课程笔记合集

CMU 15-418/618

并行计算机架构与编程

Parallel Computer Architecture and Programming

全 27 讲中文讲义

2018 年春季学期

课程：CMU 15-418/618 Parallel Computer Architecture and Programming
机构：Carnegie Mellon University
讲师：Kayvon Fatahalian、Randy Bryant 等
讲次：27 讲（L01–L27）
视频来源：B 站搬运（alexphil），含中英字幕
生成工具：`lecture-to-notes`

本讲义由视频字幕与画面整理生成，非官方课件。

目录

Lecture 01 Why Parallelism? Why Efficiency? 1

1.1 为什么必须并行：一次被功耗终结的性能曲线 . . . 1
1.2 并行的定义：处理单元必须协作，而不是简单叠加 . . . 3
1.3 四场演示：加速为什么总在打折 . . . 5
1.4 效率：加速比的另一半真相 . . . 8
1.5 这门课怎么教你处理这些代价 . . . 10
1.6 总结与延伸 . . . 11

Lecture 02 A Modern Multi-Core Processor 12

2.1 硬件到底在哪些层次提供了并行 . . . 12
2.2 最底层：指令级并行与顺序代码 . . . 12
2.3 多核：拆了芯片，但程序还没准备好 . . . 15
2.4 SIMD：用一条指令驱动多个数据 . . . 17
2.5 GPU：把控制逻辑换成 ALU . . . 21
2.6 内存：真正卡住并行的那一环 . . . 23
2.7 多线程与超线程：用吞吐换延迟 . . . 26
2.8 算术强度：峰值算力能兑现多少 . . . 28
2.9 总结与延伸 . . . 29

Lecture 03 Parallel Programming Abstractions 31

3.1 抽象与实现：并行编程最容易混淆的一对概念 . . . 31
3.2 ISPC：看起来像多线程，实际是向量指令 . . . 32
3.3 两种数据分布：interleaved 与 blocked . . . 35
3.4 归约：为什么求和不能标 uniform . . . 40
3.5 其他抽象及其实现层面的差异 . . . 41
3.6 消息传递：彻底放弃共享内存 . . . 46
3.7 数据并行：把并行交给编译器 . . . 47
3.8 总结与延伸 . . . 48

Lecture 04 Parallel Programming Basics 50

4.1 从两个例子开始：规则结构与不规则结构 . . . 50
4.2 并行化的四个步骤 . . . 52
4.3 Amdahl 定律：串行部分设下的天花板 . . . 57
4.4 分配的两种策略：静态与动态 . . . 61
4.5 动态分配：用队列换均衡 . . . 62
4.6 编排：用红黑格序消除数据竞争 . . . 63
4.7 消息传递：同一个求解器的另一种编排 . . . 69
4.8 总结与延伸 . . . 70

Lecture 05 Work Distribution and Scheduling 72

5.1 性能优化的三个目标 . . . 72
5.2 静态分配：预先决定，零运行期开销 . . . 74

5.3 半静态分配：在两者之间取一个折中 . . . 76
5.4 动态分配：让线程按需取活 . . . 77
5.5 细粒度动态分配的代价与粒度旋钮 . . . 78
5.6 分布式队列与 work stealing . . . 81
5.7 Cilk 的 fork-join：抽象如何决定调度细节 . . . 85
5.8 总结与延伸 . . . 92

Lecture 06 Recitation: Performance Tuning Tools 94
6.1 先看清机器：/proc 这一手资料 . . . 94
6.2 读架构手册：功能单元与流水线 . . . 97
6.3 优化的起点：找到内层循环 . . . 99
6.4 手工改造内层循环 . . . 103
6.5 读汇编：确认编译器做了什么 . . . 105
6.6 向量化与基准测量的纪律 . . . 106
6.7 基准测量的纪律 . . . 107
6.8 总结与延伸 . . . 108

Lecture 07 GPU Architecture & CUDA 109
7.1 GPU：从图形卡到超级计算机的心脏 . . . 109
7.2 图形管线：天然的数据并行 . . . 111
7.3 从图形卡到通用计算 . . . 114
7.4 CUDA 的抽象：线程、块与网格 . . . 119
7.5 内存层次与共享内存 . . . 123
7.6 卷积示例：从 3n 次读取到一次 . . . 125
7.7 抽象回顾与执行细节 . . . 128
7.8 总结与延伸 . . . 130

Lecture 08 Locality, Communication, Contention 132
8.1 通信：并行程序里最容易被低估的成本 . . . 132
8.2 发送与接收的几种约定 . . . 135
8.3 延迟与带宽：两个必须分开的量 . . . 137
8.4 通信耗时的模型 . . . 141
8.5 把并行系统看成扩展的内存层次 . . . 144
8.6 分块形状如何决定算术强度 . . . 147
8.7 人为通信：缓存行、缺失与遍历顺序 . . . 149
8.8 竞争与伪共享 . . . 154
8.9 总结与延伸 . . . 154

Lecture 09 Parallel Programming Case Study 156
9.1 从原理转向案例 . . . 156
9.2 硬件假设：NUMA . . . 157
9.3 案例一：海洋模拟 . . . 158
9.4 网格划分：面积与周长之比 . . . 160
9.5 案例二：星系演化与 N 体问题 . . . 166

9.6 Barnes-Hut 树 . . . 167
9.7 负载不均与动态划分 . . . 170
9.8 总结与延伸 . . . 172

Lecture 10 Parallel Programming Case Study（续） 173
10.1 加速比这个指标本身有问题 . . . 173
10.2 加速比的两个陷阱 . . . 175
10.3 超线性加速说明了什么 . . . 177
10.4 强扩展与弱扩展 . . . 179
10.5 三种约束下的定量分析 . . . 180
10.6 真实场景中的扩展与模拟方法 . . . 186
10.7 用模拟评估架构 . . . 187
10.8 总结与延伸 . . . 189

Lecture 11 Snooping-Based Cache Coherence 190
11.1 缓存一致性：一个被缓存制造出来的错觉 . . . 190
11.2 一致性问题到底出在哪 . . . 192
11.3 一致性的精确定义 . . . 195
11.4 snooping 的基本思路 . . . 199
11.5 入门协议：写直达加失效 . . . 201
11.6 MSI 协议 . . . 203
11.7 MSI 的代价与 MESI 的改进 . . . 208
11.8 总结与延伸 . . . 209

Lecture 12 Assignment 3: GraphRats 211
12.1 GraphRats：一个必须保持确定性的图模拟 . . . 211
12.2 奖励函数与移动选择 . . . 214
12.3 三种更新模式 . . . 219
12.4 提供的材料与正确性保障 . . . 223
12.5 正确性：回归测试与自建用例 . . . 224
12.6 图结构的三种形态 . . . 225
12.7 初始分布与它对语义的影响 . . . 228
12.8 总结与延伸 . . . 229

Lecture 13 Implications of Cache Coherence 231
13.1 伪共享：代码看起来没问题，性能却崩了 . . . 231
13.2 监听协议的扩展性上限 . . . 234
13.3 目录：把广播变成点对点 . . . 237
13.4 目录的收益与代价 . . . 241
13.5 目录的存储代价与压缩方案 . . . 243
13.6 真实机器的实现与优化 . . . 246
13.7 内存一致性模型：一致性没解决的那一半 . . . 249
13.8 总结与延伸 . . . 251

Lecture 14 Snooping-Based Multi-Processor Implementation 252
14.1 从状态图到硬件 . . . 252
14.2 回顾：MESI 与多级缓存的现实 . . . 254
14.3 包含属性：L2 如何知道 L1 有什么 . . . 255
14.4 三个目标与三种失败 . . . 258
14.5 简化假设与总线事务 . . . 262
14.6 多处理器控制器与 Snoop Pending . . . 264
14.7 数据从哪来：事务分裂 . . . 266
14.8 总结与延伸 . . . 266

Lecture 15 OpenMP 268
15.1 OpenMP：用编译制导表达并行 . . . 268
15.2 循环并行化时编译器替你做了什么 . . . 270
15.3 变量作用域：默认规则是个陷阱 . . . 272
15.4 CPU 与 GPU 的调度模型之别 . . . 274
15.5 两类让结果出错的写法 . . . 275
15.6 循环间同步：nowait 与 reduction . . . 277
15.7 当 OpenMP 无法并行时：重组算法 . . . 279
15.8 总结与延伸 . . . 281

Lecture 16 Memory Consistency 283
16.1 问题：并行内存层次应该表现成什么样 . . . 283
16.2 用一个实验建立顺序一致性的定义 . . . 284
16.3 横跨不同地址：更难的约束 . . . 287
16.4 反例：标志位同步为什么会失效 . . . 290
16.5 如何恢复顺序：从严格到逐步放宽 . . . 293
16.6 弱序模型与栅栏 . . . 296
16.7 总结与延伸 . . . 299

Lecture 17 OpenMP & MPI 300
17.1 为什么需要消息传递 . . . 300
17.2 MPI 是什么：规范而非库 . . . 302
17.3 运行环境与启动方式 . . . 304
17.4 基本概念：通信器、类型与标签 . . . 304
17.5 阻塞与非阻塞通信 . . . 307
17.6 集合操作与”MPI 其实很简单” . . . 310
17.7 软同步：同步发生在消息层 . . . 312
17.8 总结与延伸 . . . 313

Lecture 18 Performance Measurement and Tuning 314
18.1 “我的程序很慢”：先弄清楚是哪一种慢 . . . 314
18.2 第一层：机器上还有谁在跑 . . . 315
18.3 第二层：函数级计时 . . . 317
18.4 第三层：硬件性能计数器 . . . 320

18.5 第四层：采样分析定位到行 . . . 324
18.6 案例：一个指针间接寻址的代价 . . . 326
18.7 工具的代价与使用边界 . . . 328
18.8 总结与延伸 . . . 328

Lecture 19 Interconnection Networks 330
19.1 互连网络：一个在小规模下看不见的问题 . . . 330
19.2 为什么互连设计越来越重要 . . . 332
19.3 拓扑、路由、缓冲与流量控制 . . . 334
19.4 拓扑巡礼：从交叉开关到环 . . . 338
19.5 小规模方案：交叉开关与环 . . . 339
19.6 网格、环面与折叠环面 . . . 342
19.7 树、胖树与超立方体 . . . 344
19.8 总结与延伸 . . . 348

Lecture 20 Implementing Synchronization 350
20.1 同步原语的实现：从线程映射说起 . . . 350
20.2 同步事件的三个阶段 . . . 353
20.3 一个看似正确却有缺陷的自旋锁 . . . 356
20.4 硬件原语：test-and-set 与 cmpxchg . . . 358
20.5 理想锁的四个特征 . . . 361
20.6 三个改进方案 . . . 362
20.7 屏障与后续 . . . 364
20.8 总结与延伸 . . . 366

Lecture 21 Lock-free Programming 367
21.1 为什么不用一把大锁 . . . 367
21.2 比较并交换：无锁的基石 . . . 368
21.3 语言封装：C11 的原子类型 . . . 371
21.4 并发链表的三种方案 . . . 372
21.5 方案二：手递手细粒度锁 . . . 374
21.6 无锁算法：定义与两个实例 . . . 377
21.7 ABA 问题 . . . 381
21.8 总结与延伸 . . . 382

Lecture 22 Transactional Memory 384
22.1 上一讲的遗留：ABA 与两种解法 . . . 384
22.2 为什么需要更高层的抽象 . . . 388
22.3 事务：来自数据库的想法 . . . 389
22.4 声明式与命令式 . . . 390
22.5 事务的三个保证 . . . 391
22.6 乐观执行与冲突检测 . . . 393
22.7 乐观执行与冲突检测 . . . 393
22.8 总结与延伸 . . . 395

Lecture 23 Heterogeneous Parallelism 396
23.1 功耗：改变设计思路的主导约束 . . . 396
23.2 固定面积下的胖核与瘦核 . . . 397
23.3 异构方案：一个胖核加若干瘦核 . . . 401
23.4 现实约束：为什么不能照模型定制 . . . 403
23.5 真实系统里的异构 . . . 404
23.6 超算与专用化 . . . 407
23.7 超算排名与专用化 . . . 407
23.8 总结与延伸 . . . 410

Lecture 24 Domain-Specific Programming Languages 411
24.1 普通程序远未用尽硬件 . . . 411
24.2 即使好代码也没用尽机器 . . . 413
24.3 多个层次与多个模型 . . . 416
24.4 领域专用语言的策略 . . . 417
24.5 两个具体项目 . . . 419
24.6 总结与延伸 . . . 420

Lecture 25 Domain-Specific Programming on Graphs 421
25.1 领域专用系统：为什么值得为图单独做一套 . . . 421
25.2 设计难点：性能与可调性 . . . 424
25.3 PageRank 与 GraphLab . . . 426
25.4 一致性规则与调度顺序 . . . 431
25.5 三条经验与一个警告 . . . 433
25.6 分片：把局部性做到极致 . . . 434
25.7 GraphChi：用分片把随机访问变成顺序读 . . . 435
25.8 总结与延伸 . . . 436

Lecture 26 Parallel Deep Neural Networks 438
26.1 深度神经网络：回到结构化数值计算 . . . 438
26.2 训练与推理：两个不同的问题 . . . 441
26.3 卷积：权重共享带来的三重节省 . . . 443
26.4 推理的计算结构 . . . 447
26.5 推理的计算结构 . . . 449
26.6 训练：梯度下降与损失 . . . 451
26.7 训练：梯度下降与损失 . . . 452
26.8 总结与延伸 . . . 454

Lecture 27 Implementing Message Passing 455
27.1 消息传递为什么容易实现 . . . 455
27.2 与共享地址空间的对比 . . . 457
27.3 两种语义：同步与异步 . . . 459
27.4 eager 协议：三段流程 . . . 461
27.5 同步协议的三段流程 . . . 462

27.6 异步的麻烦与流控 463
27.7 信用式混合方案 465
27.8 总结与延伸 466

Lecture 01 Why Parallelism? Why Efficiency?

视频时长：1 小时 16 分 53 秒 来源：https://www.bilibili.com/video/BV1Xz4y1p7ZN/

1.1 为什么必须并行：一次被功耗终结的性能曲线

这堂课要回答的问题不是” 如何写并行程序”，而是更靠前的一步：为什么今天的程序员没有选择余地。答案来自 2004 年前后的一次硬件转折。理解这次转折，才能理解为什么” 效率” 会成为比” 快” 更重要的指标。

并行机并非新鲜事物：三个阶段的驱动力

并行计算机的存在时间比多数读者的编程经历长得多。讲者梳理出一条清晰的驱动力变迁线索，每一阶段的推动者都来自不同的应用领域。

第一个阶段是 1970 年代起的科学计算。当时这类机器被称为” 超级计算机”，使用者是希望更快完成数值模拟的研究者。讲者举了三个差异极大的实例：CMU 自己建造的 C.MMP，包含 16 个处理器（视频 00:35:18）；Seymour Cray 在 1980 年代设计的多 CPU 加向量处理超级计算机，处理器数量不多但每一个都极快；以及从 MIT 分出的创业公司 Thinking Machines，路线完全相反——早期机器里放了 65000 个单比特处理器（视频 00:35:57）。讲者对后者的评价带着调侃：它们擅长堆指示灯，很适合科幻电影里” 让观众看到计算机在工作” 的镜头。

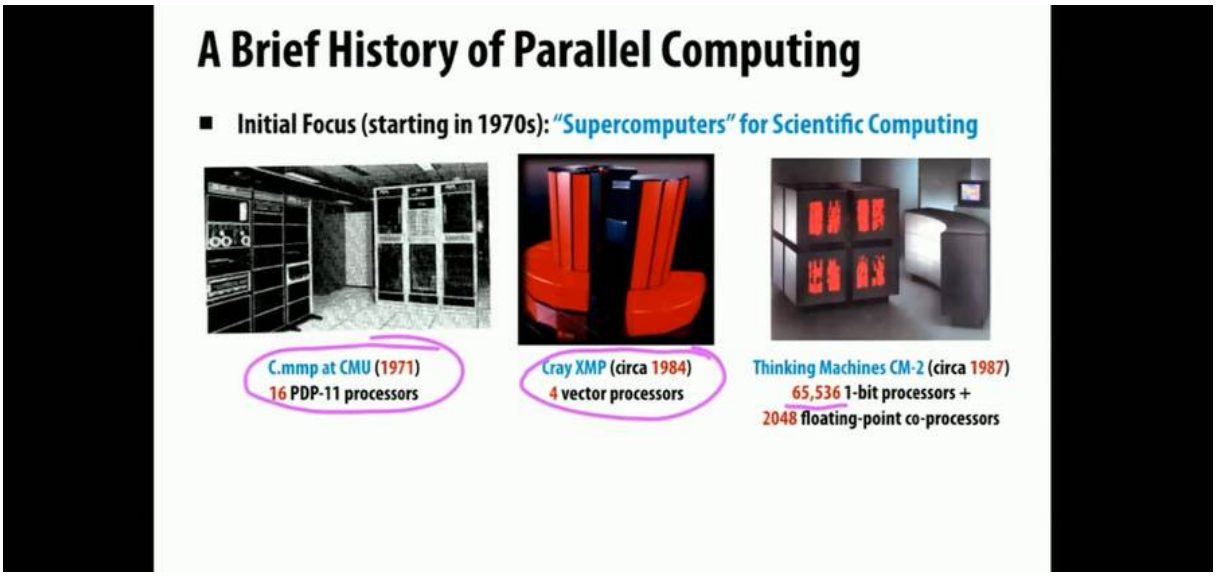

图 1: 讲者的并行计算简史幻灯片，按驱动力阶段划分并行机的应用场景[1]

第二阶段从 1990 年代开始，驱动力换成数据库。数据库研究者发现并行机是加速事务处理的有效手段，于是把关键算法与数据结构散布到多个处理器上。讲者指出这一步的意义在于市场规模的转换：数据库的市场大于科学计算，并行机因此从研究设备变成了一门生意。Oracle 与 Sun 都在卖这类机器。

[1] 视频画面时间区间：00:36:15–00:36:30。若为裁剪图，时间区间仍对应原视频画面。

第三阶段就是今天读者熟悉的形态——通用多核处理器。这一阶段并非由应用需求推动，而是被下一小节的硬件困境逼出来的。

2004 年：主频路线为什么走不下去

在很长一段时间里，处理器性能呈现稳定的指数增长，讲者的估算是大约每三年翻一倍（视频 00:38:15）。驱动这条曲线的因素有好几个：数据通路不断加宽；硬件会向前扫描指令流，找出互不依赖的指令并同时发射（讲者提到最高四路）；以及最主要的一项——时钟频率的指数提升。

时钟频率的上涨带来了一个副作用，讲者用一张 Intel 内部的功耗密度幻灯片把这个问题讲得很具体。功耗密度指芯片表面单位面积产生多少瓦特，数值越高，散热压力越大，否则芯片会因过热而失效。早期 Intel 处理器只有几瓦，Pentium 已经热到可以” 煎鸡蛋”、温度接近电热板。当时的 Intel CTO 在技术会议上顺势推演：如果主频继续指数上涨，芯片表面温度将依次达到核反应堆内部、火箭喷嘴，最终抵达太阳表面的量级。

这个推演并非玩笑。讲者提到 2004 年 5 月 17 日《纽约时报》的报道：Intel 取消了两个旗舰项目，原因是无法在不让芯片熔化的前提下解决散热问题。他在当时正供职于 Intel，因此把这一事件描述为” 一夜之间世界变了”。值得注意的是这里有一个判断失误的代价：Intel 在 1990 年代的大规模营销中教会了消费者” 性能等于主频”，这让它自己也难以轻易放弃这条路线。

把主频当成性能的同义词

主频只是性能的一个因子。同一主频下，不同架构的实际吞吐可以相差数倍。2004 年之后，这条等价关系彻底失效，但在消费市场的宣传语境中它长期残留。判断一颗处理器的性能时，主频必须与架构、核心数、缓存层次一起看。

转折之后的唯一出路

主频停滞之后，硬件设计师能做的是把多个核心放进同一颗芯片。这解释了为什么今天的手机、笔记本、服务器都提供了越来越多的并行硬件。讲者的结论很直接：单线程性能不预期会再变快，所以一切都落在并行上。

这个结论对读者意味着责任转移。过去性能提升由硬件厂商自动交付，软件无需改动就能变快；现在硬件只提供更多并行单元，能否把这些单元变成实际速度，取决于软件是否被正确地并行化。讲者在课程末尾把这一点称为” 秘密武器”——走出这门课的人会比别人更懂这些约束。

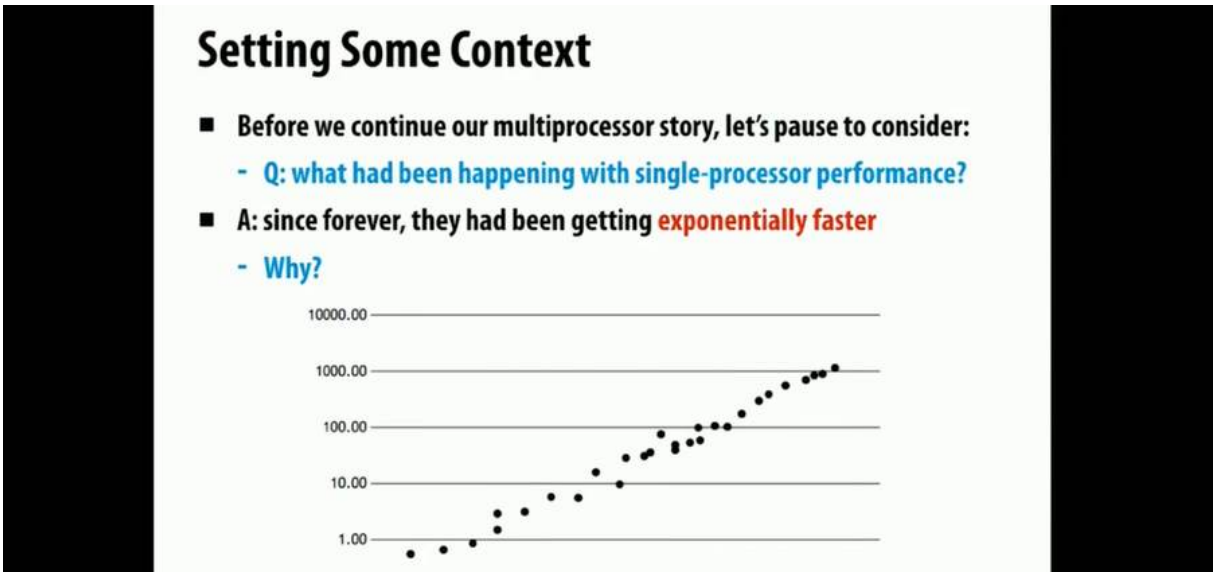

图 2: 讲者插入的背景页，用于回顾单处理器性能曾经如何指数增长[2]

本章小结

性能增长的来源已经从”更快的单核”切换到”更多的核心”，而这次切换是被功耗密度逼出来的，不是设计偏好。并行硬件因此成为默认前提：从 Intel 取消主频路线的 2004 年算起，软件若不并行就无法再获得硬件红利。

有了硬件，接下来的问题是它能否兑现。下一章给出并行的定义，并说明兑现过程为什么一再打折。

1.2 并行的定义：处理单元必须协作，而不是简单叠加

如果并行只是”装更多处理器”，性能就会随核心数线性增长。实际并非如此，原因在于定义里一个容易被忽略的动词。

教科书定义与其中的难点

讲者给出了一份”教科书式”的定义：并行计算机是一组处理单元，它们协同起来快速解决问题（视频 00:50:00）。定义包含两个要件，第二个才是关键。处理单元的存在是显而易见的，但让这些单元”协同”去解决同一个问题，才是这门课的全部难度所在。讲者的原话是：它远不只是拥有一堆处理器。

[2]视频画面时间区间：00:38:25–00:38:40。若为裁剪图，时间区间仍对应原视频画面。

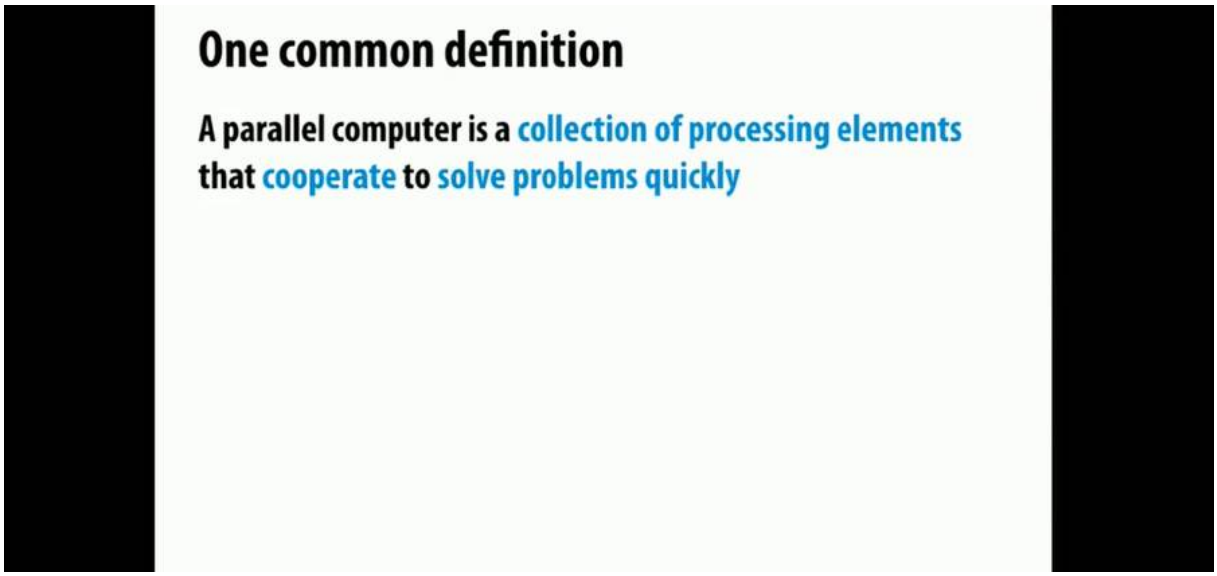

图 3: 并行计算机的定义幻灯片，强调处理单元之间必须协作[3]

并行 eq 处理器数量

判定一台机器是否在并行工作，看的是多个处理单元是否在协作求解同一个问题。各自独立处理不同任务属于并发，不是并行。这个区分决定了性能指标该按什么问题单位去度量。

定义给出之后，需要一个能把” 协作效果” 量化的工具。讲者拒绝使用” 快了百分之五十” 这类说法，因为” 快 100%” 究竟指耗时减半还是耗时归零会产生歧义。替代方案是加速比。

加速比：唯一可比较的性能语言

加速比衡量同一份工作在单处理器上的耗时与在多处理器上的耗时之比：

$$S(n) = \frac{T(1)}{T(n)}$$

- $S(n)$：使用 n 个处理单元时的加速比；
- $T(1)$：同一计算在单个处理单元上的耗时；
- $T(n)$：同一计算在 n 个处理单元上的耗时。

这个比值把” 变快” 从模糊的形容词变成可比的数量。讲者强调的关键之处在于分母是同一份工作，而不是同一台机器上的任意两次运行——否则加速比会被问题规模或实现版本的差异污染。

有了可测量的量，下一个问题自然浮现：处理器数量翻倍，加速比会不会也翻倍。讲者用一场课堂演示给出了否定答案。

本章小结

并行的定义要求处理单元协作解决同一个问题，协作本身是成本来源。加速比 $S(n) = T(1)/T(n)$ 是衡量协作效果的唯一可比较指标，它衡量的始终是同一份工作。

[3] 视频画面时间区间：00:50:08–00:50:23。若为裁剪图，时间区间仍对应原视频画面。

定义了指标之后，理论预期随之明确：若 n 个处理器完全有效，应有 $S(n) = n$。下面的演示用来检验这个预期，结果每次都不达标，而每次不达标的原因各不相同。

1.3 四场演示：加速为什么总在打折

这一章的核心材料是讲者组织的课堂演示。他请学生上台，只用纸笔传话、不许互相交谈，把”处理器之间必须通过共享介质通信”这一硬件约束变成可观察的行为。演示从 1 个处理器开始逐步扩展到全场约 150 人（视频 01:07:49）。四个数据点构成一条不理想的扩展曲线。

基准：一个人的 56 秒

第一位志愿者 Sam 需要在脑中把 16 个一位数相加。计时结果是 56 秒（视频 00:53:37）。这个数字确立了 $T(1)$ 的基准。讲者特意让 Sam 用心算而非纸笔，并说明这一轮的意义只是拿到参照点。

两个处理器：34 秒，而不是 28 秒

第二轮由两位志愿者分别计算一半的数字，最终需要合并结果。耗时降到 34 秒（视频 00:56:09）。用加速比表达：

$$S(2) = \frac{56}{34} \approx 1.65$$

- 分子 56：单处理器基准耗时，单位为秒；
- 分母 34：两处理器耗时，单位为秒；
- 结果 1.65：实际加速比，低于理论上限 2。

处理器数量翻倍，加速比不到 1.65。讲者让学生自己寻找原因，课堂给出的解释归结为两点，他都借演示设计落实了。

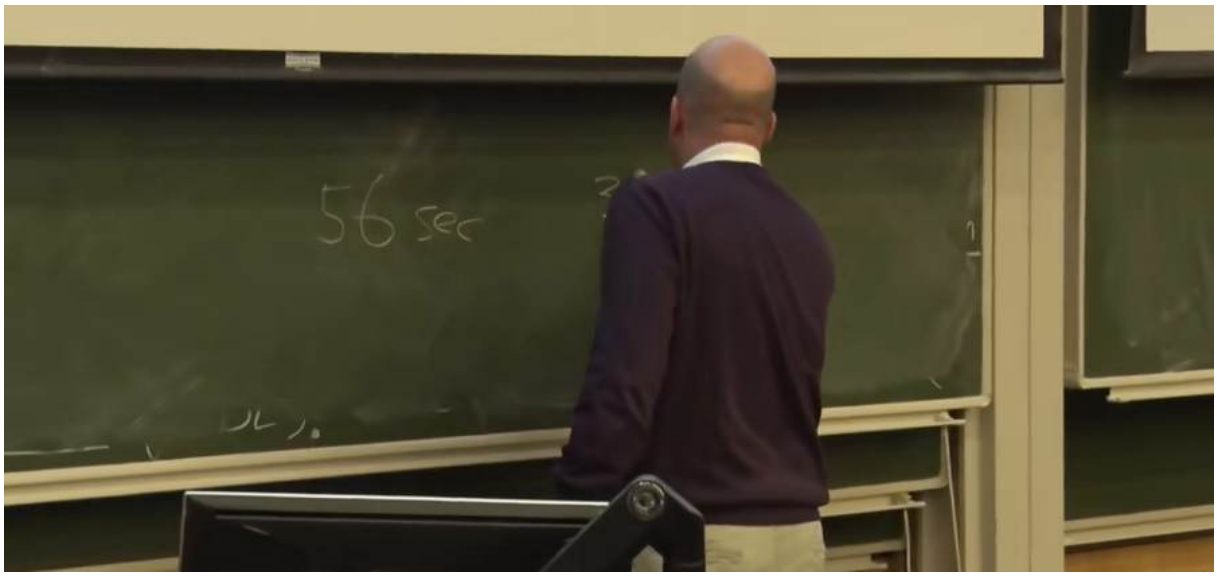

图 4: 两处理器演示结束后的计时幻灯片，记录 56 秒与 34 秒两个数字[4]

[4]视频画面时间区间：00:56:12–00:56:27。若为裁剪图，时间区间仍对应原视频画面。

两个代价的来源：通信与串行收尾

第一个代价是通信。讲者故意让其中一位志愿者 Frank 从教室后排走到前排。这不是舞台调度，而是模拟真实硬件的行为：即使多个核心在同一颗芯片上，它们交换数据也必须经过连线，而信号传输需要时间。把 Frank 的行走时间换成物理量，就是互连延迟。

第二个代价是收尾。Frank 走到前面时，搭档其实还没有算完。他必须等对方的结果写下来，读出纸上的数字，再做一次加法，才能得到总数。这段”等待加最终合并”的时间无法被并行化，它是串行成分。

为什么用人类演示讲硬件

人对”走过去看一眼再算”的耗时是有直觉的，而 CPU 核心之间的数据传递抽象掉了这种身体感。讲者用志愿者替代核心，把抽象延迟变成可看见的几秒钟。类比在讲到缓存一致性、流水线细节时会失效，但用于说明”通信有成本、归约需等待”是精确的。

四个处理器：负载不均让结果更差

按线性预期，4 个处理器应带来接近 4 倍的加速。实际耗时是 39 秒（视频 01:01:49），加速比只有约 1.44：

$$S(4)_{\text{unbalanced}} = \frac{56}{39} \approx 1.44$$

这比两个处理器的 1.65 还差。处理器数量增加了，性能反而下降。讲者在这里设置了本轮演示的关键变量：他在分配数字时故意让一位志愿者拿到了远超其余三人的工作量。学生很快指出这位志愿者分到的数字”远多于四个”。

讲者随后解释了为什么这样设计。在真实并行程序中，把工作量均匀切开并不总是容易做到，而负载不均的后果是可以预测的：完成得快的处理器进入空闲，却无法提前结束整个计算，因为最终结果必须等最后一个处理器交卷。三人在等待，一人在计算，这段时间里三个核心的资源被完全浪费。

增加核心数反而变慢

4 个处理器耗时 39 秒，劣于 2 个处理器的 34 秒。这不是测量误差，而是负载不均的直接后果。加速比的分母取决于最慢的那个处理单元，而不是平均值。评估并行方案时，只看总工作量是否被切分是不够的，必须看切分后最大分片的大小。

重新平衡：24 秒与它的代价

讲者允许四位志愿者在开始前短暂协商策略，之后仍禁止交谈，但可以移动数字。他们的做法是把数字重新分配成每人四个。耗时降到 24 秒（视频 01:06:21），加速比约为：

$$S(4)_{\text{balanced}} = \frac{56}{24} \approx 2.33$$

同一个 4 处理器配置，从 1.44 提升到 2.33，唯一的改动是负载分布。讲者同时提醒这项改进并非免费：重新平衡本身占用了时间，属于额外开销。在这个例子里开销远小于收益，但在实际系统中，动态负载均衡的调度成本可能吃掉全部收益。

图 5: 四处理器负载不均的一轮，计时板同时显示此前两轮的 34 秒与本轮的 39 秒[5]

图 6: 重新平衡后的一轮，计时板并列展示三轮结果以对比加速效果[6]

全场 150 人：通信拓扑决定归约形态

最后的演示把规模推到极限。全教室约 150 名学生每人报出自己选修的课程门数，规则是只能与前后左右相邻的人通信，不许隔空喊话（视频 01:07:49）。这相当于一台通信拓扑受限于近邻连接的并行机。

学生提出的方案体现了两种结构。第一种是先将同一排的数字横向相加，再把各排结果纵向汇总到前排；第二种是让求和从两端同时向中间推进，使并行度更高。讲者认可这些思路，因为它们在限制条件下减少了必须串行经过的步骤数。

[5]视频画面时间区间：01:02:00–01:02:15。若为裁剪图，时间区间仍对应原视频画面。

[6]视频画面时间区间：01:05:40–01:05:55。若为裁剪图，时间区间仍对应原视频画面。

这一轮演示的要点不是耗时数字，而是揭示了一条规律：当通信只能发生在近邻之间时，求和无法一步完成，必须组织成树形或多级归约，通信层数随规模增长。这正是真实并行机上大规模归约的成本来源。

本章小结

四个数据点构成一条不达标的扩展曲线：$S(2) \approx 1.65$、$S(4) \approx 1.44$、重平衡后 $S(4) \approx 2.33$；规模扩展到全场后，瓶颈转为通信层数。三种代价分别是通信延迟、串行收尾、负载不均，第四种代价隐藏在前三者之中——为了消除它们所做的协调本身要花时间。

演示回答的是” 加速比为什么小于核心数”。它没有回答另一件事：当加速比小于核心数时，这些多余的处理器是浪费还是仍有价值。这取决于如何衡量成本，是下一章的主题。

1.4 效率：加速比的另一半真相

加速比只回答” 快了多少”，不回答” 值不值”。同样一个加速比，在不同成本结构下可以是一次成功，也可以是一次失败。

效率的定义

效率把加速比放到处理器数量上归一化：

$$E(n) = \frac{S(n)}{n}$$

- $E(n)$：使用 n 个处理单元时的并行效率，取值在 0 到 1 之间；
- $S(n)$：同一计算在 n 个处理单元上的加速比；
- n：投入的处理单元数量。

$E(n) = 1$ 表示每个处理器都物尽其用，$E(n) = 0.5$ 表示有一半的算力被浪费。用演示数据代入：$E(2) = 1.65/2 \approx 0.83$，负载不均的 $E(4) = 1.44/4 = 0.36$，重新平衡后 $E(4) = 2.33/4 \approx 0.58$。同一份计算，在四个不同的组织方式下效率相差两倍以上。

同样快两倍，为什么可能是失败

讲者用一道情境题把效率的意义讲透：如果你在实习时把一个问题放到 10 个处理器上，只快了 2 倍，老板会给你升职还是解雇你？他没有立刻给出答案，而是指出结果取决于资源是否稀缺。

如果这 10 个处理器本来就在你手里——比如手机里的 GPU，无论你是否使用它都在耗电和占面积——那么把原本达不到实时帧率的软件加速到实时，就是相当大的成功。反之，若你按 10 台机器在云上租用计费，却只换来很小的收益，这次并行化的投入产出比就很差。

效率取决于边际成本

并行是否值得，不能只看加速比。判断依据是新增的处理单元是否本来就会存在：既有的、已经付费的资源上获得加速是净收益；为并行新增的按量计费资源上只获得小加速则是净损失。同一组数字在两种情境下结论相反。

效率为什么成为硬件设计的核心指标

效率的重要性不只对软件成立。讲者指出 2004 年之前，硬件设计的目标是” 在一颗芯片上放一个尽可能快的处理器”，为了榨取最后一点性能，设计变得极度投机且低效。而这种设计在当时是正确的——因为芯片面积只能容纳一个核心，不用掉就浪费了。

多个核心进入芯片之后，判断标准翻转了。每个核心的面积决定了能塞进多少个核心，因此” 每单位面积的性能” 成为关键指标；同时，服务器机房的供电能力有上限，移动设备的电池容量有上限，于是” 每瓦性能” 同样关键。讲者把这种从追求绝对速度到追求单位资源产出的转变，视为 2004 年之后硬件设计的根本变化。

维度	2004 年之前（单核路线）	2004 年之后（多核路线）
设计目标	单个处理器尽可能快	单位面积与单位能耗的性能
主要手段	提高主频、投机执行	增加核心数、异构与专用单元
是否宽容低效	宽容，面积只能用在一处	不宽容，浪费面积就少放核心
性能增长来源	硬件自动交付	软件必须并行化才能兑现

表 1: 硬件设计范式的转变及其对软件的后果

串行成分如何给加速比设上限

演示已经用具体数字展示了加速比低于核心数，讲者没有在黑板上推导公式，但给出了足够的因果结构，可以据此整理出一条硬性约束。

设一份计算中不可并行的比例记为 f，剩余 $1-f$ 部分可以完全并行。若忽略通信与调度开销，n 个处理器下的耗时至少包含那部分串行工作：

$$S(n) \le \frac{1}{f + \dfrac{1-f}{n}}$$

- f：计算中串行成分占总工作量的比例，$0 \le f \le 1$；
- n：处理单元数量；
- 当 $n \to \infty$ 时，上界趋近于 $1/f$。

这个关系解释了演示中” 两个处理器只得到 1.65 倍” 的现象：即使并行部分完美分摊，Frank 走到台前与最终核对的那一段时间仍然串行。它也解释了为什么不加区分地增加核心会迅速失去意义——上界由 f 决定，与 n 无关。本讲义据此整理为推论，讲者在本课中并未写出该式。

把加速比当成扩张性能的万能钥匙

若一份计算中有 10% 无法并行，则无论投入多少处理器，加速比都不会超过 10 倍。在资源规划阶段先估算串行成分，比事后增加机器更有效。

本章小结

效率 $E(n) = S(n)/n$ 把加速比换算为资源利用率，它能区分”既得资源上的提速”和”新增付费资源上的小提速”这两种性质相反的处境。2004 年后的硬件设计也转向同一标准：每单位面积与每瓦的性能。串行成分 f 为加速比设定与 n 无关的上界，这是”无限加核”思路的硬性天花板。

到这里，代价的种类已经清楚：通信、负载不均、串行成分、协调开销。剩下一个疑问：这些代价能否在写代码之前被预测，从而避免反复试错？讲者的回答是不能，这正是下一章要说明的课程方法。

1.5 这门课怎么教你处理这些代价

前面几章建立了并行的动机与代价结构。这一章转向方法论：面对这些代价，工程上应该采取什么样的工作方式。

性能调试是一个必须经历失败的迭代过程

讲者对课程方法的说明与多数课程不同。他明确表示，这门课关心的是性能调试与调优，而不仅是正确性。与只要求正确结果的课程相比，这里的评分标准包含了过程：他会认真阅读学生记录如何尝试、为何失败、如何推理下一步的写作。

讲者把这一点提升为课程的核心命题：这门课讲的是优化的过程，而不是优化后的那个数字。理由是系统复杂到没有人能在第一次就给出完美方案。真实世界的做法是做出最好的估计，形成策略，尝试，观察结果，修正判断。他在课程开头就要求学生在作业中完整记录这个过程。

为什么过程比结果更值得记录

并行系统的性能由多项相互牵制的因素共同决定：通信、负载均衡、缓存局部性、同步开销。这些因素之间的权衡关系因机器而异，无法从教科书条件直接推出。记录失败尝试的行为本身就是知识的积累方式——它保留了”在什么条件下某策略失效”这一无法从最终代码里读出的信息。

为什么必须懂一点硬件

讲者给出的第二个方法要点是硬件知识不可跳过。即使读者完全不打算设计硬件，也必须了解硬件层面的约束，原因是对比序列程序时的一个本质差异：在串行程序里，硬件的内部细节对性能的影响可以被很大程度地忽略，而在并行程序里，这些细节直接决定性能。

他举的例子是瓶颈。哪一部分限制了吞吐，往往取决于缓存层次、互连结构、同步原语的实现方式，这些都不是编程语言层面能表达的信息。不了解它们，性能问题就只能靠猜。

每一份作业都是不同的抽象

课程的第三个特点是抽象层次的多样性。讲者提到，每份作业对应一套完全不同的并行思维抽象。这意味着不存在一个统一的并行编程模型能覆盖所有场景，读者的能力目标是学会在给定抽象下推理开销，而不是记住某个库的调用方式。

本章小结

课程方法有三个支点：把性能调优当作迭代过程并把失败记录为知识；把硬件瓶颈当作必须理解的约束而非黑盒；把不同并行抽象当作需要分别掌握的工具。这三点的共同前提是前面建立的代

价结构——正因为通信、负载、串行成分无法事先完全预测，才需要这种以实验为驱动的工程方法。

1.6 总结与延伸

这堂导论课没有给出任何并行算法，它给出的是一套判断框架。

讲者的收尾结论可以压缩为一句话：单线程性能不预期会再变快，所以一切都落在并行上，而这件事并不容易做。他把难度转化为一种优势——理解这些约束的人在进入行业时会比同龄人更清楚瓶颈在哪里。

把全课串起来看，逻辑链条是完整的。2004 年功耗密度终结了主频路线，硬件被迫转向多核，性能红利从此需要软件兑现。兑现的前提是处理单元之间的协作，而协作必然产生通信、串行收尾与协调开销。四场演示用 56 秒、34 秒、39 秒、24 秒四个数字把抽象代价变成了可观察的事实：两个处理器只得到约 1.65 倍，四个处理器在负载不均时甚至不如两个，重新平衡后才回升到约 2.33 倍。效率指标进一步说明，这些折扣是否可接受取决于资源是否已经付费；串行成分则为任何加速设定了与核心数无关的上界。

以下三点是本讲义在讲者材料之上做的压缩，需要与讲者原话区分开：

- 加速比与效率必须成对使用。只看加速比会误判一个浪费了七成算力的方案为成功，只看效率会误判一个在既有资源上获得真实提速的方案为失败。
- 演示中的三种代价并不独立。负载不均会诱导出额外的协调开销，协调开销本身又是串行成分的一部分。把它们当成可分别优化的独立项，会在实践中反复遇到此消彼长。
- 讲者的方法论指向一个可操作的优先级：先测量，再估算串行成分与最大分片，最后才考虑增加处理器。顺序颠倒时，硬件投入的收益会被负载与通信问题吞掉。

需要说明这堂课的边界。演示只覆盖了求和一个计算形态，通信被简化为” 走一段路” 和” 看一眼纸”，没有涉及缓存一致性、同步原语竞争、伪共享等真实机制；讲者也明确表示后四到五次课才会讨论这些问题与并行软件的关系。因此本文给出的是判断框架而非调优技术。

一个具体的下一步问题可以直接从演示数据提出：4 个处理器重新平衡后达到 24 秒，而理论下限是 14 秒（56 除以 4）。差的这 10 秒里，有多少属于重新平衡的一次性开销，有多少属于每轮都要付出的归约成本？区分这两者，决定了该优化调度策略还是该改换归约结构。

Lecture 02　A Modern Multi-Core Processor

视频时长：1 小时 22 分 49 秒　来源：`https://www.bilibili.com/video/BV1Rh4y1F7aU/`

2.1　硬件到底在哪些层次提供了并行

上一讲用人类演示说明了并行会打折。这一讲换成硬件视角，回答一个更具体的问题：当程序员写下一行循环时，机器有哪些层次能同时干活，而每一层又要求程序员付出什么。讲者给出的答案是分层的——有些并行硬件自动完成，程序员看不见；有些必须由程序或编译器显式生成代码才能兑现。

起点：功耗墙

课程先回顾上一讲的结论。2004 年前后，芯片功耗密度触及上限：讲者给出的经验阈值是单芯片功耗超过约 100 瓦（视频 00:02:24）就难以散热。他用一个直白的类比说明这件事的严重性——想象把一个很亮的大灯泡塞进盒子里，你需要一台巨大的风扇。对手机尤其致命：电池会在五分钟内耗尽，而且机身会热到让人不适。

讲者的判断是人人知道这一天会来，却没人做好准备。转折之后，所有关于未来性能提升的预测都指向并行。

并行藏在层次结构里

这节课的主线可以概括为一句话：为了利用并行，硬件设计师在层次结构的多个层面上都布置了潜力，其中一部分名义上对程序员不可见，另一部分必须由程序员或编译器显式生成相应代码（视频 00:01:35）。

为什么”不可见”不等于”可以不管”

分支预测、乱序发射这类机制不需要写代码就能生效，但它们的效果高度依赖代码形态。同一份算法，写成什么形式决定了硬件能不能找到并行。讲者因此强调：必须把硬件理解到足以写出让硬件发挥潜力的软件。

讲者还点出贯穿全课的第二个主题：扩展计算规模时最大的难题往往不在计算，而在数据搬运。你可以有大量并行单元，但如果喂不进它们要的内存值，它们就没有用（视频 00:04:25）。这一讲的后半部分专门处理这件事。

本章小结

并行不是单一技术，而是分布在本章之后逐一展开的三到四个硬件层次上的能力；功耗墙是这一切的背景。既然层次不同，兑现方式也不同——有的全自动，有的要编译器配合，有的必须手写。下一章从最底层、也最”自动”的一层开始：从顺序代码中挖掘指令级并行。

2.2　最底层：指令级并行与顺序代码

即使程序是纯顺序的，硬件也在试图同时执行多条指令。这一层叫指令级并行（instruction level parallelism，ILP），理解它的上限，也就解释了为什么必须往上走到多核。

一个没有并行的例子

讲解用的例子是计算 sin 的级数项。核心汇编大致是三步：从内存把 x_i 读进寄存器、把寄存器自乘、再把结果与其中一个乘数相乘得到 x_i^3。讲者特意说明这份代码风格是”目的地写在左边”的形式，与 2-13 课程里的 AT&T 格式相反。

关键在于依赖链：必须先完成读内存才能开始乘；必须先完成第一次乘才能做第二次乘。三条指令只能严格按序执行。任何并行机制在这里都找不到空隙。

依赖链决定可并行宽度

同一段代码能否并行，先看数据依赖，不看指令条数。读内存必须先于乘法，第一次乘法必须先于第二次，这三条指令因此只能串行。判断一份代码的并行潜力，第一步是把依赖图画出来，找出真正独立的操作有多少。

超标量处理器怎么找并行

从 1990 年代末开始，微处理器普遍具备超标量（superscalar）能力：硬件中有复制的逻辑从单一指令流中提取指令，并有多条 ALU 同时执行。

约束是语义保持——程序必须表现得像严格按顺序执行。硬件的工作是找出互不依赖的指令并同时发射。讲者提到，最初的 Pentium 大致就是这种形态。

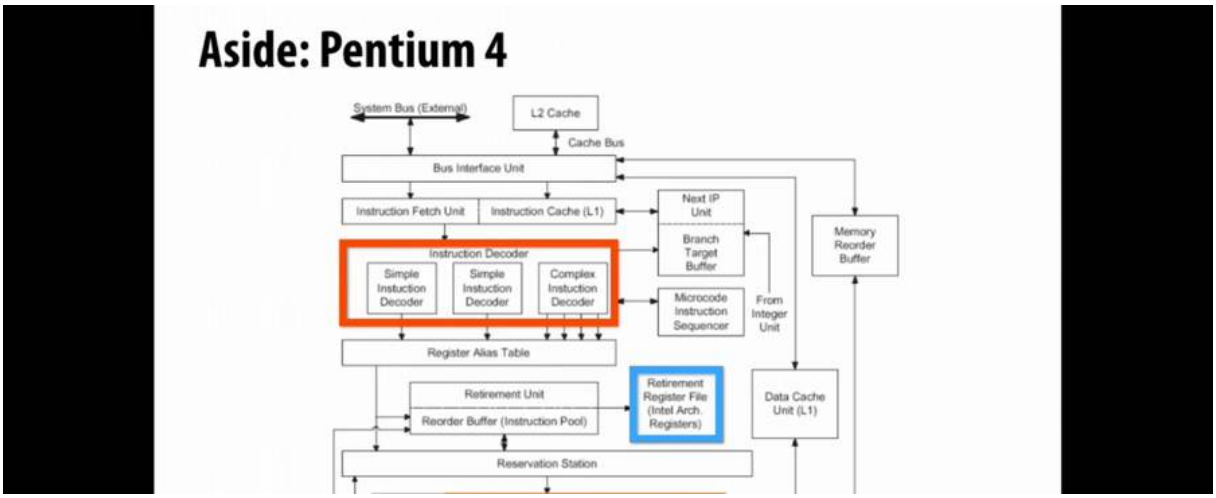

图 7: Pentium 4 的芯片结构：大片面积给了缓存与控制逻辑，ALU 只占相对小的一块[7]

乱序执行：把指令流变成数据流

1990 年代末之后，高端处理器转向乱序执行。做法是把程序计数器附近的约十条指令一并取出，映射成一种类似数据流的计算：跟踪哪个值会被哪条后续指令使用，也就是指令之间的数据依赖关系。然后通过复杂的控制逻辑，把这些操作分派到多个只能执行部分操作的独立单元上。

Pentium 4 的分工是：一个单元负责内存读写，两个做整数运算，一个做浮点运算，还有一个专门执行 SIMD 指令。较新的设计能力更强，可以同时两次加载一次存储，多个浮点与整数单元都能执行 SIMD 操作。

[7] 视频画面时间区间：00:10:45–00:11:00。若为裁剪图，时间区间仍对应原视频画面。

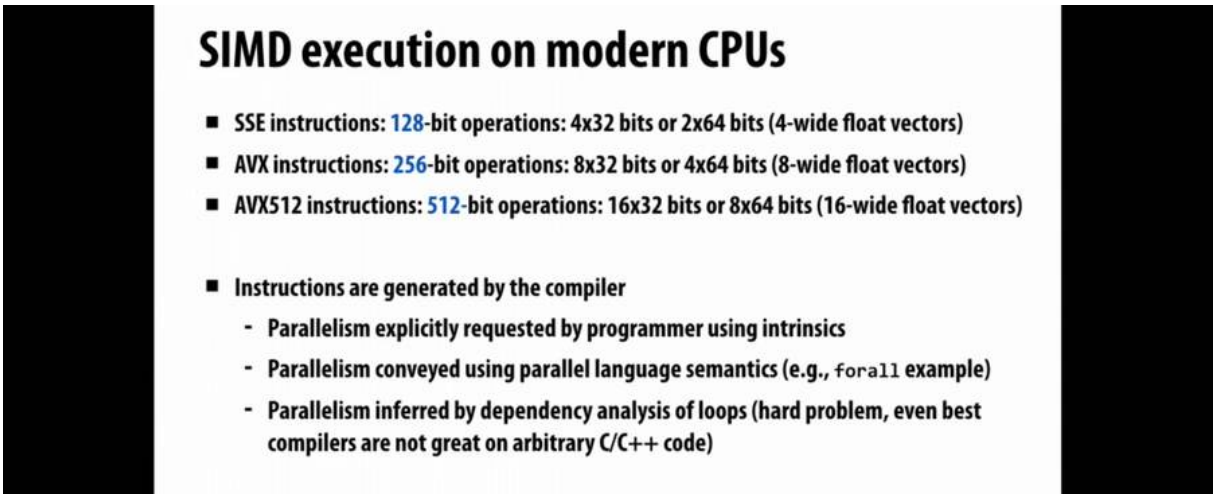

图 8: 现代 CPU 的 SIMD 能力：SSE 的 128 位与 AVX 的更宽向量[8]

讲者由此得出一个对顺序代码也成立的结论：即使完全串行的程序，运行快慢也取决于代码怎么写以及编译器有多聪明。硬件的原始算力就摆在那里，问题是有没有足够多的活喂给它。

乱序执行是硬件自动做的，但依赖代码形态

乱序执行、寄存器重命名、多发射都不需要程序员改代码就能生效，这是它们与 SIMD、多线程的根本区别。但硬件只能在指令窗口内寻找依赖关系，窗口之外无法利用。因此把独立计算写在一起、避免长依赖链，仍是程序员能直接影响这类加速的手段。

分支预测与它带来的安全问题

控制逻辑中有一块叫分支目标缓冲区，记录所有控制流指令过去跳向何处，据此预测将来也跳向同一目标。这样处理器不必等条件判断完成就能沿预判路径继续执行。

预测正确时一切顺畅；预测失败时，处理器不会把结果写回真实寄存器，而是直接冲刷掉，当作这段时间被浪费。

讲者在这里插入了一段当时正值新闻的旁白，值得单独记录：如果错误预测路径上访问了本进程无权访问的内存，虽然访问最终会被标记为失败，但缓存命中与否的差异会透露信息。这类时序与分支预测状态的泄露，正是 Spectre 一类攻击的原理。讲者强调这些机制位于系统最性能敏感的核心部位，因此堵住这类泄露很难。这也是为什么必须写清楚：这是讲者在课上给出的解释，不是本讲义补充的外部材料。

为性能而生的机制会成为攻击面

分支预测、乱序执行、预取都在缩短”看起来没用”的时间。一旦这些时间差可被观察，它们就成了信息泄露的信道。做安全敏感设计时，性能优化与信息隔离常常正面冲突。

ILP 的天花板

讲者给出这一层的量化上限：硬件能够从顺序代码中榨出的指令级并行大约是三到五倍（视频 00:19:17）。原因有两方面。其一，程序本来就不是按并行思路写的，指令之间的依赖限制了可同时执行的数量。其二，靠缩小晶体管提高主频这条路已经撞上功耗限制。

[8] 视频画面时间区间：00:46:00–00:46:15。若为裁剪图，时间区间仍对应原视频画面。

这一段是整讲的转折点：

$$S_{\mathrm{ILP}} \approx 3 \sim 5 \text{ 倍}, \qquad \text{与程序员是否改写代码无关}$$

- S_{ILP}：仅依靠超标量与乱序执行可获得的加速倍数；
- 该倍数由硬件对顺序代码的自动分析能力决定，不随处理器代际持续增长。

本章小结

指令级并行是硬件自动完成的一层，程序员无需改代码即可受益，但收益封顶在约三到五倍，且已在 2004 年前后用到尽头。要继续提速，必须让硬件同时做更多份互相独立的工作——这就需要把并行显式地暴露出来。下一章讨论由此产生的多核取舍。

2.3 多核：拆了芯片，但程序还没准备好

把一块大处理器拆成若干较小的处理器，是功耗墙之后的直接反应。这一章的重点不是多核本身，而是它暴露出的软件落差。

0.75 倍的核心与 1.5 倍的潜力

讲者设置了一个具体假设：假设每个新核心比原来的单核慢 25%，即性能是原来的 0.75 倍。那么放两个核心，理论上可望得到：

$$S(2) = 2 \times 0.75 = 1.5$$

- 0.75：每个新核心相对原单核的性能比例；
- 2：核心数量；
- 1.5：理想情况下两核并行能获得的加速倍数。

1.5 大于 1，看上去是赚了。但这只是硬件的潜力，兑现的前提是程序真的有这么多并行。

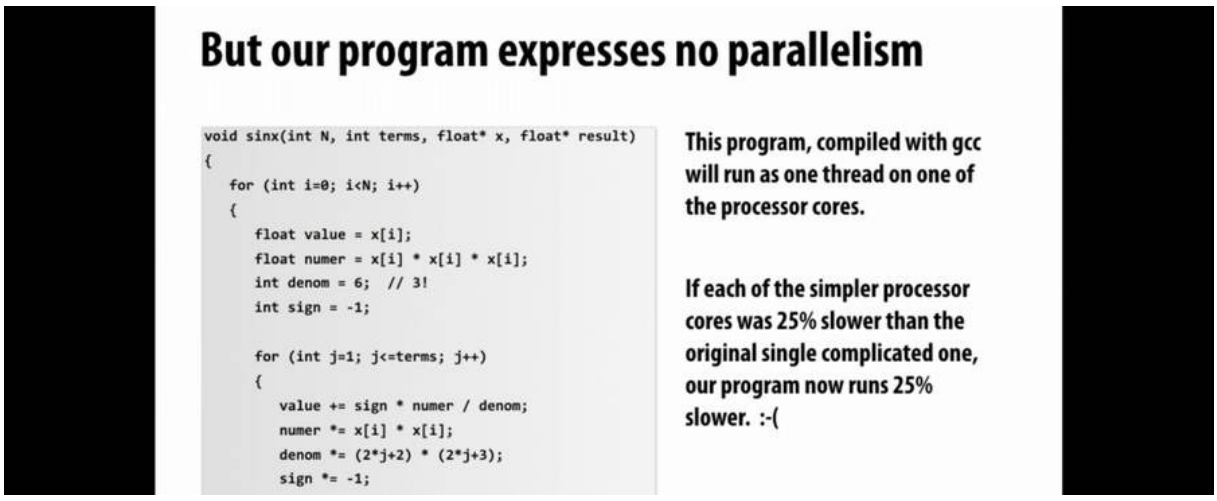

图 9: 双核并行计算两个元素的示意图，配页文字指出原程序本身不含并行[9]

[9] 视频画面时间区间：00:20:45–00:21:00。若为裁剪图，时间区间仍对应原视频画面。

纯顺序程序会变得更慢

讲者随即指出落差：如果一个纯顺序程序跑在这种双核机器上，它只会用到其中一个核心，于是只能拿到 0.75 倍——比原来的单核还慢。

这个结论的分量在于它把问题彻底翻转了过来。多核不是白送的性能，而是一次需要软件配合才能回本的交易；不配合时，硬件进步反而变成退步。

多核的红利是有条件的

$2 \times 0.75 = 1.5$ 只在程序能被切成两半并行执行时成立。纯顺序程序得到的是 0.75，比单核更差。核心数增加不自动等于性能增加，中间隔着并行化这一步。

用 pthreads 手工切分

第一条路径是手工写线程。讲者给出的做法很直接：把 n 个数分成两半，各约 $n/2$ 个，开一个线程算前半，主线程算后半。对于求这类计算，这个切分非常容易做对。

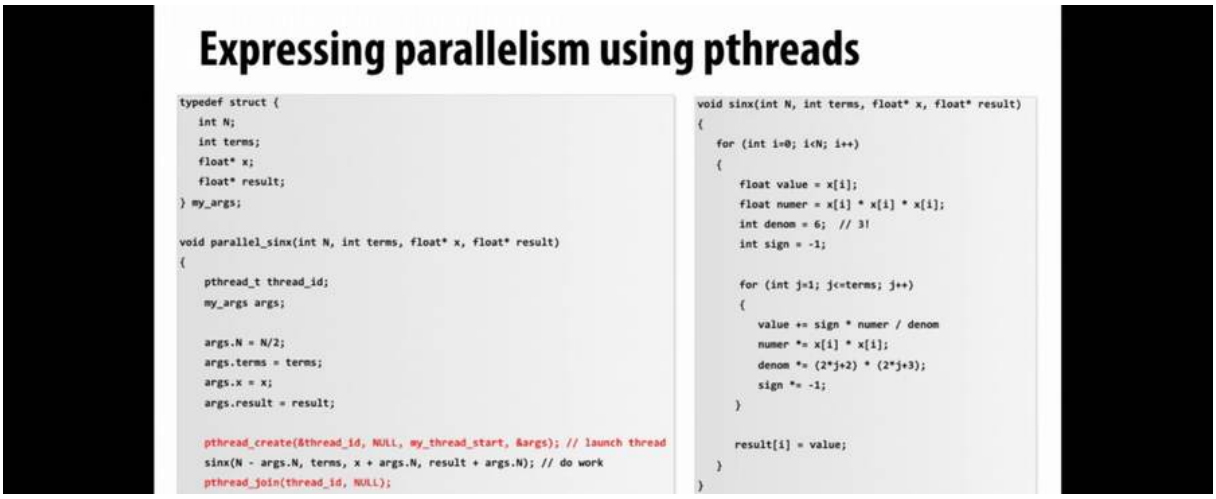

图 10: 用 pthreads 表达并行的代码：手工切分数据并创建线程[10]

但讲者对这条路的评价很坦率：pthreads 写起来不愉快，而更难的是在这套代码上做性能优化——很多地方拿不到预期的加速。这段评价呼应上一讲的三类代价：线程切分之后，负载均衡、通信同步、启动开销都仍然要程序员自己处理。

更理想的表达方式

讲者提出一个假想更好的世界：如果有语言能直接描述”工作项之间互相独立”这件事，就可以写成”对所有 0 到 $n-1$ 间的 i，执行如下操作”，把如何映射到硬件交给编译器与硬件去决定。

这种做法成立的理由是循环的每个迭代 i 都是完全独立纯粹的计算，不需要同步、协调或定序。讲者说作业一会用到具有这种风格的语言。

[10] 视频画面时间区间：00:21:30–00:21:45。若为裁剪图，时间区间仍对应原视频画面。

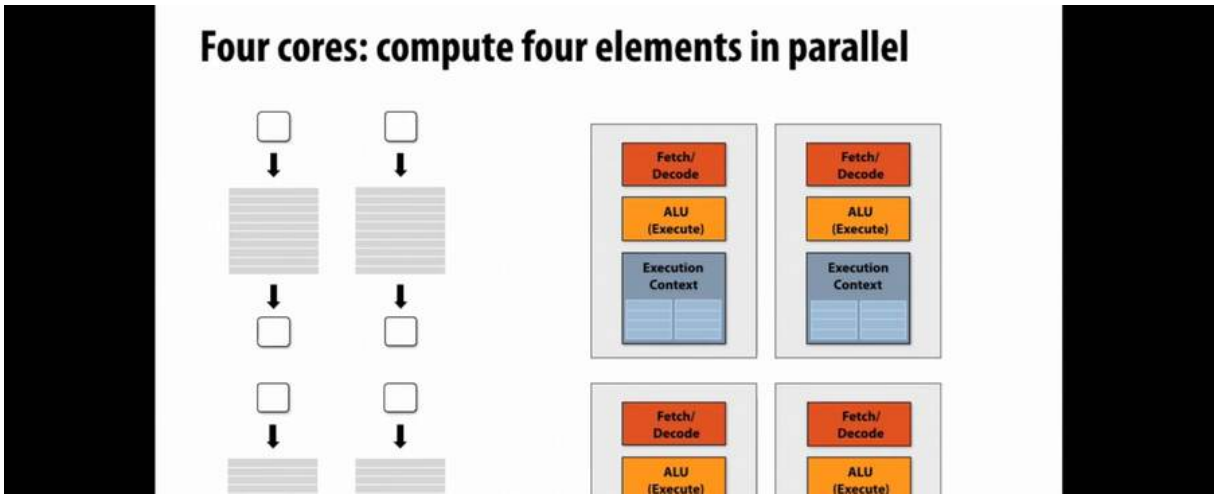

图 11: 四核并行计算四个元素，同一份计算在更多核上的映射[11]

本章小结

多核把潜力写进了硬件，但兑现要求程序显式暴露并行。手工线程可以做到，代价是程序员承担切分、同步与调优的全部负担。既然”每个迭代独立”这种结构如此规整，硬件能否用一种更廉价的方式同时处理多个元素？这就是下一章 SIMD 的出发点。

2.4 SIMD：用一条指令驱动多个数据

多线程是纵向切分——每个线程负责一部分数据，各自跑完整个流程。SIMD 是横向切分：在同一时刻，对所有数据执行同一个步骤。

纵向与横向的差别

讲者用计算 x^3 的循环来说明这两种视角。线程视角是开 k 个线程，每个线程取一段 i 从头算到尾。SIMD 视角是同时对全部 i 执行第一次乘法，再同时对全部 i 执行第二次乘法，以此类推向上走。

要实现横向做法，需要往处理器里放更多 ALU，并假设所操作的运算彼此没有依赖，然后用一条指令把这一串操作串起来。这就是单指令多数据（single instruction multiple data，SIMD）。

讲者估算，16 个核心、每核 8 路向量处理，理论上能得到 128 倍的并行度，而代价相对很低，因为这些 ALU 占据的芯片面积很小（视频 00:36:48）。但这要求同时存在 128 件彼此完全独立的工作。

纵向切分与横向切分的适用条件

多线程（纵向）要求工作能切成独立的任务块，块之间几乎不通信；SIMD（横向）要求同一时刻各通道执行完全相同的操作。前者适合任务粒度大且彼此独立的场景，后者适合对整齐数组做同一变换的场景。两者可以叠加，但叠加后的实际并行度受较弱的那个条件约束。

[11] 视频画面时间区间：00:24:30–00:24:45。若为裁剪图，时间区间仍对应原视频画面。

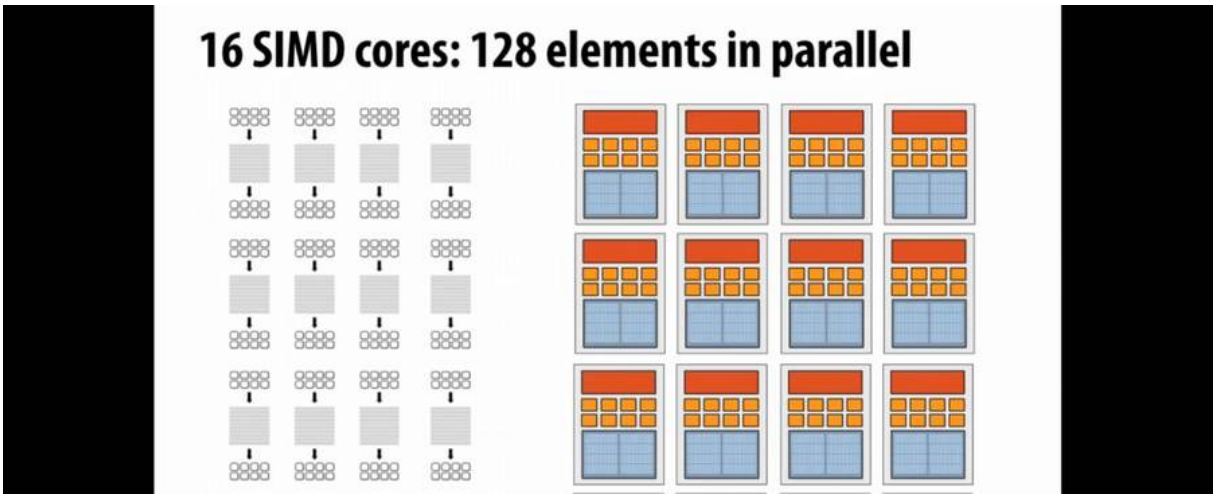

图 12: 16 个 SIMD 核并行处理 128 个元素：横向切分在多核上的叠加[12]

AVX2：真实的向量指令

Intel 当前的向量扩展叫 AVX2，即高级向量扩展第二代。它一次处理 32 字节，也就是 256 位（视频 00:33:01）；若数据是单精度浮点（每个 4 字节），则一次处理 8 个。

这套寄存器（XMM）与普通通用寄存器不同：通用寄存器只有 64 位，而 XMM 是独立的 256 位寄存器组，专为向量指令服务。现代处理器里即便非向量化的浮点运算也使用 XMM，但只用低位的几个字节。

调用这些指令的方式是 intrinsics：仍然写 C 代码，但实质是在逐条指示编译器该生成哪些汇编。命名中的 mm256 指 256 位；数据类型如 ___m256 就是 32 字节。ps 表示 packed single，即打包的单精度浮点。

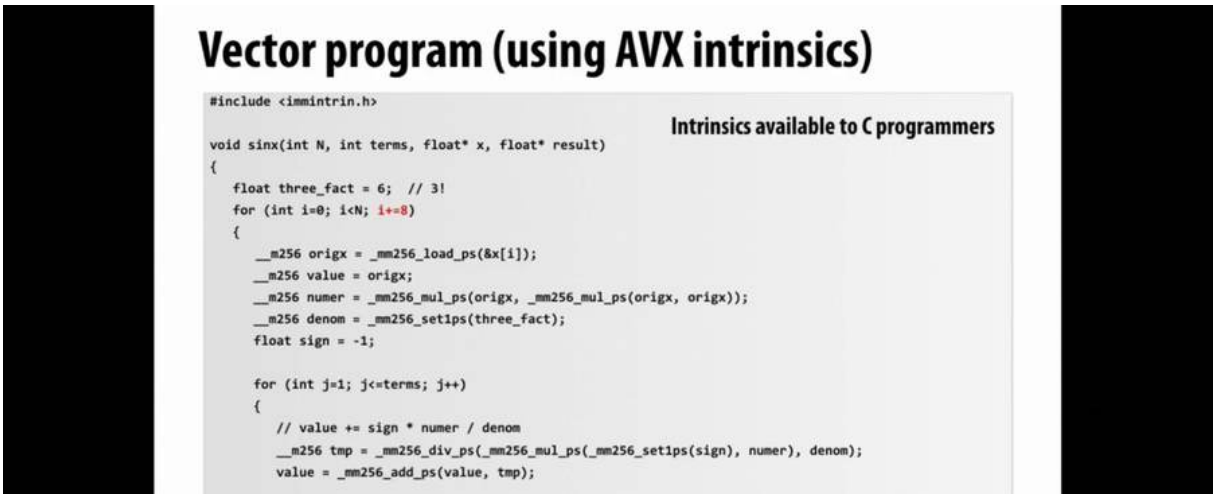

图 13: 使用 AVX intrinsics 的向量程序：循环按 8 展开并逐批加载向量[13]

按讲者的描述，这段代码的效果是把循环按 8 展开，依次对前八个数据发出向量指令，再对下一批发出向量指令。生成的汇编与原版差别极大——程序员已经明确告诉处理器该如何取出八路并行。

[12] 视频画面时间区间：00:36:55–00:37:10。若为裁剪图，时间区间仍对应原视频画面。

[13] 视频画面时间区间：00:31:40–00:31:55。若为裁剪图，时间区间仍对应原视频画面。

intrinsics 把移植成本转嫁给程序员

用 intrinsics 写的代码与具体指令集绑定，AVX2 的 256 位写法不能直接搬到 SSE 或 ARM NEON 上，换架构就要重写。同时它牺牲了可读性：代码表达的是硬件动作，不是算法意图。讲者说这是”痛苦且不愉快的体验”，正是这个代价。

```
#include <immintrin.h>

for (int i = 0; i < N; i += 8) {
    // 一次载入 8 个单精度浮点（32 字节 = 256 位）
    __m256 x = _mm256_load_ps(&A[i]);
    // 向量乘：x 自乘，得到 8 个 x_i^2
    __m256 x2 = _mm256_mul_ps(x, x);
    // 向量乘：再乘一次 x，得到 8 个 x_i^3
    __m256 x3 = _mm256_mul_ps(x2, x);
    _mm256_store_ps(&result[i], x3);
}
```

Listing 1: AVX intrinsics 的核心写法（按讲者对 ___m256 与 _mm256_load_ps 的描述整理）

为什么不能指望编译器自动完成

有人问能否让编译器自动向量化。讲者的回答是”长答案是能”，但需要给 GCC 大量提示与特殊条件，代码完全规整时才会触发。业界的共识是只有高度结构化、精心编写的代码能被自动向量化。

讲者给出的可靠验证手段很朴素：查看生成的.s 汇编文件，确认代码到底有没有被向量化。

条件分支：SIMD 效率的软肋

前面所有例子的共同优势是所有 i 执行完全相同的操作，因此容易向量化。一旦代码中出现依据 x 取值分支的情况（例如 $x > 0$ 时做一套操作、否则做另一套），问题就复杂了。

SIMD 的处理办法是掩码。对八个 x 并行做 $x > 0$ 的测试，得到一组真假值，把它当作掩码决定后续哪些操作生效：为真的通道执行，为假的通道被禁用，或虽然计算但结果不写回寄存器。然后翻转掩码，执行 else 分支的操作。

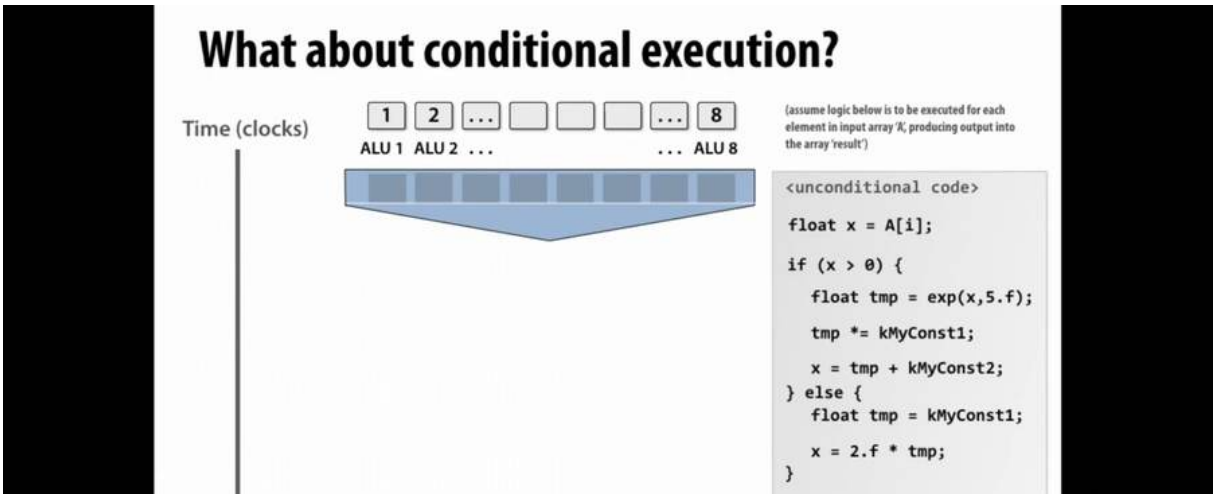

图 14: 条件执行的困境：八个通道里部分走 if、部分走 else[14]

这样做的代价是：分支被展平为直线代码，但两边的操作都要执行。若两路分支的工作量大致相当，那么任一时刻约有一半的 ALU 空转（即整体效率上限约 50%），效率最高只有 50%（视频 00:42:13）。极端情况下只有一两个元素真正需要执行，其余七个通道的算力全部浪费。

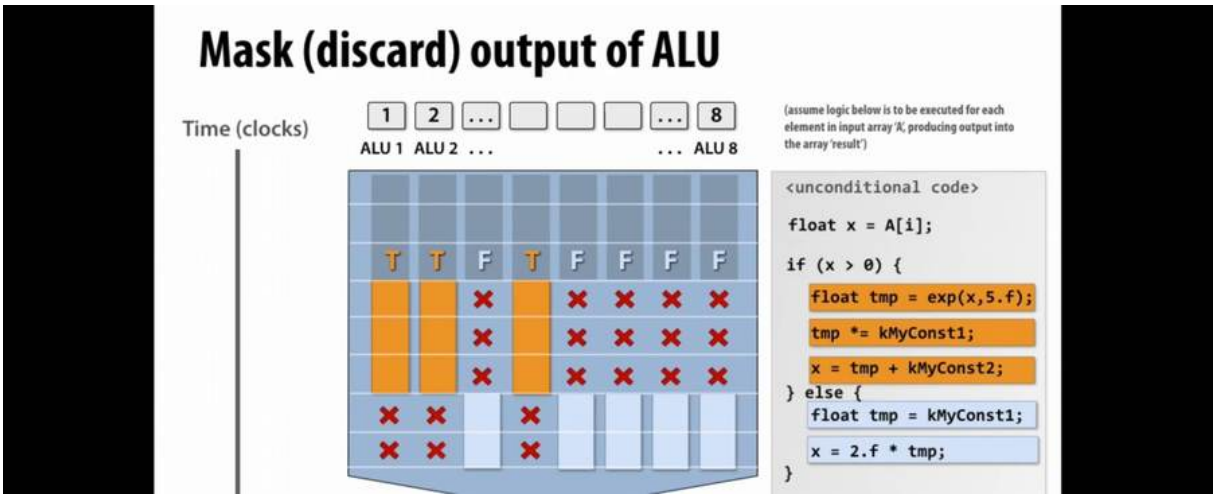

图 15: 用掩码丢弃 ALU 输出：为假的通道计算但不写回结果[15]

讲者也指出了好消息：分支结束后所有通道重新汇合到同一程序点，可以继续同步执行。

分支越细碎，向量化收益越低

掩码机制的成本取决于两路分支是否都要执行。若整批数据的判断结果一致，可以跳过另一路，效率接近满额；若两路混合，则两路都跑，效率腰斩；若只有个别通道需要走特殊路径，其余七路陪着空转，效率只剩八分之一。因此在 SIMD 代码里，把判断按数据分组往往比逐元素判断更快。

[14] 视频画面时间区间：00:40:00–00:40:15。若为裁剪图，时间区间仍对应原视频画面。
[15] 视频画面时间区间：00:41:40–00:41:55。若为裁剪图，时间区间仍对应原视频画面。

coherence 与 divergence

讲者为这一现象引入了两个术语，并特意提醒它们与后面讲缓存一致性时的 coherence 毫无关系。

coherence 与 divergence（SIMD 语境）

coherence 指同一组通道在多大程度上想做同一件事，程度越高，这批并行资源的利用率越高。divergence 指偏离这一理想的程度——通道之间的行为越不一致，被浪费的算力越多。判断一段代码是否适合 SIMD，很大程度上就是判断它会不会产生大量 divergence。

向量还能有多宽

讲者提到下一代是 AVX512，512 位。这里有一个命名上的提醒：512 是位数，与第几代无关，所以没有 AVX3。

关于是否继续加宽，他转述了内行的看法：这大概就是极限了。理由正是 divergence——真实程序里能够整齐一致的向量长度有限，一味加宽会让浪费的比例越来越高。

本章小结

SIMD 用一条指令驱动多个数据，在数据整齐时能廉价地成倍提高吞吐，16 核乘 8 路就是 128 倍。但它对数据一致性有强要求：一旦出现分支，掩码机制会让两路都执行，常态效率掉到一半，极端情况只剩八分之一。这个弱点在 GPU 上被推向极致——下一章看 GPU 如何接受这个代价。

2.5 GPU：把控制逻辑换成 ALU

GPU 是 SIMD 思路的极端化。理解它的关键不是” 核多”，而是它主动放弃了什么。

SPMD：对外是单程序多数据

GPU 给程序员呈现的模型叫 SPMD（single program multiple data，单程序多数据）：不同部分可以执行不同的代码路径。但在底层，它仍然用 SIMD 来实现大部分逻辑。

程序员表达的是” 我要对两个各含 n 个值的向量做 n 次加法”，并可以启用或禁用其中某些通道；硬件则负责把这项工作切成大小固定为 32 个值（不是字节）的块，逐块执行。这个块大小叫 SIMD 宽度，历史上在 8 到 32 之间，目前多数是 32。

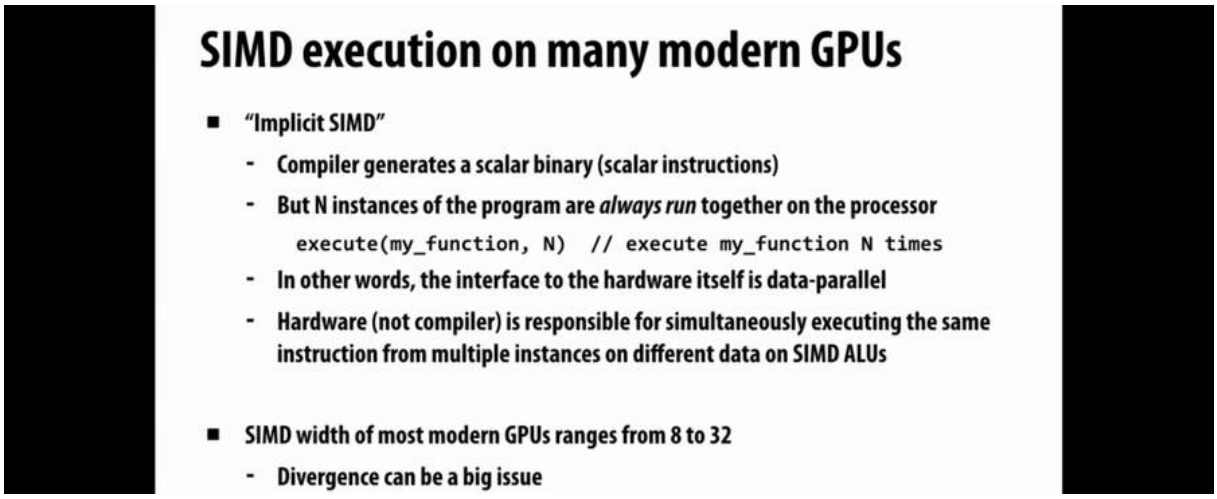

图 16: 现代 GPU 的隐式 SIMD：编译器生成标量二进制，硬件按 SIMD 宽度分块执行[16]

去掉控制逻辑换来的算力

GPU 的设计哲学可以用讲者一句话概括：分支预测、乱序控制这些逻辑是" 让你算不完的更多办法"，它们并不做程序真正要求的计算。GPU 把这些统统扔掉，塞进尽可能多的核心，用另一种特殊方式控制。

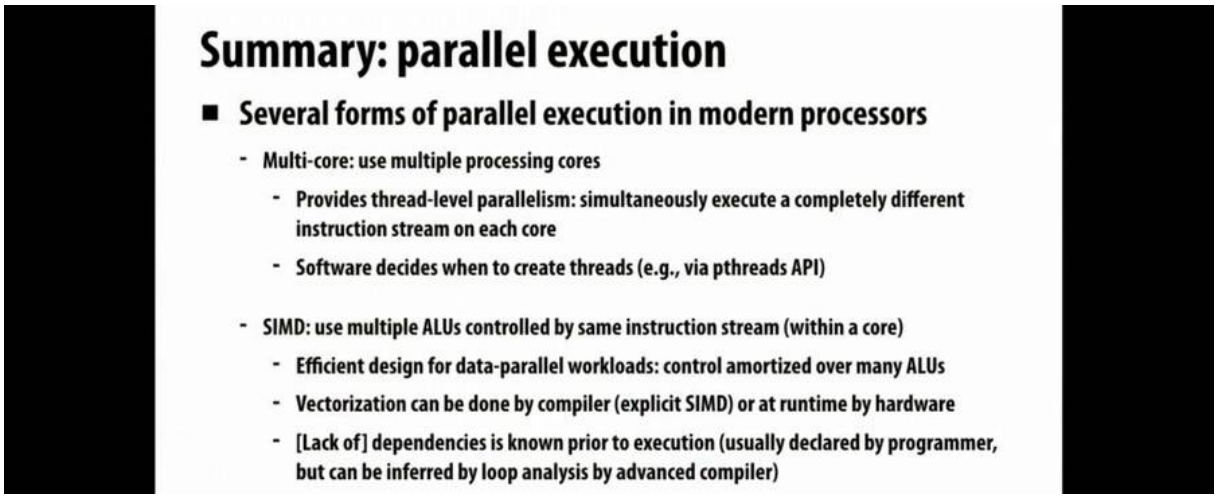

图 17: 并行执行方式汇总：多核、SIMD 与超标量三条路径[17]

讲者给出的真实机器参数让这个取舍变得具体。课程用的 Gates 机器是 2016 年前后的型号，4 个核心、每核支持 AVX2，按 4 字节数据算相当于每核 8 个并行 ALU。课程后期还会用到一台 12 核的集群。Gates 机器上还挂着 NVIDIA GPU，20 个核心、每核 32 个 ALU。

[16] 视频画面时间区间：00:49:20–00:49:35。若为裁剪图，时间区间仍对应原视频画面。
[17] 视频画面时间区间：00:53:00–00:53:15。若为裁剪图，时间区间仍对应原视频画面。

设备	核心数	每核 ALU	原始算力上限
Intel CPU（Gates）	4	8（AVX2）	不足 100 GFLOPS
集群节点	12	8（AVX2）	同量级
NVIDIA GPU（Gates）	20	32	8 TFLOPS

表 2: 课程所用机器的核心数、每核算力与峰值算力对照

算力差距是最有说服力的一点：GPU 全部加总能到 8 TFLOPS，即每秒八万亿次浮点运算；而 CPU 每核约二十几 GFLOPS，乘四核也不到 100 GFLOPS。两者之比约为 80 比 1（视频 00:52:32）。这个差距完全来自把控制逻辑的面积换成了 ALU。

上面这张表里 CPU 一列的数字来自讲者现场展示的 i7 参数页（视频 00:50:00）；GPU 一列取自同一实验室在用的显卡参数页，下图即该页。

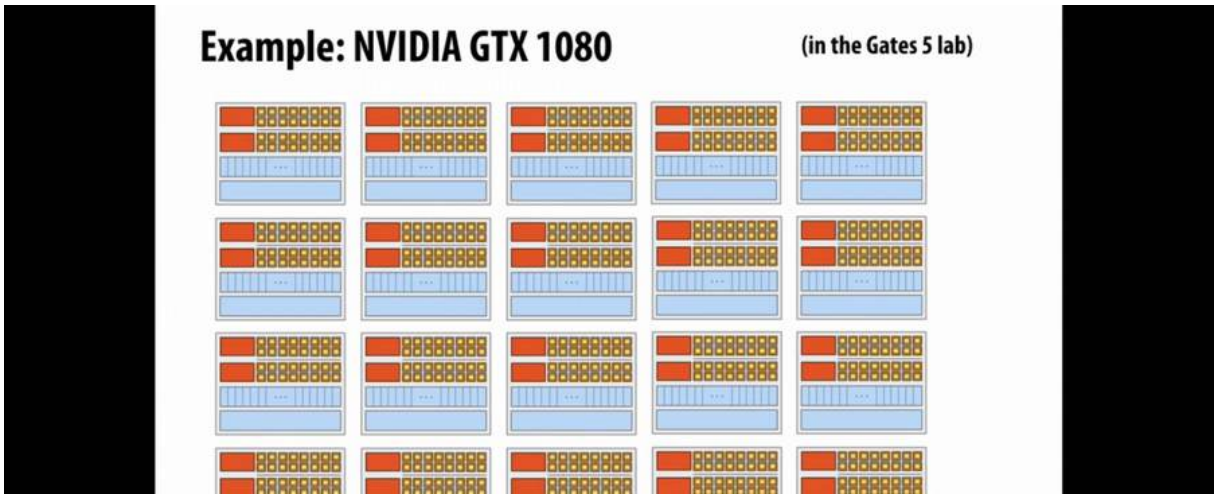

图 18: NVIDIA GTX 1080 的参数页：核心数与每核 SIMD 宽度的来源[18]

本章小结

GPU 用 SPMD 包装 SIMD，用放弃分支预测与乱序控制换来数十倍的 ALU 密度，因此峰值算力可达 CPU 的八十倍。但讲者留下一句关键限定：问题在于这些算力能不能被程序用起来。这句话的答案藏在内存里——下一章处理。

2.6 内存：真正卡住并行的那一环

计算能力已经很多了。这一章说明为什么大多数程序拿不到这些能力。

延迟的数量级落差

讲者指出，这些年芯片变快了、内存也变大了，但把一字节数据从内存送进处理器所需的时间几乎没有改进。这包含两个量：延迟，即一次访问需要多久；吞吐，即那条连接内存与处理器的相对狭窄通路上每秒能过多少字节。

[18] 视频画面时间区间：00:51:20–00:51:35。若为裁剪图，时间区间仍对应原视频画面。

两者的数量级值得记住。DRAM 访问时间约为 100 纳秒，而 1 GHz 处理器的时钟周期是 1 纳秒，现代处理器普遍在数 GHz（视频 00:54:38）。也就是说一次主存访问要等上几百个时钟周期。带宽则是每秒 25 GB/s 的量级（讲者说的数十 GB/s 按课程机器约 25 GB/s 计）。

三级缓存与共享结构

缓存的目的是让大多数访问落在更小更快的存储里，从而把 100 纳秒的延迟换成几个纳秒（讲者提到的量级是约 5 纳秒，视频 00:56:19）。典型机器有三级缓存。

多核机器上，靠近核心的缓存通常为每个核心私有，而最大的 L3 由所有核心共享。这一点有实际意义：核心之间可以通过一个写、另一个读来传递数据，这些数据只需经过 L3，不必一路写到主存。

缓存解决延迟，带宽解决不了

缓存把常见访问从主存搬到片上，改善的是单次访问要等多久。它同时也能提高有效带宽，因为片上连线短且数量多。但这两种收益都有上限：缓存容不下工作集时，访问仍会落到主存，此时决定性能的是内存总线的每秒字节数，与缓存层级设计无关。

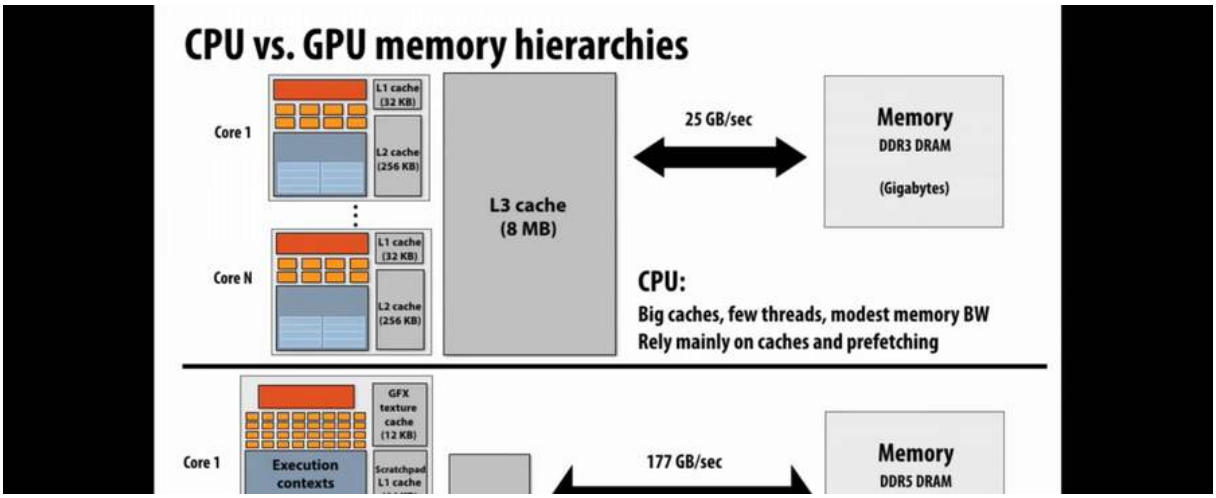

图 19: CPU 与 GPU 的内存层次对比：缓存容量、私有与共享关系、以及带宽标注[19]

预取：猜你要访问哪里

既然延迟这么高，硬件就像猜分支方向一样去猜你接下来要访问哪个内存地址，这就是预取。

它并不读你的代码，只观察地址序列：如果发现你在按固定步长（例如每隔四个位置）访问，就推断这个模式会继续，并提前把数据从主存搬进缓存。这样等你真正执行到那条加载指令时，数据已经在缓存里了。现在几乎所有硬件都做这件事。

内存山：预取改变了曲线的形状

讲者用两台机器的对比来说明预取的效果。他用家里一台 2008 年的 iMac 测过一组数据，那是当年相当不错的处理器；另一组是课程用的 Gates 机器。

[19] 视频画面时间区间：00:48:00–00:48:15。若为裁剪图，时间区间仍对应原视频画面。

横轴是数组总大小，从左端每次只访问一个字开始向右增大。左端数据量小，全部在 L1 里反复循环，带宽最高。越过 L1 容量后开始不断缺失（miss），带宽出现明显下跌，随后数据稳定落在 L2 中，这个台阶反映的是 L1 与 L2 的相对性能。再往右落入主存，又一次下跌。

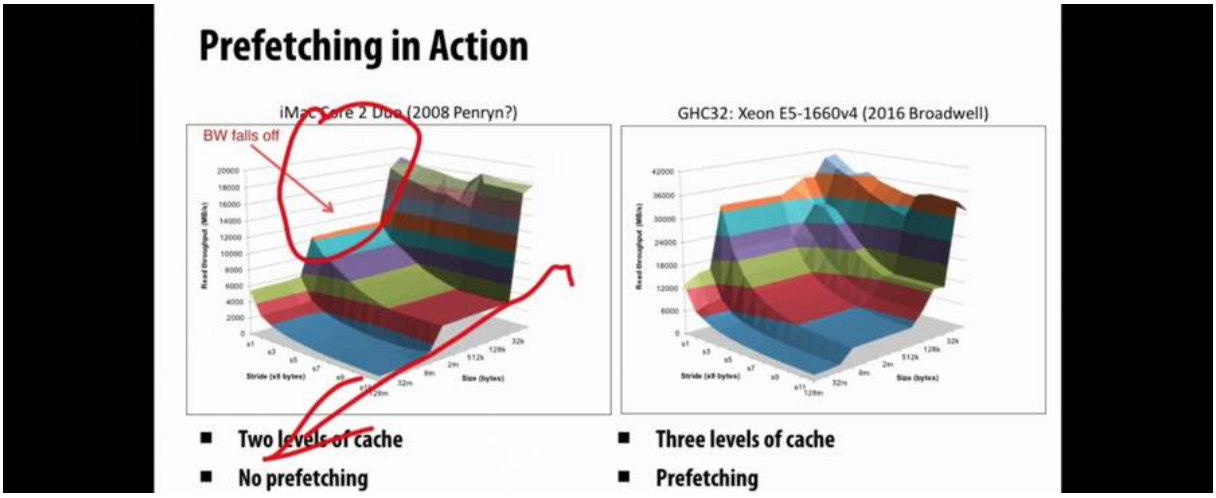

图 20: 预取生效的实证：新一代机器的带宽台阶更平缓，且出现 L3 的额外凸起[20]

Gates 机器的曲线差异极大：下跌幅度小得多，中间还多出一个小凸起，那是 L2，后面是 L3。讲者明确把这一改善归因于预取。

但预取的作用有严格边界：它能减少延迟，却不能提高吞吐。一旦访问规模超出 L3，仍然要撞上内存总线有限的带宽，此时曲线再次陡降。讲者说他跑到树莓派之类的设备上才容易观察到没有预取的原始形态。

预取对随机访问无效

硬件预取依赖可识别的地址模式，例如固定步长的循环。指针追逐、哈希查找这类访问模式没有规律，预取器无法预测，延迟会完整暴露。这类代码的优化方向是改变数据布局以提高局部性，而不是指望硬件猜准。

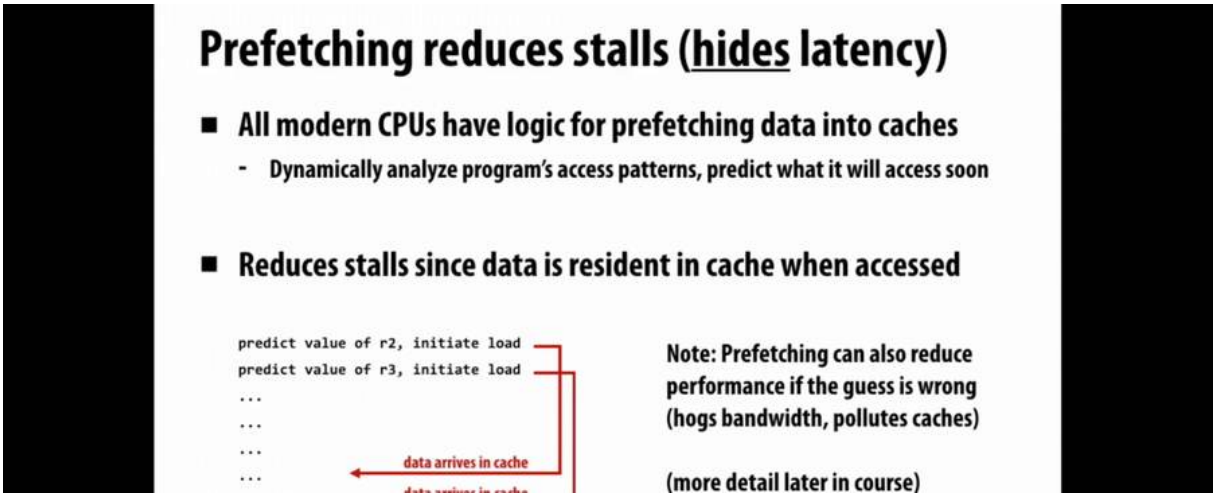

图 21: 预取的机制：根据地址访问模式提前把数据搬入缓存[21]

[20] 视频画面时间区间：01:00:00–01:00:15。若为裁剪图，时间区间仍对应原视频画面。

本章小结

主存延迟比时钟周期高两个数量级，缓存把多数访问拉回纳秒级，预取通过猜测地址模式进一步压缩等待。但预取只改善延迟，不改变带宽，一旦数据量超过共享缓存，性能就由内存总线决定。这条限制指向一个尚未使用的手段：既然处理器在等内存时闲着，能不能让它去干别的？

2.7 多线程与超线程：用吞吐换延迟

这一章回答上一章末尾的问题。核心思路是：等待内存时处理器无事可做，那就安排另一条独立的执行流在这些空隙里运行。

执行上下文：多线程需要什么硬件

做法是把多个独立线程的处理交错在同一核心上。设想第一个线程发起一次八路向量加载，数据不在缓存里，它就停顿了。如果此时另一个线程也有八次读取准备好，硬件就可以发起它，让它在同一个停顿窗口里推进。

这需要的硬件不是更多 ALU，而是多个执行上下文。讲者给的定义是：每个线程的寄存器状态，加上足以存放栈或至少是栈的当前部分以及其他必要状态的内存。所有这些线程还会共享同一批缓存。

共享缓存带来一个真实的副作用：内存不足时线程之间会互相挤占缓存。但讲者的判断是这件事总体划算，因为它们填进去的正是本来无所事事的时间。

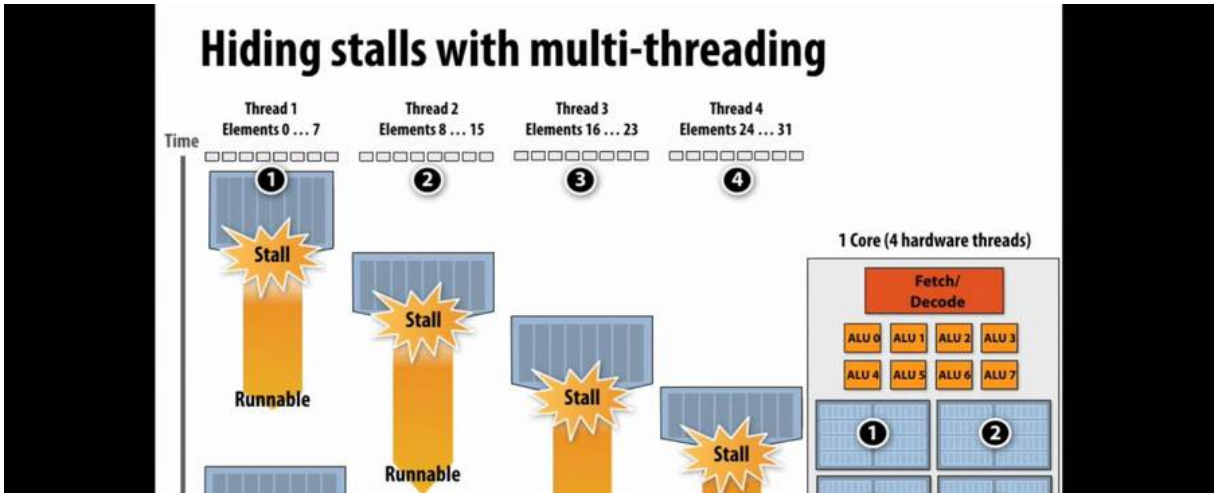

图 22: 用多线程隐藏停顿：四条线程在时间轴上交替填充彼此的等待空隙[22]

代价：单线程变慢

讲者把这类做法称为面向吞吐的计算。他坦率承认代价：上面的例子里，第一条线程的单独性能其实是下降的，因为有时别的线程会占用资源，即使它当时已经可以继续执行，也得等。

这引出了课程反复出现的一条主线：

$$\text{并行化} \Rightarrow \text{关键资源从延迟转向吞吐}$$

[21] 视频画面时间区间：00:58:00–00:58:15。若为裁剪图，时间区间仍对应原视频画面。

[22] 视频画面时间区间：01:04:00–01:04:15。若为裁剪图，时间区间仍对应原视频画面。

- 延迟：单次操作需要多长时间；
- 吞吐：单位时间内系统能送过多少工作；
- 讲者的判断是提高吞吐比降低延迟更容易实现。

代价的另一个侧面是资源分配：只跑一个线程时，整个 L1 缓存和寄存器组都可以给它用；如果要交错十六条线程，每个线程能分到的存储就更少。

超线程的实现

Intel 把这项技术叫超线程（hyperthreading），非商业的叫法是同时多线程（simultaneous multithreading）。讲者特意指出它并非 Intel 发明，所以这个名字不算他们的功劳。

实现方式是利用现代乱序处理器中本就存在的大量硬件：增加少许逻辑，并把寄存器复制若干份，让每个线程有独立的寄存器组，而执行单元、内存与缓存仍然共享。当某线程因内存问题停顿，硬件立即切换到另一线程执行。

GPU：把这条思路推到极致

在 GPU 上这种做法被推到极端。讲者以某个型号为例：每个核上相当于 48 路交错的线程，而这样的核有 15 个。相乘之后，需要有两万三千多个可独立操作的数据值，才能让整块芯片持续跑满。这个逻辑并发度极高。

代价同样清晰：GPU 的内存系统是为极高吞吐调优的，缓存相对较小，连主存容量也不能做得很大。缩小缓存腾出的面积，又被用来堆 ALU。

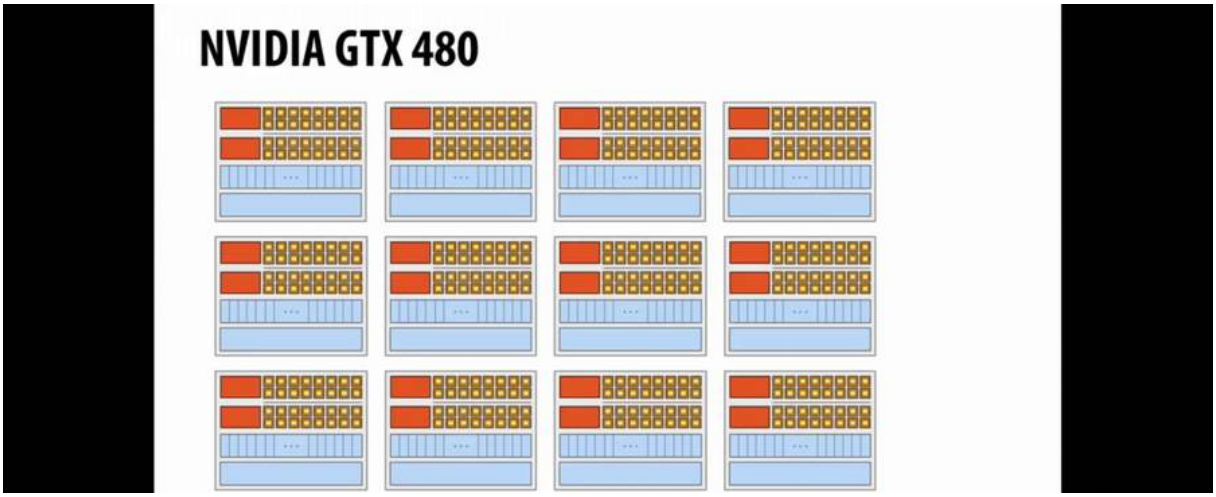

图 23: NVIDIA GTX 480 的结构细节：每核多路线程与宽向量单元[23]

本章小结

多线程用多个执行上下文填充内存等待的空隙，本质是把延迟问题转化为吞吐问题，代价是单线程性能与缓存份额。超线程让这件事在普通 CPU 上几乎免费；GPU 则把它与宽 SIMD 叠加，得到极高的逻辑并发度，同时换来小缓存与有限主存。至此所有提高算力的手段都已就位，最后要问的是：这些算力实际能用上多少？

[23] 视频画面时间区间：01:17:00–01:17:15。若为裁剪图，时间区间仍对应原视频画面。

2.8 算术强度：峰值算力能兑现多少

这一章用一次思想实验给出全讲最有说服力的定量结论。

思想实验：逐元素相乘

任务是对两个向量 A 和 B 做逐元素相乘，结果写入 C。向量包含数百万个元素。每个元素需要两次加载（A[i] 与 B[i]）、一次乘法、一次存储。

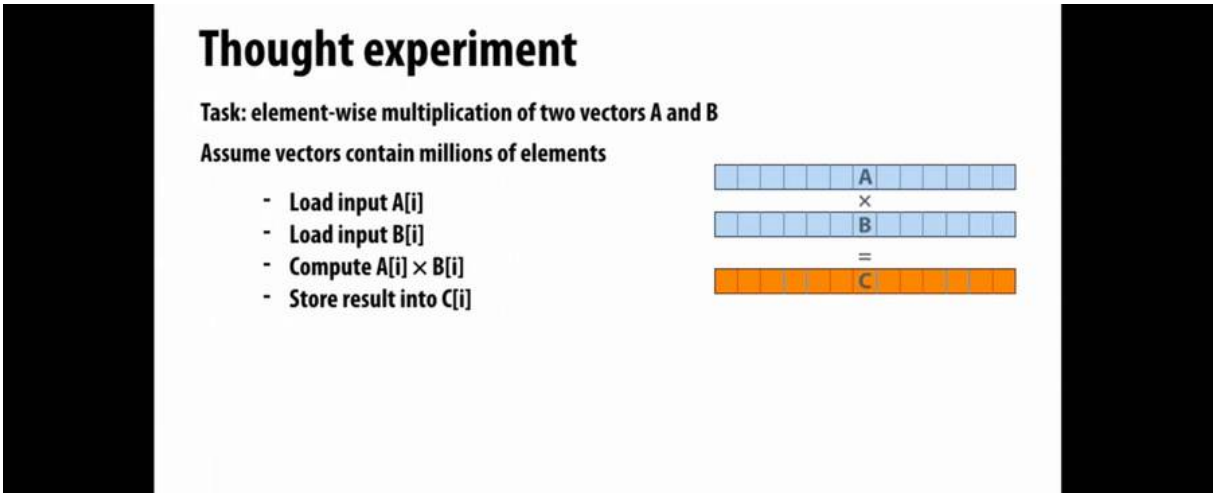

图 24: 算术强度思想实验：逐元素相乘每次运算需要两次读取与一次写入[24]

讲者指出这段代码性能表现很差，因为它对内存极度饥渴：每做一次算术运算就要两次读和一次写。这类代码的特征叫算术强度低。

算一下这笔账

讲者用某个型号的 GPU 做了具体计算。它每时钟周期能做 400 次乘法。要让这些乘法全部有数据可用，需要的带宽是：

$$B_{\text{need}} = \frac{400 \times 4\text{字节} \times 3\text{次访问}}{\text{每周期}} \approx 6.4 \text{ TB/s}$$

- 400：该 GPU 每时钟周期可完成的乘法次数；
- 4 字节：每个单精度浮点数的宽度；
- 3 次访问：每个元素两次读加一次写；
- 6.4 TB/s：喂满这些乘法所需的内存带宽。

该 GPU 实际只有 177 GB/s（视频 01:19:42），缺口超过三十倍。于是它只能被这条管道完全限制住：

$$E_{\text{mem}} = \frac{177}{6400} \approx 3\%$$

- 177 GB/s：该 GPU 的实际内存带宽；

[24] 视频画面时间区间：01:19:00–01:19:15。若为裁剪图，时间区间仍对应原视频画面。

- 6400 GB/s：充分供数所需带宽；
- 3%：实际能达到的算力占峰值的比例。

讲者的说法是这块核被内存带宽饿死了，效率只有 3%（视频 01:19:55）。

但即使饿着也快七倍

这个结论有个容易误读的地方，讲者专门澄清：即便被带宽卡死，这个 GPU 仍然比同代的 Xeon 快大约七倍（视频 01:20:00）。原因是 Xeon 的带宽更低（讲者提到约 25 GB/s，现代 Xeon 更高一些），在这种以带宽为主导的计算上同样受限，而且起点更差。

不要把 3% 误读为"GPU 不适合这类计算"

3% 的峰值效率听起来像失败，但它的比较对象是该 GPU 自己的算力峰值，不是 CPU。在算术强度低的计算里，GPU 的绝对速度仍可能是 CPU 的数倍。判断用哪个设备，要看绝对耗时，而不是各自峰值利用率的比例。

由此得到的判断规则

讲者把这条经验提升为课程的一条主线：内存速度往往比算术速度更是限制，无论你编的是 GPU 还是传统处理器。

场景	决定性能的资源	优化方向
算术强度高	算术单元	提高并行度、向量化、减少分支
算术强度低	内存带宽	提高数据复用、分块、减少搬运
数据超出共享缓存	内存总线带宽	改善访问局部性，而非增加核心

表 3: 按算术强度判断瓶颈所在的规则

本章小结

当每字节数据只参与很少几次运算时，算力规模毫无意义，性能由带宽决定：400 次乘法需要 6.4 TB/s，实际只有 177 GB/s，效率落到 3%。但这条管道劣势下 GPU 仍比 CPU 快七倍，说明比较必须用绝对速度。讲者的结论是本课最重要的判断依据——多数程序的瓶颈在供数，不在算力。

2.9 总结与延伸

这一讲没有写任何并行算法，它给出的是硬件的能力地图，以及读取这张地图的方法。

讲者开篇设下的主线在结尾闭合：为了利用并行，硬件在多个层次上都放置了潜力，其中一部分对程序员不可见，一部分必须显式生成代码。全讲把这些层次一一展开——指令级并行（自动，上限三到五倍）、SIMD（需要 intrinsics 或聪明的编译器，宽度从 128 位到 512 位）、多核（需要显式线程）、多线程与超线程（用吞吐换延迟）、GPU（放弃控制逻辑换八十倍 ALU 密度）。算力一路放大到 128 倍乃至更高，但每一层都附带一个兑现条件。

本讲义在讲者材料之上做的压缩，有四点值得单独记住，需与讲者原话区分：

- 算力的增长与可用性并不同步。峰值从不足 100 GFLOPS 涨到 8 TFLOPS，但低算术强度的核只用到 3%，实际收益取决于数据能否供上。

- SIMD 的效率由数据一致性决定。128 倍的潜力在两路分支下常态只剩一半，极端情况只剩八分之一；divergence 是判断代码是否适合向量化的核心指标。
- 延迟与吞吐是两个必须分开讨论的目标。缓存与预取处理延迟，多线程把延迟转成吞吐，而带宽是两者都无法突破的硬约束。
- 判断瓶颈要用绝对速度而非峰值利用率。3% 的效率仍可能比另一台机器快七倍。

这堂课的边界同样明确。讲者反复说明分支预测、乱序执行、超线程的细节更属于计算机体系结构课程，本课不会深挖；SIMD 与多线程的具体编程会在后续作业中展开。也就是说，本讲给出的是选型与诊断的框架，不是调优手法。

两个可以直接从本讲数据提出的下一步问题：其一，16 核乘 8 路的 128 倍潜力，在真实的分支代码里平均能兑现多少，这个折扣与第 3 讲的负载不均折扣相比谁更大？其二，如果算术强度是决定因素，那么把同一份计算分块以提高缓存复用，能使 3% 的效率提高到什么水平？二者的答案都取决于具体的数据规模与访问模式，也因此需要第 3 讲要讲的性能测量方法，而不是先验推断。

Lecture 03 Parallel Programming Abstractions

视频时长：1 小时 7 分 38 秒 来源：https://www.bilibili.com/video/BV1G94y1k7zt/

3.1 抽象与实现：并行编程最容易混淆的一对概念

这一讲要解决的问题很简单，却几乎人人都会在这一步出错：当你说”线程”时，你说的是语言的抽象，还是硬件里真的有一个线程？

为什么这两个概念必须分开

抽象是为人服务的，它让程序更容易表达、更容易调试。但抽象终归要被实现。并行领域特别容易把两者混淆，因为抽象和实现都在处理同一件事——并发与通信。语言用抽象表达并发，系统也在管理并发，两者用着相似的名词，含义却完全不同。

讲者把这一讲的题目定为”抽象与实现的区别”，并选择 Intel 的 ISPC 语言作为第一个案例，因为这个语言把反差做到了极致。

本讲的两条主线

课程后续的所有内容都可以归到一组对照上：程序员看到的模型是什么，机器实际做的事情是什么。本讲后面 ISPC 讨论部分的一张幻灯片把这对关系写成了两行，如下图所示。

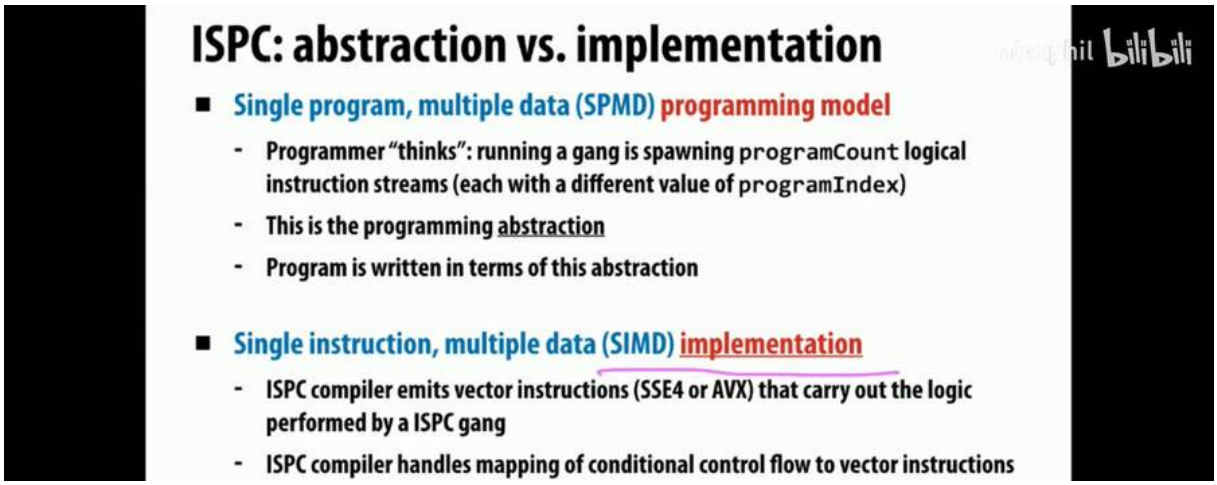

图 25: ISPC 的抽象与实现对照：抽象是 SPMD 编程模型，实现是单线程上的 SIMD 指令[25]

第二主线是通信。并行与串行真正不同之处，在于并发执行的多份代码必须相互通信、相互配合。因此每个抽象都要回答同一个问题：你用什么手段来表达通信。

本章小结

抽象是人造的便利，实现是机器的真相，并行领域因为名词重合而特别容易混为一谈。本讲用 ISPC 演示最尖锐的一例——看起来是多线程，实际是单线程加向量指令；随后讨论另外三个主流抽象，并分别指出它们的实现方式与代价。

[25] 视频画面时间区间：00:22:30–00:22:45。若为裁剪图，时间区间仍对应原视频画面。

3.2 ISPC：看起来像多线程，实际是向量指令

ISPC 是这一讲的核心案例。理解它，就能看清抽象与实现能差多远。

SPMD 抽象的定义

ISPC 的核心抽象叫 SPMD，即单程序多数据。它的含义是：有多个并发执行体，每个执行体运行同一段代码，但各自处理不同的数据。

讲者特别解释了为什么他尽量回避”线程”这个词。抽象里说的是逻辑上并发的实例，它们可能同一时刻执行，也可能不是严格同步；关键在于给它们同一段代码，让它们各自在自己的数据上执行。

与之相对的是另一种风格：完全不同的程序处理不同的数据。浏览器这类软件就是这样——多个互不相关的任务各做各的事，执行的是完全不同的过程。但在这门课里，目标是把同一件事做快，所以采用同一份代码的模式。

ISPC 的代码组织

ISPC 的模型是受限的、明确的：把需要并行执行的部分抽出来，放进独立的`.ispc` 文件。这些文件的代码看起来很像 C，只是扩展名不同。文件之外是调用它们的普通代码，例如 `main.cpp`，顺序执行，直到调用到`.ispc` 里的方法才进入并行。

ISPC 不用”线程”这个说法，而用 gang（一帮、一组）和 program instance（程序实例）。这个措辞刻意模糊，因为抽象层级上还没有落定到具体实现。

执行过程因此呈现三段：调用前顺序执行，进入 ISPC 方法时并发执行，返回后回到顺序执行。所有的并行都通过创建这些 ISPC 文件来表达。

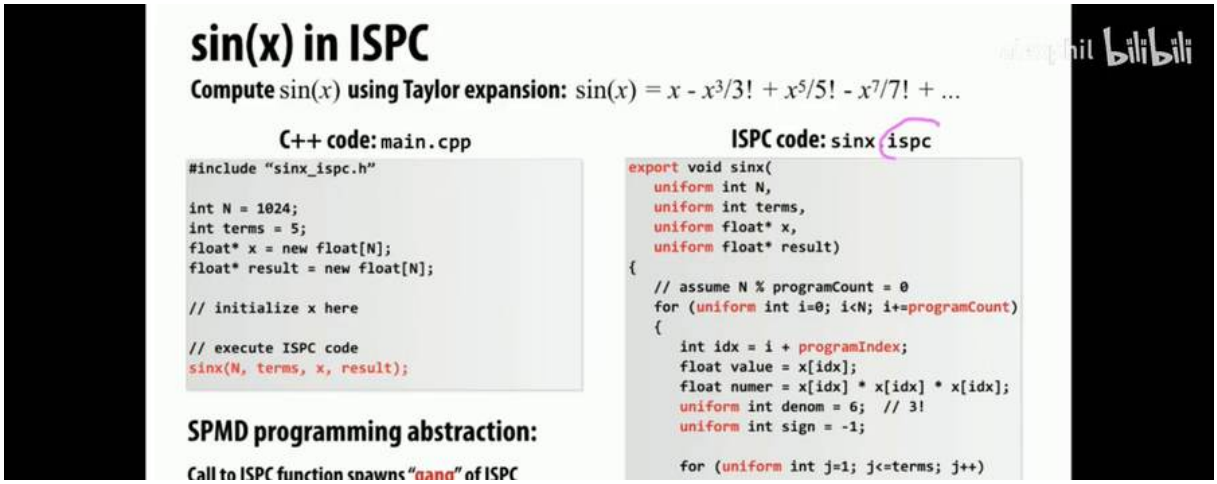

图 26: ISPC 版本的 sin(x)：C++ 侧保留顺序代码，并行部分放在.ispc 文件[26]

programCount 与 programIndex

ISPC 文件里有两个关键的内置变量。programCount 是这个 gang 中并发实例的数量，也就是同时有多少件事在并行。programIndex 则是”我是第几个实例”，取值从 0 到 $n-1$。

[26]视频画面时间区间：00:04:00–00:04:15。若为裁剪图，时间区间仍对应原视频画面。

programCount 有一个不寻常之处：程序员不能设置它。它由运行时系统决定。程序里只能读取。

利用这两个变量，原本的顺序循环被改写成跳跃式遍历：

```
// 在 .ispc 文件中
export void sinx(uniform int N, uniform int terms,
                 uniform float input[], uniform float result[]) {
    // 外层循环按 programCount 跳跃，而不是每次加 1
    foreach (i = 0 ... N) {
        // 每个实例计算自己的下标
        int index = i + programIndex;
        ...
    }
}
```

Listing 2: ISPC 用 programIndex 切分迭代（按讲者对两个内置变量的描述整理）

讲者解释了这个改写的作用：如果 programCount 是 4，外层循环一次前进 4，四个并发实例同时运行，各自的下标都由自己的 programIndex 决定，因此它们在做不同的工作。

一个必须避免的误解

如果只是把原来的 `sinx` 函数原封不动搬进 ISPC 文件并调用，会发生什么？讲者的回答是：它会创建 programCount 个并发实例，但每个实例做的工作完全相同，彼此完全冗余，因此程序不会变快。

把顺序代码搬进并行文件是最常见的空转

原样搬入 ISPC 文件后，programCount 个实例执行完全相同的工作，结果彼此覆盖，耗时不变。判断一份并行代码是否真的在工作，看的是各实例访问的下标是否互不相同，而不是文件扩展名。

这个反例值得记住。并行抽象本身不产生加速，真正产生加速的是把工作切分开。写并行代码的第一件事永远是划分数据，而不是套用并行语法。

uniform 提示

ISPC 还有一个修饰符叫 uniform。它标注的是：这个变量在所有并发实例中取值完全相同。

例如 index 就不是 uniform 的，因为每个实例根据 programIndex 和 i 算出的值不同；而参与计算的常量、数组长度这类量则是 uniform 的。

讲者说这是一个不寻常的提示——在多数并行抽象里没有这个概念。他承诺稍后揭开实现细节时，读者就会明白为什么需要它。

gap 大小的真相

到 ISPC 讨论部分，讲者揭开了底层实现。ISPC 是为一个明确目的设计的：让编写利用 SIMD 向量指令的代码变得容易。如果手写 intrinsics，要逐个指明做向量加、向量乘，相当麻烦。ISPC 的做法是让程序员写看起来普通的并发代码，由编译器在背后生成向量指令。

programCount 是实现的函数，不是程序员的参数

programCount 由运行时按机器与数据类型算出来，程序员只能读。这意味着同一份 ISPC 源码在 256 位与 512 位机器上会有不同的并发实例数，循环的跳跃步长随之改变。写 ISPC 时不能假设某个固定的 gang 大小。

因此那个不¨ 寻常的¨ 答案就出来了：

$$\texttt{programCount} = \frac{\text{向量宽度}}{\text{单元素位宽}}$$

- programCount：gang 中并发实例数，由运行时决定；
- 向量宽度：机器 SIMD 寄存器宽度，例如 256 位；
- 单元素位宽：数据类型的位数，例如单精度浮点为 32 位。

对 256 位寄存器与 32 位单精度浮点，programCount 就是 8。所谓 gang 里并发的八个实例，实际上是向量寄存器中的八个通道。

讲者紧接着给出结论：ISPC 生成的不是多个线程，而是单一的线程执行流，并行完全来自 SIMD 向量指令。

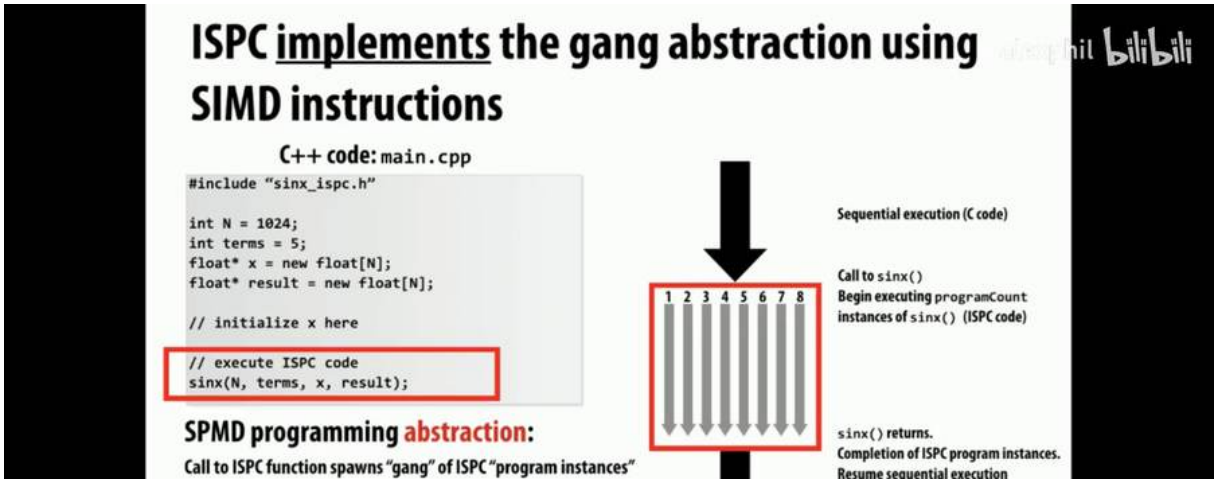

图 27: ISPC 用 SIMD 指令实现 gang 抽象[27]

跨核扩展：task

programCount 通常就是向量宽度，但机器往往有多个核心。前面所有代码只会跑在一个核上，用不到其余核心。

因此 ISPC 提供另一个机制：task。task 是可以在不同核心上运行的控制流，它会被转成真正的线程。

[27] 视频画面时间区间：00:11:30–00:11:45。若为裁剪图，时间区间仍对应原视频画面。

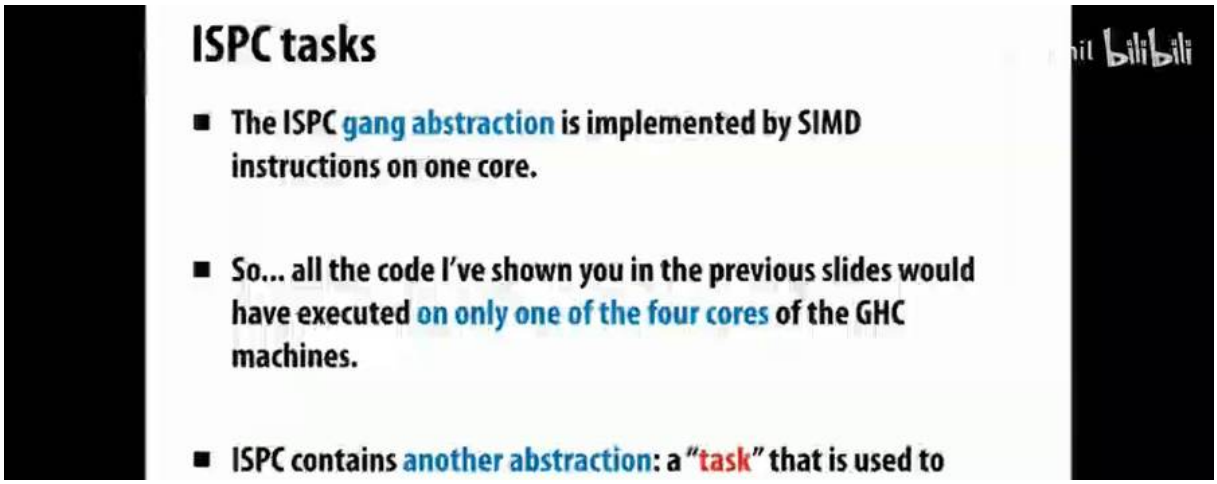

图 28: ISPC task：gang 抽象只用一个核，跨核需要显式创建 task[28]

本章小结

ISPC 的抽象是 SPMD 的多实例并发，实现是单线程加 SIMD 指令，programCount 等于向量宽度，uniform 提示正是为了让编译器能复用同一个向量寄存器。要跨越多个核心，必须显式使用 task。

这个案例确立了一条判断规则：看到" 并发实例" 时，先问它在硬件上对应什么。同样的抽象可以用完全不同的机制实现，性能特性也随之不同。

3.3 两种数据分布：interleaved 与 blocked

同样的 gang 抽象，把数据分配给实例的方式可以不同。这一节的结果出人意料：在 ISPC 上两者差距极大。

交叉分布 interleaved

前面看到的代码采用的是交叉分布：实例按 programIndex 轮流取数据元素。四个实例时，第 0 个实例处理下标 $0, 4, 8, 12\ldots$，第 1 个处理 $1, 5, 9, 13\ldots$，依此类推。

[28] 视频画面时间区间：00:26:00–00:26:15。若为裁剪图，时间区间仍对应原视频画面。

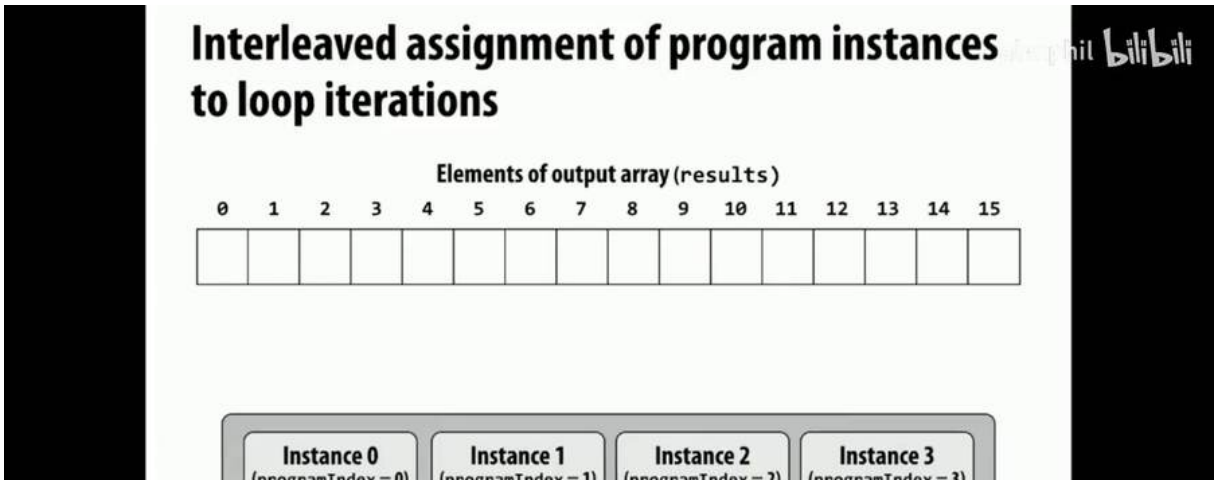

图 29: 交叉分布：程序实例与循环迭代的对应关系[29]

从某个实例的视角看，它跳着访问；但从整体看，任一时间点上四个实例正在处理连续的四个元素。这一点在后面决定了性能。

交叉分布的索引映射可以写成显式公式：

$$i_k = k + j \cdot P, \qquad k = 0, 1, \ldots, P-1$$

- i_k：第 k 个程序实例在第 j 轮迭代中处理的数据下标；
- k：programIndex，即实例编号；
- P：programCount，即 gang 中的并发实例数；
- j：外层循环的轮次。

阻塞分布 blocked

另一种做法是把数组切成连续的大块，每个实例负责一块。代码逻辑因此不同：先算出每块包含多少个连续元素，再算出自己从哪个位置开始。

阻塞分布的映射则是按块划分：

$$\text{chunk} = \left\lceil \frac{N}{P} \right\rceil, \qquad \text{start}_k = k \cdot \text{chunk}$$

- N：要处理的元素总数；
- P：程序实例数；
- chunk：每个实例负责的连续元素个数；
- start_k：第 k 个实例的起始下标。

讲者提醒暂时忽略 N 不能被 P 整除的情形，那属于边界处理而非模型本身。

[29] 视频画面时间区间：00:10:30–00:10:45。若为裁剪图，时间区间仍对应原视频画面。

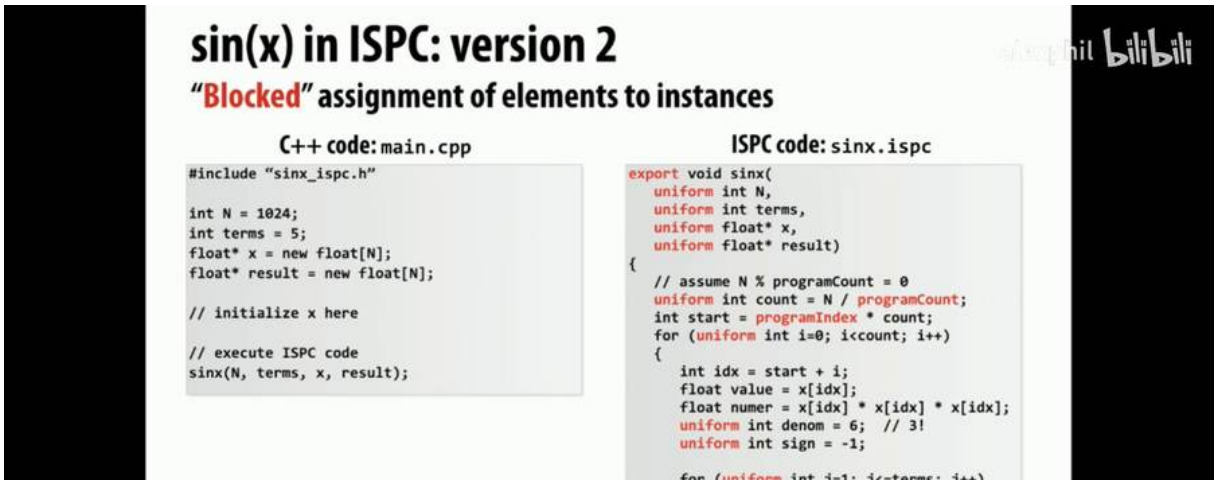

图 30: 阻塞分布的 ISPC 代码：先算块宽，再算起始位置[30]

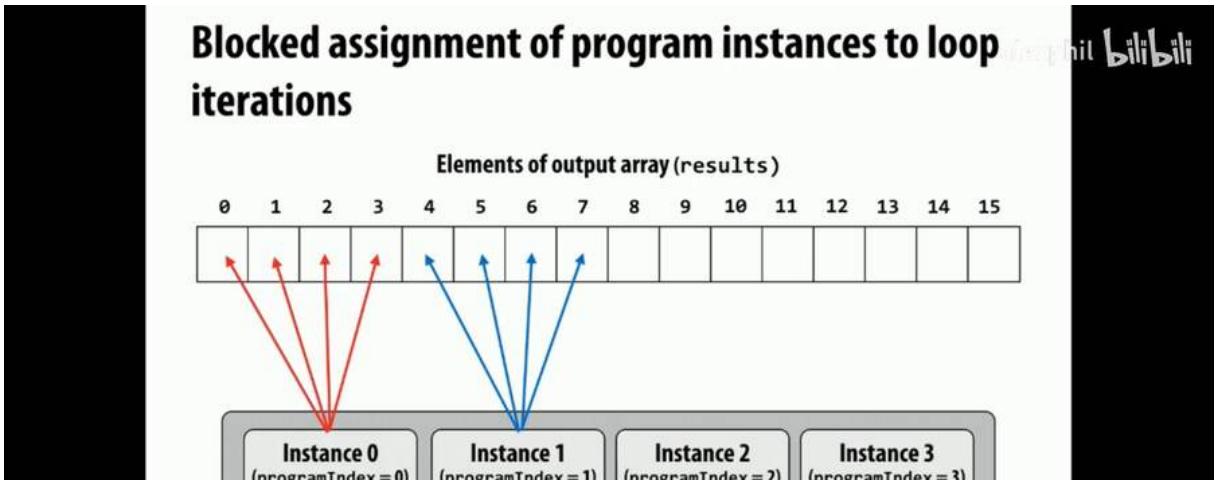

图 31: 阻塞分布下实例与迭代的对应关系[31]

两种分布各自的优缺点

讲者请学生比较两者，得到的是两个真实的权衡，但都不是最终答案。

阻塞分布的缺点是可能负载不均。如果计算时间随索引增长（例如随 n 的平方增长），那么均匀切块之后，左边那块任务很轻，右边那块任务很重。相比之下，交叉分布在计算量呈现连续变化时风险更小。

阻塞分布的优点是空间局部性好，这对缓存有利。如果多个线程各自在独立的缓存上工作，连续访问数据通常更快——这在 2 到 13 课程里是常识。

ISPC 中的真正答案

讲者随即指出，在 ISPC 上这两个分布有明显得多的胜负，而且原因不是上面任何一条。

原因是时间视角。在交叉分布下，任一时间点向量指令处理的是连续的下标 0 到 3。当要把数据装进向量寄存器时，需要的是连续内存——而这正好是交叉分布提供的。有专门的向量加载指令

[30] 视频画面时间区间：00:13:00–00:13:15。若为裁剪图，时间区间仍对应原视频画面。

[31] 视频画面时间区间：00:15:00–00:15:15。若为裁剪图，时间区间仍对应原视频画面。

可以一次搬入一大块连续数据，只要数据本身连续，效率极高：一次加载操作就取回全部需要的值。

阻塞分布则相反。时间上它需要同时加载下标 0、4、8、12，这些在内存中并不连续。硬件确实有向量 gather 指令能一次加载多个非连续元素，但代价远高于普通向量加载——它实际上相当于执行了四次或八次独立加载。从汇编程序员角度看这是一条指令，从性能角度看它非常昂贵。

其代价可以近似理解为：

$$T_{\text{gather}} \approx W \cdot T_{\text{load}}$$

- T_{gather}：一条 gather 指令的实际耗时；
- W：向量宽度，即一次装入的元素个数；
- T_{load}：一次普通标量加载的耗时。

这解释了为什么交叉分布在 ISPC 上胜出得如此干脆：它让向量加载始终能用到连续的 T_{load}，而阻塞分布只能反复付出 W 倍的代价。

抽象一致不代表性能一致

两种分布都能表达同一个 gang 抽象，都正确计算出结果，但在 SIMD 实现下一个比另一个快得多。性能差异来自实现层的数据搬运方式（连续向量加载对比 gather），与抽象层的语义无关。选择数据分布时，必须考虑目标实现，而不只是逻辑正确性。

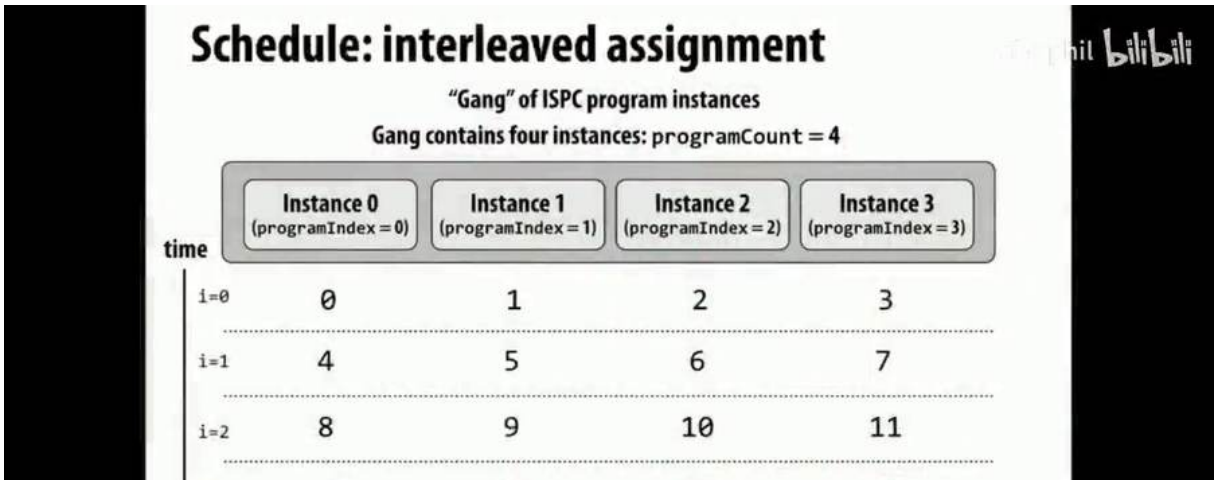

图 32: 时间轴上的交叉分布：同一时刻向量处理连续下标，可用单条向量加载[32]

[32] 视频画面时间区间：00:18:00–00:18:15。若为裁剪图，时间区间仍对应原视频画面。

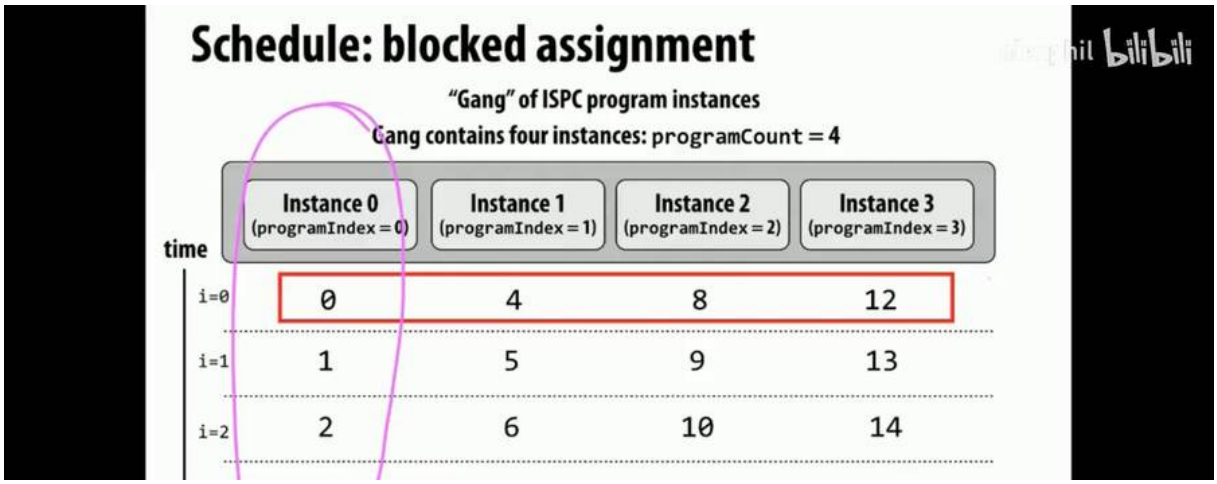

图 33: 时间轴上的阻塞分布：同一时刻需要跨步取数，只能依赖 gather[33]

让编译器决定：forEach

既然手工选择容易出错，ISPC 提供了 forEach：把 for 循环换成 forEach，等于告诉系统这些迭代可以并行、顺序由你决定。

forEach 交换的是确定性与可移植性

把迭代分配权交给系统，代码从此不绑定某一种数据分布，换实现时无需改写。代价是性能特征变得不那么直观：同一个 forEach 在不同编译器、不同目标架构下可能综合出不同的分布与访问模式。需要精确控制内存行为时，仍需退回手工分布。

系统会自行综合出代码，在 ISPC 里它选择交叉分布。

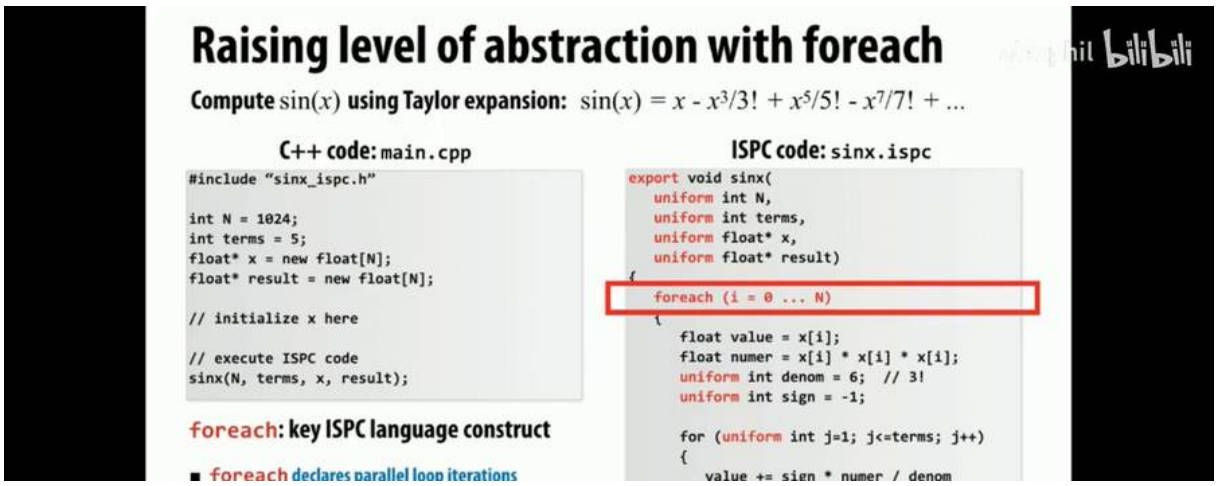

图 34: 用 forEach 提升抽象层次：交出迭代分配权，由系统综合[34]

[33] 视频画面时间区间：00:19:30–00:19:45。若为裁剪图，时间区间仍对应原视频画面。
[34] 视频画面时间区间：00:21:00–00:21:15。若为裁剪图，时间区间仍对应原视频画面。

本章小结

交叉分布与阻塞分布各有负载均衡与空间局部性的权衡，但在 SIMD 实现下交叉分布压倒性胜出，因为向量加载要求同一时刻的数据连续，而阻塞分布只能走昂贵的 gather。这个例子说明抽象正确不等于性能正确。

顺带一个限定：讲者提醒这个结论依赖 ISPC 是单线程加向量指令；若换成各自独立缓存的真线程，阻塞分布的空间局部性优势就会显现。

3.4 归约：为什么求和不能标 uniform

前一节处理的是”每个输出独立”的情形。归约则相反：所有实例共同产生一个结果。这一节看清了这个转变带来的约束。

失败的尝试

任务是把数组所有元素求和并返回。讲者提出一个自然的想法：既然最终大家应该得到同一个和，那就在 sum 声明前加 uniform。

ISPC 会直接报编译错误。

uniform 描述的是取值，不是逻辑意图

代码意图上本轮归约的最终结果确实只有一个值，但 uniform 修饰的是每个实例此刻持有的变量值。各实例累加的 x_i 不同，因此被修饰的变量在运行期间不 uniform。判断能否标 uniform，看的是运行时的取值分布，而不是作者期望的结果。

原因在数据依赖：每个实例要累加的是不同的 x_i，这些值本身不 uniform。如果让多个实例并发地往同一个变量里加，朴素实现会出错——这正是第 5 章要讨论的共享变量竞争问题在向量语境下的重现。

正确的做法：部分和加 reduce

可行的写法分两步。先让每个实例算出自己那部分的部分和，这些部分和确实每个实例不同、不 uniform。然后用 ISPC 的 reduce_add 原语把这些不同的值合并成最终结果。

归约在形式上就是把各实例的局部结果合并为单一值：

$$S = \sum_{k=0}^{P-1} S_k, \qquad S_k = \sum_{i \in I_k} x_i$$

- S_k：第 k 个实例算出的部分和，逐实例不同，因此不 uniform；
- I_k：第 k 个实例负责的下标集合；
- S：合并后的最终结果，全部实例共享同一值，可以标 uniform；
- P：程序实例数。

讲者展示了生成代码的样子：前半段是向量指令的加、加载、存储；后半段做归约时反而不用向量指令，而是用常规指令把向量的各元素逐个加起来。

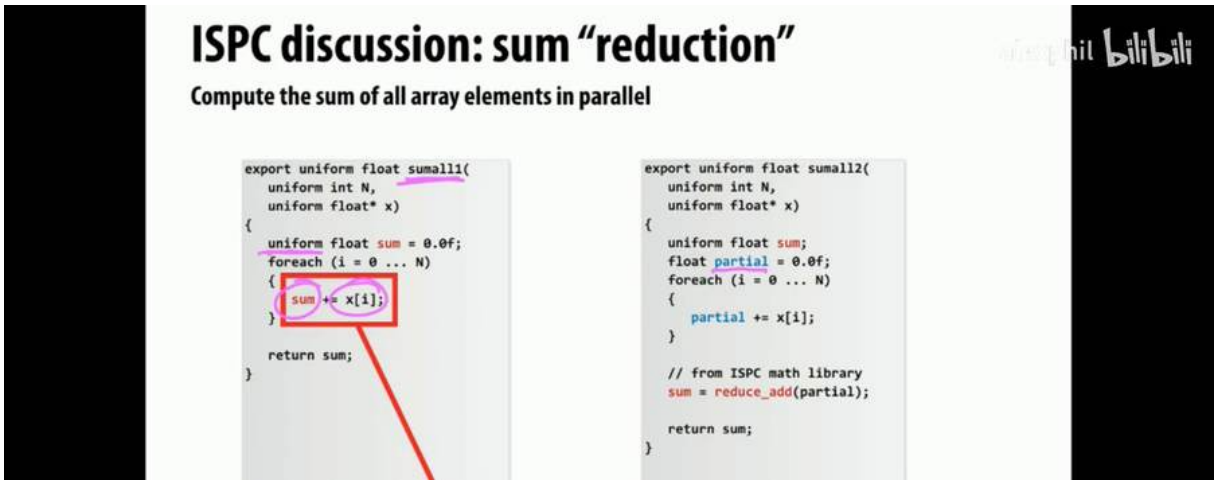

图 35: 求和归约：先算部分和，再用 reduce_add 合并[35]

归约的代价藏在最后一步

归约把并行度收拢成一个标量，这一步无法向量化，只能用标量指令逐个累加。因此归约不是免费的：它把前面并行获得的收益按元素数取对数或线性地吃掉一部分。频繁的小规模归约往往是并行程序的隐性瓶颈。

本章小结

归约破坏了”每个输出独立”的前提，因此不能声明为 uniform，必须显式用部分和加 reduce 原语，且最后一步退化为标量运算。至此 ISPC 的抽象、实现、数据分布与归约四条线索都已交代清楚。

接下来要问的是：ISPC 只是众多抽象之一，还有哪些主流抽象，它们的实现方式又差多远?

3.5 其他抽象及其实现层面的差异

ISPC 只是入口。这一节把视角拉远，比较三种主流并行抽象以及它们各自落在软件栈的哪一层。

系统分层：抽象在哪，实现在哪

讲者先建立了一幅分层的图景。程序员写高级语言，其下是编译器、运行时与库，再下面是操作系统，最下面是硬件。

抽象可以在软件实现、硬件实现，或者两者结合。而且”软件”本身也不是单一的一层。这个框架是判断性能来源的工具。

[35] 视频画面时间区间：00:24:30–00:24:45。若为裁剪图，时间区间仍对应原视频画面。

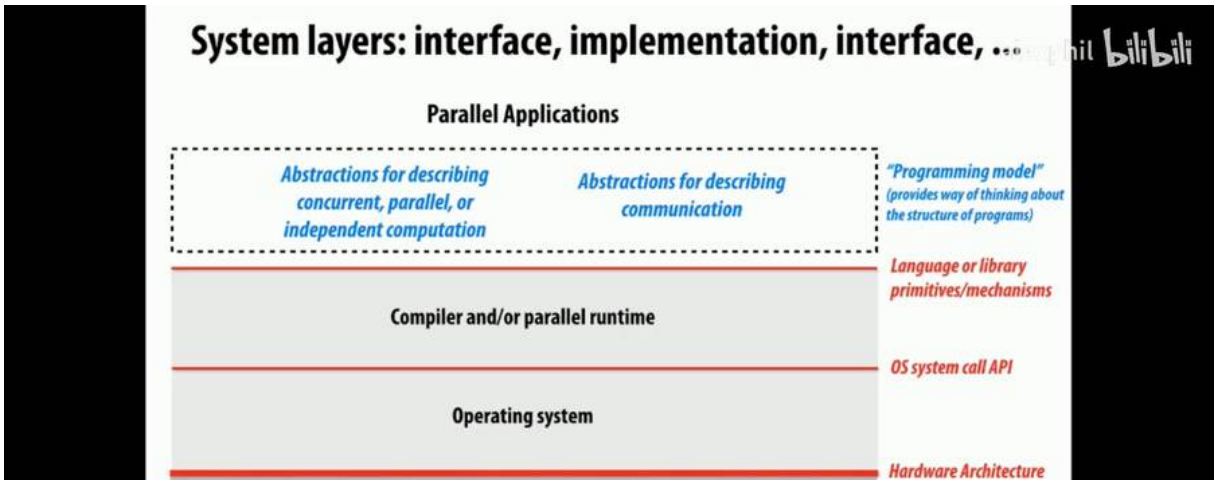

图 36: 系统分层：应用、抽象、编译运行时、操作系统与硬件[36]

pthreads：库与内核共同承担

创建线程调用 pthread_create，这是一个库函数。它通常需要内核支持——如果希望线程跑在另一个核心上，内核必须知道这个线程存在。而操作系统本来就在管理多个核心上的执行，也已经在与硬件交互。

因此 pthreads 的大部分工作发生在库层与内核层。

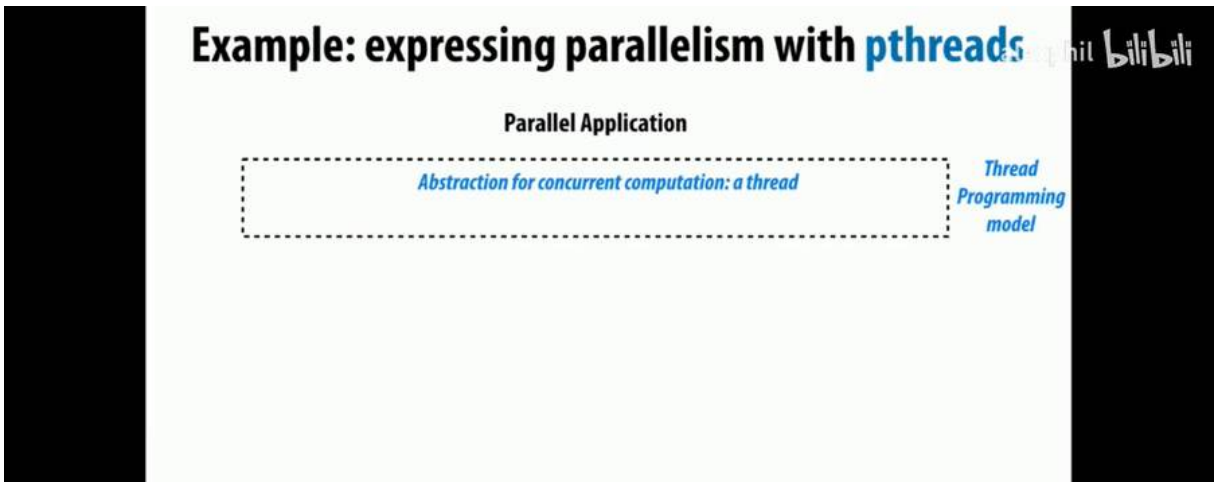

图 37: 用 pthreads 表达并行：抽象落在库与内核层[37]

对比之下，ISPC 通常完全不涉及操作系统：编译器只生成向量指令，硬件直接并行执行，不需要特殊库或内核参与。繁重的工作全部由编译器承担。

[36] 视频画面时间区间：00:28:00–00:28:15。若为裁剪图，时间区间仍对应原视频画面。

[37] 视频画面时间区间：00:28:30–00:28:45。若为裁剪图，时间区间仍对应原视频画面。

抽象	主要实现层	是否需要内核	并行单位
ISPC gang	编译器	否	向量通道
ISPC task	编译器 + 运行时	是（转成线程）	线程
pthreads	库 + 内核	是	线程
共享地址空间	硬件 + 缓存一致性协议	否（硬件内建）	线程
消息传递	网络硬件 + 库	否（只需网络）	进程

表 4: 四种抽象的实现层次对照

共享地址空间抽象

这是最常见的模型：所有并发执行体共享同一个地址空间，通过读写共享变量来通信，额外要做的是加锁与其他同步机制。

讲者指出，将来看到这类代码时会发现，多出来的代码大多是为了同步原语。他也用一个例子预告了危险：若两个线程都读出同一个值、各自加一、再写回，就可能丢掉一次自增。

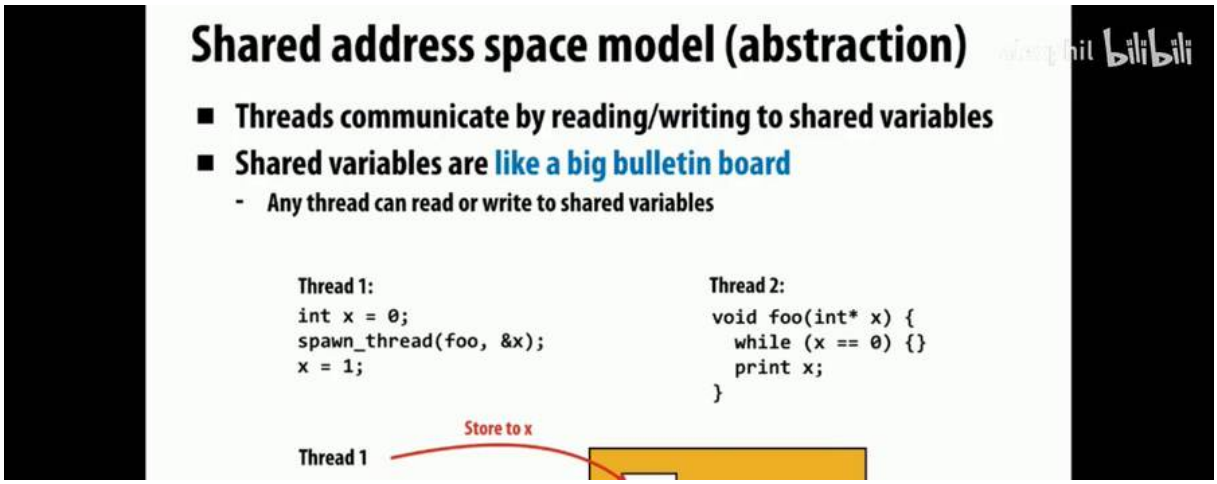

图 38: 共享地址空间模型：线程通过读写共享变量通信[38]

共享地址空间的硬件实现

程序员视角的简单，在硬件上要付出代价。最直接的做法是让处理器物理上共享同一份内存，通过总线或其他互连连接。

这种结构有个别名叫 dance hall 架构——像中学舞会，所有人都贴在墙边，没人站在中间；处理器在一边，内存在另一边。它也被称为对称多处理：每个处理器到内存的距离相等。讲者的评语是，这听起来不错，但实际上是” 对所有人一样糟”。

[38] 视频画面时间区间：00:32:30–00:32:45。若为裁剪图，时间区间仍对应原视频画面。

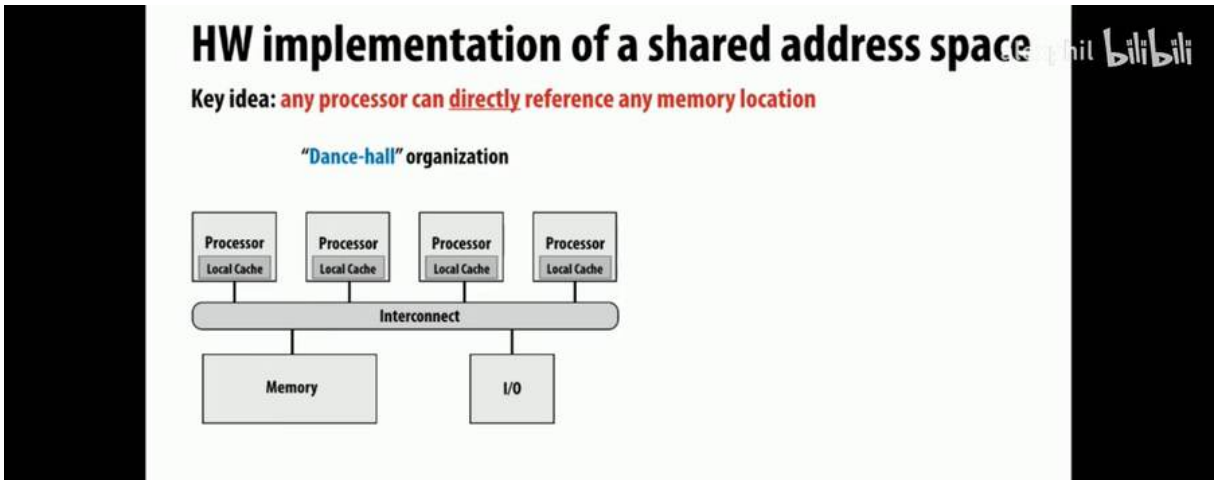

图 39: 共享地址空间的硬件实现：任一处理器都能直接引用任一内存地址[39]

问题在互连。总线很便宜，但同一时刻只能有一个设备在总线上，带宽无法扩展。要用更复杂的方式连接更多处理器与更多内存模块，成本迅速上升。

真实硬件的例子可以看出这个压力。现代处理器上若干核心共享同一个缓存，这是最贴近对称多处理的做法。而 Sun Niagara 2 为了支持大量容忍延迟的任务，放了八个处理器，代价是必须用更复杂的互连——在芯片版图上，那块交叉开关已接近一个处理器核心的面积。

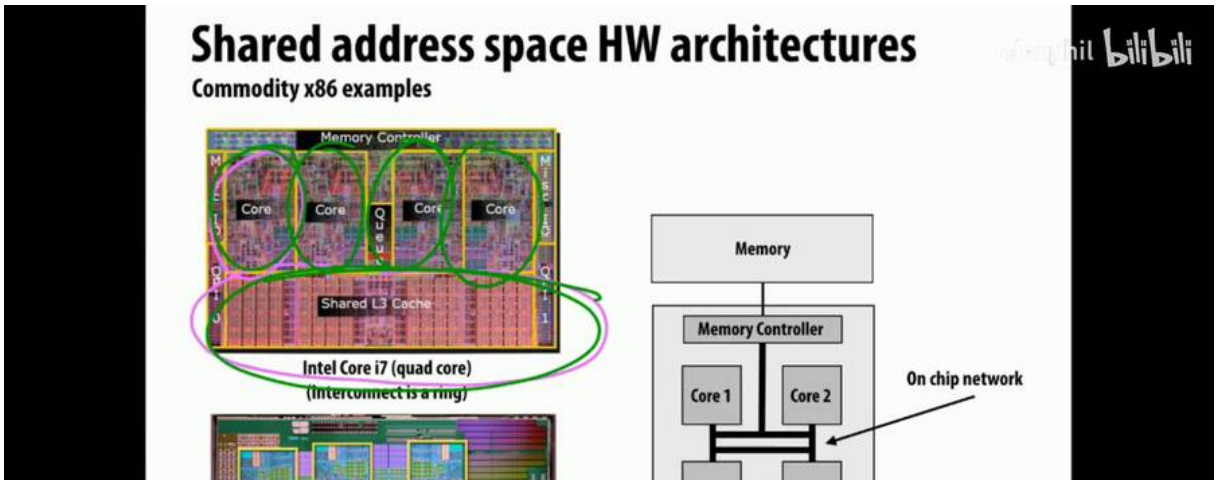

图 40: 共享地址空间的硬件实例：多核与共享缓存[40]

[39] 视频画面时间区间：00:36:00–00:36:15。若为裁剪图，时间区间仍对应原视频画面。

[40] 视频画面时间区间：00:38:30–00:38:45。若为裁剪图，时间区间仍对应原视频画面。

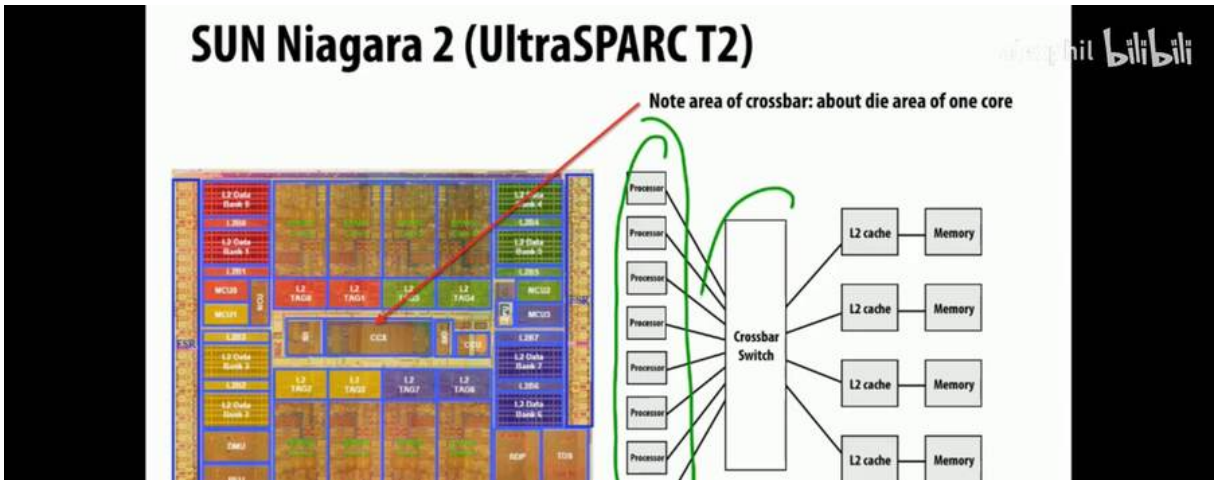

图 41: Sun Niagara 2：交叉开关的面积接近一个核心[41]

NUMA：放弃等距

要造出包含数百上千个处理器、且全部与内存等距的机器非常困难。绕开的办法是放弃等距：每个处理器配一块就近的物理内存，但仍可访问所有其他内存，只是更慢。这就是非统一内存访问 NUMA。

这个设计的收益有两个层面。私有数据（栈、代码）本来就不需要远行。更关键的是，让并行程序跑快的方式本就是划分数据，让各处理器主要处理自己的那片——讲者举的例子是把约 95% 的时间花在本地内存（视频 00:40:35）。这样既降低延迟，也减少大互连上的流量，避免拖慢他人。

代价是复杂度转移给了程序员：内存可能近而快，也可能远而慢。

NUMA 成本不会自己消失

在 NUMA 机器上忽略内存位置，程序仍然正确，只是当访问大量走远端时性能会明显下降。这类退化往往是渐进的，不会被正确性检查发现，只会在性能测量中暴露。因此在多插槽机器上做性能调优时，线程与数据的位置绑定是必须显式考虑的一项。

今天一台双插槽的笔记本或台式机就已经是 NUMA 机器。

[41] 视频画面时间区间：00:39:00–00:39:15。若为裁剪图，时间区间仍对应原视频画面。

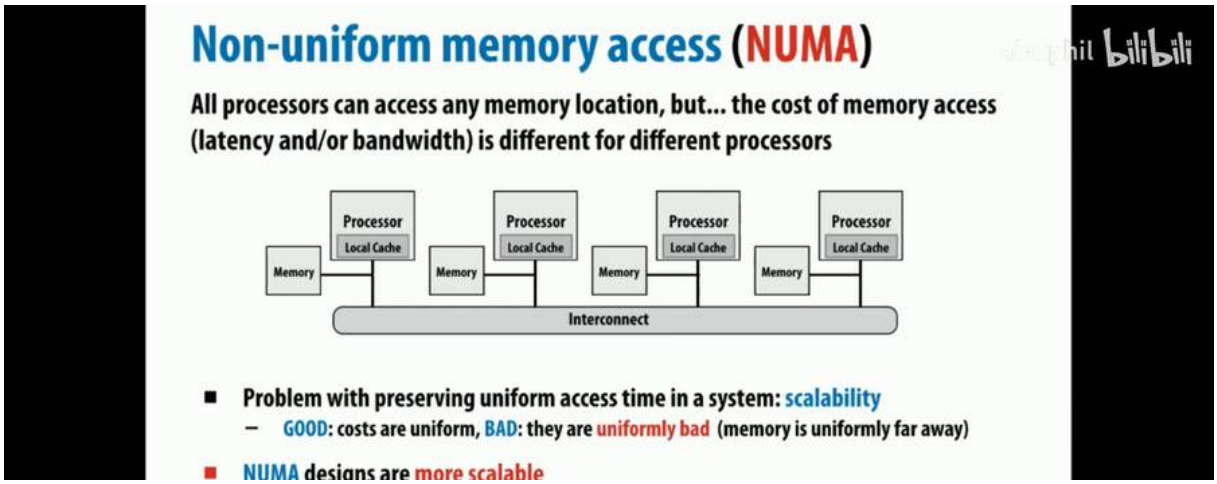

图 42: NUMA：所有处理器可访问任意内存，但远端访问更慢[42]

对称多处理为何难以做大

对称的前提是等距，而等距要求互连对所有节点一视同仁。总线的带宽不随节点数增长，交叉开关的面积则随节点数迅速膨胀。当规模上升到数百处理器时，等距的硬件成本超过收益，因此大规模机器几乎都转向 NUMA。这是被物理约束逼出的设计，不是偏好。

本章小结

同一份” 共享地址空间” 抽象，底层可以从共享缓存的核心群，扩展到交叉开关连接的对称多处理，再到 NUMA 的分层结构。程序员视角始终是” 大家都用同一个地址空间”，但硬件代价与访问延迟随规模变化。缓存一致性则是这套抽象真正的实现难点。

3.6 消息传递：彻底放弃共享内存

到此为止讨论的抽象都建立在共享地址空间上。消息传递走的是相反的路：每个执行体拥有私有地址空间。

消息传递模型的通信方式

消息传递模型里，每个执行体有自己的私有地址空间，通信靠显式的 send 与 receive。消息可以带标签，接收方可以只收特定标签的消息。

这个模型最大的优势是对硬件几乎没有要求：只要能连上网络、能在机器之间传消息，代码就能跑。因此扩展到超大规模机器很容易——只需要一条快的互连，不必处理缓存一致性这些麻烦。

讲者指出，第三、第四次作业将让同一道题分别用共享地址空间和消息传递实现，学生可以自行比较偏好。

[42] 视频画面时间区间：00:39:30–00:39:45。若为裁剪图，时间区间仍对应原视频画面。

消息传递以妥协换扩展性

它要求每次通信都显式写出 send 与 receive，配对关系必须由程序员维护，这是它被认为僵硬的原因。换来的是不需要缓存一致性硬件，只要互连够快就能扩到超大规模。选择哪种模型，本质是在编程便利与机器规模之间取舍。

他的观察是，绝大多数人更偏好共享地址空间，因为消息传递在通信方式上更僵硬。

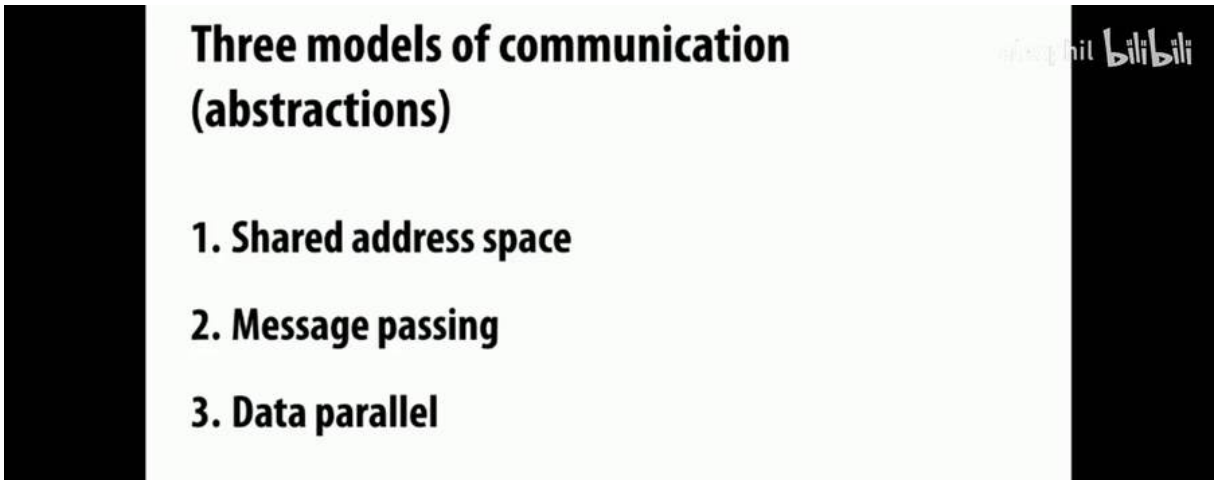

图 43: 三种通信模型：共享地址空间、消息传递、数据并行[43]

本章小结

消息传递用显式 send 与 receive 取代共享变量，代价是通信配对必须由程序员维护，收益是摆脱了缓存一致性硬件，只要互连够快就能扩到超大规模。它说明” 通信” 这件事可以用完全不同的机制表达。

另一种同样绕过共享内存的思路是把并行完全交给编译器——下一章的数据并行模型。

3.7 数据并行：把并行交给编译器

对全部数据应用同一函数

数据并行模型的思路是：同一份函数作用于大量数据。它在这门课上很重要，GPU 正是围绕这个模型构建的。

从 SIMD 到 SPMD 的语义放宽

早期向量机要求严格锁步：所有通道在同一时刻执行同一条指令，遇到分支只能靠掩码。SPMD 放松了这一要求，允许不同实例走不同代码路径，执行时机也不必完全一致。实现上仍常借助 SIMD，但抽象已不再等同 SIMD。

讲者做了历史对照。早期的向量超级计算机用向量指令并行，一条向量指令可以同时处理 100 多个元素，并行单位严格是” 一条指令”。今天的主流是 SPMD。今天的主流是 SPMD：对大量数据做同样的工作，但并不严格锁步，而是说” 这里是函数，对所有数据应用它”。

[43] 视频画面时间区间：00:30:00–00:30:15。若为裁剪图，时间区间仍对应原视频画面。

forEach 只保证控制并行

这是本讲最容易踩坑的一处。讲者明确指出：forEach 严格来说只提供控制并行——它只是告诉系统循环迭代可以并行，ISPC 并不关心循环体内对数据做了什么。

因此很容易写出错误代码。讲者举了两个例子。第一个是为每个输入生成两个输出副本（取绝对值后复制两份），这段代码能编译、能正确运行，确实是一种数据并行。

forEach 不做依赖分析

把 for 换成 forEach 只声明了”迭代之间可以并行”，编译器不会验证你的迭代是否真的独立。若循环体内存在跨迭代的数据依赖（例如根据当前元素的正负去写一个由邻居决定的输出位置），多个实例可能同时写同一位置。ISPC 因为严格按向量与循环推进，结果不会真的随机，但换成调度更自由的 task 或线程，程序就会变得不确定。

第二个例子就暴露了问题：代码检查元素是否为负，然后写入输出。当相邻元素一正一负时，两个并发实例可能写同一个输出位置。讲者说从程序员角度看，程序因此变得不确定。

stream：用纯函数避免竞争

有没有更讲原则的方式利用数据并行？一个答案是 stream 模型：数据以流的形式流入，经过一个纯函数（无副作用的 kernel），产生流出。

由于 kernel 没有副作用，各元素可以任意顺序处理，也就不会出现上面那种竞争。讲者用它重写了取绝对值的例子：只看当前元素、只写对应输出，没有不确定性。

stream 模型的两个要件

流是一批可独立处理的元素；kernel 是作用于单个元素的无副作用函数。两个要件合起来才保证可任意顺序执行。只要 kernel 引入了跨元素的状态（读邻居、累加共享变量、按条件写别处），这个保证就失效。

本章小结

数据并行以交出控制权换来了简洁：forEach 声明迭代可并行，stream 模型用无副作用 kernel 保证可任意顺序执行。但原语本身不检查依赖，安全性来自 stream 的纯函数约束，而不是来自 forEach。

至此四类抽象都已交代完毕，可以回到本讲最初的判断框架。

3.8 总结与延伸

这一讲给出了一个可以反复使用的判断框架：看任何并行抽象，都问它在硬件上落到了哪里。

讲者的历史观察值得记录：新的抽象通常起源于有人发现了某种高效的硬件实现方式，所以最初都与硬件紧密结合。久而久之人们发现，这个抽象其实没那么依赖硬件，可以移植到别的机器上。但无论怎么移植，理解抽象与实现的差别始终是必要的，因为他强调要学会从多个不同角度看待同一件事。

ISPC 给出的最尖锐教训是：一个看起来像”多线程”的抽象，底层可以是单线程加向量指令，programCount 就是向量宽度。后三个抽象则展示了实现层次的多样化：pthreads 落在库与内核，共享地址空间落在硬件与缓存一致性协议，消息传递只需要网络。同一份”通信”语义，实现在四个不同层次，代价与扩展性完全不同。

本讲义在讲者材料之上做的压缩，有四点需要与讲者原话区分：

- 抽象不决定性能，实现才决定。interleaved 与 blocked 表达同一个 gang 语义，在 ISPC 上却因向量加载与 gather 的差别而差距悬殊。选数据分布时必须知道目标实现。
- ”并行语法”不等于”并行”。把顺序函数原样搬进 ISPC 文件只会产生冗余实例，速度不变。真正的工作在数据划分。
- 归约是并行度的收口。它必须显式表达（部分和加 reduce），且最后一步退化为标量，因此频繁的小归约是隐性开销。
- 安全的数据并行需要纯函数约束。forEach 只声明控制并行，不检查依赖；stream 模型用无副作用 kernel 补上了这一环。

本讲的边界也很清楚。讲者多次表示缓存一致性、NUMA 细节、同步原语的实现都会在后续专门的课上展开，本讲只是提出这些问题的存在。因此这里给出的是抽象的坐标系，不是任何一种模型的使用手册。

一个可以直接从本讲数据提出的下一步问题：既然 interleaved 之所以胜出是因为向量加载要求连续数据，那么当程序本身需要跨步访问（例如处理矩阵的某一列）时，是否应该改变数据布局而非访问顺序？这个问题把本讲的结论从”选分布”推进到”改数据结构”，也正是后续局部性与通信那一讲要处理的。

Lecture 04 Parallel Programming Basics

视频时长：1 小时 19 分 37 秒　来源：https://www.bilibili.com/video/BV1vu4y1R7rS/

4.1 从两个例子开始：规则结构与不规则结构

在讨论方法之前，讲者先给出两个参照对象。它们会贯穿全课，也是后面讲性能优化的对象。

海洋模拟：规则的稠密网格

假设你是海洋学家，要模拟海洋的物理过程。海洋是三维的，但因为深度远小于水平范围，它像一个非常薄的立体。又因为压力与温度随深度变化，实际表示方式是一叠位于不同深度的二维平面。

每一层就是一个二维数组。空间与时间本来都是连续的，软件里必须离散化：把二维空间划成网格点，数组元素代表该坐标处的状态；再把时间切成步，逐个时间步推进。

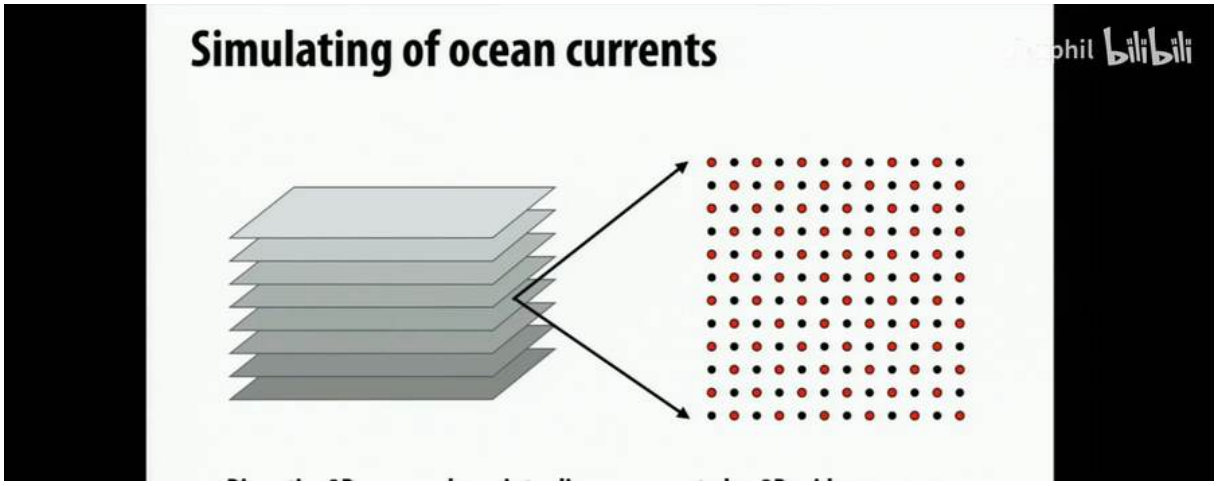

图 44: 海洋流模拟：把三维场按深度切成二维层逐层计算[44]

每一步要做的是求解一组偏微分方程，更新所有网格元素。讲者指出这正是程序花掉大部分时间的地方，也是后面并行化的重点。这个例子的显著特征是结构规整：一个稠密的二维数组，按时间步反复更新。

[44]视频画面时间区间：00:01:30–00:01:45。若为裁剪图，时间区间仍对应原视频画面。

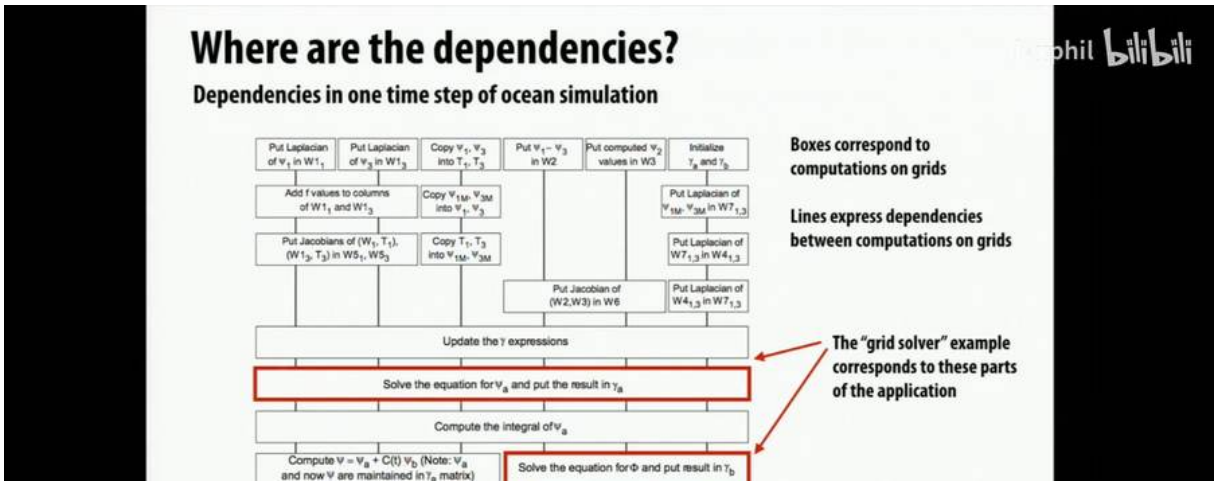

图 45: 单个时间步内的数据依赖[45]

n-body 模拟：稀疏且不规则

第二个例子模拟宇宙中的恒星。同样按时间步推进，但计算的是相互之间的引力：每个星体对其他星体施力，从而移动。

与海洋的例子不同，星体的空间分布极不均匀——有的地方挤成一团，有的地方很空。因此不能用稠密数组表示，否则大部分空间都是无意义的。这是稀疏数据结构，可以理解成一张图。

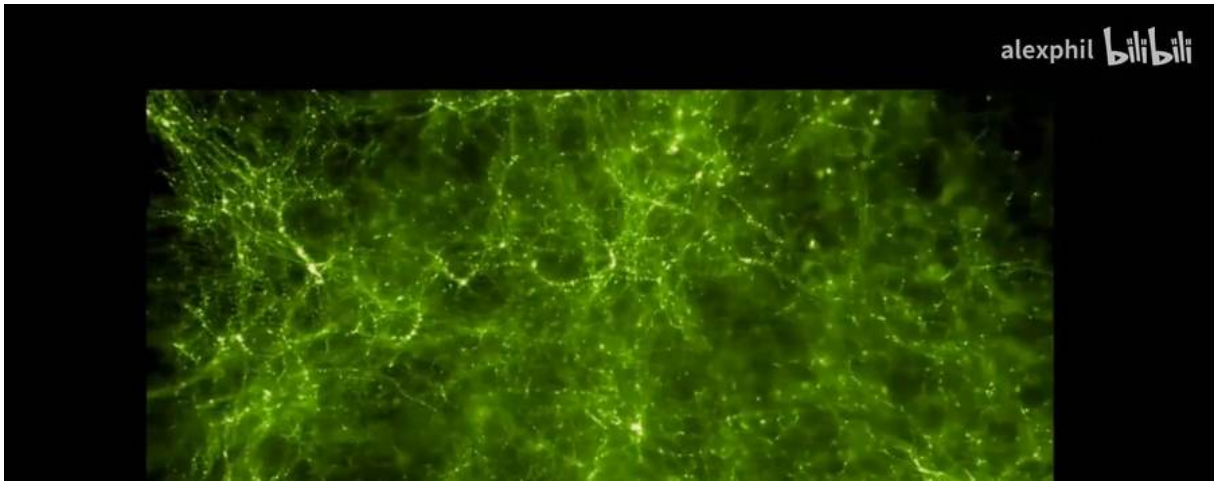

图 46: 星系演化模拟与 Barnes-Hut 算法[46]

朴素算法不可行：每个星体与每个其他星体都算一次，规模一大就完全不可扩展。

四叉树与近似

出路是近似。对某个待更新的星体，足够近的星体逐个计算；足够远的一群星体，则把它们合并成一个近似的质量，当作那里只有一颗大星。

[45] 视频画面时间区间：00:03:30–00:03:45。若为裁剪图，时间区间仍对应原视频画面。

[46] 视频画面时间区间：00:05:30–00:05:45。若为裁剪图，时间区间仍对应原视频画面。

近似算法把正确性换成了可控误差

把远处的星群合并为一个近似质量，意味着结果不再等于精确的万有引力求和。误差量级由半径参数控制：半径越大，聚合越多，算得越快，误差也越大。因此这类算法的参数选择本身就是精度与速度的权衡，而不是实现细节。

这样省下大量计算。分界由一个半径参数决定。

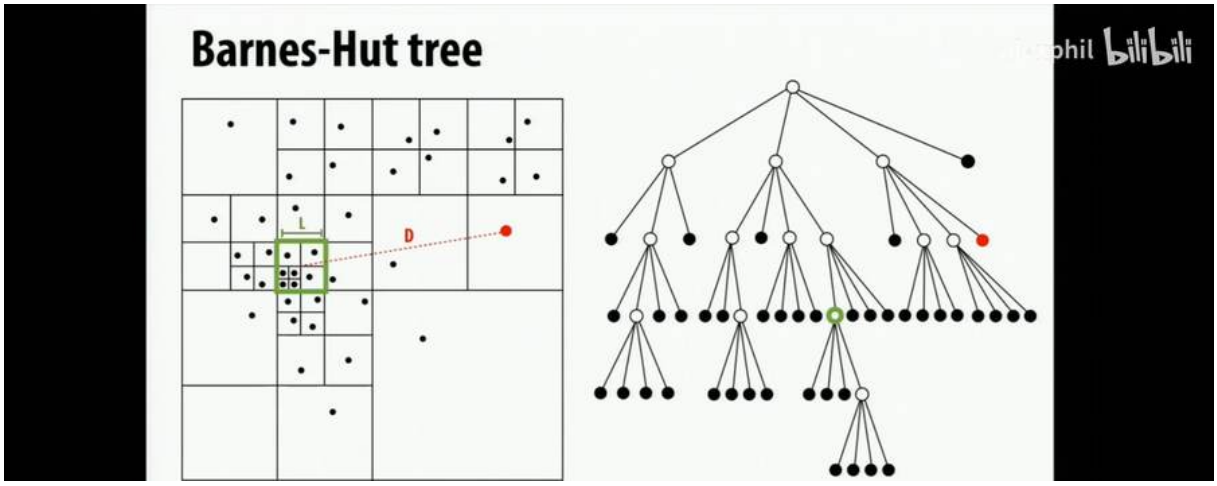

图 47: Barnes-Hut 四叉树：递归细分空间，邻近查询只需上溯几层[47]

要快速判断哪些星体近、哪些远，需要一个能表达空间位置的数据结构，这就是四叉树（三维时是八叉树）。讲者展示的是二维版本，因为便于画在幻灯片上，但真实程序是三维的。

做法是递归细分：星体稀疏的象限不再细分，密集的区域继续往下分，直到叶子节点。这种结构的便利在于，找附近的东西通常只需往树上走几层，看看兄弟或表亲节点。

因为星体在移动，可能移出原象限，所以每个时间步都要重建这棵树，然后遍历所有节点计算合力，再更新属性。

本章小结

两个例子给出了并行的两种地形：规则稠密网格与稀疏不规则图。

规则与不规则结构对应两种并行策略

规则稠密结构可以放心按几何形状切分，代价仅为边界通信。不规则结构必须先建立索引结构（树或图），否则切分出去的任务之间负载相差几个数量级。判断策略前先判断数据是规则还是不规则，这一步在分解之前就该完成。

前者容易平均切分，后者需要树结构才能找到有意义的工作划分。后面的方法讨论会交替用这两个例子验证，因为它们暴露的问题并不相同——稠密网格考负载均衡与通信边界，稀疏图考结构重建与不规则负载。

4.2 并行化的四个步骤

有了例子，讲者给出通用流程。这四个步骤是分析并行程序的基本框架。

[47] 视频画面时间区间：00:08:30–00:08:45。若为裁剪图，时间区间仍对应原视频画面。

总览

把串行程序变成并行程序，需要依次回答四个问题：哪些工作可以并行、这些工作如何分给各处理器、各处理器如何正确协作、这些逻辑线程最终落到哪些物理单元。每个问题对应一步：分解、分配、编排、映射。

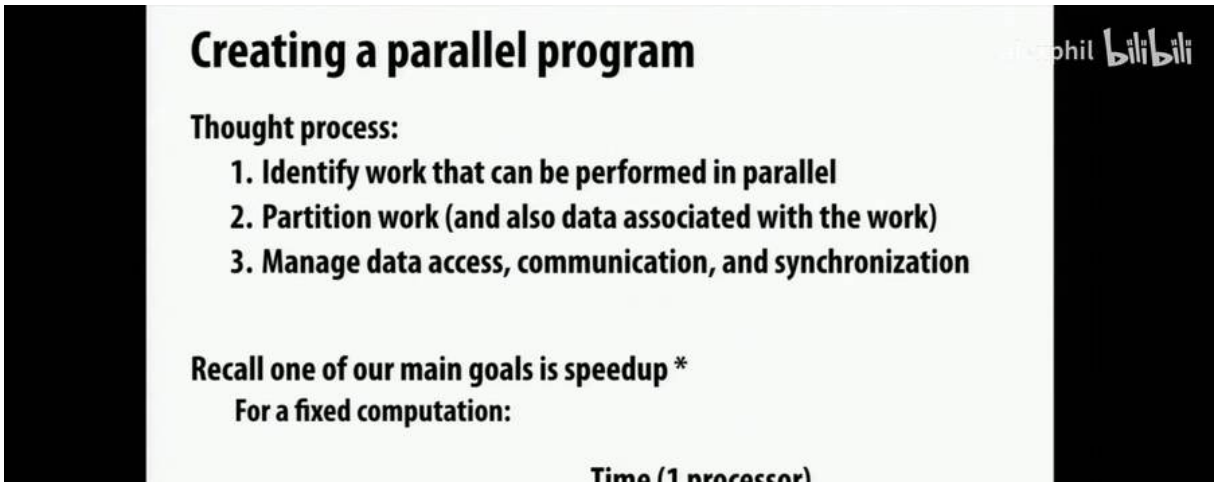

图 48: 把串行程序变成并行程序的思考步骤[48]

讲者用一张图把这层关系画出来：最上面那一整块是需要完成的计算，在单处理器上它是一个整体。并行化就是把它切成小块，而且块数至少要等于处理器数——少于处理器数，必然有处理器无事可做。

任务数与处理器数的关系可以直接写出来：

$$n_{\text{task}} \;\geq\; n_{\text{proc}}, \qquad \text{理想取 } n_{\text{task}} \gg n_{\text{proc}}$$

- n_{task}：分解出的任务（工作块）数量；
- n_{proc}：可用的处理器数量；
- 前者小于后者时，必然有处理器分不到任务而空闲。

这些工作块通常称为任务（task）。第一步就是找出这些自然的工作块。

分解

分解是纯粹的心智活动：想清楚这项工作有哪些自然的切分方式。

在海洋模拟里，二维数组的每个点都需要计算，因此每个元素是一个潜在任务。任务是独立才构成并行来源——如果一个任务必须等另一个算完才能开始，它们之间不是并行关系，而是串行的。

[48] 视频画面时间区间：00:11:00–00:11:15。若为裁剪图，时间区间仍对应原视频画面。

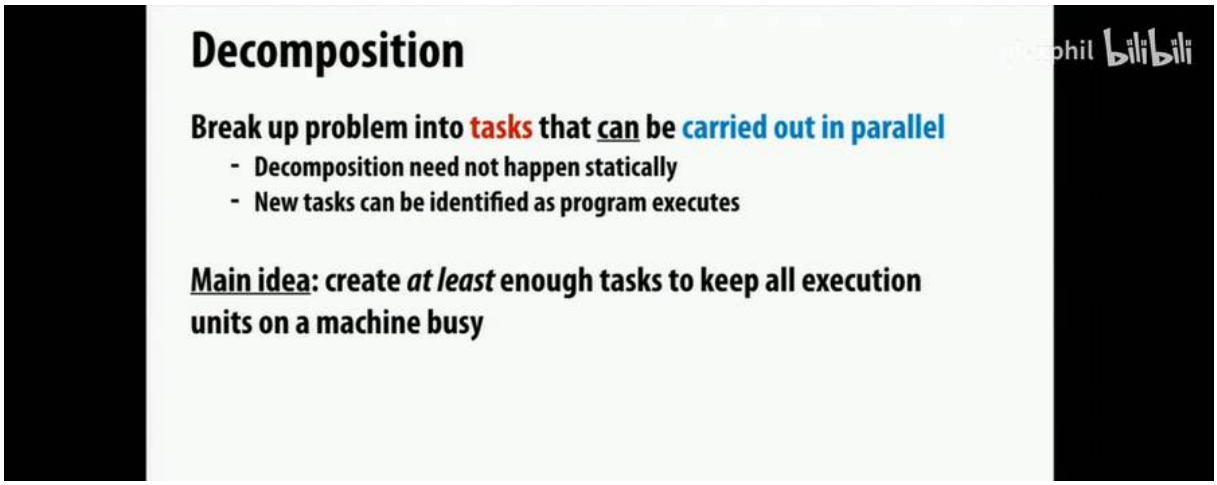

图 49: 分解：把问题拆成可并行执行的任务[49]

这一步几乎总是由程序员完成。理想情况下希望编译器或工具自动推断，但

不要指望编译器替你分解

自动并行化研究已进行多年，成果集中在高度规整的循环嵌套上。一旦出现指针别名、间接寻址或不规则数据结构，编译器无法证明迭代独立，只能放弃。实际项目中分解的合理粒度仍要由人根据算法语义决定。

讲者指出这是很难的问题——他自己也做过这方面的研究，成果有限，因为依赖关系难以整体把握。

任务粒度的选择

任务数量上的规则很清楚：至少要有线程数那么多，而且通常希望更多。原因是你很难一次就切得恰到好处，任务大小未必相等，多留一些任务才有调配余地。

但也不能走极端。任务切得太小，管理它们本身就会产生开销。因此一个合理的回答是：在星系模拟里，每颗星体是一个任务；在海洋模拟里，可能是单个元素，也可能是整行、整列——讲者说这取决于后续考虑。

粒度选择	优点	代价
任务过少	管理开销低	无法占满处理器，负载不均
任务 = 处理器数	静态即可分配	任何不均都直接变成等待
任务远多于处理器	便于动态均衡	每个任务有调度开销
任务极小	均衡极好	调度开销可能超过计算本身

表 5: 任务粒度与两项成本的关系

[49] 视频画面时间区间：00:16:30–00:16:45。若为裁剪图，时间区间仍对应原视频画面。

本章小结

分解回答” 有哪些独立工作”，规则结构可按几何切分，不规则结构需先建索引。任务的粒度要满足数量远多于处理器、又不能小到管理开销反客为主。这一步只能由人完成。

分解只确定工作的形状，还没决定谁来做。把任务实际落到处理器上，是下一步的工作，也是本课第一讲演示中负载不均教训的直接延续。

分配的目标

分配的目标是让每个处理器拿到大致相同的工作量。讲者回到第一堂课的演示：四个志愿者加十六个数，他故意让一个人分到八个数字，结果另外三个人早早完成，全体等那最后一个人。

图 50: 分配：把任务分给线程[50]

第二次他们重新分配，每人四个，情况好得多。这个对比就是本章第 4 节要展开的静态与动态两种分配方式。

编排

分解与分配只是在切分工作量。真正让线程协作起来的那一步叫编排，它的细节高度依赖编程抽象。

[50] 视频画面时间区间：00:26:00–00:26:15。若为裁剪图，时间区间仍对应原视频画面。

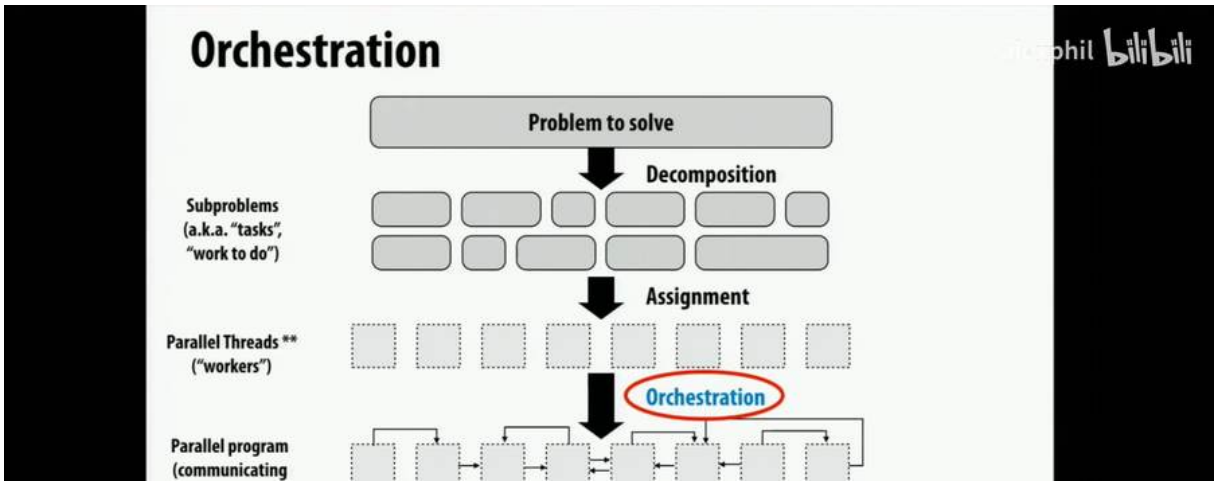

图 51: 编排：在分解出的子问题之间组织协作[51]

这是四个步骤里最有分化的一步：共享地址空间、消息传递、数据并行三种抽象下的编排代码看起来完全不同。因此本讲后面会用同一个网格求解器在两种模型下各写一遍，直接对比。

映射

最后一步是把逻辑线程映射到物理硬件单元。多数情况下程序员让系统自行决定，但有些场景需要更小心。

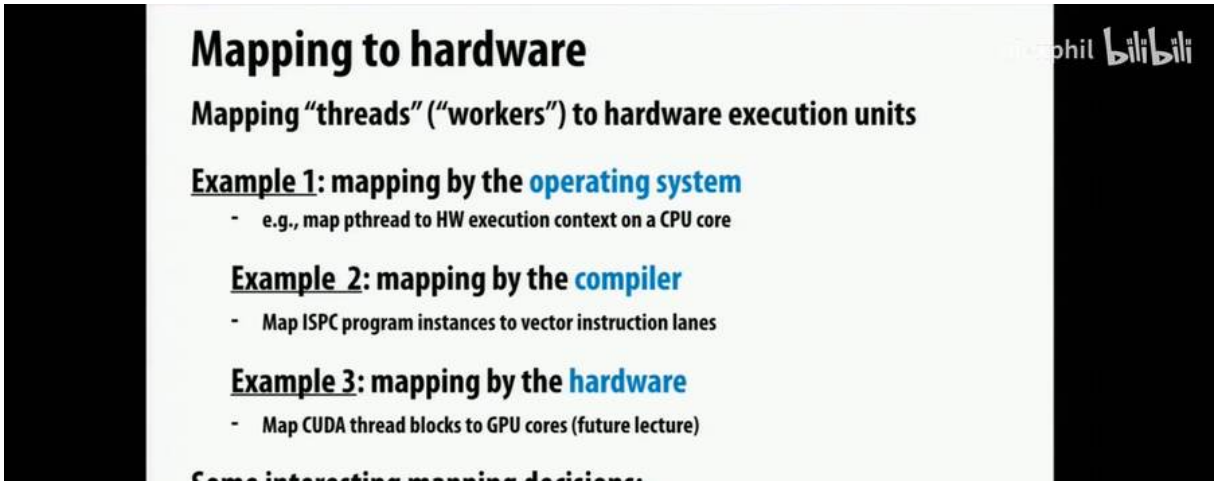

图 52: 映射：把线程（工作单元）对应到硬件的执行单元[52]

讲者给出的例子是：如果互连只允许与近邻通信，那就需要确保相邻的逻辑线程被映射到物理上相邻的单元，否则每次通信都要绕远。

本章小结

分配把任务落到处理器上，目标是让每个处理器拿到相近的工作量。四步之中，分解与分配决定理论上限，编排与映射决定实际能拿到多少。

[51] 视频画面时间区间：00:35:30–00:35:45。若为裁剪图，时间区间仍对应原视频画面。
[52] 视频画面时间区间：00:38:30–00:38:45。若为裁剪图，时间区间仍对应原视频画面。

四步中最容易出错也最容易被低估的是分配，因为它直接对应第一课演示里那个负载不均的教训。下一章先量化一件事：即使分配完美，还有另一个因素给加速设上限。

4.3 Amdahl 定律：串行部分设下的天花板

上一讲已经用演示说明加速会打折。这一节给出定量的上界，并用一个具体例子算清楚。

一个常见错误

讲者先指出一个在研究并行时常见的失误：看到某段代码容易并行，就格外兴奋地去优化它，而忽略那些难以处理的段落，还以为这是明智之举。

为了说明这并不是明智之举，他请学生回顾 Amdahl 定律。

图像处理的例子

设想一个图像处理算法分两步：把每个像素的亮度加倍，然后计算处理后所有像素的平均值。

第一步没有依赖，每个像素独立处理，容易并行。第二步是归约，有依赖关系，性质类似第一课全场加总课程数的演示。

顺序执行时，两步各要访问 N^2 个元素，因此总耗时正比于 $2N^2$。

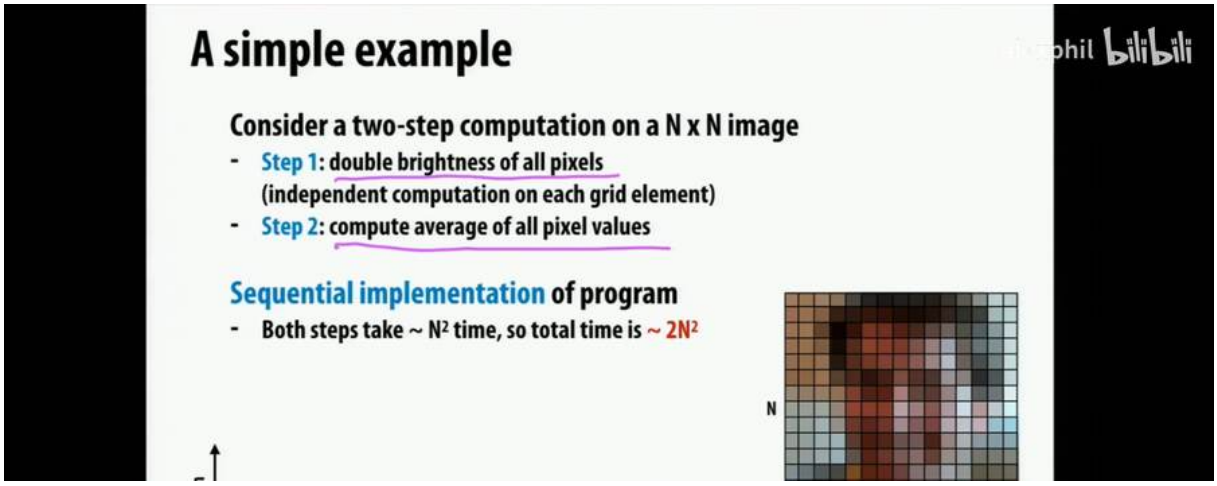

图 53: 两步计算的简单例子：先加倍亮度，再求平均[53]

只并行第一步的后果

在 P 个处理器上，第一步可以并行达到 P 倍加速：

$$T_P = \frac{N^2}{P} + N^2, \qquad T_1 = 2N^2$$

- T_1：单处理器总耗时，正比于 $2N^2$；
- T_P：P 个处理器上的总耗时；
- N^2/P：第一步在 P 个处理器上的耗时；

[53] 视频画面时间区间：00:19:30–00:19:45。若为裁剪图，时间区间仍对应原视频画面。

- N^2：第二步未并行，耗时不变。

讲者用一张时间轴图说明这个结果：横轴是时间，纵轴是正在被利用的并行度。顺序版本全程是 1；并行版本里紫色那一大段抬高到 P，但绿色那一段仍然停在 1。

于是当 P 趋于无穷时：

$$S_\infty = \lim_{P\to\infty} \frac{2N^2}{N^2/P + N^2} = 2$$

- S_∞：处理器数量趋于无穷时的加速比；
- 分子 $2N^2$：顺序基准；
- 分母中仅剩的 N^2：未被加速的第二步，它成了整个加速比的瓶颈。

投入无限多处理器，最多只能快两倍。这个数字很有说服力：一半的计算不可并行，就吞掉了一半的潜在收益。

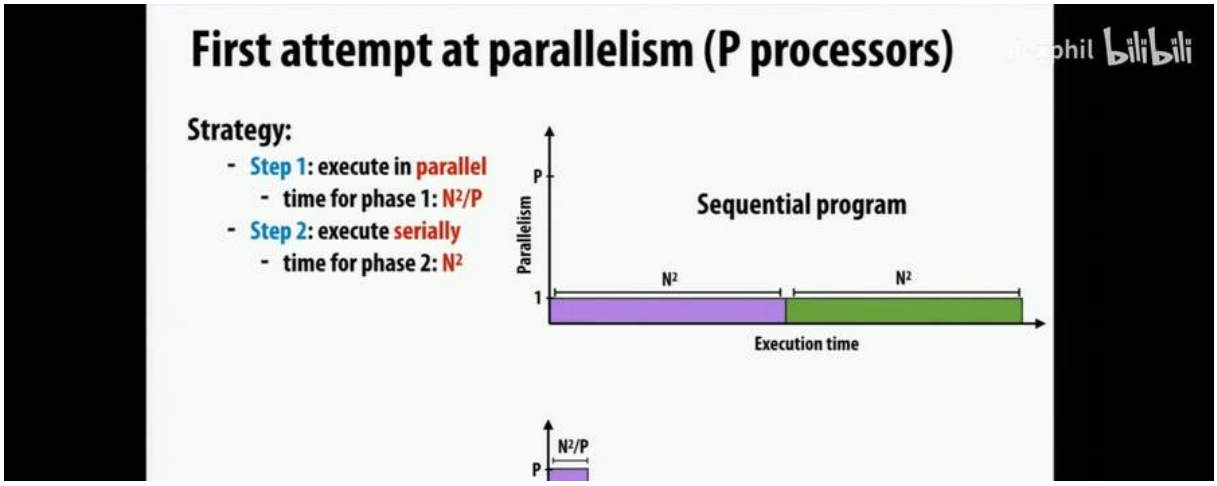

图 54: 只并行第一步的第一次尝试与其时间轴[54]

把第二步也并行：部分和

第二步虽然不是天然独立，但有一个通用技巧：让每个处理器先算出自己那部分数据的部分和。这一步彼此独立，可以并行。

然后才把 P 个部分和合并成一个总和。合并只能串行进行。

[54] 视频画面时间区间：00:20:30–00:20:45。若为裁剪图，时间区间仍对应原视频画面。

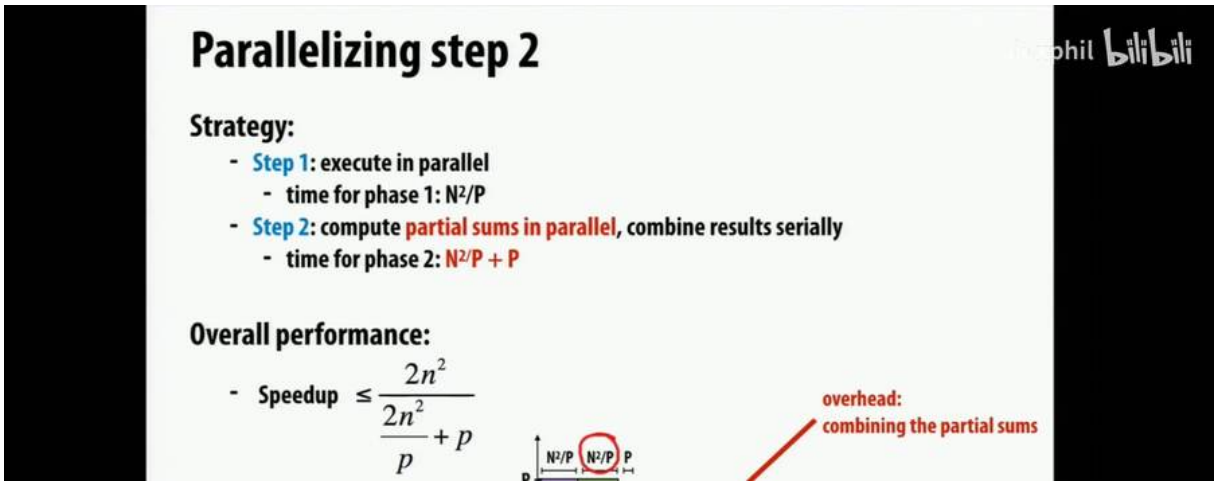

图 55: 并行化第二步：各处理器算部分和，再串行合并[55]

关键观察是新的串行部分宽度只有 P 个部分和，而不是 N^2 个元素。合并这 P 个部分和只需逐个相加，串行代价从 N 的平方规模降到 2 个量级的跨度。因为 N 通常远大于 P，这段串行开销可以被摊薄到几乎可以忽略。讲者的说法是会得到接近完美的加速。

定律的一般形式

讲者随后把结论推广。设 S 是总工作量中本质上串行的比例，则最大加速比为：

$$S_{\max} = \frac{1}{S + \dfrac{1 - S}{P}}$$

- S：总工作中无法并行的比例，$0 \le S \le 1$；
- P：处理器数量；
- 当 $P \to \infty$ 时，上界趋近 $1/S$。

代入具体数字就明白它有多严厉。若只有九成的代码可以并行，也就是串行占一成，那么无论投入多少处理器，加速都超不过 10 倍。即使可并行的部分提高到百分之九十九，上限也只有 100 倍。这两个数字来自把比例直接代入上界公式：串行比例 0.10 时上界为 10，串行比例 0.01 时上界为 100。投入再多处理器也突破不了这两个上限。

可并行比例	串行比例 S	加速上限 $1/S$
90%	0.10	10 倍
99%	0.01	100 倍
99.9%	0.001	1000 倍

表 6: Amdahl 定律的量化含义：串行比例决定加速天花板

[55] 视频画面时间区间：00:23:00–00:23:15。若为裁剪图，时间区间仍对应原视频画面。

先算上界，再决定投入

Amdahl 定律给出的是” 投入更多处理器也拿不到” 的严格上界。在规划并行化时先用它估算可达到的上限，比先写完代码再发现收益封顶更省力。若上界远低于投核计划，应该先减少串行部分（如把归约改成部分和），而不是继续增加处理器。

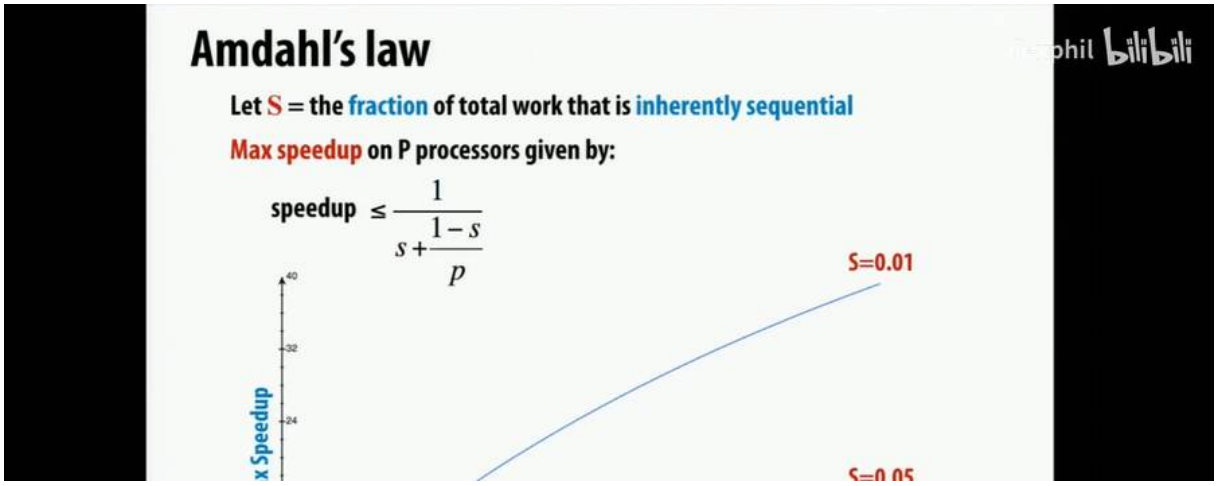

图 56: Amdahl 定律：设 S 为固有串行比例，最大加速为 $1/(S+(1-S)/P)$[56]

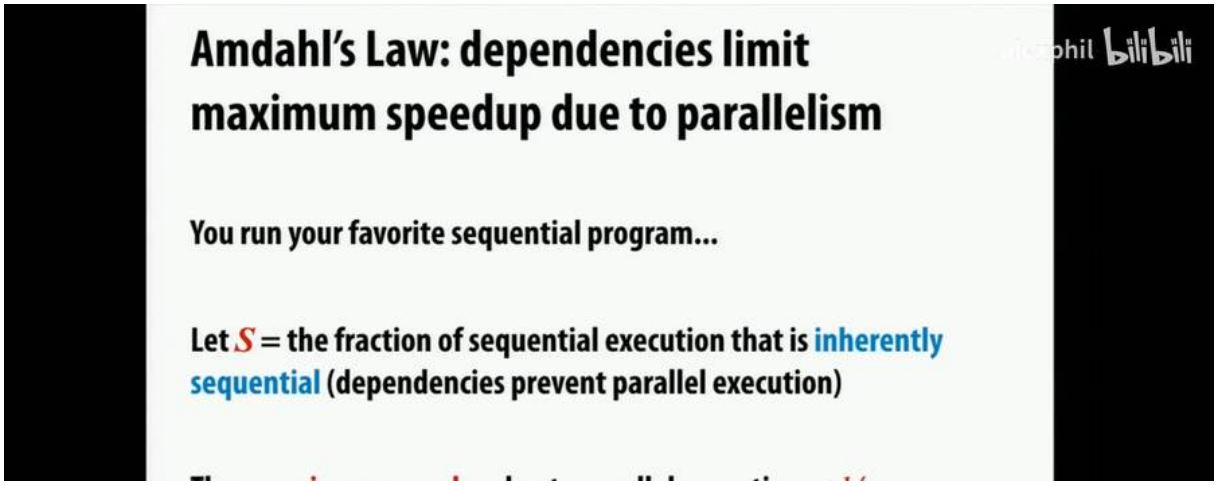

图 57: 依赖限制最大加速：Amdahl 定律的直观表述[57]

本章小结

Amdahl 定律把” 加速会打折” 量化为一个硬上界：串行比例 S 决定了加速最多为 $1/S$，与处理器数量无关。图像处理的例子给出两个具体数字——只并行第一步，极限是 2 倍；改成分部分和加串行合并后，串行宽度从 N^2 降到 P，几乎可以忽略。

这条定律回答的是” 值不值得并行某一部分”。而” 实际能拿到多少” 还取决于分配与编排，那才是下一章的内容。

[56] 视频画面时间区间：00:23:30–00:23:45。若为裁剪图，时间区间仍对应原视频画面。
[57] 视频画面时间区间：00:18:30–00:18:45。若为裁剪图，时间区间仍对应原视频画面。

4.4 分配的两种策略：静态与动态

Amdahl 定律假定分配是完美的。现实中分配方式本身就会决定性能，而分配策略可以分为静态与动态两大类，二者的开销来源恰好相反。

静态分配与它的问题

静态分配在编译期或启动时就把任务固定交给各线程，运行期不再调整。它简单、无调度开销，但对工作量分布的估计有很强依赖。

静态分配在编译期或启动时就把任务固定交给各线程，运行期不再调整。它简单、无调度开销，但对工作量分布的估计有很强依赖。

一个具体写法与它暴露的两个问题

讲者展示了一段用 pthreads 做静态分配的代码：把输入数组对半分，创建一个子线程处理一半，父线程处理另一半。

这里有两个问题。

把线程数写死等于放弃可移植性

按数组对半分并创建固定数量的线程，在核心数变化时要么浪费硬件，要么超出可用核数。可移植的写法应查询运行时的可用线程数，或把工作切成远多于线程数的任务再交给调度器分配。

第一，代码硬编码只创建两个线程；如果机器有四核，就有一半硬件闲置。第二，即使只有两核，这种对半分也不一定均等：如果某半边的计算量明显更大，先完成的那一半就要空等。

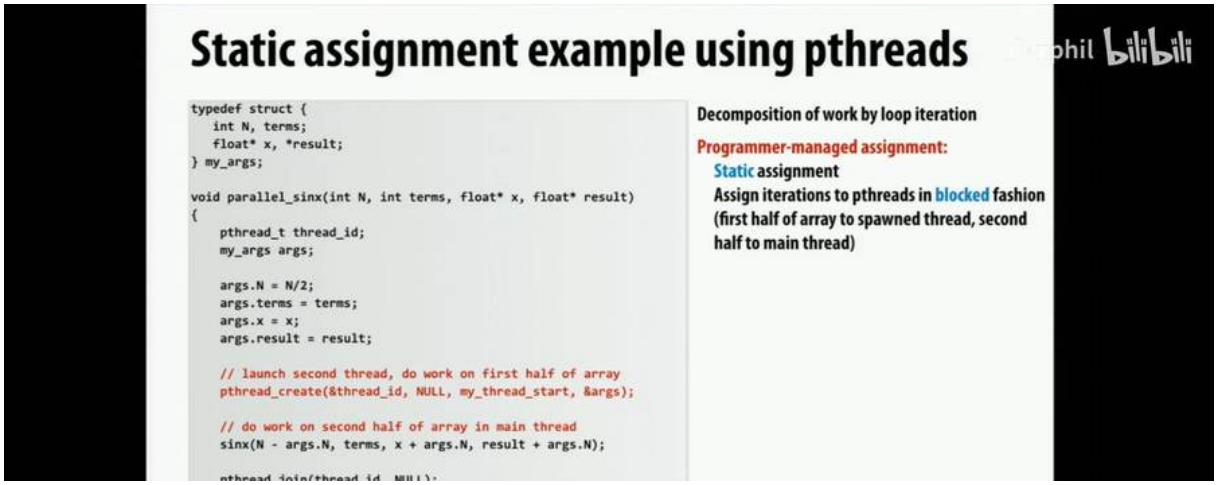

图 58: 用 pthreads 做静态分配：手工按循环迭代切分[58]

ISPC 中的静态分配写法更简洁，由 forEach 把迭代分给实例。但负载均衡的问题依然存在——只是被语言包装起来，并没有消失。

[58] 视频画面时间区间：00:30:00–00:30:15。若为裁剪图，时间区间仍对应原视频画面。

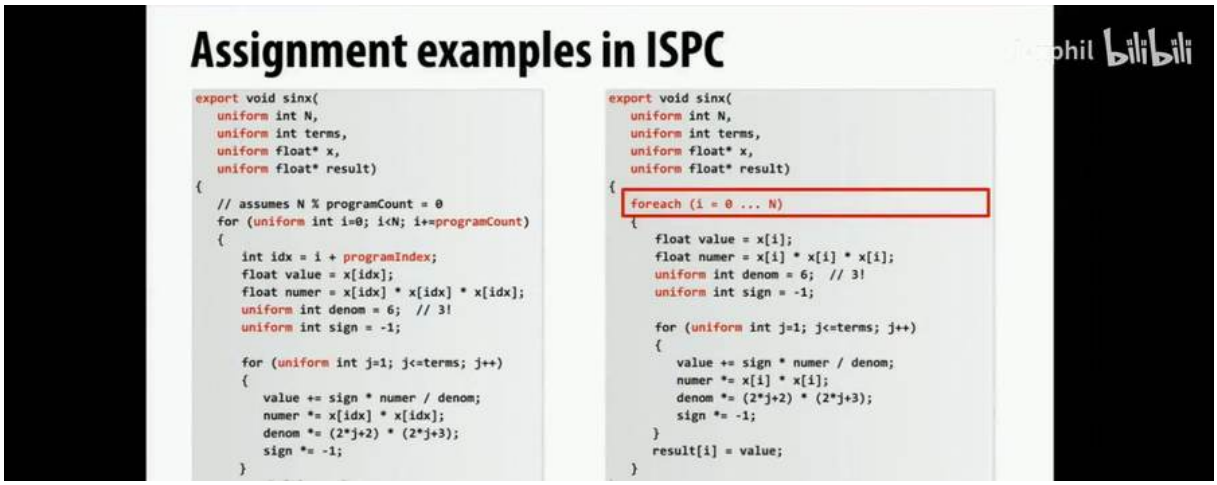

图 59: ISPC 中的分配示例：由 forEach 交出迭代分配[59]

本章小结

静态分配没有运行期开销，代价是要求事先估准工作量，且线程数写死时无法适应机器核心数。对均匀且稳定的工作，它是最省的选择；一旦工作量分布不可预测，这份节省会以空等的方式还回去。

要让程序自动适应变化的负载，就需要把分配推迟到运行期，这正是下一章的做法。

4.5 动态分配：用队列换均衡

工作队列的做法

解决思路是把工作放进一个队列，谁空了谁去取下一件。

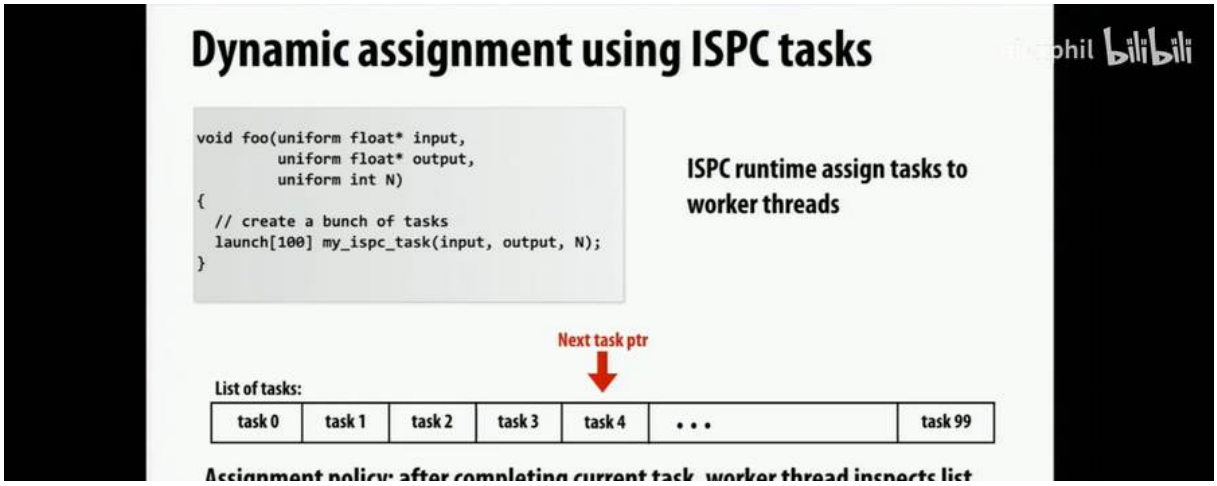

图 60: 用 ISPC task 做动态分配：运行时把任务派给空闲的执行体[60]

ISPC 的 task 就是这样一种机制：把一堆工作块交出去，系统内部维护一个指向下一个待派任务的指针。开始时每个核心拿一个，谁先完成谁回去取下一个。

[59] 视频画面时间区间：00:28:30–00:28:45。若为裁剪图，时间区间仍对应原视频画面。

[60] 视频画面时间区间：00:33:30–00:33:45。若为裁剪图，时间区间仍对应原视频画面。

讲者请学生评价这种方法的优劣，得到的正是它的本质权衡。

静态与动态两种分配的运行期总开销可以用同一形式比较：

$$T_{\text{static}} = \frac{W}{P} + \delta, \qquad T_{\text{dynamic}} = \frac{W}{P} + \frac{W}{s} \cdot c_{\text{queue}}$$

- W：总工作量；
- P：处理器（线程）数量；
- δ：静态分配的负载不均损失；
- s：单个任务的规模，W/s 即任务个数；
- c_{queue}：一次取任务（含加锁）的开销。

静态分配把成本压在 δ 上，动态分配把它转移到任务数乘以每次取任务的代价。这也是任务不能切得太细的原因：s 越小，第二项越大。

动态分配的收益与代价同源

收益：负载自动均衡，某个任务耗时很长也不影响整体，因为其余处理器会继续取新任务。代价：取任务本身要访问共享队列，通常需要加锁保护，因此多个处理器同时取任务时必须串行化。任务切得越细，这份开销占比越高。静态分配没有这项开销，但要求事先估准工作量。

本章小结

动态分配用共享队列换取自动均衡：某件任务耗时很长也不影响整体，因为其余处理器会继续取新任务。代价是每次取任务都要访问共享队列，通常还需加锁，多处理器同时取任务时必须串行化。任务切得越细，这份开销占比越高。选择依据是工作量的可预测性与任务粒度。

4.6 编排：用红黑格序消除数据竞争

前面三步处理的是”分多少、分给谁”。编排处理的是”怎么配合”，而这个问题的难度在这门课里最容易被低估。

网格求解器的串行版本

讲者引入一个具体的求解器：在 $(N+2) \times (N+2)$ 的网格上求解偏微分方程。外层是收敛循环，每次比较新旧值，把全部元素差值的绝对值求和，低于阈值就停止。

内层是对二维数组的遍历，每个元素取自身与上下左右四个邻居的平均，写回原位置。因为要计算新旧之差，必须先记住旧值。

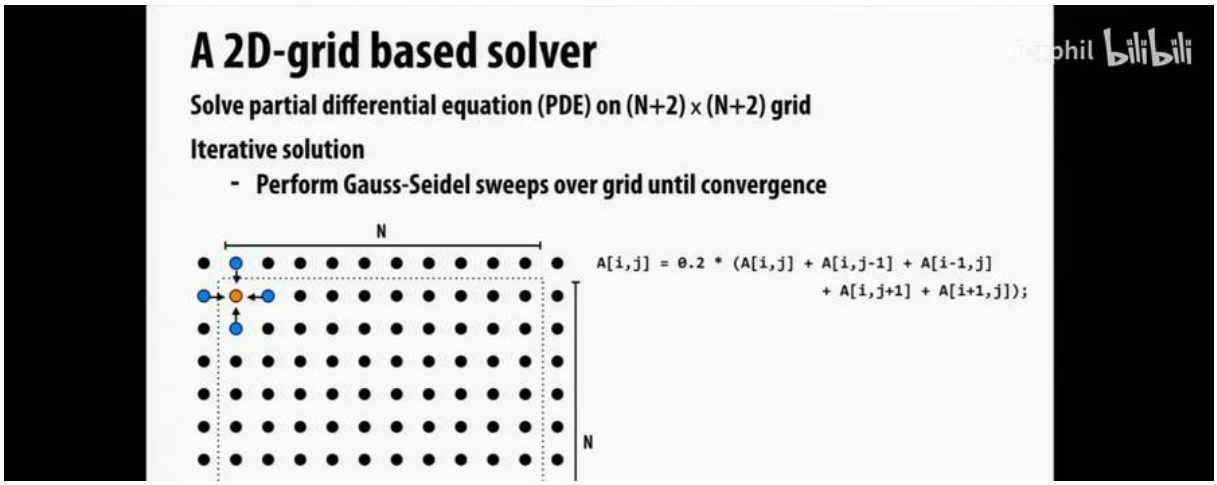

图 61: 二维网格求解器：顺序执行的伪代码[61]

依赖在哪里

如果直接把这个双重循环并行化，会发生什么？讲者让课堂思考依赖关系，答案是数据竞争：每次运行可能得到不同结果。

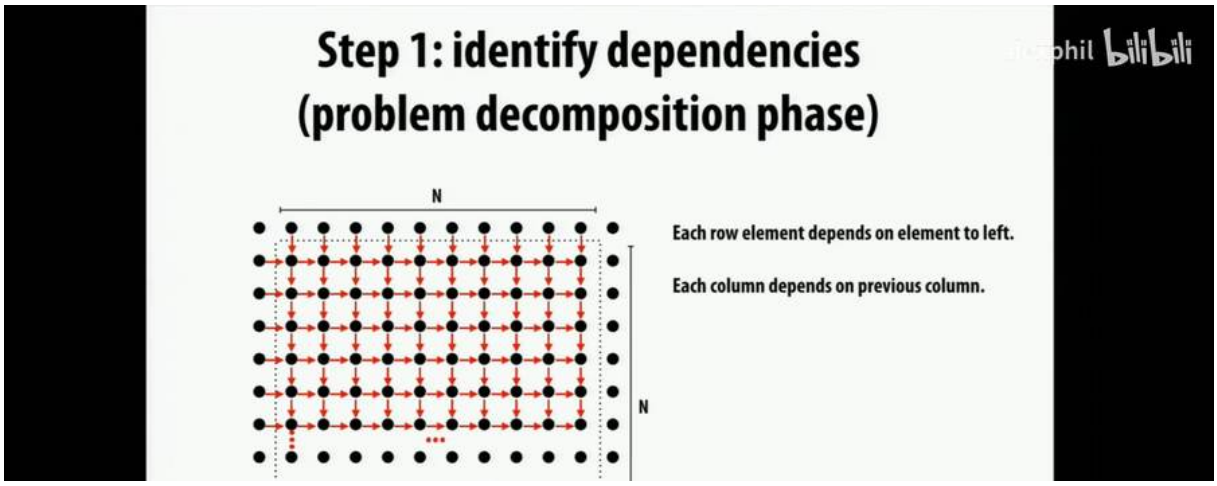

图 62: 第一步：识别依赖关系——每个元素依赖它的邻居[62]

原因很直接：写第 j 个元素时读的是它左右的邻居，而邻居可能正被另一个线程更新。于是读到的值取决于执行时序。讲者用”红色箭头”表示这些跨迭代的依赖，图上密密麻麻。

就地更新加并行等于非确定性

当写操作的目标与读操作的来源在同一数组内，并行执行的结果就取决于线程交错顺序。这类错误的危险之处在于它不总是报错——程序可能多数时候算对，偶尔给出略微不同的数值，极难复现。

[61] 视频画面时间区间：00:43:30–00:43:45。若为裁剪图，时间区间仍对应原视频画面。
[62] 视频画面时间区间：00:46:30–00:46:45。若为裁剪图，时间区间仍对应原视频画面。

双缓冲与它为什么被拒绝

最直接的修法是准备两份矩阵，一份读一份写，下一轮交换角色。这样读的永远是不变的副本，竞争消失。

但讲者说科学计算者拒绝这么做，理由是把内存填满数据是他们的重要目标，甚至不愿意砍掉一半。他的评语是可以争论这合不合理，但这就是他们的想法。

双缓冲为何在科学计算中不流行

双缓冲需要的额外内存正比于整个数据体。对受内存容量限制的大规模模拟，这直接减半可求解的问题规模。因此这个做法在图形或游戏里很常见，在科学计算里常被视为不可接受。替代方案必须在不复制全部数据的前提下工作。

红黑格序

替代方案借用了国际象棋棋盘的想法。

把网格元素按棋盘模式着色：相邻元素颜色不同。然后分两阶段更新：先只更新所有红格，再只更新所有黑格。

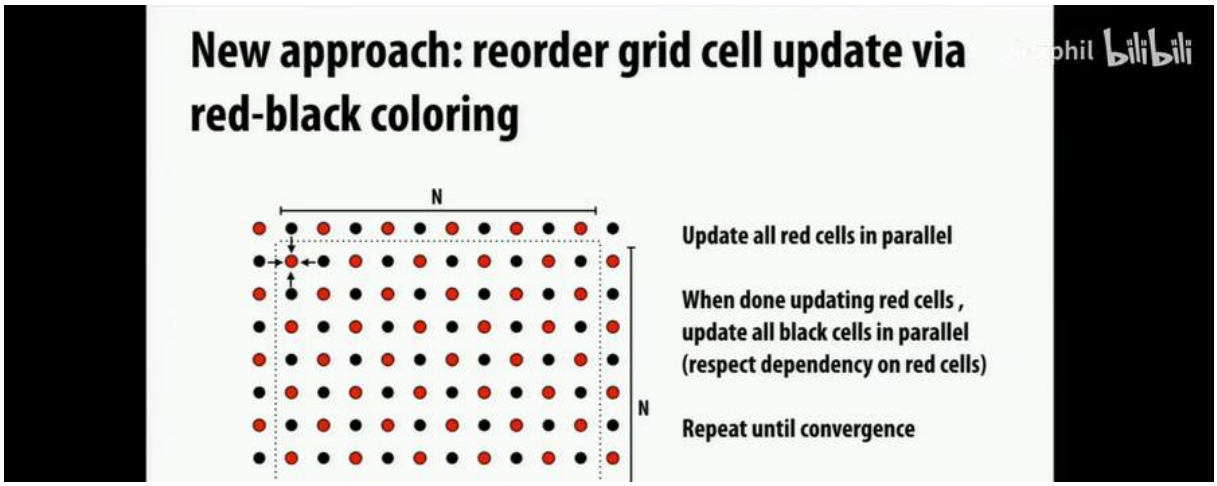

图 63: 红黑格序：分两阶段交替更新红格与黑格[63]

这个安排的效果等同于双缓冲：更新红格时，它读的是自身与黑格邻居，而黑格此刻不被修改，所以读到的值稳定。然后交换角色。

[63] 视频画面时间区间：00:51:30–00:51:45。若为裁剪图，时间区间仍对应原视频画面。

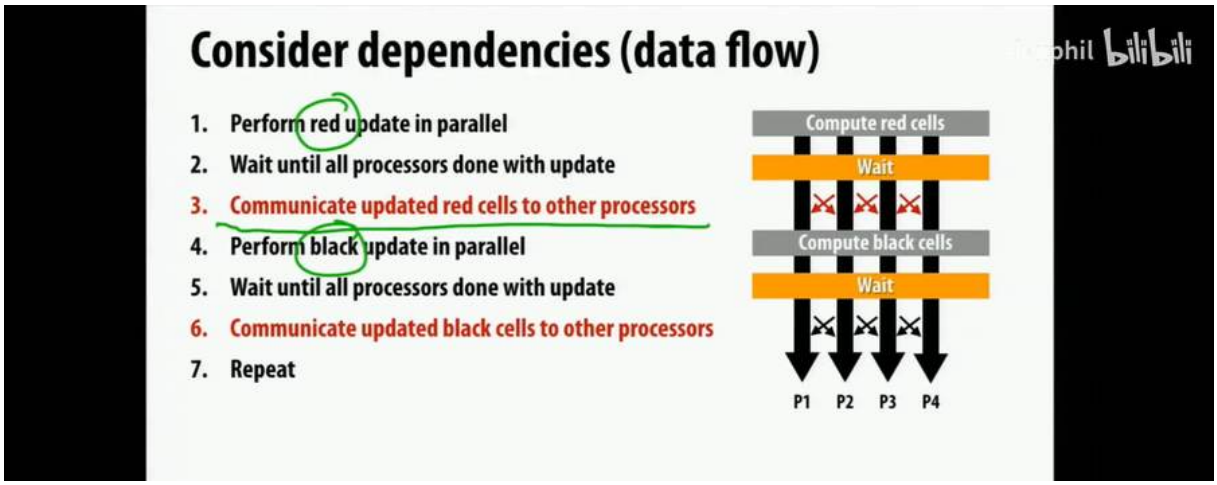

图 64：按红黑格序重排后的数据流：两阶段依赖清晰[64]

讲者坦承一个代价：结果与顺序版本并不完全相同。

确定性不等于与顺序版本等价

红黑格序每次运行结果一致，这是确定性；但这个结果与逐元素就地更新的顺序版本不同。对迭代收敛的物理模拟，这点差异通常可接受；对要求逐位复现某个参考实现的场景，则不能直接替换，必须先验证误差在容许范围内。

但对这类模拟通常足够接近，而且它是确定的——数据竞争消失，每次运行结果一致，也不需要复制全部元素。

阶段之间需要同步

学生提出一个关键追问：更新黑格时，红格已经改过了，黑格的邻居确实是被更新过的新值。

讲者的回答是：那正是我们想要的，但前提是必须确保所有对红格的更新都已正确写出并对所有人可见。这需要两阶段之间的一次同步，也就是屏障。

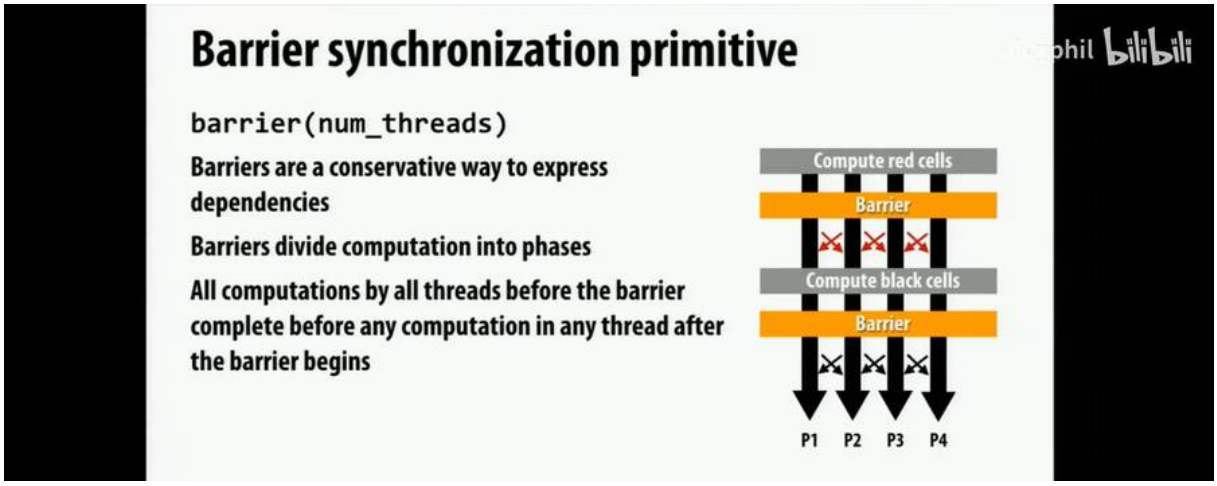

图 65：屏障同步原语：所有线程在此汇合[65]

[64] 视频画面时间区间：00:56:00–00:56:15。若为裁剪图，时间区间仍对应原视频画面。

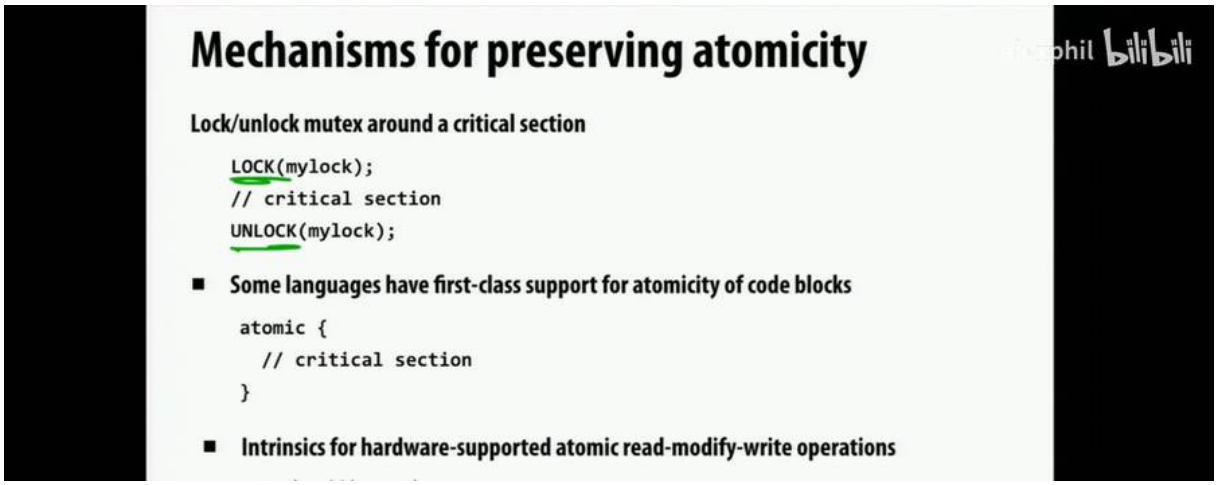

图 66: 保证原子性的机制：临界区加锁与解锁[66]

屏障的代价与更精确的依赖

讲者指出屏障是个重量级的工具：它会强迫全部线程彼此追上进度。

屏障表达的是最粗的依赖

一个屏障表示”所有线程都必须等我”，它保守地覆盖了全部依赖。当实际依赖只涉及局部数据时，用屏障会让本可继续工作的线程被迫停下。屏障越频繁，其开销占比越高；这也是后续要考察减少同步次数的主要原因。

用一¨ 刀切的¨ 方式解决依赖固然简单，但代价是失去了细粒度调度的可能。
如果知道只需等某个特定位置更新完，就可以更精确地等待，而不必让所有线程同步。

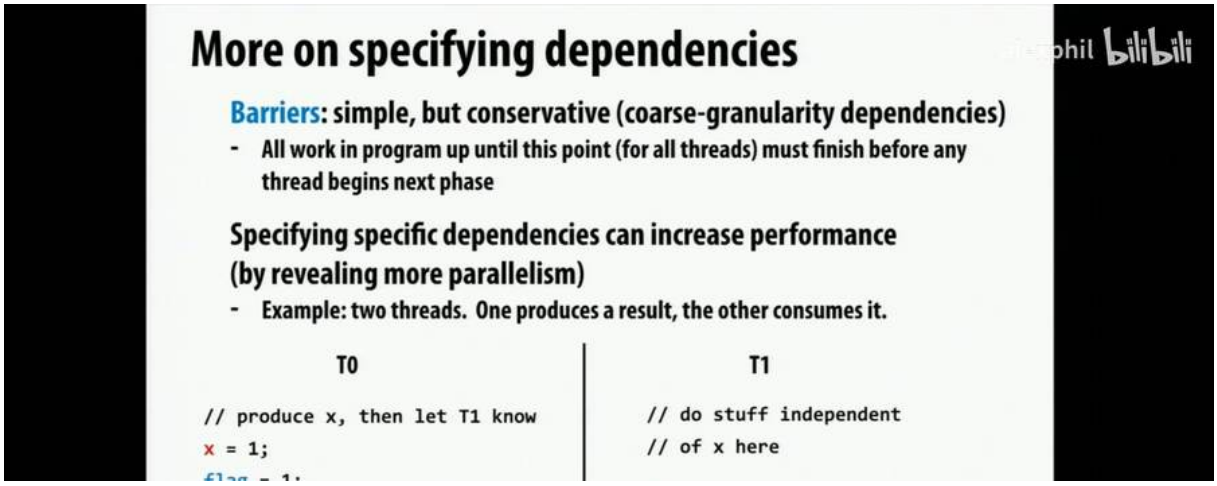

图 67: 更精确地表达依赖：屏障简单但保守[67]

[65] 视频画面时间区间：01:07:30–01:07:45。若为裁剪图，时间区间仍对应原视频画面。
[66] 视频画面时间区间：01:06:00–01:06:15。若为裁剪图，时间区间仍对应原视频画面。
[67] 视频画面时间区间：01:11:00–01:11:15。若为裁剪图，时间区间仍对应原视频画面。

不要用普通内存位置做同步

讲者在展示这种细粒度同步时明确警告：按那种写法不能直接用，中间必须有一个栅栏（fence），否则完全不能工作。用普通内存读写充当同步信号，在编译器与硬件重排面前是不可靠的。这个概念会在后面讲内存一致性模型的课上专门展开。

用多份副本减少屏障

讲者还展示了一个实用技巧。如果某个标量（例如差值累加量 diff）在多个迭代间被复用，可能出现还没用完就被下一轮覆盖的情况。

解法是准备三份副本，按下标取模交替使用。因为 diff 只是标量，三份副本几乎不占空间。这样每个线程始终指向正确的那份，于是本阶段只需要一个屏障。

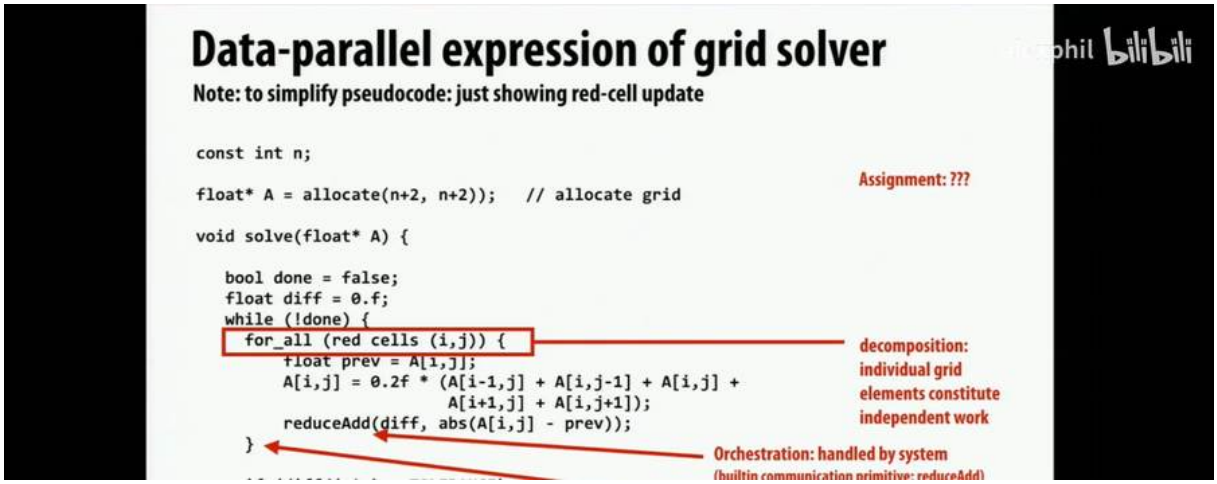

图 68: 网格求解器的数据并行表达[68]

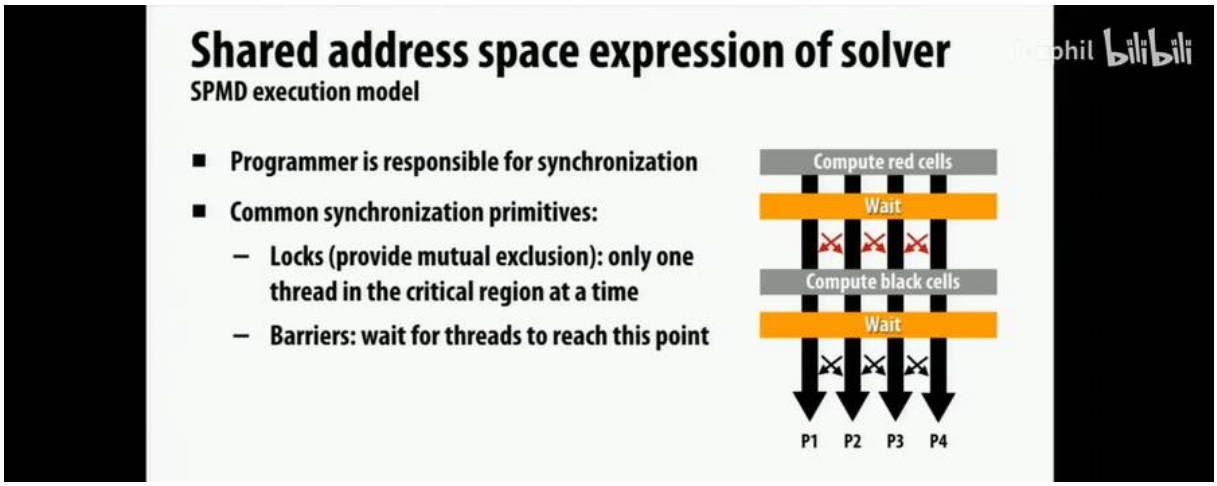

图 69: 网格求解器的共享地址空间表达：SPMD 执行模型[69]

[68] 视频画面时间区间：00:58:30–00:58:45。若为裁剪图，时间区间仍对应原视频画面。

[69] 视频画面时间区间：01:01:30–01:01:45。若为裁剪图，时间区间仍对应原视频画面。

本章小结

就地更新加并行会产生数据竞争，双缓冲能消除它但在科学计算中不可接受。红黑格序以改变更新顺序为代价，在不复制数据的前提下得到确定性，代价是两阶段之间需要屏障。

这条路径展示了编排的实质：正确性来自显式表达依赖，而不是依赖时序巧合。屏障是最简单的表达方式，也是最保守的，更精确的依赖表达需要配套的内存序保证。

到此为止的编排都建立在共享变量上。换一种抽象，同一份求解器的协作方式会完全不同，这正是下一章要对比的。

4.7 消息传递：同一个求解器的另一种编排

讲者最后用同一个网格求解器再写一遍，这次用消息传递，让读者直接看到编排的差异。

通信只能靠消息

在消息传递模型里，通信只通过给其他线程发消息、由对方接收消息来完成。没有共享变量可用。

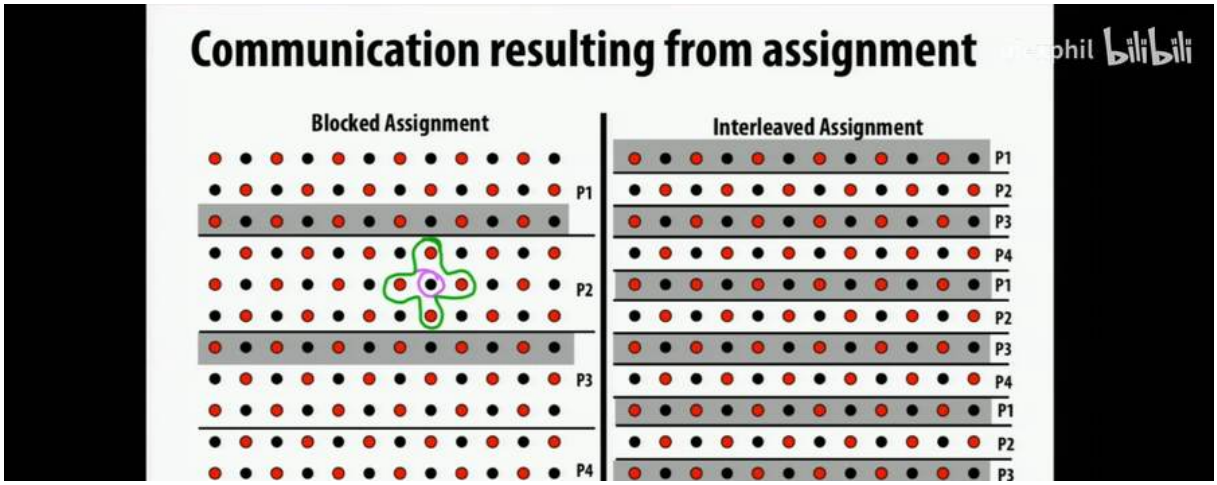

图 70: 不同分配方案产生的通信需求[70]

这意味着共享地址空间里”读一下邻居”这种自然的操作，在这里必须变成显式的发送与接收。分配方式也因此变得重要：阻塞式分配下，每个线程只需要与相邻线程交换边界数据；交叉式分配则会让一次更新需要到处取数。

死锁与它的解法

讲者展示了交换边界数据时的经典陷阱：如果所有线程都先发后收，就可能互相等待对方腾出缓冲，形成死锁。

[70] 视频画面时间区间：00:57:00–00:57:15。若为裁剪图，时间区间仍对应原视频画面。

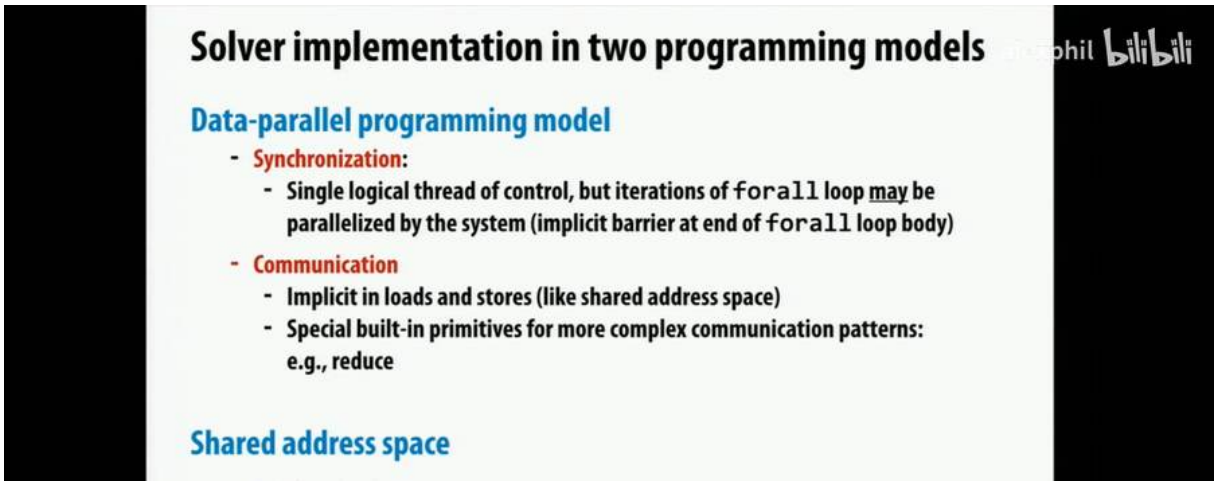

图 71：同一求解器在两种编程模型下的实现对照[71]

两种常规解法。第一种是按编号奇偶交替：偶数号先发后收，奇数号先收后发，这样总有一半线程在收，不会全部阻塞。第二种是使用非阻塞的发送与接收。

死锁是消息传递的入门门槛

共享地址空间里没有对应问题，因为读写不需要双方同时就位。消息传递里传与收是成对的阻塞操作，顺序不当就会互相等待。写消息传递代码时，必须先确认每个通信对的先后顺序，或者直接使用非阻塞原语。

本章小结

同一个红黑格序求解器，在共享地址空间里靠屏障表达依赖，在消息传递里靠交换边界数据加奇偶交替（或非阻塞收发）完成。编排的正确性标准变高了：不仅要算对依赖，还要保证通信不成环等待。

这也回到讲者开头提出的对照——编排是四步中唯一随抽象剧烈变化的一步，而它恰好是决定实际性能的关键。

4.8 总结与延伸

这一讲把”怎么写并行程序”拆成了可检查的步骤，并给出了每一步的失败模式。

讲者的收尾很简洁：我们讨论了编写并行代码的各个步骤——分解、分配，并看了不同的编程模型。接下来两讲会深入这些问题的更多细节。

把全讲串起来，逻辑是这样闭合的。分解决定有多少独立工作，任务数必须不少于处理器数、通常还要更多，但切得太细会带来管理开销。分配决定这些工作如何均衡落下，静态方式没有运行时开销却要求事先估准，动态队列能自动均衡却要付锁竞争。Amdahl 定律在此之上给出一个与处理器数量无关的上界：串行比例 S 把加速封在 $1/S$，图像处理的 $2N^2$ 例子说明只并行第一步最多快两倍。编排决定线程能否正确协作，网格求解器展示了就地更新导致的竞争，以及红黑格序如何在拒绝双缓冲的前提下恢复确定性。消息传递则把同一问题的编排换成显式通信，并引入死锁这一新风险。

本讲义在讲者材料之上做的压缩，有四点需要与讲者原话区分：

[71] 视频画面时间区间：01:12:30–01:12:45。若为裁剪图，时间区间仍对应原视频画面。

- 四步中存在”先量化再动手”的顺序。Amdahl 定律应在分解阶段就用来判断某一部分值不值得并行，而不是等写完代码再测。
- 分配与编排的代价相互牵扯。动态分配为了均衡而引入的锁，本身就是一个需要编排的共享数据结构；任务切细会同时提高均衡性和调度开销。
- 红黑格序是被内存容量逼出来的解法。它的存在前提是不许复制数据，这个前提在内存宽裕的场景下不成立，双缓冲反而更简单。
- 编排的正确性分两层。第一层是消除竞争（红黑格序），第二层是保证可见性与顺序（屏障、栅栏）。只做第一层而不做第二层，代码会时对时错。

这堂课的边界也清楚。讲者反复表示缓存一致性、内存一致性模型、同步原语的具体实现都在后续专门课程里，本讲只提出这些问题的存在。因此这里给出的是解题流程，不是任何模型的完整用法。

两个可以从本讲数据提出的下一步问题：其一，Amdahl 定律假设串行部分是常数，但当问题规模随处理器数增长时，这个假设会失效——那么在什么条件下弱扩展性才成立？其二，红黑格序把一阶段更新切成两阶段，通信次数是否相应增加，这笔账该怎么算？这两个问题都指向本讲没有展开的通信与局部性话题，也正是接下来要处理的内容。

Lecture 05 Work Distribution and Scheduling

视频时长：1 小时 20 分 24 秒 来源：https://www.bilibili.com/video/BV1bp4y1G7tJ/

5.1 性能优化的三个目标

这一讲从”能跑”转向”跑得快”。讲者开宗明义：并行编程是高度迭代的过程，第一次几乎不可能做对，第二次甚至第三次也未必。因此真正的方法不是一次设计到位，而是测量、从测量中学习、再改进。

迭代是方法的本质

讲者把这条作为全讲的前提。学生在做作业时会亲身体会：必须测量性能，从测量里读出信息，再用这些信息改进。这也直接推出了一条建议。

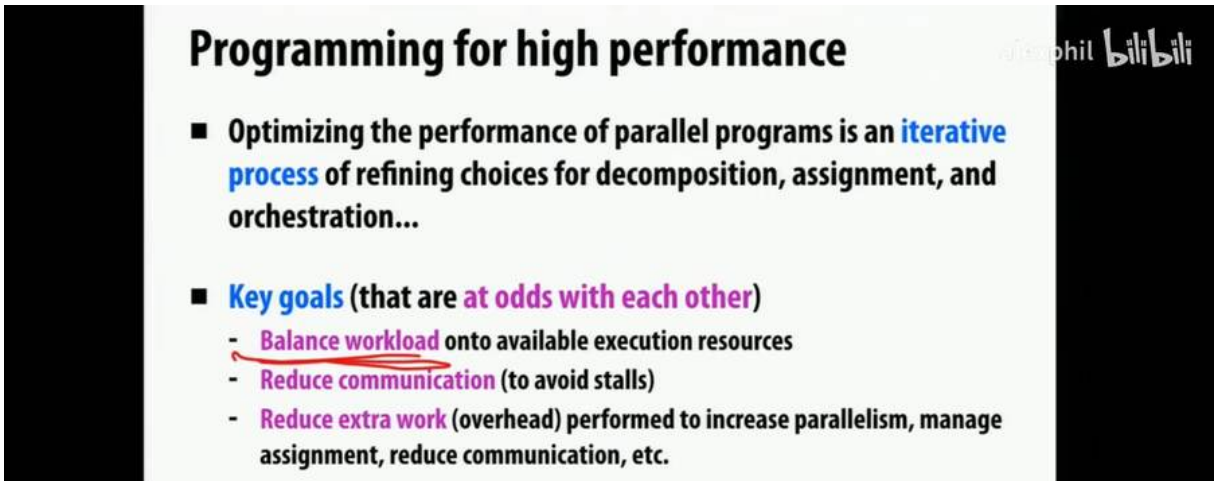

图 72: 高性能编程：优化并行程序是一个迭代过程[72]

操作建议：先写最简单的版本

讲者的建议很具体：先实现能工作的最简单方案。原因有两条。简单的实现你能更快写完，因此更早拿到测量数据；而当被测量的对象本身简单时，你也更容易理解测量结果的含义。有了这个基准，再去做复杂优化才有一个可靠的起点。

简单版本的价值是提供一个可解释的基准

复杂方案第一次尝试通常不会成功，此时你无法判断失败来自设计缺陷还是实现错误。简单版本给出的数字虽然不好看，但它干净、可复现、可解释，后续每次改动都能与它对比。这是一个测量基准，不是临时凑合的代码。

[72] 视频画面时间区间：00:02:30–00:02:45。若为裁剪图，时间区间仍对应原视频画面。

三个必须同时盯住的目标

讲者把今天要处理的问题归纳为三项，它们之间存在张力。

第一项是均衡负载，让所有线程有相同的工作量。有时很容易，有时很难，这是今天的主线。

第二项是减少通信，因为通信昂贵。

第三项是控制软件开销。为了优化而引入的任何聪明做法，往往意味着运行更多指令，这份开销同样会拖慢程序。

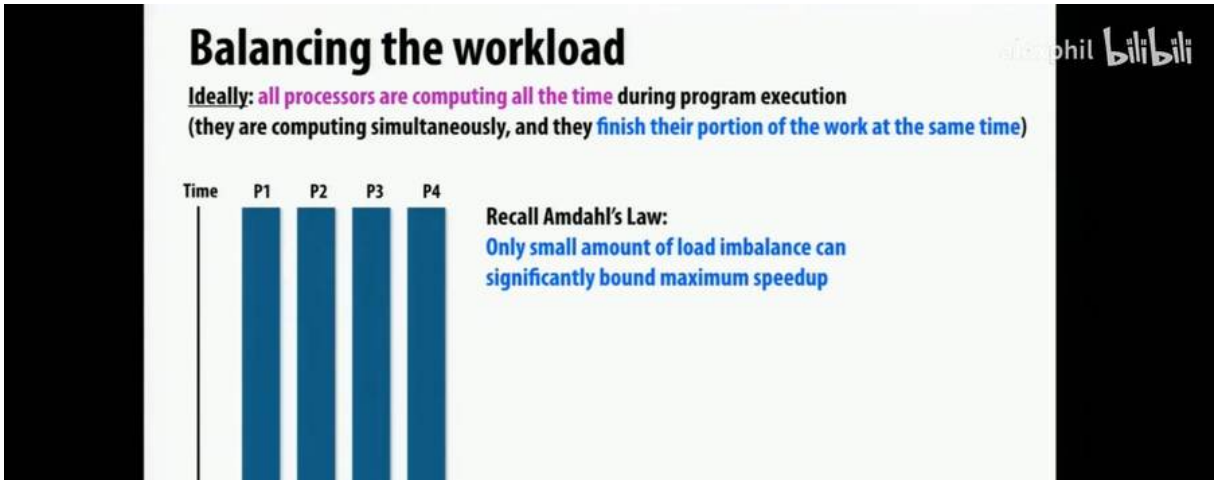

图 73: 负载均衡：理想情形是所有处理器在整个执行期间都在计算[73]

负载不均的代价有多大

讲者用第一堂课的演示作为参照，然后给出一个更精确的说明：即使把工作量分得几乎完全正确，只差一点也会很贵。

他的例子是四个处理器，其中前三个的工作量完全相同，第四个只多出一小部分，四者的差距在 20% 以内。看起来分得不错。但在那第四个多花的时间里，其余三个处理器无事可做。

并行程序的完成时间由最慢任务决定：

$$T_{\text{parallel}} = \max_{i=1\ldots P} T_i, \qquad E_{\text{balance}} = \frac{\sum_i T_i}{P \cdot \max_i T_i}$$

- T_i：第 i 个处理器分到的工作总耗时；
- P：处理器数量；
- T_{parallel}：整体完成时间，等于最慢者的耗时；
- E_{balance}：负载均衡效率，取值越接近 1 越好。

若四者中有三个相同、另一个多出两成，则 E_{balance} 约为 0.95，看似尚可，但那 5% 全部落在收尾阶段，成为无法并行的等待。

[73] 视频画面时间区间：00:04:30–00:04:45。若为裁剪图，时间区间仍对应原视频画面。

负载不均的代价由最慢者决定

并行程序的完成时间取决于最慢的那个处理器，而不是平均值。四者相差 20% 之内看似均衡，实际损失的是整个收尾阶段的并行度。判断均衡程度要看最大工作量与平均工作量之比，不是看方差。

本章小结

性能优化是迭代过程，起点应是可解释的简单实现。要同时权衡三件事：均衡负载、减少通信、控制软件开销。其中负载不均的代价按最慢者计算，20% 的差距就足以浪费一段收尾时间。

这条约束引出核心问题：既然负载不均如此昂贵，那么如何把工作分下去？下一章从最简单的做法开始。

5.2 静态分配：预先决定，零运行期开销

静态分配是在程序运行前就确定好每个处理器做哪些工作。

做法与它的最大优点

讲者回顾上一讲的网格求解器：最终版本的代码里用的是块分配，也讨论过交叉分配。无论哪种，分配逻辑都已经写进代码，因此永远按这个方式分给处理器。

所谓”预先确定”并不排斥少量运行期计算。程序仍需要读取输入文件大小、矩阵规模、处理器数量，据此算一遍。但真正的并行工作开始之前，划分已经完全确定。

它的最大优势是运行期开销接近零：既然已经想过怎么分，运行期不需要再做额外的事。

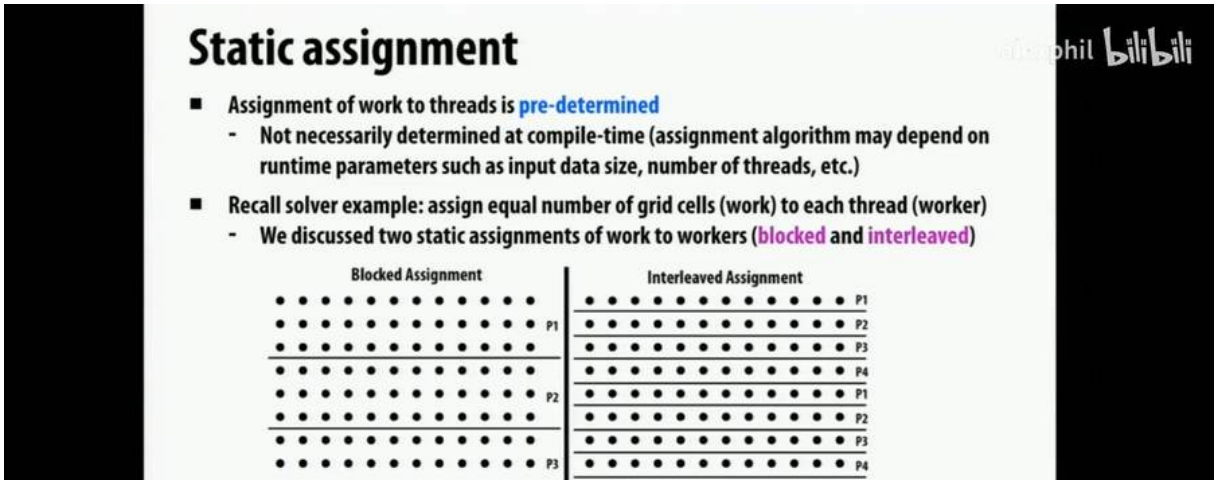

图 74: 静态分配：工作到线程的划分在运行前确定[74]

适用条件：任务时间可预测

学生很快指出了它的缺点：如果某个任务意外地比预期慢很多，你就只能跟着它。

讲者由此给出适用条件。网格求解器之所以能静态分配，是因为每个网格点的计算完全相同，除了缓存缺失等小扰动之外，各任务耗时非常一致。

[74] 视频画面时间区间：00:05:30–00:05:45。若为裁剪图，时间区间仍对应原视频画面。

更一般地说，需要的是任务运行时间可预测。最容易可预测的情形是它们全都一样。另一种情形是任务大小不同，但与某个参数成比例（例如线性相关），那么虽然大小不一，仍能预测。

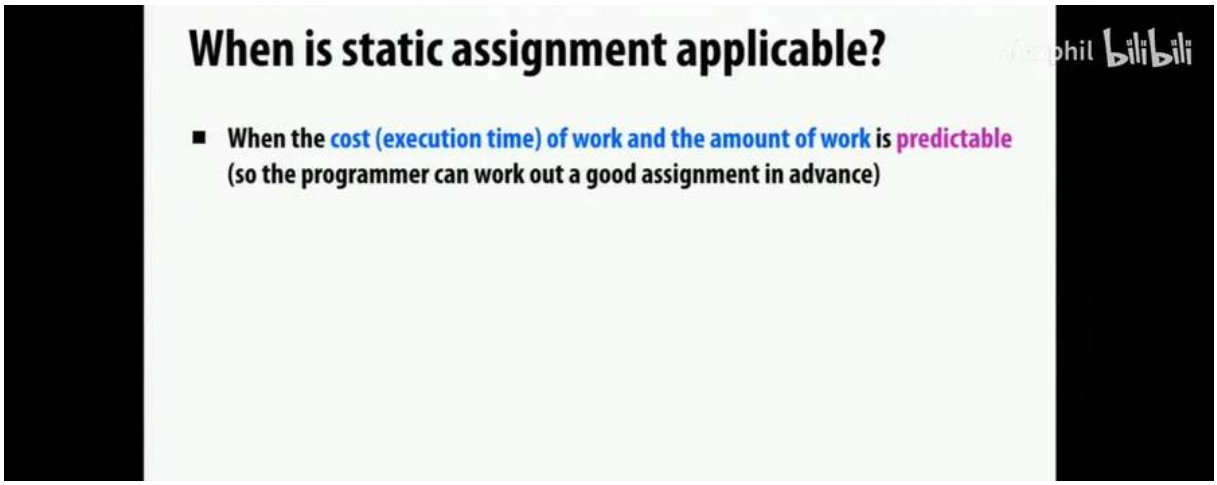

图 75: 静态分配的适用条件：工作开销与工作量可预知[75]

在后一种情形下，做法是把任务打包，使每个处理器分到的任务预期时间之和大致相等。

$$\left| \sum_{k \in S_i} \hat{T}_k - \frac{1}{P} \sum_{k=1}^{M} \hat{T}_k \right| \le \varepsilon$$

- S_i：分给第 i 个处理器的任务集合；
- $\hat{T}_k$：任务 k 的预估耗时；
- M：任务总数；
- ε：可接受的不均衡容差。

打包是个组合优化问题

把大小不一的任务分成 P 组且各组和尽量相等，一般情形下是 NP 难的。实际做法是用贪心（最大任务优先放入当前最轻的组）或按已知的规律直接排序分配。因此静态分配的”均衡”始终是近似的，只是近似误差被压在可接受范围内。

只要预估准确，这个打包就能在运行期零开销的情况下接近均衡。这样打包未必完美，但优点仍然是运行期几乎无开销。讲者的判断是：如果只牺牲一点均衡能换回极低的运行期开销，这笔交易值得做。

本章小结

静态分配把划分写在代码里，运行期开销接近零，代价是对可预测性的强依赖。任务耗时一致，或至少能按参数预估时，它是最优选择。若做不到这一点，就需要把决策推迟——但推迟也有程度之分。

[75] 视频画面时间区间：00:08:00–00:08:15。若为裁剪图，时间区间仍对应原视频画面。

5.3 半静态分配：在两者之间取一个折中

讲者特意强调，这一节的内容往往不易想到，但很值得掌握。

为什么需要中间方案

很多系统按时间步推进模拟。每个任务的耗时可能随时间变化，看起来无法静态划分。但如果这份变化很缓慢，情况就不同了。

星系模拟就是例子。上一讲提过 Barnes-Hut 算法的工作量不均匀：某个星体周围若有很多邻居，它要做大量成对比较；比较孤立的星体则可以把远处星群合并成一个质量块，省下很多计算。所以工作量既不均匀，又随时间变化。

但讲者的关键补充是：星体移动得并不快。

定期重测，期间静态

于是可以做半静态调度。每隔一段时间，测量一次当前各任务的耗时，据此重新决定划分。然后在此后若干次迭代里，就按这个划分当作静态调度执行，不再重新审视。

重划分周期的两个约束

周期太短，测量与重划分本身的开销占比过高；周期太长，划分依据会过时，负载已经漂移。合理周期取决于负载变化的时间尺度与一次重划分的代价，通常取”重划分开销只占该区间总时间几个百分点”的量级。

等过了一段时间，再回来重新测量、重新划分。

讲者说这类技术在其他物理模拟中同样适用，例如风洞里的飞机模型：物体在动、状态在变，但通常不会剧烈到需要每步重划分。

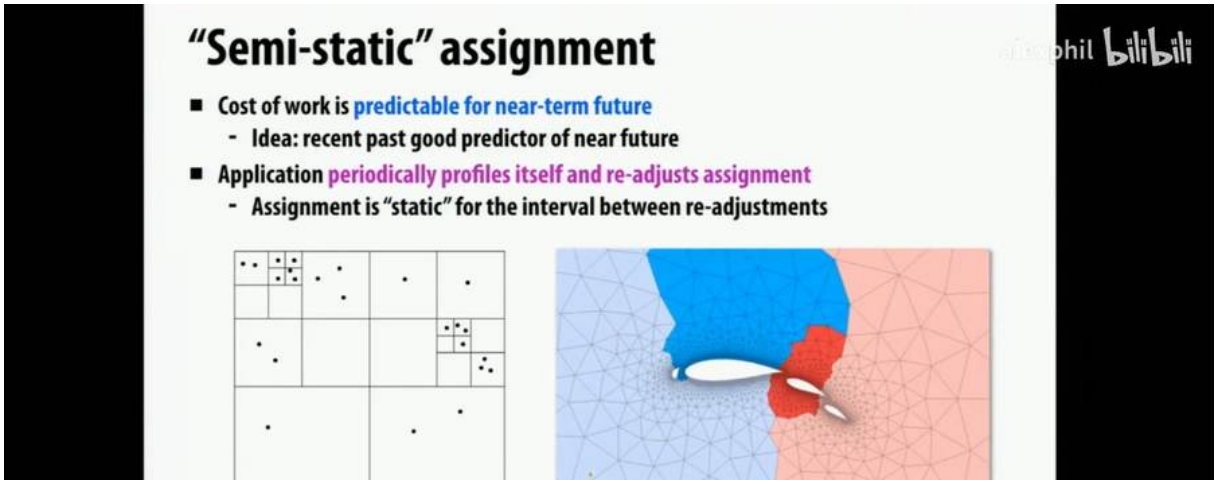

图 76：半静态分配：在近期未来内工作的开销仍可预测[76]

[76] 视频画面时间区间：00:10:30–00:10:45。若为裁剪图，时间区间仍对应原视频画面。

半静态的前提是变化慢于重划分周期

半静态只在负载变化的时间尺度远大于重划分间隔时有效。若网络流量这类负载在毫秒级剧烈波动，周期性重划分反而会一直基于过时的信息，此时纯动态调度更合适。判断依据是负载的自相关时间，而不是数据规模。

本章小结

半静态把”定期测量加期间静态”组合起来，在负载缓慢变化时用极低的平均开销获得接近动态的均衡质量。它填补了静态与动态之间的空白，也说明分配策略是一条连续谱，而非两种对立选择。

当负载真的不可预测时，就只能把决策做到运行期。下面四章都在处理这种情形的代价。

5.4 动态分配：让线程按需取活

动态分配在程序运行期间由线程自己决定拿哪份工作。

最简单的实现

讲者用一段代码说明。假设有一个顺序循环，各次迭代彼此独立，那么每次迭代就是一个任务。可以静态地按块或交叉分配，也可以动态地这样写：维护一个计数器，本质上就是循环下标；线程需要工作时读取这个计数器并自增，因此必须加锁或使用等价的原子指令，否则计数会损坏。

动态分配的均衡以正确性为前提

动态队列只保证工作被分出去，不保证迭代之间真的独立。若循环体存在跨迭代依赖，动态分配的先后顺序会改变结果，且每次运行可能不同。判断能否动态调度，先确认任务独立性，再谈粒度。

各线程拿到一个迭代就去做自己的计算。如果某些迭代远比其他耗时，最终也能大致平衡，因为你只有在准备好时才回去取新工作。

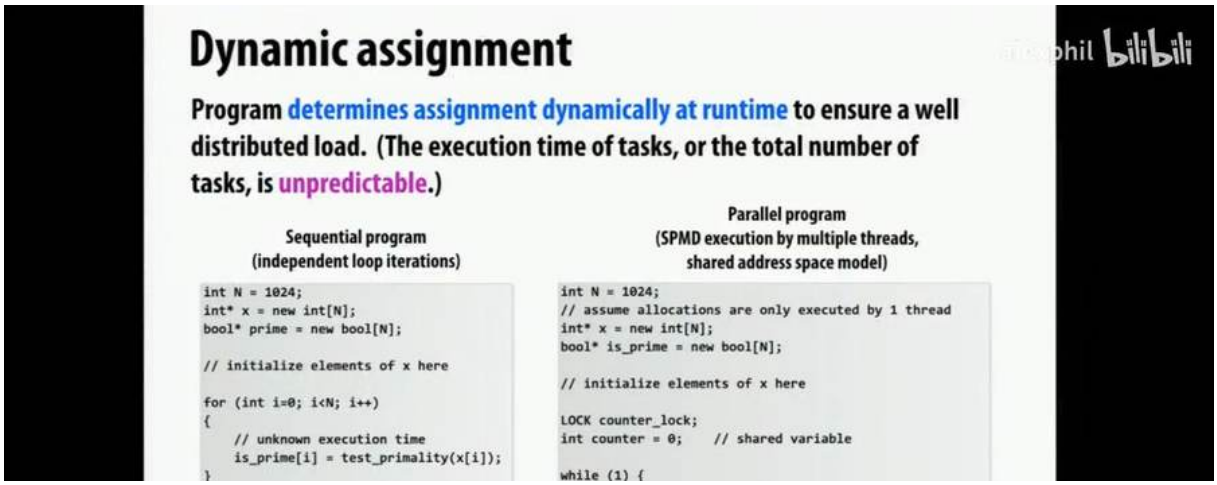

图 77：动态分配：程序在运行期决定划分以保证负载均衡[77]

[77] 视频画面时间区间：00:14:00–00:14:15。若为裁剪图，时间区间仍对应原视频画面。

动态分配的最大优势是它适用于执行时间不可预测、且波动较大的情形。这正是静态分配失效之处。

工作不必是循环迭代

讲者补充了一点：前面把工作表示成循环迭代，但更一般地，工作可以是任何需要数据结构来描述的东西。星系模拟里的每颗星就是四叉树中的一个节点，我们要访问所有节点。

若要纯动态调度，可以为每个任务建一个小数据结构，全部丢进一个大工作队列。有四个核，就有四个工作线程，谁需要工作就去队列里取。

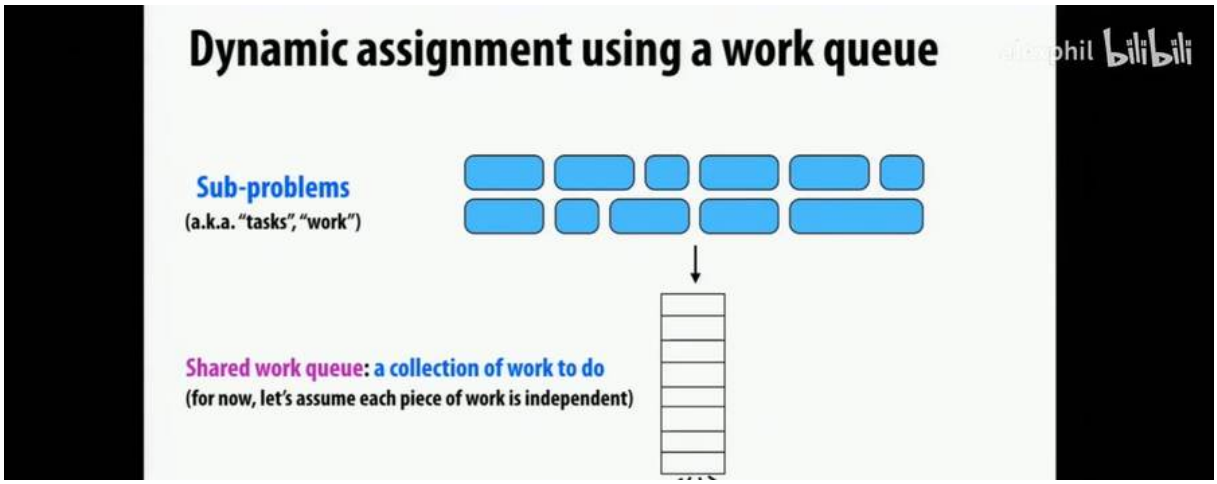

图 78: 用工作队列实现动态分配：所有任务进入共享队列[78]

本章小结

动态分配用共享计数器或工作队列把划分推迟到运行期，因此能适应不可预测的耗时。它简单、正确，但讲者随即提出一个尖锐问题：这样写的代码，性能如何？

5.5 细粒度动态分配的代价与粒度旋钮

上一章的代码在功能上没有问题。讲者故意追问：如果它跑得特别慢，你认为原因是什么？

锁竞争吃掉了时间

答案是加锁。线程频繁去抢那把保护计数器的锁，大量时间花在竞争上而非计算上。这种一次只取一个任务的写法叫细粒度动态分配。

[78] 视频画面时间区间：00:16:30–00:16:45。若为裁剪图，时间区间仍对应原视频画面。

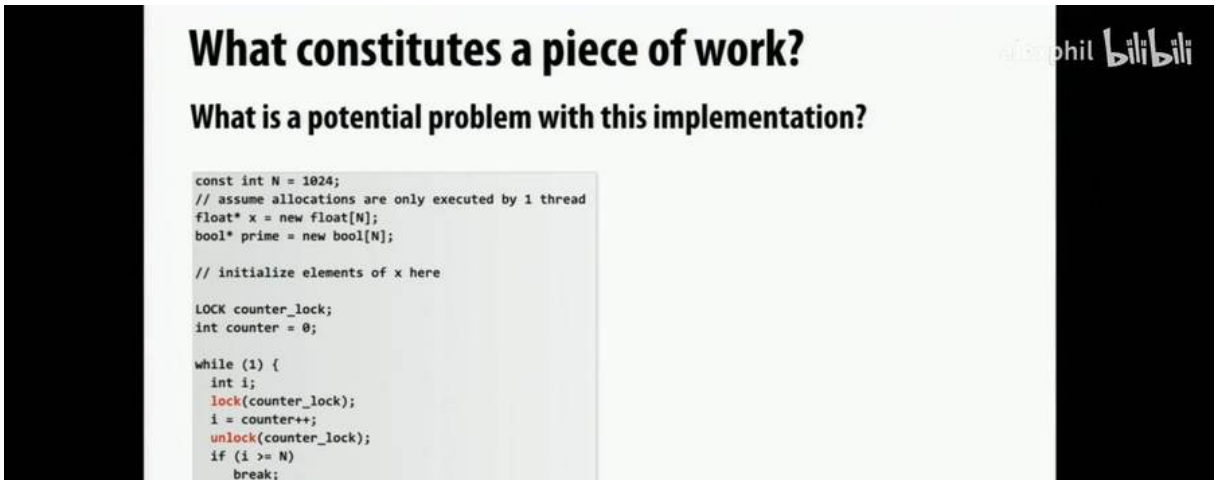

图 79: 什么算一块工作：一次只取一个迭代的写法及其问题[79]

讲者用时间轴把损失画出来：蓝色是真正做有用工作的时间，白色是去队列抢锁、访问计数器的时间。图上白色段极多。他明确指出，白色部分实际上就是 Amdahl 定律里的串行时间，完全不是有效工作时间。

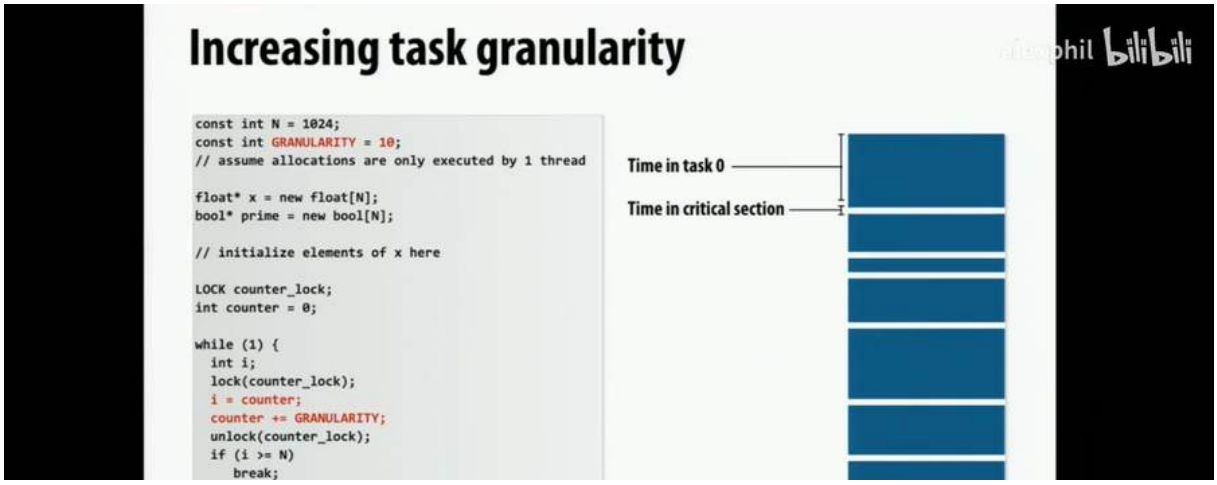

图 80: 增大任务粒度：一次抓取多个迭代来减少取任务次数[80]

一次取多个

学生提出的办法是：不要只算一个，一次取若干个（比如五个或十个）。线程一次从计数器拿到一段，然后遍历这一整段。

好处是任务变大，回去取新工作的频率下降，白色那部分开销因此减少。

把开销写成显式模型就能看清旋钮的作用：

$$T_{\text{total}} = \frac{W}{P} + \frac{W}{G} \cdot c_{\text{lock}} + \delta_{\text{imbalance}}$$

- W：总工作量；

[79] 视频画面时间区间：00:18:00–00:18:15。若为裁剪图，时间区间仍对应原视频画面。

[80] 视频画面时间区间：00:20:30–00:20:45。若为裁剪图，时间区间仍对应原视频画面。

- P：线程数；
- G：每次抓取的任务数，即粒度；
- c_{lock}：一次加锁取任务的代价；
- $\delta_{\text{imbalance}}$：粒度变粗后重新出现的负载不均损失。

G 增大时中间项下降、末项上升，因而存在最优值。

粒度是并行调度最有效的单一旋钮

一次取多少任务，直接决定两项成本的比例。取得少，均衡好但取任务开销占比高；取得多，开销低但可能重新出现负载不均。多数调度器的默认粒度值都是按这个权衡调出来的，调优时优先改这一个参数。

粒度过大的代价

讲者接着问：这么做有什么坏处？答案是粒度太粗会重新引入负载不均。

所以在光谱的两端都不好——工作太少或任务太大伤均衡，任务太碎伤开销。讲者说，并行编程里很多事情都是这样，中间存在一个合适的位置。

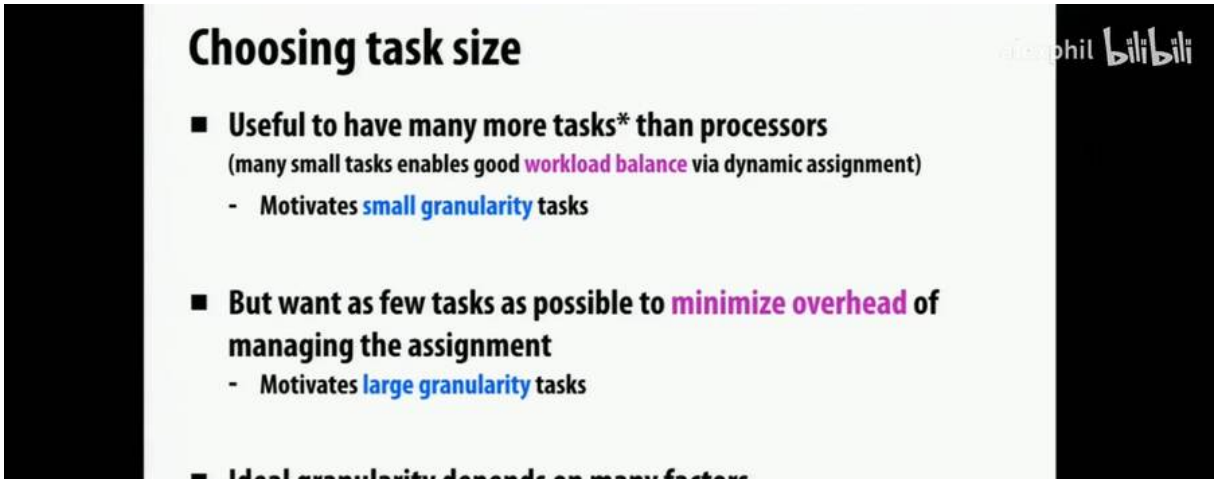

图 81：任务规模的选择：任务数应远多于处理器数[81]

收尾陷阱：最后派出的长任务

讲者随后展示了一个即使粒度合适也会出问题的情况。任务从左到右进入队列，依次派出，最后派出的那一件恰好在最右边。

如果那件恰好耗时长，就会出现这样的情形：其余处理器很快做完，然后全体坐着等这件大任务。这属于运气不好，但确实会发生。

[81] 视频画面时间区间：00:22:30–00:22:45。若为裁剪图，时间区间仍对应原视频画面。

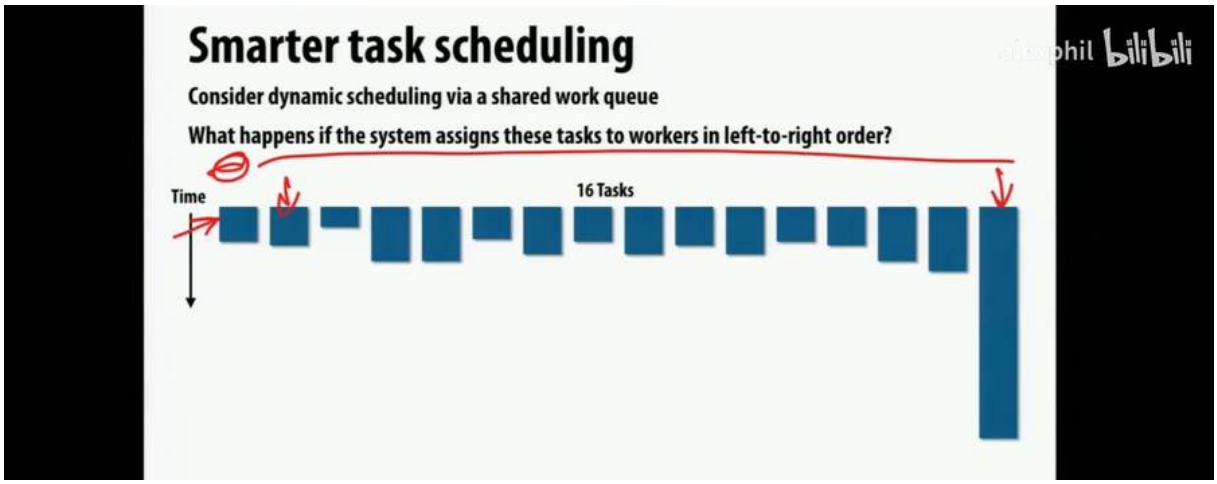

图 82: 更聪明的任务调度：最后才派出的大任务导致全体空等[82]

一个理想但需要知识的策略

学生提出给任务优先级。讲者认可：如果事先知道这是个大任务，早点派出去比放在最后好得多。若把同一份工作提前派给某个处理器，那个处理器会花较长时间做它，而其他线程能找到别的任务，总体就均衡了。

由此得到一条更深的洞察。在这样一张负载图上，最优调度应当是执行早期用较大的任务，只在接近结束时才需要较小的任务。原因是大任务能压住运行期开销，而这个优势会一直成立到收尾阶段——那时拼图块对不上了，才开始浪费时间。

讲者的说法很形象：开头用大球，结尾用沙粒。这需要事先对哪些任务更大有一些粗略知识；另一个变体是随时间动态调整粒度，例如示例中的粒度 10，可以在接近结束时逐步调小。

收尾阶段的并行度天然衰减

无论分配策略多好，最后总剩少量工作无法再被均分，此时并行度会从 P 掉到 1。这不是调度失误，而是工作量有限的必然结果。改善方向是让最后剩下的任务尽可能小（因此开头可以用大任务），而不是试图消除这个阶段。

本章小结

细粒度动态分配被锁竞争拖垮，因为每次取任务都要抢锁，这段等待本质上就是 Amdahl 定律中的串行部分。加大粒度能压住这笔开销，但过大会重新引入不均，因此存在最优区间；更精细的做法是让粒度随时间从大到小。

粒度问题解决后还有一处结构性问题：所有线程都在抢同一个队列，这份争用本身也需要消除。这是下一章的内容。

5.6 分布式队列与 work stealing

即使取任务的频率已经降下来，只要队列真的是一个中央队列，每次访问仍要与其他线程通信、争抢同一把锁。讲者由此引出替代方案。

[82] 视频画面时间区间：00:23:30–00:23:45。若为裁剪图，时间区间仍对应原视频画面。

每线程一个队列

把队列拆开，每个硬件线程一个自己的队列，这叫分布式工作队列。

做法是先按最佳猜测把工作铺到各队列，然后开始运行。此后某线程始终优先访问自己的队列；计算过程中若产生新工作，也放回自己的队列。

这样在大部分执行时间里效果极好：访问自己的队列，没有人与你争用，局部性也很好。

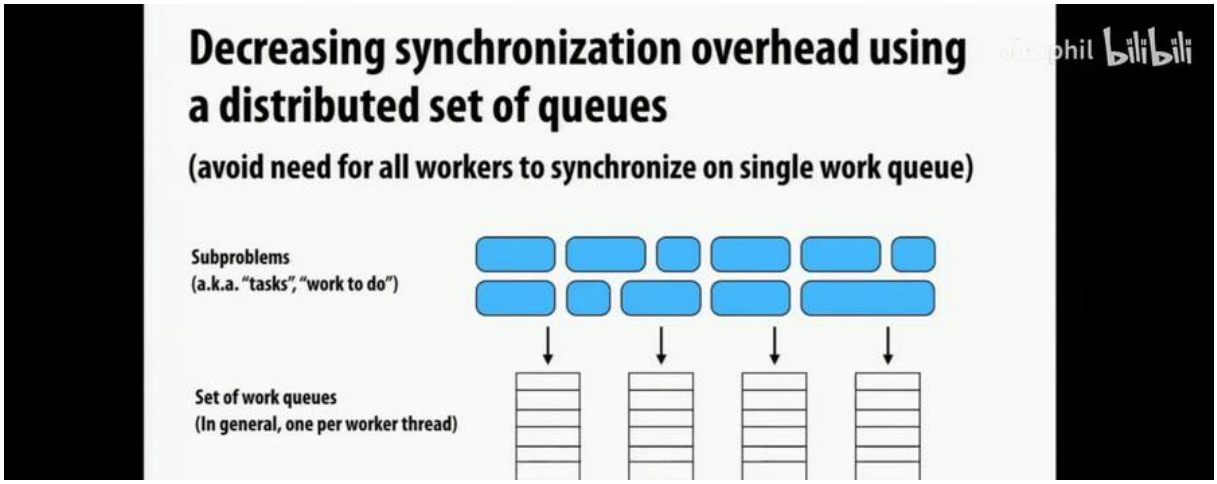

图 83: 用分布式工作队列降低同步开销[83]

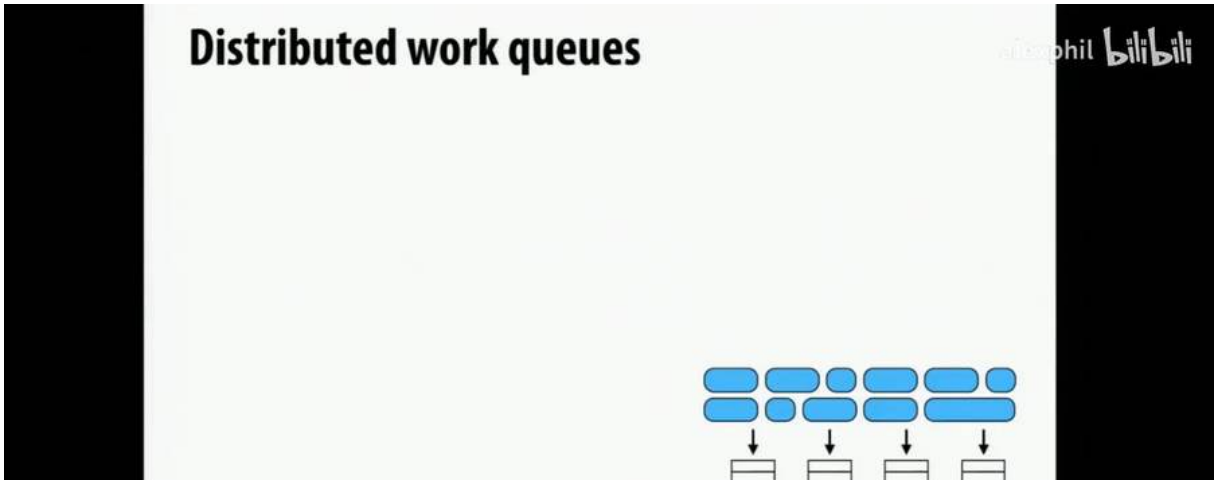

图 84: 分布式工作队列：每个处理器一个队列[84]

队列耗尽的必然性

问题随之而来：我们之所以用动态调度，正是因为难以预测执行时间并平均切分。既然一开始就分不准，就必然有线程会先把自己的队列做空。

最简单的做法是干等。也可以设一个备用队列，让早完成的线程从那里取工作，争用人数会比中央队列少。

[83] 视频画面时间区间：00:27:00–00:27:15。若为裁剪图，时间区间仍对应原视频画面。

[84] 视频画面时间区间：00:31:30–00:31:45。若为裁剪图，时间区间仍对应原视频画面。

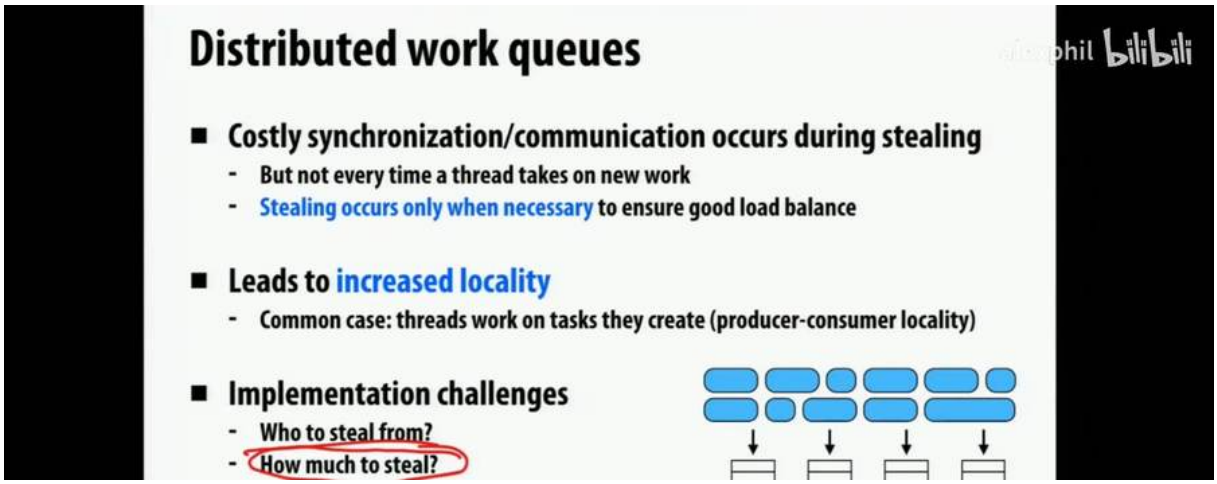

图 85: 分布式队列下的窃取：只在必要时才去别处取工作[85]

窃取的三个设计要点

讲者给出的主流答案是 work stealing：工作用完时去别的队列偷，这样不必干坐着。

偷工作不是免费的，涉及通信与同步。但好在只有有人没活时才会发生，因此大部分执行期间没有这项开销。讲者提醒仍需小心：不要一进入窃取就突然切换成极慢的模式。

关于从谁那里偷，Cilk 的运行时采用随机选择。讲者解释为什么不能总是从编号小的开始偷：如果所有线程都先去偷 0 号、再偷 1 号，反而会制造新的失衡。

窃取策略必须避免对称

让所有线程按同一顺序去偷（先偷 0 号、再偷 1 号），会在空闲线程之间制造新的热点：所有空闲者同时扑向同一个队列。随机选择或按线程编号做散列，目的都是打破这种对称，使窃取请求均匀分散。

因此要随机，或者用任何能保证大致均匀的方式。

关于偷多少，答案是一次偷多个。只偷一个的话，很可能马上又要再偷一次。

若每次只偷一个任务，窃取次数与任务数同阶；每次偷 G 个，则窃取次数降为：

$$N_{\text{steal}} \approx \frac{M}{G}$$

- N_{steal}：总窃取次数；
- M：剩余任务总数；
- G：单次窃取的任务数。

窃取本身含通信成本，因此增大 G 同样是在压这项开销，代价与动态分配相同：偷得太多会重新造成不均。

[85] 视频画面时间区间：00:35:00–00:35:15。若为裁剪图，时间区间仍对应原视频画面。

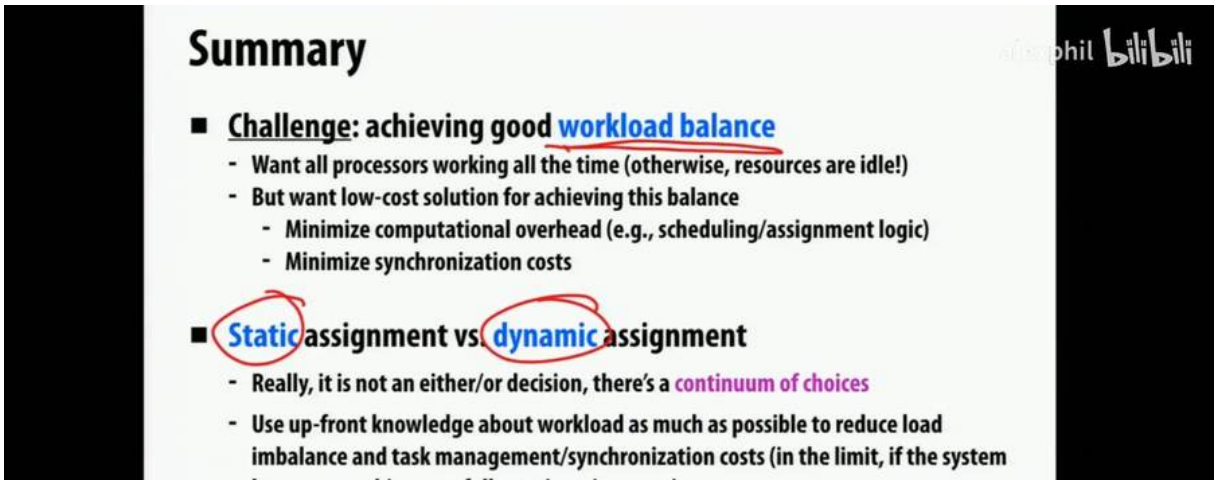

图 86: 负载均衡与低开销之间的权衡总结[86]

收尾阶段的集中争用

讲者也点出一个边界情况：真正到结束时，剩下的工作很少且都集中在一个队列里，此时争用会变得很严重。

但在通常情形下，一旦线程开始窃取，许多队列里往往还有不少工作可偷，所以实际做法是偷走其中一定比例。

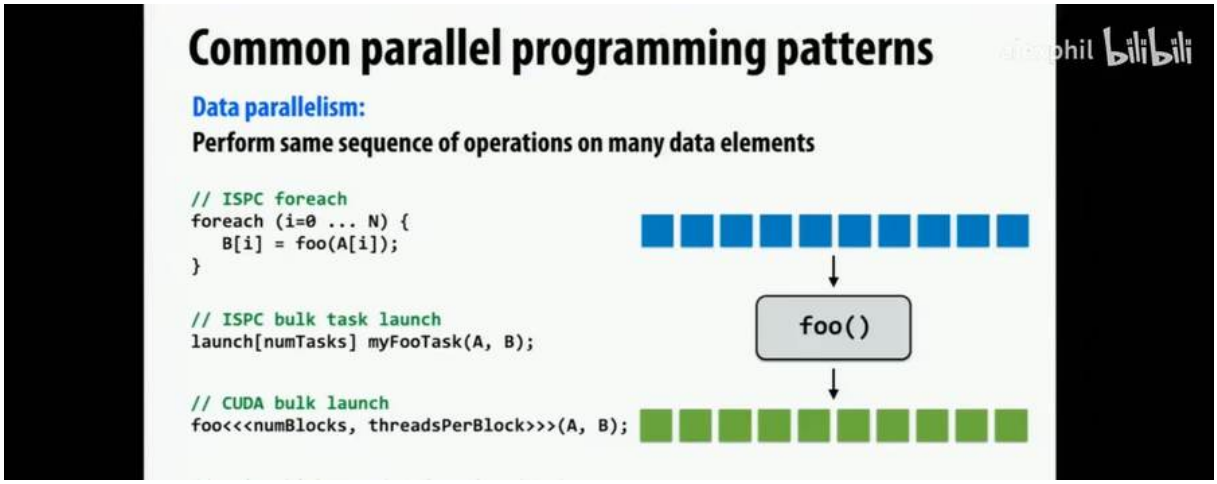

图 87: 常见并行编程模式：数据并行与 fork-join 等[87]

本章小结

分布式队列用私有性消除了中央队列的争用，work stealing 在必要时补上均衡。三个设计要点是：只在需要时窃取、随机选择目标、一次偷多个。它同时得到动态调度的均衡与静态调度般的局部性，代价是窃取发生时的通信与收尾阶段的集中争用。

讲者最后补充一个提醒：任务队列里的工作彼此不必独立，这一点在后续课程里会展开。

到这里分配与调度的原理已完整。剩下要看的是一套真实系统怎么把这些原理落实为语言原语。

[86] 视频画面时间区间：00:37:00–00:37:15。若为裁剪图，时间区间仍对应原视频画面。

[87] 视频画面时间区间：00:41:00–00:41:15。若为裁剪图，时间区间仍对应原视频画面。

5.7 Cilk 的 fork-join：抽象如何决定调度细节

讲者用 Cilk Plus 展示一个真实运行时系统的实现选择。

分治与 fork-join 模式

讲者先回顾常见并行模式，再聚焦到分治算法。

以快速排序为例：把数组分成两部分，各自递归排序，再合并。两部分互不依赖，这就是天然的并行来源。

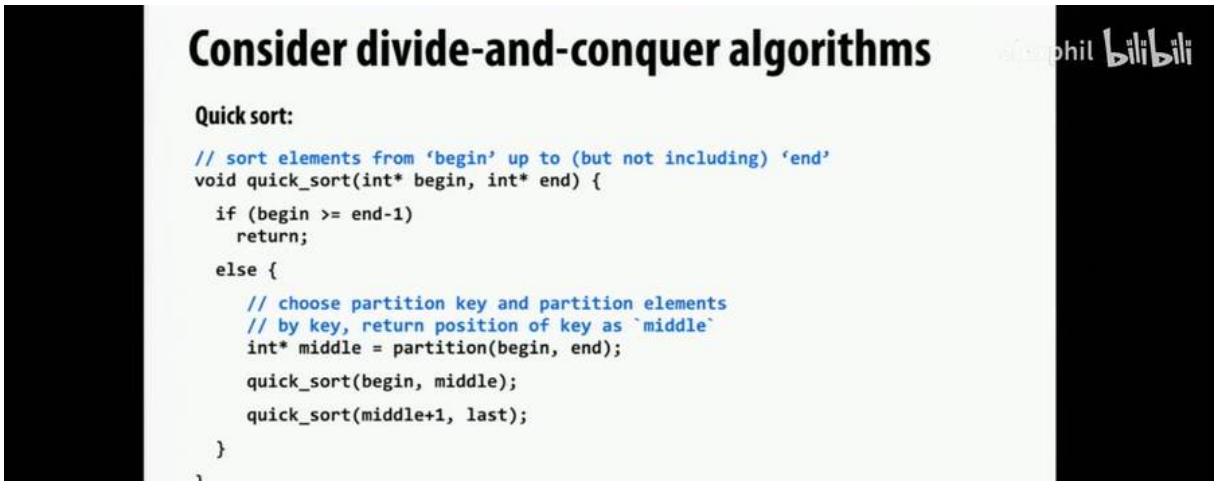

图 88: 分治算法：以快速排序为例[88]

表达这种结构的自然方式叫 fork-join：在一处 fork 出独立工作，在另一处 join 等它们完成。

图 89: Fork-join 模式：表达分治算法中独立工作的自然方式[89]

spawn 与 sync

Cilk Plus 用两个关键字实现这个模式：cilk_spawn 表示这里可以并行，cilk_sync 表示等待所有 spawn 出来的工作完成。

[88] 视频画面时间区间：00:42:00–00:42:15。若为裁剪图，时间区间仍对应原视频画面。

[89] 视频画面时间区间：00:44:30–00:44:45。若为裁剪图，时间区间仍对应原视频画面。

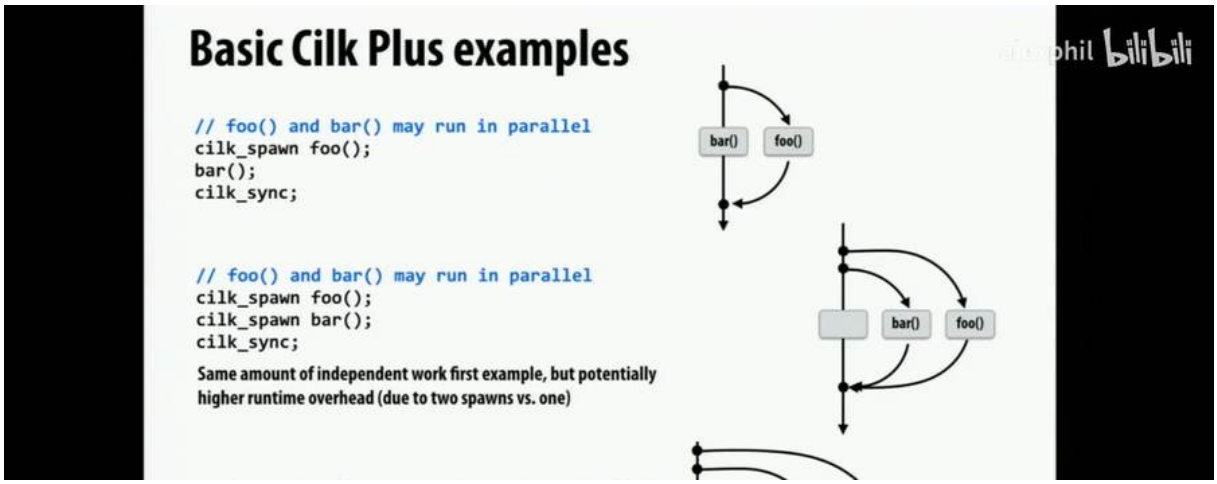

图 90: Cilk Plus 基础示例：cilk_spawn 与 cilk_sync[90]

关键在于讲者强调的一点：cilk_spawn 这个抽象并不指定哪个线程去跑哪一段。

抽象不承诺任何调度细节

cilk_spawn 只说这里可以并行，既不说在哪个核上跑，也不说谁先谁后，更不保证负载均衡。因此同一份 Cilk 程序在不同运行时、不同核数下的行为可以差别很大。这类抽象牺牲了可控性，换取了可移植性与简洁。

抽象只表达这¨ 里可以并行，¨ 具体怎么映射是实现的事。

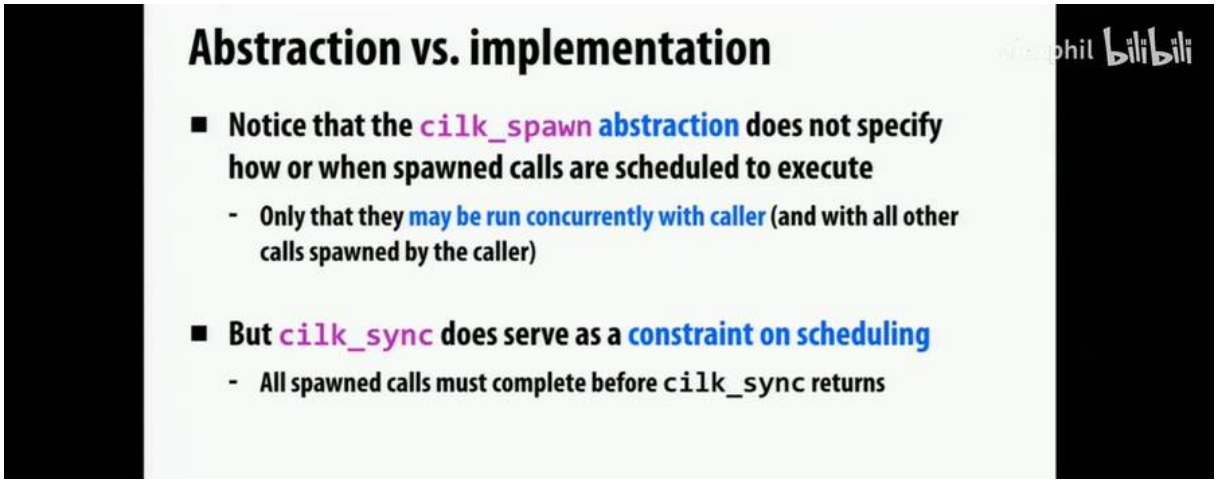

图 91: 抽象与实现：cilk_spawn 不指定线程，映射由运行时决定[91]

[90] 视频画面时间区间：00:46:30–00:46:45。若为裁剪图，时间区间仍对应原视频画面。
[91] 视频画面时间区间：00:48:00–00:48:15。若为裁剪图，时间区间仍对应原视频画面。

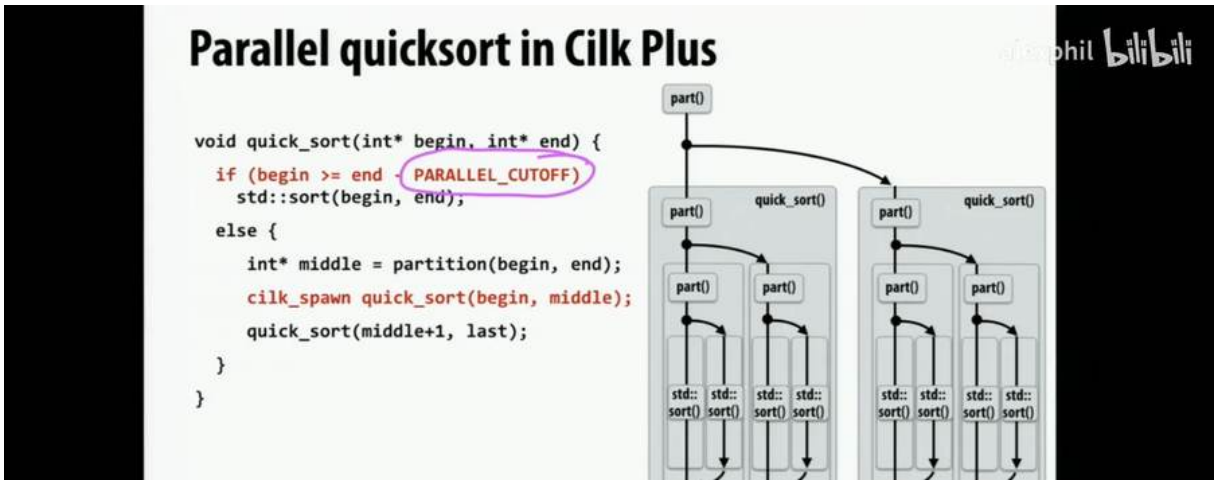

图 92: Cilk Plus 中的并行快速排序：递归部分用 spawn 并行[92]

为什么不能每个 spawn 开一个 pthread

讲者设想了最朴素的调度器：每个 cilk_spawn 就启动一个 pthread。这显然不可行——递归里会有成千上万个 spawn。

实际做法是运行时维护一个工作线程池。例如一台典型的笔记本可能四核，加上超线程就是八个硬件线程，Cilk Plus 就为你生成八个线程。讲者强调，重点不在于创建线程，而在于把线程指向该做的工作。

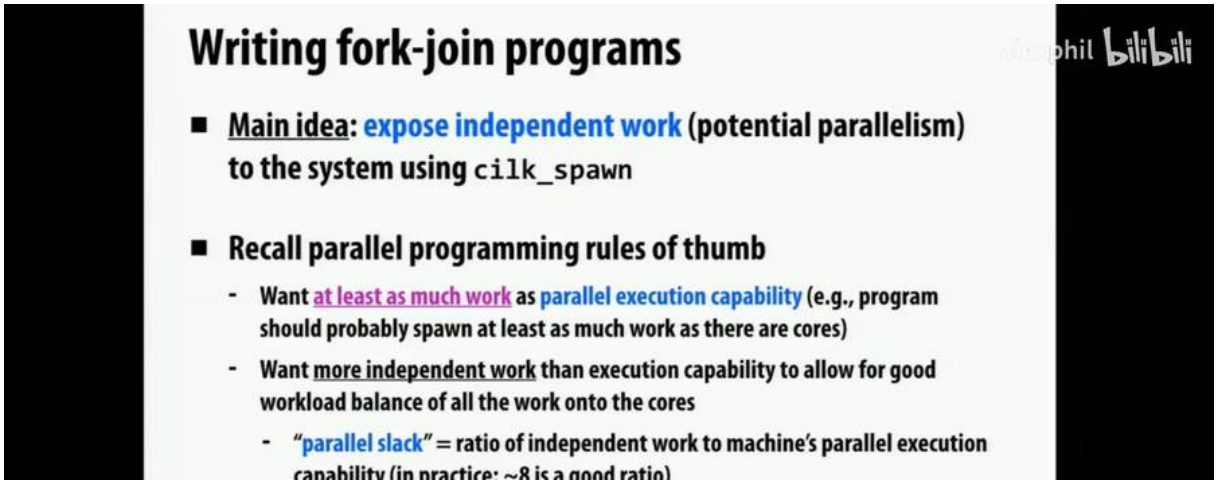

图 93: 编写 fork-join 程序：暴露独立工作并等待其完成[93]

[92] 视频画面时间区间：00:49:00–00:49:15。若为裁剪图，时间区间仍对应原视频画面。

[93] 视频画面时间区间：00:51:00–00:51:15。若为裁剪图，时间区间仍对应原视频画面。

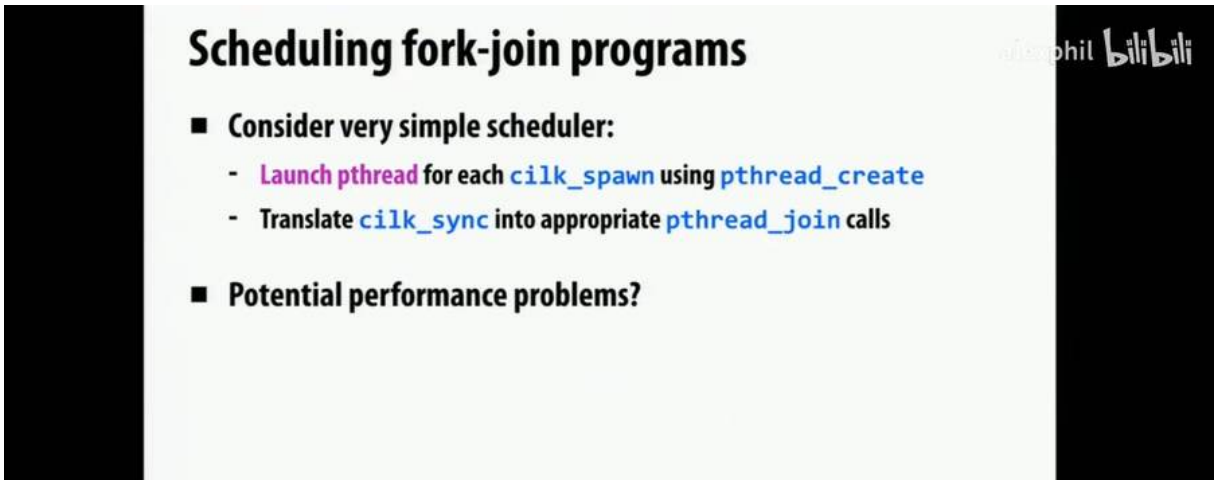

图 94: fork-join 程序的调度：最朴素的方案每 spawn 开一个线程[94]

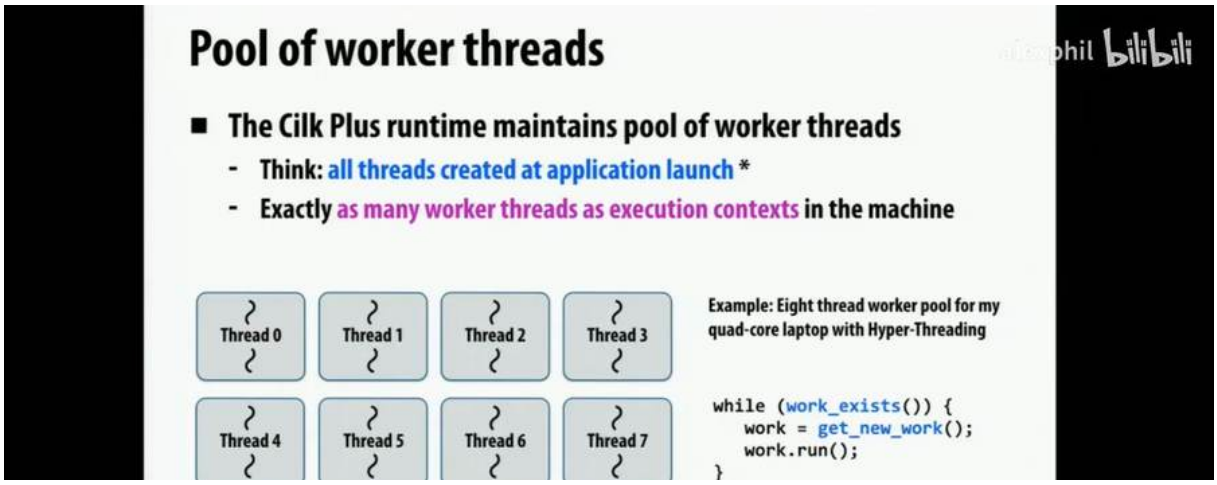

图 95: 运行时维护的工作线程池：所有线程共享同一批工作[95]

child 与 continuation

讲者引入两个术语来描述 spawn 处的两份可并行工作。被调用者 foo 叫 child（子任务）；调用之后要继续做的事叫 continuation（延续），在例子里就是 bar。

在 spawn 处，本线程要做其中一件，把另一件放进自己的队列，等待被别的线程偷走。

[94] 视频画面时间区间：00:52:30–00:52:45。若为裁剪图，时间区间仍对应原视频画面。

[95] 视频画面时间区间：00:54:30–00:54:45。若为裁剪图，时间区间仍对应原视频画面。

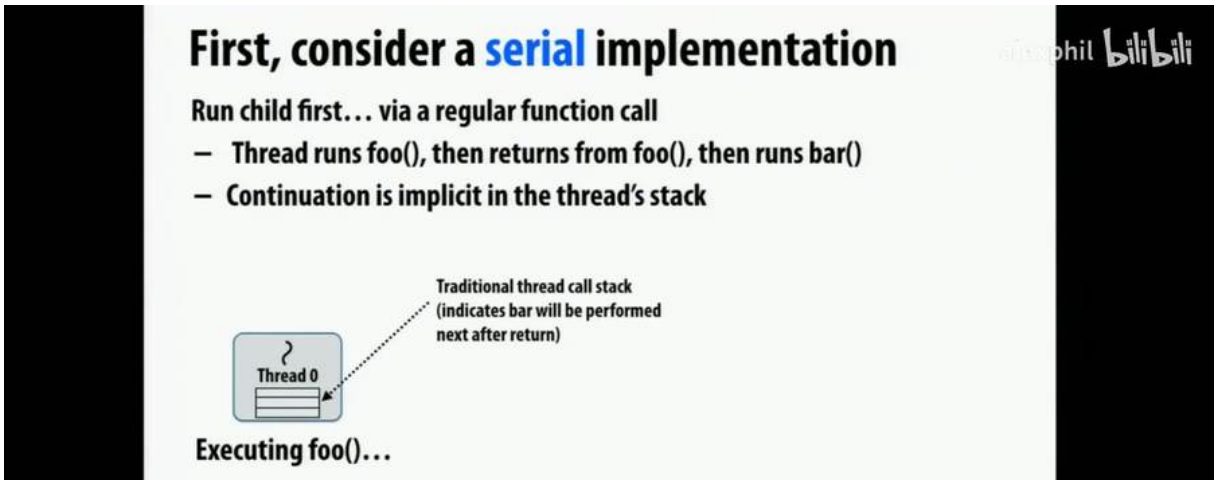

图 96: 先看串行实现：正常返回后继续执行后续代码[96]

讲者先看串行情形：没有 spawn 时，进入 foo、执行完返回，然后再做 bar。程序如何知道返回后要做什么，靠的是调用栈。

而在 Cilk 中，从 cilk_spawn 返回时不走普通返回路径，而是回到自己的队列取一件工作继续。这多了一点软件开销，但好处是若该线程随后空闲，它能发现自己的队列空了，并去别的队列偷工作。

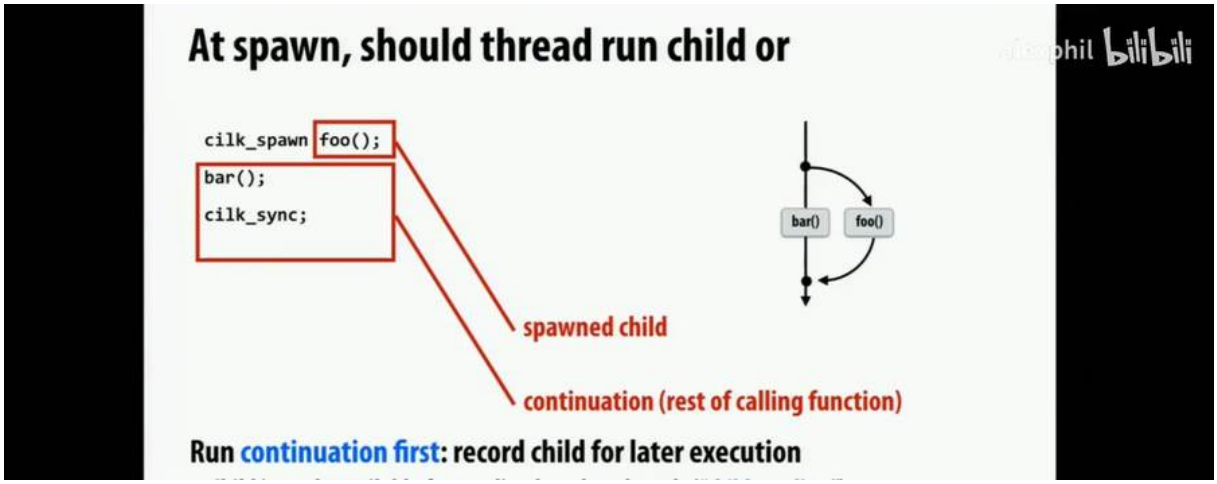

图 97: 每线程工作队列：遇到 spawn 时把 continuation 放进自己的队列[97]

[96] 视频画面时间区间：00:57:00–00:57:15。若为裁剪图，时间区间仍对应原视频画面。

[97] 视频画面时间区间：01:01:30–01:01:45。若为裁剪图，时间区间仍对应原视频画面。

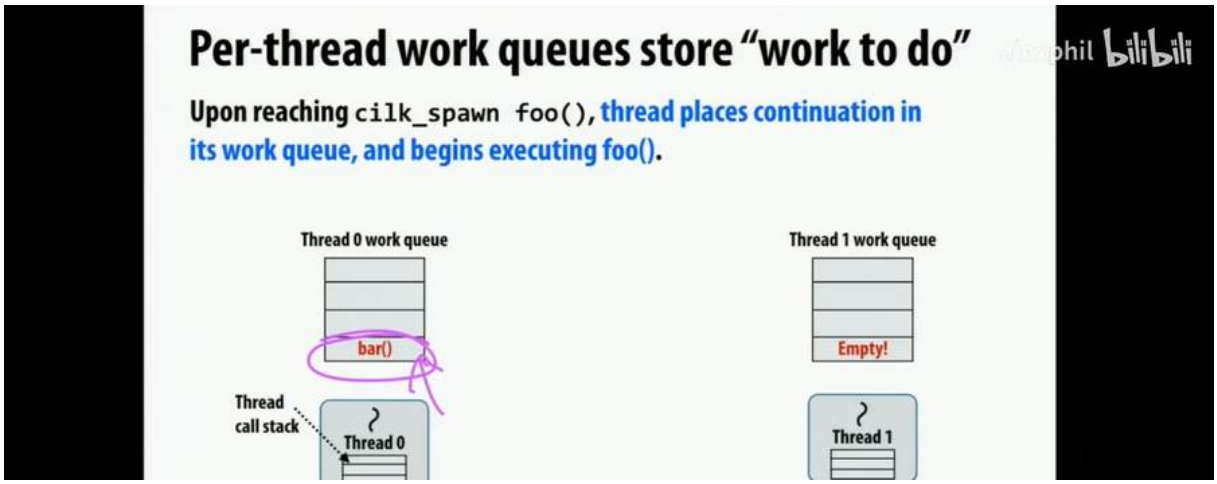

图 98: 每线程队列保存待办工作，空闲线程从别处窃取[98]

该先做 child 还是先做 continuation

这是本讲最有意思的权衡。学生提出可以直接把 foo 放进队列，自己去做 bar。讲者顺势展开讨论。

spawn 两种选择的表面优劣

先做 continuation：线程把大量后续工作堆进自己的队列，看起来储备充足，别的线程来偷也还有活干。先做 child：队列里只剩一件 continuation，一旦被别人偷走，本线程就没活可做了。仅凭这个观察，先做 continuation 似乎更安全。

讲者把这个情形推广成循环：对 n 个实例依次 spawn foo。若先做 continuation，每轮都会把 foo 放进队列然后继续，于是队列越堆越多。

若先做 child，线程执行 foo(0)，返回后发现刚过循环中那个点，于是继续下一轮执行 foo(1)。讲者指出这实际上很像深度优先遍历，访问方法的顺序很可能与顺序程序一致。

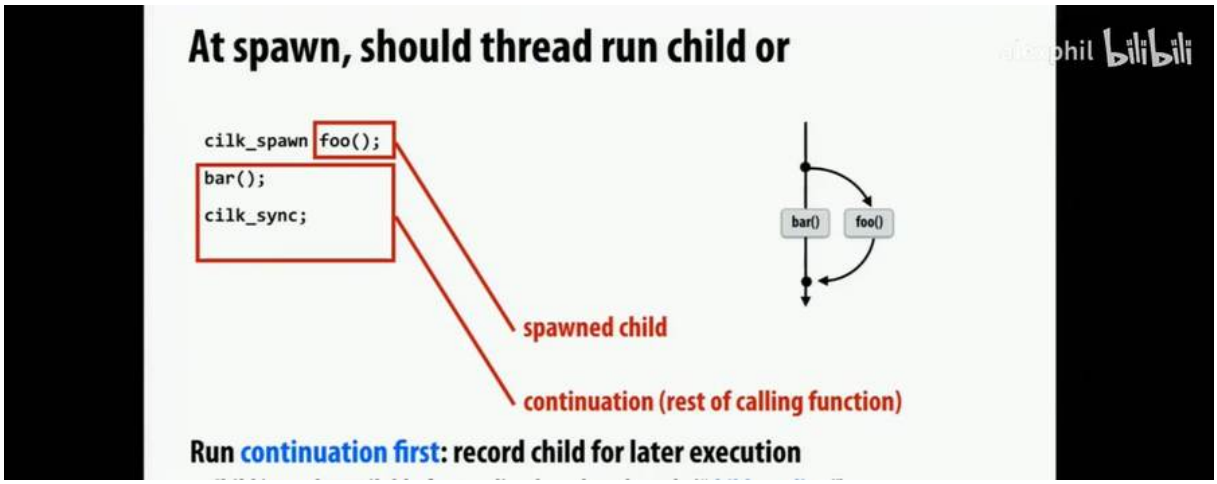

图 99: spawn 时该执行 child 还是 continuation[99]

[98] 视频画面时间区间：01:00:00–01:00:15。若为裁剪图，时间区间仍对应原视频画面。

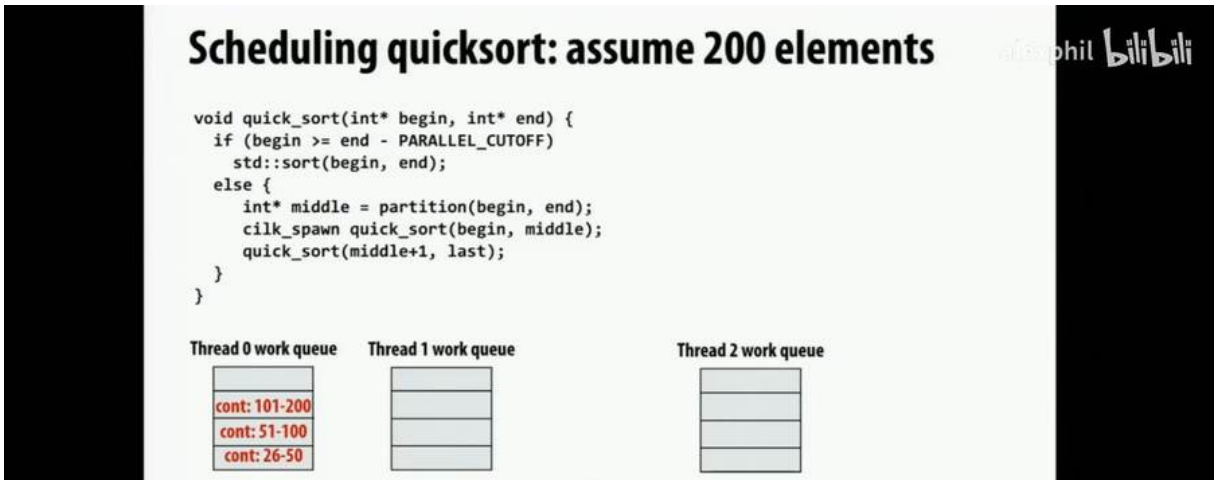

图 100: 快速排序调度示例：200 个元素时两种策略的差异[100]

Cilk 的真实选择

讲者让学生判断 Cilk 采用哪一种。答案是第二种——先做 child，尽管它表面上更不利。

理由与 work stealing 的局部性有关：先做 child 时，队列深度与递归深度成正比，被偷的任务与偷取者所需的数据在内存上也更接近，而先做 continuation 会制造出大量远端任务。此外讲者提到 Cilk 采用贪心策略：让最后完成的那份工作去执行同步，而不是固定某一方等待，这让实现更复杂但性能更好。

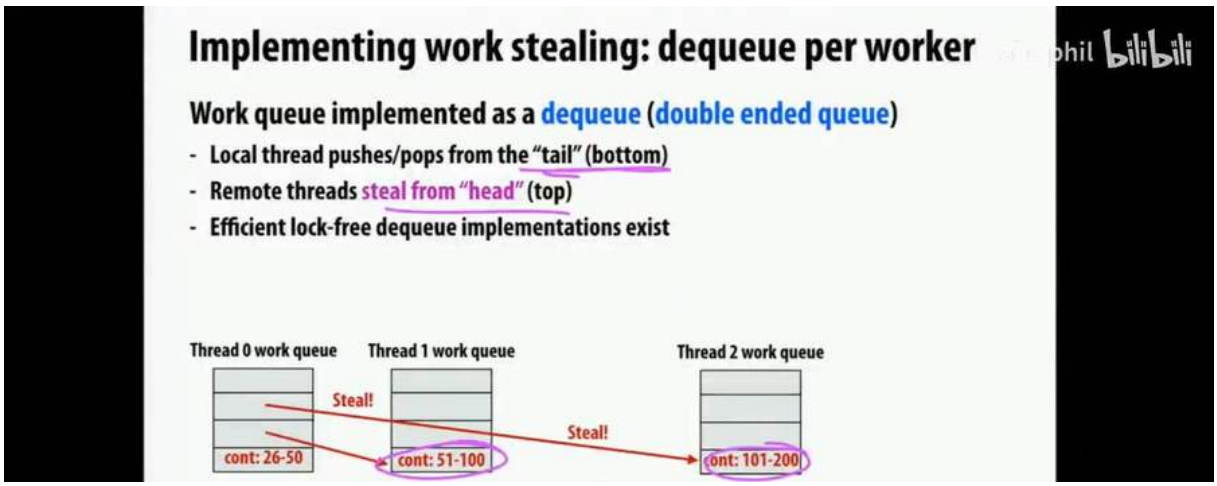

图 101: work stealing 的实现：每个 worker 一个双端队列[101]

[99] 视频画面时间区间：01:01:30–01:01:45。若为裁剪图，时间区间仍对应原视频画面。
[100] 视频画面时间区间：01:10:00–01:10:15。若为裁剪图，时间区间仍对应原视频画面。
[101] 视频画面时间区间：01:14:00–01:14:15。若为裁剪图，时间区间仍对应原视频画面。

双端队列让窃取与本地执行互不争用

work stealing 的工作队列通常实现为双端队列：本线程在自己一端压入与弹出（栈式，深度优先），窃取线程从另一端取走（队列式，取最老的任务）。两端操作不重叠，因此多数情况不需要加锁。

双端队列的两端分别服务两种角色

本线程从一端压入与弹出，行为像栈，因此走深度优先、数据局部性好；窃取者从另一端取走，行为像队列，因此拿到的是积压最久、通常也最大的任务。两端操作区间不重叠，多数情况下不需要加锁，这是 work stealing 低开销的根本原因。

这也解释了为什么先做 child（深度优先）能让窃取到的任务更大、更少。

本章小结

Cilk 用 spawn 与 sync 表达 fork-join，抽象不指定线程，运行时用线程池加分布式双端队列加 work stealing 实现。在 spawn 处它选择先执行 child，以获得深度优先的顺序与更好的局部性；表面上”队列永远不空”的 continuation-first 反而更差。

这一讲从”如何分工作”走到了”运行时如何调度”，全部结论都指向同一个结构：均衡与开销之间的取舍，以及用私有队列加按需窃取来同时争取两者。

5.8 总结与延伸

讲者的收尾很简短：我们看了 fork-join 并行，看到管理并行性上有一些有意思的权衡，目标是任务足够多但不能太多；也看了 Cilk 如何用 continuation、窃取与重新调度来做到这一点。

把全讲串起来：负载不均的代价按最慢者计算，20% 的差距就能浪费整个收尾阶段。静态分配零运行期开销，但依赖任务时间可预测。半静态在负载缓慢变化时用周期性重测取得折中。动态分配适应不可预测，但细粒度版本被锁竞争拖垮——那段等待本质上就是 Amdahl 定律中的串行时间。加大粒度能压住开销，但过大会重新引入不均，因此存在最优区间；理想策略是粒度随时间从大到小。分布式队列加 work stealing 同时取得均衡与局部性，三个设计要点是只按需窃取、随机选目标、一次偷多个。Cilk 则展示了如何在真实系统里落地这些选择，包括一个反直觉的结论：先执行 child。

本讲义在讲者材料之上做的压缩，有四点需要与讲者原话区分：

- 粒度是唯一同时影响均衡与开销的参数。加大粒度改善开销，加小粒度改善均衡，两者不可同时最优。所有调度器的默认值都在这条曲线上选点。
- work stealing 的局部性来自队列结构本身。私有队列加双端队列，使本线程与窃取者从两端操作，既避免争用，又让被偷的任务天然是较大的那批。
- Cilk 选择先做 child 是一次用局部性换表面均匀的取舍。它牺牲了”队列永远充实”的直觉优势，换取深度优先的顺序与近邻的数据。
- 收尾阶段的并行度衰减无法消除。任何分配策略最后都会只剩少量工作，改善的方向是让剩余任务更小，而不是追求全程满并行。

这堂课的边界也清楚。讲者反复表示队列中的工作彼此不必独立、通信与局部性的优化都会在后续课程展开。因此这里给出的是调度策略的选择框架，不含具体实现代码。

两个可以直接从本讲数据提出的下一步问题：其一，粒度既要足够小以均衡、又要足够大以省开销，那么在一台给定机器上，最优粒度是否与缓存容量相关？其二，work stealing 在收尾阶段会造成集中争用，这是否意味着应当让窃取本身也分级（先偷大块、再偷小块）？这两个问题都指向下一讲的通信与局部性。

Lecture 06 Recitation: Performance Tuning Tools

视频时长：1 小时 2 分 25 秒　来源：`https://www.bilibili.com/video/BV1uh4y1Q73E/`

6.1 先看清机器：/proc 这一手资料

这堂习题课由 Greg Kesden 讲授，内容是把前几讲的方法落到实际工具上。开场他就说明这是一堆与课程相关的杂项，主题围绕性能优化。

/proc 是什么

讲者先问有多少人看过 `/proc/cpuinfo`。他从历史讲起：过去想从内核取信息，必须做系统调用，而系统调用本身有固定开销。

现代操作系统提供 `/proc`，它是一个虚拟文件系统：不是真实文件系统，但看起来像。内核把大量状态导出，伪装成文件系统的形式，因为这种形式对人方便，对程序也方便——打开、读取、解析都容易。这样做同时省掉了每次都做系统调用的开销。

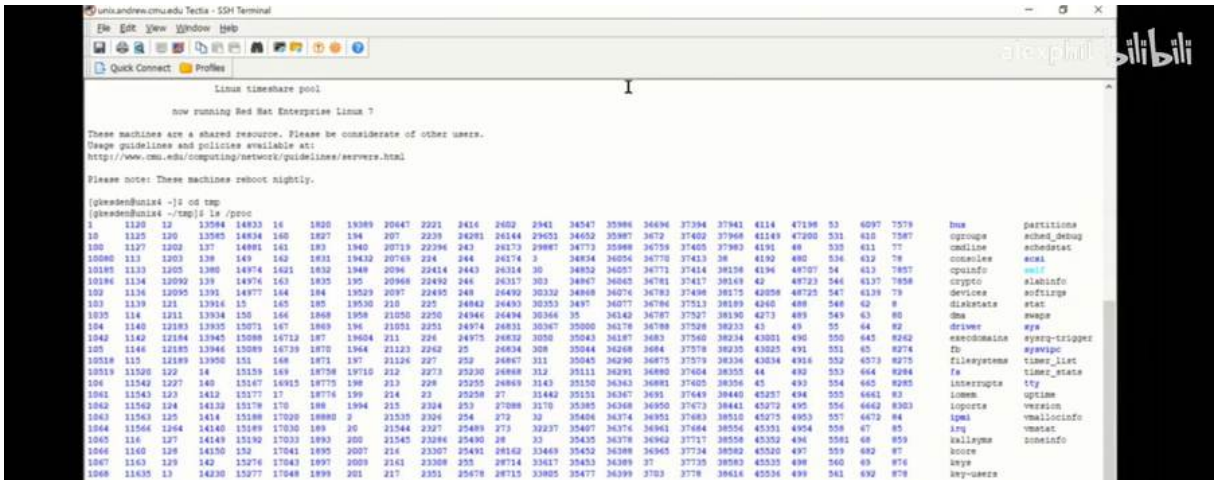

图 102: 在终端中浏览 /proc：内核状态以文件形式呈现[102]

进程目录与 cpuinfo

`/proc` 下一串以数字命名的目录代表进程，数字就是进程 ID。对某个编号做 `ls`，就能看到那个进程的各项信息。讲者现场演示时那个进程已经退出，于是换了一个。

真正与性能相关的是 `/proc/cpuinfo`。它暴露处理器数量、型号与各种属性。

[102] 视频画面时间区间：00:02:30–00:02:45。若为裁剪图，时间区间仍对应原视频画面。

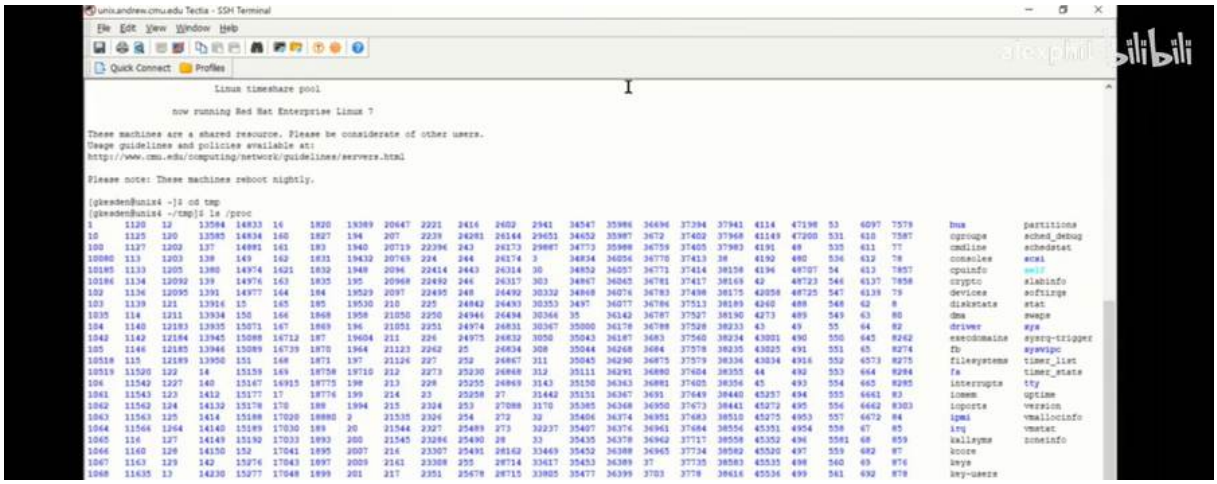

图 103: 列出 /proc 下以进程号命名的目录[103]

按逻辑处理器数规划并行度会高估算力

超线程共享同一核心的执行单元与缓存，两条逻辑线程的合计吞吐明显低于两个独立物理核。把 40 个逻辑处理器当作 40 份独立算力来切分工作，会得到过度乐观的预期，实测加速比通常远低于设想的倍数。

讲者现场展示了一台机器：报告 40 个处理器，而 siblings 是 20。他让学生猜这意味着什么。答案是有大量超线程——每核两条线程，于是 20 个物理核被操作系统看成 40 个。

处理器数与 siblings 数不一致就说明有超线程

cpuinfo 中 `processor` 的条目数是操作系统看到的逻辑处理器数，`siblings` 是同一物理封装内的逻辑处理器数。两者不等时，你拿到的核数要按物理核重新算。把超线程当成独立核心来规划并行度，会高估可用算力。

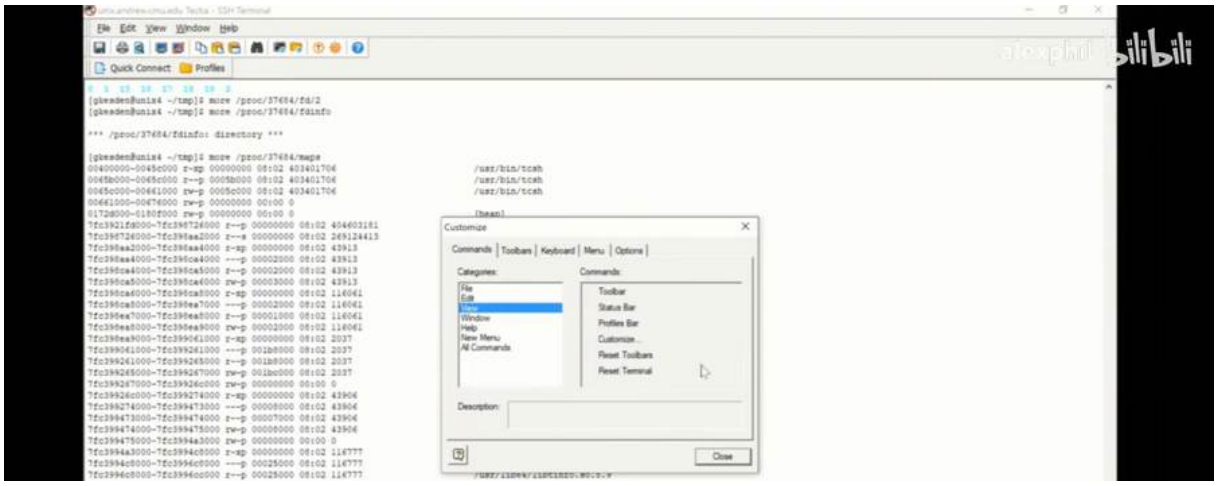

图 104: 查看 cpuinfo 的终端会话[104]

[103] 视频画面时间区间：00:03:00–00:03:15。若为裁剪图，时间区间仍对应原视频画面。

[104] 视频画面时间区间：00:05:00–00:05:15。若为裁剪图，时间区间仍对应原视频画面。

几个必须读对字段

讲者逐项解释容易误读的字段。

model name 通常标注处理器的最高频率。而 cpu MHz 是当前频率。现代处理器有降频省电机制，也有短时间的睿频模式，因此这两个数字常常不一致。看到 cpu MHz 偏低不代表处理器变慢了。

cache size 只显示最外层缓存，通常是所有核心共享的 L3。各核私有的 L1、L2 不在这一项里显示。

physical id 是插槽编号。多处理器系统里两个处理器各带若干核心时，这个字段能区分它们。

flags 列出处理器的能力，例如这门课一直在用的 AVX2 指令。

图 105: cpuinfo 关键字段：型号、当前频率、最外层缓存、siblings[105]

图 106: cpuinfo 中的 physical id 与 flags[106]

[105] 视频画面时间区间：00:11:40–00:11:55。若为裁剪图，时间区间仍对应原视频画面。

[106] 视频画面时间区间：00:13:00–00:13:15。若为裁剪图，时间区间仍对应原视频画面。

信息多不等于有用

讲者特意提醒：这里是一座宝库，但打开后很容易被海量字段带跑。做性能工作时应该带着具体问题来查——我要知道核数、当前频率、是否有 AVX2——而不是通读一遍。

还能快速查什么

讲者给出了一份”快速查”清单，其中包括内存带宽与功耗这类直接影响并行上限的量。他展示的一台机器标称约 76.8 GB/s 带宽、140 瓦功耗。

图 107：还能快速查到什么：内存带宽与功耗等参数[107]

本章小结

`/proc` 把内核信息做成文件，避免了反复系统调用的开销。读取时要抓住几个真正影响并行决策的字段：逻辑处理器数、siblings、当前频率、最外层缓存、插槽编号、指令集 flags。其中处理器数与 siblings 不一致，是判断超线程最直接的线索。

知道了机器长什么样，下一步是知道它为什么这样设计。讲者翻开了处理器架构手册。

6.2 读架构手册：功能单元与流水线

细节都在手册里

讲者推荐了两份资料：Agner Fog 的微架构文档，以及 2-13 教材第五章。他把这称为”细节里的魔鬼”——这些文档里有关功能单元、延迟、端口的具体数字。

[107] 视频画面时间区间：00:15:00–00:15:15。若为裁剪图，时间区间仍对应原视频画面。

图 108: 细节藏在架构手册里：推荐的参考资料[108]

流水线的心智模型

讲者重申了这门课使用的处理器模型：像一条装配线。指令进来、被译码、被派发，在没有数据依赖或其他结构性冒险时可以乱序执行，最后进入提交单元，从那里按序把结果落实到真实状态。

还有一个 2-13 课程里没强调的事实：寄存器在处理器内部有很多份副本，正是这些副本让并行得以发生。开启超线程后，处理器不只是并行执行一条指令流，它能同时跟踪多条指令流。

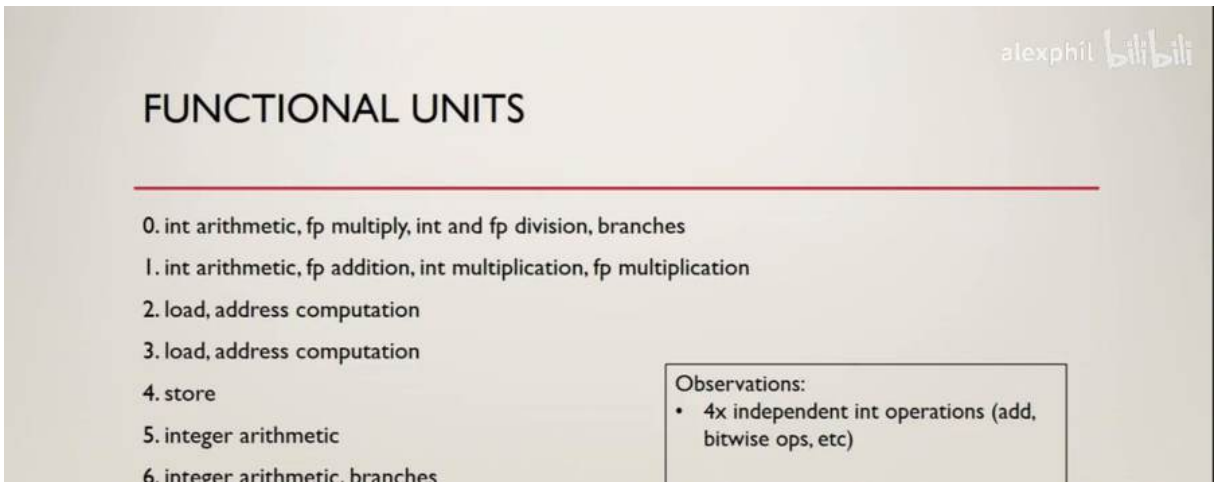

图 109: 处理器的功能单元及其分工[109]

延迟、发射时间与容量

讲者用一张表把三个常被混淆的量分开。

延迟是这条指令从开始到结果可用所需的时间。发射时间是相邻两条同类指令之间至少要隔多久，也就是吞吐的倒数。容量是有多少个功能单元能支持这类操作。

[108] 视频画面时间区间：00:18:00–00:18:15。若为裁剪图，时间区间仍对应原视频画面。
[109] 视频画面时间区间：00:21:30–00:21:45。若为裁剪图，时间区间仍对应原视频画面。

他指出了一个过去的简化不再成立的地方：早年间说”能做四个加法”就足够描述，而现在部分功能单元承担一个操作篮子，能做一个或另一个但不能同时做。因此简单地相加容量会高估实际能力。

若只考虑单一功能单元，稳定状态下的吞吐由发射时间决定：

$$\text{Throughput} = \frac{\text{单元数量}}{\text{发射时间}}$$

- 单元数量：能执行该操作的功能单元个数；
- 发射时间：相邻两条同类指令的最小间隔，单位为周期；
- 结果为单位周期内可完成的该操作次数。

注意分母是发射时间而不是延迟：延迟决定单条依赖链的长度，发射时间才决定吞吐。

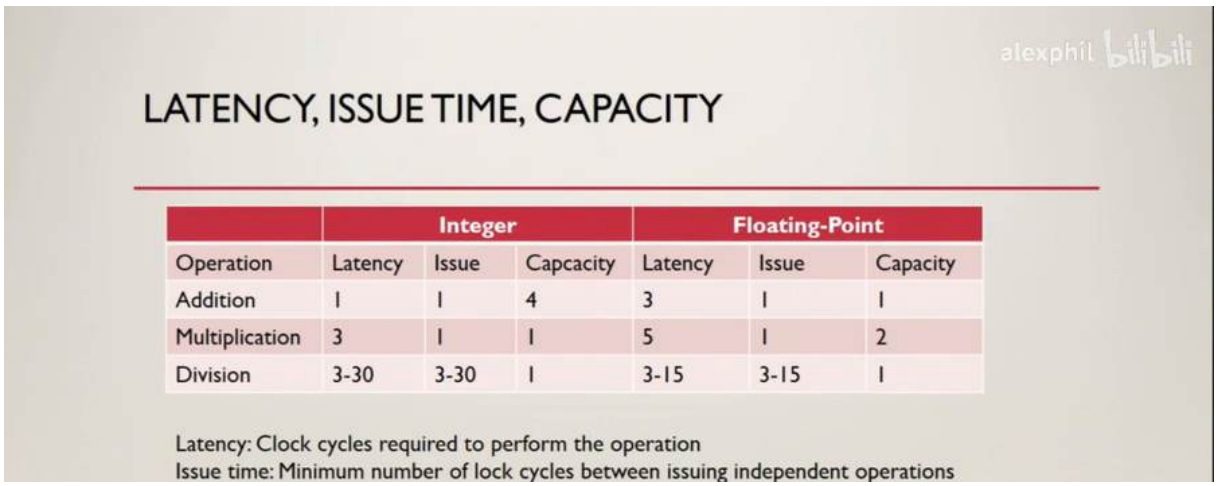

图 110：延迟、发射时间与容量：三个常被混淆的量[110]

三个量的用途各不相同

延迟决定依赖链的长度，影响单条关键路径的耗时；发射时间决定单位时间能塞进多少条同类指令，影响吞吐；容量决定同类型操作的并行上限。优化时先判断瓶颈属于哪一种，三者的改进手段完全不同。

本章小结

架构手册给出功能单元的真实参数，流水线模型解释了乱序执行与按序提交。延迟、发射时间、容量是三个必须区分的量；现代处理器中功能单元能力重叠，不能简单相加估计吞吐。

有了机器的参数与模型，就可以回到程序本身。下一章用一段具体代码演示优化从哪一步开始。

6.3 优化的起点：找到内层循环

一个贯穿全课的代码例子

讲者用的是计算 sin 的泰勒级数近似。这段代码在前几讲出现过，因为它简单、能放在一页幻灯片上。

[110] 视频画面时间区间：00:25:00–00:25:15。若为裁剪图，时间区间仍对应原视频画面。

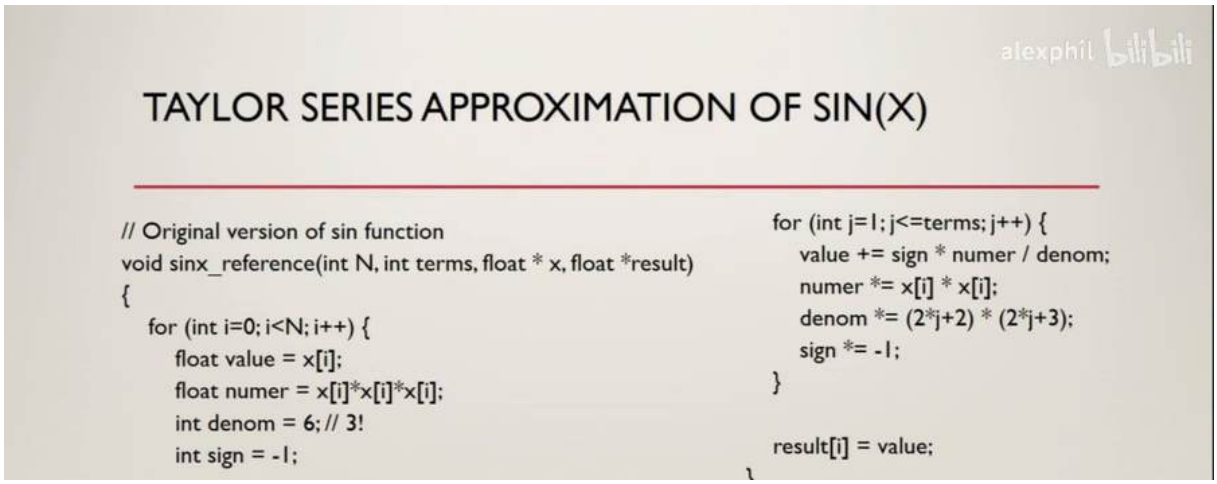

图 111: sin(x) 泰勒级数近似的完整代码[111]

Amdahl 的饼图视角

讲者问学生 Amdahl 定律是什么，然后给出他更喜欢的表述方式：把程序的活动想成一张饼图，你在优化其中某些切片。如果挑中的切片很小，你能得到的改进也只能很小。

若某个活动只占全部时间的一成，你把它优化到零，也只省下一成时间。由此得到一条排序原则：在常见活动上的小优化，收益往往大于在罕见活动上的大优化。写成式子：

$$S_{\text{total}} = \frac{1}{(1-f) + \dfrac{f}{k}}$$

- S_{total}：整段程序获得的加速比；
- f：被优化部分占总时间的比例；
- k：该部分自身被加速的倍数。

当 $f = 0.9$ 而 $k \to \infty$ 时，S_{total} 的上界只有 10；而当 $f = 0.005$ 时，无论 k 多大，整体几乎不变。

Amdahl 定律的通用形式

这条定律原本用于并行化的加速上界，但它对纯串行程序的优化同样成立：某段代码占运行时间比例 f，则它对总体加速的贡献不超过 $1/((1-f)+f/k)$，其中 k 是该段被加速的倍数。因此优化的顺序应当按占比从大到小排，而不是按改起来是否顺手排。

[111] 视频画面时间区间：00:27:00–00:27:15。若为裁剪图，时间区间仍对应原视频画面。

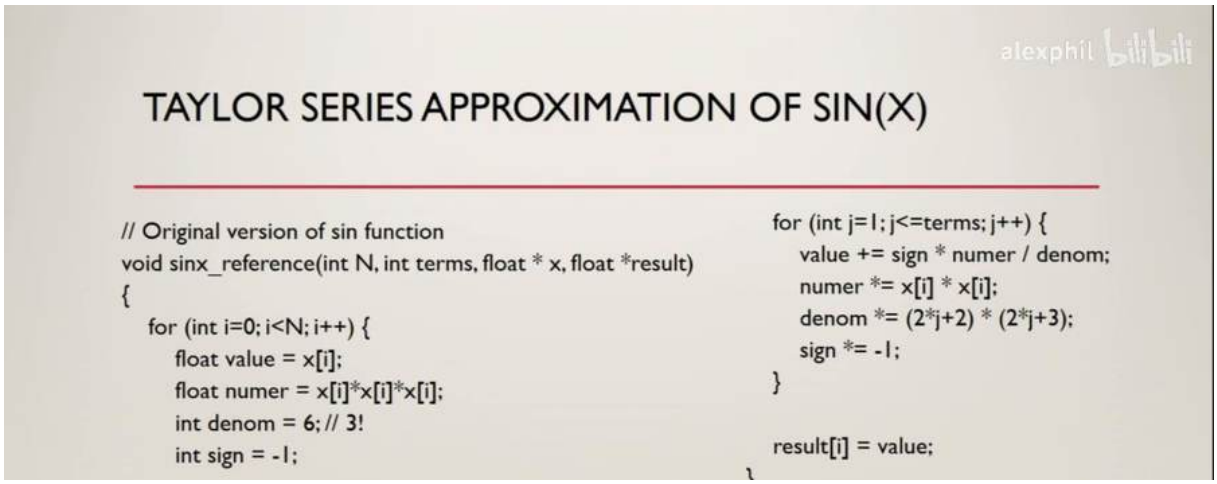

图 112: 定位内层循环：真正被反复执行的部分[112]

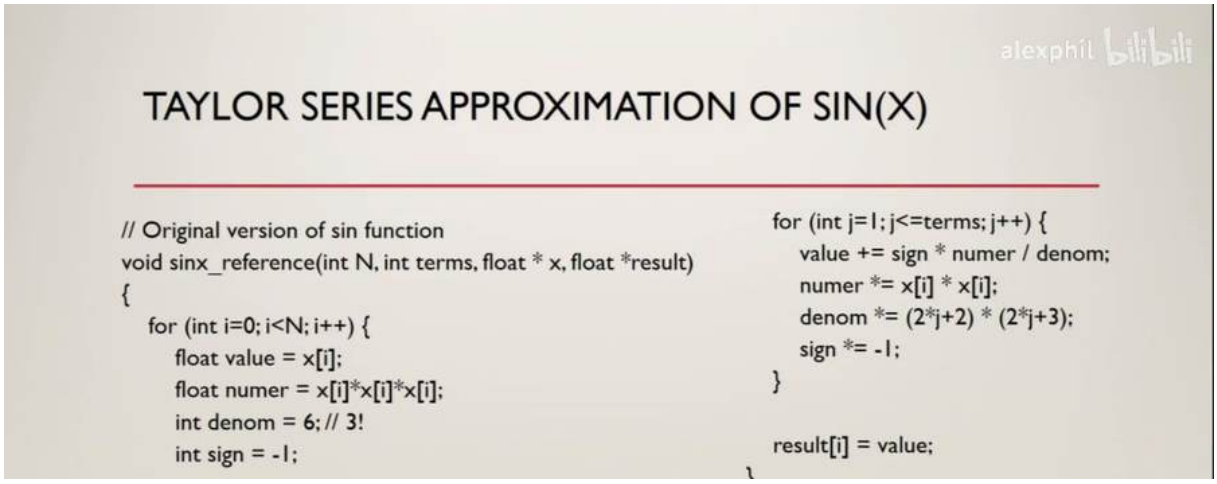

图 113: Amdahl 的饼图视角：优化的收益上限由占比决定[113]

内层循环为什么是首要目标

这段代码的循环是嵌套的：外层执行 n 次，内层执行 terms 次。因此内层循环体被执行 $n \times \text{terms}$ 次，这就是所谓的内层工作循环。

讲者的对比很有力：如果把开头的三条指令彻底消掉，整个程序只快了 0.5%，因为那三条不常执行。反过来，在内层循环里省一件事，这个节省会被执行次数放大。

内层循环体的执行次数是两层循环次数之积：

$$C_{\text{inner}} = n \times \text{terms}$$

- C_{inner}：内层循环体被执行的次数；
- n：外层循环次数，即输出元素个数；
- terms：泰勒级数保留的项数。

[112] 视频画面时间区间：00:28:30–00:28:45。若为裁剪图，时间区间仍对应原视频画面。
[113] 视频画面时间区间：00:28:00–00:28:15。若为裁剪图，时间区间仍对应原视频画面。

这解释了为什么同样省一条指令，放在内层循环与放在外层开头的效果相差几个数量级。

先测基线，不要凭直觉优化

讲者引用了上一讲 Todd 的说法：拿到初始测量非常重要。很多人会在脑子推演” 这里贵、那里也贵”，然后还没把代码写清楚就开始优化。他明确说不要这样。

他们的基线是每元素约 7 纳秒。这个数字本身没有绝对意义，它是后面每次改动的参照点。

每元素成本由总时间除以元素数得到：

$$T_{\text{elem}} = \frac{T_{\text{total}}}{N}$$

- T_{elem}：分摊到每个元素的时间，本例约 7 纳秒；
- T_{total}：一次完整运行的总耗时；
- N：处理的元素总数。

用每元素成本而不是总时间做比较，可以在元素数变化时仍保持可比。

图 114: 基准测量：先拿到初始数字再优化[114]

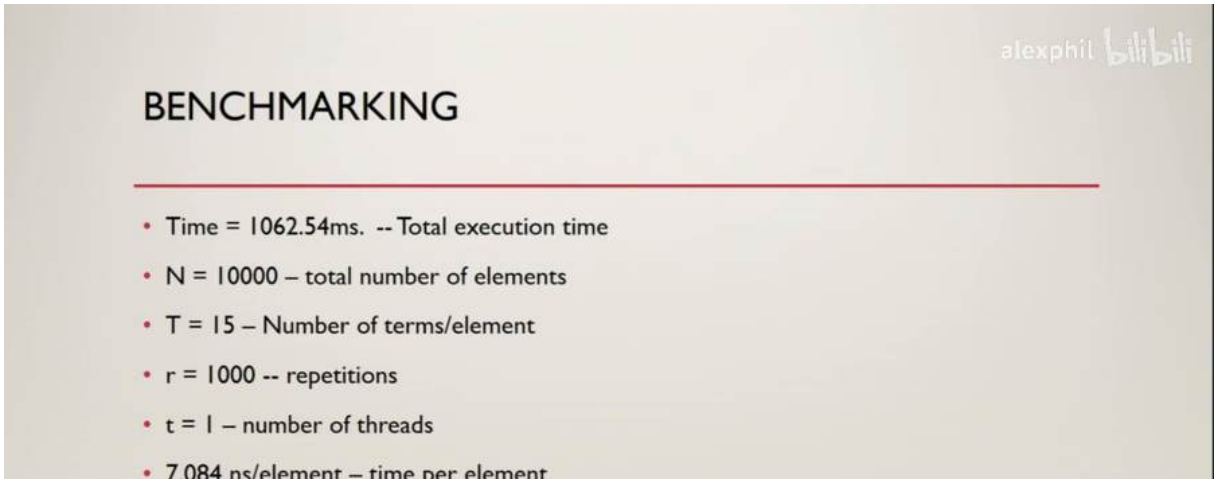

图 115: 基准测量的细节：总时间、元素数与每元素成本[115]

[114] 视频画面时间区间：00:31:00–00:31:15。若为裁剪图，时间区间仍对应原视频画面。

没有基线就无法判断改动是否有效

一次改动带来百分之几的差异，可能与测量噪声同量级。没有稳定的基线，你无法区分真实改进与随机波动，也无法在多次迭代中追踪哪一步真正起了作用。基线必须在优化开始之前建立，并且用同样的方式重复测量。

本章小结

优化从内层工作循环开始，因为它的执行次数最多，任何节省都会被放大。判断依据是 Amdahl 的饼图视角：占比决定收益上限。动手前必须先建立基线（本例为每元素约 7 纳秒），否则无法判断改动是否真的有效。

定位了目标之后，接下来看两个方向：人工改写代码，以及依赖编译器。

6.4 手工改造内层循环

从最贵的操作入手

讲者审视内层循环，找出最贵的部分：乘法与除法，其中除法更贵。因此初期目标是减少这两类操作的次数。

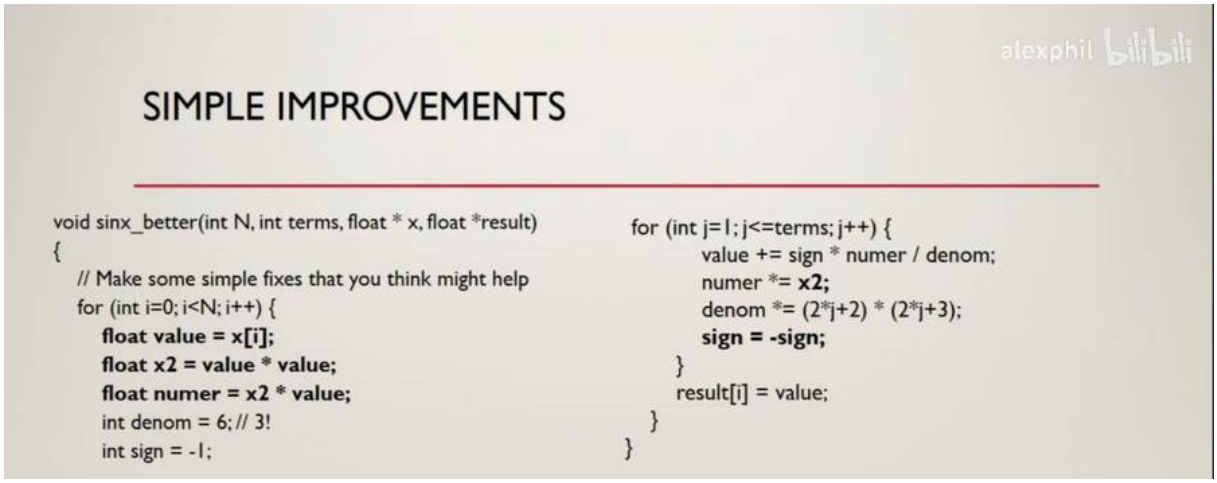

图 116: 简单改进：预先计算阶乘倒数，把除法转为乘法[116]

预先计算把循环内的工作搬到循环外

在循环外预先算好阶乘倒数，等于把每轮都要重复的计算变成一次性的表。这类改写的适用范围很广：任何只依赖循环不变量的计算都可以移出去。代价是多一份存储，收益是内层循环少一整类昂贵操作。

最直接的一步是预先算好各阶乘的倒数，避免在循环里反复做除法。这类改写不需要改变算法结构，属于低风险高回报的改动。

讲者还提到另一类改写的效果量级：把表达式重组之后，某些情况下能拿到 180

[115] 视频画面时间区间：00:34:00–00:34:15。若为裁剪图，时间区间仍对应原视频画面。

[116] 视频画面时间区间：00:38:30–00:38:45。若为裁剪图，时间区间仍对应原视频画面。

循环展开

下一步是循环展开：减少循环条件判断的次数，增加每次迭代内的工作量，从而避开分支代价。

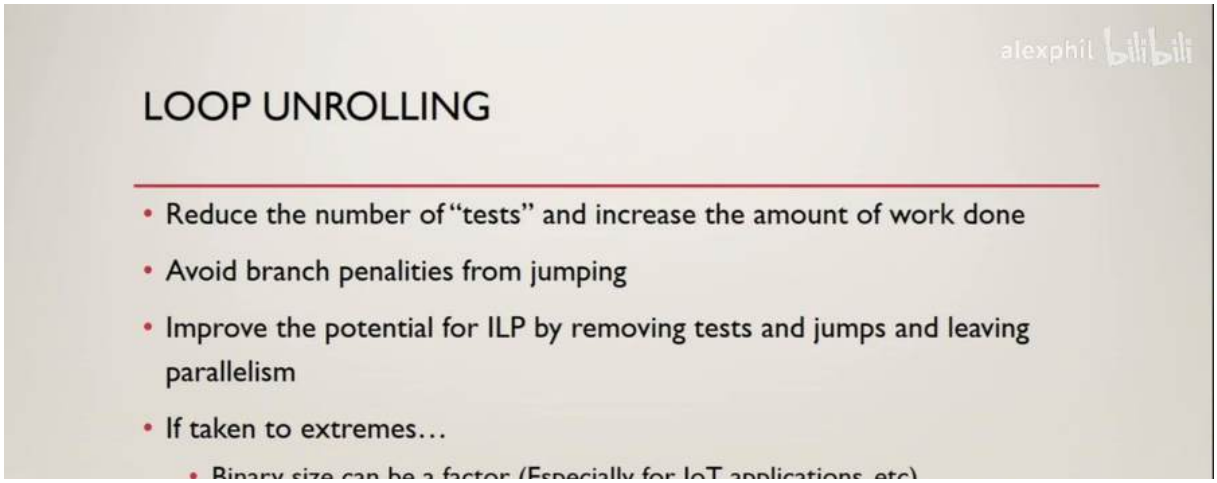

图 117: 循环展开：减少判断次数并增加单次迭代的工作量[117]

讲者也指出了代价：展开后需要更多变量来保存中间结果，寄存器压力上升。寄存器不够用时就会溢出到内存，反而变慢。

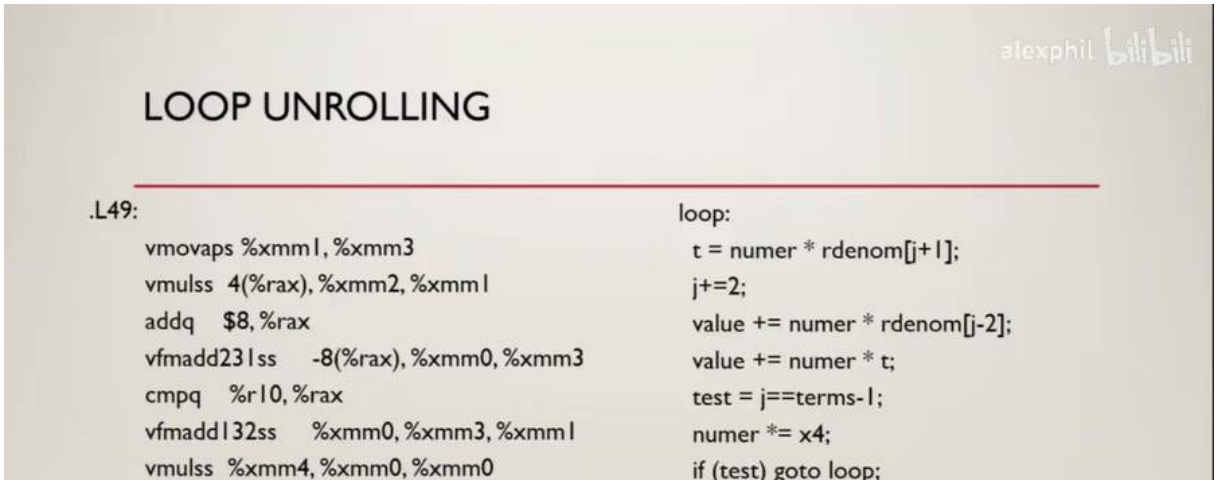

图 118: 展开后的汇编：更多寄存器被占用[118]

本章小结

手工改造内层循环的两个方向是减少昂贵操作与循环展开。前者把除法转成乘法，后者减少分支判断，但展开会增加寄存器压力，溢出到内存反而更慢。这些改动是否真的生效，需要另一种手段来确认。

源码看不出编译器做了什么，因此需要直接阅读机器生成的指令。

[117] 视频画面时间区间：00:41:00–00:41:15。若为裁剪图，时间区间仍对应原视频画面。

[118] 视频画面时间区间：00:46:00–00:46:15。若为裁剪图，时间区间仍对应原视频画面。

6.5 读汇编：确认编译器做了什么

从汇编里看真实指令

讲者展示了内层循环的汇编，其中出现了 `vfmadd231ss` 这类融合乘加指令。

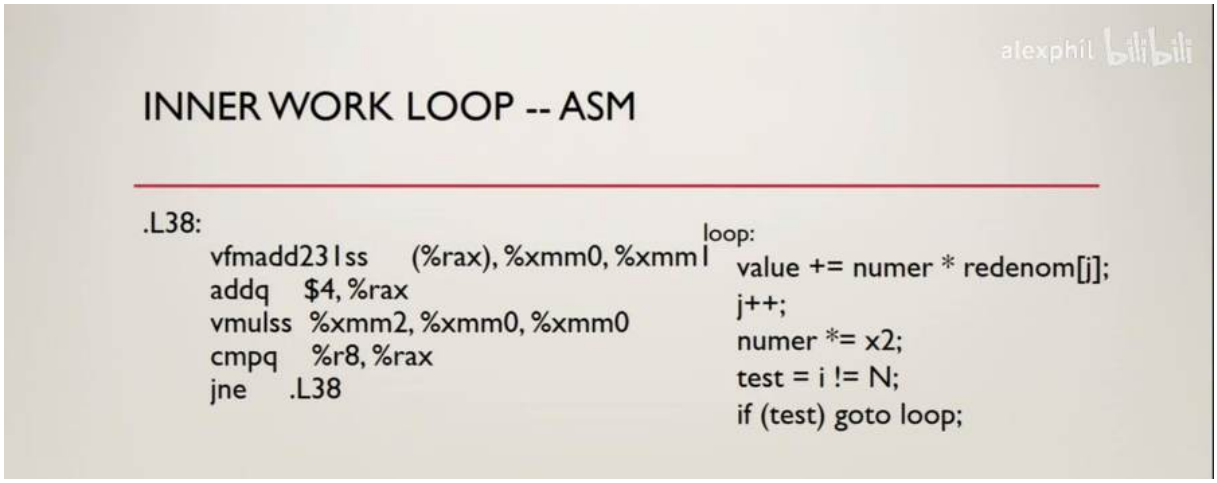

图 119: 内层工作循环的汇编代码：融合乘加等指令[119]

汇编是验证优化是否生效的唯一直接证据

阅读源码无法知道编译器做了什么。要确认循环被展开、乘加被融合、或者代码没有被向量化，只能看生成的汇编。这也是判断”我的改动被编译器接受了”与”编译器另有打算”的分界线。

三个人、三个加速比

讲者分享了这段代码的实战对照。他自己第一版优化做到 25 倍加速；看到 Randolph 去年这堂课的笔记写着 80 倍；于是他回去重新努力，做到 40 倍；最后还是忍不住看了笔记。

同一个优化目标，不同人的结果可以差数倍

25 倍与 80 倍的差距不来自编译器版本，而来自对瓶颈的理解深度。这说明优化不是套用技巧清单，而是逐步建立对这段代码在硬件上如何执行的准确认识。每一次对照更高水平的结果，都能暴露自己模型里的错误假设。

本章小结

手工改造的两大方向是减少昂贵操作（除法转乘法）与循环展开（减少判断），后者受寄存器容量限制。看汇编是确认这些改动是否生效的唯一直接方法。同一段代码的优化结果可以差数倍，差别在于对瓶颈的理解。

手工手段的收益有上限。真正数量级的提升来自另一种方式：让处理器用一条指令处理多个数据。

[119] 视频画面时间区间：00:39:30–00:39:45。若为裁剪图，时间区间仍对应原视频画面。

6.6 向量化与基准测量的纪律

向量化带来数量级差异

讲者给出了 ISPC 版本与未向量化版本的直接对照：ISPC 代码每元素 0.63 纳秒，未向量化版本 7.16 纳秒。

这大约是十倍的差距。而这个版本恰恰是增量最小的方向——讲者说向量化很简单，只换来 5.40 倍加速；而传统的手工优化做得很辛苦，换来的是 15 倍量级。

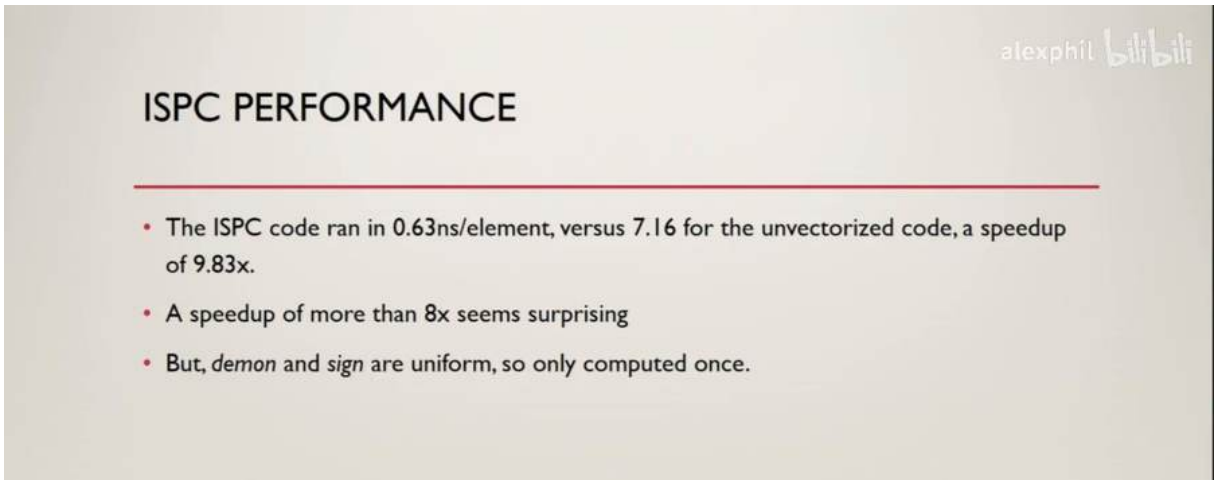

图 120: ISPC 版本与未向量化版本的性能对照[120]

向量化的投入产出比通常优于手工调优

单条指令处理多个数据，等于在不增加指令数的情况下提高吞吐。对结构规整的内层循环，重写成可向量化的形式往往只需少量改动，却能得到数倍提升。相比之下，手工展开、手工调度寄存器需要大量试错，收益还受寄存器数量限制。

阻碍向量化的常见因素

讲者也提到了让编译器无法向量化的因素：循环体内存在条件分支，或者编译器无法证明两个指针指向不重叠的内存。后者可以用语言特性（如 restrict 限定）来表达。

[120] 视频画面时间区间：00:57:00–00:57:15。若为裁剪图，时间区间仍对应原视频画面。

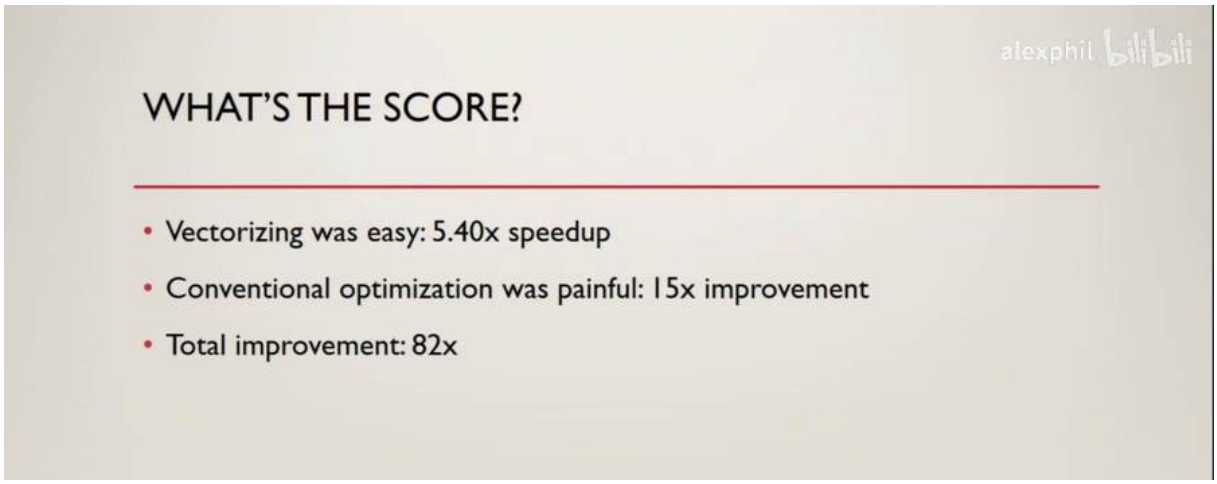

图 121：用语言特性帮助编译器消除顾虑，从而允许向量化[121]

编译器向量化的三道门槛

第一，迭代必须互相独立，不能有跨迭代依赖；第二，循环体内最好没有数据相关的分支，否则只能靠掩码执行、效率减半；第三，编译器必须能证明写目标与读来源不重叠。三者缺一，向量化就会失败或收益打折。

本章小结

向量化用一条指令处理多个数据，是投入产出比最高的方向：本例中 ISPC 版本每元素 0.63 纳秒，而未向量化版本为 7.16 纳秒，相差约十倍。它的三道门槛是迭代独立、无数据相关分支、指针不重叠。门槛不满足时编译器会放弃向量化，或者只能靠掩码执行而效率减半。

改动做完之后还要能证明它有效。这就把问题从改写代码转向了测量本身。

6.7 基准测量的纪律

测量本身要可靠

讲者最后强调测量本身要可靠。基准程序要避免被优化掉——如果计算结果没人用，编译器可能把整段计算删除，测出来的是零。

时间测量要用足够精度的时钟或性能计数器。多次运行取稳定值，并注意第一次运行的偏差（冷缓存、页错误、频率尚未升上去）。

个位数百分比的改善不可靠

讲者提到某类改动只带来 2

本章小结

向量化用一条指令处理多个数据，是投入产出比最高的方向，前提是迭代独立、无分支、指针不重叠。基准测量本身也需要纪律：避免计算被优化掉、用足够精度计时、多次运行、只相信显著大于噪声的改善。

[121] 视频画面时间区间：01:00:00–01:00:15。若为裁剪图，时间区间仍对应原视频画面。

到这里工具与方法都已交代。最后是讲者关于应有的态度的一句提醒。

6.8 总结与延伸

讲者的收尾很朴素：把风格做到最好，但不要在这上面花三周；然后转向真正的工作。

这一讲没有提出新的并行理论，它提供的是把前面几讲落到机器上的通道。`/proc` 让你知道手上是多少物理核、有没有超线程、频率是多少、支持哪些指令集。架构手册告诉你功能单元的延迟、发射时间与容量，这三个量决定了瓶颈性质。优化从内层循环开始，判据是 Amdahl 的饼图视角——占比决定收益上限。手工手段包括减少除法、循环展开，受寄存器容量约束，效果必须靠看汇编确认。向量化通常性价比最高，但有迭代独立、无分支、指针不重叠三道门槛。而所有这一切的前提是可靠的基准测量。

本讲义在讲者材料之上做的压缩，有三点需要与讲者原话区分：

- 这一讲的工具有明确的移植边界。/proc 是 Linux 专有；在 Windows 或 macOS 上没有对应物，那里要用系统 API 或平台专用工具。它不是通用技能，而是 Linux 环境下的必备技能。
- 加速比的对照（25 倍、40 倍、80 倍）说明的是理解深度而非技巧数量。三人的差异来自对同一段代码瓶颈认识的精确程度，这提示优化工作的重心应当放在建立准确的性能模型上。
- 优化存在应当停手的时刻。讲者明确说不要在代码风格上花三周。当边际收益低于测量噪声、或低于继续投入的机会成本时，继续优化就是浪费。

这堂课的边界在于它是习题课性质，内容按” 学生实际会用到什么” 选取，不追求体系完整。因此本文给出的是工具清单与优先级顺序，不含完整的微架构分析。

两个可以直接从本讲数据提出的下一步问题：其一，向量化带来十倍级提升，而手工优化只有数倍，那么在什么条件下手工优化仍然值得做——是否只在无法向量化的不规则计算上？其二，寄存器压力的临界点在哪里：循环展开几次之后，溢出到内存的代价会吞掉减少分支的收益？这两个问题都需要下一讲要处理的局部性与缓存层次来回答。

Lecture 07 GPU Architecture & CUDA

视频时长：1 小时 20 分 8 秒 来源：https://www.bilibili.com/video/BV1Hr4y1Z7tn/

7.1 GPU：从图形卡到超级计算机的心脏

讲者开场给出一个判断：GPU 是过去几十年高性能计算领域最令人兴奋的进展之一，因为它把性能可能性整体抬高了。今天几乎所有主要超级计算机的主体，都是一堆 GPU 拼起来。

这份算力并非自动可得。讲者点明本讲的双重主题：在某一层抽象上，你甚至不需要理解硬件就能用起来；但要拿到真正的性能，你必须知道下面发生了什么。

基本结构

回顾前几讲的结构：一颗很大的芯片上有许多处理器。每个处理器在很粗略的层面像一个常规 CPU——能通过 SIMD 指令支持多个 ALU，也能像 Intel 所谓超线程（更恰当的叫法是同时多线程）那样支持多条线程。

NVIDIA 把这些单元称作 SM（streaming multiprocessor，流式多处理器）。讲者展示的图上有 16 个，那是稍早的型号；课程用的 GHC 机器上有 20 个。

它们连接的内存，在内存世界里相对很小，但带宽相当可观。

整卡的原始 ALU 总量可以这样估：

$$N_{\text{ALU}} = N_{\text{SM}} \times N_{\text{ALU per SM}}$$

- N_{SM}：芯片上 SM 的数量，课程机器为 20；
- $N_{\text{ALU per SM}}$：每个 SM 的 ALU 数，课程 GPU 为 32；
- N_{ALU}：整卡可同时执行的算术操作数上限。

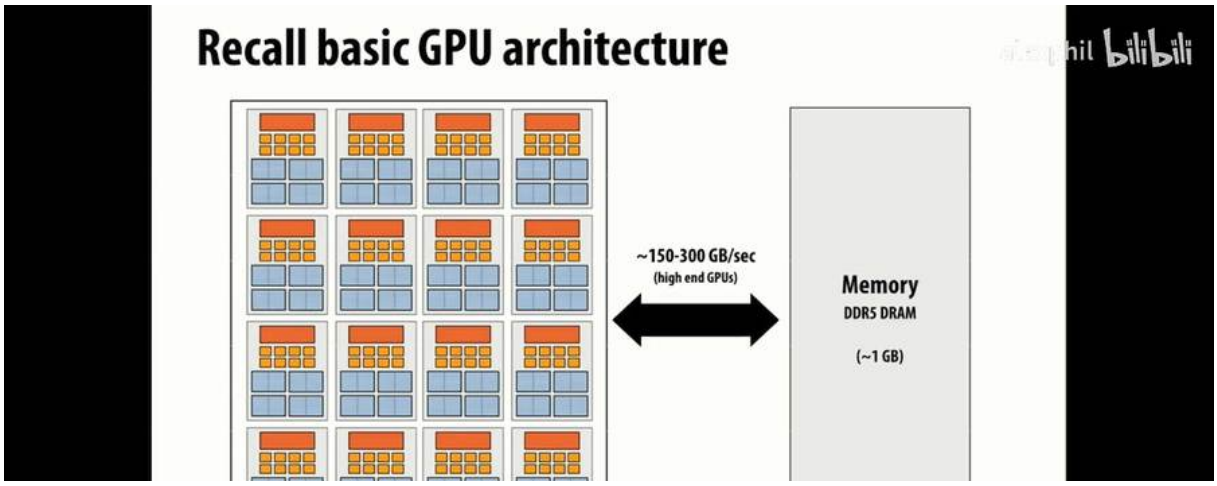

图 122: 基本 GPU 架构：多个 SM 共享高带宽内存[122]

[122]视频画面时间区间：00:02:00–00:02:15。若为裁剪图，时间区间仍对应原视频画面。

历史：为三维游戏而生

讲者从历史讲起。GPU 最初的设计目的是加速三维游戏这类应用。

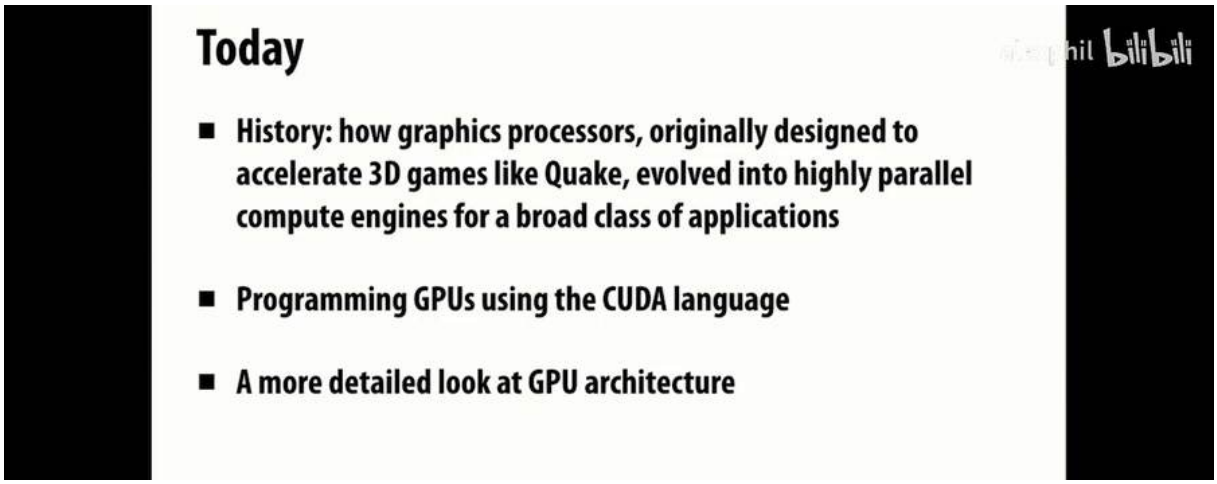

图 123: GPU 历史：图形处理器原本被设计来加速三维应用[123]

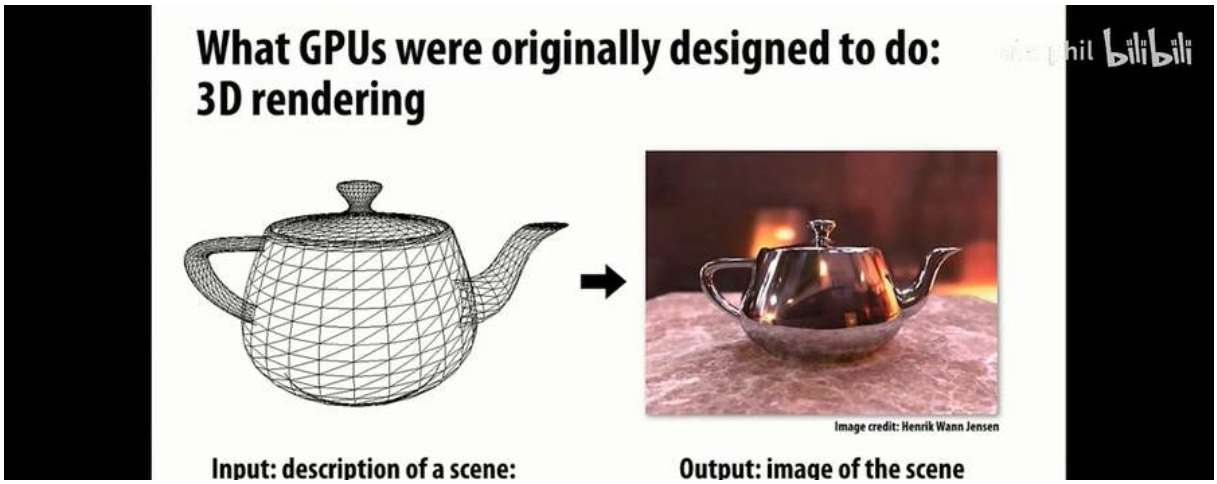

图 124: GPU 最初的用途：三维渲染[124]

实时渲染的帧率预算决定了一切

每秒 30 帧意味着每帧只有约 33 毫秒，其中还要扣掉物理模拟、输入处理与合成。这个固定预算解释了 GPU 为何偏爱近似手段（如纹理映射）而不做精确的物理建模：在预算内给出足够真实的观感，比算出精确的光照更重要。

至今这仍是它的主要用途：在高端的 GPU 上实时（约每秒 30 帧）渲染画面。

[123] 视频画面时间区间：00:01:30–00:01:45。若为裁剪图，时间区间仍对应原视频画面。
[124] 视频画面时间区间：00:03:30–00:03:45。若为裁剪图，时间区间仍对应原视频画面。

图 125: GPU 至今仍在做的事：高端 GPU 上的实时渲染[125]

追求真实感是一条指数级变难的路。讲者用一张风景图逐项举例：远处的景物比近处的略微模糊，这是大气效果；山间那些丝缕状的云，需要先跑一个扩散模型再做渲染。光照、光源位置、反射，每增加一层真实感就增加一层计算。若再加一个在草丛中跑动的男孩，还要渲染符合人体运动规律的姿态；若起风，还得让每根草摆动。

这些需求不只来自游戏，也来自建筑可视化——客户想绕着尚未建成的楼房从各个角度观察，还想看到云的倒影落在那面玻璃幕墙上。每一步都在增加复杂度。

本章小结

GPU 的算力来自大量 SM 的堆积与高带宽内存。它原本为实时三维渲染而生，真实感需求的指数级增长持续把它推向更高算力。理解这一点才能明白后面 CUDA 为何采用那样的抽象——它的形状是从图形管线继承来的。

7.2 图形管线：天然的数据并行

实体与图元

图形学要处理的对象是表面。讲者说，实时图形用三角网格表示表面：给定每个顶点在 x、y、z 上的三个坐标，每三个顶点组成一个三角形，一系列三角形构成物体的外观。

[125] 视频画面时间区间：00:04:30–00:04:45。若为裁剪图，时间区间仍对应原视频画面。

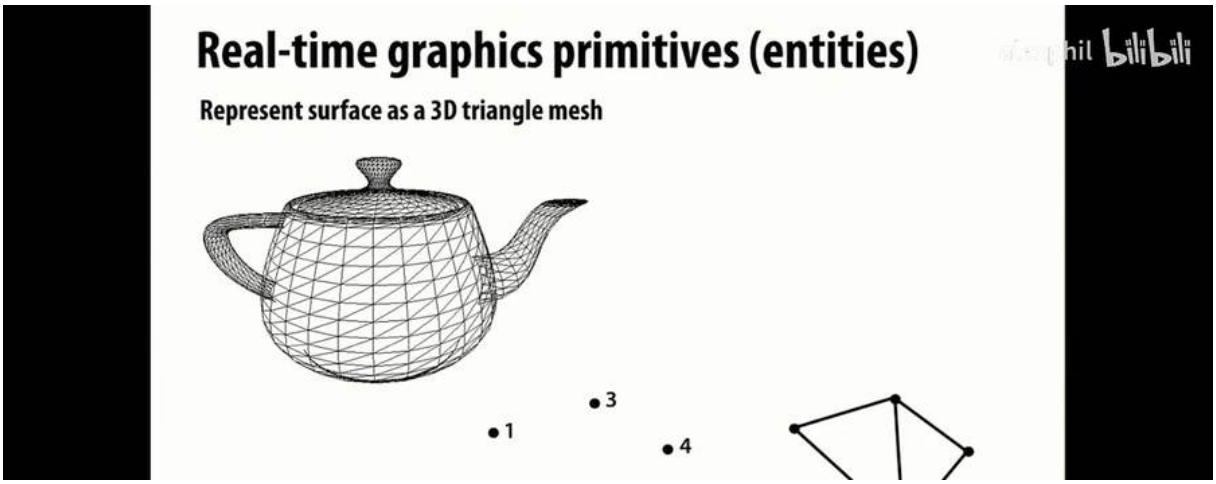

图 126: 实时图形的实体：用三维三角网格表示表面[126]

从三维到二维的管线

渲染一张图的过程可以拆成几步。

输入是顶点列表，它们存在于三维空间中。接下来要把这些三角形投影到二维屏幕上，如同从一个观察者或摄像机的视角看过去。这个映射取决于观察者的角度与位置。

投影之后得到一系列图元，在此例中是被压平到画面上的三角形。再往下，因为要转成离散像素，每个三角形内部要划分成小方格，并为每一格赋予颜色值。最后完成着色。

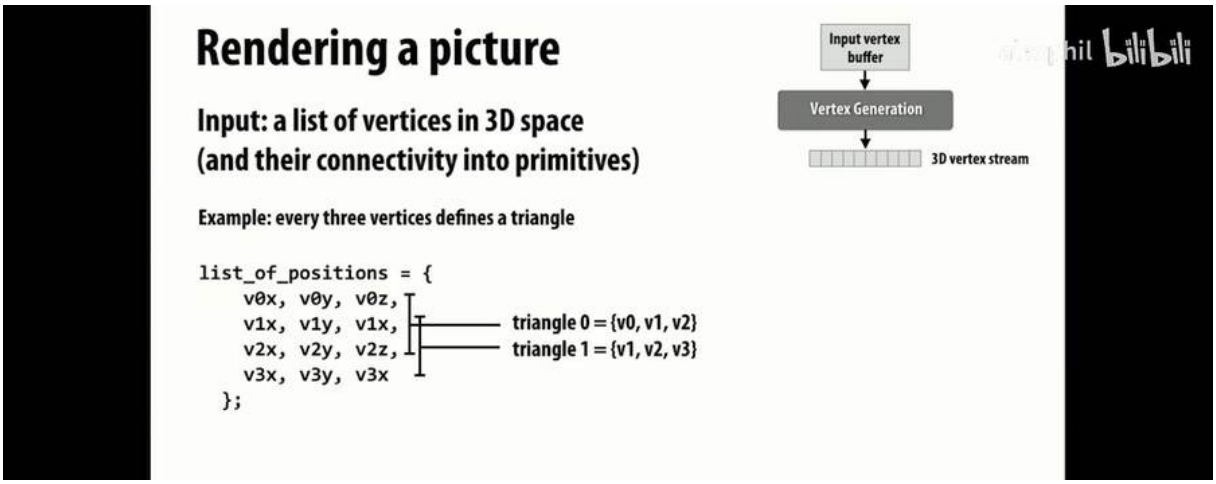

图 127: 渲染一张图：从顶点缓冲经投影到像素的完整管线[127]

图形管线的每一步都是天然的数据并行

处理顶点时，每个顶点与其他顶点无关；处理三角形时，遮挡计算只依赖各自的深度关系；处理像素时，每个小方格独立着色。对象在变（顶点、三角形、像素），操作也在变（投影、遮挡、着色），但”一堆东西各自独立地被处理”这个结构始终不变。GPU 的硬件形态正是为这个结构设计的。

[126] 视频画面时间区间：00:08:00–00:08:15。若为裁剪图，时间区间仍对应原视频画面。

[127] 视频画面时间区间：00:10:00–00:10:15。若为裁剪图，时间区间仍对应原视频画面。

片元处理与材质复杂度

讲者接着讨论片元处理，也就是逐像素计算光照。他用一系列球体展示材质的差异：大理石的光不只从表面反射，还会进入表面内部再折回，同时具有半透明性，薄的地方会透光；金属则有镜像效果，理论上要反向追踪被反射的一切；皮肤更难，皮革又要再加一层复杂度。

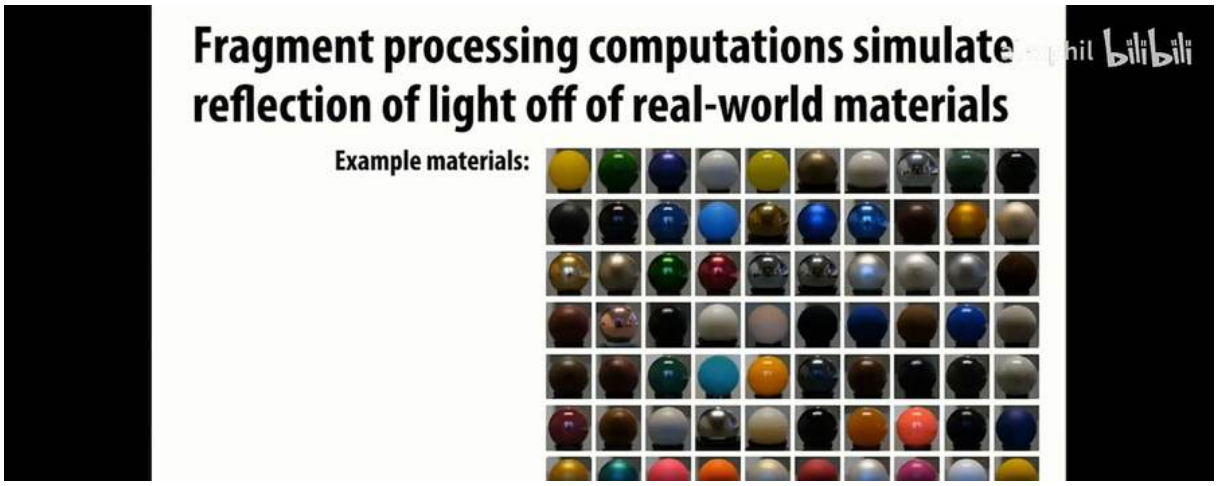

图 128: 片元处理：模拟光线在真实材质上的反射[128]

图 129: 现实世界材质与光源的极大多样性[129]

也有取巧的办法。桌布有规则的图案，于是可以用纹理映射：把一张平铺图案投影到物体表面。这以极低成本伪造出复杂图案的观感。

着色器：可编程的管线阶段

讲者由此得出关键结论：不应把整条管线都固化在硬件里。每次增加新特性，你可能都想用软件来描述要执行什么处理。

于是出现了着色器。它是在片元生成之后运行的一小段代码，例如给远处像素调整颜色，模拟光线穿过的扩散效果。讲者强调这是一种相对局部的处理——不必真的模拟光的物理。

[128] 视频画面时间区间：00:13:30–00:13:45。若为裁剪图，时间区间仍对应原视频画面。

[129] 视频画面时间区间：00:15:30–00:15:45。若为裁剪图，时间区间仍对应原视频画面。

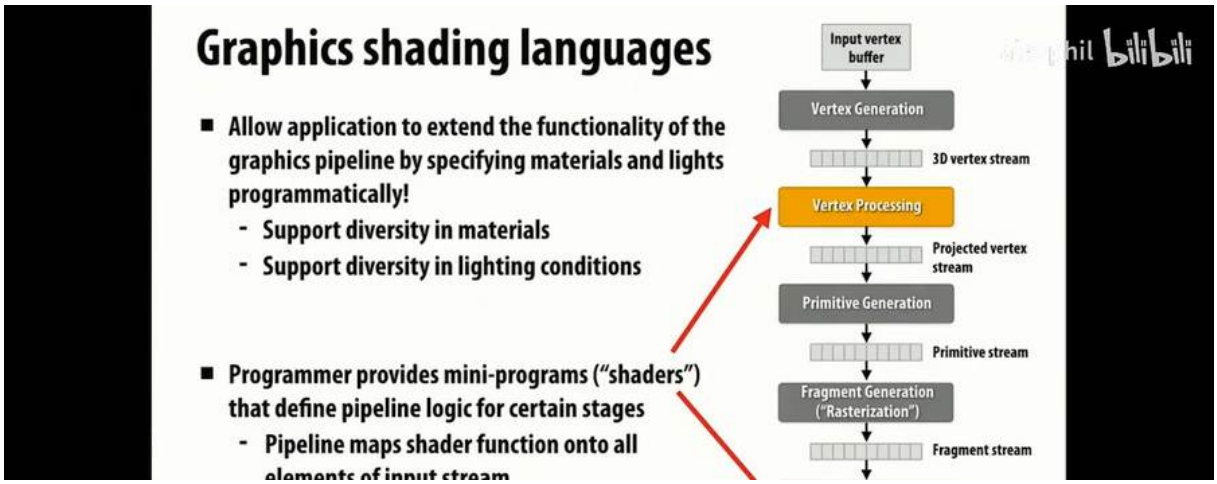

图 130: 图形着色语言：顶点与片元着色器可替换管线阶段[130]

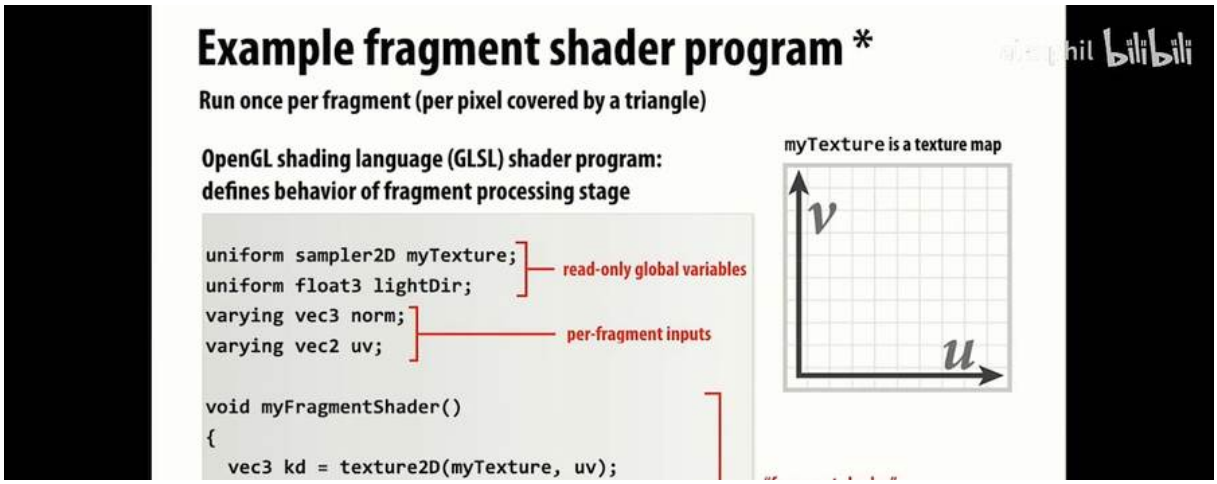

图 131: 片元着色器示例：每个片元运行一次[131]

本章小结

图形管线把渲染拆成顶点、三角形、片元、像素几个阶段，每个阶段都是对一堆独立对象做同样的操作。材质的复杂度推动管线不断加长，而着色器让这些阶段可以用软件替换。这个”对大量独立数据执行同一段程序”的结构，正是后来 CUDA 抽象的原型。

7.3 从图形卡到通用计算

历史背景

讲者回顾：这类工作早年都由普通 CPU 完成，2004 年单核无法继续提速，但晶体管数量仍能增加。

[130] 视频画面时间区间：00:19:30–00:19:45。若为裁剪图，时间区间仍对应原视频画面。
[131] 视频画面时间区间：00:20:00–00:20:15。若为裁剪图，时间区间仍对应原视频画面。

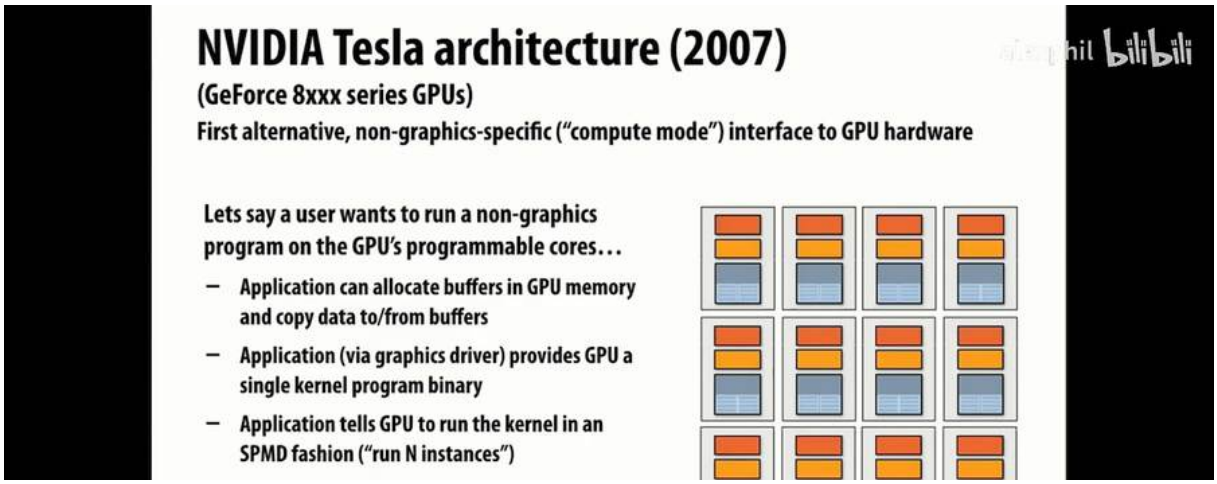

图 132: NVIDIA Tesla 架构（2007）：第一个非图形的替代方案[132]

GPU 由此诞生：可以造出大量处理单元，只是不能让单个巨型处理器跑单一指令流继续变快。

早期 GPU 纯粹为实现图形管线而设计。它们可编程，但只限于与图形操作直接相关的约束范围内——是可编程的专用硬件。

把科学计算伪装成图形操作

讲者讲了一段有意思的历史。一些聪明人意识到这东西能力很强，只要肯"倒立着思考"，就能把许多经典计算问题投射成图形操作。

抽象的形式由硬件能力决定

GPGPU 时代把计算伪装成图形操作，不是因为这些映射自然，而是因为当时的硬件只暴露图形接口。这说明抽象的形状往往由可用硬件的形态倒推出来，而不是从问题本身推出。识别这一点，才能判断某个抽象何时应当被替换。

第一个障碍是它只能处理三角形。解决办法很直接：任何方块都能用两个三角形覆盖。其次，可以创建特定尺寸的数组，并把其中的值当作数字使用。

于是有大量工作证明这些专用设备能解决科学问题、做科学模拟。

[132] 视频画面时间区间：00:28:00–00:28:15。若为裁剪图，时间区间仍对应原视频画面。

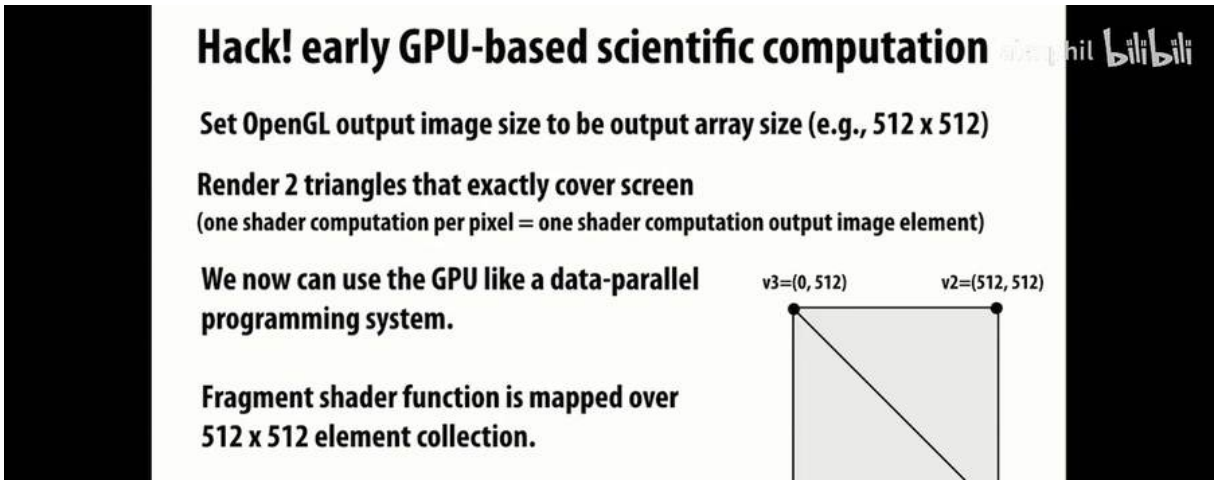

图 133: 早期 GPU 科学计算的技巧：把输出图像当作数据、把计算伪装成图形操作[133]

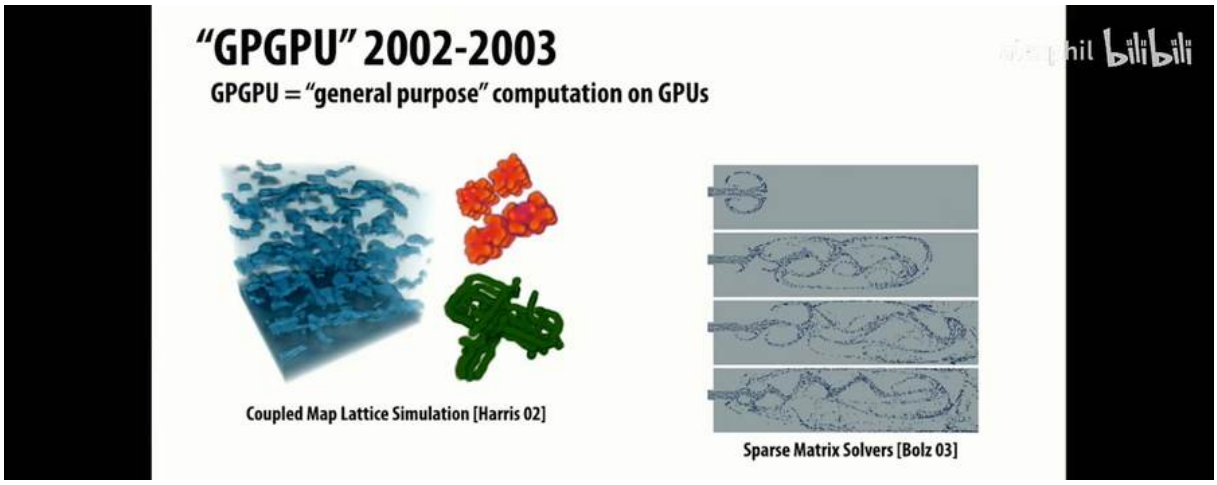

图 134: GPGPU 2002 到 2003：通用计算在 GPU 上的萌芽[134]

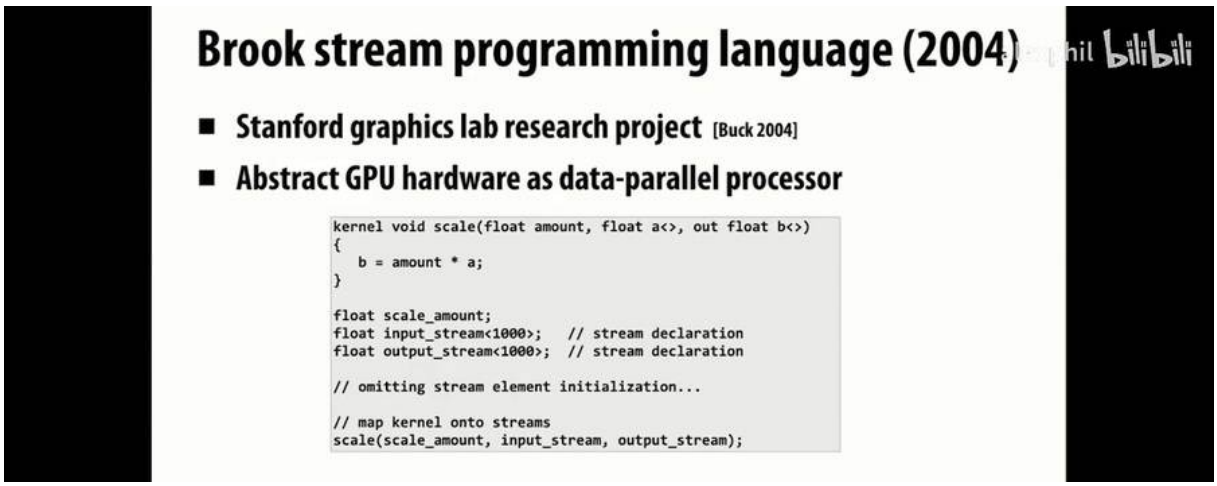

图 135: Brook 流编程语言（2004）：斯坦福图形实验室的研究项目[135]

[133]视频画面时间区间：00:22:00–00:22:15。若为裁剪图，时间区间仍对应原视频画面。
[134]视频画面时间区间：00:23:00–00:23:15。若为裁剪图，时间区间仍对应原视频画面。

这一段的历史意义在于：它证明了 GPU 的硬件形态足以承担通用计算，问题只在于编程方式太别扭。

为什么不能让操作系统调度

讲者在这里做了关键对比。CPU 上跑代码，是操作系统控制调度：它把程序加载进内存、选择哪个核执行、用中断机制不断切换进程，以支持多任务、IO 与虚拟内存这些可能造成长时间暂停的操作。

这套机制很重。讲者给出的数字是：一次 pthread 的上下文切换要花 10000 到 20000 个时钟周期——虽然绝对时间只有微秒级，但对一个追求高效利用硬件的程序来说，这个比例完全不可接受。

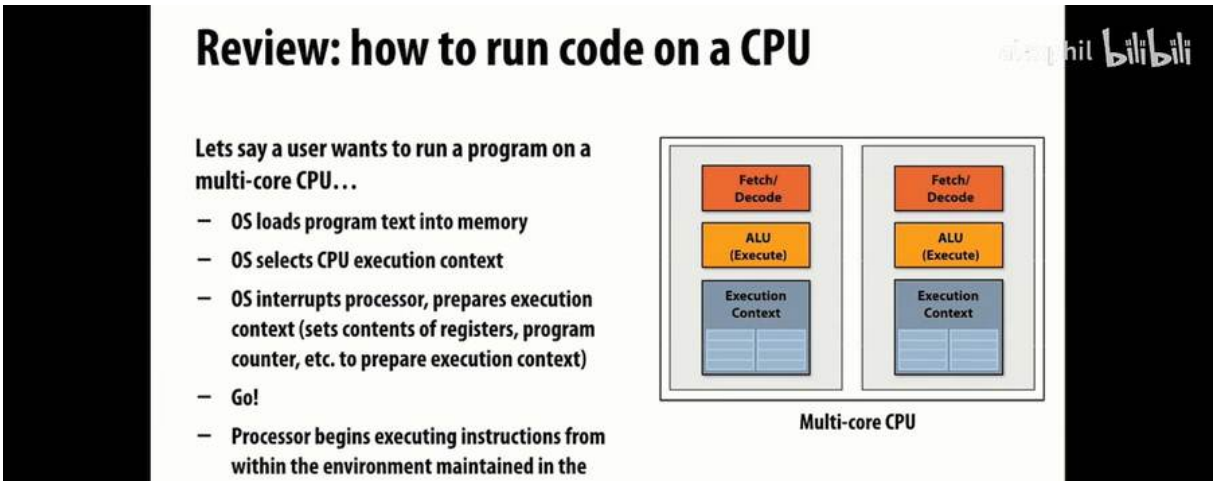

图 136: 回顾：代码如何在 CPU 上运行，操作系统掌控调度[136]

早期在 GPU 上运行的方式则是：把配置信息加载进 GPU，告诉它每个阶段该做什么，然后说”开始”，管线就跑起来。但这不被视为频繁操作——配置好就让它一直跑。

新的方向要让 GPU 支持非图形应用，但必须绕开操作系统：硬件的原始能力很强，可调度决策必须由硬件自己掌控。

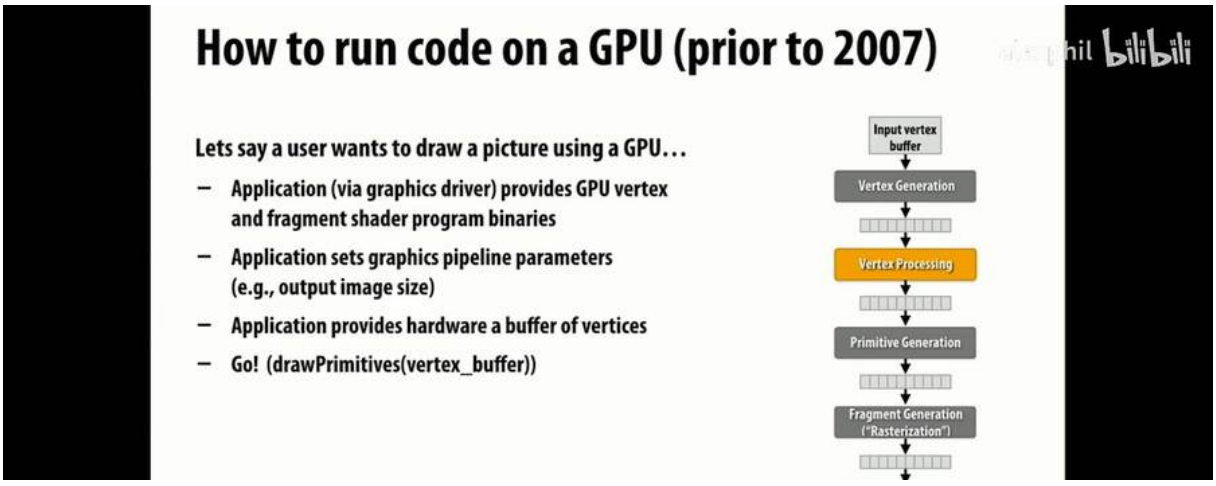

图 137: 2007 年以前代码如何在 GPU 上运行：配置管线后一次性启动[137]

[135] 视频画面时间区间：00:23:30–00:23:45。若为裁剪图，时间区间仍对应原视频画面。
[136] 视频画面时间区间：00:24:30–00:24:45。若为裁剪图，时间区间仍对应原视频画面。

CUDA 的定位

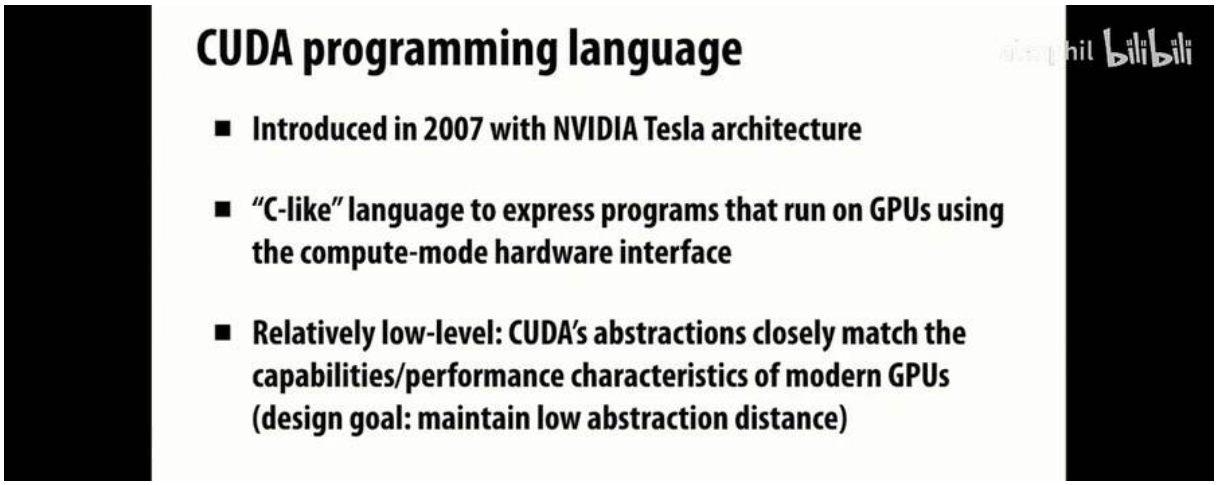

图 138: CUDA 编程语言：2007 年随 Tesla 架构推出[138]

CUDA 由 NVIDIA 在约十年前推出。写起来你会隐约觉得在写 C 或 C++，但并不是——你在描述一种高度数据并行的计算。

讲者认为 CUDA 做得好的一点是：既给出足够高层的抽象模型作为起点，又提供足够的接口让你能下探到 GPU 内部去调优。而这种调优是可以增量进行的——先看时间花在哪里，再决定改什么。他还建议始终保留一个功能正确的"黄金实现"，即使慢，也能在每次调优后对照验证结果。

他也坦率指出缺点：CUDA 随硬件架构演进不断变化，年度之间的兼容性不佳。开源的对应物 OpenCL 外观很像，但 CMU 的课程通常偏好开源方案以避免绑定单一公司——这一处是例外，专有版本确实明显更好、使用也更广。

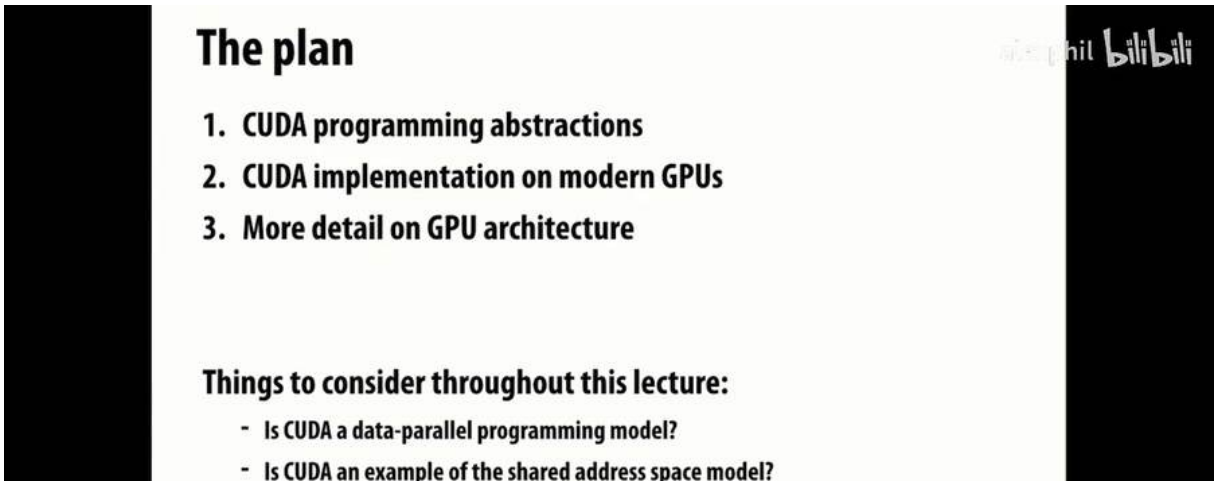

图 139: 本讲计划：CUDA 抽象、硬件映射与性能细节[139]

[137] 视频画面时间区间：00:27:00–00:27:15。若为裁剪图，时间区间仍对应原视频画面。
[138] 视频画面时间区间：00:30:00–00:30:15。若为裁剪图，时间区间仍对应原视频画面。
[139] 视频画面时间区间：00:32:00–00:32:15。若为裁剪图，时间区间仍对应原视频画面。

代价：功耗与价格

有学生问 GPU 为什么没撞上功耗墙。讲者回答它们也撞了——GHC 集群加上这些 GPU 后不得不增加空调能力。GPU 功耗约 180 瓦，芯片也很大，高端型号价格量级相当于数十万美元，500000 美元一台的高端卡并不罕见（视频 00:42:39）。

真正的优势在于可以扔掉多年来为榨取顺序程序并行度而添加的大量控制逻辑：庞大的分支预测、乱序执行、推测执行，以及大量缓存。扔掉这些，芯片面积就能腾出来做加法与乘法。

GPU 不是绕过功耗墙，而是换了一条花钱方式

GPU 的功耗与价格都远高于 CPU，它并没有解决散热问题，只是把每瓦预算从控制逻辑转移到了算术单元。因此在评估是否使用 GPU 时，不能只看峰值算力，还要看功耗预算、散热能力与采购成本是否可接受。

本章小结

GPU 从固定管线走向可编程，再被证明能承担通用计算。关键的技术障碍是调度：操作系统的上下文切换开销对 GPU 的利用方式无法承受，因此调度必须由硬件完成。CUDA 由此出现，提供高层抽象又能下探调优。它的优势来自砍掉控制逻辑，代价是功耗与价格。

7.4 CUDA 的抽象：线程、块与网格

先澄清一个术语陷阱

讲者提醒：同一个术语在不同语境下含义差别很大，其中”线程”尤其如此。

pthread 的线程是非常独立的执行流：有自己的控制、可以有自己的数据、与其他线程几乎无耦合。

CUDA 线程完全不同。它是程序在一份数据上的执行。多个 CUDA 线程彼此强耦合，能做的交互形式也受限得多。讲者说接下来的内容里，”线程”一律指 CUDA 线程。

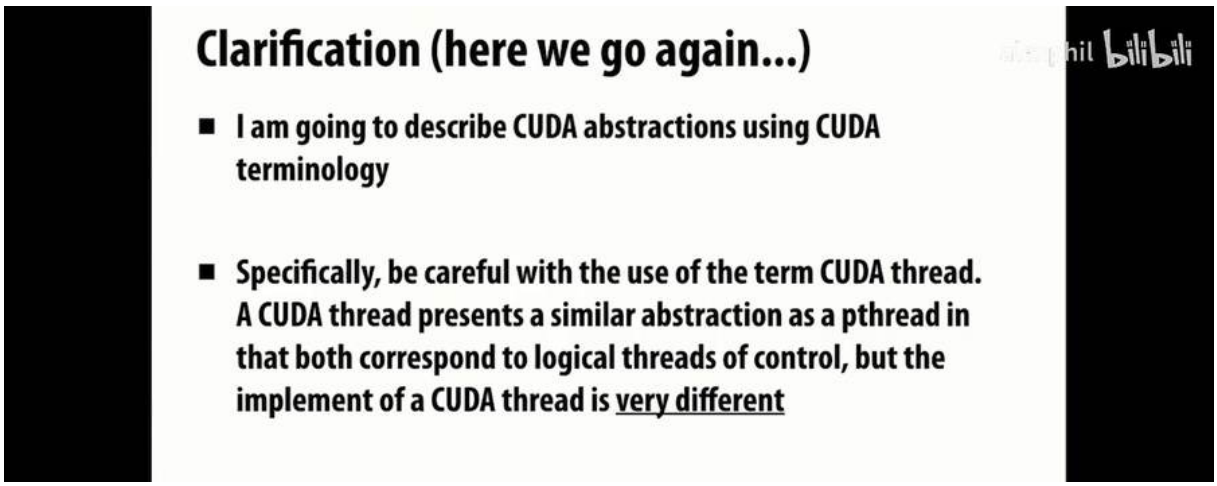

图 140：术语澄清：CUDA thread 与 pthread 语义差别极大[140]

[140] 视频画面时间区间：00:33:00–00:33:15。若为裁剪图，时间区间仍对应原视频画面。

三维的线程索引

在高层看，CUDA 是数据并行模型：一个线程就是对某个数据元素的处理。CUDA 支持最多三维的矩阵或数组表示，并为此提供直接支持——不过讲者强调，这更多是一种错觉，硬件里并没有三维结构。

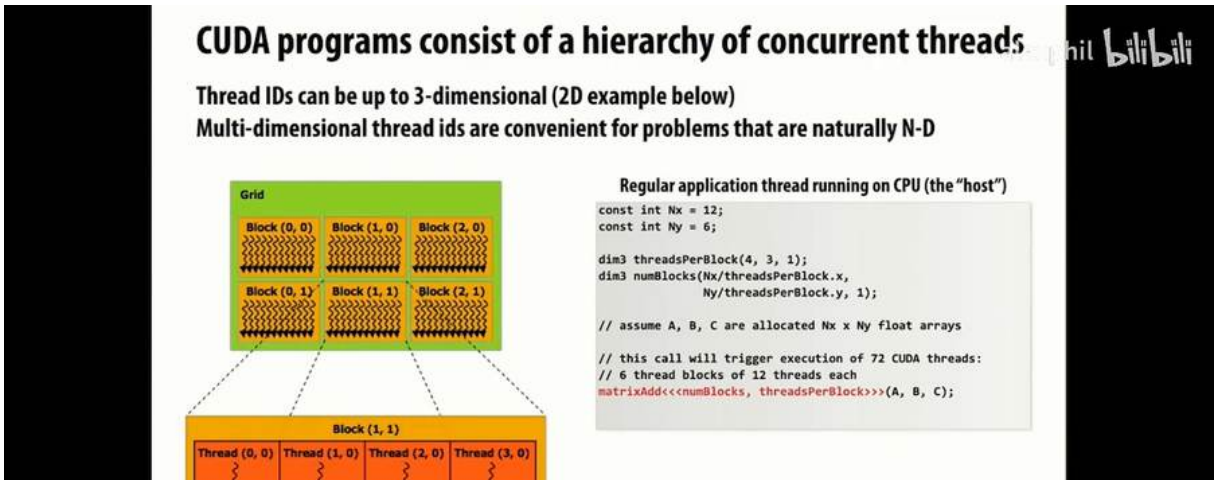

图 141: CUDA 程序由并发线程的层次结构组成，线程索引最多三维[141]

block 与 grid

以矩阵加法为例：一个 12×6 的矩阵，划分成若干 block，每个 block 是四列宽、三行高。于是得到六个块。每个 block 内是若干线程，每个线程负责一个元素。

若一个 block 是 4×3，那么所需的 block 数就是行数除以每块行数、列数除以每块列数。

写成式子：

$$N_{\text{block}} = \left\lceil \frac{N_x}{B_x} \right\rceil \times \left\lceil \frac{N_y}{B_y} \right\rceil$$

- N_x, N_y：数据在 x 与 y 方向的元素数，本例为 12 与 6；
- B_x, B_y：每个 block 在两方向上的线程数，本例为 4 与 3；
- 向上取整保证数据不被 block 尺寸整除时仍能覆盖全部元素。

讲者指出这背后是 2-13 里学过的行主序多维数组概念。

[141] 视频画面时间区间：00:34:00–00:34:15。若为裁剪图，时间区间仍对应原视频画面。

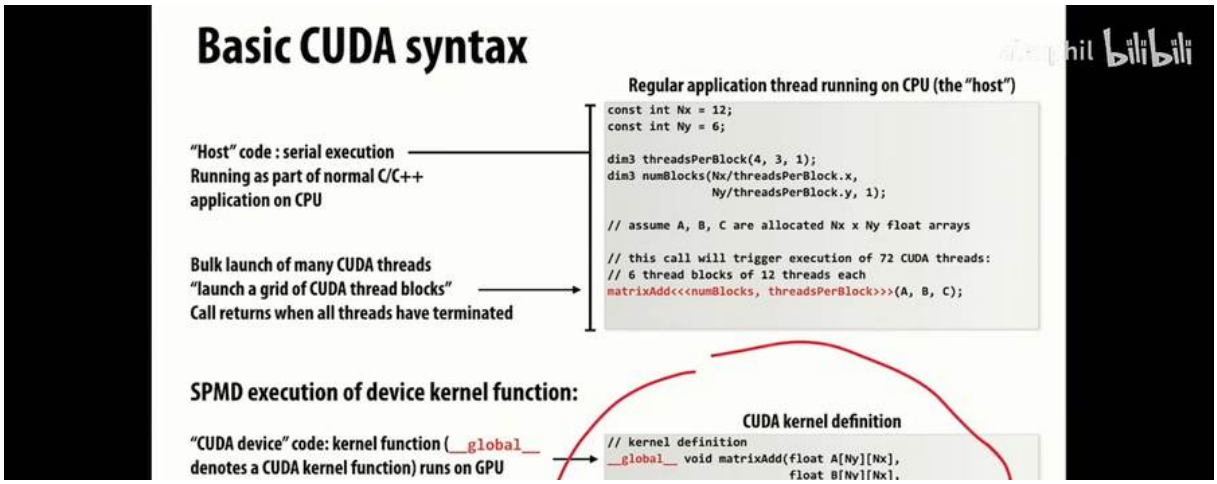

图 142: CUDA 基本语法：host 端配置 block 结构并启动核函数[142]

核函数

GPU 执行的那段代码叫核函数。它的写法基于图形学的思路：把自己想象成矩阵的第 (i, j) 个元素，做 a_{ij} 加 b_{ij} 并存到 c。

核函数需要一个全局下标，由三个量算出来：

$$i = \texttt{blockIdx.x} \times \texttt{blockDim.x} + \texttt{threadIdx.x}$$

- `blockIdx`：本 block 在 grid 中的编号，由 CUDA 在启动时分配；
- `blockDim`：每个 block 的尺寸，本例中 x 方向为 4；
- `threadIdx`：本线程在 block 内的编号；
- i：对应的全局元素下标。

讲者强调，核函数启动时它只知道自己是这个 block 里的某个线程，而这个 block 是众多 block 之一。

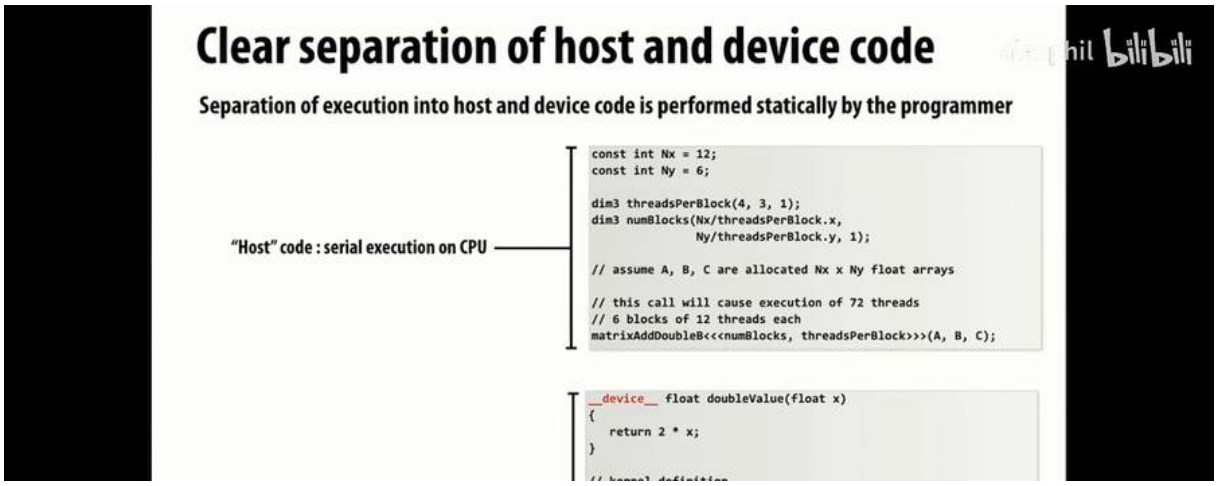

图 143: host 与 device 代码的清晰分离[143]

[142] 视频画面时间区间：00:38:00–00:38:15。若为裁剪图，时间区间仍对应原视频画面。

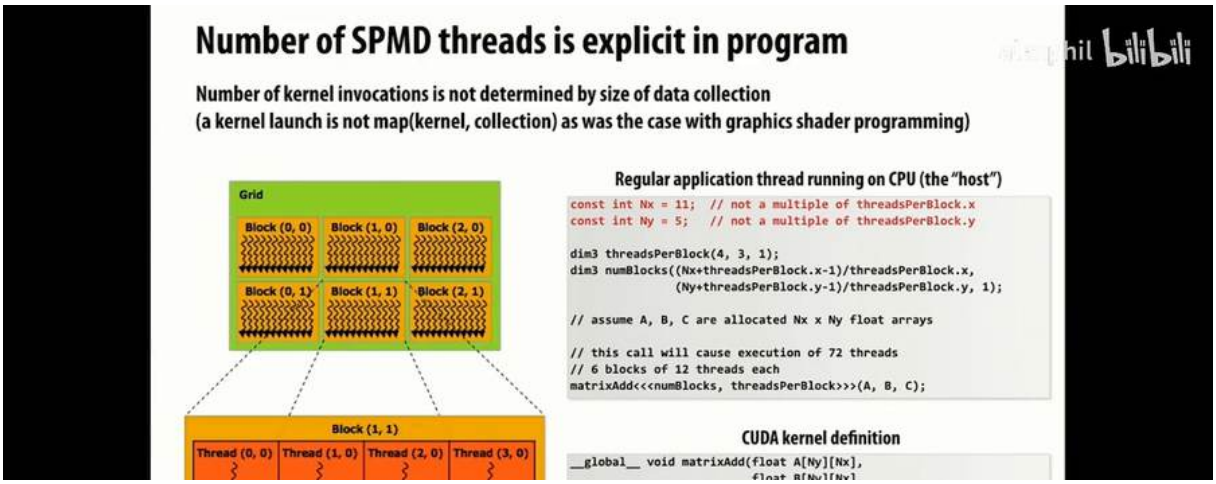

图 144: SPMD 线程数在程序中显式给出[144]

三个修饰符

同一份文件里既有 host 代码也有 device 代码，靠修饰符区分。

`__global__` 标记核函数，从 host 端调用、在设备上执行。`__device__` 标记设备端调用的普通函数，看起来就像普通函数。host 端则可以执行任意 C/C++，但会做一种特殊操作——线程启动，即那对三尖括号 `<<<...>>>`，用来指定 grid 与 block 的尺寸。

三尖括号是 host 与 device 的分界线

`kernel<<<gridDim, blockDim>>>(...)` 这一行是唯一同时属于两个世界的地方：由 host 发出，却描述设备上的线程拓扑。它不返回执行结果，只发出启动请求；设备上的执行与 host 的后续代码可以并行进行，需要显式同步才能保证顺序。

block 尺寸的硬限制

block 内线程数有硬件上限

讲者指出单个 block 的总线程数上限是 1024，这是硬件限制。实际写代码时还应优先选 32 的倍数这类对硬件友好的尺寸，因为线程会被按固定宽度分组执行。超出上限或尺寸不佳时，要么编译失败，要么执行效率下降。

学生问 block 是不是纯粹的虚拟概念。讲者回答：block 在组织计算的意义上是虚拟的，但它有上限，而且每个 block 会被映射到一个执行单元（SM）上执行。所以它更像是对计算的切块，而不是某块具体硬件；CUDA 让你看到硬件的这一点点结构，是为了方便后续调优。

[143] 视频画面时间区间：00:44:00–00:44:15。若为裁剪图，时间区间仍对应原视频画面。
[144] 视频画面时间区间：00:45:00–00:45:15。若为裁剪图，时间区间仍对应原视频画面。

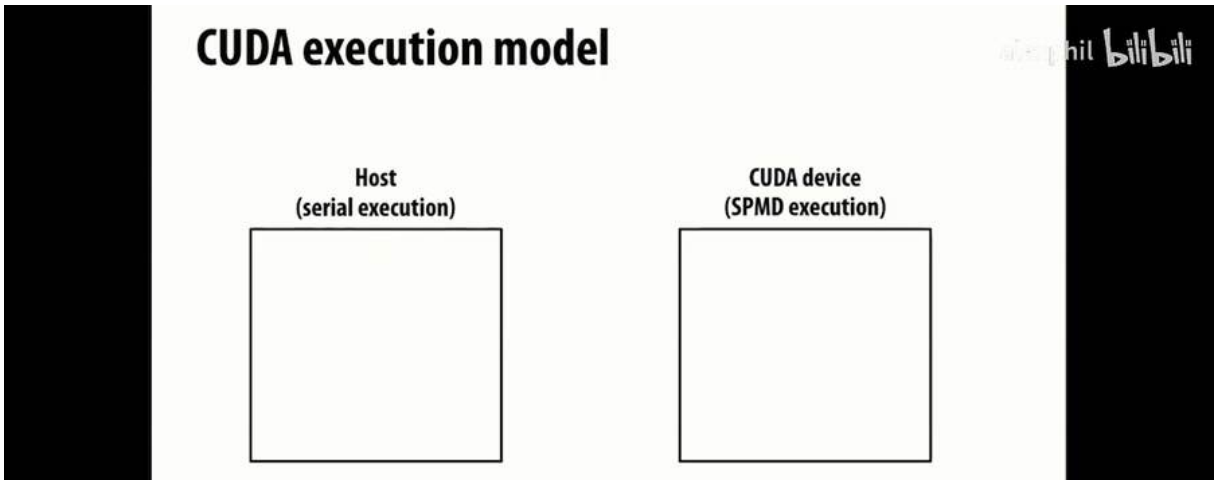

图 145: CUDA 执行模型：host 串行执行，device 以 SPMD 方式执行[145]

越界处理

如果数据规模不是 block 尺寸的整数倍，代码必须处理越界。讲者说这是很标准的做法：判断下标是否在范围内，越界就不做事。

他的分析值得记住：这个检查几乎零成本。唯一的代价是没有用满所有功能单元。这与 CPU 上的情形根本不同——在 CPU 上每个条件判断都可能是一次分支预测失败，是巨大减速的来源。而在这里，我们本来就有远多于硬件的活儿要干，因此调度可以相当粗放，仍能最大化利用性能。

越界检查在 GPU 上便宜的原因

GPU 的线程远多于硬件通道，多余线程被禁用只是让某些批次空转，不产生分支预测失败的惩罚。而在 CPU 上条件分支会打断流水线，代价高得多。同一个习惯用法在两种硬件上的成本完全不同，这是不能把 CPU 优化直觉直接搬到 GPU 的原因之一。

本章小结

CUDA 用 grid、block、thread 三级结构描述并发，核函数内用 `blockIdx` 与 `blockDim` 与 `threadIdx` 算出全局下标。block 上限 1024 线程，每个 block 映射到一个 SM。CUDA 是 SPMD 而非 SIMD，硬件负责把海量线程映射到有限通道上，因此程序员可以只写高层代码。

抽象既然回避了硬件，性能就被藏了起来。要把它拿回来，必须理解内存层次——那是下一章的主题。

7.5 内存层次与共享内存

地址空间是分离的

CUDA 内存模型最需要记住的一点：host 与 device 有各自独立的地址空间。

[145] 视频画面时间区间：00:50:00–00:50:15。若为裁剪图，时间区间仍对应原视频画面。

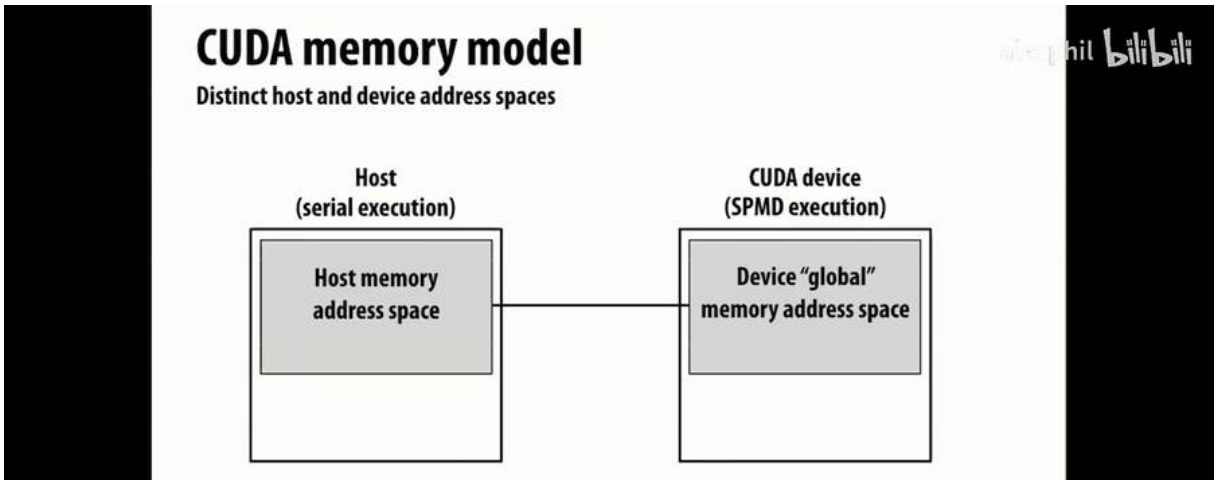

图 146: CUDA 内存模型：host 与 device 地址空间彼此独立[146]

跨地址空间的拷贝是最容易漏掉的成本

每次把数据搬到设备、把结果搬回主机，都要占用 PCIe 带宽，且需要显式同步。若内核本身的计算量很小，拷贝时间可能超过计算时间，使得 GPU 版本比 CPU 还慢。优化 GPU 程序时，先算清计算与传输各占多少时间，再决定是否值得搬到设备上。

因此数据必须在代码里显式搬运。讲者介绍了 `memcpy` 原语，用于在两个地址空间之间移动数据。

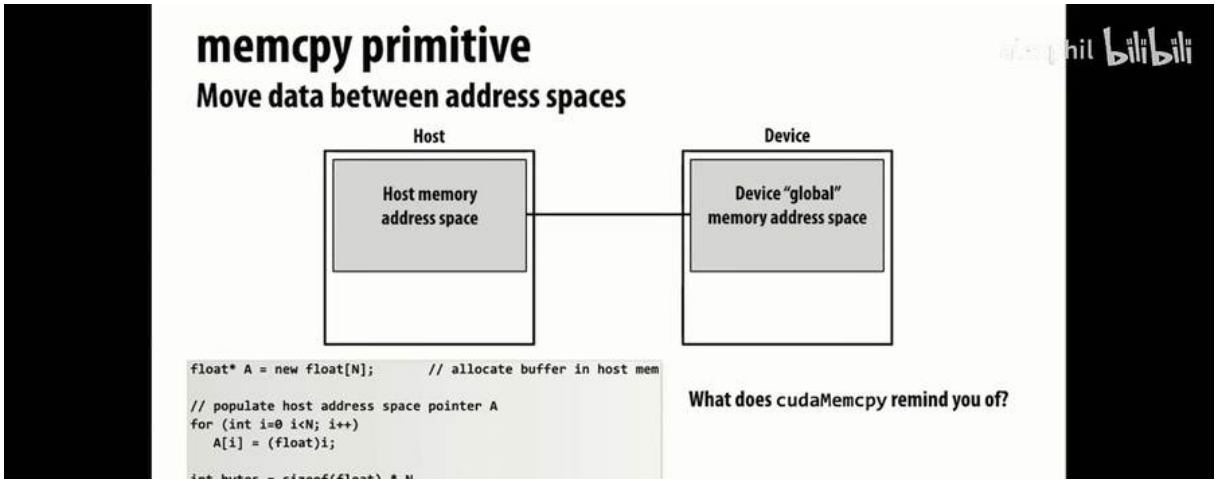

图 147: memcpy 原语：在 host 与 device 地址空间之间搬数据[147]

分配设备内存用 `cudaMalloc`。讲者特意指出它与常规 malloc 的差别：`cudaMalloc` 不返回指针，而是返回错误码，真正的指针通过参数传出。他顺带评论了早期 C 库以返回值表示结果的做法，以及为什么现代库改用错误码。

设备内存容量在 4 GB 到 8 GB 这个量级（视频 00:51:10）——不算大，但也不差。讲者也强调，这里的并发模型与 pthread 大不相同，对内存操作的一致性假设必须非常小心。

[146] 视频画面时间区间：00:51:00–00:51:15。若为裁剪图，时间区间仍对应原视频画面。
[147] 视频画面时间区间：00:52:00–00:52:15。若为裁剪图，时间区间仍对应原视频画面。

三类内存

讲者给出内核可见的三类内存。

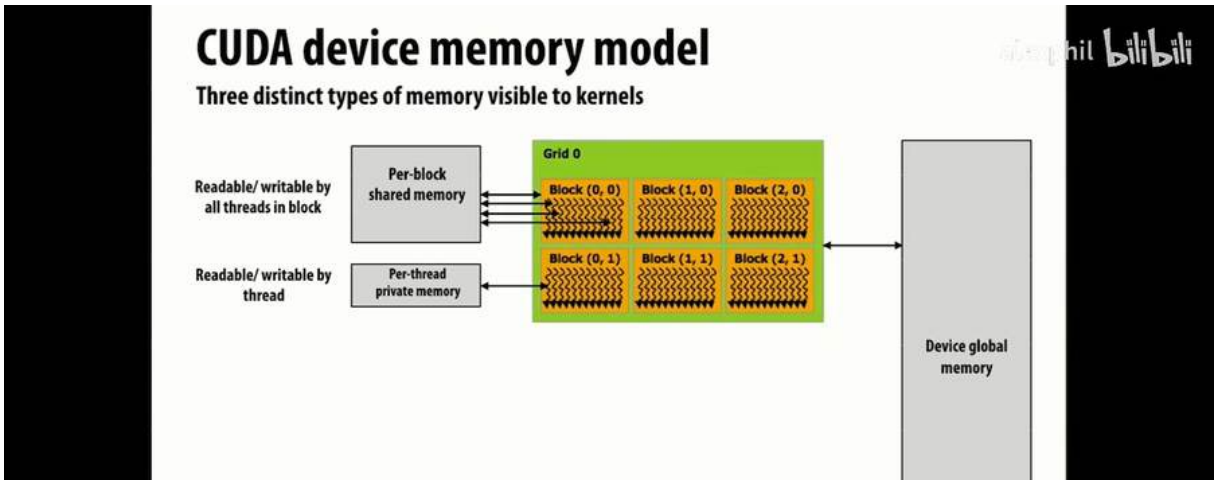

图 148: CUDA 设备内存模型：内核可见的三类内存[148]

第一类是全局内存，所有线程都能访问，容量最大、延迟最高。第二类是每个 block 的共享内存，由该 block 的所有线程共同使用。第三类是每个线程的极少私有存储。

关键差异在于控制权。CPU 的缓存完全由硬件控制，程序员无法指定什么留在缓存里。GPU 上这些内层存储更受软件控制，可以当作”软件缓存”使用：由你的代码决定往里放什么。

共享内存不会自动装数据

读一次全局内存，数据不会自动出现在 block 的共享内存里。必须由代码显式写入。这是它与硬件缓存的根本区别：缓存自动生效，共享内存需要你主动管理。正因如此，它会带来额外的编程负担，但也带来了缓存无法提供的确定性。

本章小结

CUDA 的 host 与 device 内存彼此独立，必须显式拷贝；`cudaMalloc` 返回错误码。设备端有三类内存：全局、block 共享、线程私有，其中共享内存由软件管理，是可控的”软件缓存”。

这套内存模型解释了一件事：GPU 上重复的全局读取没有缓存来救。下一章用卷积的例子把这个代价算清楚。

7.6 卷积示例：从 3n 次读取到一次

版本一：一个线程一个输出

讲者引入一维卷积作为贯穿示例。版本一的策略是一个线程负责一个输出元素，block 尺寸取 128。

[148] 视频画面时间区间：00:53:30–00:53:45。若为裁剪图，时间区间仍对应原视频画面。

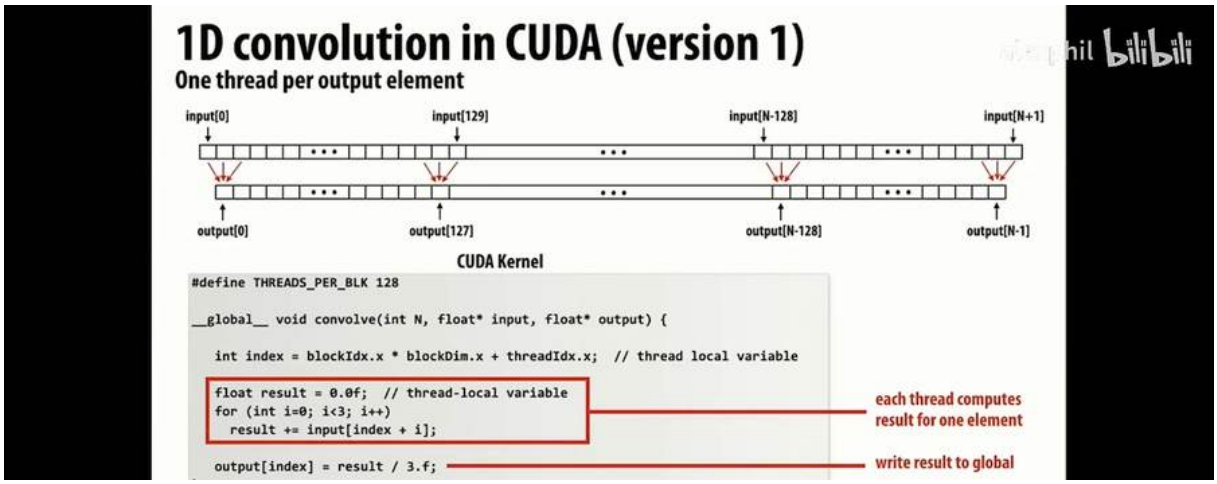

图 149: 一维卷积版本一：每个输出元素一个线程[149]

这段代码能正常工作，但不够快。讲者引导学生算一笔账，并特意叮嘱：不要假设有缓存。

答案是 $3n$ 次全局读取——每个线程都要做三次读取，而且它们之间不共享。因为不存在缓存，每次读就是真的去读，慢就是慢。

于是出现了一个明显的低效：三个线程都需要同一个输入值，它们却完全没有协作，各自独立地读了一遍。

重复的全局读取是 GPU 内核最常见的浪费

在 CPU 上，同一地址的多次读取可能被缓存吸收，因此这类浪费常被掩盖。GPU 的全局内存没有这种自动机制，重复读取全部落到带宽上。判断一个内核是否低效，先数它对全局内存做了多少次访问，再问这些访问能否被合并。

版本二：先用共享内存装一遍

既然 block 内有共享内存，就可以把它当缓存用——但必须显式使用。

讲者展示了版本二：在核函数里声明一个局部数组，用 `__shared__` 修饰。这样这块内存被分配在处理器（SM）的共享内存里，而不是全局内存。

[149] 视频画面时间区间：00:56:30–00:56:45。若为裁剪图，时间区间仍对应原视频画面。

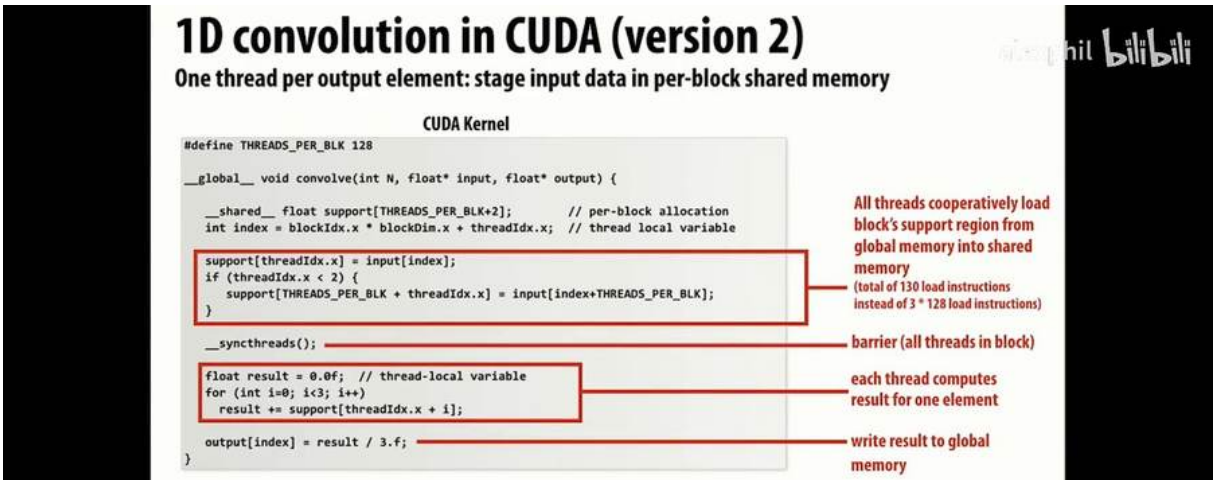

图 150: 一维卷积版本二：先用共享内存暂存输入数据[150]

代码分两个阶段。第一阶段每个线程从全局数组读入自己需要的那一段，写进共享数组；第二阶段所有线程改用共享数组做卷积计算。

由于一个 block 的所有线程共用这块共享内存，同一个输入值现在只被从全局读了一次，而不是三次。

把两种版本的全局读取次数写成式子是：

$$R_{\text{naive}} = 3n, \qquad R_{\text{shared}} \approx n$$

- R_{naive}：版本一的全局读取次数，每个输出做三次读；
- R_{shared}：版本二装载阶段总共读入的元素数，约为 n 加上块间重叠的少量边界；
- n：输入元素总数。

节省的比例接近 3 比 1，而这正是 GPU 上最稀缺的资源——全局内存带宽。

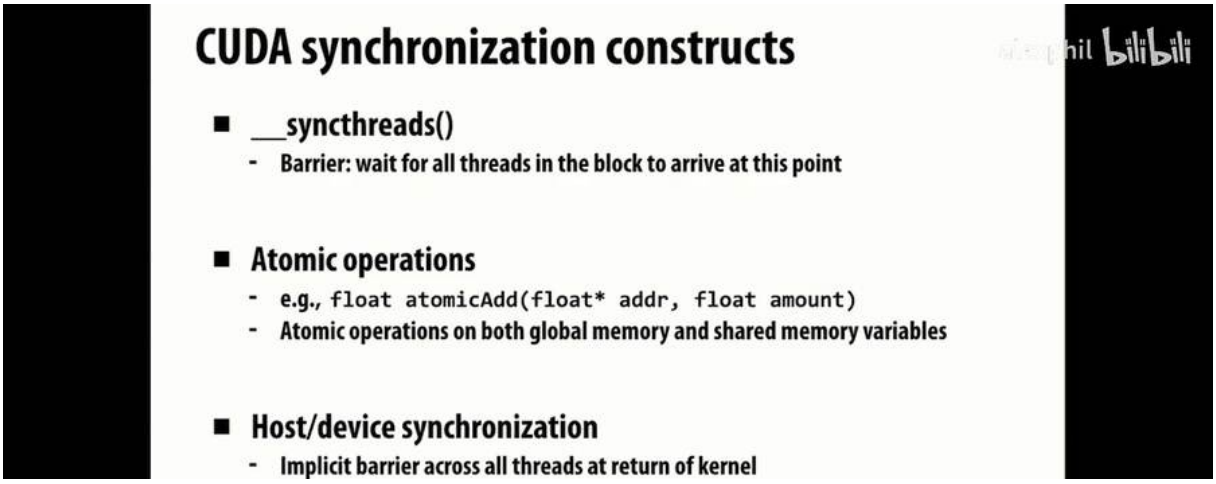

图 151: CUDA 同步原语 ___syncthreads：等待 block 内所有线程到达屏障[151]

[150] 视频画面时间区间：01:02:00–01:02:15。若为裁剪图，时间区间仍对应原视频画面。

[151] 视频画面时间区间：01:07:30–01:07:45。若为裁剪图，时间区间仍对应原视频画面。

为什么必须同步

两个阶段之间需要 `__syncthreads()`。它是一条屏障：等待 block 内所有线程都到达此处再继续。

理由与此前的红黑格序一致——如果不等待，第二阶段可能有线程读到了别人尚未写入的共享内存，结果是错的。讲者用”相当巧妙”形容这段代码，正是指这种手工分阶段加屏障的写法。

共享内存优化必须配同步

`__syncthreads()` 只保证 block 内同步，不跨 block。若误以为它能同步整个 grid，跨 block 的数据依赖就会出错而没有任何报错。此外屏障必须被 block 内所有线程执行到，放在条件分支内会导致死锁。

本章小结

朴素实现的每个输入需要 $3n$ 次全局读取，因为同一数据被多个线程各自读了一遍。用 `__shared__` 把输入按 block 装一次、再用 `__syncthreads()` 分阶段，可以把重复读取降到一次。这一步手工管理正是换取性能的代价。

讲者随后总结：CUDA 的高吞吐模型让”提高吞吐比降低延迟容易”这句话落到了实处。

7.7 抽象回顾与执行细节

CUDA 抽象的两层

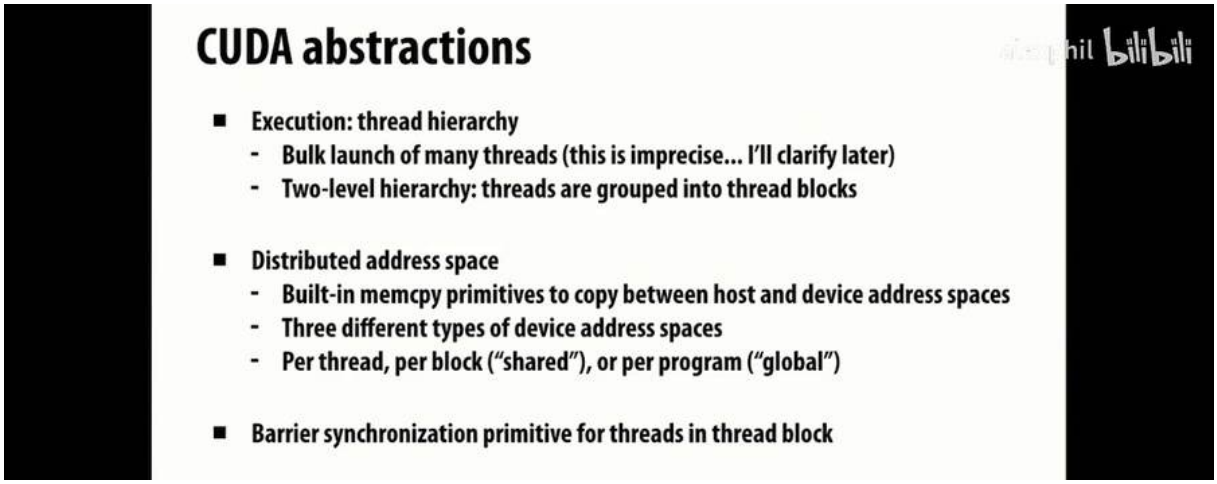

图 152：CUDA 抽象汇总：执行层次与内存层次[152]

CUDA 的抽象可以分成两条轴。执行轴上是从 thread 到 block 到 grid 的层次，核心特征是批量启动大量线程。内存轴上是从线程私有到 block 共享到全局的分层，核心特征是后两者可由软件控制。

[152] 视频画面时间区间：01:08:30–01:08:45。若为裁剪图，时间区间仍对应原视频画面。

层次	作用域	谁管理	容量与速度
线程私有	单个 CUDA 线程	编译器	极小、最快
block 共享	一个 block 内全部线程	程序员显式	小、快
全局内存	整个 grid	程序员显式	大、慢

表 7: CUDA 三类内存的作用域与管理方式

warp 与 SMM

讲者最后展示了执行细节。以 2014 年的 GTX 980（Maxwell 架构）为例，其中每个 SMM 单元（讲者的引号表明这相当于一个” 核”）都有若干 warp。

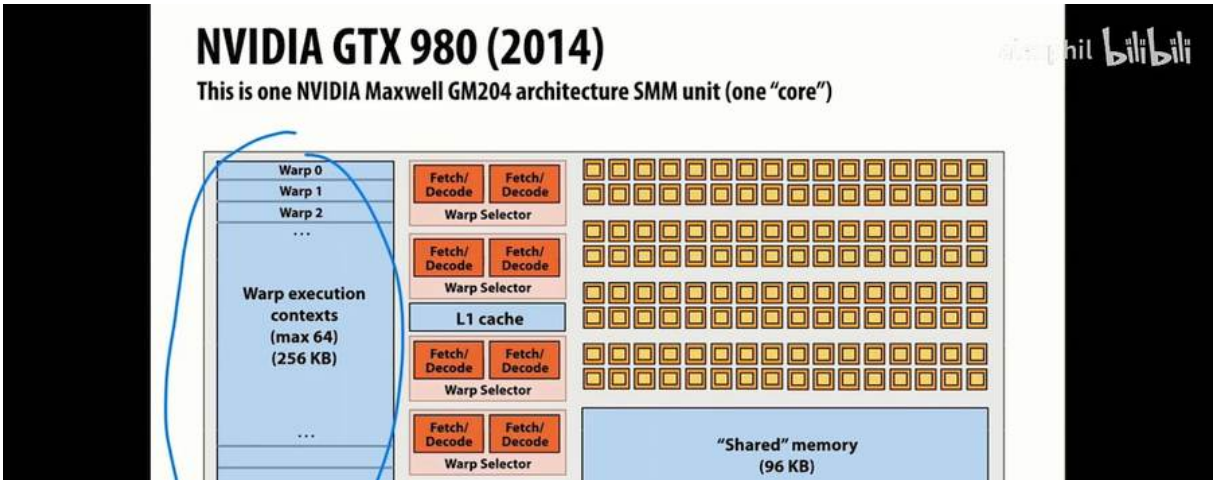

图 153: NVIDIA GTX 980（2014）的 Maxwell SMM 单元[153]

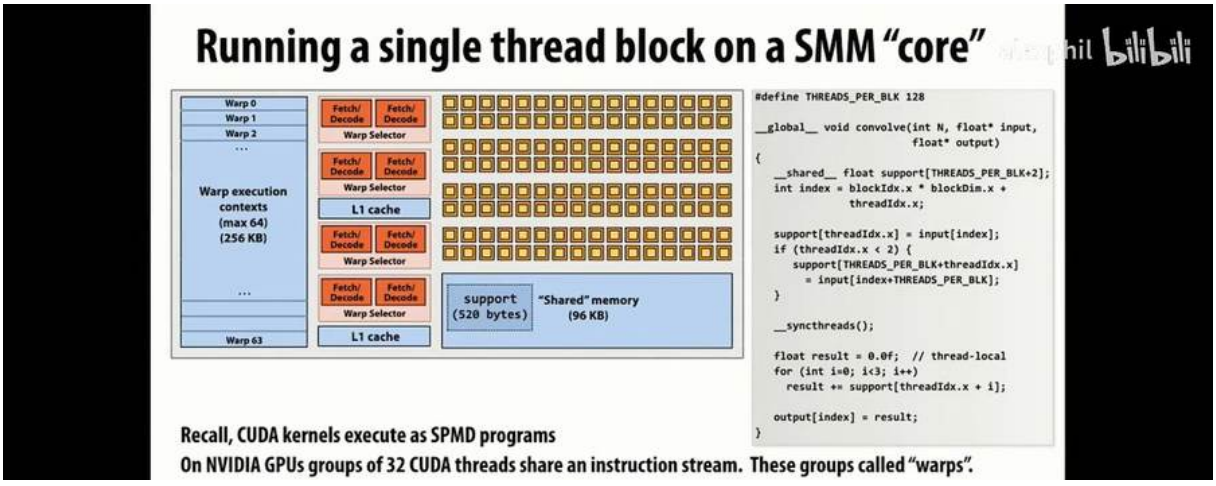

图 154: 在一个 SMM 核上运行单个线程块：线程被组织成若干 warp[154]

[153]视频画面时间区间：01:14:00–01:14:15。若为裁剪图，时间区间仍对应原视频画面。
[154]视频画面时间区间：01:16:00–01:16:15。若为裁剪图，时间区间仍对应原视频画面。

讲者展示了一个含 128 线程的 block 如何在 SMM 上运行。这 128 个线程被组织成固定宽度的 warp，warp 是硬件调度的基本单位。这也解释了前面为什么建议 block 尺寸取 32 的倍数：若尺寸不是 warp 宽度的整数倍，最后一个 warp 会有空闲通道。

warp 是 CUDA 隐藏的关键执行单位

线程在抽象上是独立的，但硬件按 warp 成组执行。同一 warp 内的线程若走不同分支，就会依次执行各分支、互相等待，这就是 warp 内的分歧。因此 CUDA 代码中把相邻线程安排去处理相邻数据，可以让同一 warp 的行为保持一致，避免分歧。

本章小结

CUDA 的抽象沿执行与内存两条轴展开，block 是这两条轴的交点：它既是调度单元，也是共享内存的作用域。硬件层面线程按 warp 组织，这决定了 block 尺寸与数据排布的选择。

到这里，写出一份正确且高效的 CUDA 内核所需的全部要素都已出现。

7.8 总结与延伸

讲者的收尾带着务实的态度：即使只有 10% 的效率，你跑得也很快；真正的性能调优者能把利用率推上去，但你往往会发现推不上去。不过只要计算能被表达成对独立数据做同样的操作，你就能让它在 GPU 上跑得很好。他还承诺在作业中会遇到一些看似不独立、但可以通过原语操作改写的情况。

把全讲串起来：GPU 为实时三维渲染而生，图形管线天然是数据并行，因此硬件形态与这条管线同构。真实感需求推动管线加长，着色器让阶段可以用软件替换，最终引来把科学计算伪装成图形操作的 GPGPU 时代。操作系统的调度开销（一次 pthread 上下文切换上万时钟周期）使 GPU 必须自己做调度，CUDA 由此出现。它用 grid、block、thread 描述并发，用 host 与 device 分离的地址空间和显式拷贝处理数据流，用三类内存给出可控的局部性手段。卷积示例把这一切落到具体数字上：朴素版本需要 $3n$ 次全局读取，用 `__shared__` 加同步可以降到一次。

本讲义在讲者材料之上做的压缩，有四点需要与讲者原话区分：

- CUDA 的抽象刻意隐藏了硬件，性能必须靠下探找回。讲者展示的 block、共享内存、warp 都不是纯粹的抽象概念，而是让程序员能够干预硬件的接口。用高层抽象写出的代码正确但慢，这不是缺陷而是设计意图。
- 共享内存是可控性的来源，也是同步负担的来源。缓存自动生效，共享内存需要手动装载与屏障。因此它并非总是更优：当数据访问本来就有良好局部性时，硬件缓存可能足够，手工管理反而增加出错机会。
- warp 的存在解释了多个看似无关的建议。block 尺寸取 32 的倍数、相邻线程处理相邻数据、避免 warp 内部分歧，这些都源于硬件按固定宽度成组执行。
- 10% 效率仍可能很快，这句话要放在绝对速度上理解。与第七讲（算术强度）的结论一致：峰值利用率低不代表不划算，比较要看绝对耗时。

这堂课的边界在于它是 GPU 入门，讲者明确表示调优细节与更多原语会在作业里遇到。因此本文给出的是 CUDA 的概念地图与第一个可量化的优化范式，不含完整的最佳实践清单。

两个可以直接从本讲数据提出的下一步问题：其一，共享内存容量有限，一个 block 能装载的数据量有上限，那么当卷积核更长时该如何分块？其二，warp 内的分歧会让同一批线程串行执行各

分支，那么怎样的数据排布能让同一个 warp 尽量走同一分支？这两个问题分别指向下一讲的局部性优化与 GPU 性能调优。

Lecture 08 Locality, Communication, Contention

视频时长：1 小时 27 分 2 秒 来源：https://www.bilibili.com/video/BV1vr4y1Z78J/

8.1 通信：并行程序里最容易被低估的成本

上一讲解决了如何把工作分下去。这一讲处理分配完成之后真正决定性能的部分：数据如何在处理器之间流动。

为什么通信能抹掉并行收益

讲者开门见山地描述了最常见的失败模式：你可能对如何把问题切成若干块有一个很好的想法，也确实并行起来了，然后被来回搬运数据的成本淹没，把并行带来的机会全部抵消掉。

这种成本出现在所有形态的并行系统上。共享内存的多核处理器里，它是缓存与总线上的流量；消息传递的大系统里，它是网络传输；而 GPU 只是这个问题的一种特殊口味——box 内部有些部分像共享内存，另一些则像消息传递，必须显式搬数据才能继续算。

三者的差别其实只在”距离”上。讲者说这些东西位于机器具体距离之上的更高层次，它们的共同问题是同一件事。

通信成本与并行收益是同一枚硬币的两面

切分带来并行，切分也带来跨块的数据需求。因此不存在”只提高并行度而不同时增加通信”的方案。判断一次并行化是否值得，必须同时算出两笔账：计算被分摊到多少份，以及块之间要搬多少数据。只算前一笔，就会得到上一讲那种 4 核比 2 核更慢的结果。

一个把通信暴露得最清楚的例子

讲者选消息传递版的网格求解器作为主线，理由很直接：在这个模型里通信是显式的，每一条数据交换都写在代码里，无法假装不存在。

问题本身是求每个网格点与其上下左右四个邻居的平均值。实现上加了一圈边缘行列，这样边界点也有合理的邻居值，不必为它们写特殊判断。因此数据是 $(N+2)\times(N+2)$ 的网格，实际计算的是内部的 $N\times N$ 区域。

加一圈边缘是为了消除边界分支

边界点缺少某个方向的邻居，若不特殊处理，内层循环里就要为每个元素判断”我是否在边界”。这会在最热的循环里引入分支，而且让向量化失效。加一圈哨兵行列把这些判断移出内层循环，代价只是少量存储与一次边界填充通信。

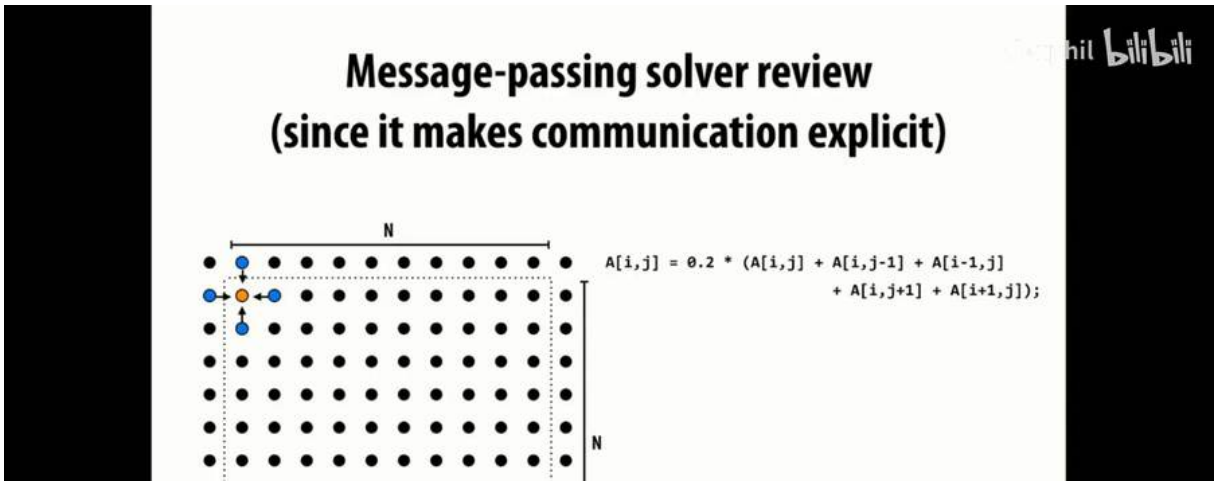

图 155: 消息传递求解器回顾：通信在这个模型里是显式的[155]

最自然的分法是按行切成块，块数等于可用线程数。为简化，讲者假设线程数能整除 N。

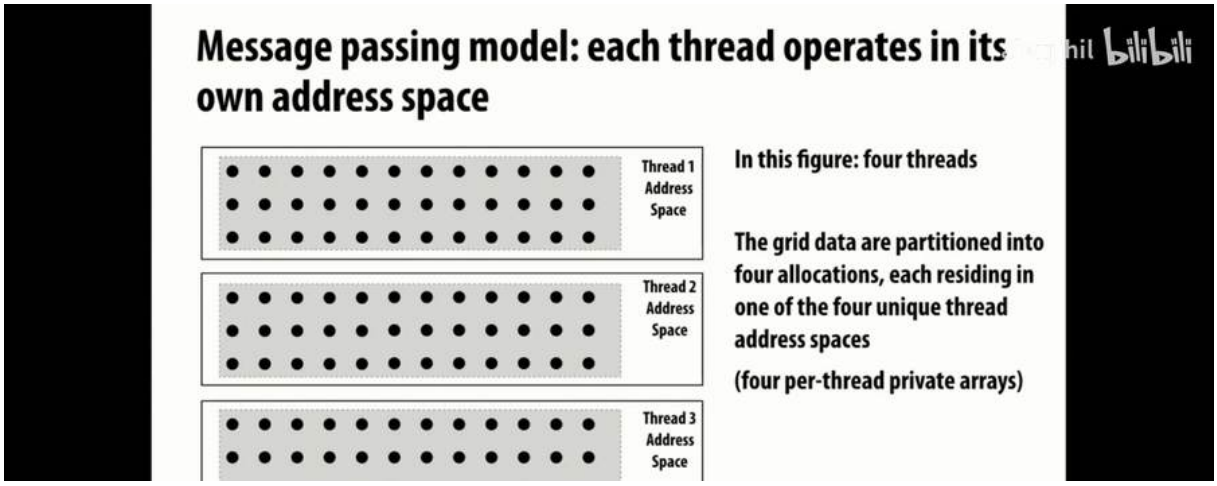

图 156: 消息传递模型：每个线程在自己的地址空间内运行[156]

每块需要什么

在消息传递世界里，每个块跑在一个独立处理器上，跨处理器取数据只能靠发消息。

假设是迭代计算：对全网格做一遍平均，更新状态，再重复，直到满足收敛条件。

那么第 2 号线程要算它那几行，就需要拿到它上方和下方相邻块的那一行数据。除了最上和最下的块，每个线程都要接收两组数据，也要向外发出两组——各一行。

[155] 视频画面时间区间：00:07:30–00:07:45。若为裁剪图，时间区间仍对应原视频画面。

[156] 视频画面时间区间：00:09:00–00:09:15。若为裁剪图，时间区间仍对应原视频画面。

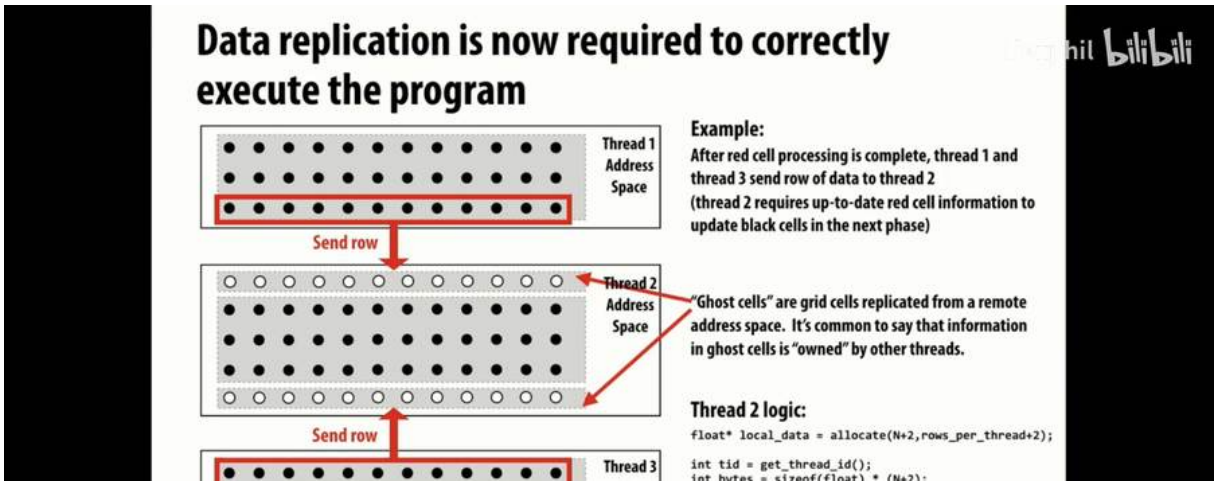

图 157: 消息传递中正确执行需要数据复制：每个线程保存自己的块与邻居的边界[157]

因为无法共享内存，每个线程必须自己保存一份邻居边界数据的副本。这就是”复制”的必要性。

收敛判断需要一次全局归约

除了取邻居数据，还有第二种通信。收敛条件是检查每个网格点的新旧差值、取绝对值、在全网格上求和，低于阈值就停止。

在消息传递里这必须分两步：所有线程把自己的差值发给 0 号线程；0 号线程负责接收并累加，判断是否足够小；然后把”继续”或”结束”的状态广播给所有线程。

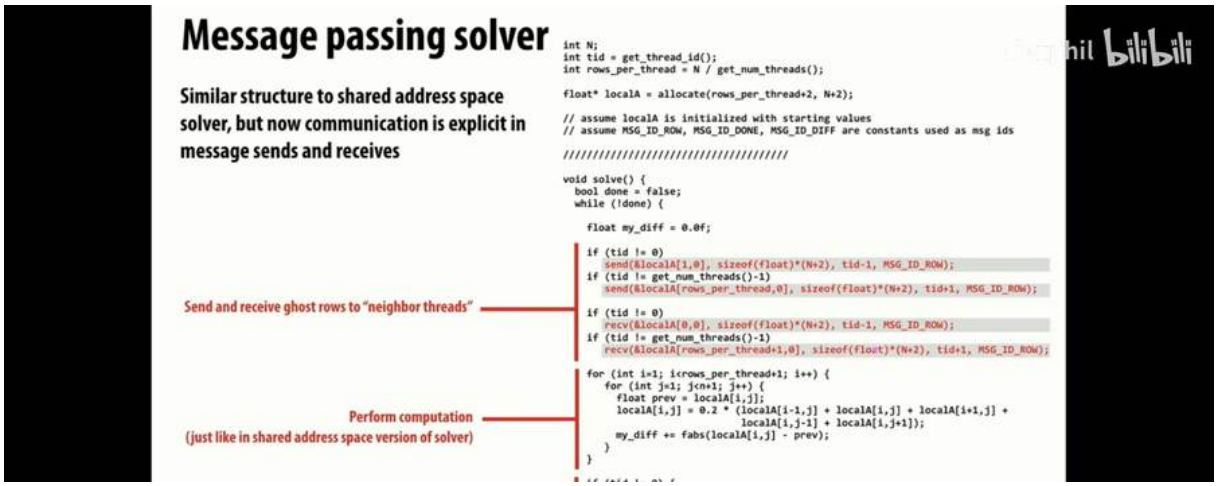

图 158: 消息传递求解器的代码：邻居交换与收敛归约两种通信并存[158]

于是这份程序里同时存在两种通信模式：邻接交换（局部、可并行）与主从归约广播（全局、串行化）。讲者说下一步看 MPI 时会发现，这类程序用 MPI 写起来相当容易。

[157] 视频画面时间区间：00:10:00–00:10:15。若为裁剪图，时间区间仍对应原视频画面。

[158] 视频画面时间区间：00:12:30–00:12:45。若为裁剪图，时间区间仍对应原视频画面。

本章小结

通信成本足以抹掉并行收益，而它在三种并行形态中本质相同，只是距离不同。网格求解器把这件事暴露得最清楚：按行分块后每个线程必须保存邻居边界数据的副本，收敛判断还需要一次把所有差值汇总到 0 号线程的全局归约。

既然通信无法避免，下一个问题是它的耗时由什么决定。这需要先把”延迟”与”带宽”这两个常被混用的量分清楚。

8.2 发送与接收的几种约定

在讨论性能之前，讲者先交代消息传递原语的语义差异，因为这直接决定了程序能否正确运行。

阻塞发送：会合

从系统实现角度看最直接的是阻塞发送，也叫会合：发送方与接收方必须同步，传输才能发生。

软件上的表现是双向的。接收时如果消息还没到，你会等待；而发送时，代码也会阻塞，直到接收方确认收到。这像一次握手：发送方想发、接收方想收，双方达成一致，数据被拷贝过去，然后各自返回。

讲者指出系统实现者喜欢这种方式，因为它要求最少的缓冲——不需要在接收方跑得比发送方慢时囤积数据。他还提到可以让发送方领先若干条消息的变体，但会复杂得多。

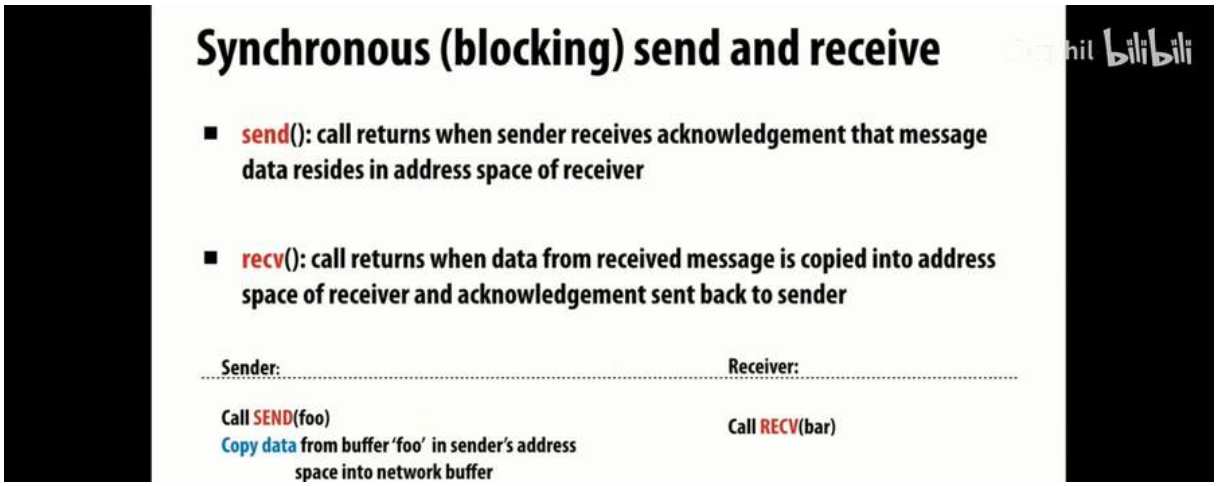

图 159: 同步（阻塞）发送与接收：调用在收到确认后返回[159]

死锁与奇偶解法

学生很快指出风险：如果所有人都在发、没人在收，就全部挂死。讲者确认这正是阻塞发送的典型陷阱。

而在网格求解器里，那段代码如果直接用阻塞发送，恰好会死锁———一开始除最上面的线程外，所有线程都在尝试发两条消息，而没有任何线程在尝试接收。

解法是奇偶交替：让偶数号的行先发后收，奇数号的行先收后发，然后各自翻转角色。这样总有一半线程在收，握手能够完成。

[159] 视频画面时间区间：00:15:00–00:15:15。若为裁剪图，时间区间仍对应原视频画面。

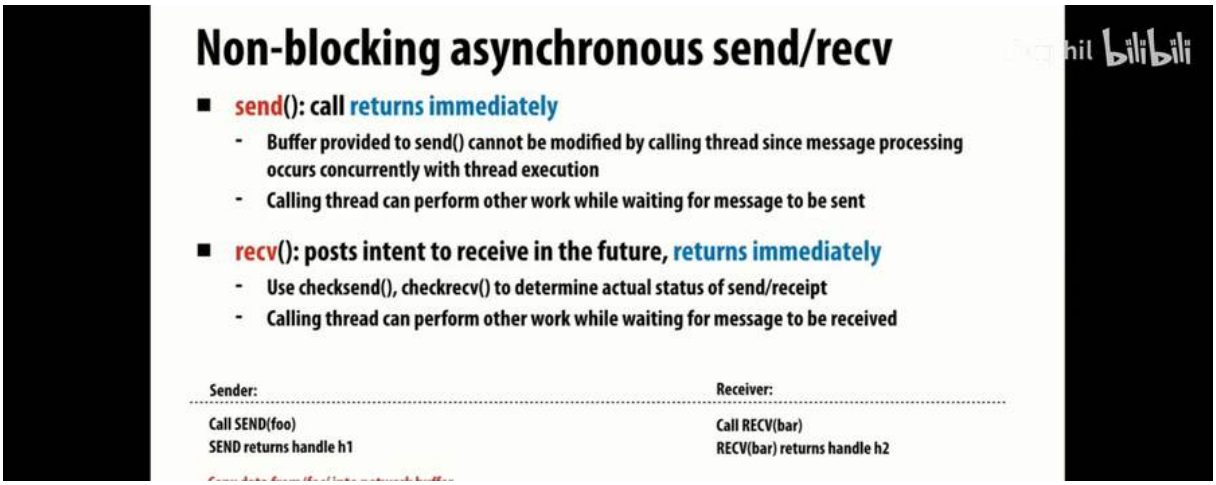

图 160：非阻塞异步发送与接收：调用立即返回，稍后再确认完成[160]

阻塞发送的配对顺序必须由程序员保证

奇偶交替不是优化技巧，而是正确性要求。它把”每一对收发都有一方先就位”这件事编码进了线程编号。一旦线程拓扑变化（例如改成二维分块，每个块有上下左右四个邻居），奇偶规则就要重新设计。这也是消息传递被认为”僵硬”的具体体现。

非阻塞发送

另一种约定是非阻塞发送：调用立即返回，真正的传输在后台进行。

问题是这样可能堆积出无界的待发消息。因此通常返回一个令牌（句柄），发送方之后可以用它查询发送是否真的完成了。

还可以做得更彻底：收发两端都完全异步。接收方可以声明”我想收某个消息，好了告诉我”，然后去做别的事，等消息真正到达时触发一个回调。

讲者的评价是：如果你能设计出让人人有事可做的应用，这种机制很有用——不必干等消息到来。

非阻塞原语把等待从调用点移到了别处

阻塞版本的等待发生在 send/recv 调用处，代码结构因此被迫与通信顺序绑定。非阻塞版本把等待推迟到显式检查或同步点，让程序员能把通信与计算重叠。代价是需要管理句柄、处理未完成的请求，代码复杂度上升。

本章小结

阻塞发送要求收发双方会合，缓冲需求最低但容易死锁，奇偶交替是标准解法。非阻塞发送立即返回并给出句柄，可通过查询或回调确认完成，能把等待与计算重叠。

语义确定之后，就可以讨论一个更基础的问题：一次通信究竟要花多少时间？

[160] 视频画面时间区间：00:19:00–00:19:15。若为裁剪图，时间区间仍对应原视频画面。

8.3 延迟与带宽：两个必须分开的量

讲者用一个生活化的类比来建立直觉，这段讨论值得完整展开，因为它是后面所有通信分析的出发点。

公路类比

设想从旧金山开车到匹兹堡。车的速度是每小时 100 公里，路程 4000 公里，因此单程需要 40 小时。

但如果我们有多辆车呢？如果车之间保持 1 公里间距，那么任一时该公路上可以有 4000 辆车同时在路上，而到达率是每小时 100 人。

图 161: 延迟与吞吐：公路类比（时速 100 公里，全程 4000 公里）[161]

图 162: 公路类比：车速与车距共同决定吞吐[162]

这是提高吞吐而不改善延迟的经典例子。讲者的判断是：在大量系统设计里，改善吞吐比改善延迟容易得多。

[161] 视频画面时间区间：00:23:00–00:23:15。若为裁剪图，时间区间仍对应原视频画面。
[162] 视频画面时间区间：00:24:30–00:24:45。若为裁剪图，时间区间仍对应原视频画面。

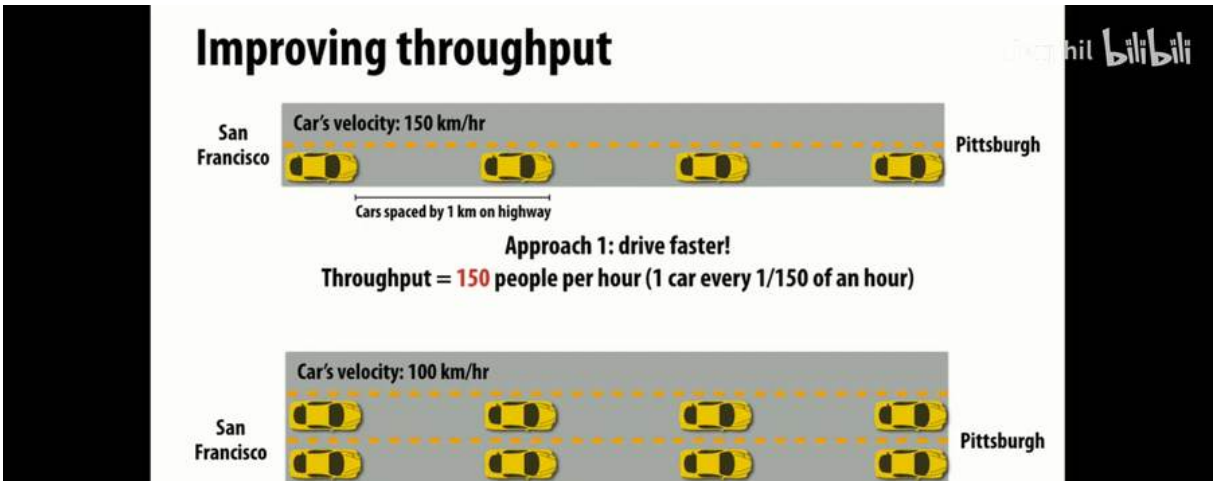

图 163: 提高车速到 150 公里/小时：同时改善延迟与吞吐[163]

当然也可以开得更快。若车速提升到 150 公里/小时，吞吐随之升到每小时 150 人，延迟也有所下降。或者可以修更多的路、把路修宽，同样能提高容量。这两种手段性质不同：开快车同时改善两者，加车只改善吞吐。

两个量的定义

讲者随即给出一组可用于任何系统的定义。

延迟是一次操作从开始到完成的端到端时间。它在系统各处都会出现：缓存缺失的处理时间、从 A 点发消息到 B 点的时间、从 Piazza 拿到回复的时间。

带宽是单位时间能完成的工作量，通常按字节而不是按操作计。

在纯串行情形下，带宽就是延迟的倒数（配上适当的单位）。但除此之外还有别的提高带宽的手段。

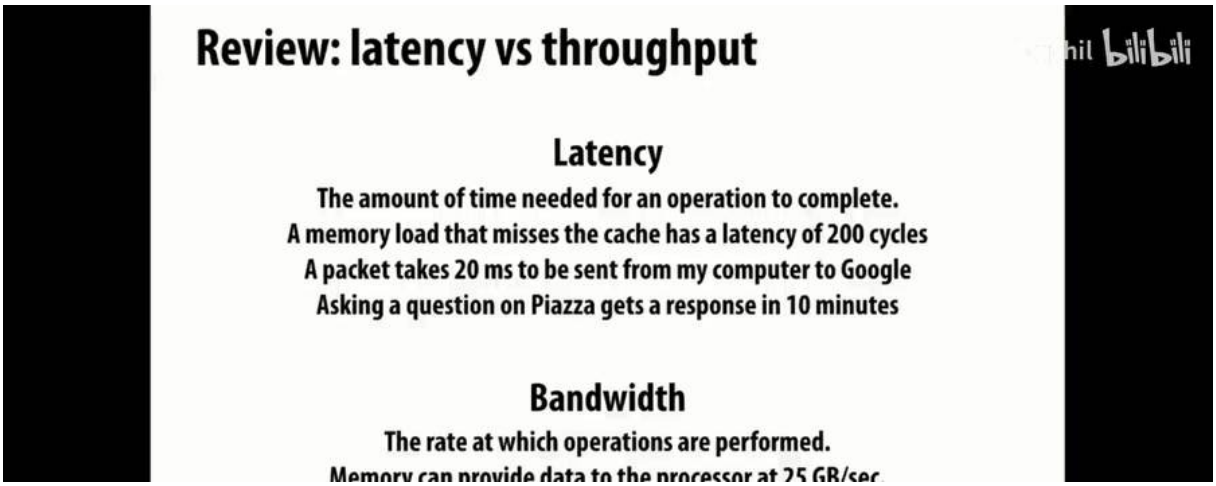

图 164: 延迟与吞吐的定义：前者是单次耗时，后者是单位时间完成量[164]

[163] 视频画面时间区间：00:25:00–00:25:15。若为裁剪图，时间区间仍对应原视频画面。

[164] 视频画面时间区间：00:26:00–00:26:15。若为裁剪图，时间区间仍对应原视频画面。

先判断瓶颈是延迟还是带宽

两者对应完全不同的改进手段。若瓶颈是延迟（关键路径上的一次访问），增加并行度无济于事，只能缩短路径或预取。若瓶颈是带宽（大量数据搬运），提高单次速度作用有限，应当增加并发通道或减少搬运总量。判断错方向，投入会完全浪费。

洗衣与流水线

讲者用一个更贴近并行计算的例子：洗衣服。

假设洗衣机 45 分钟、烘干机 60 分钟、折叠 15 分钟，那么完成一批衣服的总延迟是两小时。

要提升吞吐，最直接的办法是把资源加倍——两倍洗衣机、两倍烘干机，还得找个愿意帮你叠衣服的朋友。这样吞吐翻倍，而延迟一点没变。

更好的办法是流水线。第一批洗完后立刻开始第二批，不必等第一批全部完成。当然会有等待：烘干机要 60 分钟而洗衣机只要 45 分钟，所以第二批洗完后必须等 15 分钟才有空闲烘干机。

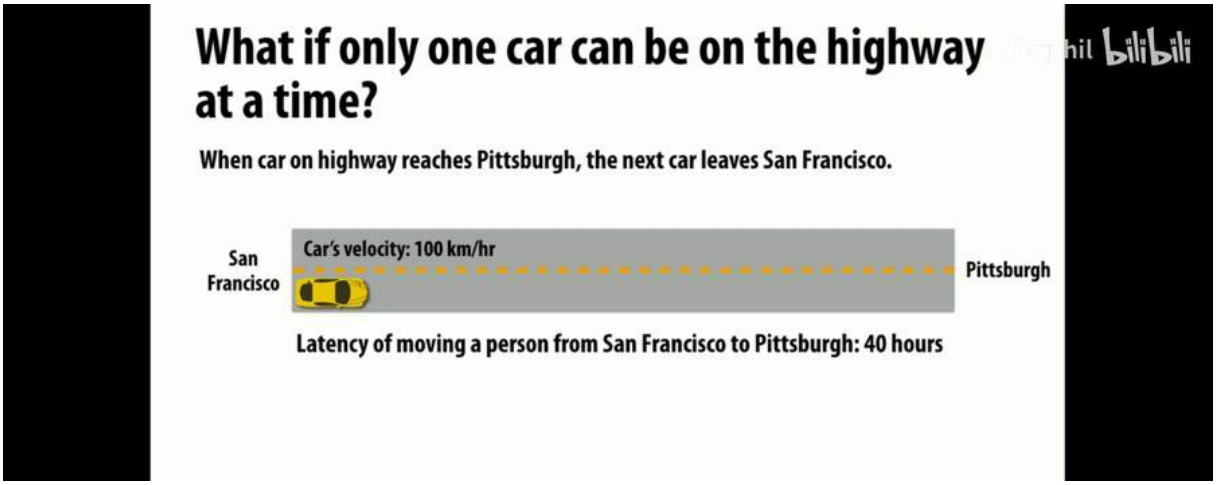

图 165：若公路上同时只能有一辆车会怎样：串行化的极端情形[165]

图 166：用洗衣流程提升吞吐：目标是最多大化多批衣服的吞吐[166]

[165] 视频画面时间区间：00:27:00–00:27:15。若为裁剪图，时间区间仍对应原视频画面。

[166] 视频画面时间区间：00:28:30–00:28:45。若为裁剪图，时间区间仍对应原视频画面。

关键结论是：

$$\text{Throughput}_{\max} = \frac{1}{\max_i L_i}$$

- L_i：第 i 个流水阶段的延迟，本例中洗衣机 45 分钟、烘干机 60 分钟、折叠 15 分钟；
- $\max_i L_i$：最慢阶段的延迟，即 60 分钟；
- 结果：这个系统最多每小时完成一批。

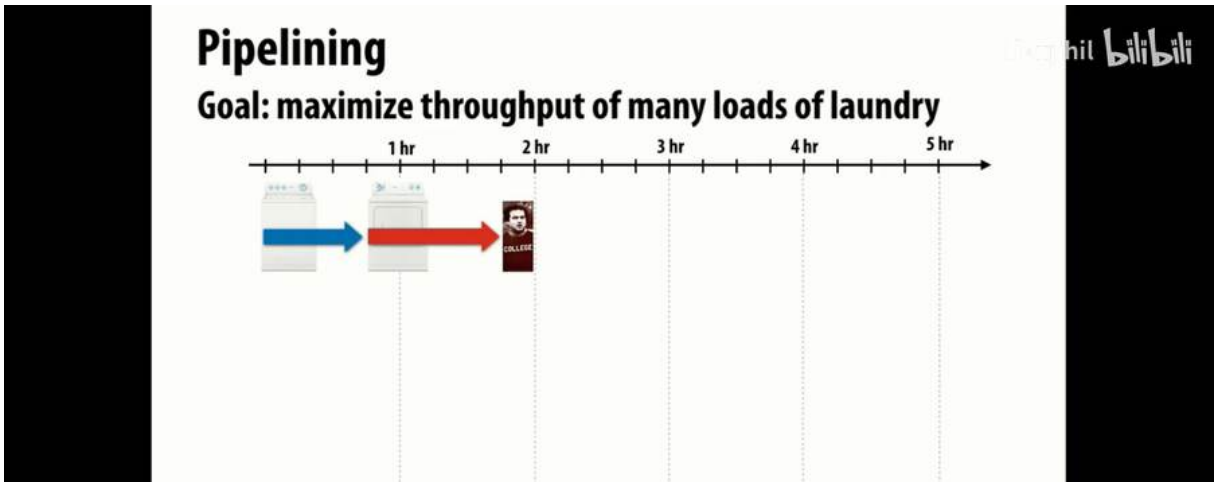

图 167：流水线：让各阶段始终有活在干，吞吐由最慢阶段决定[167]

讲者指出好消息：吞吐翻倍而没有增加任何资源。坏消息是你每小时得花 15 分钟叠衣服——但洗衣机与烘干机反正闲着，不用白不用。

处理器也是这么做的

讲者说这正是处理器设计的原理：把指令执行拆成若干阶段——取指、译码、执行、写回。最基础的流水线只有这几级，而现代处理器有 15 到 20 级，每级耗时可能不同，并配有大量动态调度的执行单元。

用公路类比再讲一遍：可以把 4000 公里切成 4000 个一公里的区段，规定每段同时只能有一辆车。这其实很像铁路调度——把轨道分成离散区段，保证同一段上不会有两列车。

如果能不增加开销地把流水级做得更细，例如改成 500 米一段，那么同时可用的资源更多，吞吐还能翻倍。

[167] 视频画面时间区间：00:29:00–00:29:15。若为裁剪图，时间区间仍对应原视频画面。

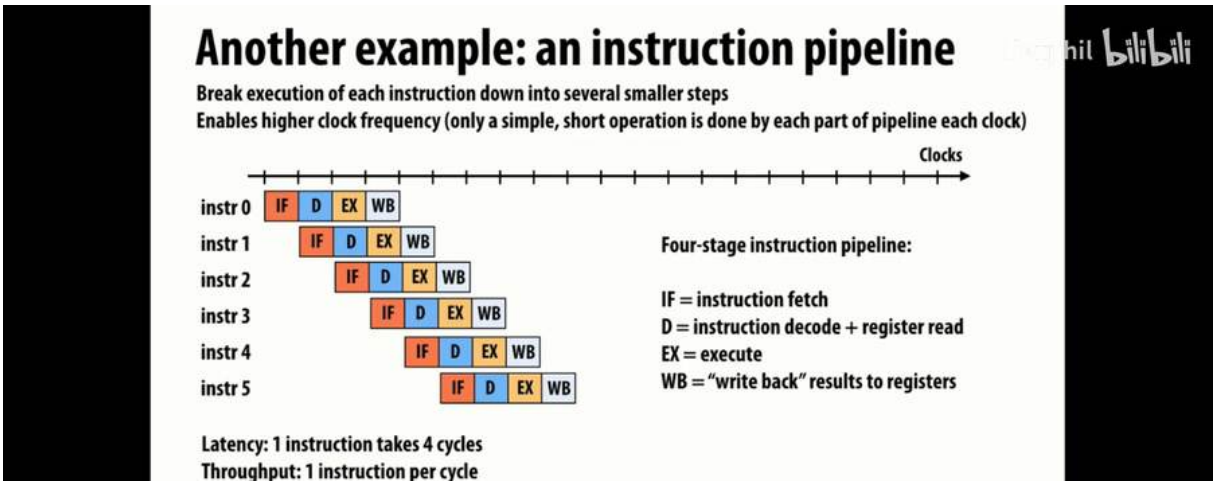

图 168: 指令流水线：把执行拆成取指、译码、执行、写回等阶段[168]

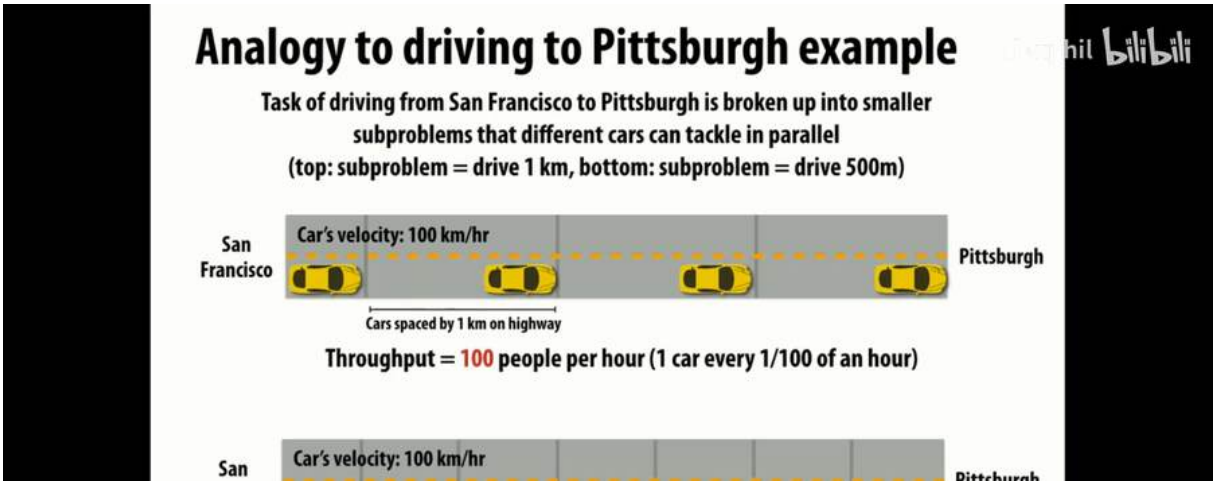

图 169: 与驾车类比对应：把公路切成区段当作流水阶段[169]

本章小结

延迟是单次操作的端到端时间，带宽是单位时间完成量，两者的改进手段不同。公路类比说明增加并发可以提升吞吐而不改善延迟；洗衣例子给出流水线的核心结论：吞吐上限由最慢阶段决定，而流水可以在不增加资源的情况下翻倍吞吐。

这个结论直接适用于通信。下一章把它写成可以用来估算的公式。

8.4 通信耗时的模型

固定开销加传输时间

讲者用公路类比建立通信的简单模型：发送一条 n 比特的消息。

$$T(n) = T_0 + \frac{n}{B}$$

[168] 视频画面时间区间：00:30:30–00:30:45。若为裁剪图，时间区间仍对应原视频画面。

[169] 视频画面时间区间：00:32:00–00:32:15。若为裁剪图，时间区间仍对应原视频画面。

- $T(n)$：发送 n 比特消息的总耗时；
- T_0：固定开销，包括消息从 A 到 B 的传播时间与协议处理，跨美国也要几秒的量级；
- n：消息的比特数；
- B：链路带宽，单位比特每秒。

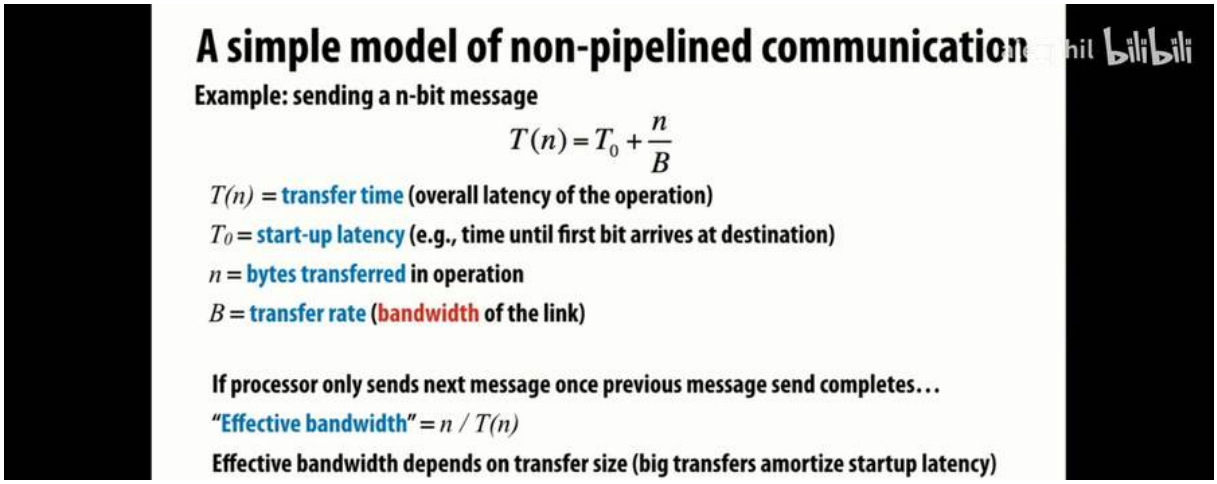

图 170: 非流水线通信的简单模型：$T(n) = t_0 + n/B$[170]

讲者说这是许多通信模型不错的第一近似。它立刻带来一条实用推论：如果把消息合并成数量更少、每条更大的消息，T_0 带来的影响在总量上就被摊薄了。

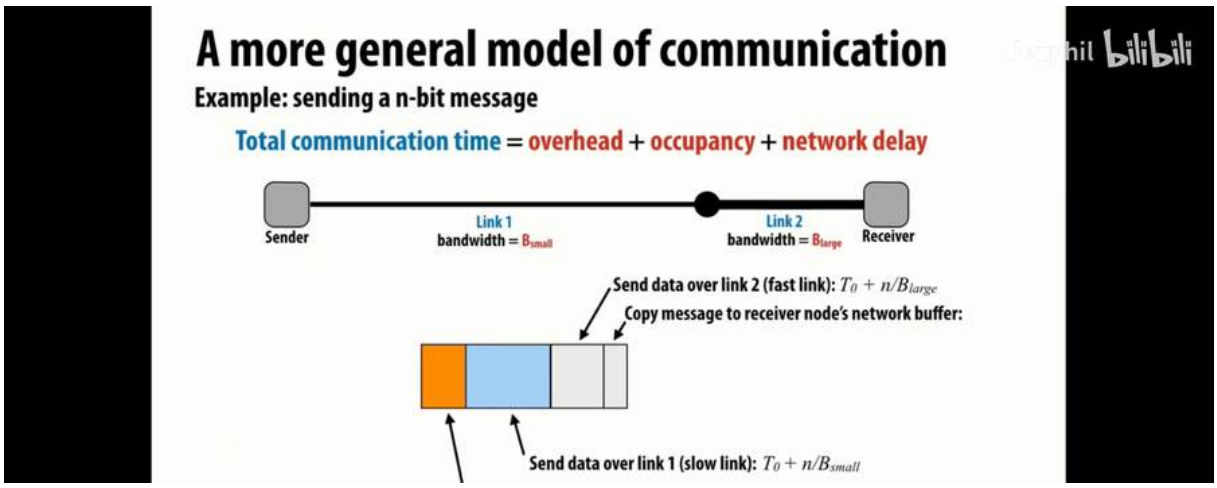

图 171: 更一般的通信模型：固定开销加传输时间，并考虑最慢链路[171]

他的原话是：几乎所有的通信介质都更擅长发送长消息而不是短消息。

有效带宽

由此可以定义有效带宽：用总字节数除以包含 T_0 的实际总时间。

[170] 视频画面时间区间：00:33:00–00:33:15。若为裁剪图，时间区间仍对应原视频画面。
[171] 视频画面时间区间：00:35:30–00:35:45。若为裁剪图，时间区间仍对应原视频画面。

$$B_{\text{eff}} = \frac{n}{T(n)} = \frac{n}{T_0 + n/B}$$

- B_{eff}：实际达到的带宽；
- n：本次传输的字节数；
- 分母包含固定开销，因此 B_{eff} 总是低于名义带宽 B；
- 当 n 很大时 B_{eff} 才趋近 B。

这就把”发小消息吃亏”变成可计算的量：消息越小，固定开销占比越高，有效带宽越低。

频繁的小消息是并行程序的隐形杀手

按上式，若每次只发几十字节，有效带宽可能只有名义带宽的百分之几。这在网格求解器里很容易发生：每个时间步都要交换边界，如果每步只交换一行而步数极多，T_0 就被重复支付了几千次。合并消息、减少同步次数，往往比优化计算本身收益更大。

最慢链路与流水线化通信

讲者进一步指出：通信系统里有快有慢，而吞吐通常由最慢的那条链路决定——就像洗衣例子里烘干机是关键瓶颈。

他画出一段通信的分解：橙色是建立调用（相当于 T_0）的开销，蓝色是最慢的那段链路，灰色是其余部分（下游链路与接收端固定成本）。从吞吐角度看，只有蓝色部分真正决定上限。

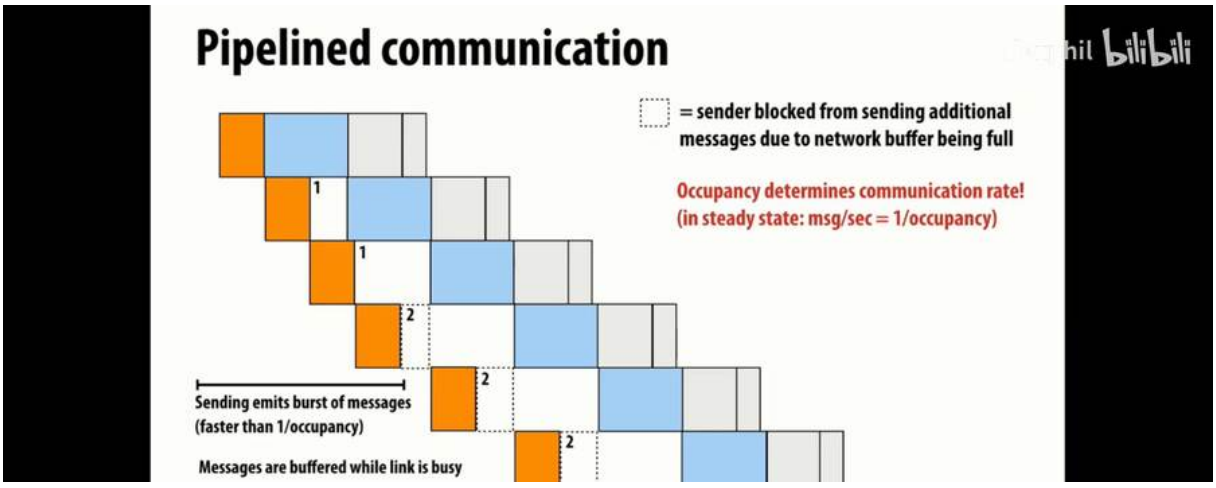

图 172：流水线化的通信：让最慢链路持续忙碌，其余开销被重叠[172]

好消息是可以在通信系统里也建流水线。之前之所以受限，是因为缺少足够多的离散部分来重叠，于是每次都重复支付开销。若能建立流水线，T_0 与下游操作都能与传输重叠，让最慢链路始终保持忙碌，有效带宽就趋近于那条链路的带宽。

[172] 视频画面时间区间：00:36:30–00:36:45。若为裁剪图，时间区间仍对应原视频画面。

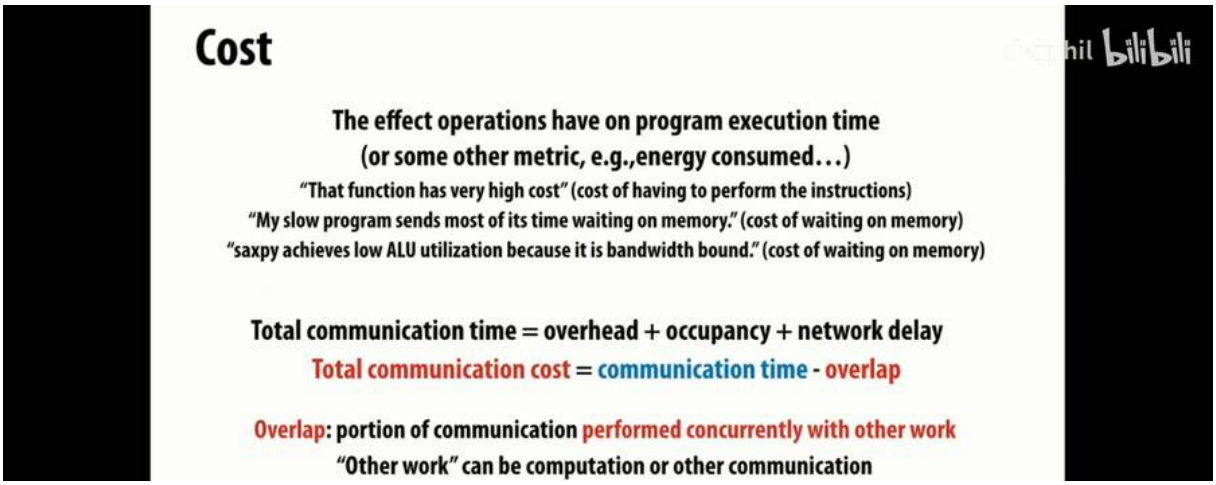

图 173: 成本的定义：各种操作对程序执行时间（或能耗）的影响[173]

流水线化通信的前提是消息可分割

把通信做成流水线要求消息能被切成多段，各段依次流过链路。这与”合并大消息”看似矛盾，实际解决的是不同层面的问题：合并处理 T_0 的摊薄，流水处理段间的重叠。两者可以并用——先合并成少数大消息，再让大消息内部流水传输。

本章小结

通信耗时可建模为固定开销加传输时间，有效带宽因此总低于名义带宽，小消息尤其吃亏。系统吞吐由最慢链路决定，而流水线化能让最慢链路持续忙碌、把其余开销重叠掉。这三条结论构成了后面所有优化的依据。

讲者在这里给出”成本”的总定义：成本是各种操作对程序执行时间的影响，也可以按能耗等其他资源来衡量。接下来要把并行系统放进内存层次的框架里看。

8.5 把并行系统看成扩展的内存层次

缓存既是资源也是成本

讲者提出一个统一的视角：通信在并行系统里扮演的角色，与缓存在单机里的角色同类。它提供了与其他线程交换数据的机制，这很有用；但同时也是一笔开销。

这解释了为什么我们总想最大化局部性——把正在操作的数据留在尽可能小、尽可能近、尽可能快的存储层次里。

[173] 视频画面时间区间：00:37:30–00:37:45。若为裁剪图，时间区间仍对应原视频画面。

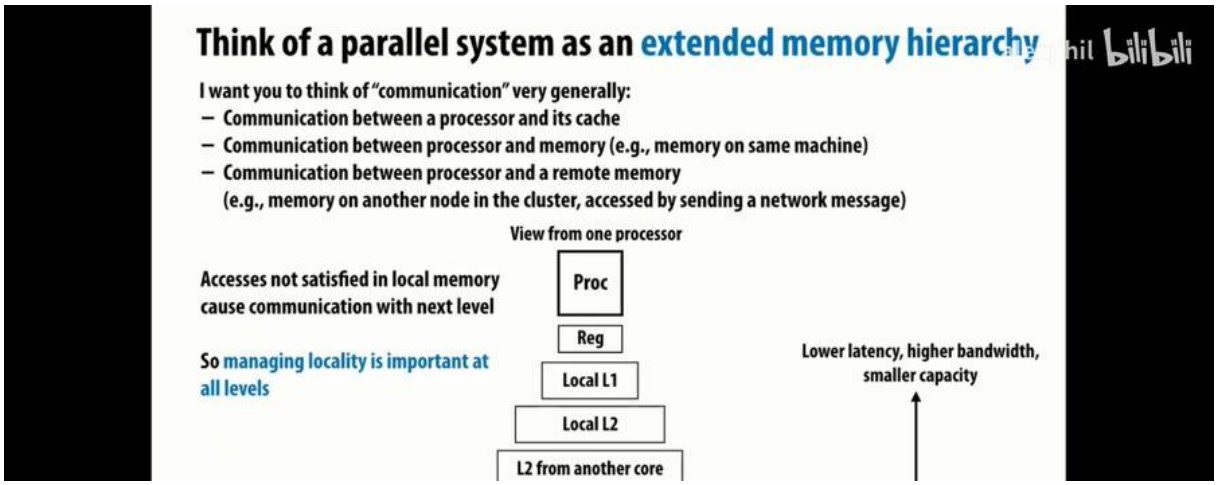

图 174: 把并行系统看作扩展的内存层次：越远的存储越慢但容量越大[174]

放大到大规模系统，除了本地内存，还有别的处理器所管理的内存——它们仍然可访问，只是更慢、容量更大。再往外，就是必须通过消息传递才能拿到的、存放在其他处理器上的信息。

无论哪一层，规律一致：越远越慢，越多越大。

固有通信与人为通信

讲者给出本讲最重要的分类。

固有通信是算法本身要求发生的通信。在网格求解器里，必须把相邻块的那一行数据传过去，否则算不出来。这部分无法消除。

人为通信是实现方式引入的、本可以避免的通信。它可能来自硬件设计、软件设计或系统设计的任何一部分，但在理想世界里本该不存在。

区分固有与人为通信决定优化方向

固有通信只能通过改变算法或分块方式来减少，例如换一种划分让边界更短。人为通信可以通过改实现来消除，例如提高传输粒度、调整数据布局、避免伪共享。把两者混为一谈，就会在错误的方向上投入——试图用调参解决算法层面的通信量。

[174] 视频画面时间区间：00:39:30–00:39:45。若为裁剪图，时间区间仍对应原视频画面。

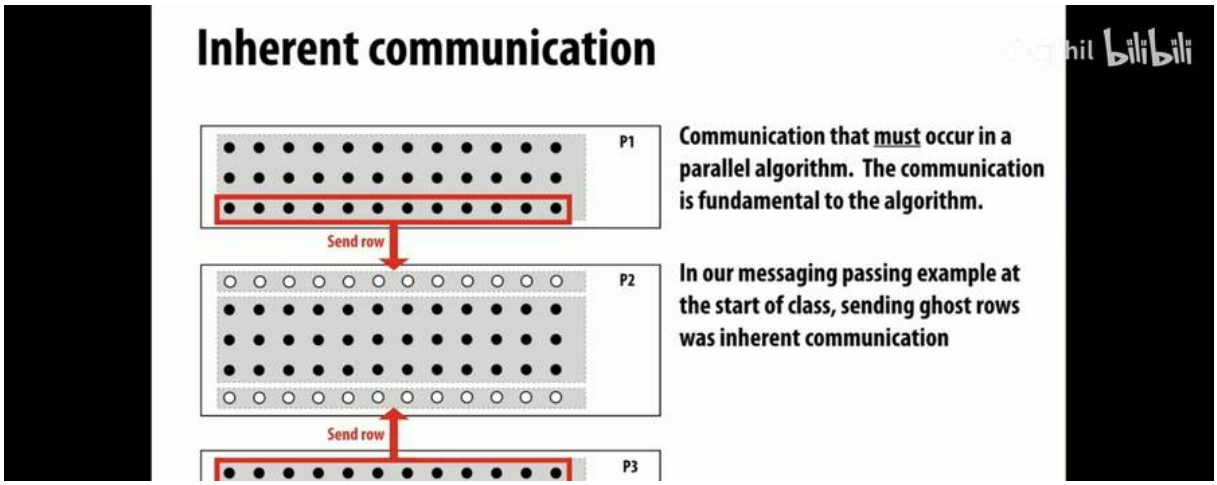

图 175: 固有通信：并行算法中必然发生的通信[175]

算术强度：把通信与计算放在一起看

讲者引入一个常用的比率：通信量与计算量之比。他习惯把它翻过来，称为算术强度：

$$\text{AI} = \frac{\text{计算量}}{\text{通信量}}$$

- 分子是完成的有用计算，例如浮点操作数；
- 分母是需要搬运的数据量，例如字节数；
- 比值越高，说明每一次数据搬运换来了更多计算，通信越不构成瓶颈。

他举了 SAXPY 作为反面例子：单精度 $a \times x + y$ 只做两次浮点操作（一次乘、一次加），却要读两个源向量、写一个目标向量。按任何口径算，都是每三次内存操作换两次算术操作。讲者的评语是：这不太好。

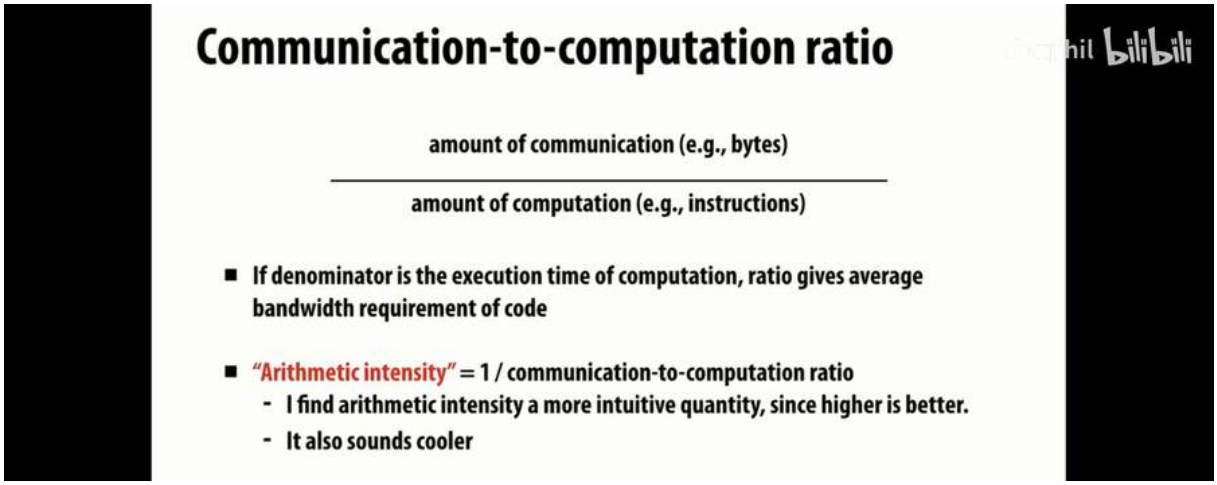

图 176: 通信与计算之比：算术强度的定义[176]

[175] 视频画面时间区间：00:42:30–00:42:45。若为裁剪图，时间区间仍对应原视频画面。
[176] 视频画面时间区间：00:43:00–00:43:15。若为裁剪图，时间区间仍对应原视频画面。

本章小结

并行系统可以视为扩展的内存层次：越远越慢、越大。在这个框架下，通信分为算法必然要求的固有通信与实现引入的人为通信，前者只能靠改变分块减少，后者可以靠改实现消除。算术强度把计算量与通信量放在同一个比值里，是判断通信是否构成瓶颈的通用指标。

有了这个指标，就可以比较不同分块方式的优劣。这正是下面两章的内容。

8.6 分块形状如何决定算术强度

行分块

讲者回到网格求解器，假设是 $N \times N$ 的网格，按行分给 P 个处理器。

每个处理器分到的计算量是 N^2/P。而它需要与相邻线程交换的数据是 $2N$——上下各一行。

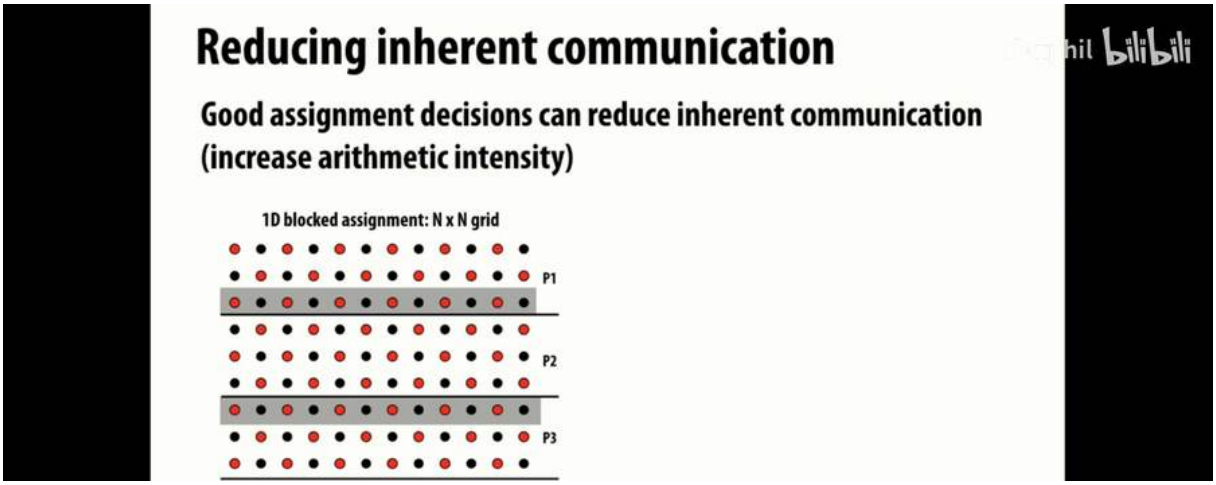

图 177: 减少固有通信：分配决策可以改变通信量[177]

于是算术强度为：

$$\mathrm{AI}_{\mathrm{row}} \propto \frac{N^2/P}{2N} = \frac{N}{2P}$$

- 分子 N^2/P：每个处理器负责的元素数；
- 分母 $2N$：交换两行所需的元素数；
- 结果与 N/P 成正比。

好消息是 N 越大，算术强度越高。直觉上也讲得通：N 大时每个处理器分到很多行，其中大部分是完全本地的，根本不需要通信。

坏消息是 P 越大，算术强度越低。讲者说这通常是普遍规律：越想并行，算术强度就越差。

交叉分配：一个反面教材

讲者接着分析另一种分配：让每个处理器负责第 1 行、第 4 行、第 5 行等等，交错分布。

每个处理器的计算量仍然是 N^2/P。但通信量呢？学生很快答出来：几乎是全部。

理由是：每算一个元素，都要把它的值向上和向下各传一次。因此算术强度只有约 0.5。

[177] 视频画面时间区间：00:44:30–00:44:45。若为裁剪图，时间区间仍对应原视频画面。

$$\mathrm{AI}_{\text{interleaved}} \approx \frac{1}{2}$$

这个数字与 N、P 都无关，恒定为一个极差的值。讲者的结论很直接：这是个很糟糕的主意。

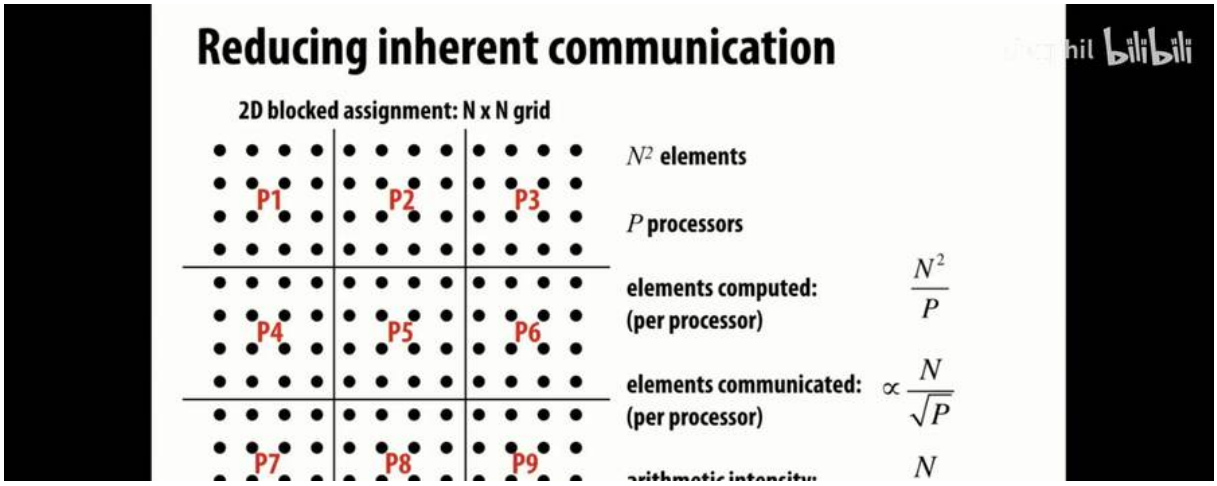

图 178：二维分块分配：用面积换周长，大幅减少边界通信[178]

方形分块：面积与周长之比

讲者问：能否做得比行分块更好？他提示想象水滴——表面张力会让水滴倾向于最小化表面积与体积之比。

答案是把网格分成方形的块，而不是长条。

这样每个处理器分到的计算量不变，但通信量正比于块的周长，而周长随面积的平方根增长。因此算术强度变成：

$$\mathrm{AI}_{\text{2D}} \propto \frac{N^2/P}{\sqrt{N^2/P}} = \frac{N}{\sqrt{P}}$$

- 通信量正比于周长，即 $\sqrt{N^2/P} = N/\sqrt{P}$；
- 结果与 $N/\sqrt{P}$ 成正比；
- 与行分块的 $N/(2P)$ 相比，多出了一个 $\sqrt{P}$ 的优势。

讲者的评价是这对扩展性很有利：随着处理器增加，性能受到的负面影响逐步变小。

通信正比于边界、计算正比于面积

这条原则可以推广到任意维度：如果计算量随体积增长而通信量只随表面积增长，那么块越大，算术强度越高。三维时通信正比于边界面（面积的平方根关系变成 $P^{2/3}$），四维更好。因此分块形状的选择本质上是在选择一个几何量——在固定体积下让表面积最小。

[178] 视频画面时间区间：00:48:30–00:48:45。若为裁剪图，时间区间仍对应原视频画面。

固有通信也可以减少

讲者在这里做了一个重要的澄清：虽然这部分通信是”固有的”，但并不意味着不能对它做优化。

改变分块形状就是减少固有通信的手段：通信量仍然由算法决定必须有，但不同划分下它的总量不同。所以”固有”指的是不可消除，而不是不可优化。

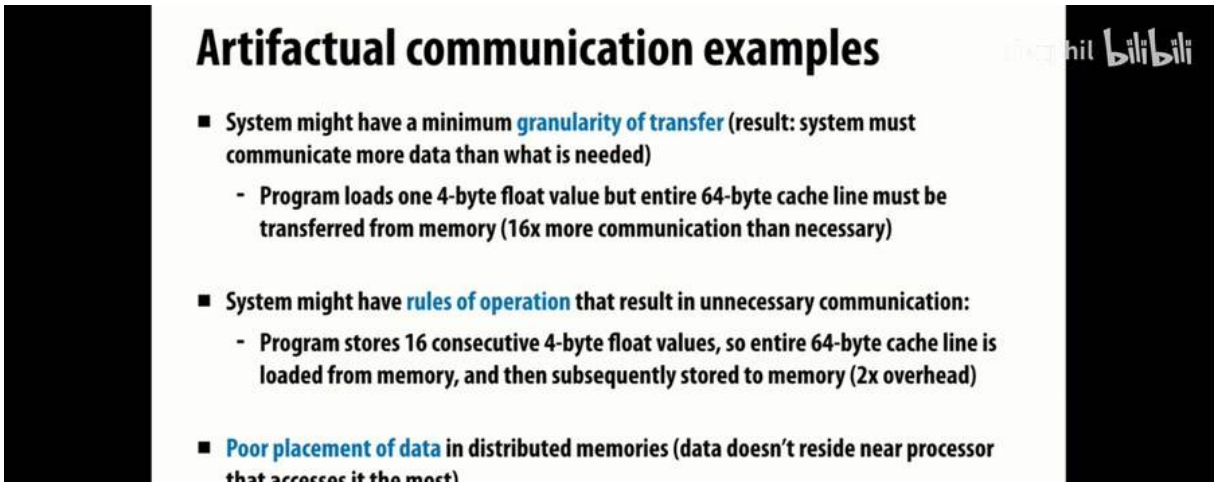

图 179：人为通信的例子：最小传输粒度等带来的额外搬运[179]

本章小结

按行分块时算术强度正比于 $N/(2P)$，交叉分配只有约 0.5，方形分块则是 $N/\sqrt{P}$。差别来自通信量正比于边界而计算量正比于面积。固有通信虽然不可消除，但可以通过改变分块形状来减少总量。

形状问题解决之后，还有一类通信与算法无关，纯由实现细节产生。那是下一章的内容。

8.7 人为通信：缓存行、缺失与遍历顺序

最小传输粒度

讲者先给出最普遍的一种人为通信：系统往往有最小传输粒度，例如一条缓存行。

即使只想读其中的一个字节，也要把整行搬过来。如果不巧只有这一个字节有用，这份额外的搬运就完全是浪费。

写的情况更糟。讲者举了 SAXPY 的例子：即使你直接指向目标地址去写，每次进入新的一行时系统仍要先把它读进来，因为它并不知道你即将写满整行。于是第一次写操作会把原有数据从内存搬到缓存，再更新、再写回。这些旧数据你根本用不上。

写分配导致的读放大

写命中的前提是数据已在缓存里，因此写缺失会先触发一次读填充。对连续写满整个缓存行的场景，这次读完全是多余的。避免办法是让写操作按完整缓存行对齐、成批进行，或用非临时写指令绕过缓存填充。这类浪费在计数器、标志位这类零散写入上尤其明显。

[179] 视频画面时间区间：00:50:30–00:50:45。若为裁剪图，时间区间仍对应原视频画面。

第四类缺失：通信缺失

讲者回顾了从 15-213 课程就该掌握的三类缺失。冷缺失是第一次访问某数据，缓存里没有，通常无法避免。容量缺失是工作集太大装不进缓存。冲突缺失是缓存的组织方式（例如组相联索引）导致它没能选到最好的淘汰对象，从纯程序角度看本不该发生。

现在加入第四类：通信缺失。指在系统更高层次上，我们付了本不需要的额外通信代价。

讲者提醒这些定义在细究时会互相重叠。

从工作集看通信

讲者用一张图把通信放在工作集框架里：系统有一部分能以很高带宽工作，当工作集超出它的容量就掉进更慢的资源，依此类推。最底下是本来就无法避免的固有通信与冷缺失。

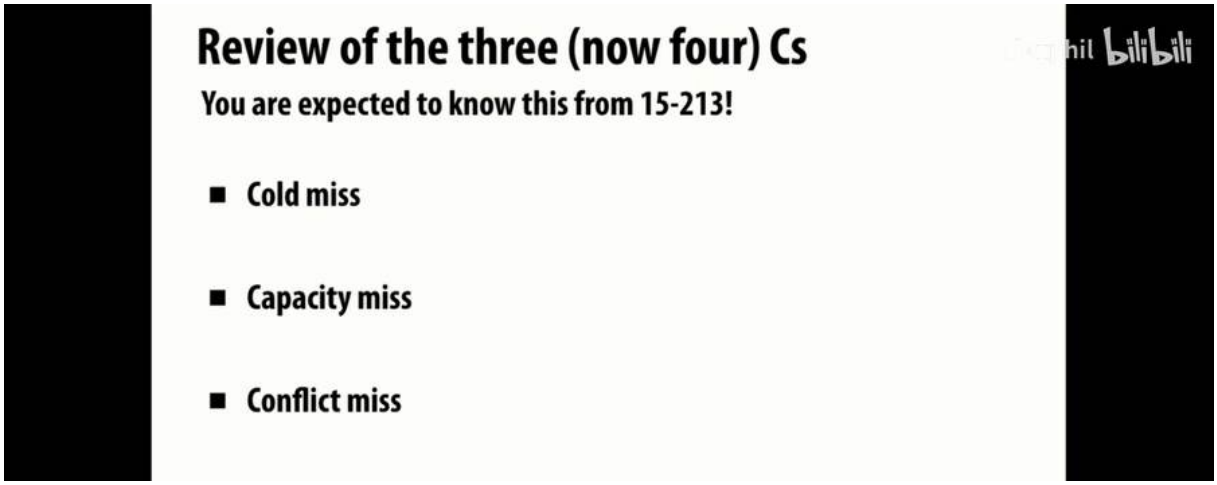

图 180: 回顾四类缺失：冷缺失、容量缺失、冲突缺失与通信缺失[180]

他指出这张图其实与之前的内存山是同一件事：取一个截面，横轴是工作集大小，小工作集留在 L1 里飞快，变大就掉进 L2、L3，最后卡在内存。

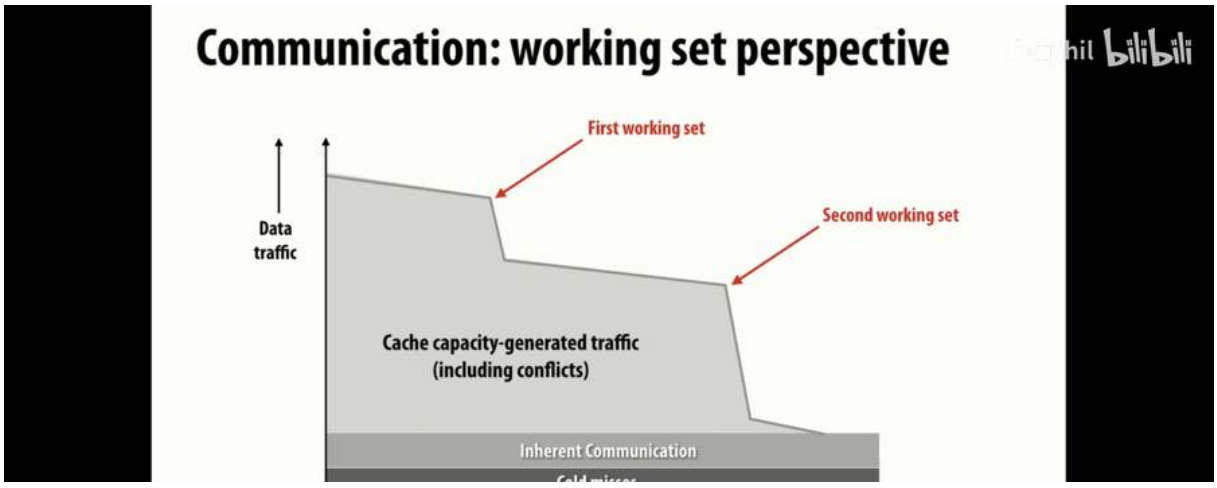

图 181: 从工作集角度看通信：跨越层次时成本骤升[181]

[180] 视频画面时间区间：00:52:30–00:52:45。若为裁剪图，时间区间仍对应原视频画面。

[181] 视频画面时间区间：00:54:30–00:54:45。若为裁剪图，时间区间仍对应原视频画面。

行主序遍历的容量缺失

讲者用一个具体例子把上面说得抽象的结论落地。假设按行主序访问网格：访问 [0] → [1] → [2] → [3]，然后回到下一行。

若缓存能装 24 个元素，且按每 4 个元素一组对齐，那么遍历完一整行再回到下一行开头时，第一行开头的数据已经被挤出去了——它在完成上一行时被淘汰。

于是每次跨行都恰好缺失一次。讲者的评价是：空间局部性还算不错，一次缺失三次命中；短期时间局部性也可以。但正是因为容量缺失，跨行时无法享受复用。

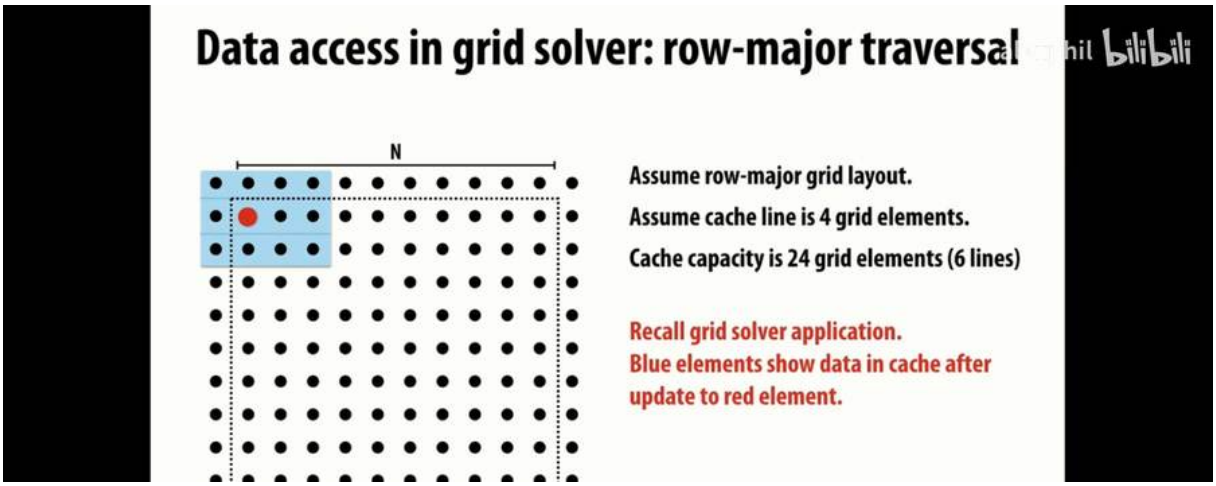

图 182: 网格求解器的行主序访问：每次跨行都要重新缺失[182]

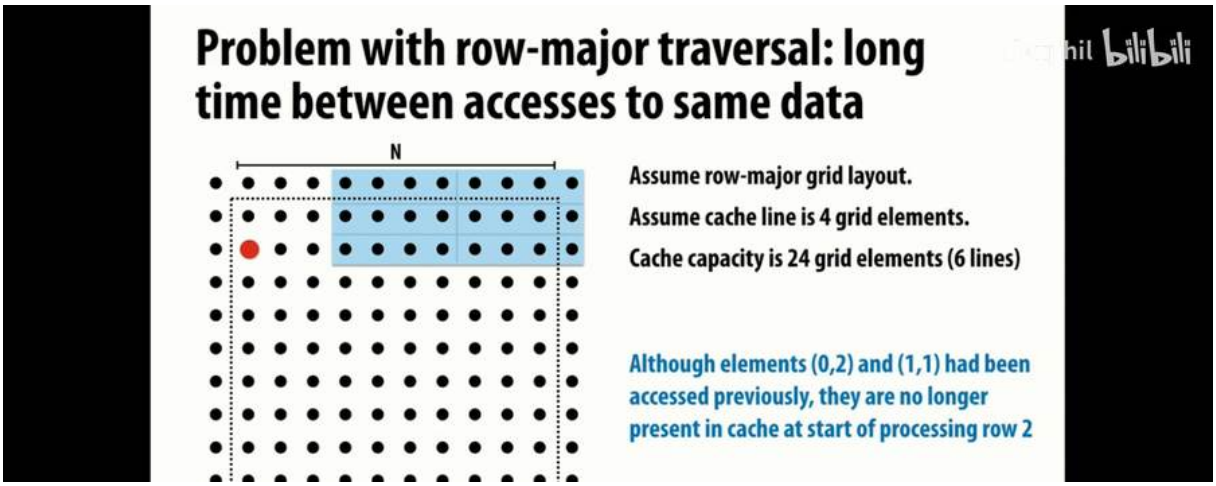

图 183: 行主序的问题：同一数据两次使用之间间隔太长[183]

改遍历顺序与循环融合

解法是改变访问顺序：先处理前几列，让这些列的上下行复用发生在短时间内，再回来处理剩下的列。行主序或列主序都可以，关键在于不要按标准顺序傻走。

[182] 视频画面时间区间：00:57:00–00:57:15。若为裁剪图，时间区间仍对应原视频画面。

[183] 视频画面时间区间：00:58:00–00:58:15。若为裁剪图，时间区间仍对应原视频画面。

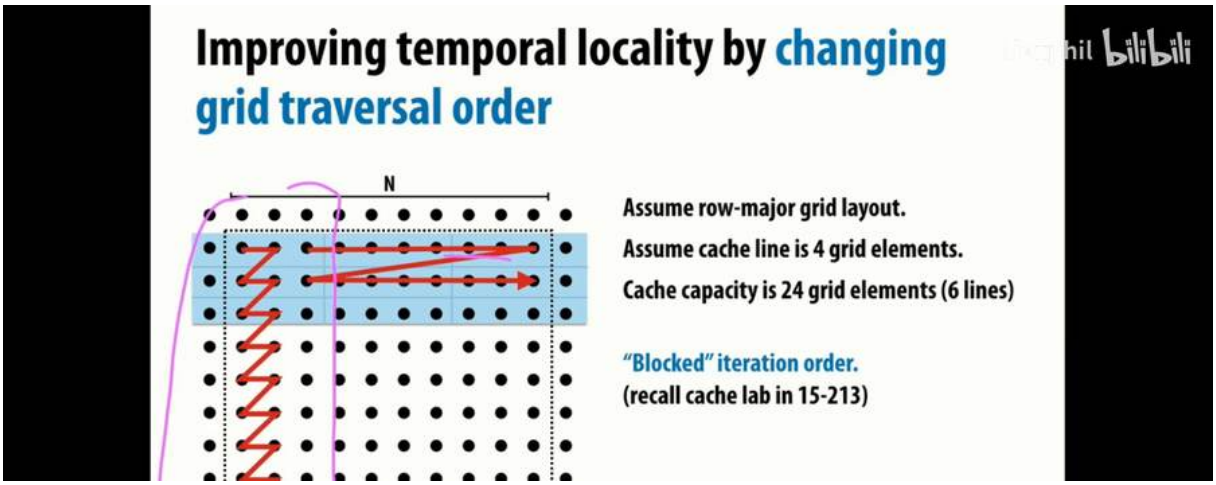

图 184: 改变网格遍历顺序以改善时间局部性[184]

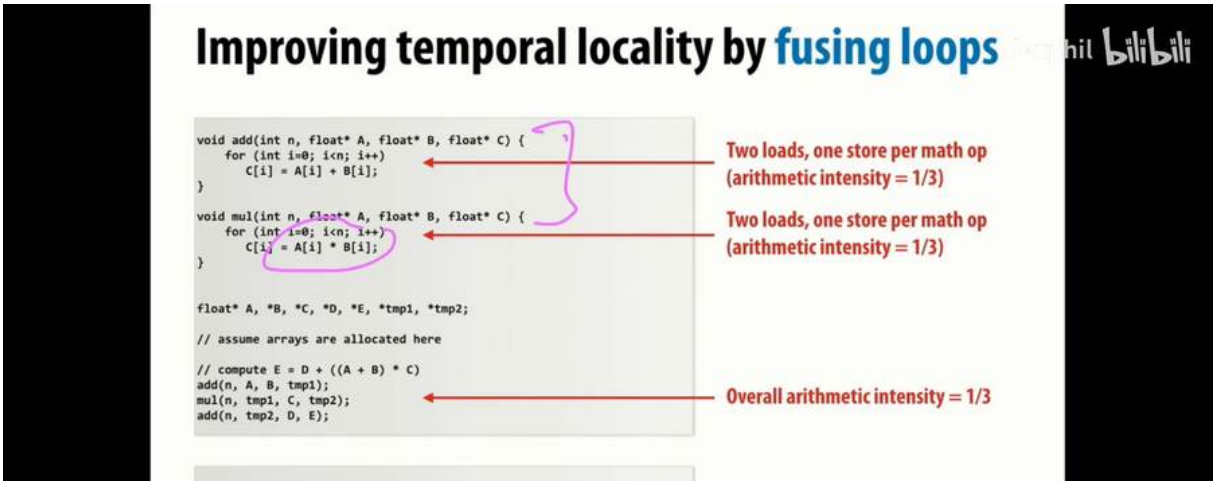

图 185: 循环融合改善时间局部性：把两个遍历合成一个[185]

另一个手段是共享数据以提高算术强度：把操作同一批数据的任务安排在一起，让这份数据只被搬一次。

[184] 视频画面时间区间：00:59:30–00:59:45。若为裁剪图，时间区间仍对应原视频画面。
[185] 视频画面时间区间：01:01:00–01:01:15。若为裁剪图，时间区间仍对应原视频画面。

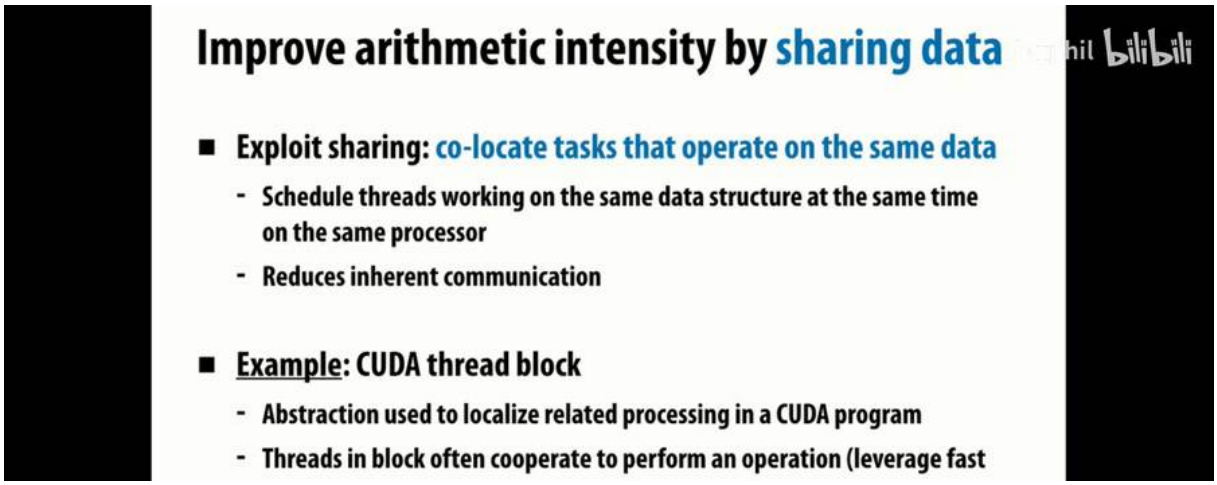

图 186: 共享数据以提高算术强度[186]

数据并行写法与性能的张力

讲者在这里指出了一个现实矛盾。用高度抽象的方式写代码很舒适：想问题时就想着数组级别的操作。比如 Python 的 numpy 里直接写 `A + B`，三维数组也知道你要逐元素相加。CUDA 的线程启动也可以这样想：先做一次向量加，再做一次向量加，读起来很优雅。

问题是这种写法从性能角度看很糟。

讲者的解释是：这类抽象把”数据在哪、怎么搬”完全隐藏了，而通信与局部性恰恰是决定性能的全部。因此数据并行的优雅与性能之间常常需要取舍。

图 187: 利用空间局部性：通信粒度也很重要[187]

本章小结

人为通信来自最小传输粒度（读一个字节要搬整行、写之前先读入旧行）与布局不佳。第四类缺失——通信缺失——把这类浪费纳入缺失分类。行主序遍历会导致跨行的容量缺失，改变遍历顺序或融合循环可以修复。而抽象层次越高，越容易在无意中制造这类浪费。

[186] 视频画面时间区间：01:03:30–01:03:45。若为裁剪图，时间区间仍对应原视频画面。

[187] 视频画面时间区间：01:05:30–01:05:45。若为裁剪图，时间区间仍对应原视频画面。

8.8 竞争与伪共享

伪共享的机制

讲者在最后讨论了另一类人为成本：竞争，也就是多个线程同时访问同一个共享资源。

其中最容易忽视的一种叫伪共享：不同线程操作的数据虽然不同，却恰好落在同一条缓存行里。缓存行是失效与传输的单位，因此一个线程写自己的数据，会让另一个线程持有的同一行副本失效，迫使它重新取数。两个线程之间其实没有任何数据依赖，却在缓存协议层面互相干扰。

伪共享的判据是缓存行而不是变量

判断是否存在伪共享，要看多个线程的写目标是否落在同一缓存行，而不是看它们是否访问同一个变量。两个完全无关的计数器若相邻存放，同样会互相拖慢。解决办法是填充（padding）或按缓存行对齐分配，把每个线程的写目标推到独立的行上。

原子操作与热点

讲者提到另一类问题：用原子操作做累加之类的更新时，所有线程都会打向同一个地址，形成热点。硬件必须把这些操作串行化，代价随线程数上升。

给出的对策是避免原子操作，改用更高层次的并行原语：先让各线程算局部结果，最后再合并。这与前面反复出现的”部分和加归约”是同一个模式。

全局计数器是并行程序的典型瓶颈

每个线程都去自增一个全局计数器，等于把所有线程串行化在同一个地址上。随着线程数增加，这份争用会成为无可避免的上限。可行替代是每线程一个局部计数器、最后汇总，或使用分片计数器（sharded counter）把争用分散到多个地址。

本章小结

竞争与伪共享属于人为成本中最隐蔽的一类：它们不会让结果出错，只让性能莫名下降，且难以从代码表面看出来。判据是缓存行与共享地址，而非变量与逻辑依赖。避免手段是分散写目标、用局部累加替代原子操作。

讲者最后的时间分配说明这一讲已覆盖了成本的主要来源与对策。

8.9 总结与延伸

讲者收尾时说：我们走过了成本的各个部分，以及如何减少它们、提高性能。

把全讲串起来，是一条从”通信不可避免”到”如何把它压到最低”的完整链条。通信成本足以抹掉并行收益，而它在共享内存、消息传递、GPU 三种形态中本质相同，只是距离不同。消息传递把这一点暴露得最清楚：按行分块后每个线程必须复制邻居边界，收敛判断还需要一次全局归约，而阻塞发送若不配奇偶交替就会死锁。延迟与带宽必须分开看待——公路类比说明增加并发可以只提升吞吐；洗衣例子给出流水线的核心结论：吞吐上限由最慢阶段决定。通信耗时可建模为 $T_0 + n/B$，因此合并消息能摊薄固定开销，有效带宽总低于名义带宽。并行系统可视为扩展的内存层次，通信分为算法必然要求的固有通信与实现引入的人为通信；算术强度把两者放在同一个比值里。分块形状决定算术强度：行分块是 $N/(2P)$，交叉分配只有 0.5，方形分块是 $N/\sqrt{P}$，差别来自通信正比边

界而计算正比面积。最后，最小传输粒度、遍历顺序、伪共享与原子操作构成了人为成本的主要来源。

本讲义在讲者材料之上做的压缩，有四点需要与讲者原话区分：

- 通信优化的抓手是几何量而非代码细节。分块形状决定边界长度，边界长度决定通信量。这比微调传输参数有效得多，也解释了为什么行分块与方形分块的差距会随处理器数扩大。
- 固定开销 T_0 的代价随调用次数放大，而不是随数据量放大。因此减少同步次数、合并消息，往往比增大带宽更直接。这与 Amdahl 定律的结构相同：不能并行的那部分反复出现，就是上限。
- 固有通信可以优化但不可消除。改变划分能让它变少，但算法要求的跨块数据依赖始终存在。设计时应先承认这部分不可避免，再看剩余空间。
- 抽象层次越高，越容易制造人为通信。numpy 式数组操作与 CUDA 的高层线程启动都很优雅，但它们隐藏了数据布局，而布局恰恰决定通信量。

这堂课的边界在于它讨论的是成本的结构，而不是某个具体系统的调优手册。讲者明确表示伪共享、原子操作这些细节会在后续课程与作业中继续展开。

两个可以直接从本讲数据提出的下一步问题：其一，方形分块需要处理器数能凑成完全平方，实际划分时如何处理不能整除的情形，代价有多大？其二，既然算术强度随 P 增大而下降，那么强扩展（固定问题规模加机器）本质上必然失效，弱扩展（问题规模随机器增长）是否才是唯一可行的路径？这两个问题都指向接下来的缓存一致性与扩展性讨论。

Lecture 09 Parallel Programming Case Study

视频时长：1 小时 20 分 47 秒　来源：https://www.bilibili.com/video/BV1ej41167sy/

9.1 从原理转向案例

这一讲开始用真实程序演示前几讲的原理如何落地。讲者把课程位置交代得很清楚：到目前为止讨论的是一般原则——如何思考并行计算、它的执行部分与通信部分；接下来用一些例子说明这些问题在不同领域是怎么被解决的，然后才进入如何测量与评估系统性能。

两个核心关切

讲者点明这一讲要看的重点：这些程序用了什么技术来提取并行、处理我们已经谈过的问题。最主要的两个是保证处理器之间有合理的负载均衡，以及避免通信带来的过高开销。

图 188: 课程路线图：从原理转向案例研究[188]

案例的来源

这些案例来自课程网页上列出的一本教材。讲者说那本书已有二十年历史，但其中的原则与概念在今天是普遍有效的。

他的判断值得记录：除了 GPU 的出现之外，并行计算领域在过去二十年里概念上没有太大变化，变的只是可行的规模。

[188] 视频画面时间区间：00:03:00–00:03:15。若为裁剪图，时间区间仍对应原视频画面。

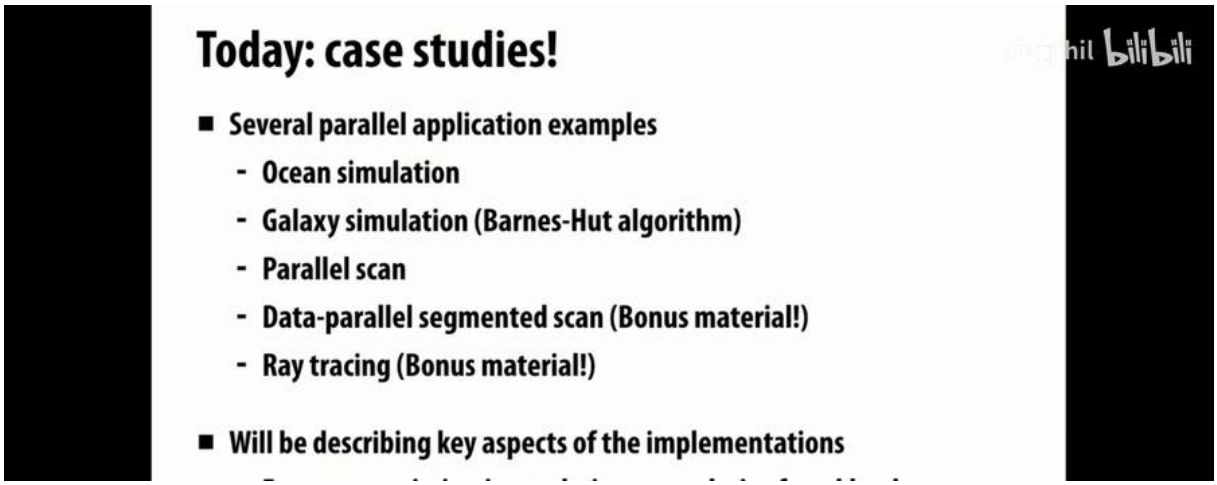

图 189: 今日主题：几个并行应用案例[189]

今天要看的有四个：海洋模拟（典型的网格求解问题）、星系建模（不规整的空间划分问题）、从扫描推广到分段扫描、以及光线追踪与图形学。

本章小结

这一讲用真实程序演示两个核心关切：负载均衡与通信开销。案例取自二十年前的教材，但概念至今有效——除 GPU 外，并行计算的变化主要在规模而非原理。

下一章先明确硬件假设，因为后面的划分策略都建立在这个假设上。

9.2 硬件假设：NUMA

统一的思考模型

讲者说为了讨论方便，请大家统一按一个模型思考：大规模共享内存多处理器。也就是说，通信上可以寻址整个内存空间。

图 190: 假设：共享地址空间的多处理器[190]

[189] 视频画面时间区间：00:04:00–00:04:15。若为裁剪图，时间区间仍对应原视频画面。

[190] 视频画面时间区间：00:05:00–00:05:15。若为裁剪图，时间区间仍对应原视频画面。

但这个共享内存是 NUMA 的——非一致内存访问，意思是某些部分的内存比其他部分更容易、更快地访问到。

原因有两层：一是缓存效应，二是物理构造方式。一旦超出单芯片多核处理器，通常的做法是各个处理器分别管理一段 DRAM，可以跨过去请求访问，但要经过另一颗芯片的 DRAM 控制器。

NUMA 让”内存”这个词失去单数含义

在 NUMA 机器上，一次访问的代价取决于地址落在哪个节点。这意味着数据布局不只是缓存友好性问题，而是直接决定了通信是否跨越慢速链路。因此后面所有划分策略的共同目标都是让大部分访问落在本地节点。

具体机器

讲者给出下一份作业会用到的集群：每个处理器是 12 核 Xeon，但物理上其实是两颗芯片。因此一半内存比另一半更快。

NUMA 是共享内存与消息传递之间的过渡形态

共享内存中一次访问的代价由地址决定；消息传递中一次通信的代价由显式发送决定；NUMA 介于两者之间——编程模型仍是共享地址空间，但代价已随位置变化。因此 NUMA 上的优化技术（本地性优先、按节点划分）在消息传递中大多仍适用。

他也说明这些想法可以推广到消息传递、分布式地址空间，以及在较小程度上推广到 GPU。

本章小结

硬件假设是大规模共享内存多处理器，且为 NUMA：部分内存访问更快。原因是缓存效应与物理构造——跨芯片访问要经对方的 DRAM 控制器。集群中每个处理器是 12 核 Xeon 但物理上两颗芯片，因此一半内存更快。这些思想可推广到消息传递与分布式地址空间。

9.3 案例一：海洋模拟

问题设定

图 191：模拟洋流：把海洋离散成二维网格，只取一个水平切片[191]

目标是模拟洋流。讲者说像许多这类问题一样，我们不必真的懂背后的科学，只需对计算类型有感觉。

做法与之前看过的类似：把空间离散成网格，只考虑海洋的一个水平切片——比如海面或某个特定深度。因此今天只把它当作二维问题。

要建模的是状态参数：温度、含氧量以及其他对海洋重要的量，看它们如何在时间与空间上演化。典型做法是设定一个小的时步 Δt，每隔 Δt 更新一次系统状态。

为什么这个问题难以扩展

讲者给出一个关键约束：对任何物理模拟来说，想要更细的分辨率、更细的空间分布，就必须同时缩短 Δt。

原因是 Δt 必须小到让效应在一个时间步内不传播超过最近邻。

他举了一个同行的例子：做地面运动、地震类模拟的人说，每当他们想把空间分辨率提高一倍，就需要 16 倍的算力。这是因为要在三个空间维度上都变细，同时时间也要变细。

$$\text{算力} \propto \left(\frac{1}{\Delta x}\right)^3 \cdot \frac{1}{\Delta t}$$

- Δx：空间步长，三个维度同时缩小；
- Δt：时间步长，受 Δx 约束而必须同比缩小；
- 若 Δx 减半，则空间项变为 $2^3 = 8$ 倍，时间项变为 2 倍，合计 16 倍。

分辨率提升的代价是四次的而非线性的

这是所有物理模拟共同面对的现实：提高精度不是多花一点算力，而是按四次方增长。这解释了为什么这类问题”非常难以扩展”，也解释了为什么超算排名长期被这类应用主导。

核心计算与两类工作

讲者说只关注图中标红的部分，也就是网格求解部分。它是我们上次讨论内容的稍复杂版本：在每个空间点上，计算该点与四个邻居的平均值。

当然实际计算比求平均更有意思，但这个计算是”对系统状态及其演化建模”这类工作的良好代表。典型做法是迭代直到满足某个收敛条件。

[191] 视频画面时间区间：00:06:30–00:06:45。若为裁剪图，时间区间仍对应原视频画面。

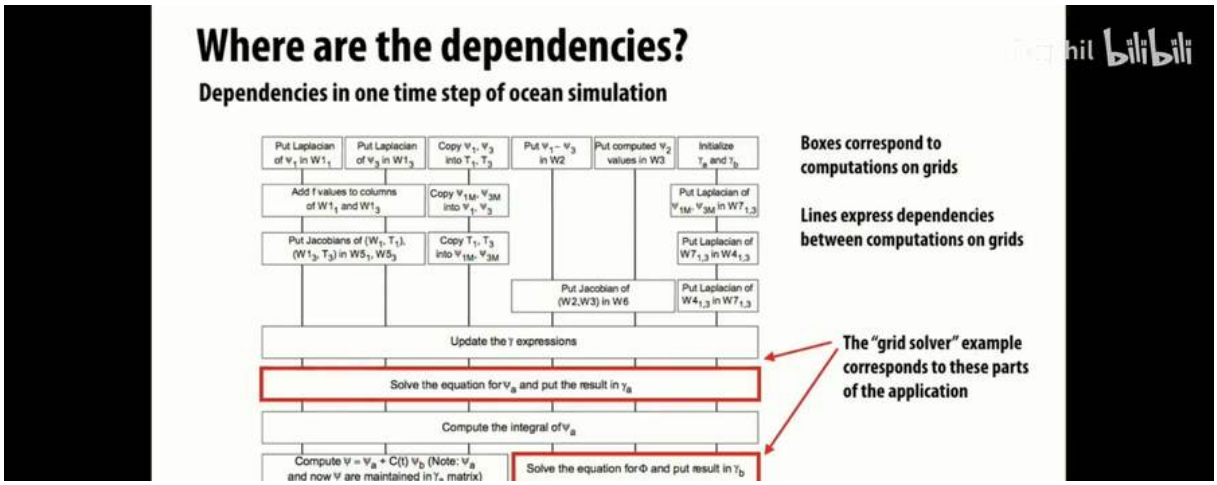

图 192: 依赖在哪里：一个时间步内海洋模拟的依赖关系[192]

与之前一样，这里有两类计算：一是更新每个元素；二是确定与上一步相比的变化量，并把这个变化信息聚合到某个中心位置，由它判断模拟是否收敛。

本章小结

海洋模拟把空间离散成二维网格（实际是三维的一个切片），状态参数含温度、含氧量等，按小时步更新。关键约束是提高空间分辨率必须同时缩短时间步，因此分辨率翻倍需要 16 倍算力。核心计算是求该点与四邻居的平均值，代表状态更新类计算；每个时间步包含两类工作：逐元素更新，以及把变化量归约到中心判断收敛。

下一章讨论如何把这个二维网格划分到多个处理器上。

9.4 网格划分：面积与周长之比

为什么越方越好

讲者回顾上一讲的结论：最好的做法是把空间划分成瓦片，而且瓦片越接近方形越好。

通信量由边界数据量决定：

$$V_{\text{comm}} = P \cdot w, \qquad V_{\text{comp}} = A$$

- P：瓦片周长，即需要交换的边界长度；
- w：每个边界元素需要传输的字节数（含光晕层）；
- A：瓦片面积，即该处理器负责的元素数。

因此每字节计算量正比于 $A/(P \cdot w)$，最大化它就等于最小化周长面积比。

[192] 视频画面时间区间：00:08:30–00:08:45。若为裁剪图，时间区间仍对应原视频画面。

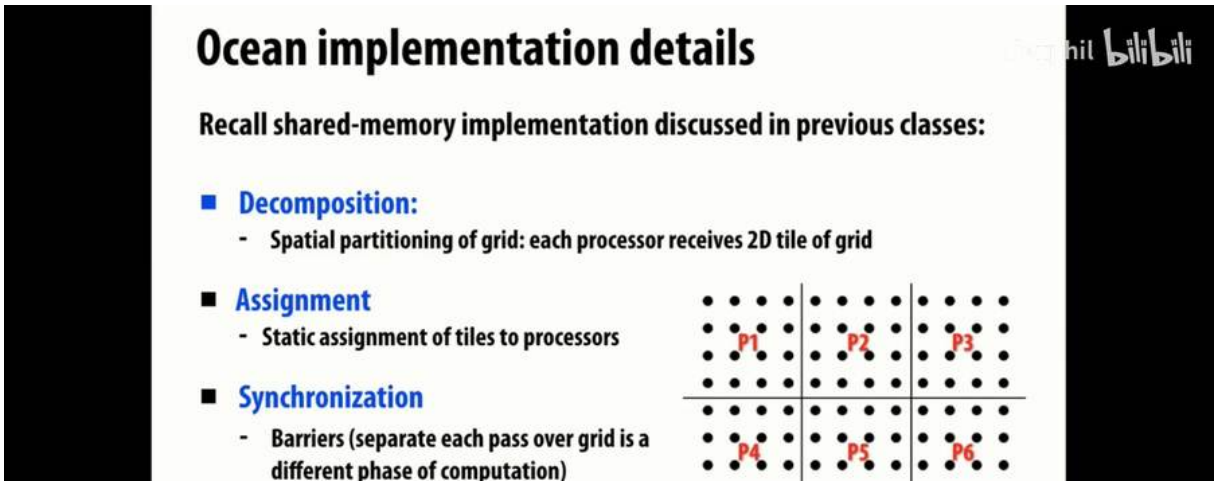

图 193: 海洋模拟的实现细节：共享内存实现要点[193]

理由很直接：计算量正比于面积，通信量正比于周长。因此要提高算术强度，就要让面积相对于周长尽量大，而正方形（或其他非常紧凑的形状）能最小化周长与面积之比。

$$\text{算术强度} \propto \frac{A}{P}, \qquad \left.\frac{A}{P}\right|_{\text{方形}} = \frac{s^2}{4s} = \frac{s}{4}$$

- A：瓦片面积，决定该处理器要做的计算量；
- P：瓦片周长，决定需要交换的边界数据量；
- s：方形瓦片的边长，因此瓦片越大，算术强度越高。

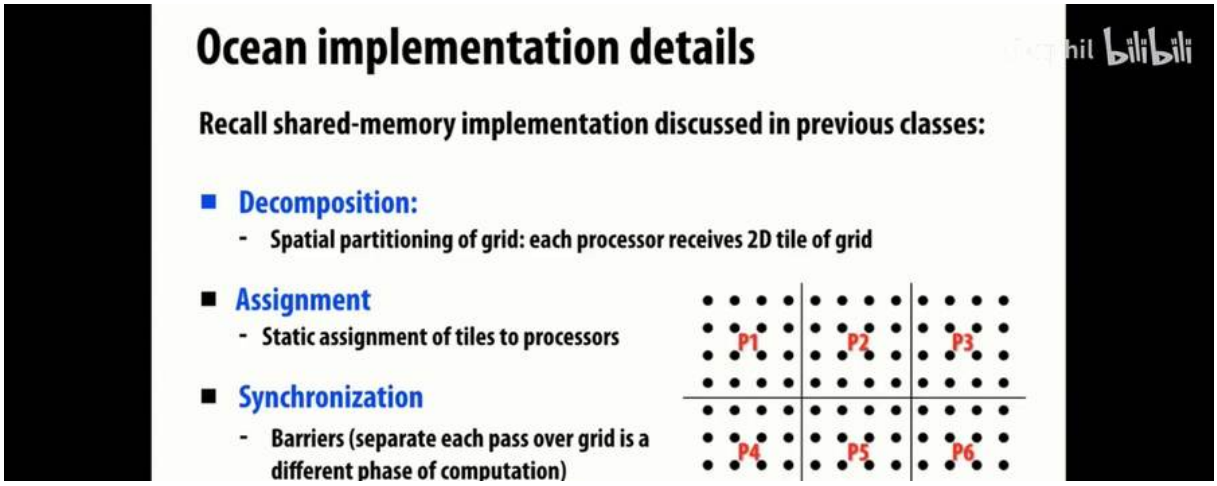

图 194: 实现细节续：分块与屏障同步[194]

[193] 视频画面时间区间：00:09:00–00:09:15。若为裁剪图，时间区间仍对应原视频画面。
[194] 视频画面时间区间：00:11:00–00:11:15。若为裁剪图，时间区间仍对应原视频画面。

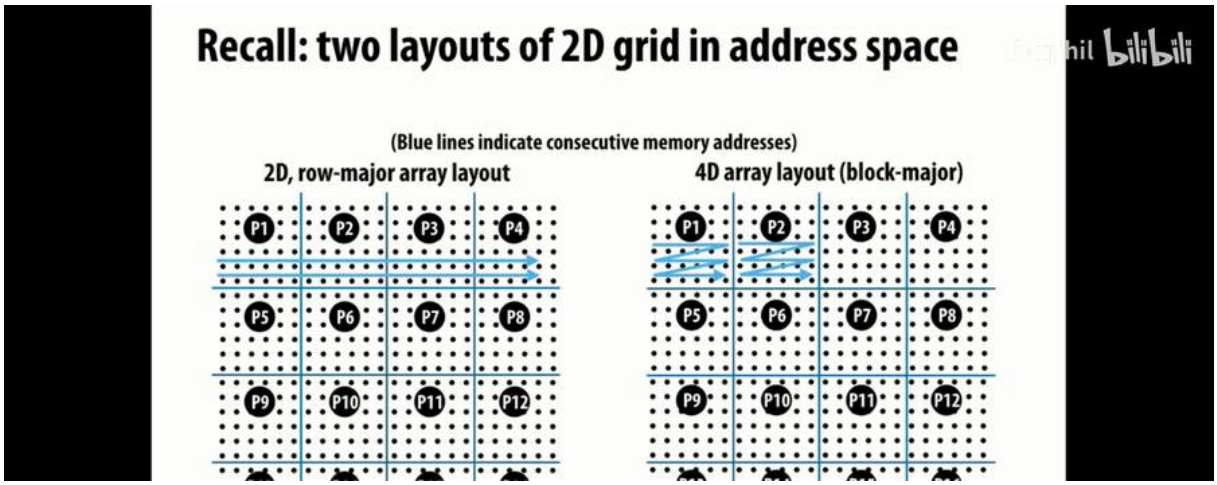

图 195: 回顾：两种静态划分方案——块划分与条带划分[195]

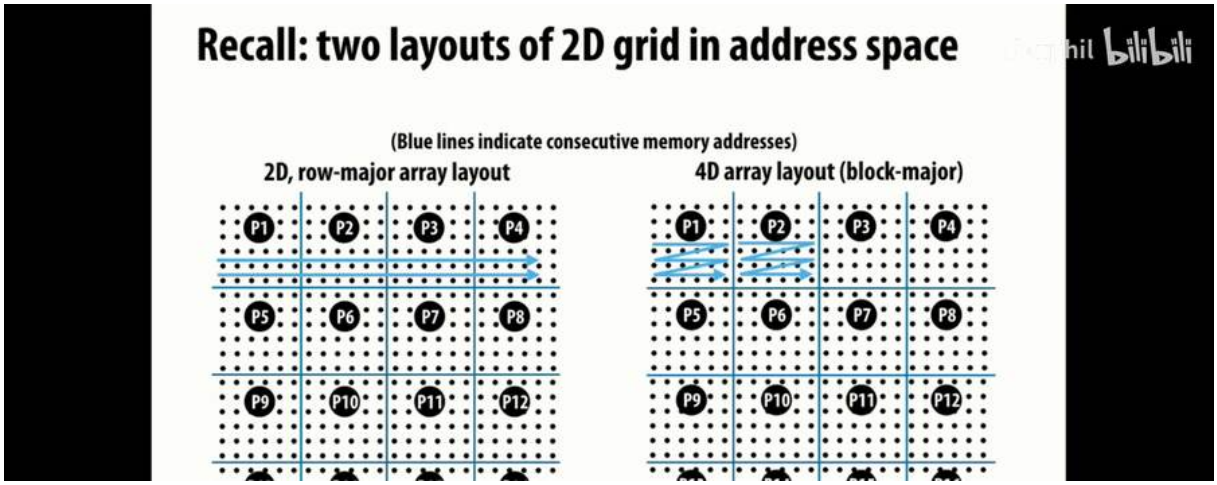

图 196: 回顾：二维网格在地址空间中的两种布局，蓝线表示连续地址[196]

屏障与掉队者

讲者接着讨论屏障同步的成本。它有两个代价。

第一是让所有人就”该停下来了”达成一致所需的时间。第二，若某个元素跑得比其他人慢一点——这叫掉队者（straggler）——它会迫使所有人按它的节奏走。

掉队者的代价随同步频率放大

若每步都同步，则总耗时约等于步数乘以最慢者的单步耗时。因此减少同步次数（增大每步工作量）可以把掉队者的影响摊薄，代价是并行粒度变粗、负载均衡更难。这是同步型应用的核心权衡之一。

讲者说这在这类计算中是个普遍的大问题。

[195] 视频画面时间区间：00:12:30–00:12:45。若为裁剪图，时间区间仍对应原视频画面。

[196] 视频画面时间区间：00:13:00–00:13:15。若为裁剪图，时间区间仍对应原视频画面。

屏障把最慢者的延迟放大为全局延迟

在每步都要同步的结构里，单步耗时不取决于平均值，而取决于最大值。因此负载不均的影响不是”稍微慢一点”，而是”整台机器按最慢分区的速度运行”。这解释了为什么网格划分要追求均衡，以及为什么后面 N 体的动态划分是必要的。

工作集的时间尺度

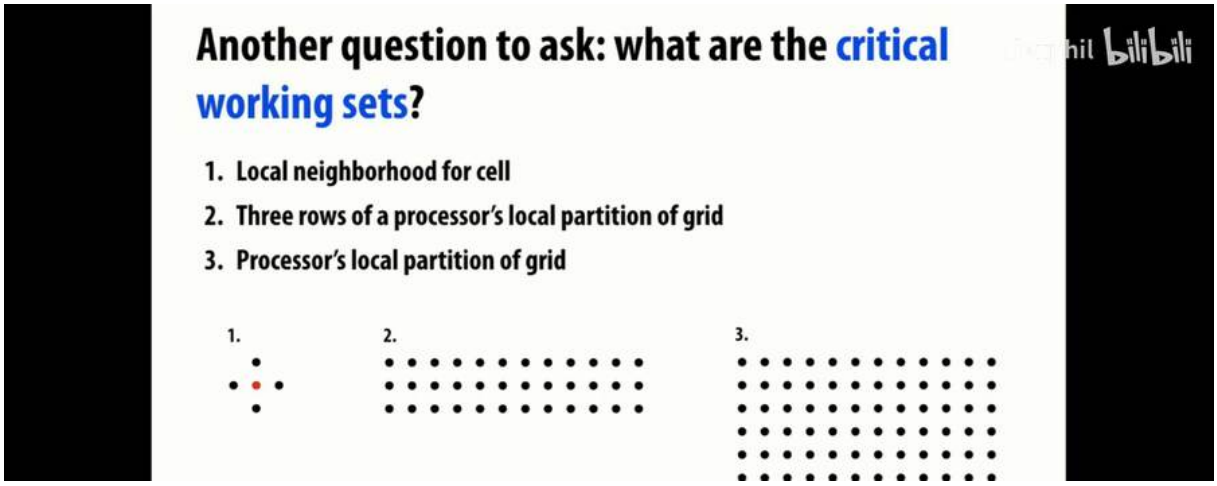

图 197：关键问题：工作集有多大，局部邻域与整块瓦片[197]

讲者提醒”工作集”这个词用得很含糊——它指你在某段时间内需要的内存或数据规模。但工作集大小取决于所讨论的时间尺度。

在这个例子里，最短时间尺度下每个处理器需要访问一个网格点及其四个邻居。若我们在行内扫描，那就是一种工作集。更一般地，某个给定瓦片内所有节点的状态集合也可视为工作集。

而缓存层次正是为应对”工作集大小随时间变化”这一事实而设计的：短期内用很小很快的缓存，然后逐级更大更慢。

[197] 视频画面时间区间：00:11:30–00:11:45。若为裁剪图，时间区间仍对应原视频画面。

静态分配的适用条件与实测

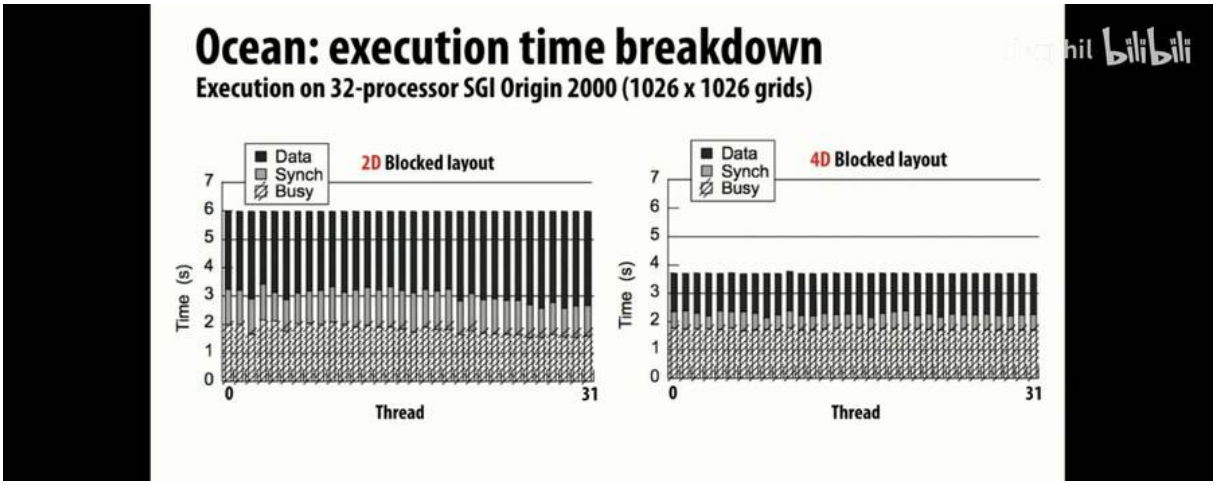

图 198: 海洋模拟的执行时间分解：32 处理器 SGI Origin 2000，1026×1026 网格[198]

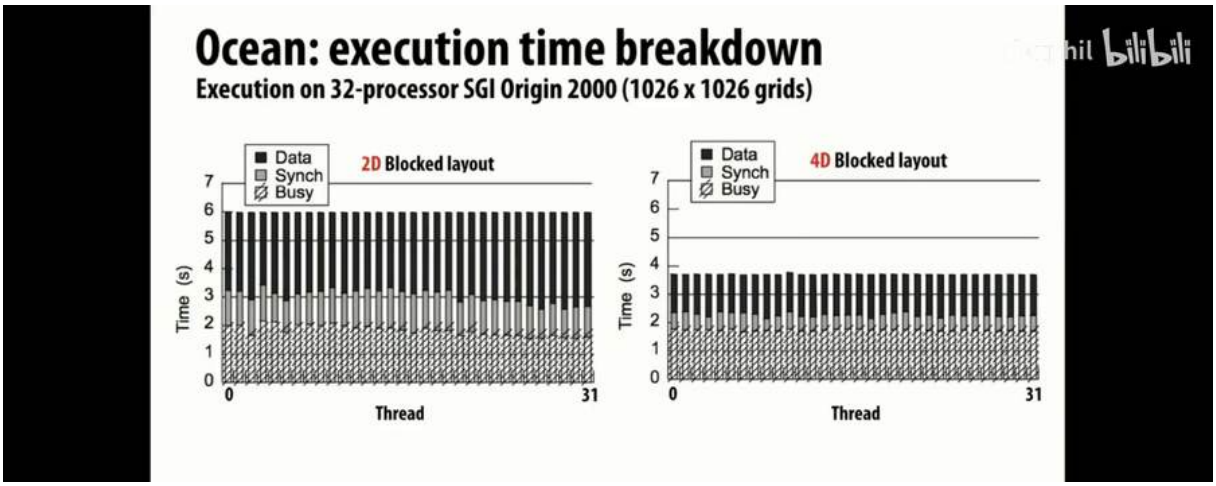

图 199: 执行时间分解续：各阶段占比[199]

讲者说明这是纯粹的静态分配，而它在这个应用上工作得很好，因为所有网格点基本上都在做同样的事，因此工作量在各瓦片间均匀分布。

分块形状的收益与机器类型无关

无论共享内存还是消息传递，计算正比面积、通信正比周长这条几何事实都成立。因此”用紧凑块划分”的结论可以跨机器类型迁移。差别只在于通信的实现方式与固定开销，而不在于最优形状。

他也确认这种被称作四维分块的做法确实改善了通信性能，而且它在分布式消息传递环境中同样表现极好，因为能获得相当好的算术强度。

[198] 视频画面时间区间：00:14:00–00:14:15。若为裁剪图，时间区间仍对应原视频画面。
[199] 视频画面时间区间：00:15:30–00:15:45。若为裁剪图，时间区间仍对应原视频画面。

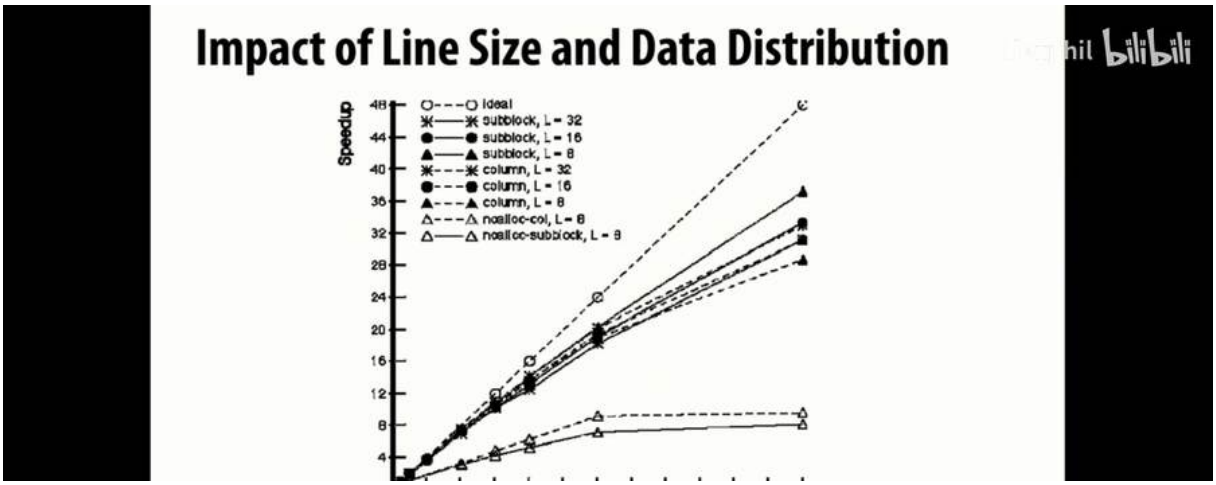

图 200: 缓存行大小与数据分布对加速比的影响[200]

他展示了一张关于加速比的图，说明不同内存分配机制下的差异：最上方是理想加速比（达不到），右下方则是完全没有局部性的糟糕分配，其余是各种分配选择形成的分布带。

讲者说自己认为这张图比他今天需要讲的更细，但它代表了一类典型问题。

他也指出一个重要连带效应：改善内存布局不仅减少通信时间，还能提供更规整的计算模式，从而避免因个别访问慢而拖累整体。

内存布局同时影响通信与抖动

好的布局既降低平均通信量，也降低通信时间的方差。在屏障同步的结构下，方差与均值同样重要——因为决定每步耗时的是最大值而非平均值。因此布局优化在同步型应用中的收益常大于其平均带宽改善所暗示的程度。

本章小结

网格划分的原则是让瓦片越方越好，因为计算正比面积、通信正比周长，方形使二者之比最大。屏障同步有两个代价：达成一致的时间，以及掉队者拖慢所有人。工作集大小取决于时间尺度，缓存层次正是为此设计。海洋模拟的静态分配之所以有效，是因为所有网格点工作量相同；四维分块确实改善了通信，在消息传递环境下同样有效。改善内存布局还能减少通信抖动。

[200] 视频画面时间区间：00:17:00–00:17:15。若为裁剪图，时间区间仍对应原视频画面。

9.5 案例二：星系演化与 N 体问题

为什么比网格问题难

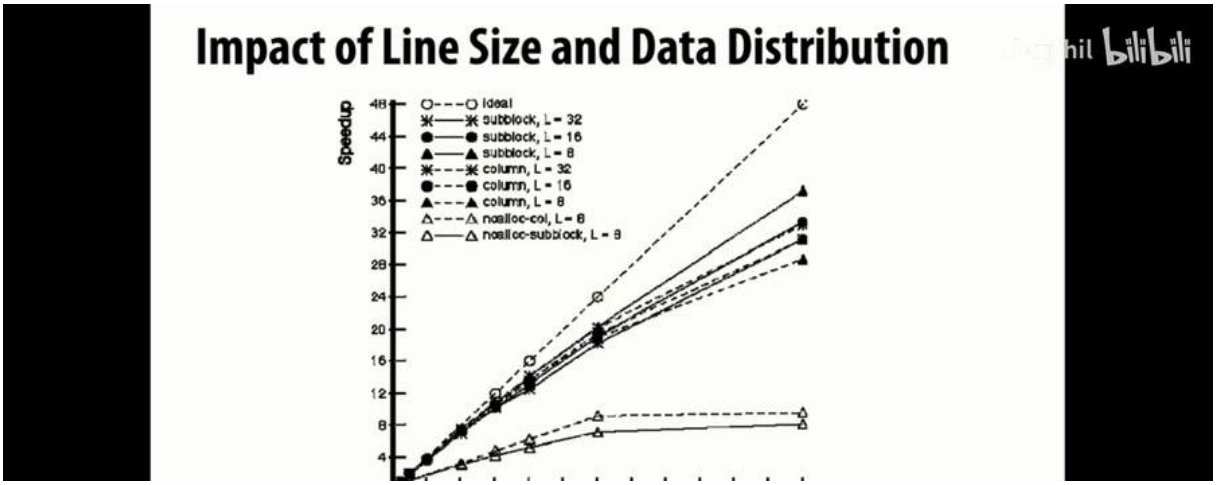

图 201: 星系演化：用 Barnes-Hut 算法建模引力相互作用[201]

讲者说更有挑战性的是星系模拟。基本设定是：把宇宙中所有恒星看作引力体，它们相互吸引，因此会形成轨道、旋臂等结构。我们要建模的是星系随时间的演化，而演化由这些恒星之间的全部相互作用决定。

困难在于：恒星在星系中并非均匀分布，它们以各种方式成团聚集。因此不能简单地把空间切成均匀网格、假设每块都一样。

$O(n^2)$ 与它的出路

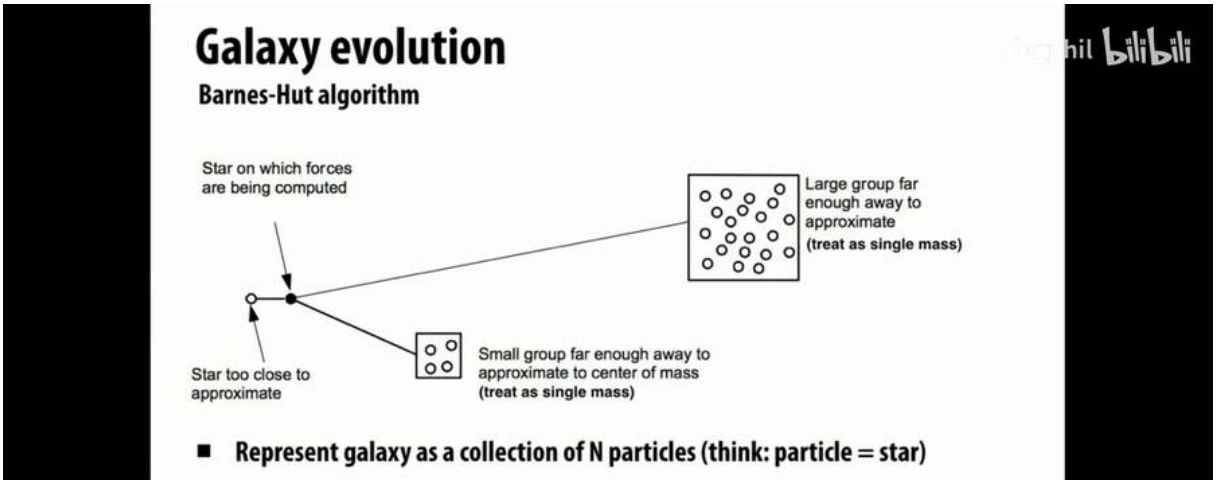

图 202: Barnes-Hut 算法：用远处星团的质心近似其引力影响[202]

朴素做法的问题是：对某一颗恒星，原则上必须计算其他 $n-1$ 颗星对它的作用力，因此是 n 平方的问题。虽然力是对称的，但那只能节省 2 倍。

[201] 视频画面时间区间：00:17:30–00:17:45。若为裁剪图，时间区间仍对应原视频画面。
[202] 视频画面时间区间：00:19:00–00:19:15。若为裁剪图，时间区间仍对应原视频画面。

关键观察是：如果某个星团足够远，就可以把它抽象成一个点质量——计算该远处星团的质心、把质量加起来，就得到它对目标恒星影响的相当好的近似。

于是可以利用”远处物体可被聚合”这一性质，把多个天体合并成单点模型。最终得到的算法大致是 $n\log n$ 而不是 n^2。

讲者特意限定：这不是最坏情况下的算法保证，而是对现实分布而言的实用结果。

是否把某星团当作点质量，由展开判据决定：

$$\frac{s}{d} < \theta$$

- s：该星团所在方格的边长；
- d：星团中心到目标恒星的距离；
- θ：阈值，通常取 0.5 左右；
- 满足则用质心近似，否则继续深入其子节点。

阈值越大越快但越不准，因此它同时是性能参数与精度参数。

近似是复杂度的来源，也是正确性的边界

Barnes-Hut 用”远处星团视作点质量”换取 $O(n\log n)$，这个近似的误差取决于星团张角与距离之比。因此它不是精确算法。工程上必须接受这个误差，并据此选择展开判据；若应用要求严格精度，就必须退回 $O(n^2)$ 或使用更高阶展开。

本章小结

星系模拟比网格问题难，因为星体分布不均而成团聚集，不能划分均匀网格。朴素做法是 $O(n^2)$，对称性只能省 2 倍。Barnes-Hut 的关键观察是远处星团可抽象为质心点质量，于是复杂度降到实用的 $O(n\log n)$，代价是引入近似。

9.6 Barnes-Hut 树

树的构造

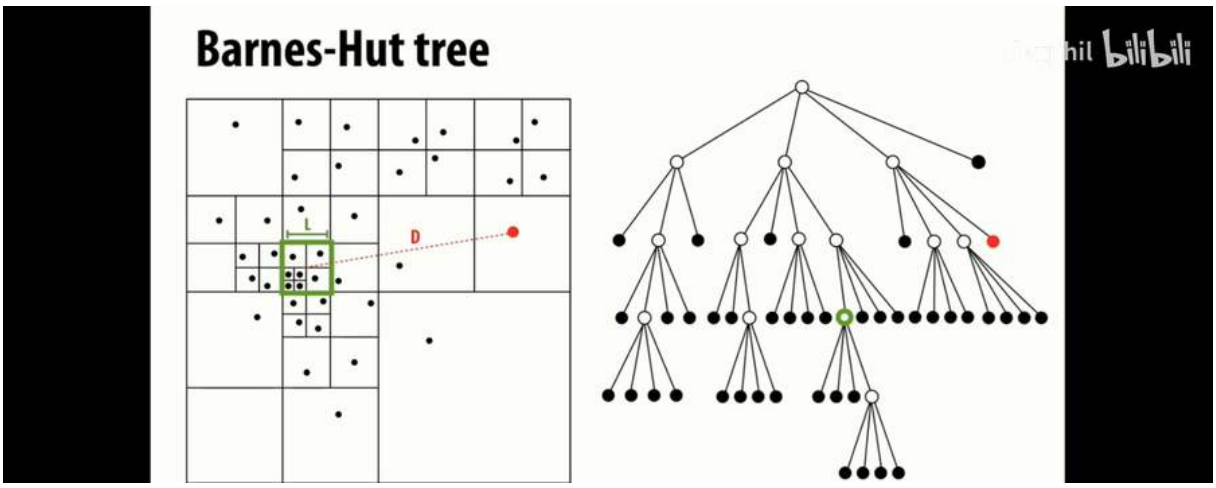

图 203: Barnes-Hut 树：反复二分空间直至每格只剩一颗星[203]

做法是把空间反复二分。讲者用二维演示（真实宇宙是三维，同样的思路在三维进行）。

图中不断把方格切得更细，直到某个方格中只剩一颗恒星——这就是最深的一层，它们会成为树的叶子。

这些方格构成一个层次结构，对应右侧那棵树，叫做四叉树：每个节点代表一块空间，它的四个子节点代表四个象限，如此递归。每一步在 x 与 y 方向各分一次。

他补充说，三维版本叫八叉树，是八路分裂，即在三个维度上各分一次。

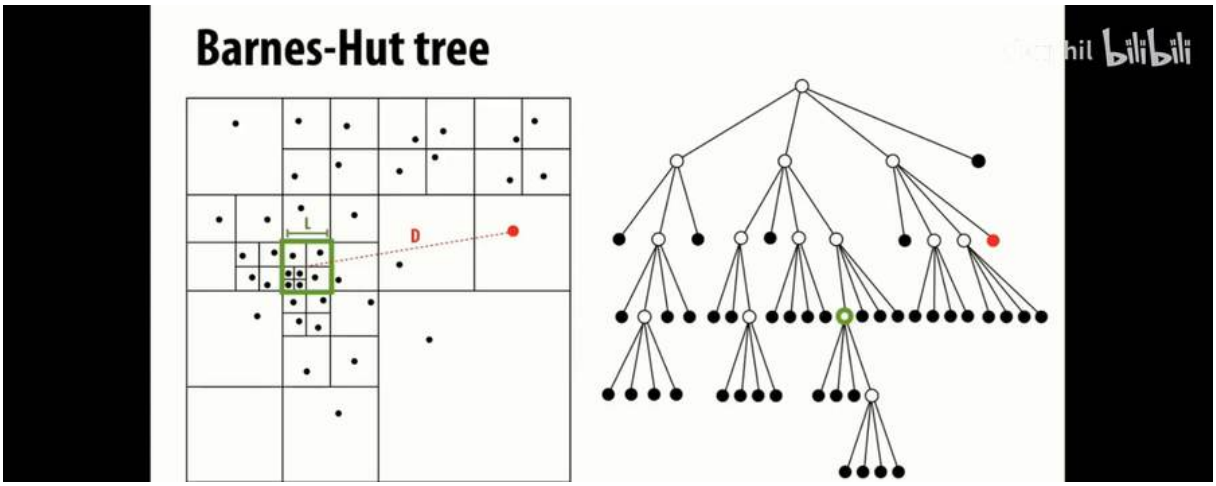

图 204: 树结构续：内部节点记录质心与总质量[204]

质心的逐层合成

这个结构提供了很好的封装方式：把一块区域内的信息聚合成紧凑形式，即单个点质量。

黑色的节点是叶子，确实是单个点质量。而那些空白的内部节点，若计算该方格内所有质量的质心与总质量，那就是一个合理的代表。而且可以继续向上传播：基于四个（或八个）扇区的质心，计算逐级更粗方格的质心。

因此这个结构利用了引力本身的特性——质心性质。

质心聚合之所以有效，是因为引力按平方反比衰减

若相互作用随距离衰减得更慢，远处星团的内部结构就会显著影响结果，点质量近似便不成立。正因为引力衰减快，远处细节才可忽略。这个性质决定了树方法适用的相互作用类型，也是它不能直接套用到所有 n 体问题的原因。

[203] 视频画面时间区间：00:21:10–00:21:25。若为裁剪图，时间区间仍对应原视频画面。

[204] 视频画面时间区间：00:23:30–00:23:45。若为裁剪图，时间区间仍对应原视频画面。

建树与遍历的代价

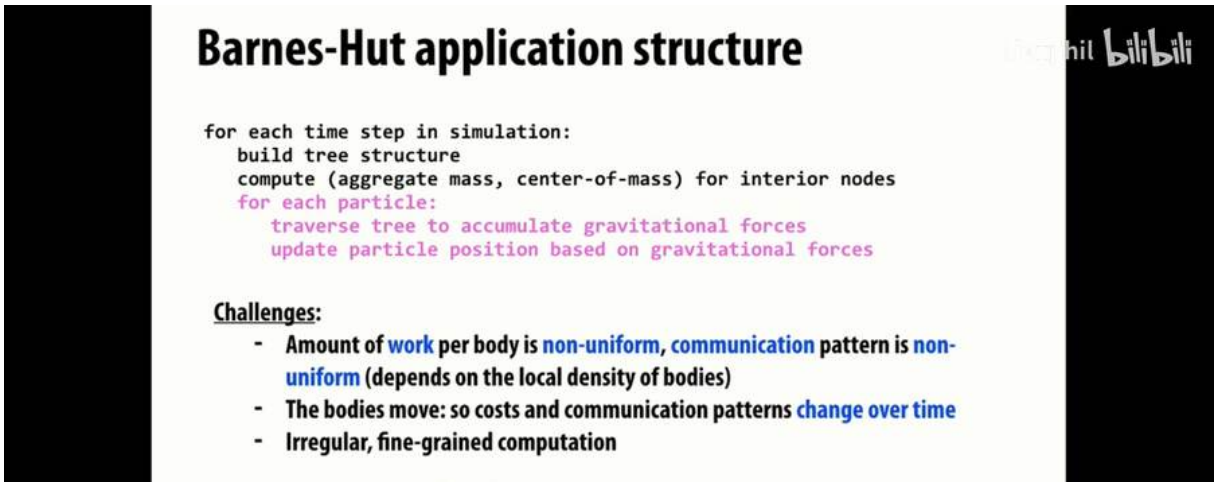

图 205: Barnes-Hut 的应用结构：每个时间步建树、计算力、更新位置[205]

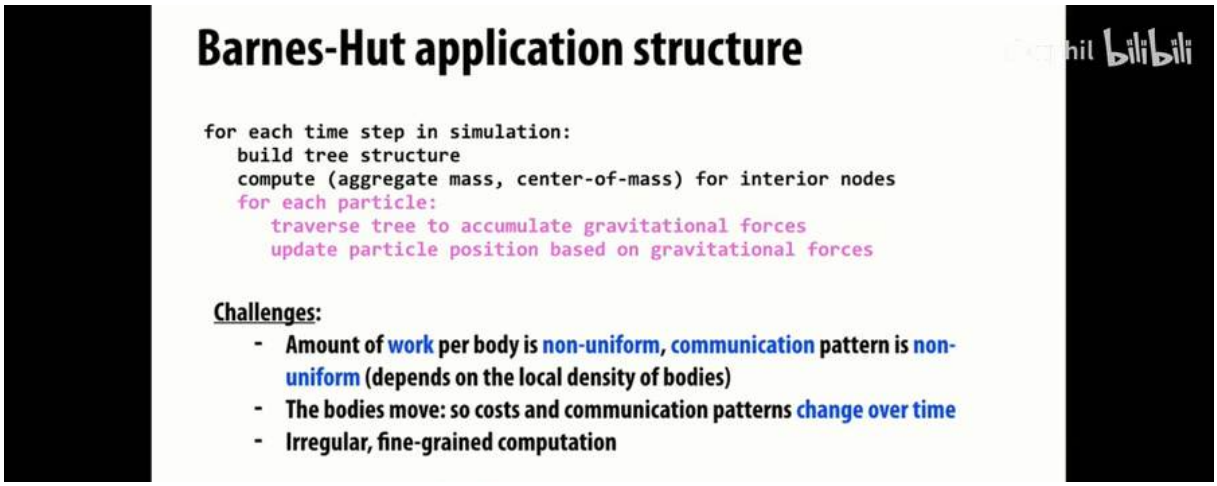

图 206: 应用结构续：建树、遍历、更新位置的循环[206]

讲者先讨论了建树。有学生问：如果只有少数恒星移动，是否真的需要重建整棵树?

讲者回答可以动态做：若只是把一颗星从 A 点移到 B 点，通常只是跨越相邻方格，可以更直接地处理。所以这当然可行。

但他说无论如何，假设我们建这棵树并完成聚合过程，可以看到它的代价只正比于树的节点数，也就是与天体数线性相关。因此这里有一个明显的并行来源。

然后是遍历。对树中每个叶子，都要计算作用在它上面的所有引力：有些纯粹是局部的，有些来自更远处。为此要从该节点向上走，再沿某些分支向下，有的分支走得比其他分支更深。

本章小结

Barnes-Hut 用反复二分把空间切成层次结构，二维为四叉树、三维为八叉树，叶子是单颗星，内部节点记录质心与总质量。质心可逐层向上合成，其有效性依赖引力的平方反比衰减。建树代价

[205] 视频画面时间区间：00:25:00–00:25:15。若为裁剪图，时间区间仍对应原视频画面。

[206] 视频画面时间区间：00:28:00–00:28:15。若为裁剪图，时间区间仍对应原视频画面。

与天体数线性相关，因此有明显并行性；遍历则对每个叶子上下走树，深度因距离而异。

9.7 负载不均与动态划分

问题的形态

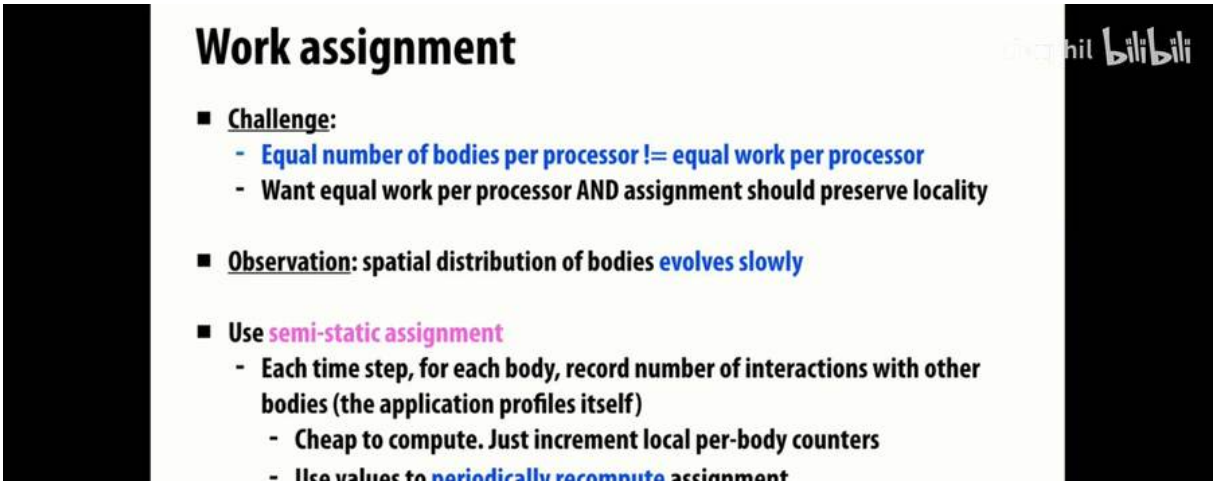

图 207: 工作分配：每个处理器分配相同数量的天体不等于相同的工作量[207]

讲者指出，对给定物体而言，尤其是考虑到分布如此不规整，工作量分布很不均匀，通信模式也不均匀。

但另一方面也不算太糟，因为我们主要是在这棵树上上下移动，而不是遍历整个宇宙。

他还提到一个随之而来的问题：若把恒星按空间分布到各处理器上，那么随着恒星移动，这个分配可能需要改变。好消息是这些问题都可控，而且由于树带来的局部性，大部分通信会相当局部。

不能按数量分配

于是引出一个有趣的问题：应该如何把工作分配给处理器？

讲者的回答很明确：由于工作量在天体间并不均匀，有些需要更多工作，所以实际做法是在运行中动态测量每个天体需要多少工作量，然后据此管理，使各处理器承担大致相同的工作量——但这并不意味着给它们相同数量的天体。

负载均衡的目标是等时间，不是等数量

在规整网格上，等数量恰好等于等时间，因此静态分配就够。在树结构上，不同叶子访问的节点数相差很大，等数量会导致等时间严重失衡。因此判据必须换成”每个处理器的预计耗时相等”，而这需要运行时测量。

[207] 视频画面时间区间：00:29:00–00:29:15。若为裁剪图，时间区间仍对应原视频画面。

用代价区域分配

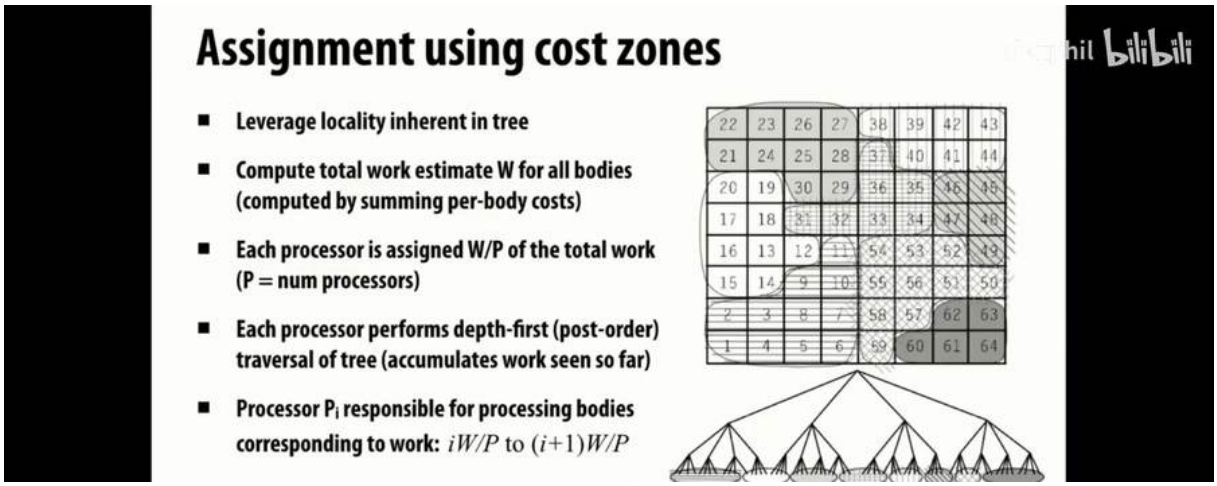

图 208: 用代价区域做分配：利用树固有的局部性[208]

讲者展示了具体做法：由于四叉树划分，可以把树的一部分分配给处理器，这在空间上对应把星系或宇宙切成一些区域——这些区域不是方形或矩形，但每一个都连续。

这是个有意思的处理不均匀性的方式：既保留了一定的空间局部性，又能通过相当简单的算法得到相当均匀的划分。

近似划分的代价必须与均衡收益一起评估

划分本身要花时间，且划分越频繁收益越小。若每步都重新划分，划分开销可能超过均衡带来的收益。因此实际系统采用”半静态”策略：在一段时间内固定分配，只在失衡累积到一定程度时才重划。

他坦率地说，找到完美划分很可能是 NP 完全问题，但可以设想各种分治算法，它们会相当有效。

这种被称为半静态的方案是：在一段时间内让处理器 1 处理横条纹那一组，处理器 2 处理下一组，依此类推。

工作集视角下的树遍历

讲者再次回到工作集的概念。以图中那个红色节点为例，它需要遍历以找出所有作用在它上面的力，因此会沿树上下走到不同深度，访问树的一些部分，而这些部分在空间上可能并不局部。

但访问的总量相当有限，因此若有任何形式的缓存，表现会比较好——因为沿树上下时访问的树的部分会被复用。

树的局部性是”结构局部性”而非”空间局部性”

遍历 Barnes-Hut 树时，一个叶子访问的节点在几何空间上可能分散，但在树结构上集中在少数路径及其子树。因此缓存仍能受益，因为上层节点被大量叶子共享。理解这一点才能解释为什么这种看似”乱跳”的访问模式实测表现并不差。

[208] 视频画面时间区间：00:30:00–00:30:15。若为裁剪图，时间区间仍对应原视频画面。

本章小结

树结构导致工作量与通信模式都不均匀，因此不能按天体数量平均分配，而要在运行中动态测量每个天体的工作量。用代价区域分配可以利用四叉树的空间连续性，得到连续但非矩形的区域，既保留局部性又相当均衡；完美划分很可能是 NP 完全问题，但分治算法足够有效。树的局部性是结构性的，因此缓存表现仍好。

9.8 总结与延伸

讲者在结尾点出一个一般性结论：这些方法并不完全依赖科学领域或问题领域，而更多与问题的一般特征有关，而不是与你要解决的具体问题有关。

把全讲串起来，这是一条从原理到案例的落地路径。这一讲的两个核心关切是负载均衡与通信开销。硬件假设是 NUMA 共享内存：部分内存访问更快，跨芯片要经对方 DRAM 控制器。海洋模拟把空间离散成二维网格，按小时步更新状态；提高分辨率必须同时缩短时间步，因此分辨率翻倍要 16 倍算力。网格划分的原则是让瓦片越方越好，因为计算正比面积、通信正比周长；屏障同步要付出达成一致的时间，且掉队者会拖慢所有人；工作集大小取决于时间尺度，缓存层次正为此设计。海洋模拟的静态分配有效，因为所有网格点工作量相同；四维分块确实改善通信，在消息传递环境下同样有效。星系模拟因星体成团而更难：朴素做法 $O(n^2)$，对称性只省 2 倍；Barnes-Hut 用远处星团可聚合为质心这一观察，把复杂度降到实用的 $O(n \log n)$，代价是近似。树在二维为四叉树、三维为八叉树，叶子是单星，内部节点记录质心与总质量，质心逐层向上合成。建树代价与天体数线性相关，遍历深度因距离而异，因此工作量不均，必须动态测量并按代价区域划分，而不是按数量平均。

本讲义在讲者材料之上做的压缩，有四点需要与讲者原话区分：

- 两个案例的差别不在应用领域，而在” 工作量是否可预知”。网格问题的工作量均匀且静态可知，因此静态划分足够；树结构问题的工作量取决于运行时分布，因此必须动态测量。这是划分策略选择的真正判据。
- 分辨率与算力的四次方关系是所有物理模拟的共同约束，不是某个应用的特殊困难。理解这一点，才能理解为什么这类问题需要超算，以及为什么它们对硬件演进极其敏感。
- 屏障同步把最慢者的延迟放大为全局延迟。因此在同步型应用中，通信时间的方差与均值同等重要，甚至更重要。这解释了为什么布局优化在这类程序中的收益常超出带宽改善所暗示的程度。
- Barnes-Hut 的 $O(n \log n)$ 是实用复杂度而非最坏情况保证，且以近似为代价。因此它不是” 更好的算法”，而是” 在可接受误差下更快的算法”。评估这类方法时必须同时报告精度与耗时。

这堂课的边界在于它是案例综述：讲者明确说部分图表比他今天需要讲的更细，也未深入具体实现的调优细节。

两个可以直接从本讲数据提出的下一步问题：其一，代价区域划分假设工作量在短时间内变化缓慢，那么当星体分布剧烈变化时，重新划分的频率应如何确定？其二，Barnes-Hut 的近似误差由展开判据控制，那么在并行环境下，不同处理器采用不同判据是否会造成结果依赖线程调度？

Lecture 10　Parallel Programming Case Study（续）

视频时长：1 小时 7 分 9 秒　来源：https://www.bilibili.com/video/BV1TV4y1e7s5/

10.1　加速比这个指标本身有问题

这一讲从一次质疑开始：怎样才算”在这个应用上取得了良好性能”？讲者指出，回答这个问题比想象中难，因为最常用的指标——加速比——在多种情况下会给出误导性的结论。

先从功耗说起

讲者在开场提到一个与性能常被混为一谈的量：手机里的处理器若跑到峰值要烧掉约 5 瓦，放在口袋里会让你觉得腿很热。因此它典型运行功耗只有 100 到 200 毫瓦。

这解释了为什么芯片设计要做得如此复杂：很大一部分动机不是让它在最高速度下表现更好，而是让它能在低功耗下更高效地运行。

这个落差可以用两个量的比值来表达：

$$R_{\text{power}} = \frac{P_{\text{peak}}}{P_{\text{typical}}}$$

- P_{peak}：芯片能维持的峰值功耗，本例约 5 瓦；
- P_{typical}：实际持续运行的功耗，约 100 到 200 毫瓦；
- R_{power}：两者之比，可达数十倍。

峰值只能短暂维持，因此持续性能由 P_{typical} 决定，而非峰值算力。

移动端的性能目标是每瓦产出而非峰值

在功耗受限的设备上，峰值性能往往无法长时间维持。设计的重心因此从”最快”转向”每单位能耗完成多少工作”。评估这类系统时，用峰值算力比较会得出与实际体验完全不同的结论。

一个用来检验测量方法的例子

讲者回顾上一讲的粒子分箱问题：一批分布在空间中的粒子，要按位置分到各个网格里，好让后续处理能够局限在局部。

他列出三种并行实现。第一种是遍历所有粒子，判断它属于哪个格子，然后追加到一个列表上——若这个列表是全局的，就有跨处理器的争用，必须同步，代价可能很高。第二种是遍历所有格子，对每个格子再看一遍所有粒子，挑出属于它的。第三种最复杂：每个处理器先为自己处理的那批粒子建立局部数据结构，再以某种方式合并成描述整个系统的全局结构。

讲者点出一个很好的检验标准：把 P 设为 1，三种实现都应当退化为串行算法。第一种不需要锁；第二种退化为对每个格子跑一遍；第三种不需要排序与合并，直接得到想要的结构。

这个检验可以写成一条约束：

$$T_{\parallel}(1) \approx T_{\text{serial}}$$

- $T_{\parallel}(1)$：并行实现在 1 个处理器上的耗时；
- T_{serial}：纯粹的串行实现耗时；
- 若两者相差很大，说明并行实现在单线程下仍背负着并行开销。

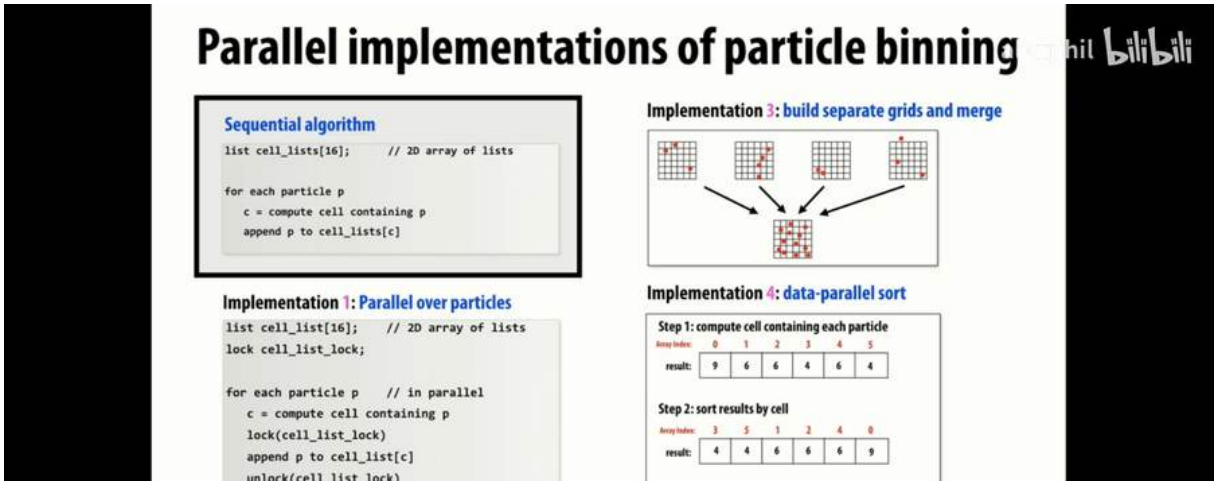

图 209: 粒子分箱的三种并行实现：第三种先建局部结构再合并[209]

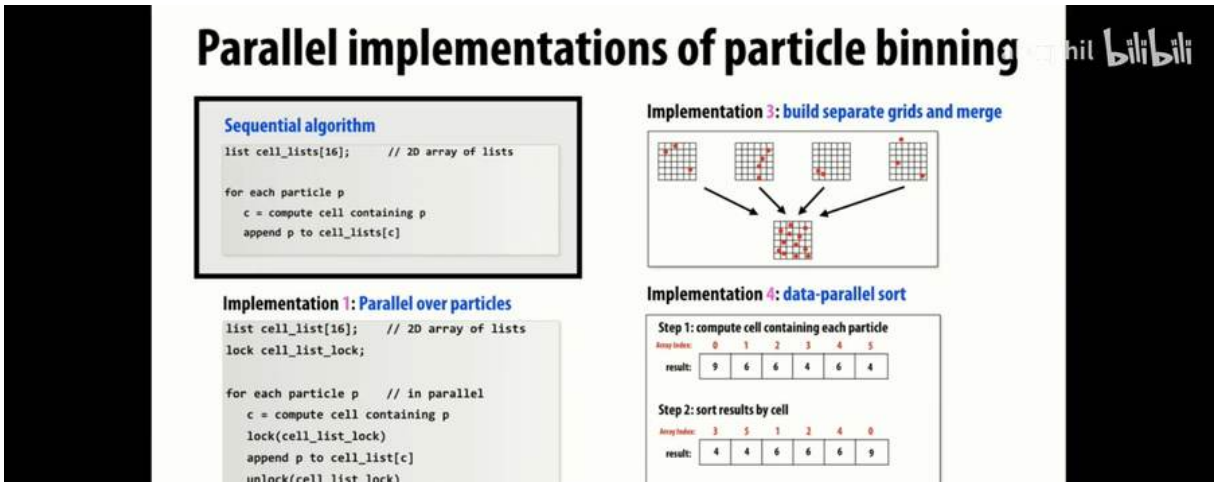

图 210: 粒子分箱的并行实现与串行基线对比[210]

本章小结

加速比的问题不在于这个比值本身，而在于分子分母是否可比：基线要选最优串行实现，问题规模要足够大到能喂饱所有处理器。两个陷阱方向相反，一个高估收益、一个低估收益。

在具体分析这些陷阱之前，先看清它们各自的表现形式与成因。

[209] 视频画面时间区间：00:01:30–00:01:45。若为裁剪图，时间区间仍对应原视频画面。

[210] 视频画面时间区间：00:02:10–00:02:25。若为裁剪图，时间区间仍对应原视频画面。

10.2 加速比的两个陷阱

第一个陷阱：基线和谁比

讲者说这里有一个在论文里常见的错误。如果拿第一种实现和”它自己的单处理器版本”比——那个版本里仍然调用着锁的接口，只是没有争用——加速比看起来不会完美，但也不难看。

但他的判断是斩钉截铁的：如果在真实世界里把它和上面那个最优的串行版本对比，结果会很难看，因为锁很贵，争用程度高的锁尤其贵。

由此得到这条原则：

加速比必须对照最优串行实现

分母应当是你能写出的最好的串行代码，而不是并行代码去掉并行之后的版本。后者往往保留了为并行而付出的开销（锁、原子操作、额外的数据结构），用它作基线会系统性高估并行收益。这种错误在学术论文中也并不少见。

第二个陷阱：问题太小

第二个问题更普遍。讲者用海洋模拟的实测数据说明：把同一份问题分别跑在 1 个和 P 个处理器上，随处理器增加，加速比先是不错地上升，到 16 个处理器时几乎不再提升，到 32 个时反而开始变差。

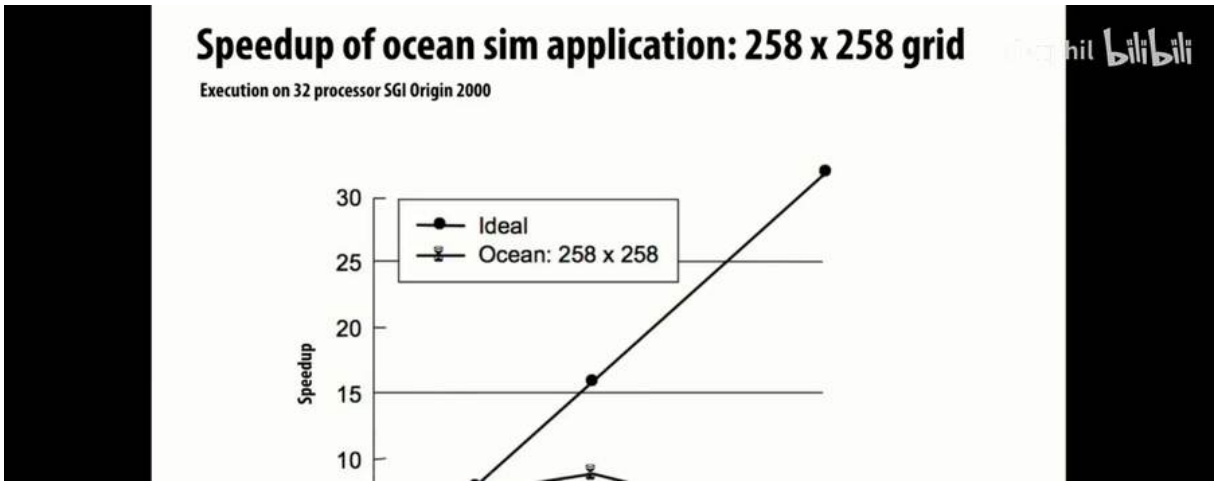

图 211: 海洋模拟在 258 × 258 网格上的加速比：16 核后停滞，32 核反而下降[211]

原因是问题太小。当处理器增加到一定程度，每个处理器分到的工作已经不足以让它保持忙碌。

讲者用上一讲的结论解释这件事。这个例子采用二维分块，算术强度为：

$$\text{AI} \propto \frac{N}{\sqrt{P}}$$

- N：网格的线性边长，网格共 N^2 个元素；
- P：处理器数量；
- 结果与 $N/\sqrt{P}$ 成正比，即指数为 -0.5。

[211] 视频画面时间区间：00:04:30–00:04:45。若为裁剪图，时间区间仍对应原视频画面。

P 增大时算术强度按 $\sqrt{P}$ 下降。讲者说这不是灾难——不是随 P 线性下降——但迟早会追上来，让你到达”算术做得不够、通信占比过高、于是拿不到任何性能提升”的境地。

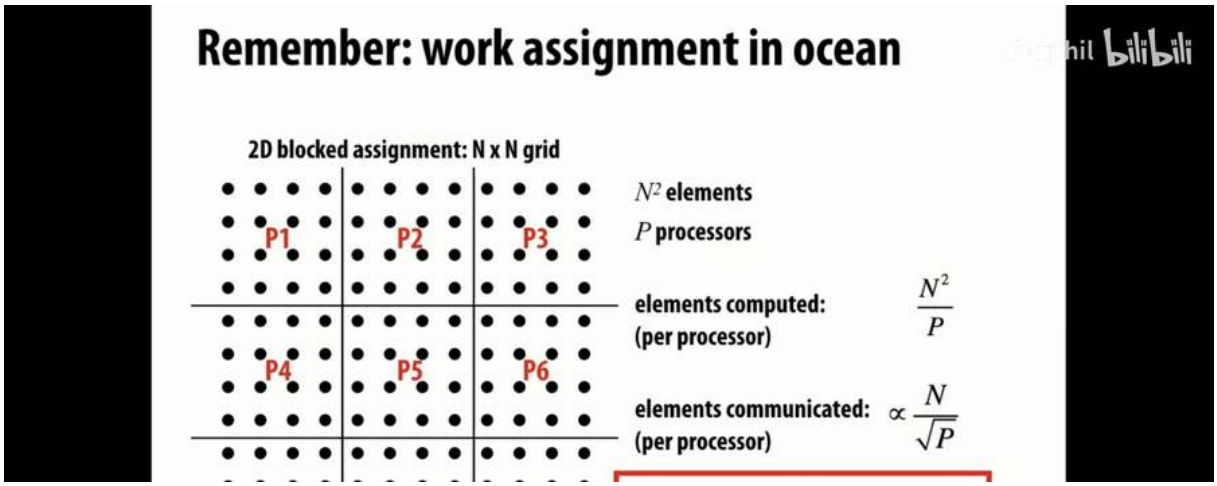

图 212: 回顾：海洋模拟采用二维分块分配，算术强度随处理器数下降[212]

换个问题规模，结论完全不同

讲者接着做了一个对照实验，这是本讲最有说服力的地方。

之前展示的只是 258×258 的网格——他纠正说其实更糟，是 130×130，对 32 个处理器而言太小了。而换成 1026×1026 的网格，同一份代码在 32 个处理器上能达到 25 倍的加速。

扩展性结论必须绑定问题规模

同一个实现在 130×130 网格上 32 核反而变慢，在 1026×1026 上却达到 25 倍加速。因此”这个程序能扩展到多少核”这个说法没有意义，必须补上问题规模。报告加速比时省略规模，等于给出了一个无法复现的数字。

讲者的结论是：在较小规模下加速比甚至负增长，恰恰说明问题不够大、留不住这么多处理器，越来越多的时间浪费在通信上。

[212] 视频画面时间区间：00:06:30–00:06:45。若为裁剪图，时间区间仍对应原视频画面。

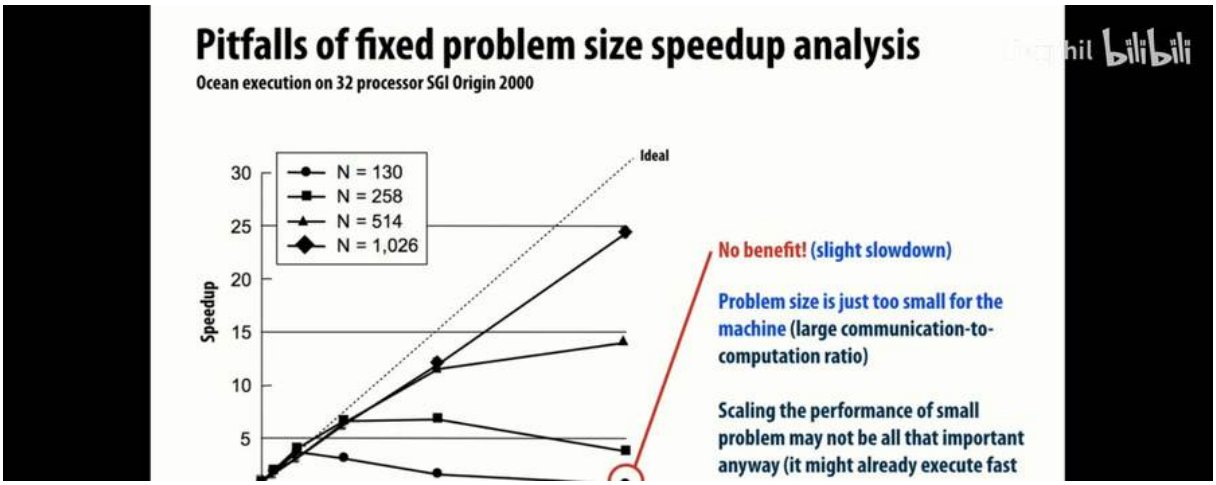

图 213: 固定问题规模加速比分析的陷阱：小网格上曲线明显下弯[213]

本章小结

加速比这个指标有两个必须警惕的陷阱：分母用了带并行开销的伪串行版本，以及问题规模太小以至于每一核都喂不饱。前者会高估收益，后者会低估收益。两个陷阱方向相反，说明单一加速比数字本身并不足以描述”性能好不好”。

这也引出本讲的主线：与其固定问题看加速，不如研究问题规模应如何随资源一起变化。

10.3 超线性加速说明了什么

上一节的问题规模太小。讲者随即给出反方向的另一个异常：有些情况下性能随问题规模增长得比线性还快。

缓存合并带来的超线性

讲者请学生猜原因，答案很快给出：缓存。

多加一个处理器，系统就多出一块缓存。而这个问题的并行性很好，可以干净地切分开。当你把问题切成 P 块，每一块的工作集，也就是每个处理器需要访问的数据量，可能刚好落进了某一级缓存的容量之内。

于是每个处理器都从被缓存覆盖的状态下工作，而单处理器版本的数据量远超缓存容量，全程在访问主存。两相对比，加速比就超过了 P。

[213] 视频画面时间区间：00:07:00–00:07:15。若为裁剪图，时间区间仍对应原视频画面。

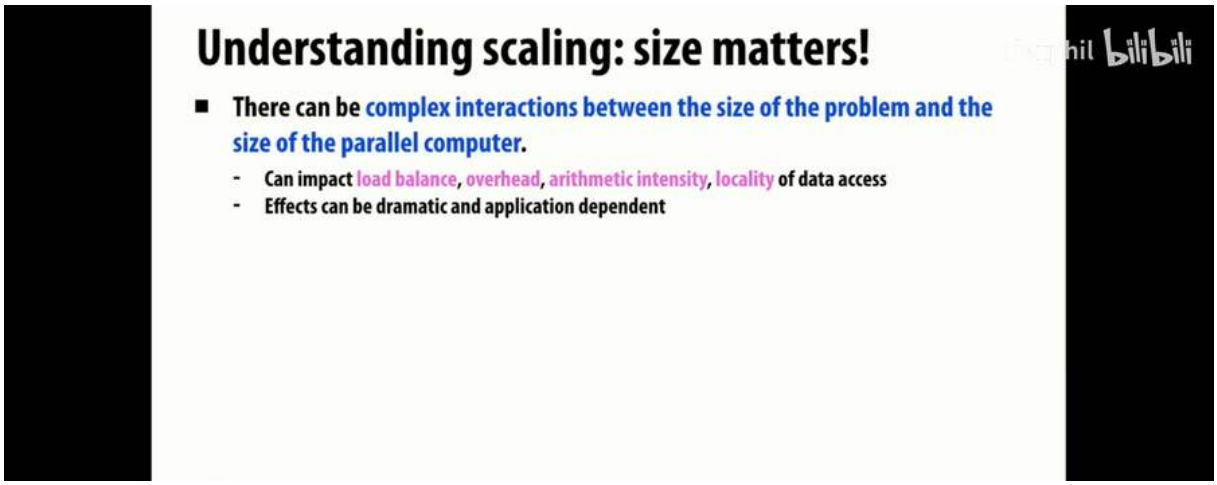

图 214: 理解扩展性：问题规模与其他因素之间存在复杂相互作用[214]

超线性加速通常意味着基线选得不合适

加速比超过 P 在数学上不应发生在同一台机器上，因为它暗示每个处理器都比单处理器时更快。真实原因几乎总是”单处理器版本装不进缓存”。因此看到超线性时，应当先怀疑基线问题，而不是庆祝算法优越。这类数字如果出现在论文里，通常说明对照设置不公平。

另一个真实的超线性来源

讲者补充了一种确实存在的例外：组合搜索，例如布尔可满足性求解。

这类算法的特点是并行地尝试多个不同方向，任意一个成功就停下。同时跑的方向越多，找到解的时间就越可能缩短。这种超线性是算法性质带来的，不是测量假象。

搜索类问题的超线性来自停止条件

当算法满足”任一分支成功即完成”时，并行版本相当于对解空间做多重采样，期望耗时会下降得比 $1/P$ 更快。这与网格计算的性质完全不同——后者的每份工作都是必需的，无法被”跳过”。

扩展性不是单一概念

讲者由此提出一个总的观察：当你说”我要把问题扩大若干倍”时，即使这句话本身代表什么都不是简单的事。负载均衡、开销、算术强度、局部性这些因素都会以不同方式影响性能。改变问题或改变处理器数量，得到的关系可能非常复杂。

于是可以这样概括前面的两个例子：如果只固定一个问题问”快了多少”，那么问题太小或太大都可能让你量不出任何有意义的东西。

[214] 视频画面时间区间：00:10:00–00:10:15。若为裁剪图，时间区间仍对应原视频画面。

”把问题放大”本身需要定义

扩大网格边长、增加时间步数、增加粒子数，三者对性能的影响并不相同。同一份代码在”边长翻倍”与”迭代次数翻倍”下的算术强度变化趋势可能相反。因此讨论扩展性之前，必须先明确是哪个维度在变。

本章小结

超线性加速多数是缓存合并造成的测量假象，少数来自搜索类算法的停止条件。它提醒我们：加速比的分子分母必须可比。而”放大问题”这句话本身含义模糊，这直接引出了本章之后要讨论的扩展性分类。

10.4 强扩展与弱扩展

两种术语

讲者指出业界，尤其是超级计算领域，有两种讨论问题规模扩展的方式。

强扩展是经典的固定问题：拿同一个问题 X，看它在 P 个处理器上比在 1 个处理器上快多少，取倒数就是加速比。固定问题，变化资源。

弱扩展则不同：随着资源增长，同时让问题规模成比例增长，然后看效率能多接近 1.0。也就是说，能否用 P 倍的资源在同样时间内解决 P 倍大的问题。

讲者指出后者其实更贴近真实应用想做的事。

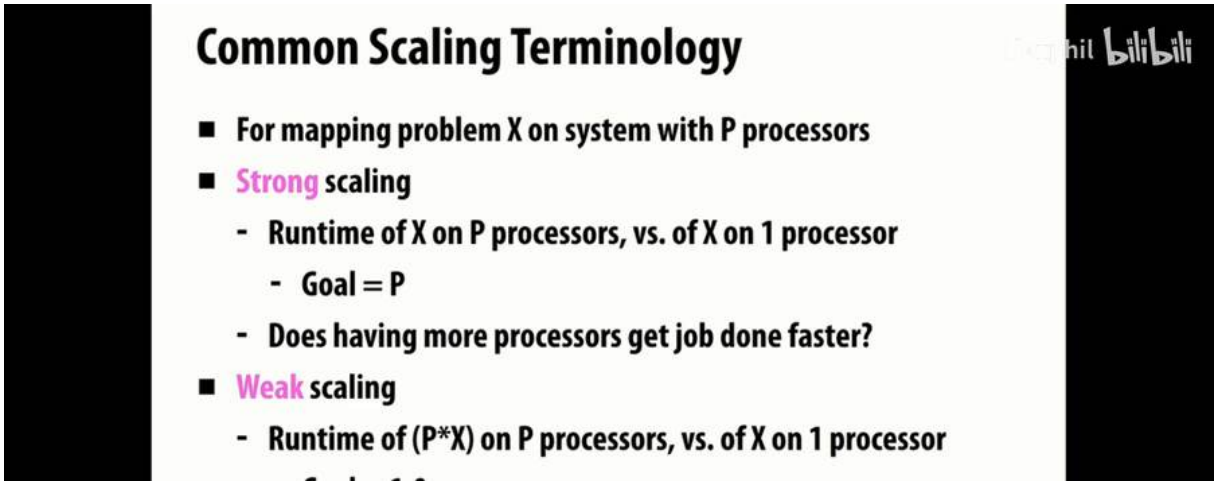

图 215：常见扩展性术语：强扩展与弱扩展[215]

看懂架构的可扩展设计

讲者提到架构师也在谈扩展性，但他们的问题不同：一种架构能否靠复制单元覆盖从手机到集群的整个产品线。

他给出的例子：几百美元就能买到一块包含两个这类核心加一个 ARM 作为主机的片上系统，装在小盒子里，广泛用于移动设备。而 GHC 机器里那台是个庞然大物——20 个这样的核心，功耗

[215] 视频画面时间区间：00:14:30–00:14:45。若为裁剪图，时间区间仍对应原视频画面。

高、价格贵。从设计角度看，这意味着厂商可以通过复制同一个单元派生出整个产品谱系，这是可扩展设计的好例子。

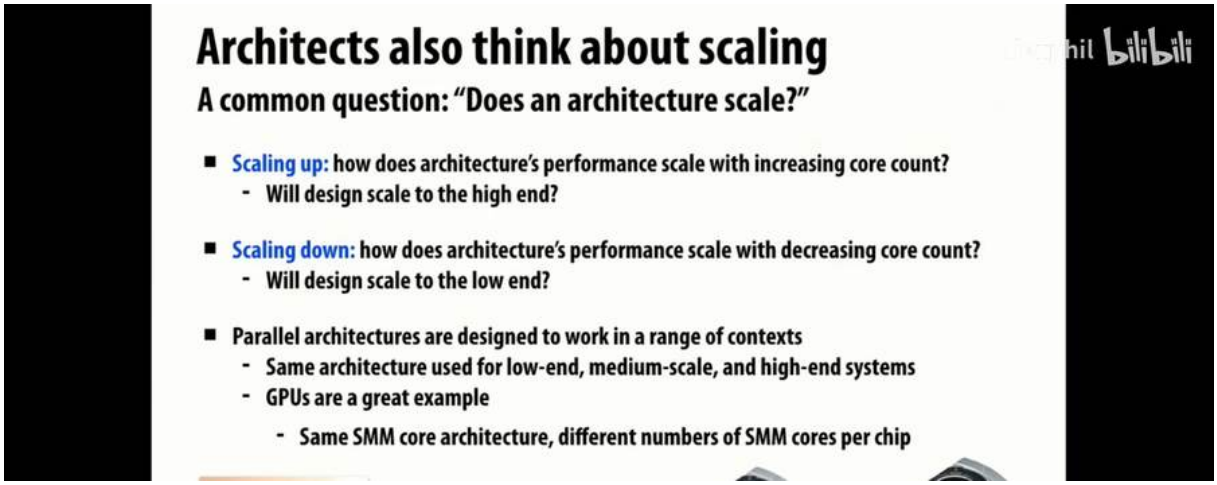

图 216: 架构师也考虑扩展性：能否靠复制单元覆盖不同规模的产品[216]

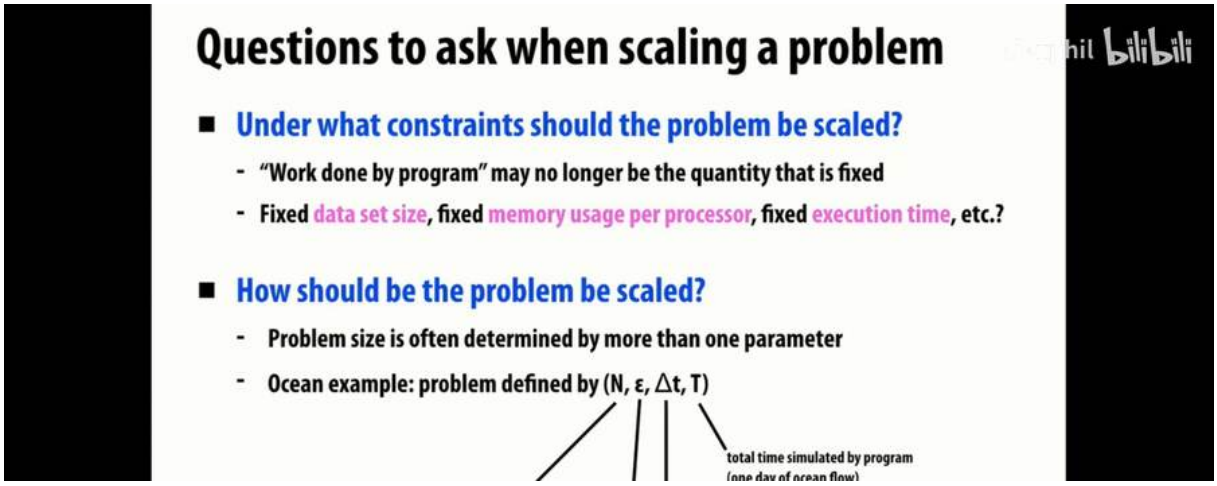

图 217: 扩展一个问题时需要先问的问题[217]

本章小结

强扩展固定问题、增加资源，衡量的是加速比；弱扩展让问题与资源同比增加，衡量的是效率接近 1 的程度。硬件层面的扩展性是另一回事——指同一设计单元复制后能覆盖多大范围的产品。

术语确定后，接下来的问题是：这三种衡量方式在真实的网格求解器上分别表现如何？

10.5 三种约束下的定量分析

这是本讲的核心。讲者把扩展目标分成三类，并用同一份网格求解器推出各自的通信计算比。

[216] 视频画面时间区间：00:12:00–00:12:15。若为裁剪图，时间区间仍对应原视频画面。

[217] 视频画面时间区间：00:16:00–00:16:15。若为裁剪图，时间区间仍对应原视频画面。

三维参数：扩展本身是多变量的

在进入分类之前，讲者先说明问题的复杂程度。以海洋模拟为例，参数包括：$N \times N$ 的网格、收敛误差容限 ε、时间步长 Δt、以及要模拟的总时长 T。

这些参数彼此纠缠。相对独立的是 T——跑 10 年还是 100 年的洋流，这一项对性能的影响大致线性。

讲者的结论是：一旦深入某个应用去问” 我该怎么扩展它”，就会发现这是个多元问题。而即使确定了各参数，把它们折算成一个” 问题规模” 的度量也不那么直接。

两类视角回答的是不同问题

应用导向问的是” 这门科学想把问题做多大”，答案由物理模型与精度要求决定。资源导向问的是” 这台机器能让我做多大”，答案由内存与时间预算决定。前者设定目标，后者设定边界，实际可解的问题落在两者的交集里。

他因此区分两种视角：应用导向的扩展关心具体参数怎么变；资源导向的扩展则从机器约束出发。

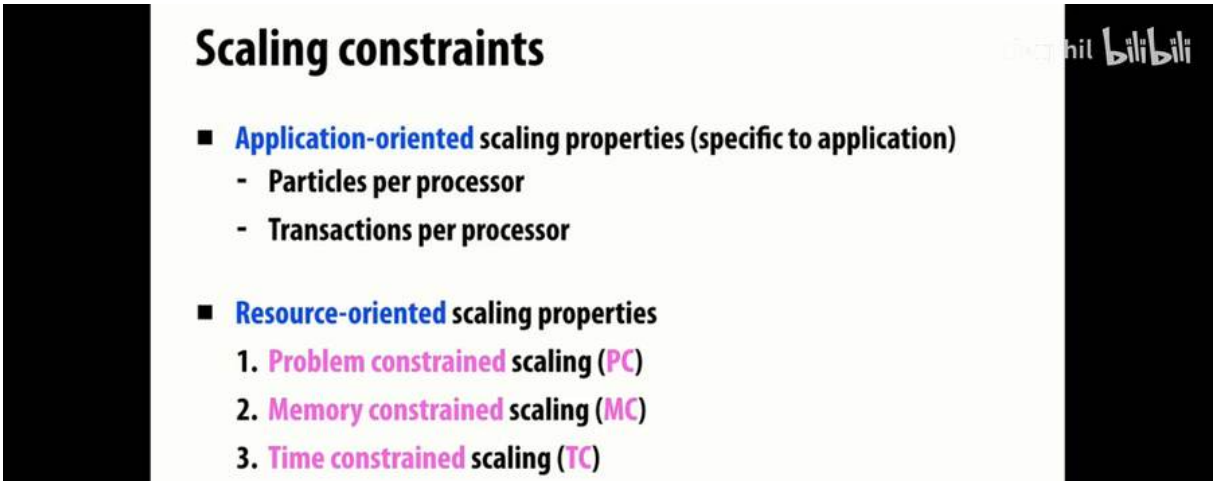

图 218: 扩展约束：应用导向与资源导向两类属性[218]

约束一：问题约束（经典加速比）

问题约束就是固定问题、增加资源，也就是强扩展。

在网格求解器上，每个处理器每一步的计算量是 N^2/P，通信量正比于 $N/\sqrt{P}$。因此通信与计算之比为：

$$\left(\frac{C}{W}\right)_{\text{problem}} = \frac{N/\sqrt{P}}{N^2/P} \propto \frac{1}{\sqrt{P}}$$

- N：网格边长，问题固定意味着 N 不变；
- P：处理器数；
- N^2/P：每处理器每步的计算量；

[218] 视频画面时间区间：00:17:30–00:17:45。若为裁剪图，时间区间仍对应原视频画面。

- $N/\sqrt{P}$：每处理器每步的通信量，正比于块周长。

坏消息很明确：P 增大时这个比值按 $1/\sqrt{P}$ 恶化，计算越来越被通信压住，于是 P 的扩展能力被限制住。

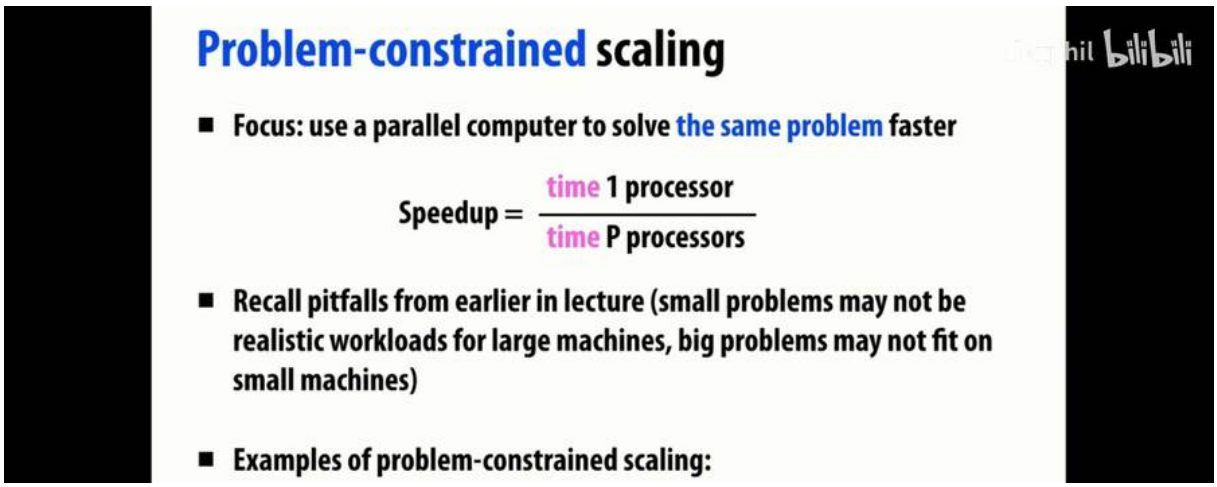

图 219：问题约束扩展：用并行系统更快地解决同一个问题[219]

约束二：时间约束

时间约束的目标是：从 1 个处理器换到 P 个，希望解决尽可能大的问题而总时间不变。

讲者先假设线性加速，即算力变为 P 倍。那么问题规模能放大多少？关键细节在于：网格边长变大时，收敛所需的迭代次数也会增加，因为达到稳定状态的传播时间正比于网格直径。

于是问题规模的两个维度都要按 $P^{1/3}$ 增长：

$$N' = N \cdot P^{1/3}, \qquad I' = I \cdot P^{1/3}$$

- N'：新的网格边长，按 $P^{1/3}$ 放大，即指数为 0.333；
- I'：新的迭代次数，同样按 $P^{1/3}$ 放大；
- 两者相乘使总计算量变为 $P^{2/3} \cdot P^{1/3} = P$ 倍，正好用掉 P 倍算力。

代回去求通信计算比：

$$\left(\frac{C}{W}\right)_{\text{time}} \propto P^{-1/6} \approx P^{-0.167}$$

讲者说这仍然随 P 恶化，但比问题约束好得多——因为我们在放大网格的同时也做了更多时间步，这一项对我们有利。

[219] 视频画面时间区间：00:18:30–00:18:45。若为裁剪图，时间区间仍对应原视频画面。

图 220: 时间约束扩展：在固定时间内完成更多工作[220]

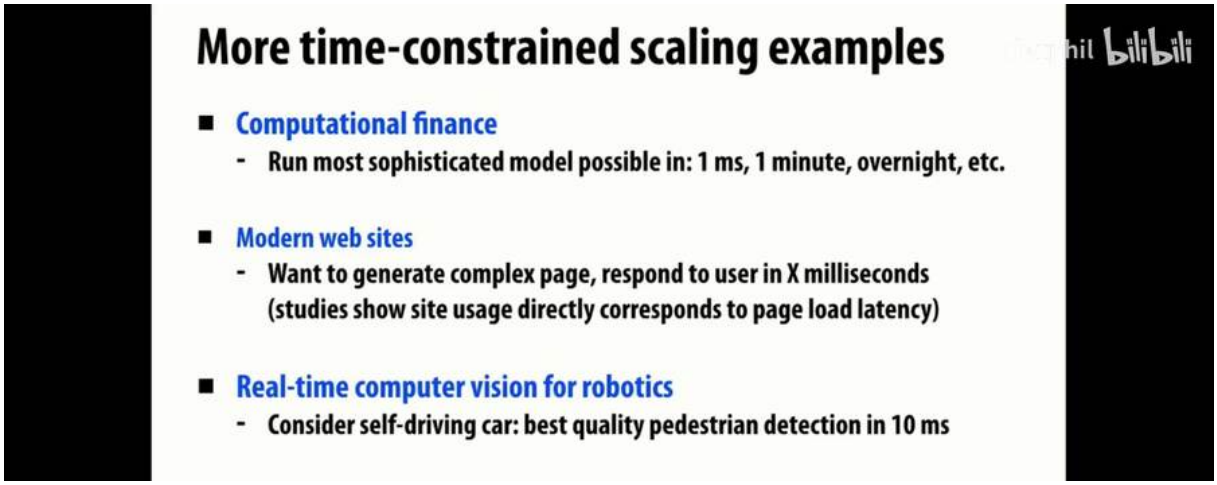

图 221: 时间约束扩展的更多例子：如计算金融中运行更精细的模型[221]

约束三：内存约束

内存约束的目标是：既然有了 P 倍的处理器，就有了 P 倍的内存，那么装下能装的最大问题并跑完它。

总运行时间会更长，但能解决的问题比原来大。关键在于这里的扩展方式：网格边长按 $P^{1/2}$ 增长，使得每处理器的元素数保持不变。

$$\left(\frac{C}{W}\right)_{\text{memory}} = \text{常数}$$

- 每处理器的数据量随 P 增长而保持同比例增长；
- 于是每处理器的计算量与通信量之比不变；
- 通信计算比不随 P 恶化。

[220] 视频画面时间区间：00:20:00–00:20:15。若为裁剪图，时间区间仍对应原视频画面。
[221] 视频画面时间区间：00:24:00–00:24:15。若为裁剪图，时间区间仍对应原视频画面。

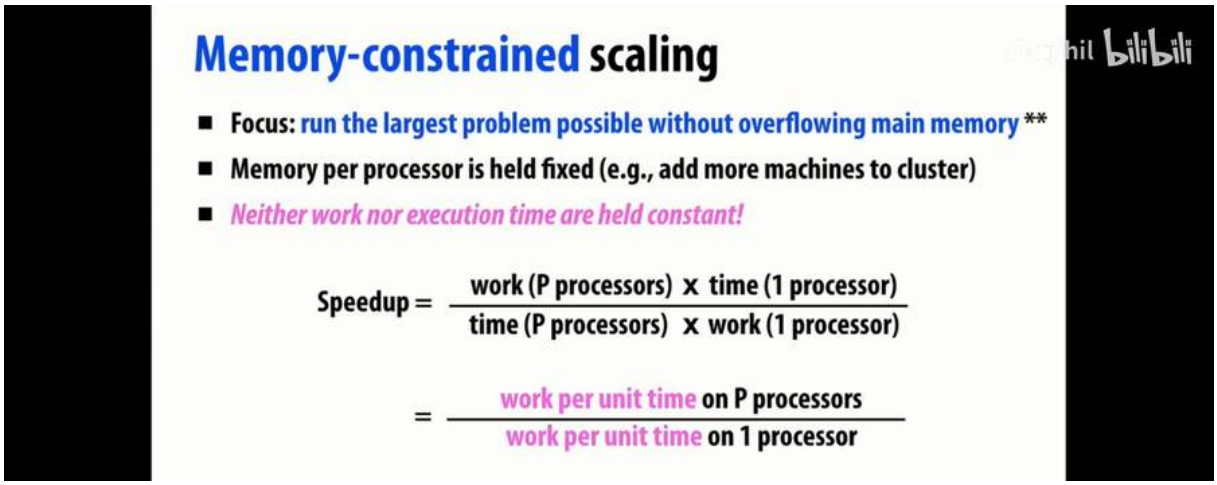

图 222: 内存约束扩展：在内存装得下的前提下运行最大的问题[222]

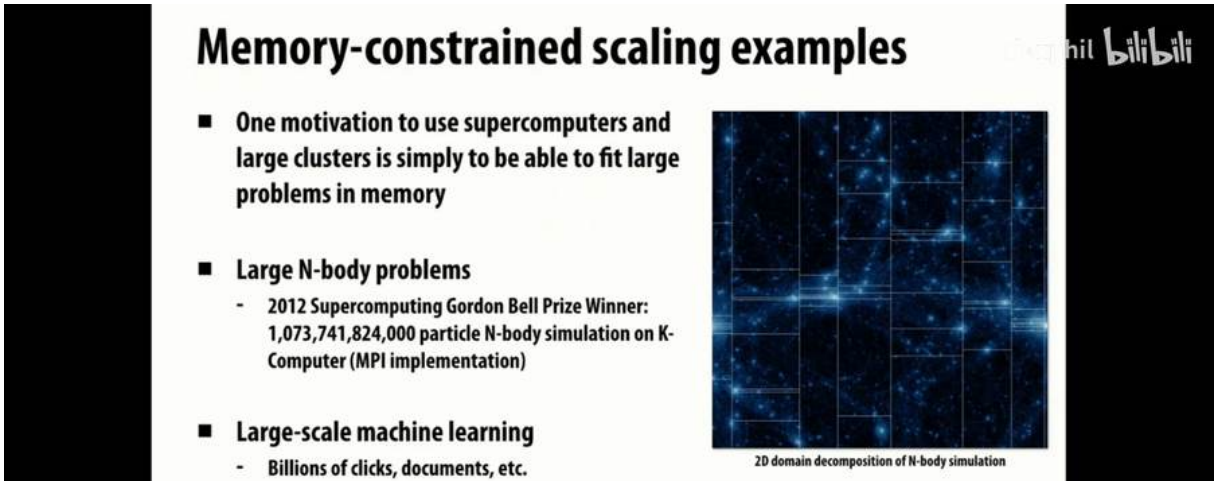

图 223: 内存约束扩展的例子：这是使用超级计算机的主要动机之一[223]

三种约束的排序

讲者给出结论性的排序。

内存约束下效率稳定的几何原因

按内存线性放大时，每处理器分到的元素数保持不变，因此块的面积与周长之比保持不变。通信正比周长、计算正比面积，比值自然恒定。这也说明为什么超算的采购逻辑强调内存墙——扩内存往往比扩算力更能保持并行效率。

内存约束最容易满足：如果原来就是好的，这样扩展会很顺畅。

时间约束次之：仍然会变差，但速率温和。

问题约束最难：也就是经典的加速比，是最难达成扩展目标的一种。

[222] 视频画面时间区间：00:25:30–00:25:45。若为裁剪图，时间区间仍对应原视频画面。

[223] 视频画面时间区间：00:26:30–00:26:45。若为裁剪图，时间区间仍对应原视频画面。

约束类型	问题规模如何变	通信计算比	难度
问题约束	不变	$\propto P^{-1/2}$	最难
时间约束	边长与迭代各按 $P^{1/3}$	$\propto P^{-1/6}$	中等
内存约束	按内存线性放大	不变	最易

表 8: 三种扩展约束下的通信计算比随处理器数的变化

扩展性结论取决于你问的是哪个问题

同一份代码、同一台机器，按问题约束衡量可能”扩展性很差”，按内存约束衡量却”扩展性很好”。这两句话并不矛盾，因为它们问的是不同的问题。因此在评价一个系统的扩展性之前，必须先说明采用哪种约束，否则数字本身没有意义。

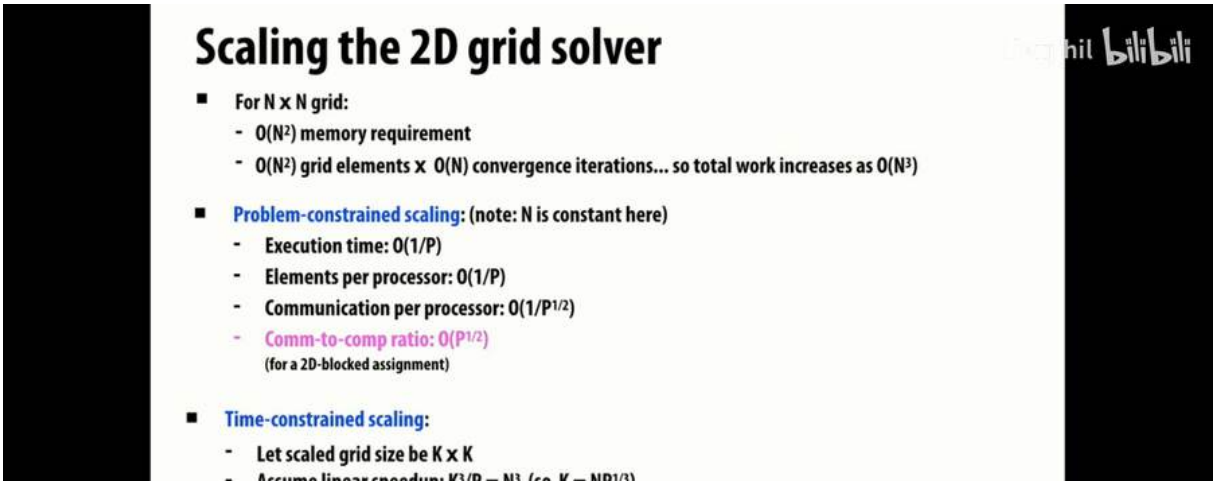

图 224: 扩展二维网格求解器：三种约束下通信计算比的推导[224]

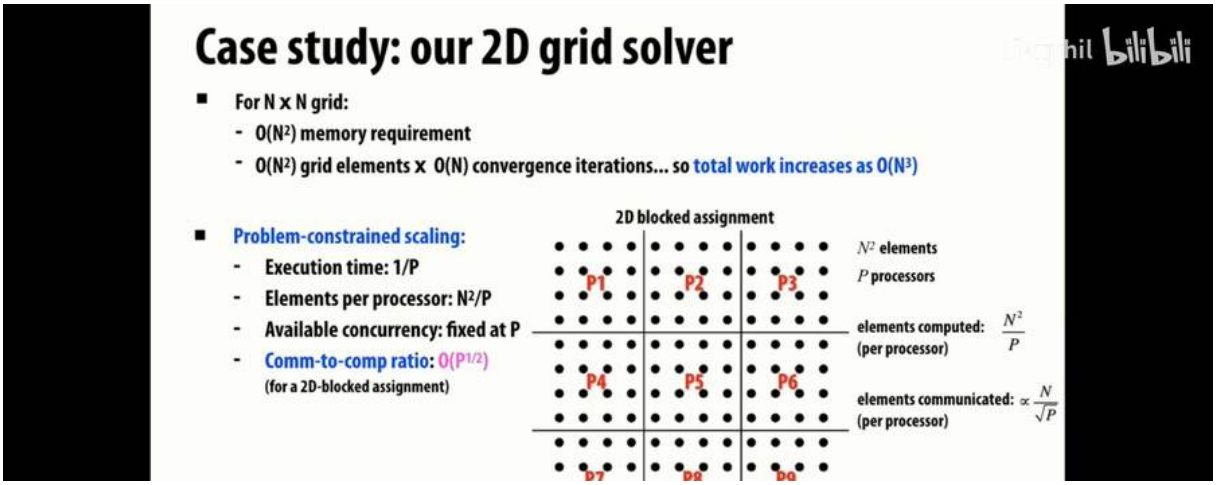

图 225: 案例研究：二维网格求解器的内存与元素复杂度[225]

[224] 视频画面时间区间：00:33:00–00:33:15。若为裁剪图，时间区间仍对应原视频画面。

[225] 视频画面时间区间：00:31:00–00:31:15。若为裁剪图，时间区间仍对应原视频画面。

本章小结

问题约束下通信计算比按 $P^{-1/2}$ 恶化，时间约束下按 $P^{-1/6}$ 温和恶化，内存约束下保持不变。难易顺序依次是问题约束最难、时间约束次之、内存约束最易。这说明” 扩展性问题” 的答案取决于选了哪种约束——离开约束谈扩展性没有意义。

讲者随后提醒，真实情形比这张表更复杂，因为各参数会互相牵动，而实际的使用场景也各不相同。

10.6 真实场景中的扩展与模拟方法

同一应用，不同约束

讲者用 Pixar 的例子说明这一点。渲染一个” 镜头”（即电影中的一段连续帧）时，管线前面各阶段的约束不同，而最后有一遍大规模渲染把整段镜头算出来——那一步受各种因素共同限制。

他的论点不是关于 Pixar 本身，而是一般性的：即使在同一类应用内部，不同的使用场景下也会有不同的约束。

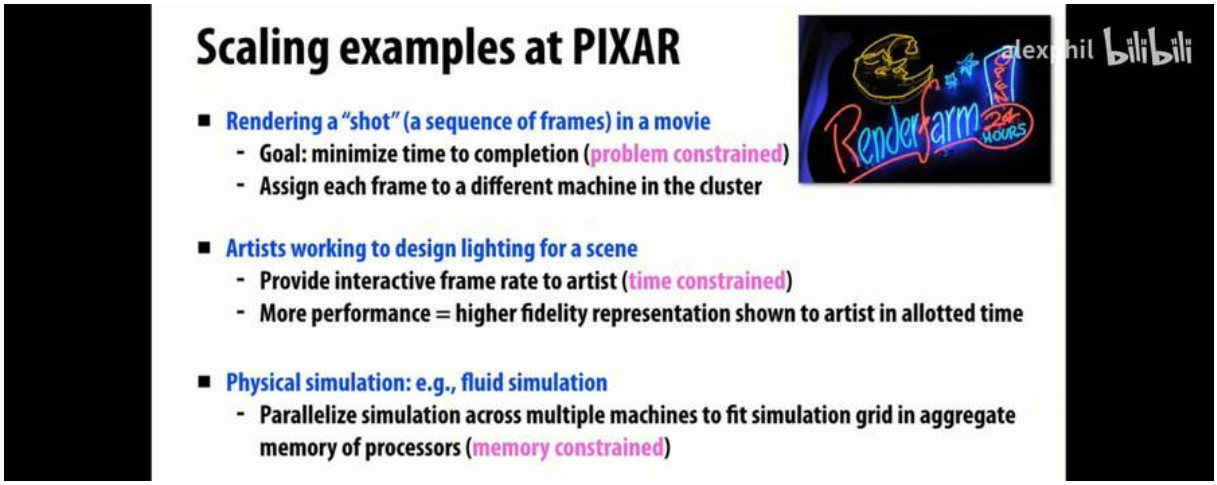

图 226: Pixar 的扩展性案例：渲染一个镜头时各阶段的约束不同[226]

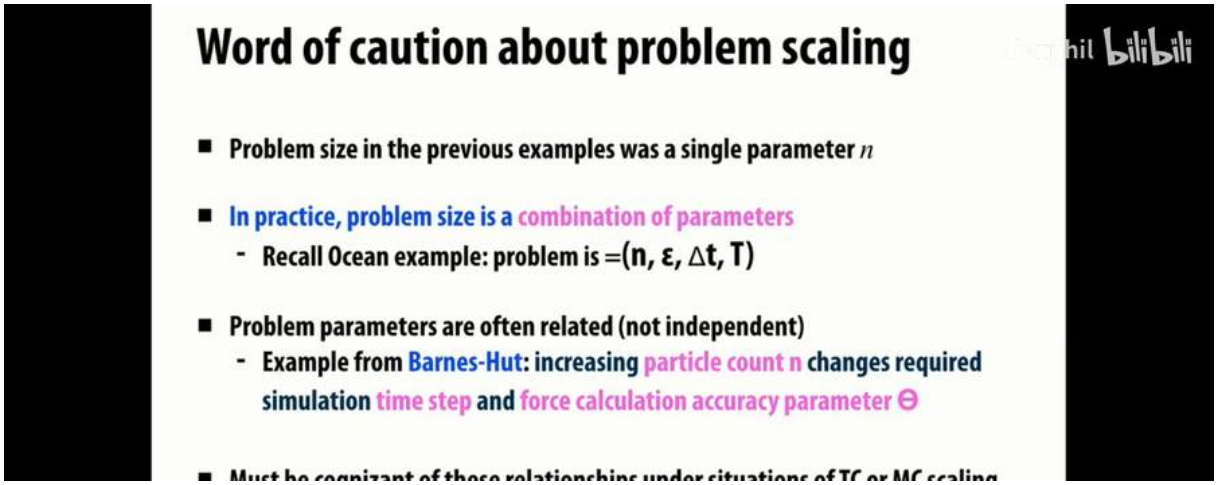

图 227: 关于问题扩展的注意事项：前例中的问题规模只是单项参数[227]

[226] 视频画面时间区间：00:27:30–00:27:45。若为裁剪图，时间区间仍对应原视频画面。

本章小结

同一应用在不同使用场景下有不同的约束，Pixar 的渲染管线就是一例：前面各阶段与最后的整段渲染，瓶颈并不相同。因此扩展性分析必须绑定具体场景，而非笼统地谈论应用。

当要评估的是尚未存在的硬件时，只能依靠模拟。这是下一章的主题。

10.7 用模拟评估架构

为什么要靠模拟

讲者转向另一个话题：架构师怎么评估一个新的设计想法。答案是模拟——因为你不能为了评估就把芯片造出来。

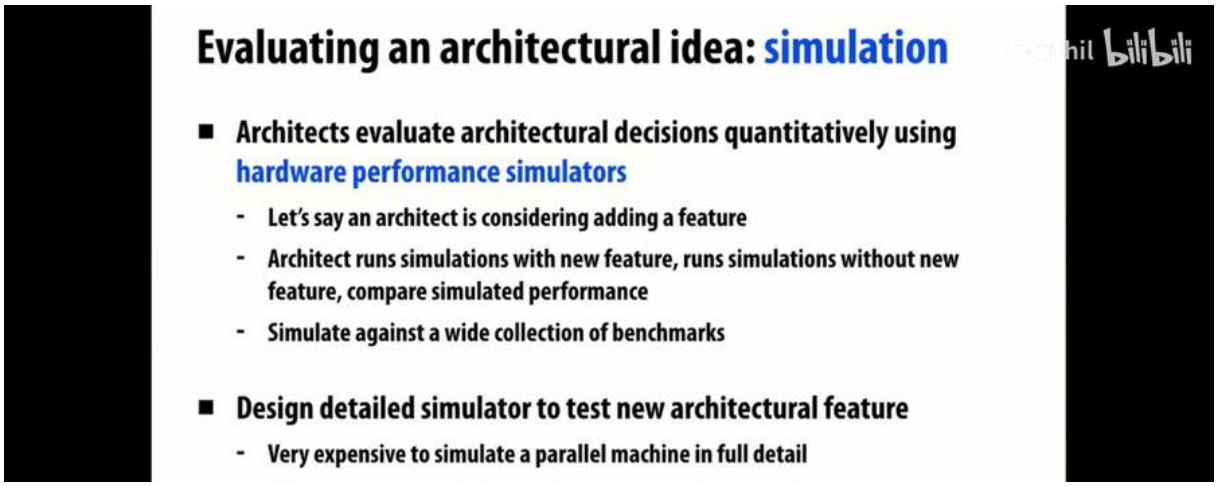

图 228：评估架构想法：架构师靠模拟来量化设计决策[228]

有两种主要做法。

两种模拟方法的保真度与成本互换

跟踪驱动把”程序行为”固定在轨迹里，模拟器只需处理存储层次，因此单次运行快、可扫描大量架构参数。执行驱动需要完整模拟指令流，能捕捉程序对架构的适应，但慢几个数量级。选择哪一种取决于要评估的改动是否会影响程序自身的行为。

跟踪驱动：在真实机器上运行真实代码，记录下所有内存访问等事件的轨迹，然后在模拟器里重放这条轨迹。它不需要真的建模执行，因此快。

[227] 视频画面时间区间：00:37:00–00:37:15。若为裁剪图，时间区间仍对应原视频画面。

[228] 视频画面时间区间：00:38:00–00:38:15。若为裁剪图，时间区间仍对应原视频画面。

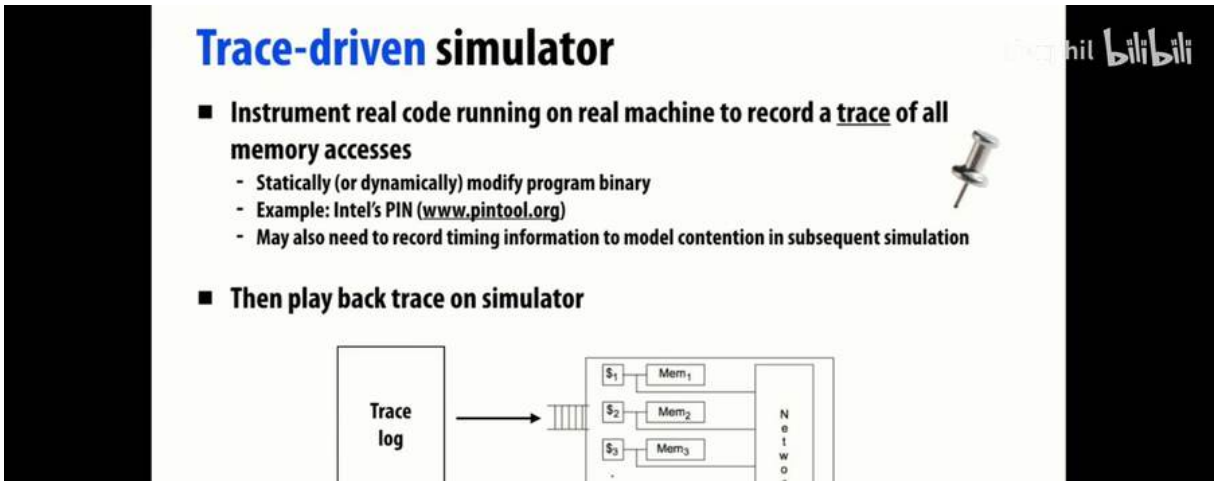

图 229：跟踪驱动的模拟器：对真实代码插桩以记录访问轨迹[229]

评估结论的偏差方向要能预判

知道跟踪驱动会低估新硬件收益，就能解释为什么某些架构改动在模拟中收益平平、在真实原型上却效果显著。反之，若模拟显示巨大收益而实测没有，也应怀疑轨迹没有反映新架构带来的访问模式变化。

执行驱动：直接模拟执行，包含系统的反馈——系统会动态适应计算的实际执行情况，因此不同反馈会引出不同轨迹。它能处理更多情形，但更难构建、运行更贵。

跟踪驱动的两个风险

讲者花了相当篇幅讨论跟踪驱动方法的隐患，这部分对做性能评估的人很有价值。

第一个风险是过拟合。如果反复用同一组轨迹调优，很容易写成专门照顾这些特例的代码。他的建议是理想情况下每次都重新采样问题空间，生成新的样本集——但现实中往往做不到。

第二个风险是开放环路假设。轨迹方法默认新系统中的流量与旧系统完全相同。但在时间约束扩展中，我们希望用同样的时间做更复杂的计算，那么真实轨迹代表的只是固定问题，未必能代表放大后的问题。

用回放轨迹评估新系统会系统性高估收益

轨迹固定了访问模式，而新架构往往会改变访问模式（更大的缓存减少了缺失、更快的互连改变了同步频率）。因此在旧机器上采集的轨迹，用来预测新机器的表现时，会低估新硬件缓解访存压力的效果。执行驱动模拟能部分修正这一点，代价是构建成本。

本章小结

真实应用中不同使用场景对应不同约束，Pixar 的渲染管线就是一例。架构评估依赖模拟，跟踪驱动快但有开放环路与过拟合的风险，执行驱动能反馈但更贵。这些方法问题与前面的扩展性分析是同一件事的两面：都在追问”用什么条件下的数字来代表性能”。

讲者的收尾建议因此很务实：不必靠随机猜测与试错，可以让真实数字驱动决策。

[229] 视频画面时间区间：00:39:30–00:39:45。若为裁剪图，时间区间仍对应原视频画面。

10.8 总结与延伸

讲者收尾时说，做了这么多工作，重点是你不必靠随机猜测和试错去看发生了什么，可以更有原则、由真实数字驱动地做决定。

把全讲串起来，是一次对”性能”这个概念的拆解。加速比有两个陷阱：基线若用带并行开销的伪串行版本会高估收益；问题太小会让每核喂不饱而低估收益。超线性加速多数是缓存合并的假象，少数来自搜索类算法的停止条件，它提醒我们基线必须可比。要问”扩展得好不好”，先得区分强扩展（固定问题）与弱扩展（问题同比放大），而更实用的分法是按约束：问题约束下通信计算比按 $P^{-1/2}$ 恶化，时间约束下按 $P^{-1/6}$ 温和恶化，内存约束下保持不变——所以内存约束最容易、问题约束最难。同一份代码在不同约束下可以同时”扩展性很差”与”扩展性很好”。评估架构则要用模拟，跟踪驱动快但有开放环路与过拟合风险。

本讲义在讲者材料之上做的压缩，有四点需要与讲者原话区分：

- 加速比不是性能的同义词。它衡量的只是”相对于某个基线快多少”，而这个基线的选择本身就是一个需要论证的决定。报告中给出加速比时，必须同时说明基线是什么、问题规模多大。
- 三种约束的排序有几何原因。通信正比边界、计算正比面积，因此只有让每处理器的数据量随 P 同比增长（内存约束），比值才能保持稳定。其余两种都在稀释这个比值。
- 超线性是诊断信号而非成绩。看到它首先该检查基线是否装不进缓存，而不是宣称实现优越。真实存在的例外（搜索类）有明确的机制解释：任一分支成功即完成。
- 模拟方法的偏差方向可以预判。跟踪驱动会系统性低估新硬件的收益，因为轨迹固化了旧的访问模式。知道偏差方向，就能判断结论应当偏向哪一侧。

这堂课的边界在于它讨论的是测量与评估的方法论，不涉及具体应用的调优。讲者明确表示后续作业会让学生自己面对这些选择。

两个可以直接从本讲数据提出的下一步问题：其一，既然问题约束最难，那么在必须做固定问题加速的场合（例如实时渲染有硬性帧率要求），唯一出路是否只剩提高算术强度——也就是改变算法而非改进实现？其二，跟踪驱动假设开放环路，那么当被评估的硬件改动会影响程序行为（例如更强的缓存导致线程调度改变）时，什么条件下回放仍然可信？

Lecture 11 Snooping-Based Cache Coherence

视频时长：1 小时 20 分 47 秒　来源：`https://www.bilibili.com/video/BV1HN411z7gD/`

11.1 缓存一致性：一个被缓存制造出来的错觉

这一讲进入一个新主题，讲者说它介于计算机体系结构与系统设计之间。当你需要写出能在大型共享内存机器上运行的代码时，它变得相当重要。

问题本身是：在一个有多份缓存、但内存是共享的系统里，如何让这些缓存彼此保持一致。

讲者指出这对程序员有实际影响：你的目标机器如何实现一致性，会相当深刻地影响性能。

先把缓存的基本行为过一遍

讲者用一个具体例子回顾缓存设计。假设代码里执行 `volatile int x = 1`。在 C 或 C++ 中，`volatile` 意味着写就是真的写进内存，而不只是落在寄存器里。

假设这个 4 字节变量位于地址 `12345604`，缓存行是 64 字节（相当典型）。那么看低 6 位就能确定行内偏移——找到偏移是 4。于是从这一行的第 4 个字节开始写入，小端序下看起来是 1 0 0 0。

行内偏移由地址的低位直接决定：

$$\text{offset} = A \bmod 2^{\log_2 L}$$

- A：变量在内存中的地址，本例为 `12345604`；
- L：缓存行的字节数，本例为 64，故指数为 6；
- offset：该字节在缓存行内的起始位置，本例为 4。

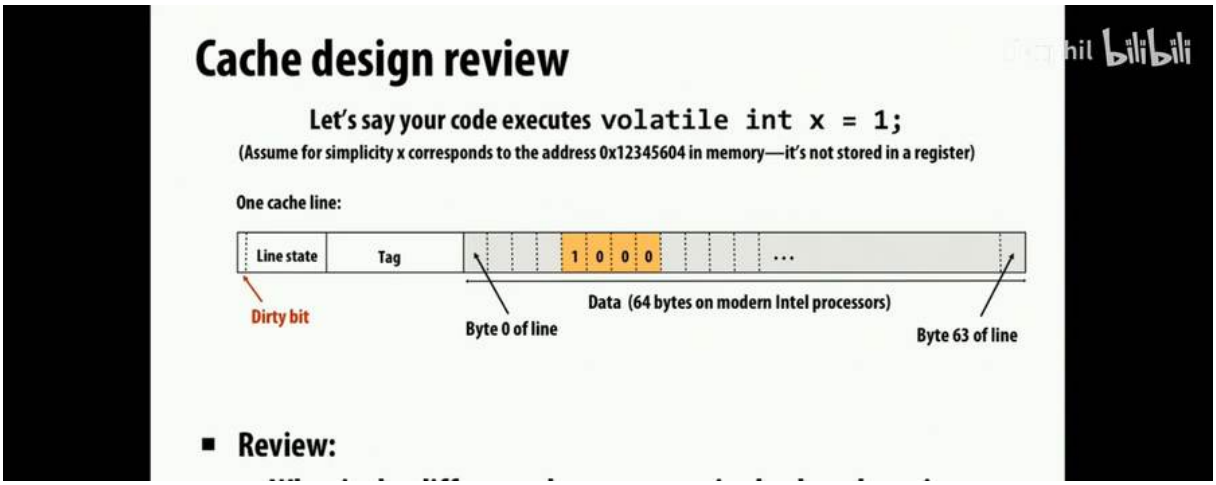

图 230: 缓存设计回顾：volatile 写与缓存行内的偏移定位[230]

[230] 视频画面时间区间：00:02:00–00:02:15。若为裁剪图，时间区间仍对应原视频画面。

回写与写直达

讲者接着回顾两种写策略，这个区分在后面会直接决定协议的复杂度。

回写是指写只落在缓存里，等到某一刻必须覆盖这一行时（容量淘汰，或进程结束），才强制把它写回内存。

写直达是指每一次写都一直穿透到内存。

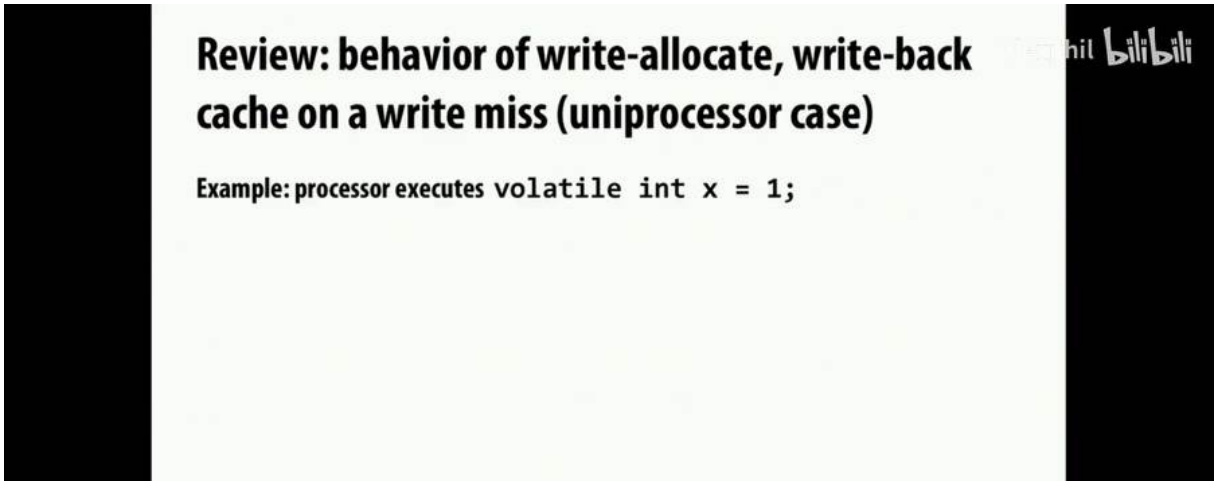

图 231：回顾：写分配、回写缓存在写缺失时的行为[231]

写直达更容易实现，但性能明显更低。它能站住脚的原因是：大多数数据被读的次数远多于被写的次数。

写策略与一致性协议是一对绑定关系

写直达让内存始终是最新的，因此一致性协议只要处理”让别人丢弃副本”这一件事。回写让缓存可能比内存更新，协议就必须额外处理”谁持有最新值、何时交出”的问题。这就是为什么本讲先讨论写直达的两状态协议，再推进到 MSI 三状态协议。

写缺失时的完整步骤

讲者把一次写缺失的处理拆开：如果之前没有读过这个地址、缓存里没有这一行，就要先按放置策略决定它该放哪。放置策略通常是组相联映射限制可能性，再用类似 LRU 的方式在一组里挑牺牲者。

然后分配这个块，把它的当前内容读进缓存，再更新我们关心的那些字节，并把这一行标记为脏。

这里的”脏”有明确含义：缓存里的值与内存里的值不一致。

脏位是”内存已过时”的标记

脏位不是”这个数据很重要”，而是”这份缓存副本比内存新”。它决定了淘汰时必须先写回内存，也决定了多处理器下必须由持有脏副本的缓存负责交出数据。理解这一点，才能理解后面 MSI 中 M 状态为何要承担额外责任。

[231] 视频画面时间区间：00:04:30–00:04:45。若为裁剪图，时间区间仍对应原视频画面。

本章小结

单处理器上的缓存机制本身已包含若干细节：行大小与偏移决定一次写落在何处，写直达与回写决定内存是否会过时，脏位记录两者的差异。这些都是”单机自洽”的。

一旦有多个处理器各自缓存同一批数据，这套机制就会给出彼此矛盾的视图。下一章把这个矛盾具体化。

11.2 一致性问题到底出在哪

一个最小反例

讲者把问题摆出来：在一个共享内存多处理器上，通常我们希望，如果某个处理器写了位置 x，另一个处理器读同一位置时，应当看到那次写。否则就谈不上”共享”。

这个要求听起来天经地义，但缓存让它变得困难，尤其是回写缓存——某个处理器持有的副本可能比主存里的更旧或更新。

他给出的最小场景是这样的：处理器 1 读到 x 的初值 0，把它带进缓存；处理器 2 也读 x，同样从内存拿到 0。到这一步都是对的。

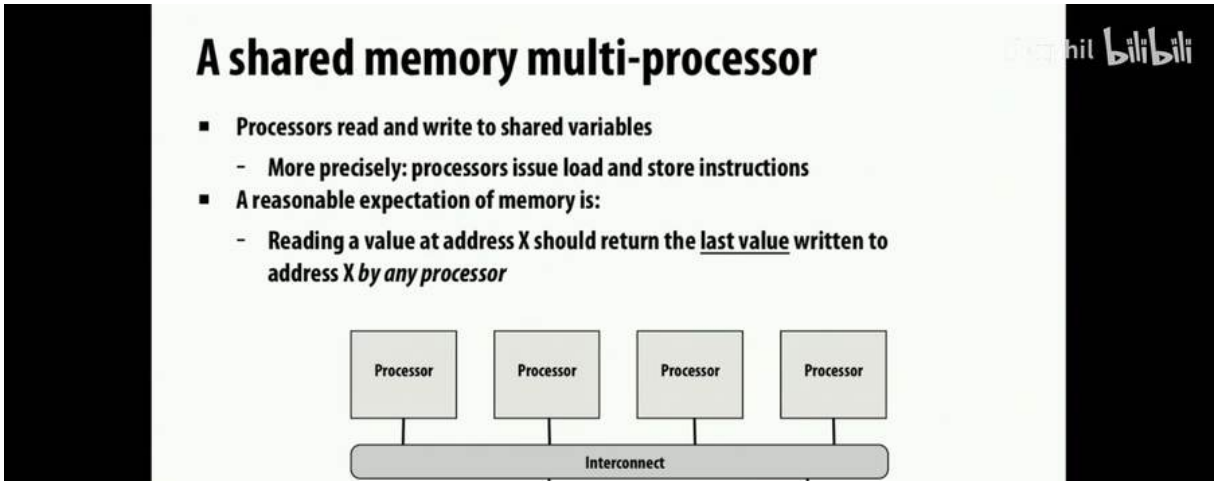

图 232: 共享内存多处理器：处理器读写共享变量[232]

然后处理器 1 往 x 写入新值。即便用写直达，处理器 2 仍然以为它缓存里有 x，而它持有的值仍是 0。除非我们做点什么，否则它会一直认为自己看到的是 0。

若用的是回写缓存，情形更糟：处理器 3 来读 x 时，会从内存读到仍然是 0 的旧值。

[232] 视频画面时间区间：00:06:30–00:06:45。若为裁剪图，时间区间仍对应原视频画面。

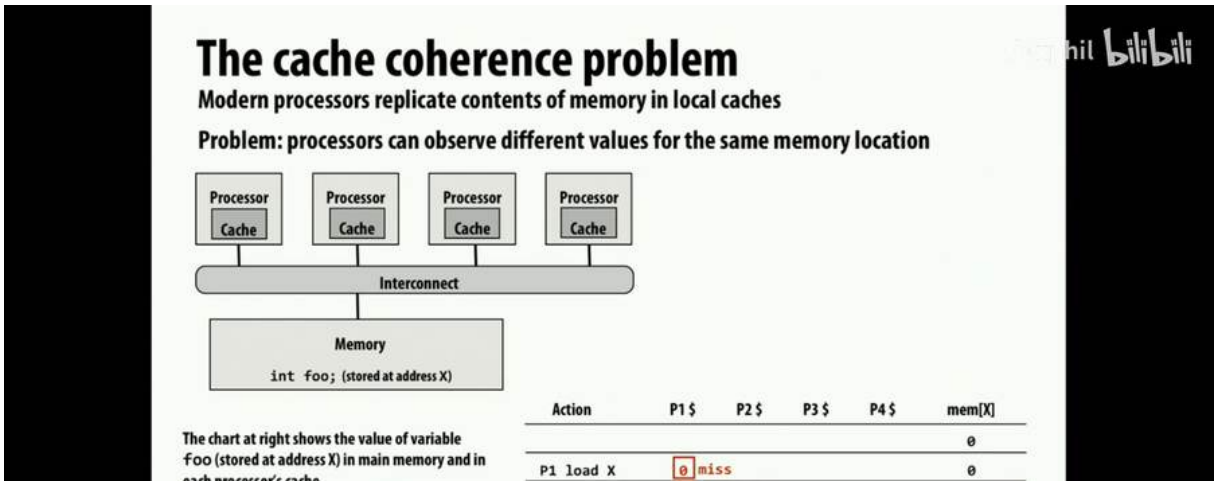

图 233: 缓存一致性问题：现代处理器在本地缓存中复制可写数据[233]

为什么必须由硬件解决

讲者强调这件事必须在硬件中处理：缓存的运作在纳秒量级，靠软件手段管理不可能有可接受的性能。

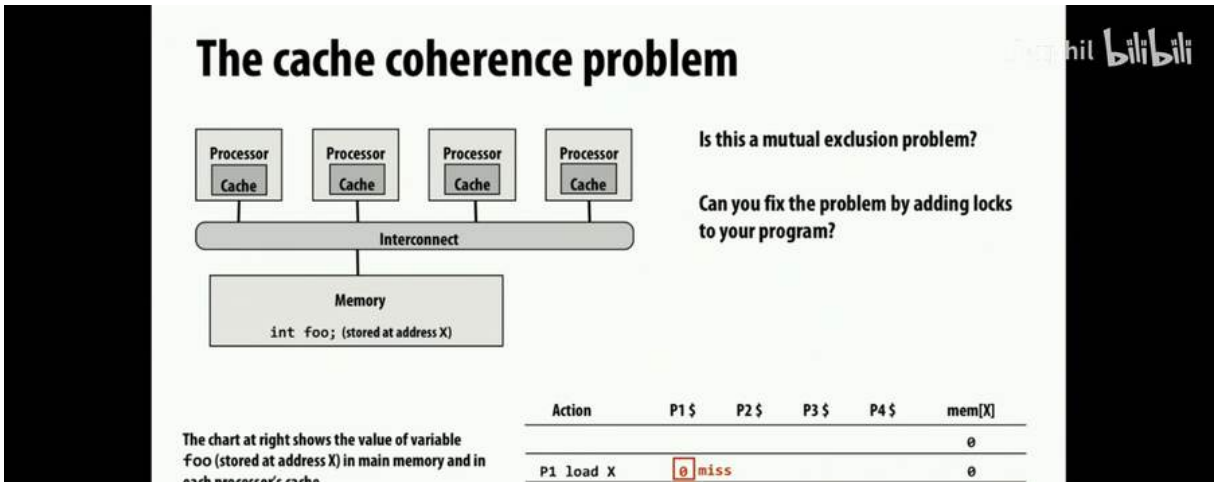

图 234: 这算是互斥问题吗：一致性问题的另一种表述[234]

他在这里做了一个概念澄清：我们想要的是——每当读取位置 x，读到的应当是该地址被任何处理器最近写入的值。

这个目标其实在维护一个错觉。缓存的全部意义是把内存层次藏起来，让程序员以为存在一份单一的内存。但物理上那只是一堆分层的存储。讲者的原话是：这是我们作为缓存设计者的职责，把这个错觉修好。

[233] 视频画面时间区间：00:07:30–00:07:45。若为裁剪图，时间区间仍对应原视频画面。
[234] 视频画面时间区间：00:10:00–00:10:15。若为裁剪图，时间区间仍对应原视频画面。

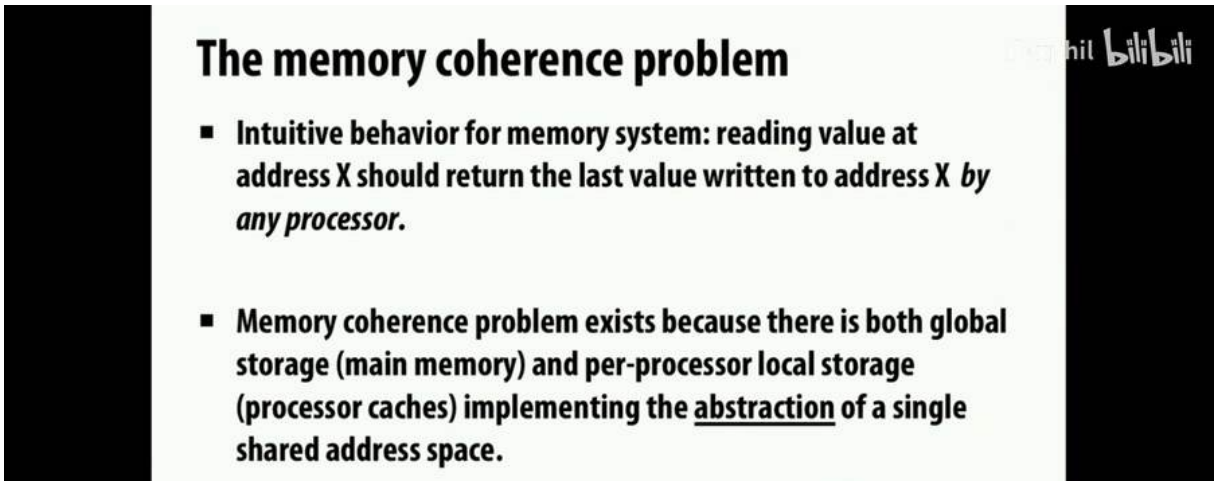

图 235: 内存一致性问题：对内存系统行为的直觉预期[235]

不依赖一致性保证时，正确性由软件承担

在不提供一致性的平台上，跨线程共享可写数据会得到不确定结果。若确实需要共享，必须使用平台明确支持的同步原语（原子操作、栅栏），而不能依赖”读完就能看到别人写的值”这一直觉。这是移植代码时最常见的隐性 bug 来源。

他也指出，市面上确实存在不保证一致性的系统。但如果我们希望用户把内存当成抽象的单一对象、让缓存与否不改变程序算出的结果，那就必须处理它。

一致性动作发生在哪一层

讲者展示了常见的缓存层次：从处理器向外，有两级每核私有的缓存，然后是一个大得多的、所有核心共享的 L3，再往外是主内存。

关键在于：处理器之间的一致性动作发生在 L2 与 L3 之间，那里正是可以插入硬件维持一致性的位置。

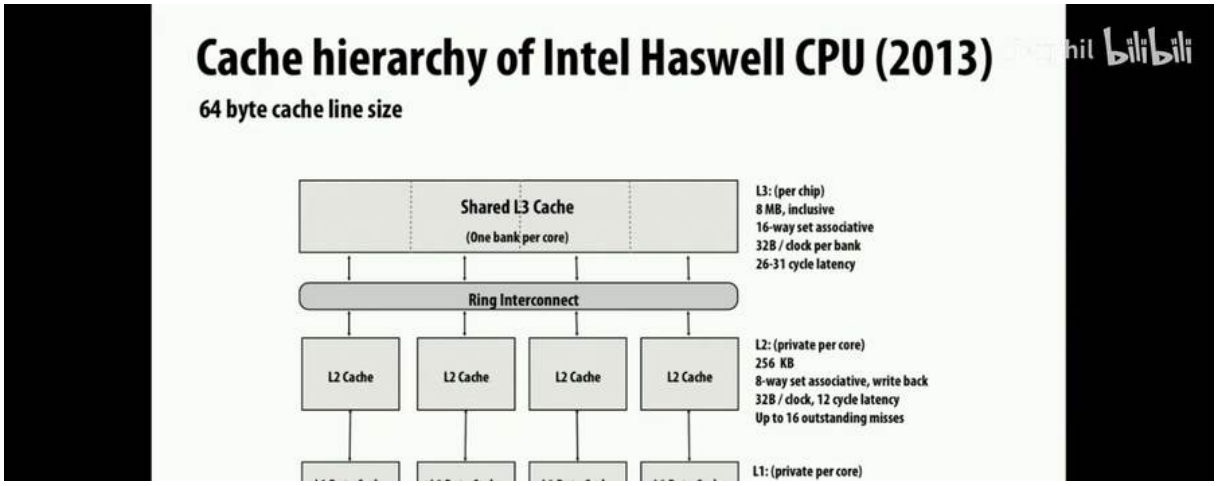

图 236: Intel Haswell（2013）的缓存层次：L1/L2 私有，L3 共享[236]

[235] 视频画面时间区间：00:10:30–00:10:45。若为裁剪图，时间区间仍对应原视频画面。

他还提到一个实现细节：物理上 L3 被切成若干扇区或分块，每一块与某个核心物理关联，但硬件把它当作单一的一致单元来管理。因此除了轻微的性能影响，程序员看不到这个划分。

单核也不简单

讲者顺带指出：即使在单个处理器内部，保持一致也不平凡。由于流水线设计，处理器维护着一个待定写操作的缓冲队列，类似 FIFO。每次加载都必须扫描所有待定的写，找出最近的那条。

另外，DMA 也是麻烦来源——IO 通常通过它工作：处理器填好内存缓冲，然后通知控制器取走；读磁盘时则是给控制器一个内存位置，让它做完后发中断通知。于是即使在单 CPU 上，也有两个主体在读写同一批内存。

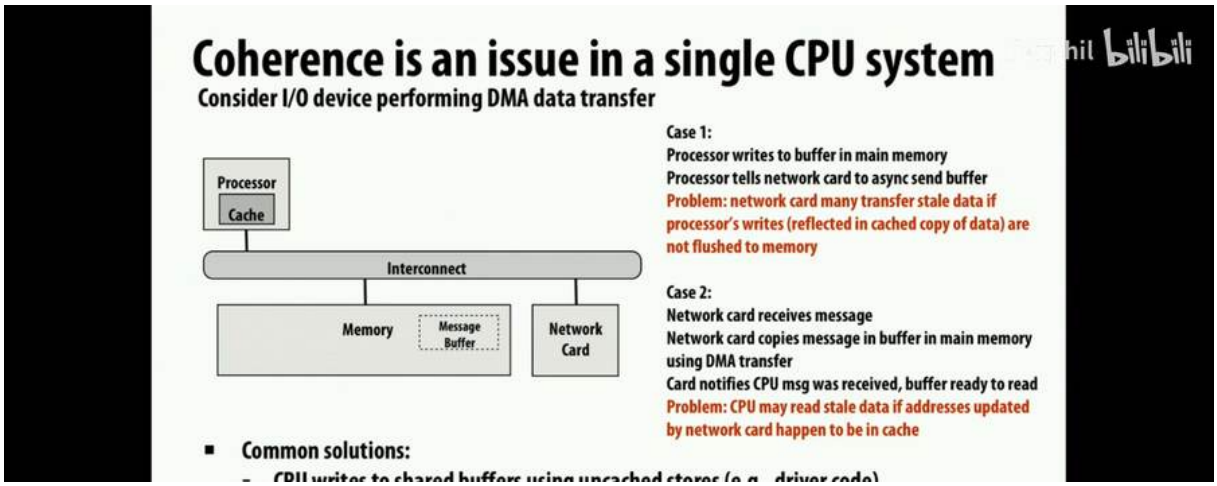

图 237: 单 CPU 系统中一致性同样是问题：DMA 让设备成为第二个主体[237]

讲者说这些单处理器的问题我们今天先假定已解决，专心对付多处理器的问题。

本章小结

一致性问题的最小形态是：两个处理器缓存了同一地址，一个写入后，另一个仍持有旧副本。回写缓存让情况更糟，因为内存本身也可能是过时的。这类问题必须在硬件里解决，因为时间尺度在纳秒级。在真实机器上，一致性逻辑插在私有缓存与共享缓存之间。

要谈” 解决”，先要说清” 保证什么”。这需要一个精确定义。

11.3 一致性的精确定义

为什么不能直接要求顺序一致性

最直觉的定义是：看起来像是所有内存事务按某个全局顺序发生。

讲者说这不是一个好的操作性定义。除非真的放一个单一权威机构来决定事件顺序，否则顺序一致性并不是可行的实现策略。

[236] 视频画面时间区间：00:12:00–00:12:15。若为裁剪图，时间区间仍对应原视频画面。

[237] 视频画面时间区间：00:15:30–00:15:45。若为裁剪图，时间区间仍对应原视频画面。

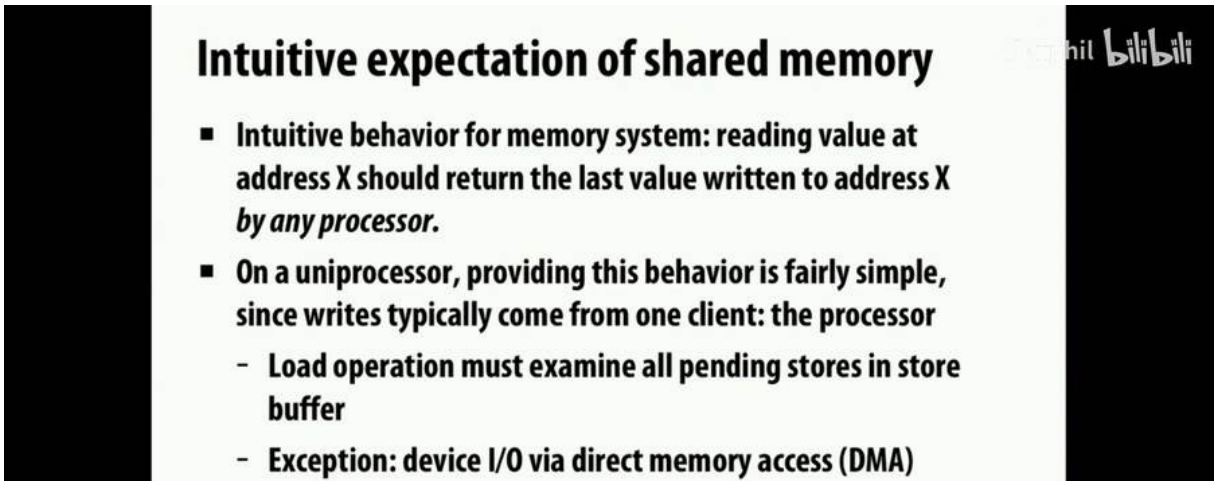

图 238: 对共享内存的直觉预期：读到最近的写[238]

于是他改为按” 实际能实现什么” 来描述。

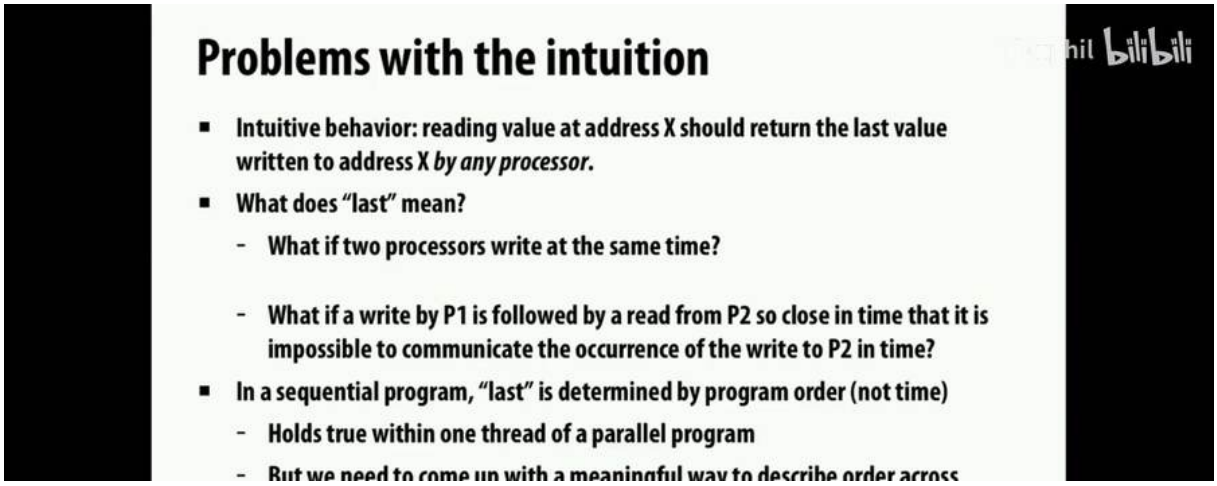

图 239: 直觉的问题：读写交互的实际行为需要更精确的定义[239]

[238] 视频画面时间区间：00:13:00–00:13:15。若为裁剪图，时间区间仍对应原视频画面。

[239] 视频画面时间区间：00:16:00–00:16:15。若为裁剪图，时间区间仍对应原视频画面。

三条不变量

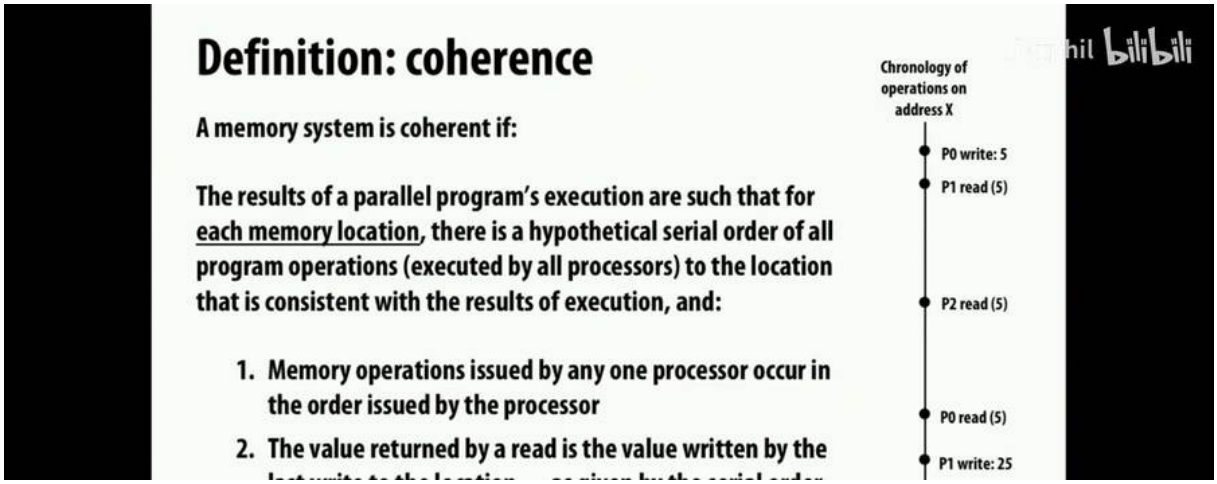

图 240: 一致性的定义：每个处理器对同一地址的操作必须符合程序顺序[240]

第一条是本地视角：若处理器 p 读取地址 x，它应当读到 p 自己最近写入的值，除非期间有其他处理器对该地址写过。这相当于单处理器上的串行化。

形式化地写成：

$$\text{read}_p(x) \Rightarrow \text{value} = \text{last_write}_p(x) \quad \text{若无他人介入}$$

- $\text{read}_p(x)$：处理器 p 对地址 x 的读；
- $\text{last_write}_p(x)$：p 自己最近一次对 x 的写；
- 条件”若无他人介入”排除期间其他处理器写过 x 的情形。

第二条是最终可见：若处理器 1 对 x 写入，经过足够时间后处理器 2 读 x 就能读到那个值。讲者承认”足够时间”这个说法有点含糊，但这就是最终一致性或最终可见的意思。

写串行化是全局共识而非局部顺序

第一条只约束单个处理器自己的读写顺序，第三条要求不同处理器之间对同一地址的写顺序达成共识。前者靠本地信息即可维护，后者必须借助某种全局串行化机制（往往是互连）。这正是协议实现中最昂贵、也最容易出错的部分。

第三条是写串行化：如果对同一地址有多次写，所有可能观察到这些写的处理器，必须以相同的顺序看到它们。

三条不变量合起来才等于可线性化

第一条保证每个处理器自己的读写顺序不被颠倒；第二条保证写最终会被看到；第三条保证所有观察者对同一地址的写入顺序有共识。少了任何一条，整个系统就不存在一个能解释全部观测的全局顺序。

[240] 视频画面时间区间：00:17:30–00:17:45。若为裁剪图，时间区间仍对应原视频画面。

违背写串行化会怎样

讲者用构造性例子说明第三条为什么必要。

设处理器 1 向 x 写入值 A，处理器 2 向 x 写入值 B。处理器 3 与处理器 4 各自做一串读。

若处理器 3 读到的是先 A 后 B，而处理器 4 读到的是先 B 后 A，那就没有任何办法把这个读写序列串行化并得到一致的结果。这种情况违背了顺序一致性这个全局目标，因此是不可接受的方案。

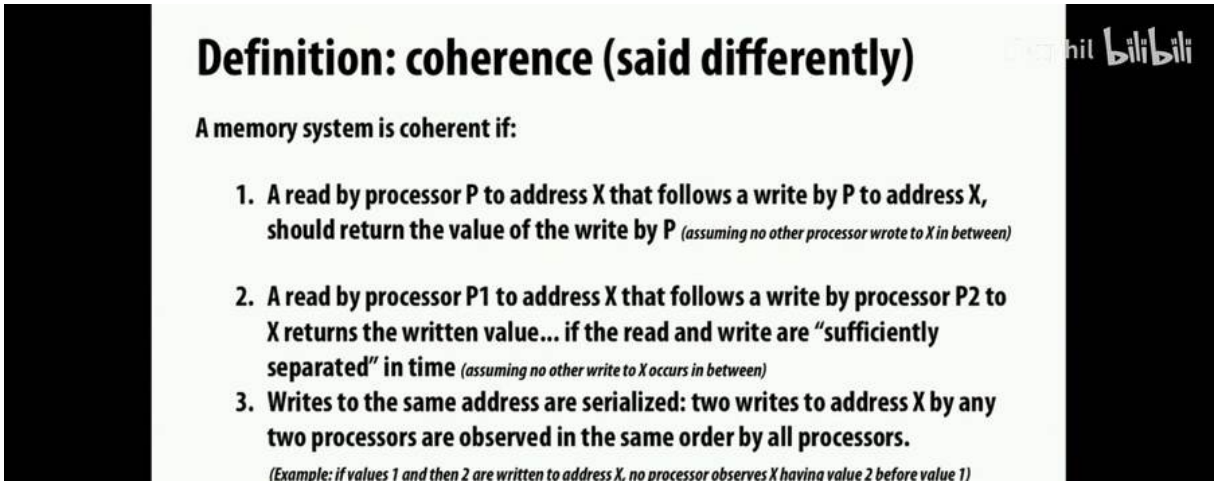

图 241: 换一种说法的一致性定义：三条要求并列表述[241]

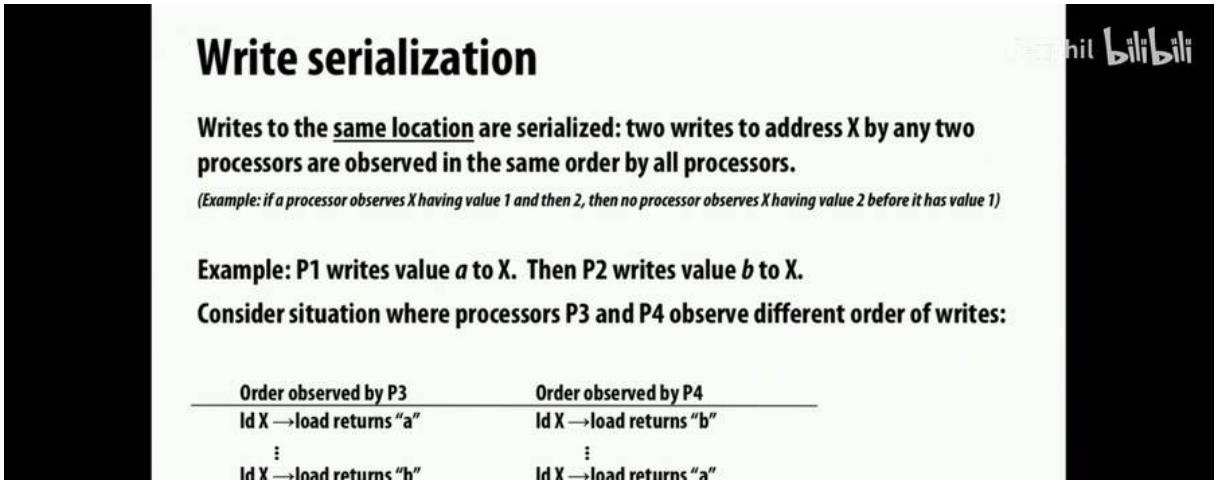

图 242: 写串行化：同一地址的两次写必须让所有观察者看到相同顺序[242]

讲者说可以证明：只要遵守这三条，就能保证所需的可串行化，但这个证明超出本课范围。

本章小结

一致性的可行定义由三条不变量组成：单处理器的读写顺序、写的最终可见性、同一地址写的全局顺序一致。第三条的必要性可以用”两个读者看到相反顺序”的构造证明。这个定义是可以落地的，而”存在一个全局全序”的表述则难以直接实现。

[241] 视频画面时间区间：00:19:30–00:19:45。若为裁剪图，时间区间仍对应原视频画面。

[242] 视频画面时间区间：00:22:30–00:22:45。若为裁剪图，时间区间仍对应原视频画面。

有了定义，就可以讨论实现方案。

11.4 snooping 的基本思路

方案选择

讲者先列出实现一致性的几条路。软件方案（例如操作系统用页错误机制传播更新）太慢——那是毫秒量级，而这里需要纳秒量级。

剩下的主流做法是：让共享缓存承担中心角色。既然一个缓存是共享的，它就是一个能监听内存事务的全局实体，可以据此决定如何搬运数据以保持一致。

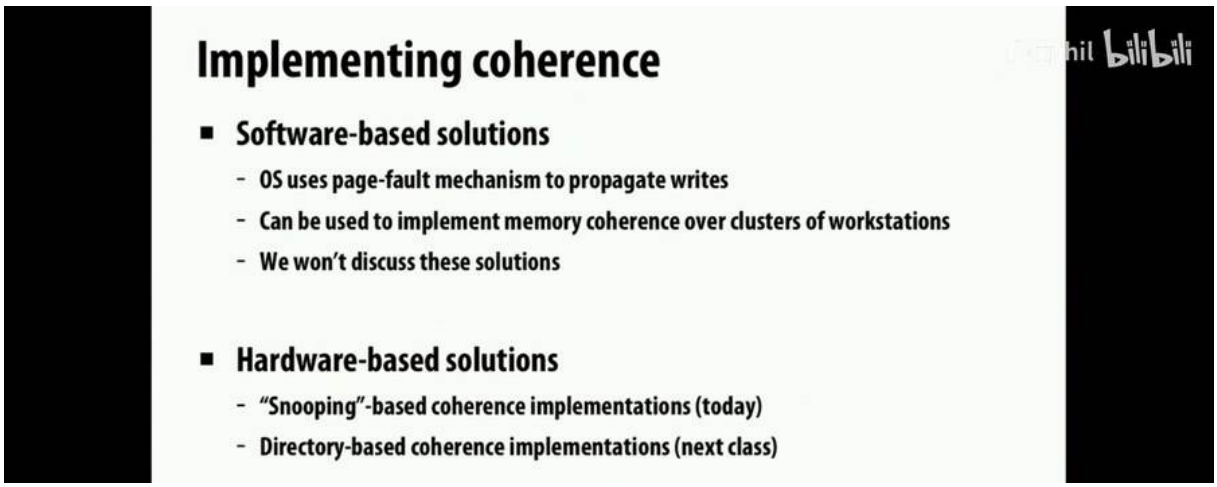

图 243: 实现一致性的方案选择：软件方案太慢[243]

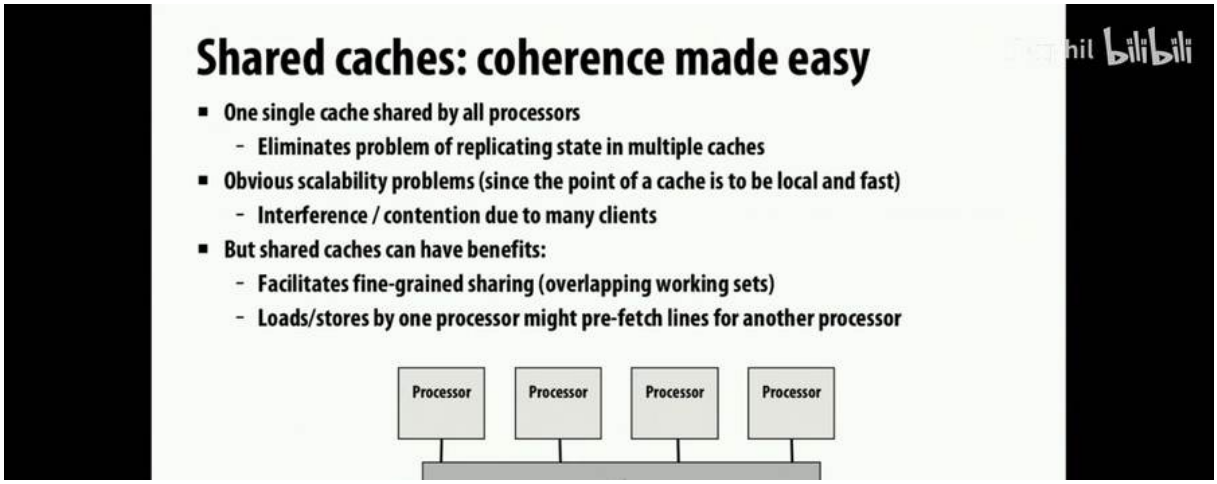

图 244: 共享缓存让一致性变得容易：单一缓存消除了私有副本[244]

snooping 的机制

今天讲的是最简单、也最标准的做法：基于监听（snooping）的缓存一致性协议。讲者预告下一讲会讨论更具扩展性的方案——目录式。

[243] 视频画面时间区间：00:23:30–00:23:45。若为裁剪图，时间区间仍对应原视频画面。
[244] 视频画面时间区间：00:25:00–00:25:15。若为裁剪图，时间区间仍对应原视频画面。

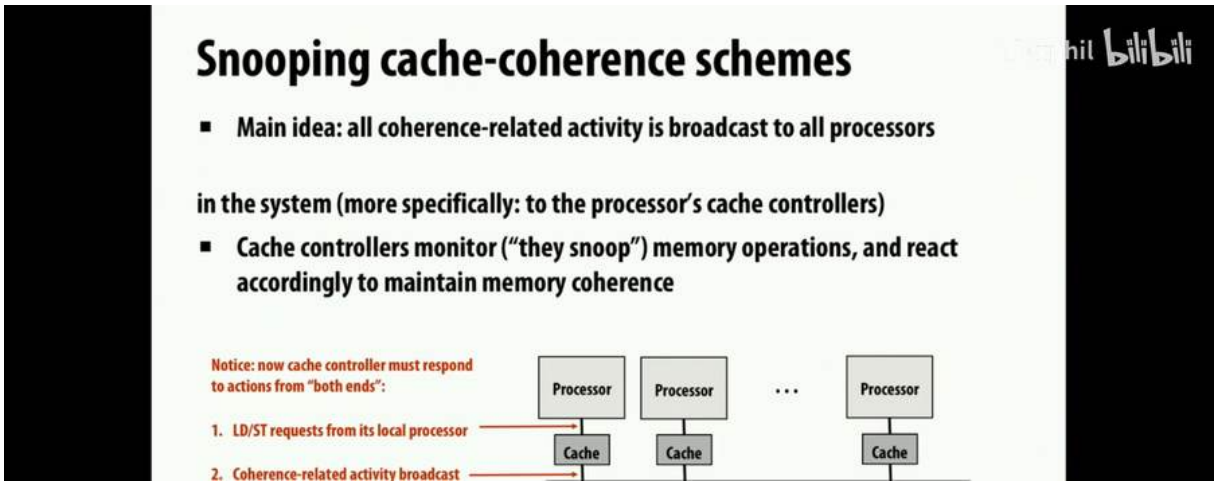

图 245: snooping 方案：所有与一致性相关的活动广播到共享互连[245]

核心是利用缓存的集中性。这个共享缓存提供了一个全局实体，可以监听内存事务、决定如何移动数据，让它看起来是一致的。

具体做法是给缓存控制器增加逻辑，使它们不只响应自己处理器的请求，也响应来自互连的请求。互连上会有消息飞来飞去，是各缓存之间在互相通告" 我关心的那些行现在是什么状态"。

代价随之而来：互连上会有流量，不只是缓存到内存的信息，还有缓存到缓存的通信。讲者指出这会限制可扩展性，但方案相当直接，也是当今大多数多核处理器采用的做法。

snooping 的代价随核心数增长

广播机制意味着每个一致性事件都要通知所有缓存。核心数增加时，互连流量按参与者数量增长，而互连带宽增长有限。因此 snooping 的可扩展性有天然上限，这也正是后续要引入目录式协议的原因。在核数很多时，即使正确性无问题，性能也会被一致性流量拖住。

协议讨论的三个前提

讲者交代了讨论协议时要注意的几点。

第一，每个缓存控制器负责管理自己本地缓存的状态。它们之间通过消息互相告知该做什么，同时也响应与自己缓存绑定的处理器的加载与存储请求。

第二，和任何协议一样，前提是所有缓存都遵守同一套规则。如果有一个缓存行为异常（对抗性的或设计错误的），它就能破坏整个系统。

第三，状态是以缓存行为单位记录的。为简单起见，暂时只考虑单字缓存行，因此不必处理" 为了改一个字节要先读入整行" 的问题。

[245] 视频画面时间区间：00:25:30–00:25:45。若为裁剪图，时间区间仍对应原视频画面。

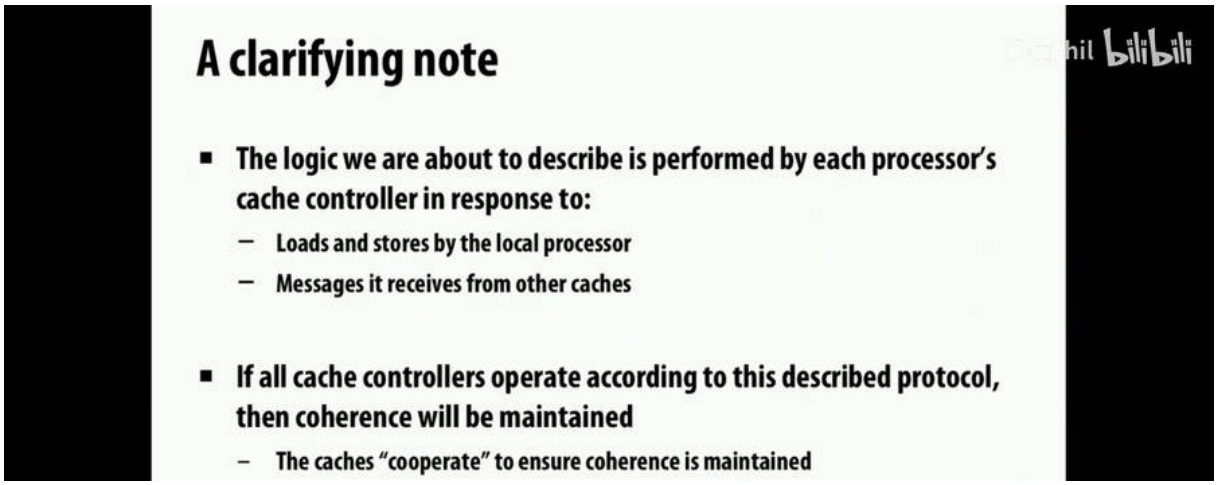

图 246: 一处澄清：描述的逻辑由每个处理器的缓存控制器执行[246]

本章小结

一致性必须由硬件实现，主流方案是把共享缓存当作监听中心，各缓存控制器在互连上广播事务。代价是互连流量随参与者增长，限制了可扩展性。协议的讨论以缓存行为单位、并假设所有控制器遵守同一套规则。

下面从最简单的协议开始。

11.5 入门协议：写直达加失效

假设与做法

讲者先设定最简化的情形：写直达缓存，单字缓存行。

于是每次处理器写，缓存就向内存写直达，并且在互连上发出消息说" 我在写地址 X"。任何其他持有该地址副本的处理器，在这个简化版本里就应当作废自己的条目。

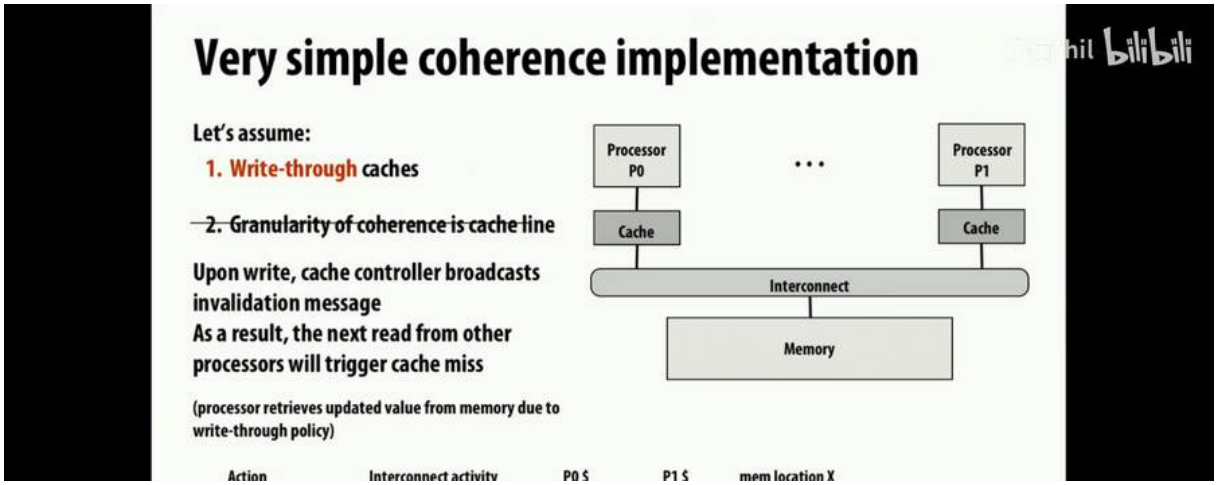

图 247: 非常简单的实现：写直达加失效[247]

[246] 视频画面时间区间：00:30:30–00:30:45。若为裁剪图，时间区间仍对应原视频画面。

[247] 视频画面时间区间：00:27:00–00:27:15。若为裁剪图，时间区间仍对应原视频画面。

这是一个基于失效（invalidation）的协议：通过作废可能变得陈旧的副本，来维持一致。讲者提到还有另一类基于更新（update）的协议，会在稍后瞥一眼。

失效与更新是处理陈旧副本的两条路

失效协议广播”这行变脏了，你们作废”，之后谁要用谁再取一次。更新协议广播”这行的新值是……”，让其他副本直接跟上。前者在写频繁而读稀少时更省带宽，后者在多个读者反复读同一行时更省延迟。商用多核几乎都选失效。

两状态状态图

这样一个协议可以用极简的状态图描述：每个缓存行只有两个状态，有效或无效。

因为是写直达，不存在”脏行”这种东西。

讲者说明了状态图的约定：实线表示由处理器动作引发的迁移，虚线表示由其他缓存控制器发来的消息引发的迁移。每条弧上左边写触发事件，右边写控制器应当执行的动作。

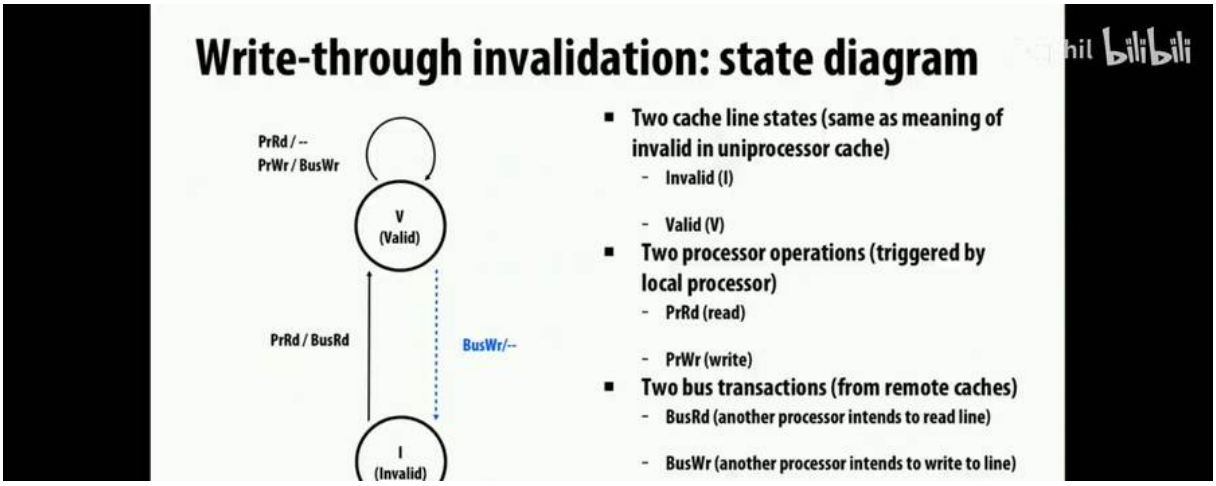

图 248: 写直达加失效的状态图：只有有效与无效两个状态[248]

典型路径是这样的：从一个无效行开始，读取它，把它带入缓存并标记有效，然后读值。此后若只是反复读，它可以一直保持有效，无需产生任何额外流量。

为什么不够

讲者随即指出这个策略效率不高：每次写操作都要走到内存，带来很高的总线流量。

在简单场景下这还能接受，但它把一致性的成本与写操作的频率直接绑定了——写越多，总线越挤。这就促使向回写缓存下的协议演进。

[248] 视频画面时间区间：00:32:00–00:32:15。若为裁剪图，时间区间仍对应原视频画面。

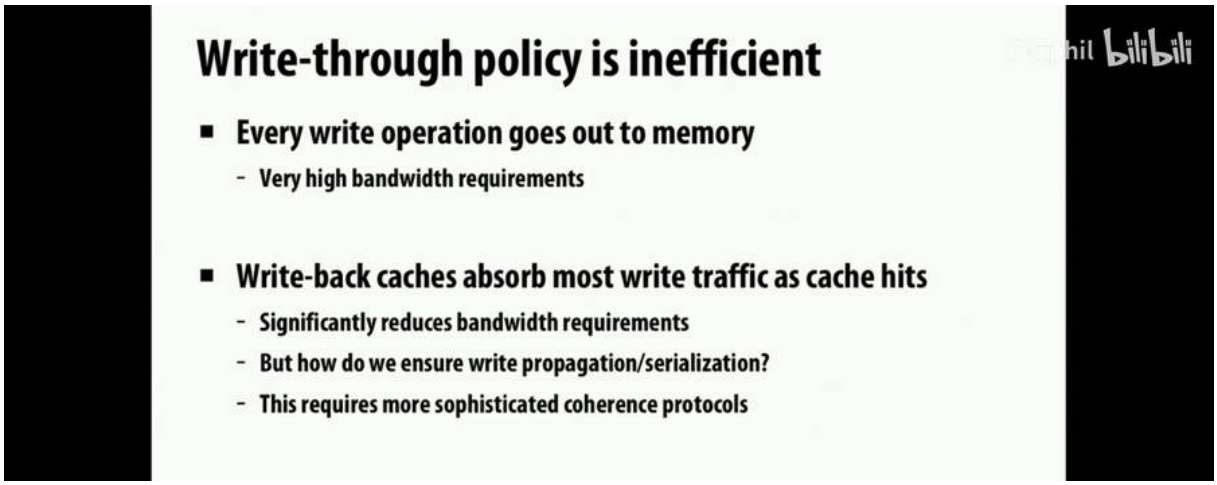

图 249: 写直达策略效率不佳：每次写都要占用总线[249]

本章小结

写直达加失效的两状态协议是把一致性引入缓存的最小可行方案：写时广播、他人作废。它逻辑清楚，但因为每次写都要上总线而不够高效，也无法利用回写带来的本地写优势。

要在回写缓存上做一致性，就必须面对”缓存可能持有比内存更新的副本”这个问题。

11.6 MSI 协议

回写缓存带来的新问题

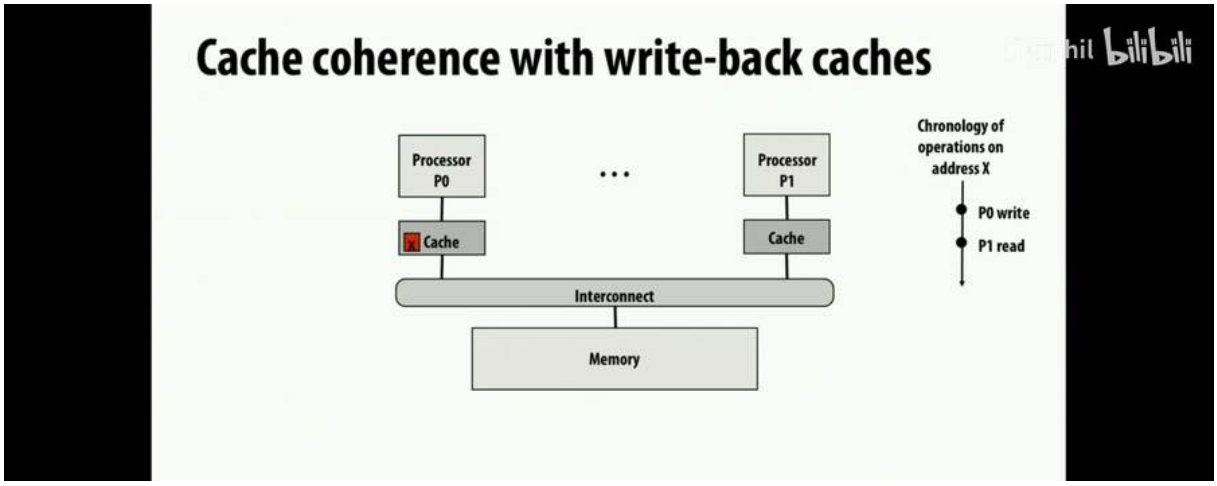

图 250: 回写缓存下的一致性问题：内存可能持有过时的值[250]

讲者说明为什么要专门讨论回写缓存下的一致性：此时内存里的值可能已经过时，最新的副本藏在某个缓存里。协议必须知道最新副本在哪。

[249] 视频画面时间区间：00:36:30–00:36:45。若为裁剪图，时间区间仍对应原视频画面。

[250] 视频画面时间区间：00:37:30–00:37:45。若为裁剪图，时间区间仍对应原视频画面。

三个状态

MSI 的名字来自它的三个基本状态。讲者的做法是把” 有效” 拆成两部分。

共享状态：存在可能的多份副本，它们彼此相同。

已修改状态：只存在于某一个缓存中，且可能与内存不同（脏）。

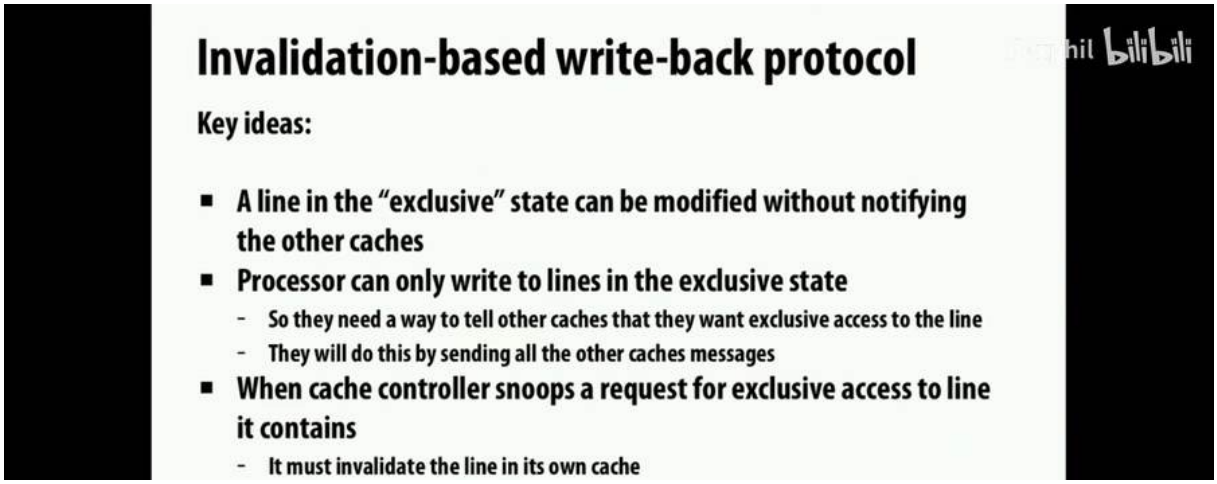

图 251: 基于失效的回写协议：核心要点是独占状态下可本地任意修改[251]

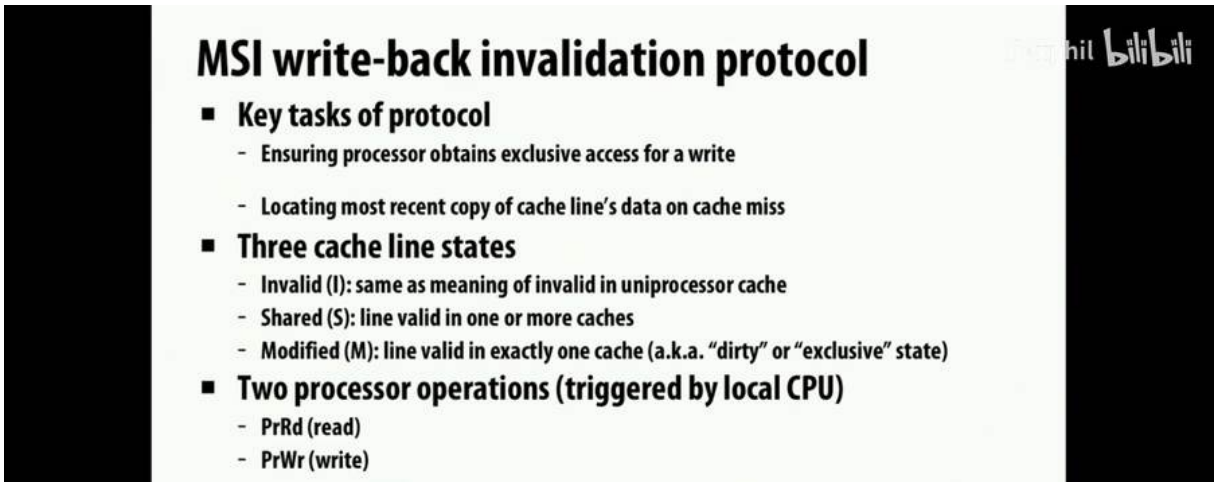

图 252: MSI 回写失效协议：协议需要完成的关键任务[252]

总线上的三类事务

讲者列出互连上可能出现的三类事务：

只读读入——想要一份副本，但计划只读。

独占读入——想要读入，但计划随后就修改。之所以还需要读，是因为缓存行可能不止一个字，需要保留其余部分。

写回或刷新——把修改过的数据交出去。

[251] 视频画面时间区间：00:40:30–00:40:45。若为裁剪图，时间区间仍对应原视频画面。

[252] 视频画面时间区间：00:41:30–00:41:45。若为裁剪图，时间区间仍对应原视频画面。

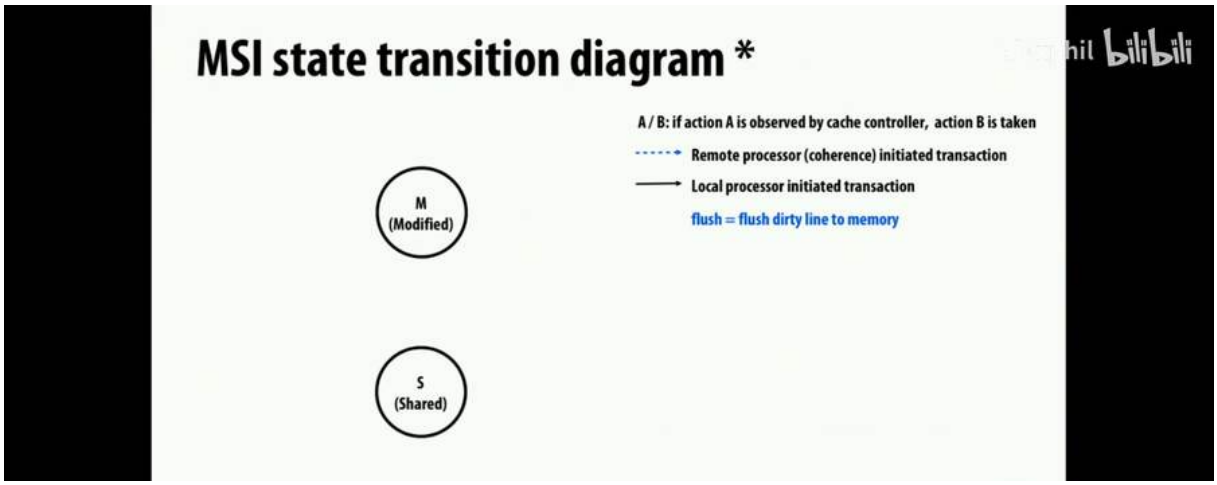

图 253: MSI 状态转移图：标注触发动作与随之执行的动作[253]

状态转移的关键行为

讲者逐条讲了迁移，其中最有价值的是几个” 为什么”。

处于已修改状态时，处理器可以在本地反复写而无需通知任何人。这正是回写的优势：只要没有其他处理器想要这个值，它就表现得和普通的回写缓存一样。

处于共享状态时，如果听到别的缓存要读这一行，那没关系——只是在更多人之间共享而已。讲者强调这里的” 共享” 不是” 真的在共享”，而是” 可能在共享”：其他处理器后来丢弃副本也不影响。

但若听到某个缓存要做独占读（即有人要写），就必须放弃自己的副本，直接标记为无效，不必再通告什么。

M 状态承担了” 数据唯一持有者” 的责任

处于 M 状态意味着内存是过时的，最新值只存在于这一个缓存。因此任何他人要读或要写，都必须由它负责交出数据。这既是 M 状态的收益来源（本地可自由写），也是它的负担来源（必须随时准备响应他人）。协议设计中” 脏” 与” 独占” 绑定在同一个状态上，正是这个权衡的体现。

若是自己处于已修改状态而别人要读，则必须把数据刷出——从自己的缓存推出去，让对方在数据经过时装载；同时因为别人要写，自己这一份也要作废。

一个执行示例

讲者用一个两个地址、两个处理器的读写序列把协议走了一遍，展示状态如何随时间演进。

[253] 视频画面时间区间：00:43:30–00:43:45。若为裁剪图，时间区间仍对应原视频画面。

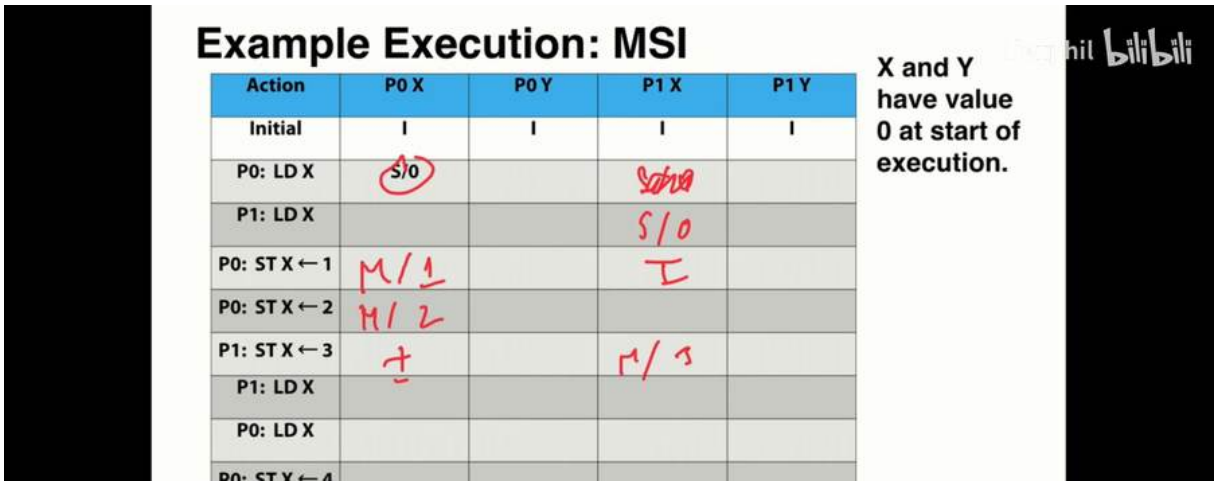

图 255: MSI 执行示例续：状态随操作推进[255]

图 254: MSI 执行示例：X 与 Y 的读写序列与状态变化[254]

他的结论是：每个缓存为它持有的每一行维护状态；若某行不在缓存里，隐含就是无效。持有的行必须标记为无效、共享或已修改三者之一。

[254] 视频画面时间区间：00:51:00–00:51:15。若为裁剪图，时间区间仍对应原视频画面。

[255] 视频画面时间区间：00:58:00–00:58:15。若为裁剪图，时间区间仍对应原视频画面。

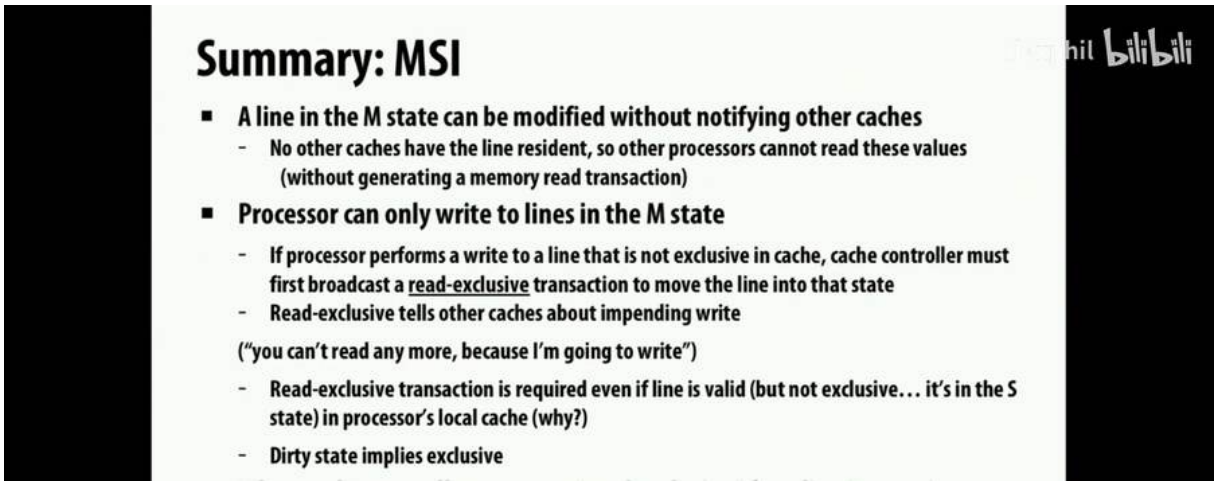

图 256: MSI 小结：M 状态的行可本地修改而无需通知其他缓存[256]

本章小结

MSI 把状态从两个扩展为三个：无效、共享、已修改。核心收益是已修改状态允许本地反复写而不产生流量，这是回写缓存的优势得以保留的地方。

但代价也清楚了：一旦别人要写，就必须来回交接。

MSI 是否满足一致性

讲者在这一节点上回过头检验定义。

写传播是通过"失效加刷出"的组合实现的：一方要写就使他人作废，持脏者则把数据交出。写串行化则依靠互连：他假设互连上所有消息按全局顺序发送、也按同一顺序接收，通常由某种总线实现。正是这条互连提供了所需的全序。

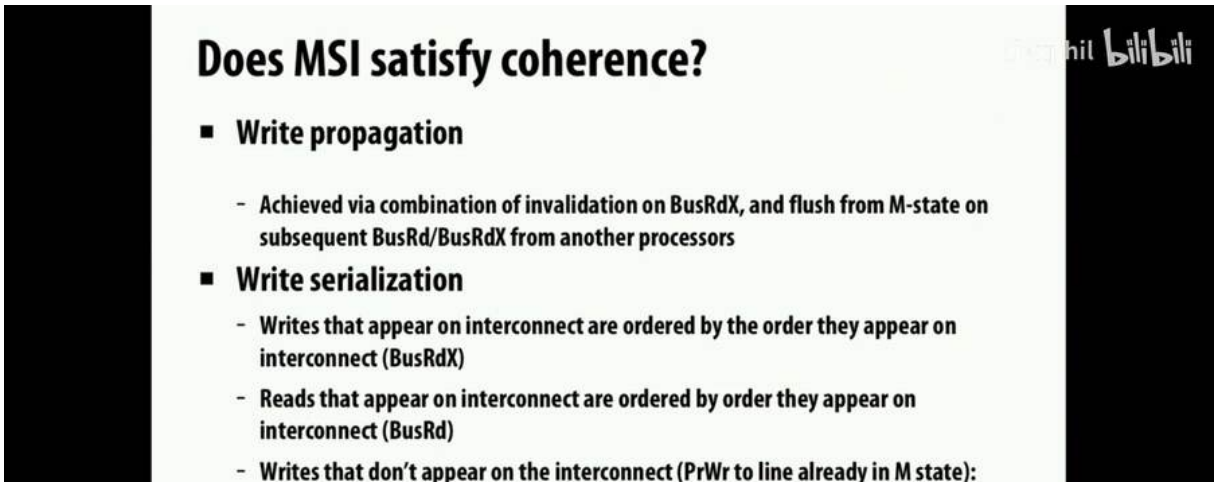

图 257: MSI 是否满足一致性：写传播与写串行化的讨论[257]

[256] 视频画面时间区间：00:58:30–00:58:45。若为裁剪图，时间区间仍对应原视频画面。

[257] 视频画面时间区间：01:00:00–01:00:15。若为裁剪图，时间区间仍对应原视频画面。

一致性协议的正确性依赖互连的顺序保证

协议的串行化能力来自互连，而不是来自缓存自身。若互连不保证全局顺序（例如改用某些非一致性的网络拓扑），同一份协议就不再满足写串行化，需要额外机制补上。这解释了为什么 snooping 通常与总线型互连绑定。

11.7 MSI 的代价与 MESI 的改进

来回争抢

讲者指出 MSI 一个明显的问题：单核执行时它表现得和普通缓存一样，这很好；但它会导致大量来回争抢。

MSI 的问题在于它无法区分”独享”与”共享”

共享状态只表示”可能存在多份”，不表示”确实有多份”。因此即使实际只有自己持有，也必须按最坏情况广播。MESI 的独占状态正是补上这条缺失信息：明确记录”只有我有，且与内存一致”，从而省掉无谓的广播。

每次想写，都得先升级自己的副本，同时让其他人的副本失效。而且第一次写时，即便别人都没有副本，你也要在总线上说一声我"马上要写这个。"

于是可以想象加入更多状态，以减少总线流量、避免这类通信开销。

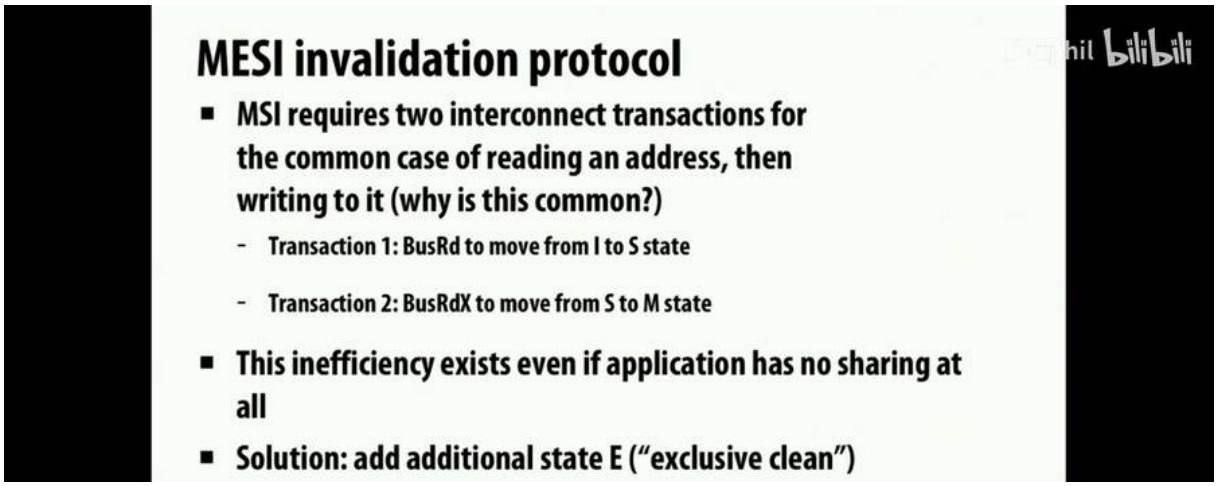

图 258：MESI 失效协议：MSI 在最常见情形下需要两次互连事务[258]

讲者给出的动机很具体：MSI 在最常见的情形下需要两次互连事务。加入独占（Exclusive）状态后，若一行只被一个处理器持有且是干净的，它就可以知道”没有别人有副本”，从而直接写而不必广播。

事务数可以写成对比：

$$N_{\mathrm{MSI}} = 2, \qquad N_{\mathrm{MESI}} = 1$$

- N_{MSI}：MSI 下独占升级所需的互连事务数，一次独占读加一次后续交互；

[258] 视频画面时间区间：01:02:00–01:02:15。若为裁剪图，时间区间仍对应原视频画面。

- N_{MESI}：有了独占状态后，本已独占比时的情况只需零额外事务；
- 前提是该行无人共享，私有写占主要比例。

伪共享：程序员的视角

讲者最后转向这个协议的实践意义：你可能遇到大量的来回争抢，而原因往往与逻辑上共享的数据无关。

伪共享的机制是这样的：缓存行是状态记录与失效的单位，而不是变量。两个变量落在同一行里，即使分别只被不同处理器使用，一个处理器的写也会让另一个的副本失效。

讲者描述的场景是两个处理器各自反复写自己那个变量，结果这一行在两个缓存之间来回弹跳，每次都产生互连流量。而这些变量在程序逻辑上毫无关系。

弹跳产生的互连事务数近似为：

$$N_{\text{bounce}} \approx \min(W_1, W_2)$$

- W_1, W_2：两个处理器各自对同一缓存行的写次数；
- 每次交接需要一次失效与一次数据传递，因此事务数与较少的那个写次数同阶；
- 若两者写次数相近，互连流量几乎与总写次数成正比。

伪共享的代价与数据布局直接相关

把每个线程高频写的计数器、标志位放到独立的缓存行上（通常用填充或对齐声明），就能消除这类弹跳。判断依据是”写的目标是否落在同一行”，而不是”逻辑上是否共享”。这是并行代码里最常见、也最容易忽略的一类性能损失。

不同系统的取舍

讲者最后提到一个对照：GPU 放弃了通用的一致性保证。它的论证是，让软件自己保证不做任何”花哨”的内存操作，就可以提供成本更低、性能更高的实现，因为它不需要那套同步论证。

他说明这正是 GPU（至少 NVIDIA 的 GPU）相对于常规处理器所做的设计取舍。至此本讲结束。

一致性是一份可选的保证

并非所有并行硬件都提供通用一致性。提供它意味着硬件必须在每次访问时维护跨核的顺序与可见性，这份成本由所有代码分担。放弃它则要求软件显式管理，换取更简单的硬件与更高的峰值效率。选择哪种，取决于目标应用是否需要任意的共享内存通信。

11.8 总结与延伸

这一讲从”缓存制造了单一内存的错觉”出发，走完了一条从问题到协议的完整路径。

缓存行、偏移、回写与写直达、脏位，这些单机机制在多处理器下互相冲突：两个处理器各持副本，一个写入后另一个仍是旧值。问题必须在硬件里解决，因为时间尺度在纳秒级；而在真实机器上，一致性逻辑插在私有缓存与共享缓存之间。要让”解决”有明确含义，需要三条不变量：单处理器的读写顺序、写的最终可见性、同一地址写的全局顺序一致。实现方案中软件太慢，于是把共

享缓存当作监听中心：各控制器在互连上广播事务、互相作废副本。写直达加失效的两状态协议是最小可行版本，但每次写都上总线。MSI 把有效拆成共享与已修改，使持有独占副本的处理器可以本地反复写而无需通告；代价是一旦有他人要写就要来回交接，最常见情形需要两次互连事务。协议的正确性依赖互连提供的全局顺序。最后，伪共享揭示了这套机制的实践代价：状态以行为单位，因此逻辑上无关的变量也会互相拖慢。

本讲义在讲者材料之上做的压缩，有四点需要与讲者原话区分：

- 状态数增加是为了省事务，不是为了正确性。两状态、三状态、四状态协议都满足同样的三条不变量，区别只在互连流量。因此评价一个协议，要看它在常见访问模式下产生多少次事务。
- 一致性保证的是” 单一地址的顺序”，不是” 多个地址之间的顺序”。三条不变量只约束同一地址上的操作，跨地址的先后关系属于内存一致性模型的范畴，那是另一讲的主题。
- 伪共享是协议设计的直接后果。只要状态以缓存行为单位，就必然存在这种可能。因此它是程序员需要通过数据布局来规避的问题，而不是协议本身的缺陷。
- 一致性是一份可选的能力。GPU 放弃通用一致性换取效率，说明” 所有并行系统必须一致” 是误解。是否需要，取决于应用是否依赖任意的共享内存通信。

这堂课的边界是它只讲了 snooping，且下一讲才会讨论目录式协议等更具扩展性的方案。讲者反复用” 暂不考虑缓存行多字” 来简化讨论，因此实际协议中” 为改一个字先读整行” 的问题并未展开。

两个可以直接从本讲数据提出的下一步问题：其一，既然 snooping 的流量随核心数增长，那么在多少核之后目录式方案会胜出，判据是什么？其二，MESI 的独占状态只在” 没有其他人有副本” 时才有益，那么当两个线程确实频繁共享同一变量时，独占状态是否完全无益，此时应当改用何种同步方式？

Lecture 12 Assignment 3: GraphRats

视频时长：1 小时 22 分 37 秒 来源：https://www.bilibili.com/video/BV1iX4y1x7KN/

12.1 GraphRats：一个必须保持确定性的图模拟

这一讲是作业三的讲解。讲者先把模拟本身的语义讲清楚，因为不理解语义就无法判断并行化是否改变了程序行为。

基本设定

设想有一大群鼠，被困在一个结构化的迷宫里，迷宫用图表示。最简单的形式是一个 $k \times k$ 的方格网格，只连最近的邻居。

把全部鼠放在右下角，然后做一系列迭代。每一次迭代中，每只鼠选择下一步去哪：可以留在原地，也可以移动到相邻位置——相邻关系由图中的边定义。

因为都挤在角落里，实际可选项只有三个：原地不动、向上、向左。

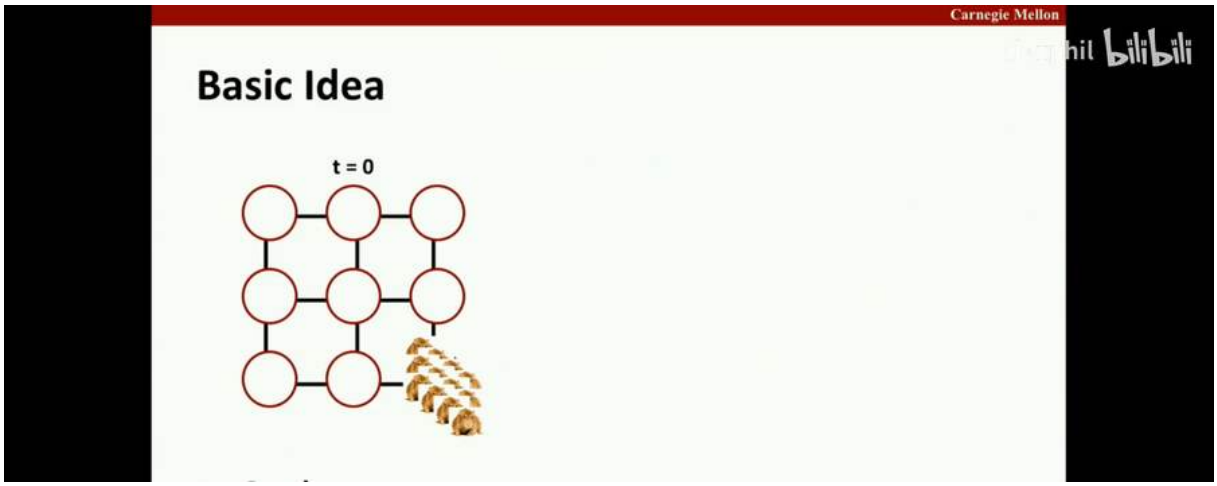

图 259: 基本想法：鼠群在图上逐轮选择移动[259]

[259] 视频画面时间区间：00:05:30–00:05:45。若为裁剪图，时间区间仍对应原视频画面。

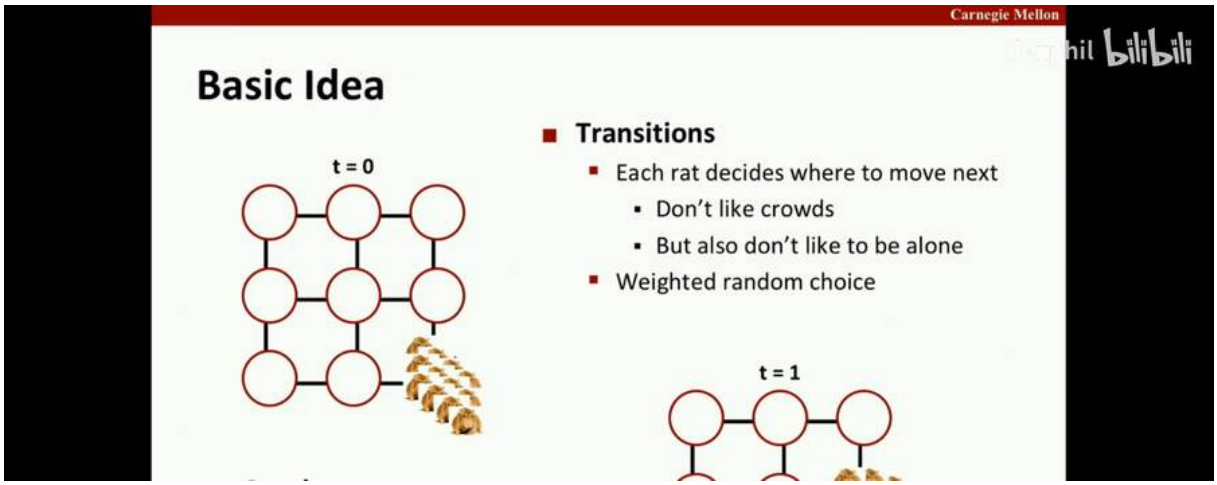

图 260: 状态转移：每只鼠决定下一步去哪[260]

奖励函数表达的行为偏好

选择并非完全随机。讲者设定了一个奖励函数来描述鼠的偏好，他的描述很形象：我不喜欢待在人群里，但我也是只鼠，我也不喜欢独自一个。

于是下一步选择虽然带有随机性，但各选项被选中的权重来自这个奖励函数。结果是有些鼠上移、有些左移、有些不动的混合。

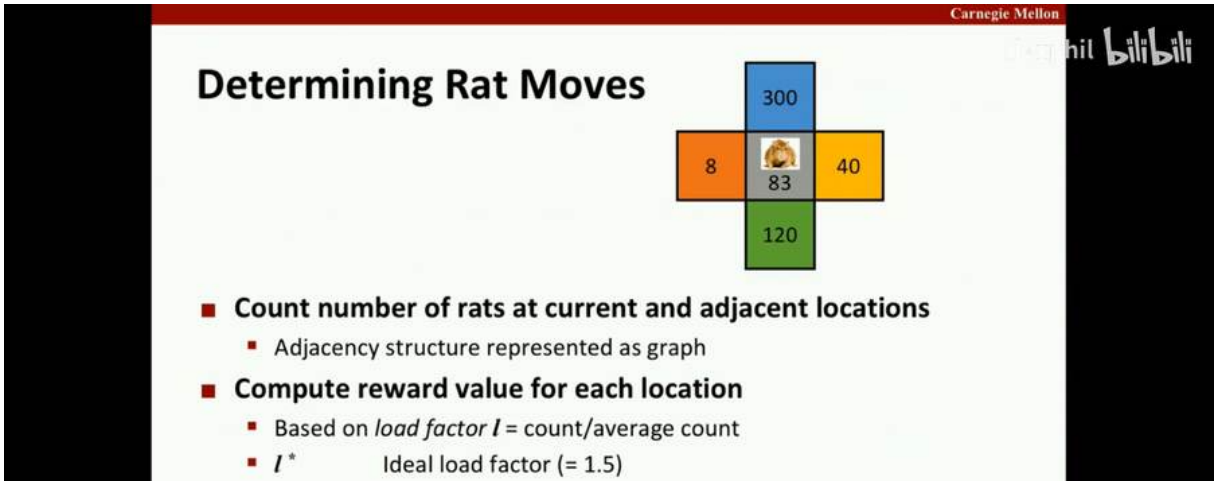

图 261: 如何决定每只鼠的下一步：统计周围各格的鼠数[261]

可视化只需要计数

真正的作业规模不是小图上的少数鼠，而是 100 万只鼠分布在 25000 个节点的图上（视频 00:06:56）。

讲者指出一个简化：因为所有鼠的偏好完全相同，模拟本身需要知道每只鼠的位置，但可视化只需统计每个格子里有多少只鼠。

[260] 视频画面时间区间：00:07:00–00:07:15。若为裁剪图，时间区间仍对应原视频画面。

[261] 视频画面时间区间：00:13:00–00:13:15。若为裁剪图，时间区间仍对应原视频画面。

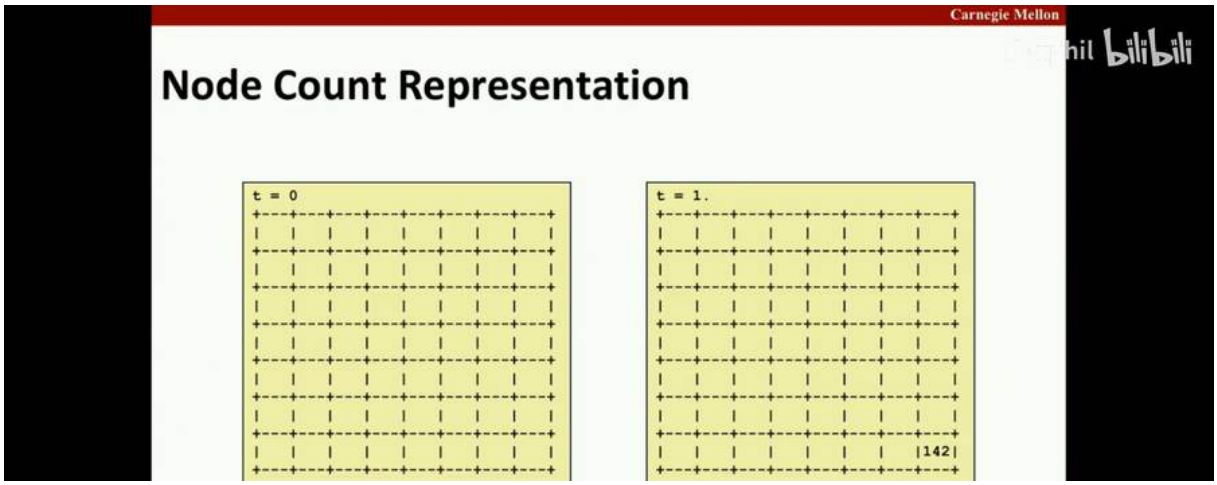

图 262: 节点计数表示：用每格鼠数代替逐只跟踪[262]

计数表示把状态从 O(N) 压到 O(V)

逐只跟踪需要保存百万只鼠的位置，而计数表示只需保存每个节点的鼠数。这让可视化与回归对比的成本从与鼠数成正比降为与节点数成正比。但注意模拟本身仍需逐只决策，因为每只鼠的随机数必须独立。两者不可混淆。

他用字符画的形式把这件事呈现出来：从 320 只鼠开始，有的上移、有的左移、有的不动。

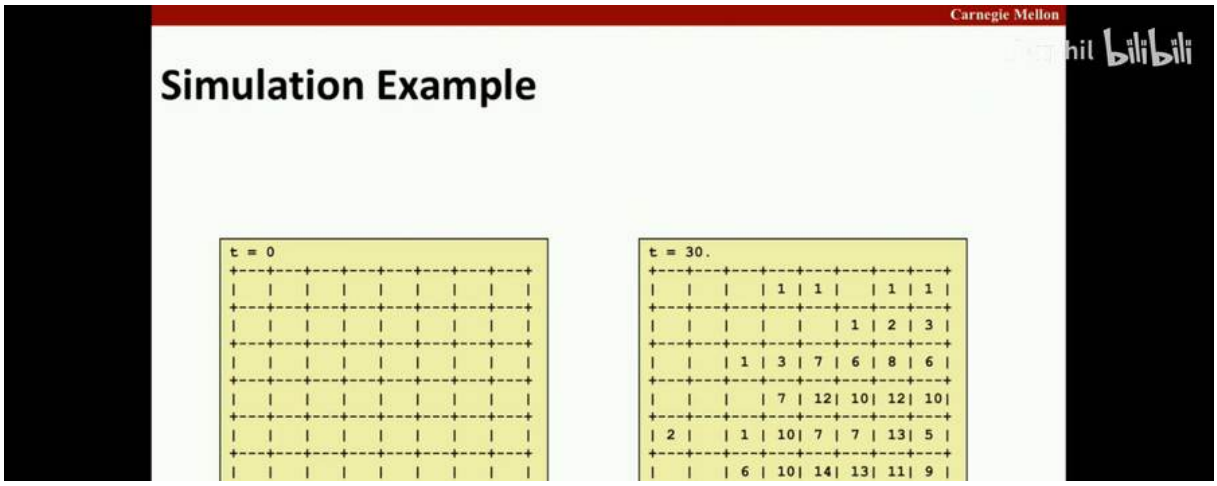

图 263: 模拟示例：字符画展示鼠群扩散[263]

在热图版本里，颜色深浅表示密度：最暗的紫罗兰色代表那一格只有一只鼠，越接近红橙代表越多。

[262] 视频画面时间区间：00:07:30–00:07:45。若为裁剪图，时间区间仍对应原视频画面。

[263] 视频画面时间区间：00:08:00–00:08:15。若为裁剪图，时间区间仍对应原视频画面。

图 264: 在终端里运行模拟演示[264]

本章小结

GraphRats 的语义很简单：每轮每只鼠依据局部密度选择停留或移动，权重由奖励函数给出。作业规模是百万鼠、两万五千节点。可视化只需节点计数，但模拟本身必须逐只决定。

这个”逐只决定”正是计算量的来源，也是并行化的切入点和难点。

12.2 奖励函数与移动选择

这是全讲计算量最集中的部分。讲者用一个看似复杂的公式说明设计意图，值得逐步拆开。

负载因子

所有计算的基准是一个叫负载因子的量：把某个格子里的鼠数，除以全图的平均值。

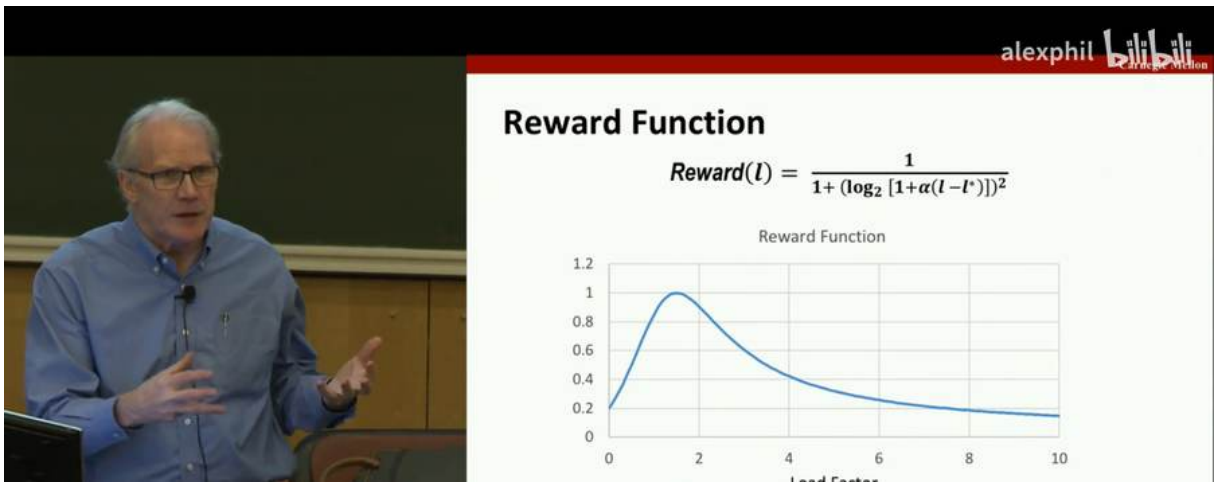

图 265: 奖励函数：以负载因子为自变量，理想值为 1[265]

他举例说，假设全图平均每个节点 10 只鼠。那么负载因子就是该节点鼠数除以这个平均值。

[264] 视频画面时间区间：00:10:30–00:10:45。若为裁剪图，时间区间仍对应原视频画面。

[265] 视频画面时间区间：00:15:30–00:15:45。若为裁剪图，时间区间仍对应原视频画面。

$$l = \frac{n_{\text{node}}}{\bar{n}}, \qquad \bar{n} = \frac{N_{\text{rats}}}{N_{\text{nodes}}}$$

- n_{node}：某一节点上的鼠数；
- N_{rats}：鼠的总数，作业中为 1000000；
- N_{nodes}：图的节点数，作业中为 25000；
- $\bar{n}$：全图平均每节点的鼠数，本例假设为 10；
- l：负载因子，全图平均值对应 $l = 1$。

理想负载与曲线形状

讲者设定理想负载因子为 1.5——也就是鼠倾向于聚集在比全图平均略大一些的群体里。这个设定本身会带来有趣的性质。

把它画成负载因子的函数，可以看到曲线在 $l = 1.5$ 附近达到奖励值 1，向左侧衰减，到空节点时降到 0.2。

右侧衰减非常缓慢。讲者解释原因：像百万鼠全挤在一处这种情形，你并不希望奖励变成零，否则那只鼠完全没有偏好可依。因此在对数坐标下，即便负载高到 1000，奖励仍保留最大值的约 1%。

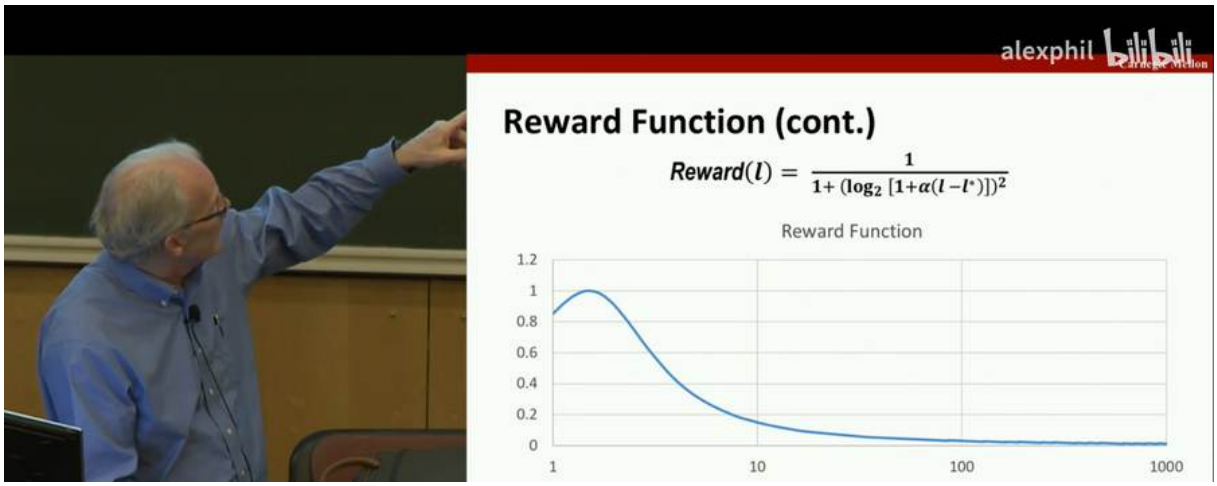

图 266: 奖励函数曲线：理想值处为 1，空节点降为 0.2[266]

公式为什么长这样

讲者坦承这个函数是” 从空中飘来的”，但它是被构造成具有上述特征的。他逐项解释。

函数形式是：

$$\text{Reward}(l) = \frac{1}{1 + \left(\log_2\left[1 + \alpha(l - l^*)\right]\right)^2}$$

- l：负载因子，节点鼠数除以全图平均值；
- l^*：理想负载因子，取 1.5；

[266] 视频画面时间区间：00:16:00–00:16:15。若为裁剪图，时间区间仍对应原视频画面。

- α：缩放参数，取 0.25 或 0.5，用来让曲线形状合适；
- 分子固定为 1，因此理想情形下奖励恰为 1。

最内层是 $(l - l^*)$：低于理想值为负，高于理想值为正。

它被某个系数加权后加 1，再取以 2 为底的对数。加 1 是必须的——不能对负数或零取对数，括号内必须为正。

对数内的括号必须恒为正

$1 + \alpha(l - l^*)$ 在 l 取最小值 0 时等于 $1 - \alpha l^*$。若 α 与 l^* 取值使这一项小于等于零，空节点处就会出现 $\log_2$ 非法。因此参数不是随意可调的：$\alpha l^* < 1$ 是这条公式能成立的前提。

若 $l = 0$，则 $1 - \alpha l^*$ 约为一半（α 取 0.5 时），对数结果才有定义。

对数的作用正是提供那种” 即使负载极高也保持非零” 的缓慢衰减。

外面再加平方，是因为大于 1 的数对数结果为正、介于 0 与 1 之间的数对数结果为负，平方后统一变为正数。

最后这个正数进到分母的 $1 + (\cdot)$ 里，于是理想情形（对数为 0）给出奖励 1，而稍微偏高或偏低都会使奖励下降。

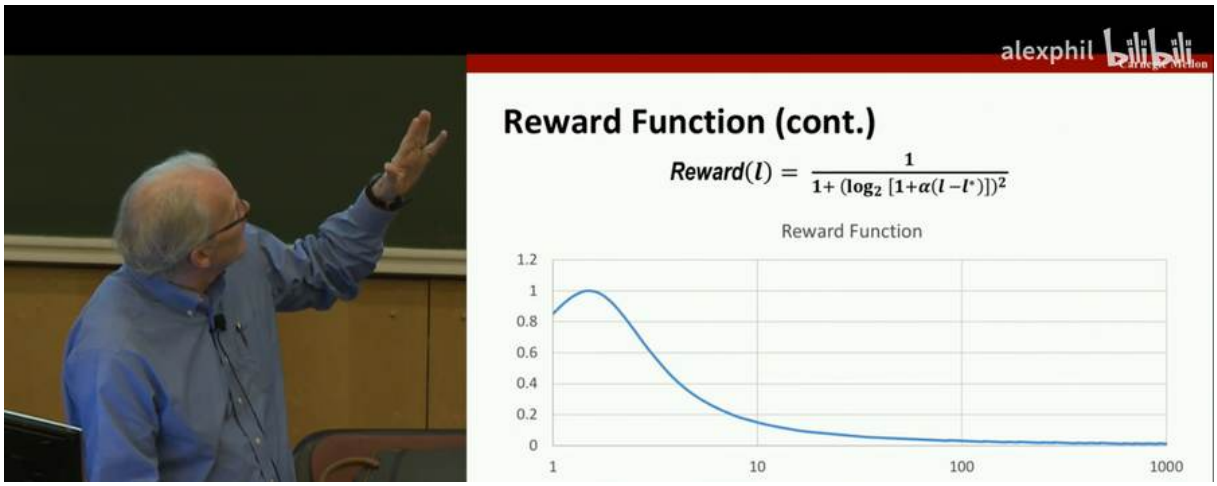

图 267: 奖励函数继续解析：对数缩放保证高负载下仍非零[267]

[267] 视频画面时间区间：00:17:00–00:17:15。若为裁剪图，时间区间仍对应原视频画面。

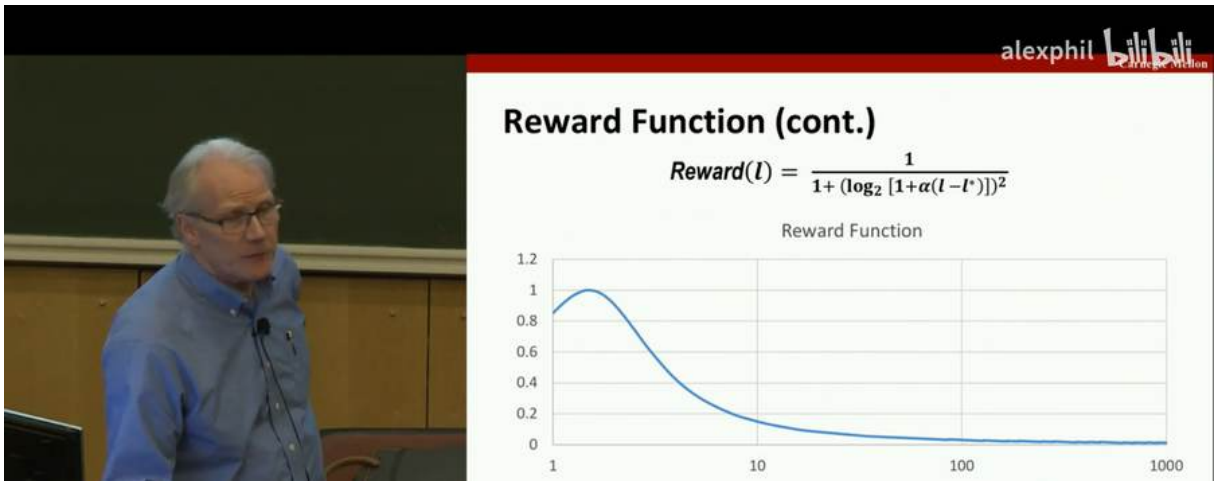

图 268: 奖励函数参数 α 的作用[268]

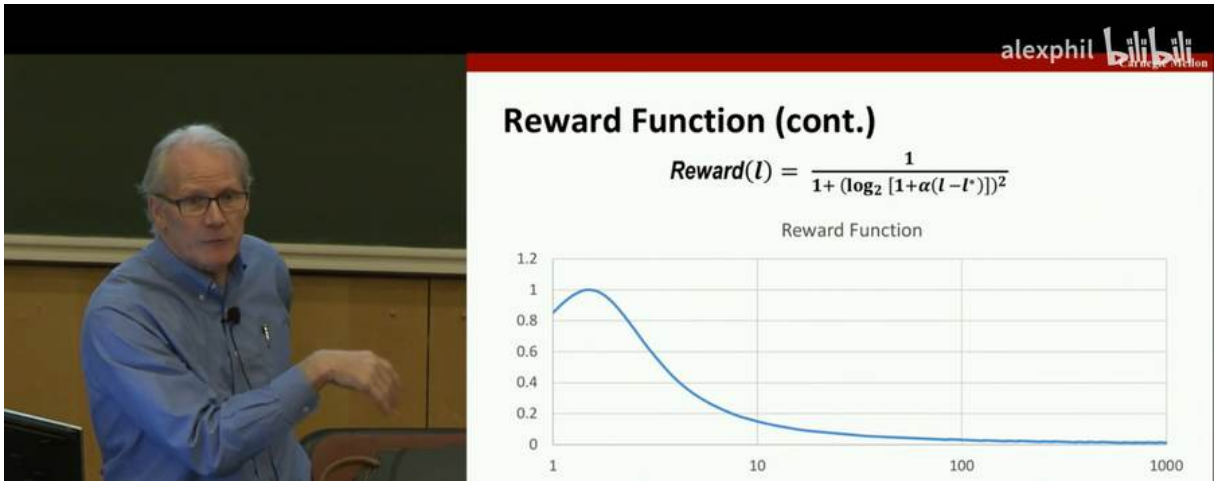

图 269: 对数坐标下的奖励衰减：负载到 1000 时仍保留约 1%[269]

奖励函数的设计目标决定了它的形状

这个函数要同时满足三件事：理想负载处精确取 1；偏离则单调下降；极高负载下仍严格为正。前两条要求分母随偏离量增大而增大，第三条要求增长速度足够慢。对数的慢增长与平方的正则化组合起来，恰好满足三者。这不是任意拟合，而是性质驱动下的构造。

把奖励变成选择区间

有了五个候选目的地（原地与四邻）的奖励值，接下来要据此做随机选择。

讲者的做法是把奖励当作区间长度，依次累加。假设平均值为 10，那么 8 只鼠的格子负载为 0.8，略低于平均，得到相当高的奖励 0.7；而顶上 300 只鼠的格子太拥挤，奖励很低，只有 0.061。

[268] 视频画面时间区间：00:17:30–00:17:45。若为裁剪图，时间区间仍对应原视频画面。
[269] 视频画面时间区间：00:18:00–00:18:15。若为裁剪图，时间区间仍对应原视频画面。

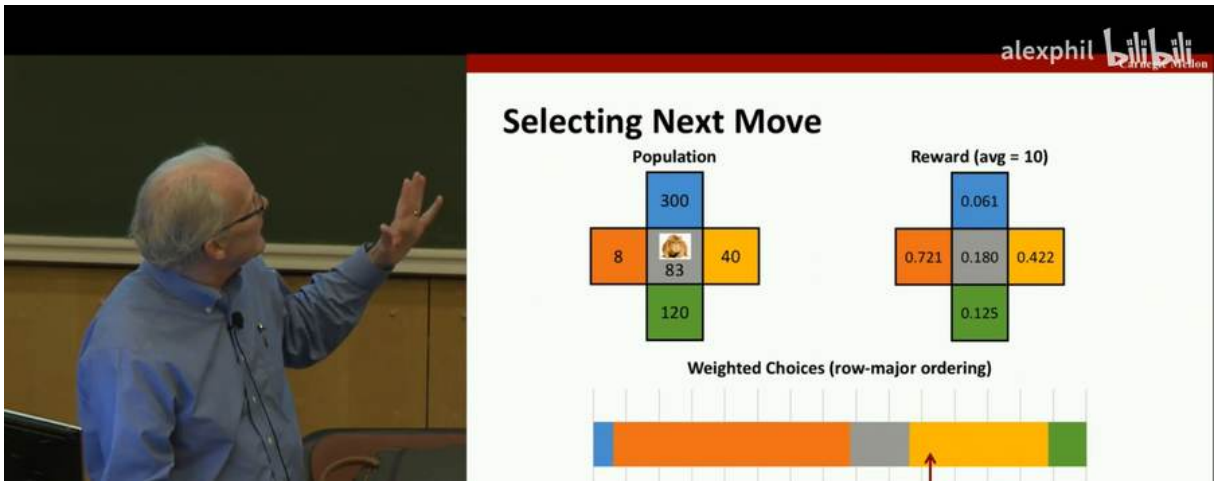

图 270: 选择下一步：把各候选的奖励累加为区间[270]

于是得到一个从零到总长度的连续区间。取一个均匀随机数，落在哪个区间就选哪一项。累加过程可以写成：

$$c_k = \sum_{i=1}^{k} \text{Reward}_i, \qquad k = 1, \ldots, 5$$

- Reward_i：第 i 个候选目的地的奖励值；
- c_k：前 k 个候选的累积长度；
- c_5：区间总长，本例略超过 1.5。

取 $u \sim U(0, c_5)$，选择满足 $c_{k-1} \le u < c_k$ 的那个候选。
讲者举例：这里总和略超 1.5，如果随机数生成到 1.04，就意味着向右移动。

图 271: 在总长度上取随机数，落入哪段就选哪个方向[271]

[270] 视频画面时间区间：00:18:30–00:18:45。若为裁剪图，时间区间仍对应原视频画面。
[271] 视频画面时间区间：00:19:30–00:19:45。若为裁剪图，时间区间仍对应原视频画面。

图 272: 选择过程的示例计算：随机数 1.04 对应向右[272]

加权随机的两种等价实现

按概率权重选择，可以显式算累积分布再二分查找，也可以像这里一样在总和上取均匀随机数再线性扫描区间。后者在候选只有 5 个时更简单，也不需要做除法归一化。两者结果分布相同，选择取决于候选数与是否需要重复使用同一分布。

本章小结

奖励函数把负载因子映射为一个正权重：理想值 1.5 处为 1，空节点降到 0.2，高负载按对数缓慢衰减但仍非零。五个候选的奖励累加成区间，随机数落点决定选择。讲者强调这正是程序的核心计算，且必须对全部候选都算一遍——这就是算术密集所在。

公式既然只是权重的来源，那么程序的行为在这套机制下是否可重复，就成了并行化的关键约束。

12.3 三种更新模式

确定性：一个硬约束

讲者在这里指出整个程序的一个关键性质：它完全确定。

所有随机数生成都由一组完全可预测的种子值控制，随机数生成算法已经内建在代码里。因此反复运行会得到完全相同的移动序列、完全相同的节点计数。

[272] 视频画面时间区间：00:20:30–00:20:45。若为裁剪图，时间区间仍对应原视频画面。

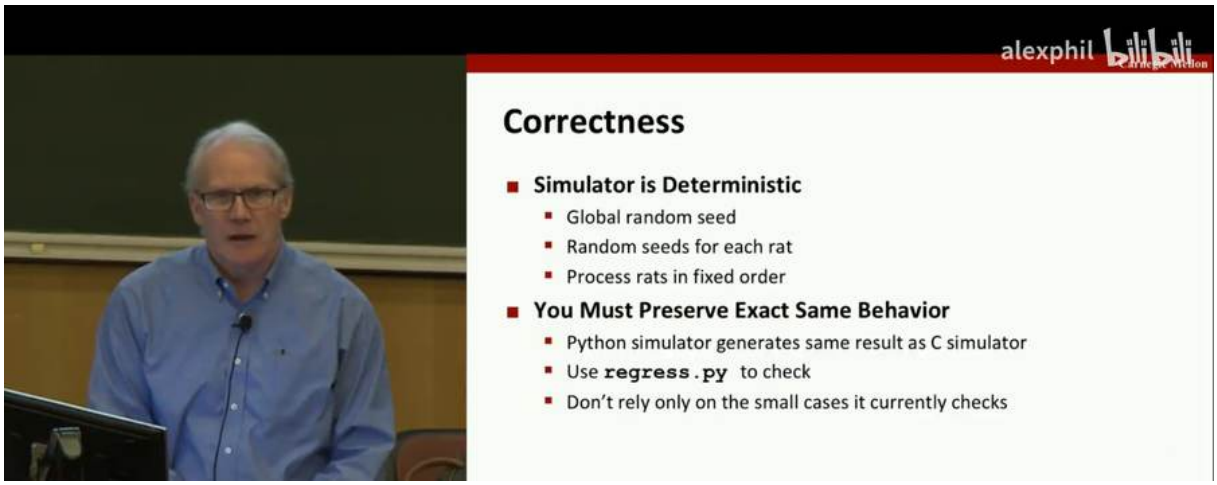

图 273: 正确性要求：模拟器是确定性的，随机种子固定[273]

而正确性检查的要求是必须精确保留这个性质。他评价这从软件设计角度是件好事：能既可扩展又完全确定，是非常有用的特性。

确定性要求会排除一批并行化方案

若并行化引入了非确定性的归约顺序、动态调度或原子竞争，即使统计行为相同，逐位结果也会变化，从而无法通过正确性检查。因此在动手之前，必须先确定哪些并行手段能保持与串行版本逐位一致。

同步模式与它的振荡

讲者接着讨论更新顺序，这是本讲最有意思的部分。

他先演示的模式叫同步模式：所有鼠按 0 到 $R-1$ 编号，但先为全部鼠计算各自想做的移动，然后统一执行。

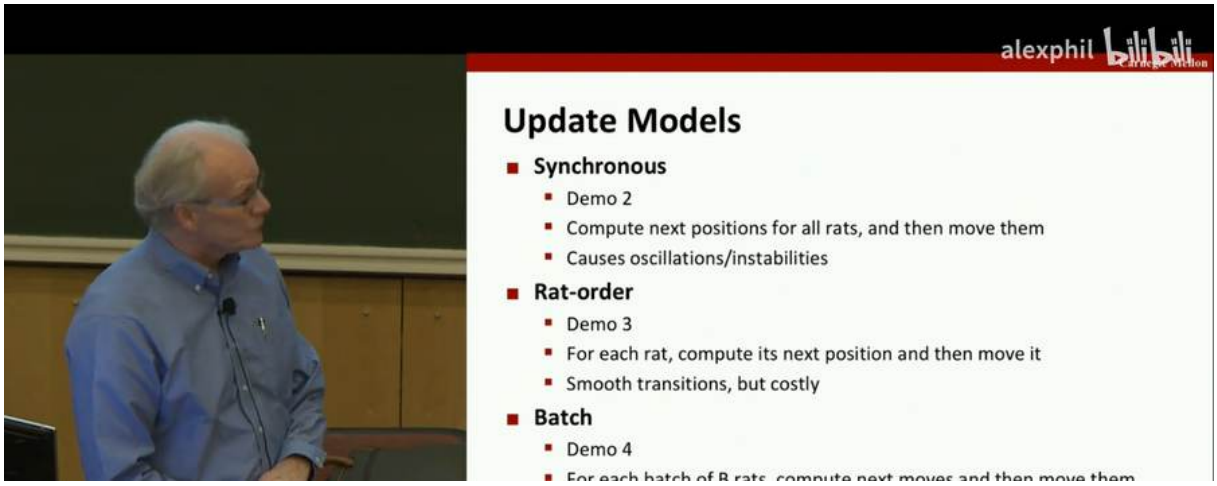

图 274: 更新模式：同步模式先算全部再一起移动[274]

[273] 视频画面时间区间：00:32:00–00:32:15。若为裁剪图，时间区间仍对应原视频画面。

[274] 视频画面时间区间：00:22:30–00:22:45。若为裁剪图，时间区间仍对应原视频画面。

这会产生振荡。他解释得非常具体：第一步时角上有 320 只鼠，它们都判断” 这里太挤了，我要走”，于是除少数留下外全部移动。结果它们到达了新的位置（那里变挤），而原来那个格子变得完全空旷。

于是那些鼠又集体判断那里太挤——如此反复。讲者说这种同步更新会长时间持续振荡，即使图很大也会产生棋盘式的花纹。

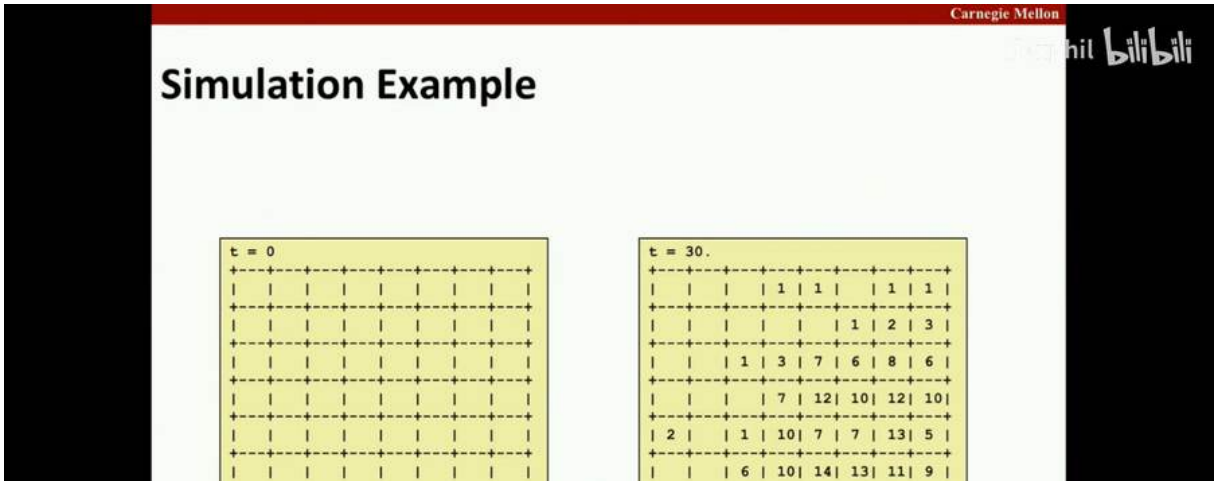

图 275: 同步更新下的模拟：棋盘状振荡花纹[275]

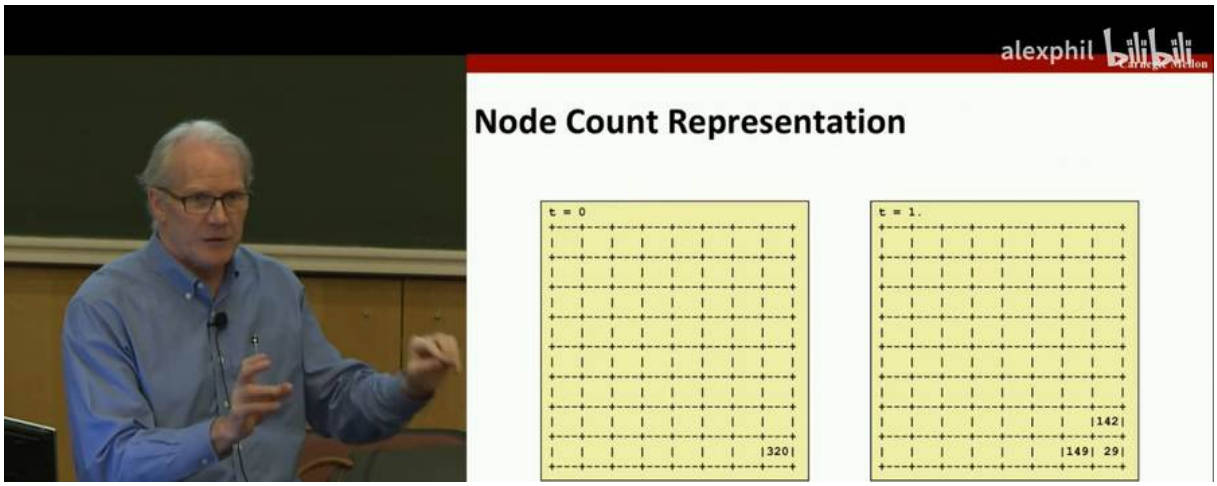

图 276: 节点计数在连续时间步上的变化：交替摆动[276]

原因是整个系统基于同一时刻的状态集体做决定，然后一起进入新状态，结果就是过冲。

按鼠序与按批次

更好的办法是按鼠序：鼠 0 环顾四周做出选择并移动，然后鼠 1 照做，依此类推。每只鼠基于此前所有鼠已经做出的选择来决策。

[275] 视频画面时间区间：00:23:00–00:23:15。若为裁剪图，时间区间仍对应原视频画面。

[276] 视频画面时间区间：00:23:30–00:23:45。若为裁剪图，时间区间仍对应原视频画面。

全局顺序依赖是并行化的硬边界

按鼠序要求每只鼠看到此前所有鼠移动后的状态，这意味着第 k 只鼠必须等前 $k-1$ 只全部完成。这个依赖链长度与鼠数同阶，无法通过增加处理器缩短。凡是”每一步都依赖上一步全部结果”的结构，都属于此类不可并行模式。

讲者说这在行为上完全合理，但几乎无法并行化。

于是设计了折中方案，即按批次模式：取某一比例（例如前 2%）的鼠一起做出选择，然后一起移动；接着下一批。

批次数的计算很直接：

$$N_{\text{batch}} = \frac{N_{\text{rats}}}{N_{\text{rats}} \cdot f} = \frac{1}{f}$$

- N_{rats}：鼠的总数，作业中为 1000000；
- f：每批占全体的比例，取 0.02；
- N_{batch}：总批次数，此例为 50。

批次数与总鼠数无关，只由比例决定。因此批次大小是纯粹的并行粒度旋钮。

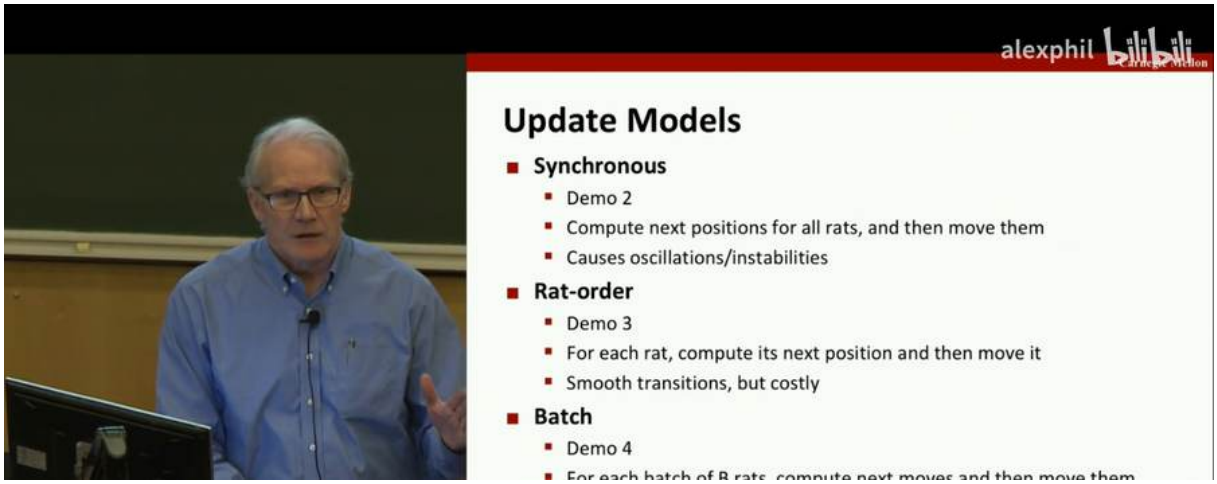

图 277：三种更新模式的对比：同步、按鼠序、按批次[277]

从可视化上看，同步模式呈现明显的振荡；按鼠序模式则是从角上全红逐渐扩散，变成橙色、黄色、绿色；批量模式虽然不完全相同，但看不出与按鼠序模式有实质差别。

因此作业选用了批量模式。

作业任务

讲者的要求很明确：需要同时加速同步模式与批量模式。

他坦率地说同步模式其实没什么用——这个模拟本身也没什么实际用途，纯粹是个模型——但它更容易跑快，算是给的一点便利。而批量模式需要下功夫。

[277] 视频画面时间区间：00:24:30–00:24:45。若为裁剪图，时间区间仍对应原视频画面。

模式	决策依据	行为特点	并行难度
同步	全部鼠基于同一时刻状态	振荡、棋盘花纹	最易
按鼠序	基于此前全部已决策结果	最自然	几乎不可并行
按批次	每批 2% 基于之前各批	近似按鼠序	中等

表 9: 三种更新模式的对比

批量模式是”语义可接受性”与”可并行性”的折中

按鼠序在语义上最自然，但它把全局顺序变成了硬依赖，无法并行。同步模式最容易并行但行为失真。批量模式用”每批内同时、批间顺序”换取了中间地带：批内可以并行，批间保留顺序依赖。批次越小越接近按鼠序，代价是并行粒度越细。

本章小结

程序必须逐位确定，因此并行方案不能引入非确定的顺序。三种更新模式中，同步最容易并行但会产生振荡，按鼠序最自然但无法并行，批量模式以 2

作业要同时加速后两者中的可用版本：同步与批量。

12.4 提供的材料与正确性保障

两个模拟器

仓库里其实有两个模拟器，功能相同、行为一致。一个是 Python 实现，前面所有演示都是它跑的；另一个是 C 实现，比 Python 快，但仍然远远不够快。

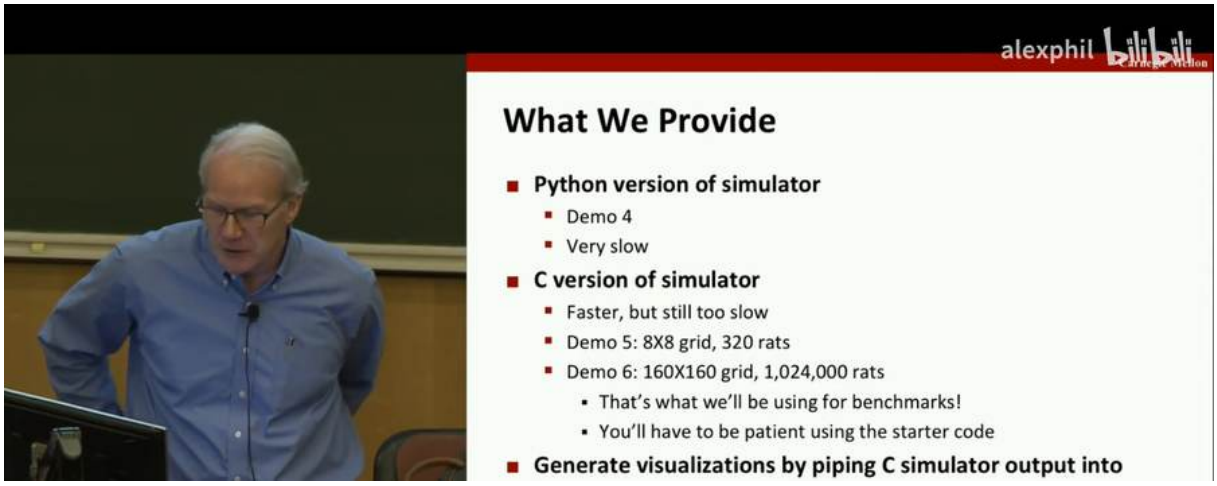

图 278: 提供的材料：Python 与 C 两个模拟器[278]

[278] 视频画面时间区间：00:27:30–00:27:45。若为裁剪图，时间区间仍对应原视频画面。

有意思的是可视化部分的组织方式：C 代码不必做任何复杂图形，它只需把” 每个节点上有多少只鼠” 这个文本表示写到标准输出。然后管道接给 Python，由它读取并生成图像。

这解释了演示命令里的那些参数：`-g` 给图文件名，`-r` 给初始鼠位置文件，`-u b` 表示批量更新模式，再指定步数、用热图可视化、以及每帧 0.3 秒。

讲者也说明这对运行环境没有特殊要求，只要装了必要的 Python 库就能在任何机器上跑。

本章小结

仓库提供 Python 与 C 两个等价实现，C 版通过文本管道把节点计数交给 Python 可视化，这种分离让模拟与绘图互不干扰。C 版虽快于 Python，但距离作业要求仍远。

要判断优化是否破坏了行为，还需要一套可靠的对照手段。

12.5 正确性：回归测试与自建用例

回归测试的作用

仓库自带一个回归测试程序。讲者解释回归测试的通常含义：每次改动后重跑同一套测试，确认没有破坏原本可用的功能。

这个程序做的是对比 Python 版与 C 版的结果。因为 Python 慢，它运行时会创建一份预计算结果的缓存，之后复用，因此回归测试只需几秒。

用参考实现校验优化版本是标准做法

一个慢但正确的实现，加上一个快但可能出错的实现，两者对比就构成了可靠的防线。缓存参考结果让这条防线成本极低，因此可以频繁运行。这也回应了第四讲提到的” 黄金实现”：保留它，是为了随时能验证优化后的结果仍然正确。

不要只依赖给定的测试

讲者给出两条经验。

第一，务必养成频繁运行回归测试的习惯。他描述的情况很常见：花大量时间调优、代码跑得很漂亮，最后发现结果是错的——而它之所以快，正是因为它是坏的。

第二，回归测试只覆盖较小的用例。他本人在这个作业里制造的所有 bug，都能在很小的图、很短的运行中被发现。但他提醒不要假设这一点总成立，建议修改 `regress.py` 加入更大的测试。

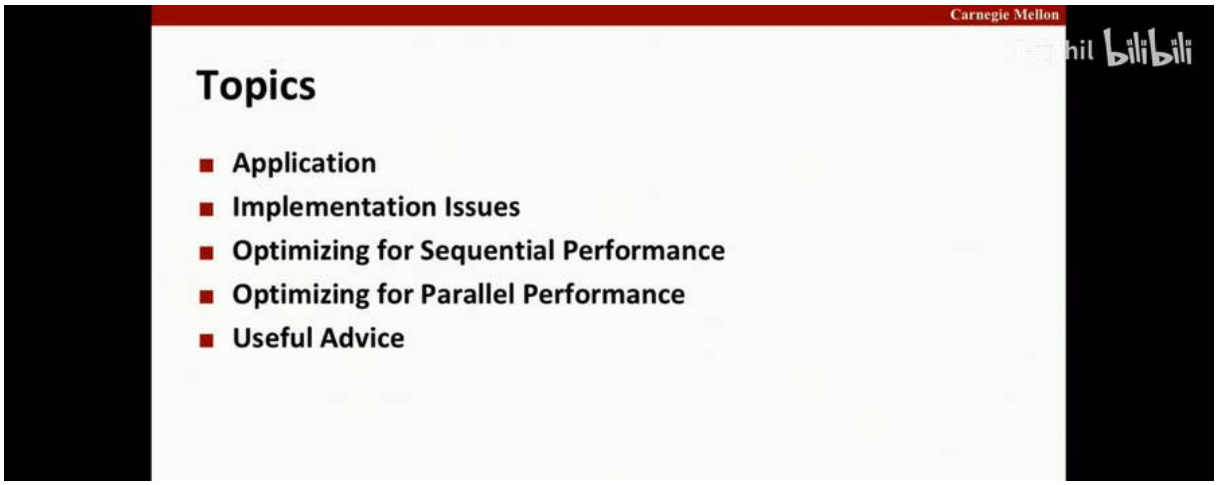

图 279: 本讲主题概览：应用、实现问题与性能优化[279]

”跑得快”可能是”算错了”的症状

并行化引入的错误常常表现为计算量减少（例如某些鼠被漏算、某批被跳过），于是程序变快而结果偏离。因此性能提升必须与正确性验证同步进行，且验证要覆盖到性能测试所用的同等规模。

本章小结

提供了 Python 与 C 两个等价实现，可视化通过文本管道分离，回归测试用缓存让对比变得廉价。但讲者强调正确性责任在程序员自己：给定测试只是起点，规模更接近真实负载的测试需要自己补充。

12.6 图结构的三种形态

前面所有演示用的都是格图。讲者在这里说明更有趣的变体。

格图：规则且稀疏

格图是 $k \times k$ 个节点、只连最近邻居。它的特点是每个节点的度最多为四，加上自身共五个候选目的地。

规则稀疏图为何容易划分

格图的边只连接几何相邻节点，因此按空间划分（行块或方块）时，跨块通信只发生在边界上，且数量正比于边界长度。这与第八讲的结论一致：计算正比面积、通信正比周长，分块越大越划算。不规则图则不具备这种性质。

而且它非常规则，所有通信与移动都高度局部化。从顺序和并行两个角度看这都很好——可以设想各种划分方式。

讲者补充说这是稀疏图：一般图的边数随节点数的平方增长，而这里的边数只随节点数线性增长。

[279] 视频画面时间区间：00:04:00–00:04:15。若为裁剪图，时间区间仍对应原视频画面。

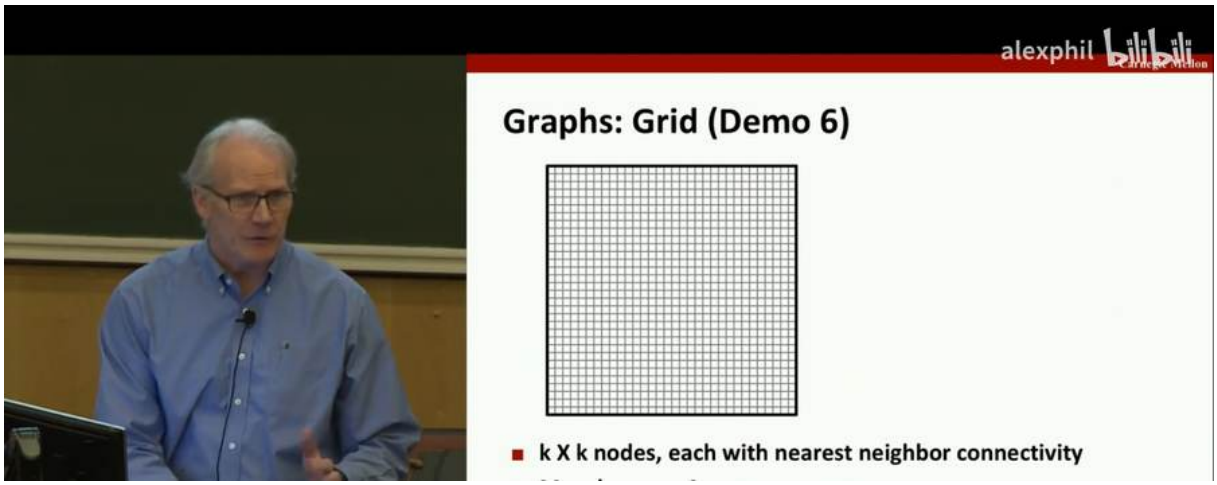

图 280: 格图：$k \times k$ 节点，只连最近邻居，度最多为四[280]

tile 图：叠加 hub 节点

为了不那么规则，他们又造了几种图。其中一种叫 tile 图：图中有若干 $d \times d$ 的小区域，每个区域里有一个红色标记的 hub 节点。

hub 节点的定义是：它与本区域内所有其他节点都有边相连。区域本身没有别的特殊之处，这些连接是叠加在格图之上的，格连接依然存在。

讲者特意澄清：图上那些黑线不是障碍，只是标出每个 hub 的连接范围。

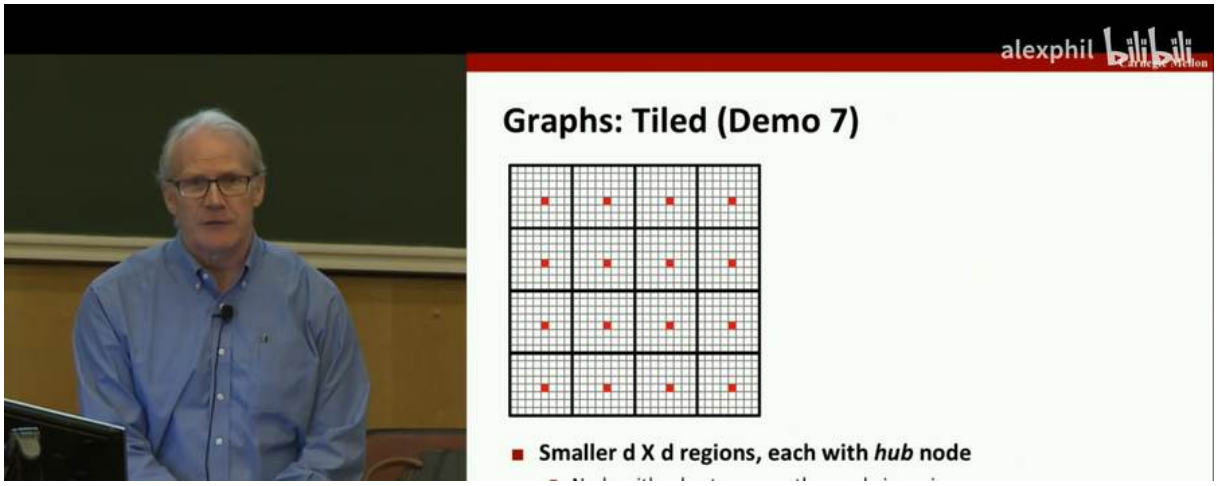

图 281: tile 图：在格图之上叠加与全区域相连的 hub 节点[281]

行为因此明显不同。鼠仍从角落开始，但很快整个下角区域被填满，然后开始向其他区域扩散，呈现逐块向外传播的模式。

更值得注意的是 hub 节点在热图上显示为更亮的颜色——它们非常拥挤，却仍吸引大量鼠。原因很清楚：它们的连接数极多，随机选择中相当一部分会落到它们身上，即使奖励函数对如此高的负载给出很低的值。讲者的说法是它们有 99 倍于普通节点的潜在来源。

若每个 tile 是 $d \times d$，则 hub 的度数与其所在区域大小同阶：

[280] 视频画面时间区间：00:35:30–00:35:45。若为裁剪图，时间区间仍对应原视频画面。

[281] 视频画面时间区间：00:36:30–00:36:45。若为裁剪图，时间区间仍对应原视频画面。

$$\deg(\text{hub}) \approx d^2$$

- d：tile 区域的边长；
- deg(hub)：hub 节点连接的邻居数，覆盖本区域全部节点；
- 普通格节点的度最多为 4，因此 hub 的候选来源高出约 $d^2/4$ 倍。

这解释了为何奖励值极低时 hub 仍被大量选择：候选数量压倒了权重。

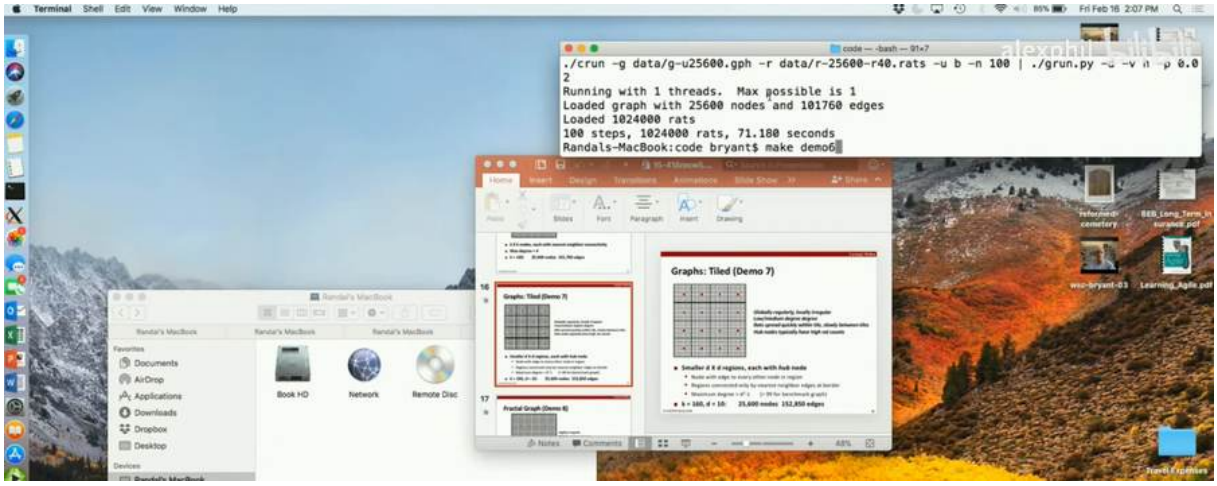

图 282: tile 图上的模拟：按区域逐块扩散，hub 节点更亮[282]

hub 节点体现了”连接数胜过奖励值”

一只鼠的候选集合由图的边决定。hub 与区域内所有节点相连，因此对区域内大量鼠而言它都是候选，累积起来的选择概率远超普通节点。这说明在非规则图上，节点度分布会直接决定负载分布，进而决定并行时的负载均衡难度。

分形图

讲者还展示了分形图上的模拟，从命令输出可以看出是一个 160×160 的迷宫，并标明了负载参数。

[282] 视频画面时间区间：00:37:30–00:37:45。若为裁剪图，时间区间仍对应原视频画面。

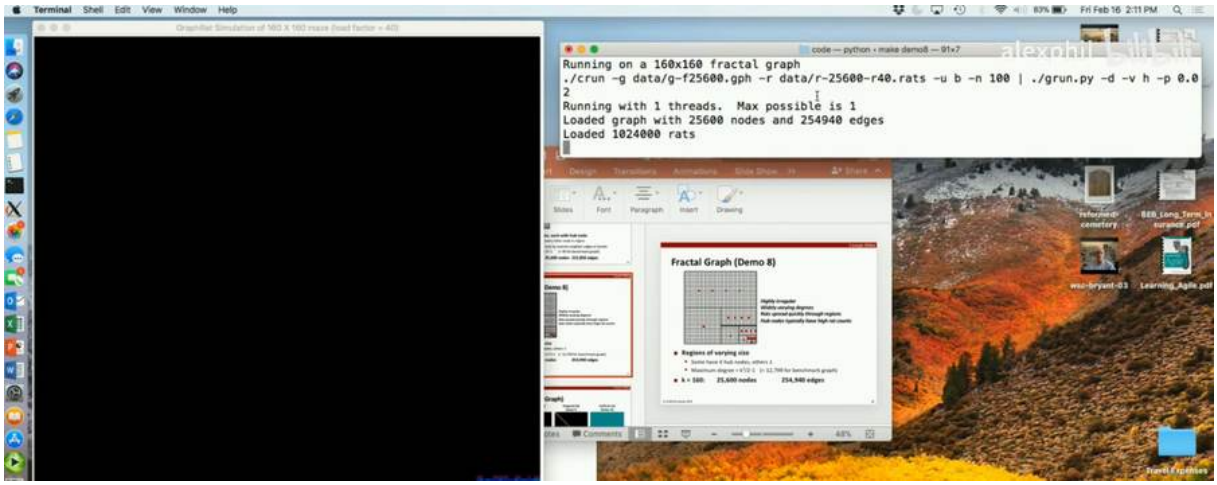

图 283: 分形图上的模拟演示：160 × 160 迷宫[283]

本章小结

图从规则的格图扩展到带 hub 的 tile 图与分形图。格图均衡易划分；tile 图的 hub 因连接数极高形成结构性热点，即使奖励很低仍会吸收超比例的鼠。热点来自图结构，不来自调度。

除图结构之外，初始分布也是基准的一部分，而它影响着批量模式是否成立。

12.7 初始分布与它对语义的影响

三种初始状态

除了鼠全在角落，还有两种初始状态被用作基准。

代号 r 表示右下角，也就是前面一直在看的。代号 d 表示对角，鼠均匀分布在图的对角线上。代号 u 表示均匀，鼠几乎完全均匀地铺满全图。

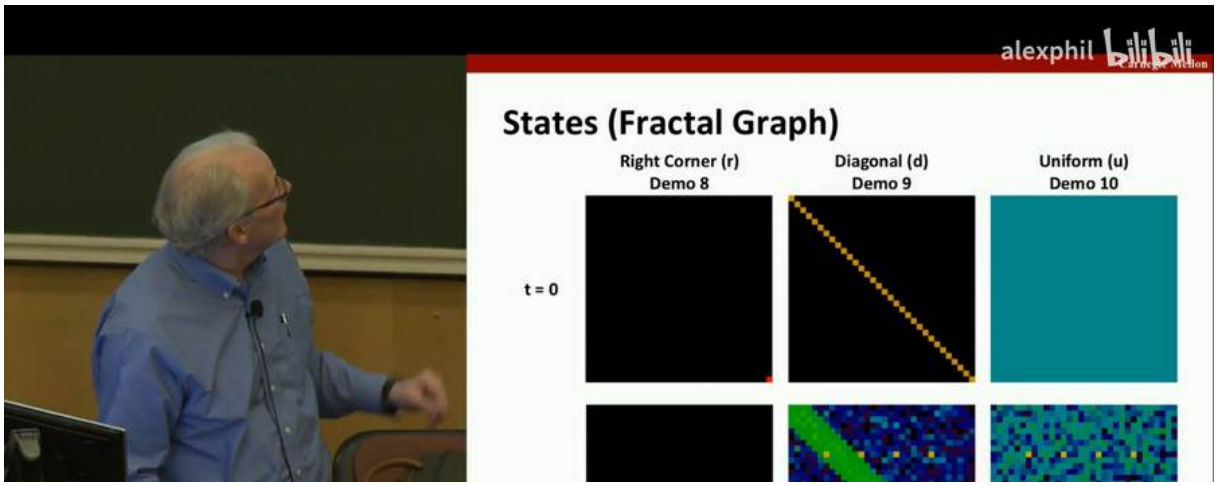

图 284: 三种初始分布：角落、对角、均匀[284]

[283] 视频画面时间区间：00:41:30–00:41:45。若为裁剪图，时间区间仍对应原视频画面。

[284] 视频画面时间区间：00:45:30–00:45:45。若为裁剪图，时间区间仍对应原视频画面。

讲者指出这些都在读入的文件里，但鼠的编号顺序被特意打乱，使其在后两种分布下完全随机。对全在角落的情形这不重要，但另外两种情形下，鼠在各自所在节点之间是随机分布的。

而这正是批量模式合理的原因：每一批里哪些鼠会移动是随机的，因此不会出现某一批集中在同一区域的情况。

他强调必须保持这个编号顺序。

打乱编号顺序不是随意设定而是语义的一部分

批量模式的行为依赖于”每批随机抽取鼠”，而随机性来自被打乱的编号顺序。若在并行化时按位置重新排列鼠，批次就会变成空间上聚集的一群，模拟行为随之改变，结果不再匹配参考实现。

从对角分布出发

讲者演示了对角分布：鼠很快从对角线向外扩散。因为都挤在一处、都想离开，而其中不少会选择通过 hub 移动，之后去向就完全随机，于是东西扩散得非常快。

他点出一个对并行计算有利的侧面：这意味着负载在位置上相对均衡。

本章小结

图从规则的格图，扩展到带 hub 的 tile 图与分形图；初始分布有角落、对角、均匀三种。图结构决定负载分布：规则图均衡、hub 会形成热点；而编号顺序被特意打乱，这是批量模式语义成立的前提，并行化时必须保留。

12.8 总结与延伸

讲者在结尾讲了一段来历：这个作业源自图处理系统的一系列工作，其中有个叫 GraphChi 的系统（”图吉娃娃”），他因此想到从狗转到鼠，于是有了 GraphRats。

把全讲串起来，这是一份”语义先于优化”的作业说明。模拟语义很简单：每轮每只鼠依据局部密度选择停留或移动，权重由奖励函数给出；规模是百万鼠、两万五千节点。奖励函数把负载因子映射为正权重，理想值 1.5 处取 1，空节点降到 0.2，高负载按对数缓慢衰减但仍非零——三项性质对应了函数的具体形状。移动选择把五个候选的奖励累加成区间，随机数落点决定选择，这是全部算术密集所在。程序必须逐位确定，随机数来自固定种子，这约束了并行方案。三种更新模式中，同步最容易并行但产生振荡，按鼠序最自然却无法并行，批量模式以 2

本讲义在讲者材料之上做的压缩，有三点需要与讲者原话区分：

- 确定性约束是本题的核心难点，不是附加要求。它排除了任何引入非确定顺序的并行手段，因此优化的方向必须是”保持逐位一致的前提下提高并行度”，而不是”提高并行度再想办法回归”。
- 批量模式的价值在于它把一个不可并行的全局顺序，改造成了批内并行加批间有序。批次大小是并行粒度与语义保真度之间的旋钮，而编号顺序的打乱是这个折中能够成立的前提。
- 非规则图会产生结构性热点。hub 节点因连接数极高而吸引超比例负载，这与奖励值无关。这类不均来自图结构本身，无法通过调整调度策略消除，只能通过划分策略处理。

这堂课的边界在于它是作业说明，讲者明确表示实现的细节需要学生自己解决，也提醒给定测试不足以覆盖真实规模。

两个可以直接从本讲数据提出的下一步问题：其一，同步模式与批量模式的可并行度不同，同一套并行化方案是否可能同时高效地服务两者？其二，hub 造成的热点在 tile 图上不可避免，那么应当按位置划分还是按度数划分，才能让负载均衡与通信量同时可接受？

Lecture 13 Implications of Cache Coherence

视频时长：1 小时 6 分 59 秒　来源：https://www.bilibili.com/video/BV1wN4116725/

13.1 伪共享：代码看起来没问题，性能却崩了

讲者开场补上第 11 讲没讲完的内容，他说这可能是程序员最需要理解的一件事：伪共享。

机制

设想有一段数据，其中若干部件恰好落在同一条缓存行里，但被不同核上的不同线程使用。它们开始为这份数据争抢。

假设计算是可写的，例如两个线程各自读写自己的计数器。原则上这些操作完全独立，各自的 L1 缓存可以分别装载这些值，系统本可以跑得很好。

但因为它们恰好被分配在同一条缓存行里，缓存系统认为这相当于要对每个值提供独占访问。于是两个核陷入协议层面的争夺，而写操作的开销远高于读。讲者的原话是：共享完全不可能发生，程序就失控了。

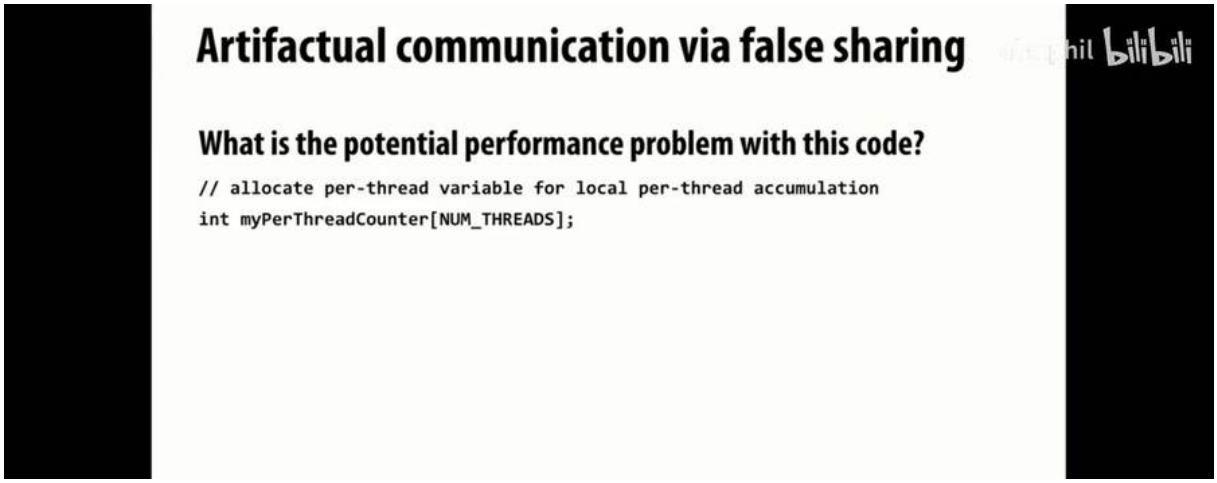

图 285: 人为通信：伪共享造成的额外开销[285]

[285] 视频画面时间区间：00:01:30–00:01:45。若为裁剪图，时间区间仍对应原视频画面。

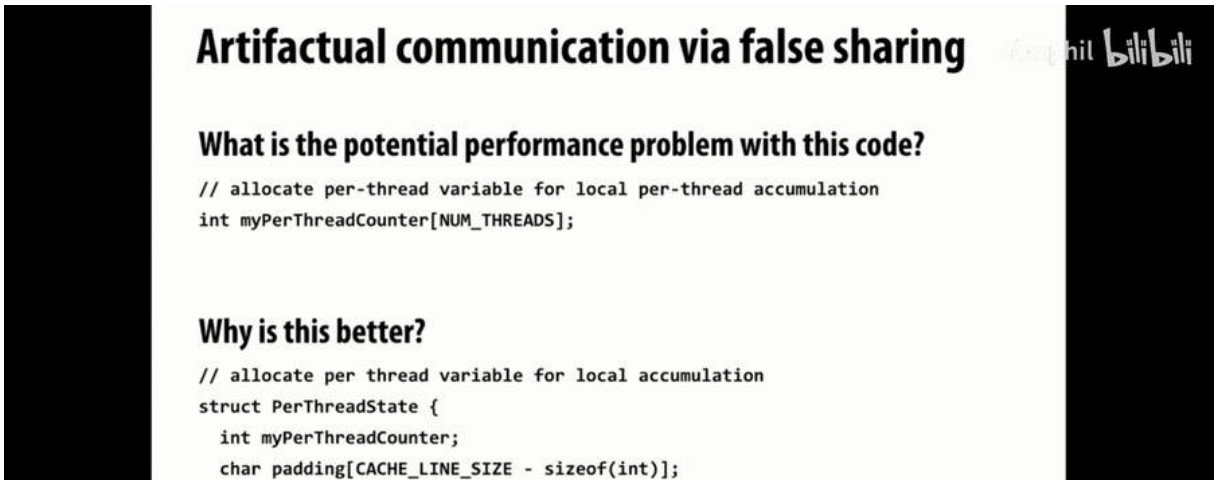

图 286: 伪共享示意：两个处理器写不同地址却争用同一缓存行[286]

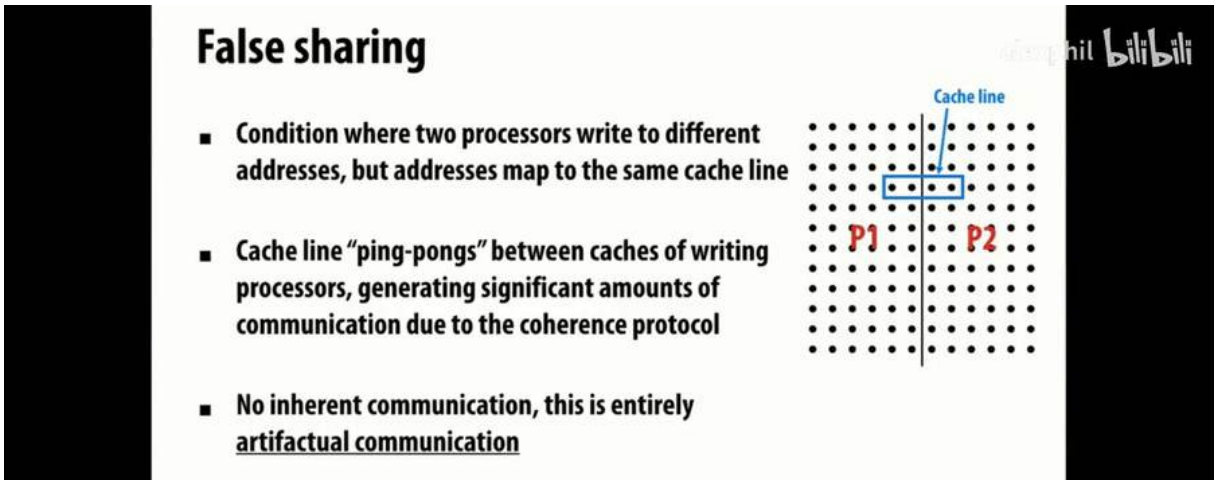

图 287: 伪共享的精确条件：两个处理器写不同地址，但落在同一缓存行[287]

伪共享的判据是缓存行，不是变量

两个线程是否伪共享，取决于它们的写目标是否落在同一缓存行内，与它们是否访问同一个变量无关。一条 64 字节的行可以容纳十六个 int 计数器，若每个线程各用一个，表面上毫无冲突，实际每次写都在抢占同一行的所有权。

一个能直接看到差别的例子

讲者用 pthreads 举例。分配一个与线程数等长的计数器数组，每个线程独立自增自己那个计数器。

另一个版本做了填充：数组里每个计数器后面跟一些空位，使它们各自落进不同的缓存行。

[286] 视频画面时间区间：00:03:00–00:03:15。若为裁剪图，时间区间仍对应原视频画面。

[287] 视频画面时间区间：00:05:00–00:05:15。若为裁剪图，时间区间仍对应原视频画面。

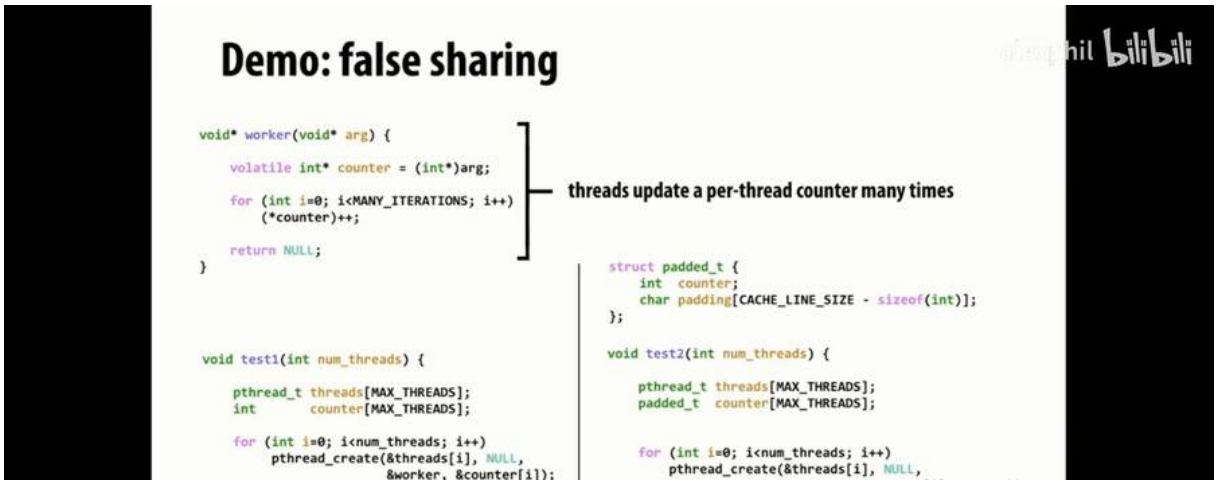

图 288: 伪共享演示：无填充的计数器数组与带填充版本[288]

第一个版本里所有计数器都在同一行，伪共享极其严重；第二个版本各自独占一行，彼此完全独立。

可以把伪共享的代价形式化为一行的往返次数：

$$N_{\text{bounce}} \approx \min(W_1, W_2), \qquad T_{\text{extra}} \approx N_{\text{bounce}} \cdot t_{\text{line}}$$

- W_1, W_2：两个线程各自对该缓存行的写次数；
- N_{bounce}：该行在两次写之间被交接的次数；
- t_{line}：一次缓存行失效加传输的耗时。

填充后 W_2 对应的行与 W_1 不再相同，N_{bounce} 直接降为零。

在一台 12 核机器上的实测是：一种情况 5 秒，另一种 2 秒。讲者强调这已经是相当温和的情形，实际可以差得多。

$$\frac{T_{\text{false sharing}}}{T_{\text{padded}}} = \frac{5}{2} = 2.5$$

- 分子：所有计数器落在同一缓存行时的耗时；
- 分母：每个计数器独占一行时的耗时；
- 比值越大，说明这行数据在核间弹跳得越厉害。

无同步代码也可能被缓存协议拖慢

伪共享的代码里没有任何锁、没有任何同步原语。从并发正确性角度看它完全正确，从性能角度看却在滥用整个缓存系统。这类问题不会报错、不会给出错误结果，只在性能曲线上表现为莫名其妙的退化，因此必须在测量中主动排查。

[288] 视频画面时间区间：00:04:30–00:04:45。若为裁剪图，时间区间仍对应原视频画面。

与分块布局的关系

讲者指出这正是为什么第 8 讲讨论二维分块时采用那种四维的布局方式：让每个处理器拿到一段连续地址。

这样伪共享最多只发生在这些区段的交界处，影响相对温和，而不会在每一行都发生。

布局把伪共享从”每行”降为”每边界”

若按行分配给不同处理器，那么每行的两端都会与邻居产生共享行，伪共享发生次数与行数同阶。若按块分配，只有块边界处才有共享行，次数与块数同阶。同一个问题，仅仅改变数据归属方式，伪共享的量级就降了一个层次。

本章小结

伪共享是缓存行粒度造成的：逻辑上独立的变量因同行而互相作废。它不破坏正确性，只破坏性能，实测可以差两倍以上。对策是填充使各线程的写目标独占缓存行，或改变布局让共享只发生在边界。

理解了这个代价，才能理解为什么一致性协议本身的实现方式会直接决定程序性能。下一章转向协议如何扩展。

本章小结（续）

讲者用一句话收束这段补充：即使没有任何同步，在这些机器上编程也必须理解这类现象。

这为整讲定下基调：一致性不是硬件内部的细节，它的实现方式会渗到程序性能里。

13.2 监听协议的扩展性上限

问题在哪

讲者先回顾第 11 讲的监听协议：某个缓存发生任何值得关注的操作时，它必须广播给所有其他缓存。

只要同时监听的缓存只有 6 个左右，这没问题。但规模再往上，流量就太多了——总线上的通信本身成为整个系统的中心瓶颈。

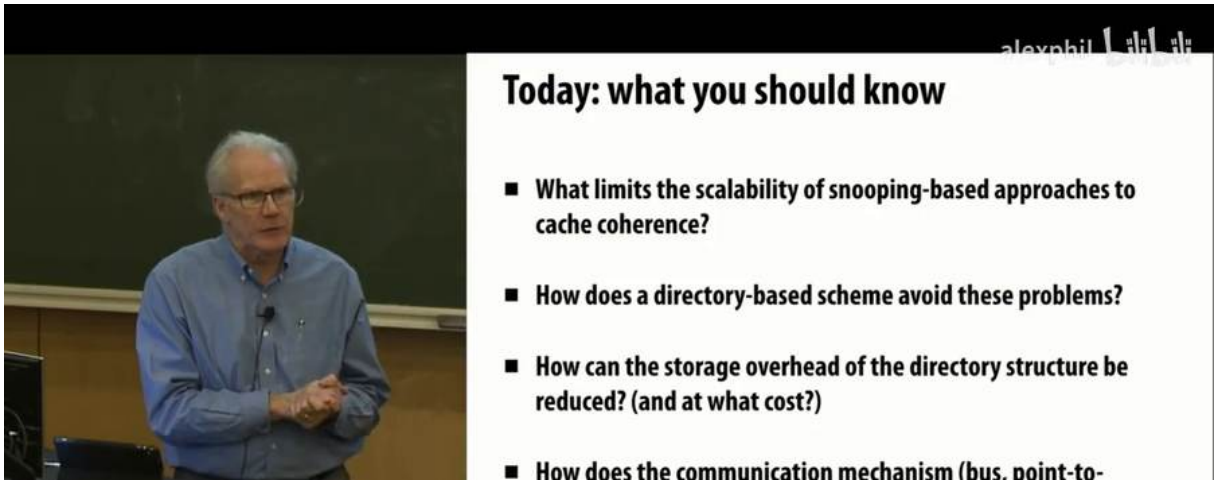

图 289: 本讲要点：什么限制了一致性的可扩展性[289]

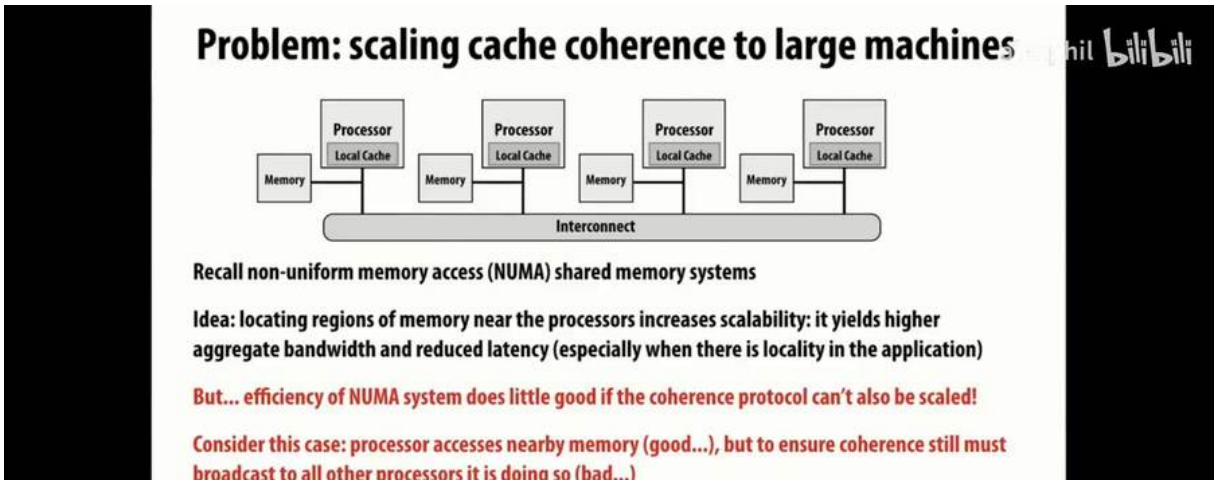

图 291: 问题：把缓存一致性扩展到大型机器[291]

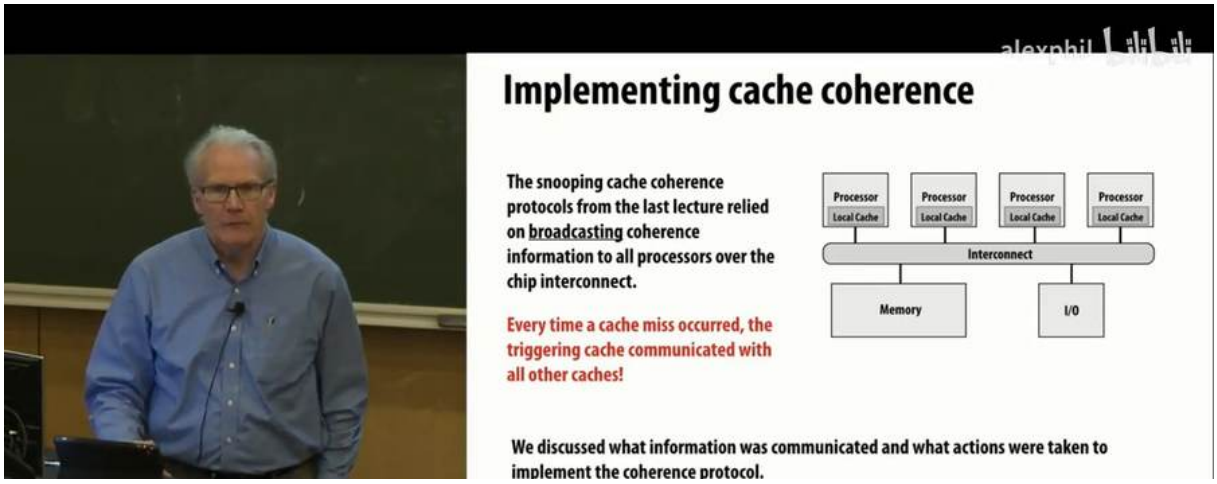

图 290: 实现缓存一致性：监听协议依赖处理器间广播[290]

讲者解释总线的含义：它就是一组共享的导线，任何人放上去的数据所有人都会看到并能接收。可以把它想成一个所有消息进出的中心点。

具体来说，一个缓存要访问某数据，就得在总线上发出请求说” 我要这行的独占读或非独占读”；要升级自己的副本为可写，就得广播并作废他人的副本。互连上的流量因此非常重。

理想与现实的差距

理想方案是点对点：只在持有数据的处理器与想要数据的处理器之间通信。

[289] 视频画面时间区间：00:07:30–00:07:45。若为裁剪图，时间区间仍对应原视频画面。
[290] 视频画面时间区间：00:08:30–00:08:45。若为裁剪图，时间区间仍对应原视频画面。
[291] 视频画面时间区间：00:10:30–00:10:45。若为裁剪图，时间区间仍对应原视频画面。

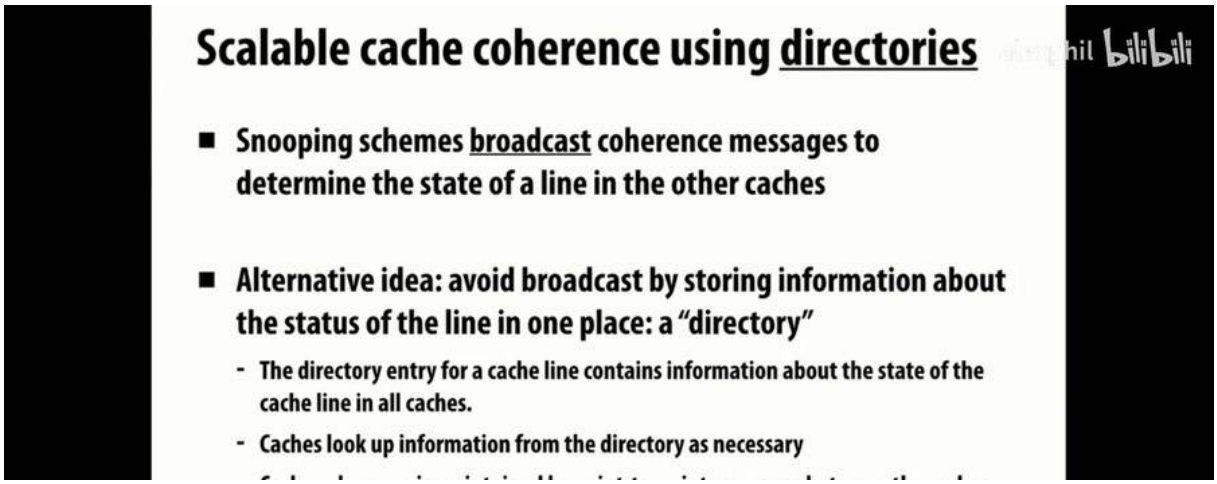

图 293: 用目录实现可扩展的缓存一致性[293]

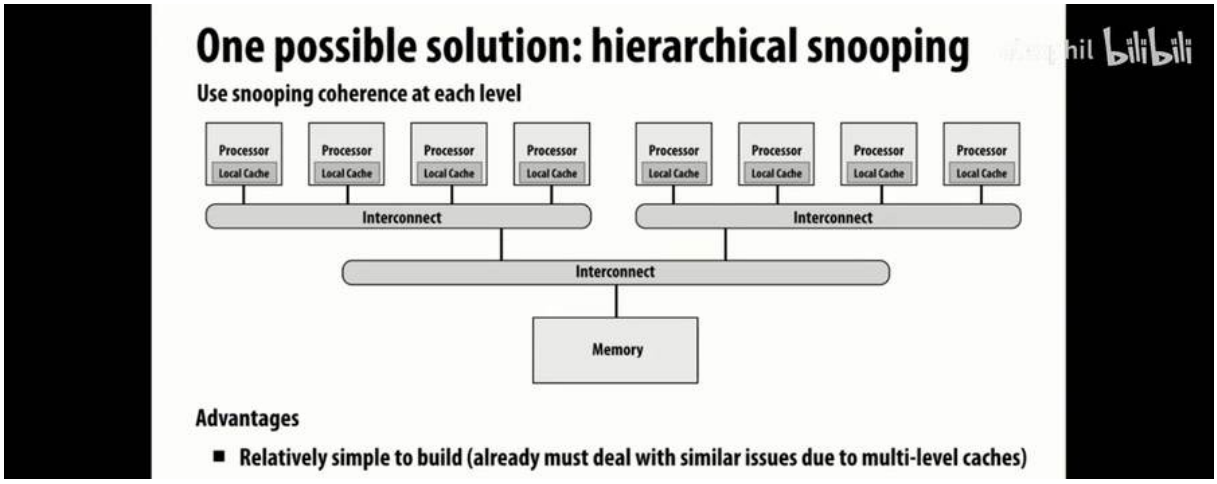

图 292: 一种解法：层次化监听，在每一层使用监听一致性[292]

讲者说现实中没人用简单版本，而是用层次化结构：小规模的例子容易跑，系统变大就叠加更复杂的方案。

一个常见做法是把互连做成树状，让能在局部第一层总线上解决的流量留在那里。比如某处理器要读，而持有者恰好在同一个局部总线上，这次读就在本地解决。若持有者在别处，才需要穿越更高层次的树。

优点是降低中心拥塞。但讲者明确点出最坏情况：如果工作负载在内存流量上没有被很好地划分，树根就会成为新的瓶颈。

他也提到这种结构在 NUMA 场景下更有效——内存本身被切成多个片段，本地化带来的收益更大。

本章小结

监听协议的瓶颈是广播：流量随参与者数量增长，超过约半打缓存后互连成为中心瓶颈。层次化监听用树结构把局部流量吸收在低层，降低中心拥塞，但最坏情况下树根仍是瓶颈，除非负载在

[292] 视频画面时间区间：00:13:00–00:13:15。若为裁剪图，时间区间仍对应原视频画面。
[293] 视频画面时间区间：00:14:30–00:14:45。若为裁剪图，时间区间仍对应原视频画面。

内存访问上已经划分良好。

要真正把通信变成点对点，需要一份记录”谁持有哪份数据”的信息。这就是目录。

13.3 目录：把广播变成点对点

基本思想

讲者设定场景：多个处理器各自能访问内存，内存被分区到各处理器上，希望实现全局的缓存一致访问。做法是引入目录。

目录记录哪些处理器持有某个数据块的副本。有了这份信息，读的时候可以直接去找持有者取，作废的时候只需给持有副本的那些处理器发消息，而不必广播。

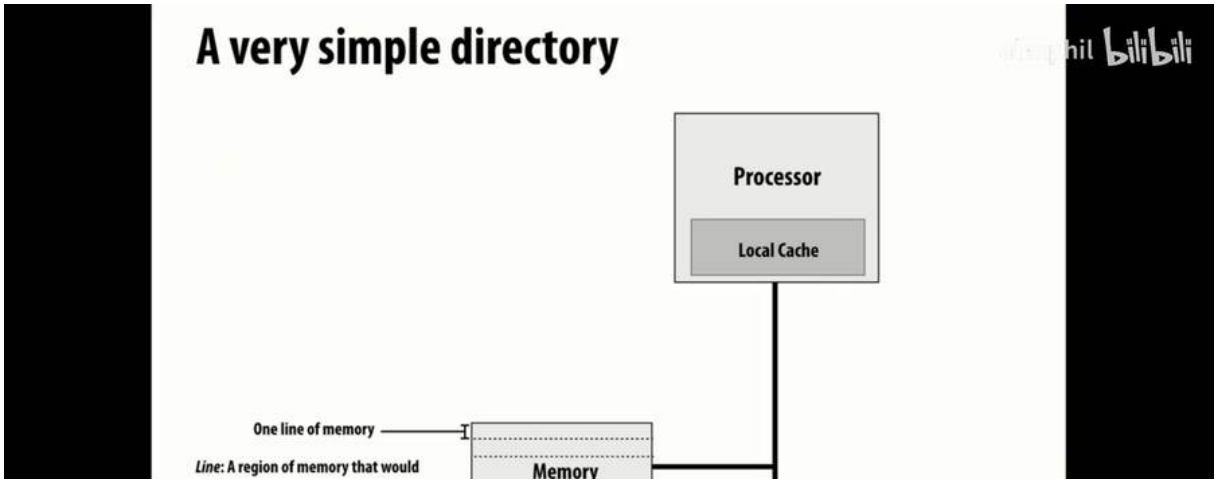

图 294: 一个非常简单的目录：内存按行划分，每行配一组状态位[294]

数据结构

内存和缓存一样按块组织，这里称为行，为讨论方便取 64 字节一段。

在此基础上，为内存中每一行增加一组位：为系统中每个可能的处理器准备一位，再加一个位表示该行是否脏。这组位向量说明哪些处理器在自己的缓存里有这一行的副本。

为简化，只考虑单级缓存。

与此配套的几个术语：每一行 64 字节的内存在某个处理器上有它的 home node，也就是它在这台机器的内存空间中归属的位置——这个位置与该数据实际在哪被使用完全无关。另外，发起读或写的处理器称为 requesting node。

[294]视频画面时间区间：00:15:30–00:15:45。若为裁剪图，时间区间仍对应原视频画面。

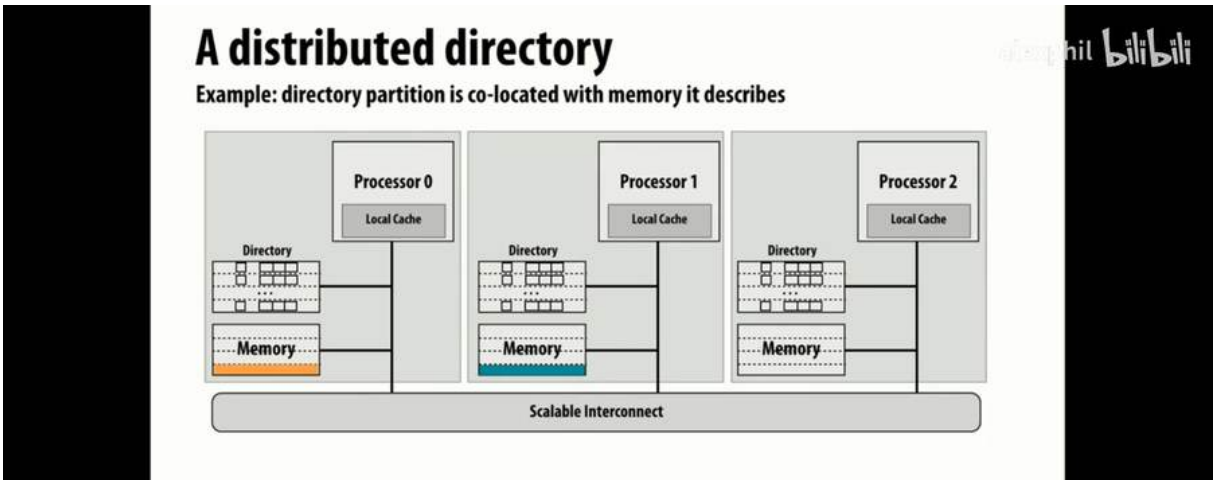

图 295: 分布式目录：目录分区与其描述的内存共置[295]

home node 与持有者是两个不同的概念

home node 由地址决定，是固定的；持有者随程序执行而变化。目录本身存在 home 上，因此请求总是先发往 home，再由 home 决定是直接返回数据，还是把请求转给真正的持有者。混淆这两者就无法理解后面"home 没有副本" 的情形。

三种典型情形

讲者用三个例子说明协议。假设内存某行由处理器 1 拥有，处理器 0 想读。

情形一：读缺失且该行干净。处理器 0 发现自己没有副本，查地址得知归属处理器 1，于是发出一个读请求。处理器 1 直接回一份副本，并在位向量里标出处理器 0 现在有副本。

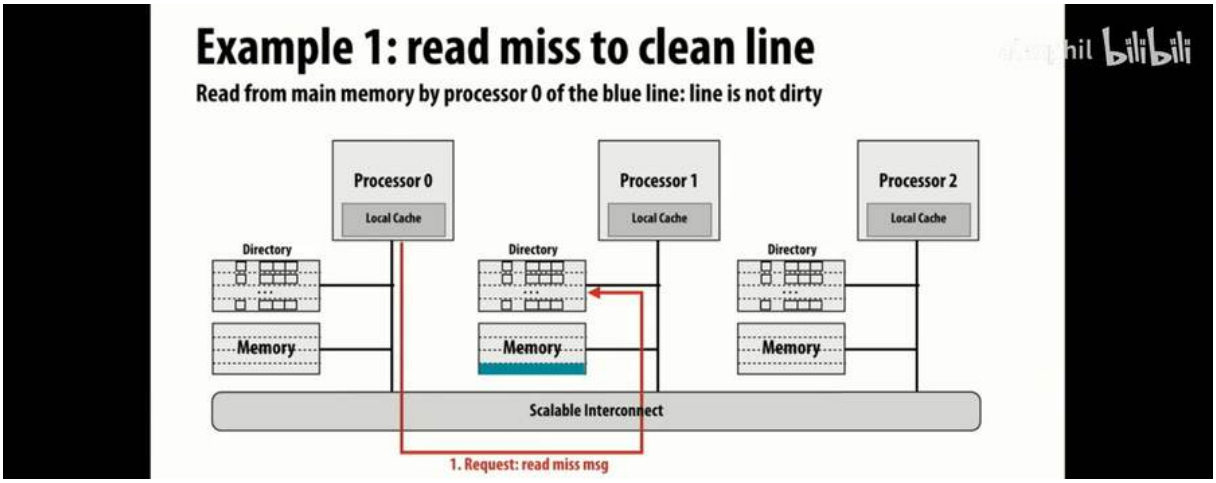

图 296: 例一：读缺失，行为干净，直接由 home 返回数据[296]

情形二：读缺失但该行脏。请求同样发往 home。这时 home 发现自己这份是脏的——或者不是最新副本，需要处理更复杂的情形。

[295] 视频画面时间区间：00:17:00–00:17:15。若为裁剪图，时间区间仍对应原视频画面。
[296] 视频画面时间区间：00:18:00–00:18:15。若为裁剪图，时间区间仍对应原视频画面。

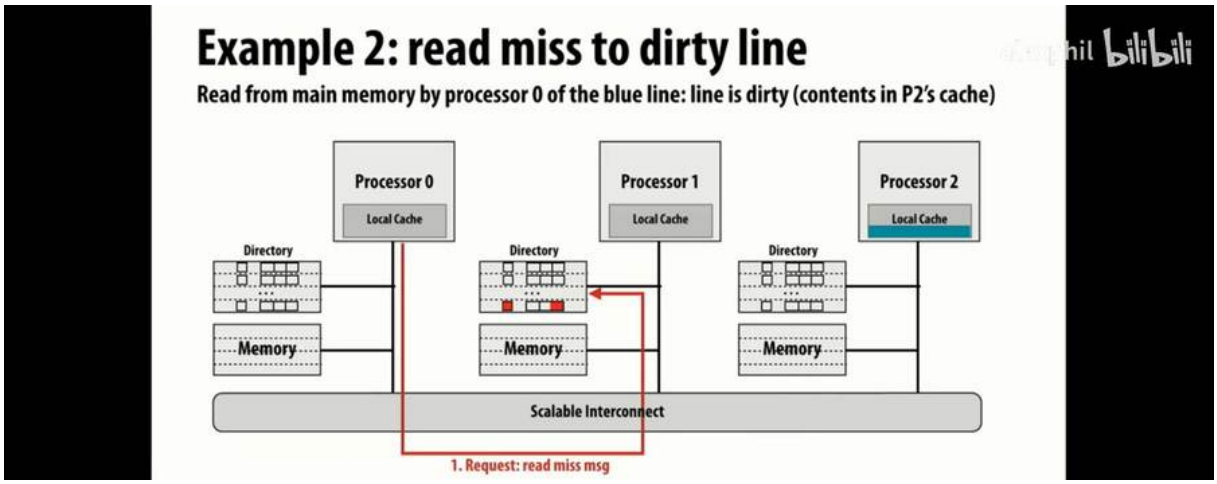

图 297: 例二：读缺失但行为脏，需要先取得最新副本[297]

情形三：写缺失，该行干净但被处理器 1 和 2 持有。这是最复杂的例子。

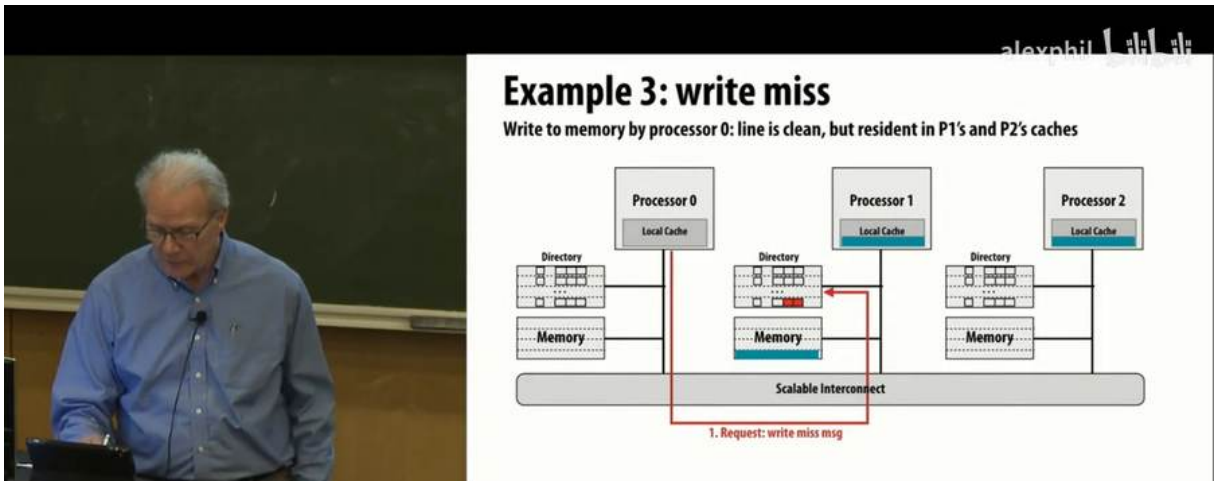

图 298: 例三：写缺失，行干净但驻留在 P1 与 P2 的缓存中[298]

[297] 视频画面时间区间：00:19:00–00:19:15。若为裁剪图，时间区间仍对应原视频画面。
[298] 视频画面时间区间：00:21:00–00:21:15。若为裁剪图，时间区间仍对应原视频画面。

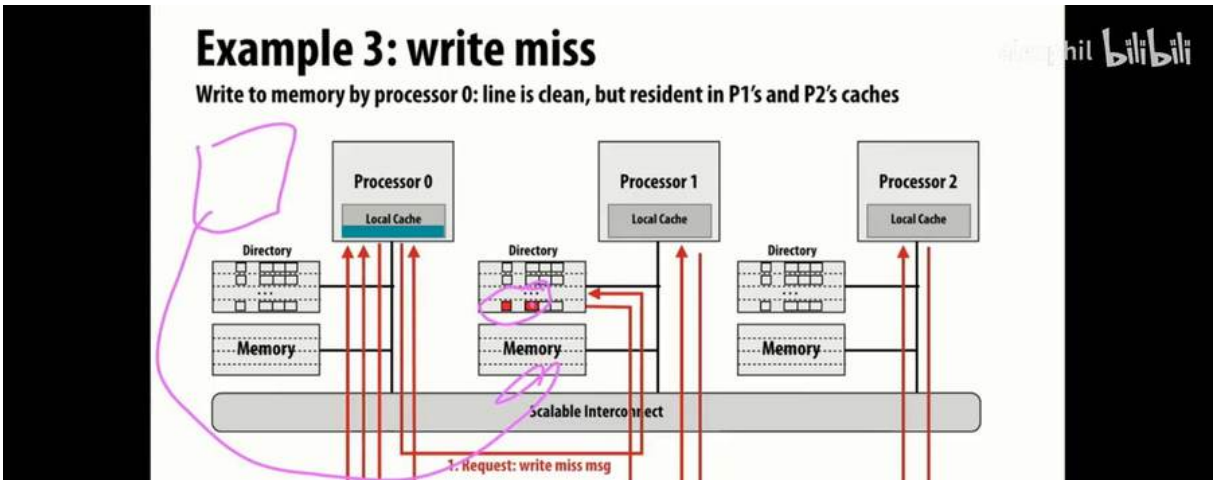

图 299: 例三续：必须等到全部失效确认后才能继续[299]

为什么必须等待确认

讲者在这里强调了正确性的核心。

前面看过的失效协议里，必须确保失效真的发生了，否则处理器 0 和 1 可能跑在前面，访问一个本该被标记为无效的内存位置。

因此处理器 0 必须等到收到全部确认之后才继续。而这时持有者才把该行标记为脏、持有在处理器 0 的缓存里。

失效确认是正确性的关键，不只是性能细节

没有等待全部 ack 就继续执行，可能出现两个处理器都认为自己持有可写副本的局面，那就是灾难。协议必须保证” 独占” 这个判断在全局唯一，而唯一性靠的是确认链，不是靠时序巧合。这类细节在多核一致性协议中极其复杂，讲者也承认真实协议远比示意版本麻烦。

还有一个细节值得记录：需要适当的 FIFO 队列，保证各处消息不相互越过，否则在等待期间若有其他处理器发来请求，可能得到一个错误的副本。

home 没有副本的情形

学生提出一个讲者没走到的情形：处理器 0 是请求者，而 home 自己也没有副本。讲者承认这确实是个未覆盖的例子，并分析说这本质上是对脏行的写缺失。

这时 home 无法提供数据，必须增加一步：home 回话说” 其实这行在处理器 2 或 3 的缓存里”。好处是，如果 home 拿不出来，就说明存在一份未共享的可写副本。于是请求者可以直接发消息要求对方交出。

讲者评价说真实协议” 毛发一样多”（极其繁琐），他展示的只是最基础版本。

本章小结

目录用位向量记录每行的持有者，使通信从广播变成点对点：读时直接找持有者，作废时只通知持有者。正确性依赖失效确认与消息顺序，真实协议的复杂度远超示意版本。

[299] 视频画面时间区间：00:24:00–00:24:15。若为裁剪图，时间区间仍对应原视频画面。

这一章解释了机制，但没说明它是否值得。下一章用实测统计回答这个问题。

13.4 目录的收益与代价

收益的前提

讲者给出关键判断：目录减少总线流量的前提是共享者很少。

设想系统有 256 个处理器，但某行只有一份副本在外面。那么只需给那一个发失效消息，而不是广播给 256 个。在这种相当常见的场景下，流量大幅下降。

反过来说，如果共享者很多，目录相对广播协议并不能省什么。

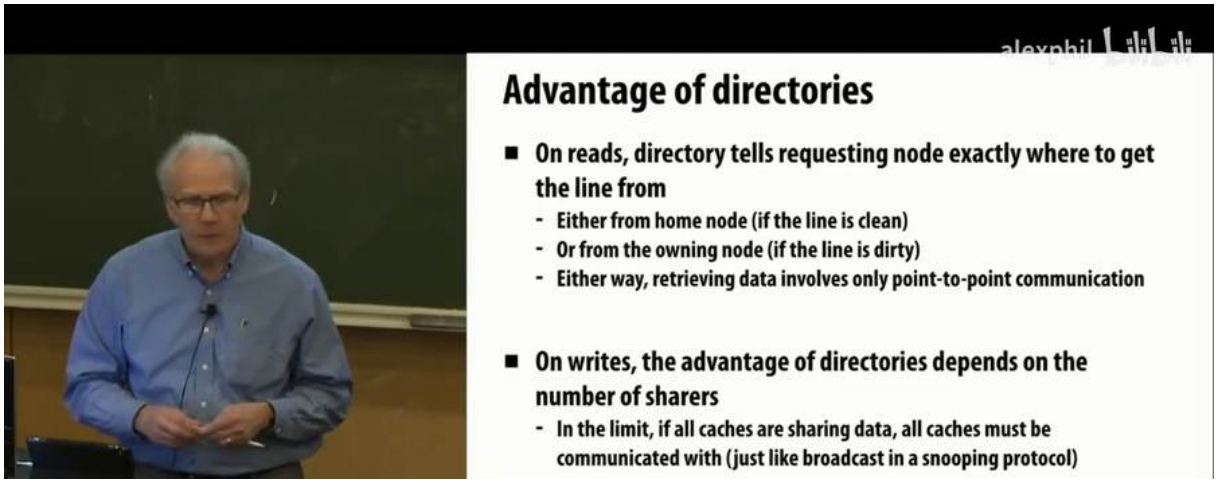

图 300: 目录的优势：读时告知确切来源，失效时只通知持有者[300]

实测统计

讲者引用了一本教科书里的统计数据：在若干基准程序上，统计每次写时该行被多少处理器共享。

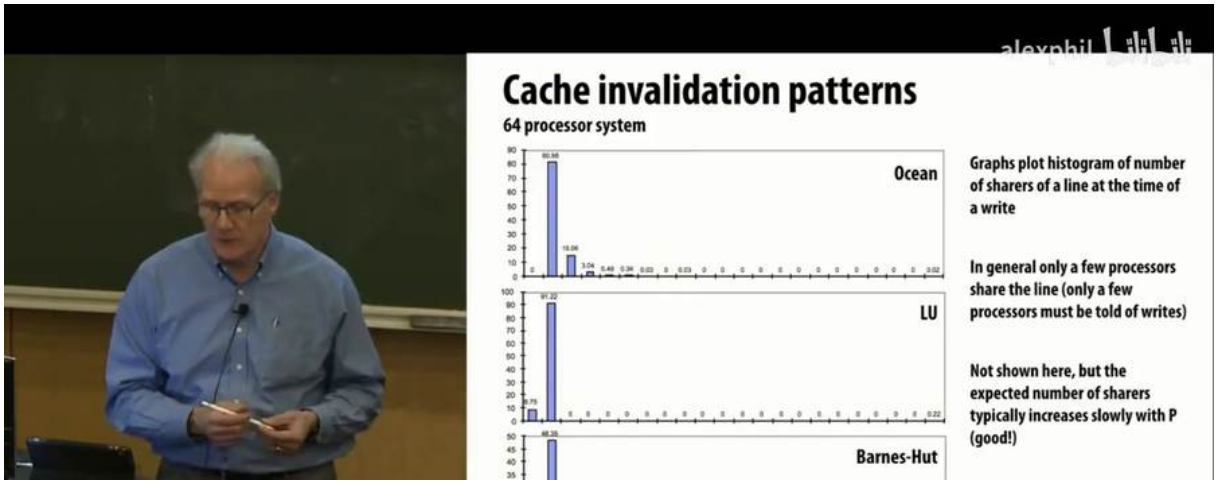

图 301: 64 处理器系统中的失效模式统计[301]

[300] 视频画面时间区间：00:27:00–00:27:15。若为裁剪图，时间区间仍对应原视频画面。

[301] 视频画面时间区间：00:28:00–00:28:15。若为裁剪图，时间区间仍对应原视频画面。

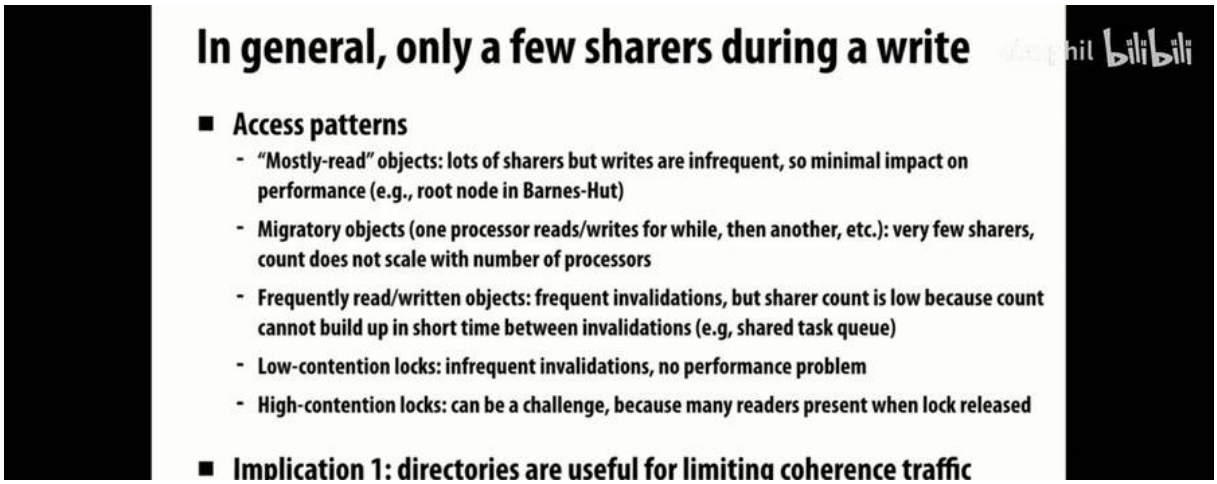

图 302: 统计显示写时通常只有很少共享者[302]

结果很有说服力：在多数程序里，90

唯一的例外是 Barnes-Hut，其中只有 48% 是单一共享。讲者解释原因：那棵树（四叉树）意味着要经常上下遍历，因此树的上层结构被大量共享。

目录的收益来自分布的形状而非平均值

如果共享者数均匀分布，目录的平均收益有限。真正带来收益的是分布严重偏斜：绝大多数情形是零或一个共享者，少数情形共享者很多。这种长尾分布让”优化常见情形”的收益远大于平均值所暗示的程度。

访问模式的分类

讲者把数据访问模式分为几类，用于定性判断各方案表现。

主要是读取的：数据很少被写。这类在任何合理方案下表现都好——各处理器都拿到干净副本，享受缓存的最大收益。

迁移型的：一处连续写一段，然后交给另一处连续读。传输本身可能有相当流量，但相对其他操作是一次性的，因此可以接受。

[302] 视频画面时间区间：00:30:30–00:30:45。若为裁剪图，时间区间仍对应原视频画面。

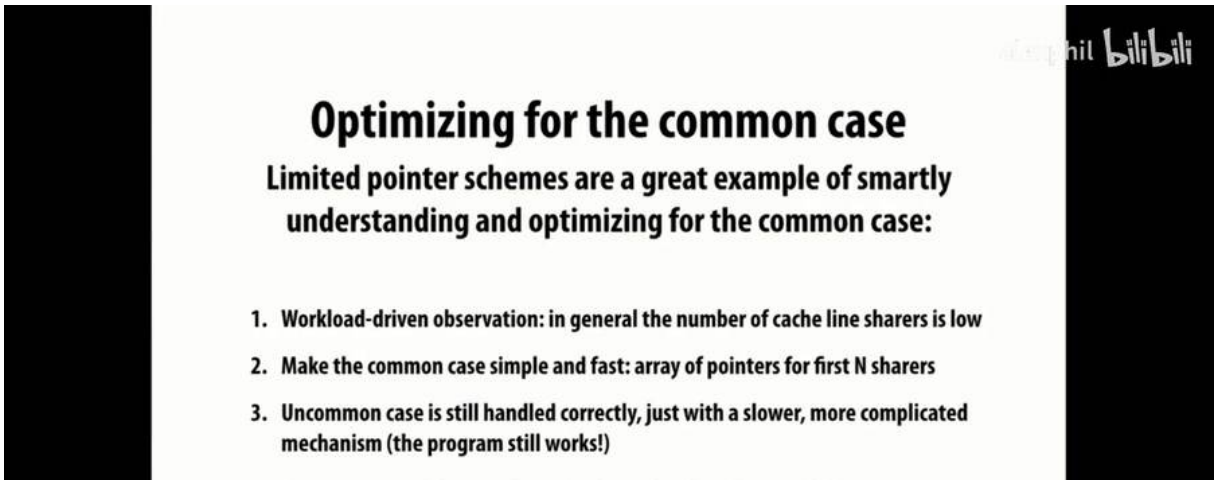

图 303: 为常见情形优化：有限指针方案是典型例子[303]

只按平均情况设计会错过优化的重点

讲者反复强调”优化常见情形”。若某类访问占八成且每次只需一条消息，那么把它做到极致，比均匀优化所有情形收益大得多。这也是有限指针、稀疏目录这些方案的设计出发点，它们都牺牲了对罕见情形的效率来换取常见情形的成本。

本章小结

目录的收益来自分布偏斜：多数应用的写只涉及零到一份其他副本，因此点对点失效远优于广播。Barnes-Hut 因四叉树上层被大量共享而成为例外。访问模式上，主要是读取的数据在任何方案下都好，迁移型数据的一次性传输相对可接受。

收益有前提，代价也有具体数字。下一章处理目录自身的存储成本。

13.5 目录的存储代价与压缩方案

全位向量的开销

简单的全位向量方案有一个明确代价：每个内存行都要 P 位，P 是处理器数。

整个目录的总存储可以估出来：

$$S_{\text{dir}} = \frac{M}{L} \cdot (P + 1)$$

- M：物理内存总字节数；
- L：缓存行字节数，本例为 64；
- P：处理器数量，全位向量为每个处理器一位；
- 额外的 1 是脏位。

当 P 远大于 L 的位数时，目录本身就与数据同量级，这是必须压缩的原因。

讲者算了一笔账。若 P 是 1024，每行要 1024 位说明共享状态。

[303] 视频画面时间区间：00:45:00–00:45:15。若为裁剪图，时间区间仍对应原视频画面。

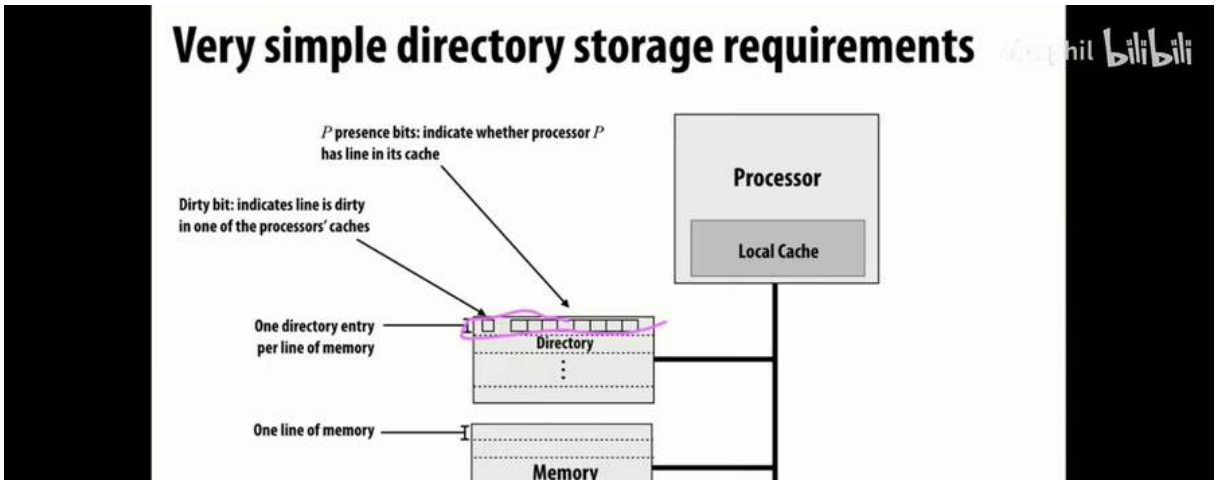

图 304: 简单目录的存储开销：每行 P 个存在位[304]

优化思路是有限指针方案：既然数据预期只出现在少数缓存里，就只存有限数量的指针。若每个指针 10 位，那么存 5 个共享者只需 50 位——远小于 1024 位。

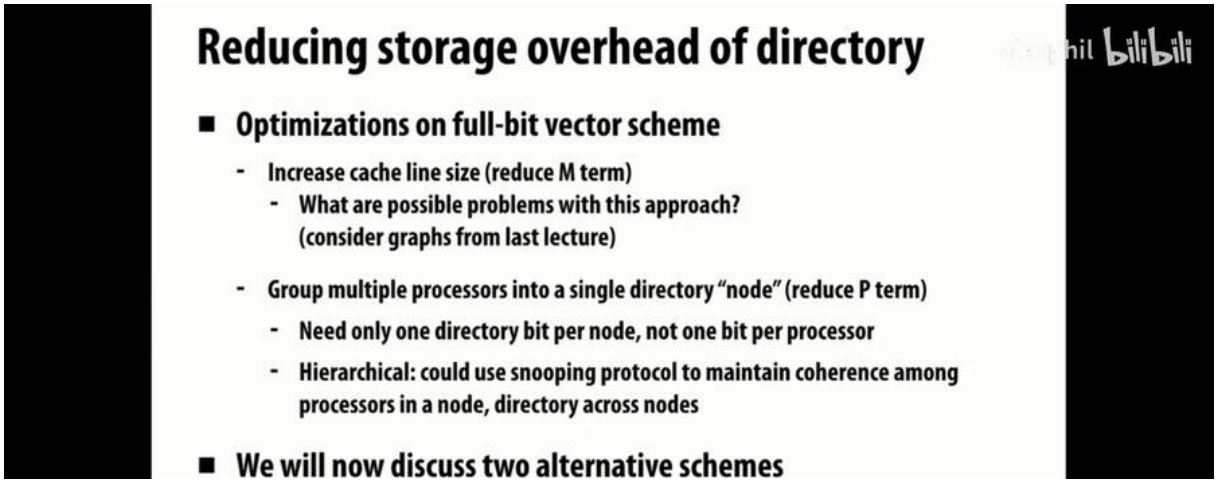

图 305: 降低目录存储开销的优化：从全位向量到有限指针[305]

[304] 视频画面时间区间：00:36:00–00:36:15。若为裁剪图，时间区间仍对应原视频画面。
[305] 视频画面时间区间：00:40:30–00:40:45。若为裁剪图，时间区间仍对应原视频画面。

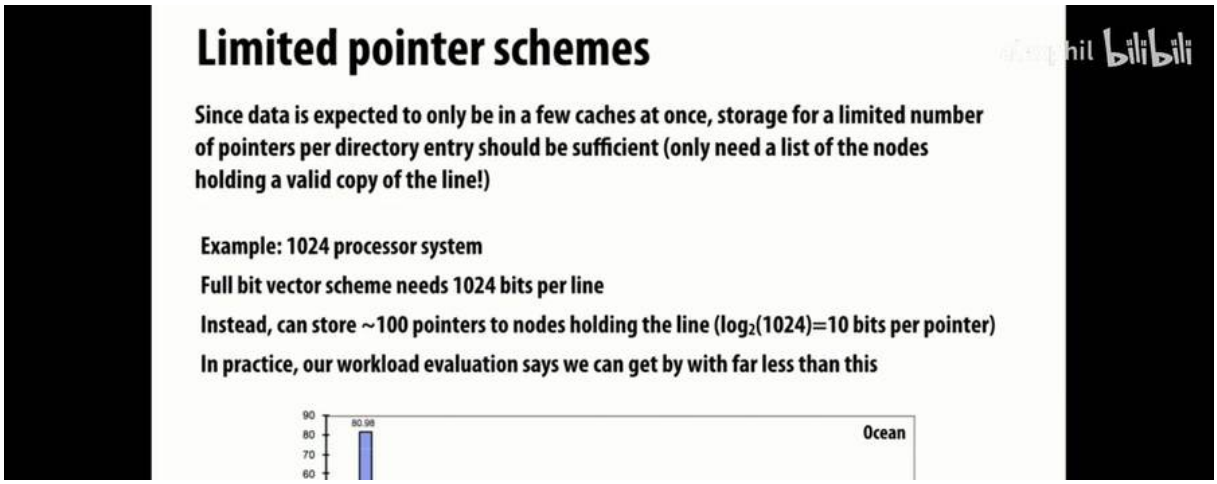

图 306: 有限指针方案：只记录少数共享者的位置[306]

$$N_{\text{bits}}^{\text{full}} = P, \qquad N_{\text{bits}}^{\text{limited}} = k \cdot \log_2 P$$

- P：处理器数量，本例中为 1024；
- k：允许记录的最大共享者数，例如 5；
- 全位向量需 1024 位，有限指针只需 $5 \times 10 = 50$ 位，约为前者的 5%。

溢出怎么办

受限方案必然遇到共享者超过 k 的情形。讲者的处理是回退到广播：声明这一行进入”无限共享者”模式，之后对该行的失效就退回广播。

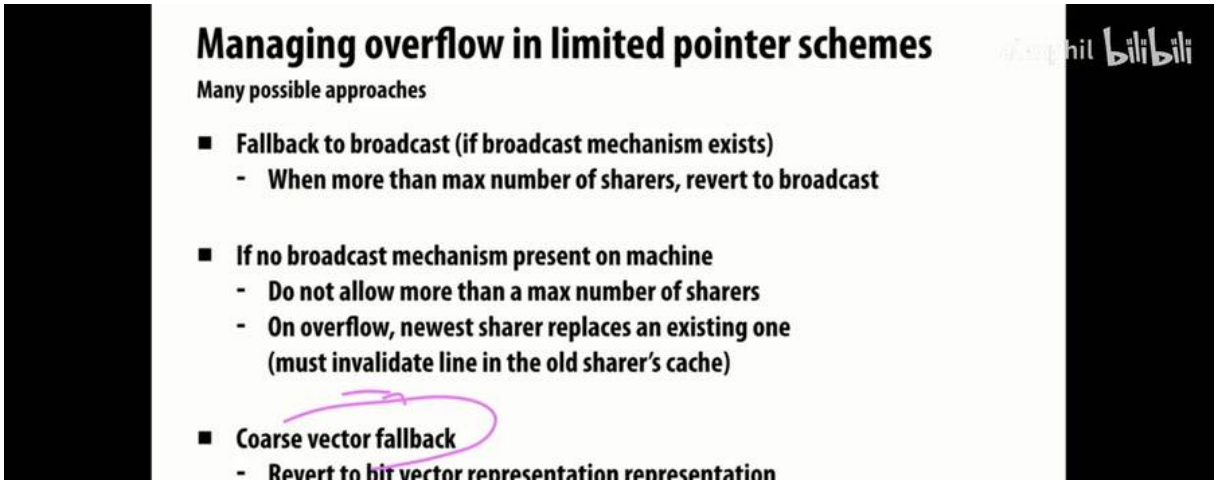

图 307: 有限指针方案的溢出处理：多种可能的回退策略[307]

他提到另一种思路：限制共享者数量本身。就像缓存满了会淘汰，目录满时可以说”我已经有最多共享者了，你得走一个”，然后向最久未使用的那个副本发失效。

[306] 视频画面时间区间：00:41:30–00:41:45。若为裁剪图，时间区间仍对应原视频画面。
[307] 视频画面时间区间：00:43:30–00:43:45。若为裁剪图，时间区间仍对应原视频画面。

溢出处理是一种刻意的容量缺失

让共享者超过上限时被迫退出，本质上是把” 目录容量不足” 转化为一次失效。这与缓存容量缺失同构：硬件用性能换空间。因此设计目录时必须同时考虑条目数和每条的宽度，两者都在与内存预算竞争。

稀疏目录

还有一个维度可以省：并非每一行都有共享副本。讲者说关键观察是内存中大部分内容从未被共享。

因此不必为每一行都保留目录项，只为那些确实存在共享副本的行保留即可。

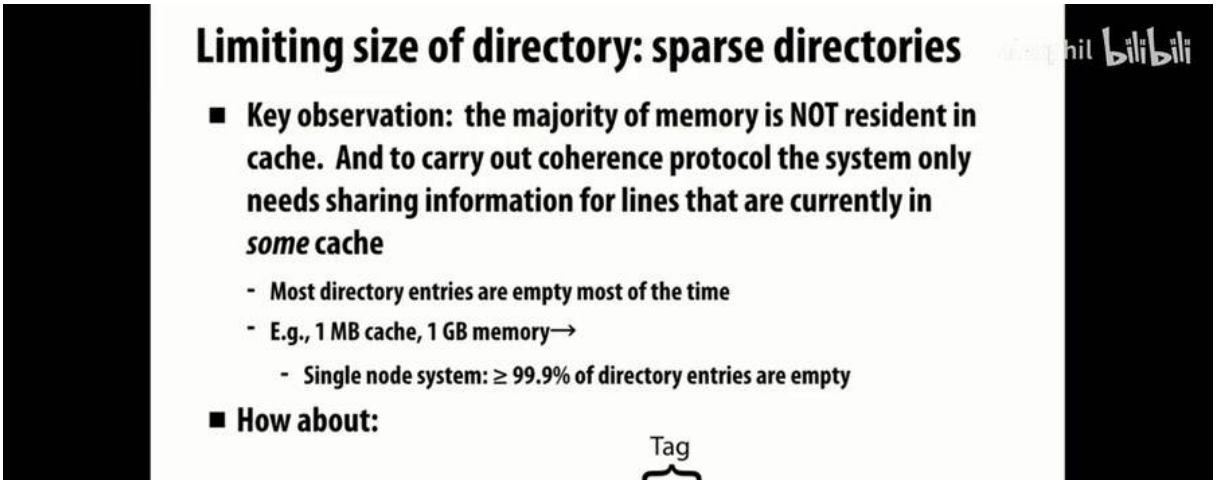

图 308: 用稀疏目录限制目录规模：只为被共享的行保留条目[308]

本章小结

目录的收益来自” 共享者通常很少” 这一分布事实：八成到九成的写只涉及零到一份其他副本。代价是每行 P 位的存储开销，因此有全位向量、有限指针加溢出广播、稀疏目录三条压缩路径。Barnes-Hut 这类共享密集的程序是例外，其树结构上层被大量共享。

讲者也说明这种代价收益权衡是所有硬件设计共同的思维方式：先弄清典型工作负载，把容易且重要的情况做好。

13.6 真实机器的实现与优化

转发介入

讲者介绍了一个把读延迟降低的优化。回顾” 读缺失但行为脏” 的情形：原本是 home 把请求转给持有者，持有者再把数据送回 home，home 再给请求者——绕了一圈。

转发介入的做法是让持有者直接把数据回给请求者，省掉中间的往返。

Intel Core i7：L3 充当目录

讲者展示了真实处理器中的做法：Core i7 的共享 L3 缓存充当集中的目录。

[308] 视频画面时间区间：00:46:00–00:46:15。若为裁剪图，时间区间仍对应原视频画面。
[309] 视频画面时间区间：00:48:00–00:48:15。若为裁剪图，时间区间仍对应原视频画面。

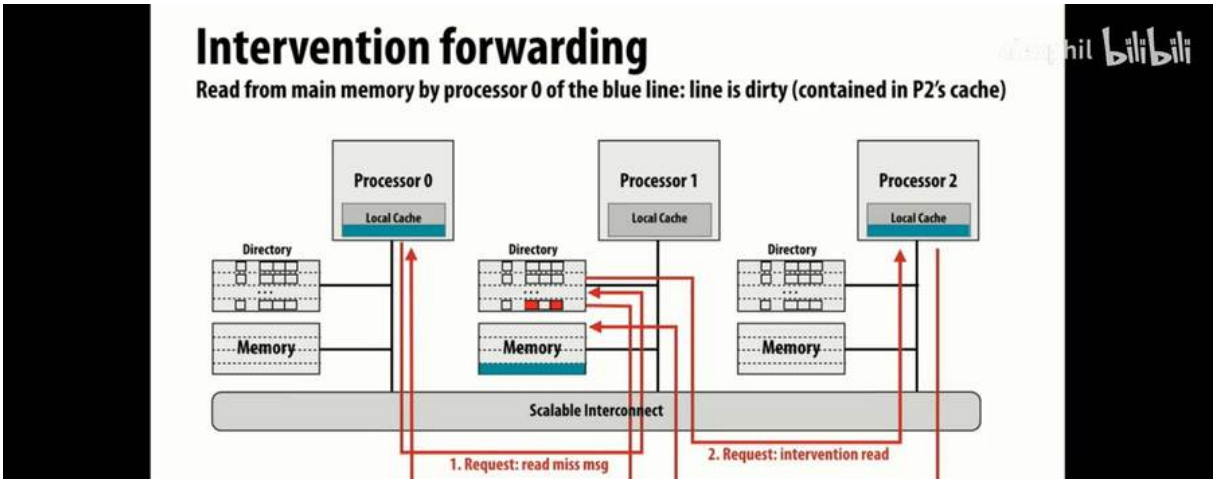

图 309: 转发介入：让持有者直接回应请求者，省去绕回 home[309]

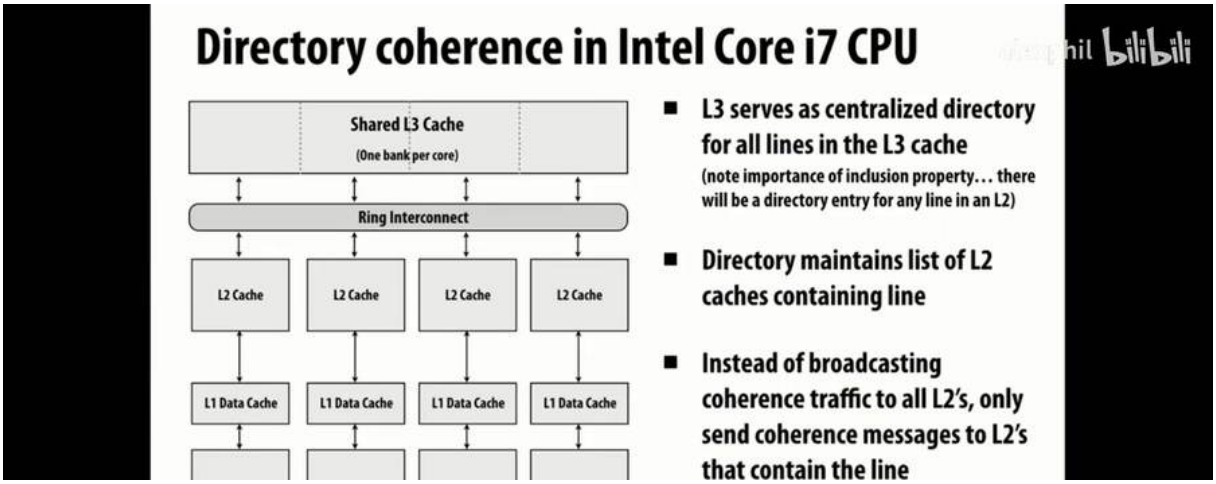

图 310: Intel Core i7 中的目录式一致性：L3 作为集中目录[310]

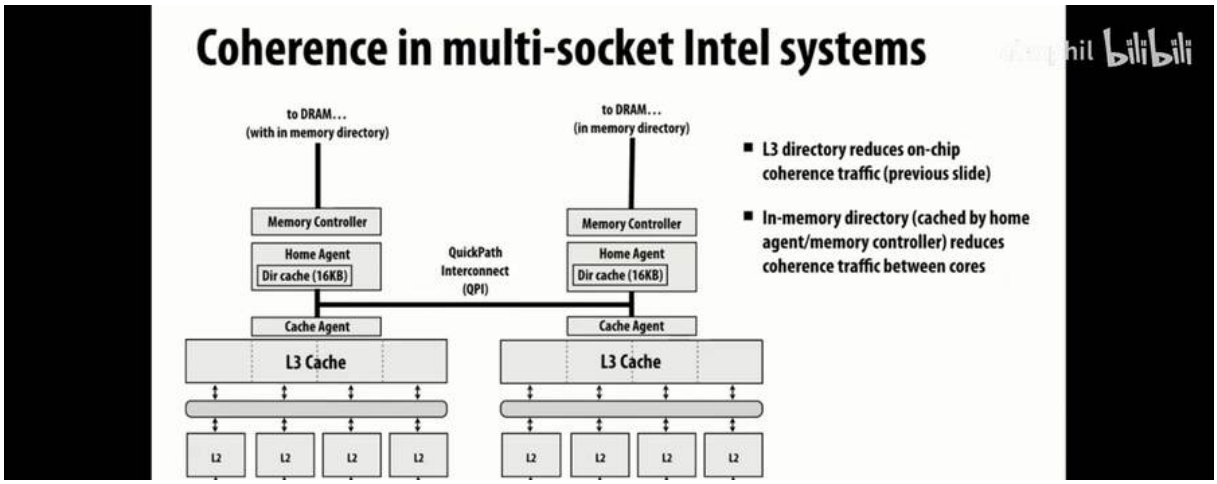

图 311: 多插槽 Intel 系统中的一致性[311]

多插槽系统

多插槽 Intel 系统里，一致性要跨插槽完成，内存侧的目录与插槽间互连共同承担这件事。

Xeon Phi

讲者最后介绍了一类很不同的处理器：Xeon Phi，是 Intel 试图与 NVIDIA 争夺高性能计算的产物。

它的定位是”众核”系统，单芯片上有大量 x86 核心，并为它们提供一致的缓存内存接口。课程机器上挂着 14 块这种卡。讲者说他们能用的版本略旧，代号 Knight's Corner。

他顺带提到一个有意思的事实：天河二号就是用这类芯片搭起来的，曾多年位居世界最快计算机，现在是第二或第三。

讲者随后说明了它的互连方式，这一点值得记住，因为它把前面两种方案结合了起来。

这些处理器之间用一条环形网络通信，而这条环同时被用作维持缓存一致性的基础。内存物理上被划分给不同的内存控制器（大约四个或八个），这些控制器可以看作环上的节点。消息沿环流动，在缓存控制器与内存控制器之间传递。

因为全都在一颗芯片上，环可以做得很宽：数据总线是 64 字节宽，而不是 64 位。这让带宽很高、延迟很低。

环的好处是可以灵活地广播与定向：发一条消息绕环一整圈就实现了广播；若目标是附近的节点，消息只需走到能到达的地方就停。而且它是双向的，两个方向都能发。

于是协议在形式上又回到了前面看过的那种全局协议：发出”我需要一份副本”或”你必须作废你的副本”这类消息。作废消息绕环一整圈，请求消息则走到附近的持有者即可。

[310] 视频画面时间区间：00:49:00–00:49:15。若为裁剪图，时间区间仍对应原视频画面。
[311] 视频画面时间区间：00:51:00–00:51:15。若为裁剪图，时间区间仍对应原视频画面。

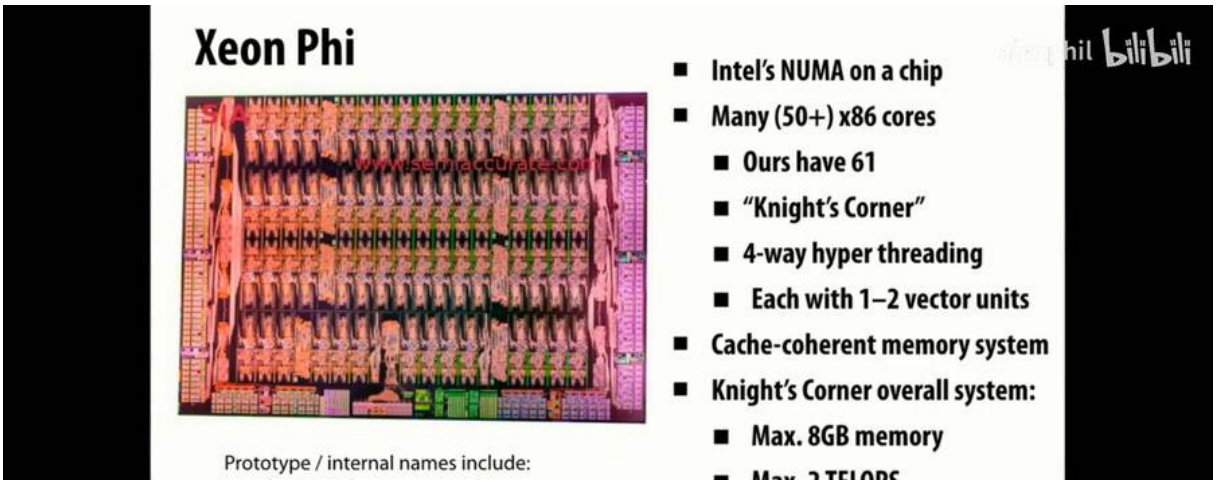

图 312: Xeon Phi：片上 NUMA，50 个以上 x86 核心与 4 路超线程[312]

层次化是真实系统的共同答案

从双插槽到 Xeon Phi，一致性都不是靠单一机制解决的，而是多层叠加：核内缓存、插槽内共享缓存、插槽间目录、片上分区内存。每层处理自己范围内的通信，只有跨层时才付出更高代价。这解释了为什么讲者说现实中没人用简单版本。

本章小结

真实系统在目录基础上加入了转发介入（降低读延迟）、集中目录（i7 的 L3）、跨插槽目录与片上 NUMA（Xeon Phi）。共同点是层次化：让大多数通信在局部完成，只有必要时才跨越更高层。

讲者在此结束了硬件部分的讨论。接下来是这一讲最容易被忽视、也最容易让程序出错的部分：内存一致性模型。

13.7 内存一致性模型：一致性没解决的那一半

讲者在最后转向一个与前面完全不同的问题，他明确说这是本讲对程序员最重要的部分之一。

缓存一致性不等于程序直觉

前面所有的讨论都在保证一件事：对同一个地址，所有处理器看到一致的顺序。但这并不保证不同地址之间的操作顺序符合程序员的直觉。

考虑两个线程：线程 A 写数据，然后置一个标志位表示”数据已就绪”；线程 B 循环等待标志位，看到后就读取数据。

从程序员角度看这个模式天经地义。但在允许重排的硬件上，A 的两条写可能在别人看来是反序的——标志位先可见、数据后可见。于是 B 看到标志却读到旧数据。

[312] 视频画面时间区间：00:55:30–00:55:45。若为裁剪图，时间区间仍对应原视频画面。

一致性的作用域是单一地址，不是跨地址的顺序

缓存一致性只保证同一地址上的写对所有观察者有序。至于”先写 X 再写 Y”这个顺序是否对别人可见，属于内存一致性模型的范畴。前者是所有共享内存机器的必备，后者因架构而异，可以从严格顺序一路放宽到几乎不保证。

为什么会允许重排

允许重排是为了性能。顺序执行所有内存操作意味着每一次访存都要等上一次完成，这对流水线与多级缓存都是巨大浪费。

放宽模型让硬件可以把写放入缓冲、可以先执行后续不相关的读、可以让不同位置的写并行完成。代价是把”保持顺序”的责任部分转移给了程序员。

此外还有第二个重排来源：编译器。即使硬件严格保序，编译器也可能为了优化而调换两条不相关的访存语句。因此单靠硬件保序不足以让直觉成立。

双重检查锁一类的模式在弱模型上会失效

这类模式依赖”先构造对象再发布引用”的顺序，而顺序可能被硬件或编译器破坏。正确做法是使用语言提供的内存序原语：写侧用释放语义发布，读侧用获取语义接收，或在初始化时使用原子操作加栅栏。只加锁访问一部分字段而让另一部分裸露，是典型误用。

怎么写出正确的代码

讲者给出的方向是原则性的而非具体的原语清单：不要依赖自己没验证过的顺序假设。

实用规则有两条。第一，跨线程共享的数据要么全部通过同步原语访问，要么用带有明确内存序语义的原子操作。第二，任何”先准备数据再发出信号”的模式，都必须用释放与获取语义把两侧配对起来，让硬件和编译器都清楚这个顺序不能被打破。

顺序的建立需要两侧配合

只在一侧加栅栏不足以建立跨线程的先后关系：发布方需要用释放语义确保之前的写先可见，接收方需要用获取语义确保之后的读不会提前。两侧缺一，顺序就可能被破坏。这与锁的配对使用是同一个道理。

布局是最后一项可调的手段

讲者在结尾提到，当人们真正开始调优程序时，会去看数据布局、访问模式这些条件，并可以通过各种提示帮助编译器与硬件。

他同时提醒这不总是可靠，也不容易做到。这一点与前面的伪共享、分块形状、目录共享分布互相呼应：性能的决定因素往往在数据怎么摆，而不在算法怎么写。

本章小结

缓存一致性保证单一地址的顺序，内存一致性模型才决定跨地址的顺序。允许重排是为性能付出的代价，而重排可能来自硬件也可能来自编译器。因此跨线程的通信必须借助带序语义的原语配对使用，不能依赖”写的先后看起来应该如此”。

13.8 总结与延伸

这一讲补上了第 11 讲没讲完的一块，然后把话题推向协议实现与程序语义两侧。

伪共享是最容易被忽视的一种人为通信：数据落在同一缓存行、被不同核使用，逻辑上毫无冲突的代码会在协议层面互相作废，实测可以差两倍以上。对策是填充或改变布局，让共享只发生在边界。这也解释了第 8 讲分块布局选择的动因。

监听协议的瓶颈是广播：流量随参与者增长，超过约半打缓存后互连成为中心瓶颈。层次化监听把局部流量吸收在低层，但树根仍可能成为瓶颈。目录用位向量记录每行的持有者，把通信变成点对点；实测显示八成到九成的写只涉及零到一份其他副本，这正是目录收益的来源。代价是每行 P 位的存储，因此有有限指针加广播回退、稀疏目录等压缩方案，Barnes-Hut 这类树结构程序则是共享密集的反例。真实系统在此基础上加了转发介入、集中目录、跨插槽目录与片上环网，共同点是层次化。

最后，缓存一致性只保证单一地址的顺序；跨地址的顺序属于内存一致性模型，可以被放宽以换取性能，而重排可能来自硬件也可能来自编译器。因此跨线程的通信必须靠带序语义的原语配对完成。

本讲义在讲者材料之上做的压缩，有四点需要与讲者原话区分：

- 伪共享与真正的共享是两个不同的性能问题。前者因为同一缓存行而互相干扰，后者因为访问同一变量而需要同步。前者的解法是改布局（不改语义），后者的解法是改协议（加锁或原子操作）。混用两种解法不会有效。
- 目录的价值来自分布的形状。若共享者数均匀分布，全位向量与有限指针的差距有限；正因为现实中严重偏斜，压缩方案才有意义。评估任何”优化常见情形”的技术，都要先确认分布确实偏斜。
- 溢出与稀疏化都是把硬件资源压力转化为性能损失。它们不改变正确性，只改变退化方式。理解这一点，才能在容量与性能之间做出有依据的取舍。
- 一致性模型的责任划分是被刻意设计的。硬件放宽顺序是为了性能，代价是程序员必须显式表达依赖。因此”我的代码在单核上是对的”从来不构成它在多核上正确的理由。

这堂课的边界在于它仍是导论：讲者多次表示真实协议细节极其复杂，内存模型的完整规则需要专门讨论。因此本文给出的是机制与陷阱的地图，不含可照抄的协议实现。

两个可以直接从本讲数据提出的下一步问题：其一，目录的存储开销与共享者上限之间存在直接的权衡，那么在什么工作负载下应当选择全位向量而非有限指针？其二，内存一致性模型允许的重排范围因架构而异，若要写一份既正确又能在多种架构上保持性能的代码，应当采用最严格的模型还是最弱的模型，判据是什么？

Lecture 14 Snooping-Based Multi-Processor Implementation

视频时长：1 小时 17 分 7 秒 来源：https://www.bilibili.com/video/BV1Wp4y1J78s/

14.1 从状态图到硬件

前两讲讲了一致性协议：先是基于总线（集中式）的，然后是目录式的。这一讲深入总线式的实现细节，目的是让大家体会真正把它做出来时的复杂性。

为什么值得单独讲

讲者的开场很坦诚：他认识工业界以此为生的人，这些协议确实极其复杂、极难做对。

而”如何验证这类协议”本身就是一个有趣的研究方向，而不是只靠跑一堆模拟。他提到这是形式验证工具取得相当成功的一个领域，部分原因在于用别的方法根本做不到。

协议验证为何必须形式化

一致性协议的出错场景往往需要特定的消息交错才触发，随机模拟很难覆盖。而形式验证把「所有可达状态」作为检查对象，因此能证明不存在违反不变量的交错。这与测试的关系类似「证明」与「抽样」的区别。

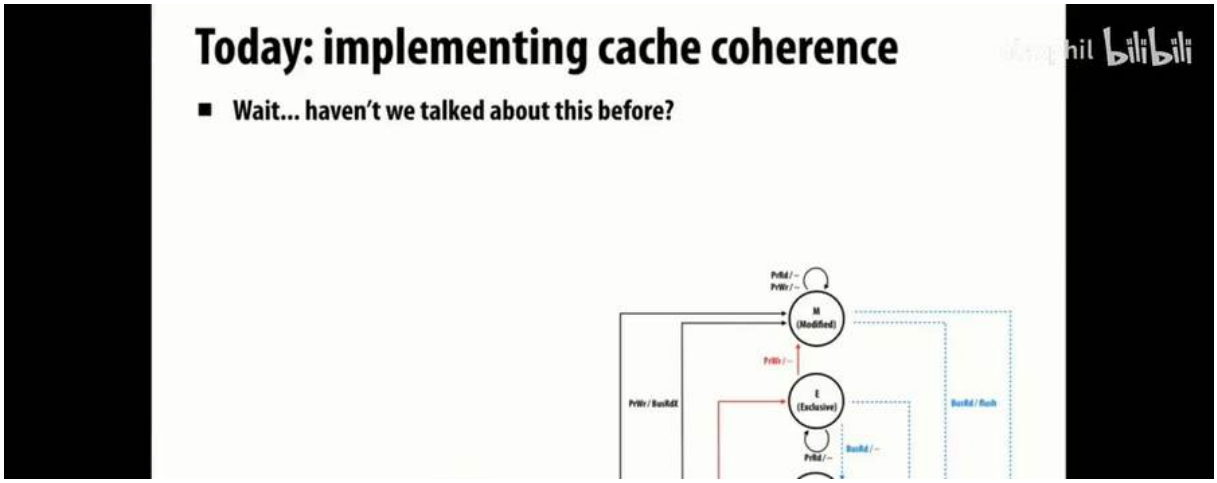

图 313: 今日主题：实现缓存一致性，状态图之外的细节[313]

[313] 视频画面时间区间：00:02:30–00:02:45。若为裁剪图，时间区间仍对应原视频画面。

图 314: 今日概念的范围远超硬件实现本身[314]

这是一类普遍问题

讲者强调这些思想不只属于缓存协议。这是你走向社会、编写系统时都会遇到的一般问题：只要系统涉及并发、有大量事情通过不同主体的独立行动同时发生，就有很多地方会出错。

因此需要跟踪它们，并且有一套系统化的方法。

状态图描述的是规则，不是实现

状态图能告诉你" 从 M 到 I 需要发什么消息"，但不能告诉你" 这条消息发出去时另一个缓存正在做什么"。实现层面的困难来自并发交错、多级层次、仲裁与超时，这些都不在状态图上。因此协议正确性论证必须针对实现，而不能只针对状态机。

本章小结

这一讲深入总线式一致性的实现细节。工业界认为这类协议极其复杂、极难做对，形式验证在此领域相当成功。这些思想不限于缓存协议：任何有并发、多个独立主体的系统都有大量出错方式，需要系统化方法。

下一章先回顾协议状态，再看真实机器的多级缓存。

[314] 视频画面时间区间：00:03:00–00:03:15。若为裁剪图，时间区间仍对应原视频画面。

14.2 回顾：MESI 与多级缓存的现实

状态的含义

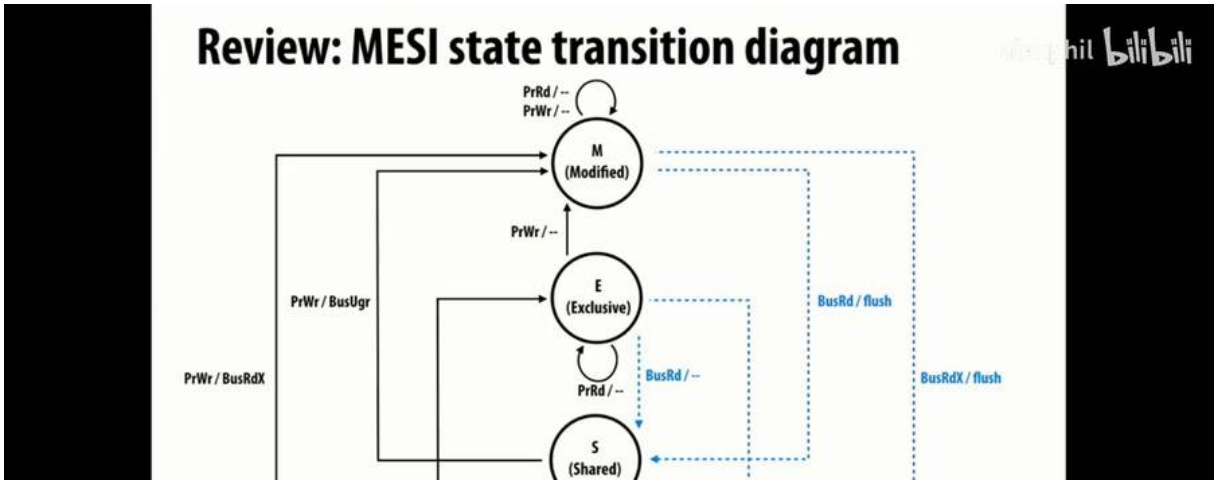

图 315: 回顾：MESI 状态转移图[315]

讲者回顾各状态的含义。独占态表示持有副本且是干净的，同时知道自己是这份数据的唯一持有者。已修改态表示脏副本——被本地缓存写过，与主存中的值不同。

因此若有人需要这份数据，我们可能得做点什么。

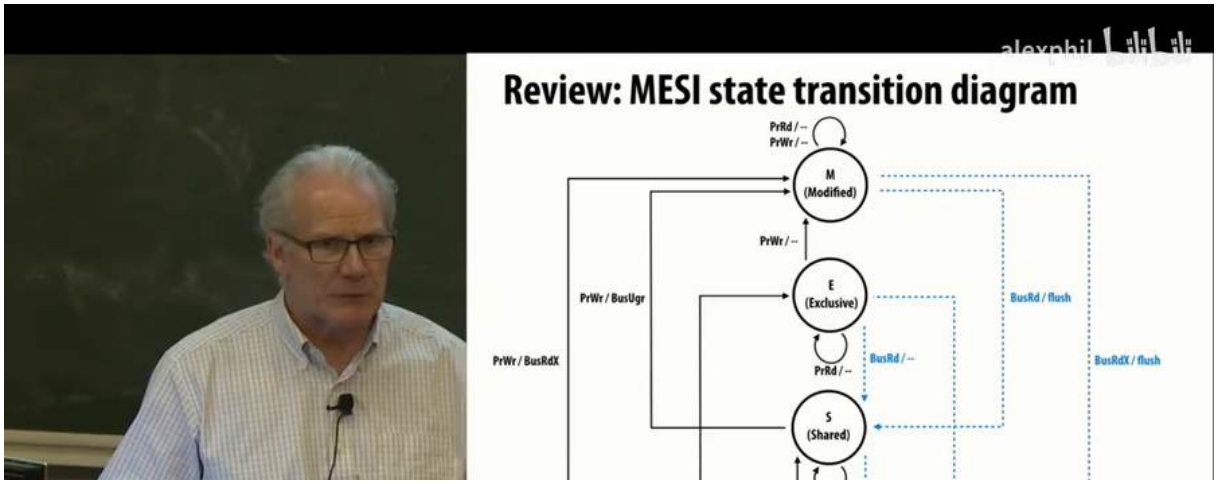

图 316: MESI 状态转移图续：黑线为本地处理器发起，蓝线为系统其余部分发起[316]

讲者解释图中颜色的约定：黑色表示代表本地处理器发生的事务——处理器请求对数据的某种读或写访问；蓝色表示代表系统其余部分发生的事务。

缓存控制器就是那个监听一致性流量、并依据这些缓存命令调整本地状态的部件。

[315] 视频画面时间区间：00:03:30–00:03:45。若为裁剪图，时间区间仍对应原视频画面。

[316] 视频画面时间区间：00:04:30–00:04:45。若为裁剪图，时间区间仍对应原视频画面。

多级缓存的现实

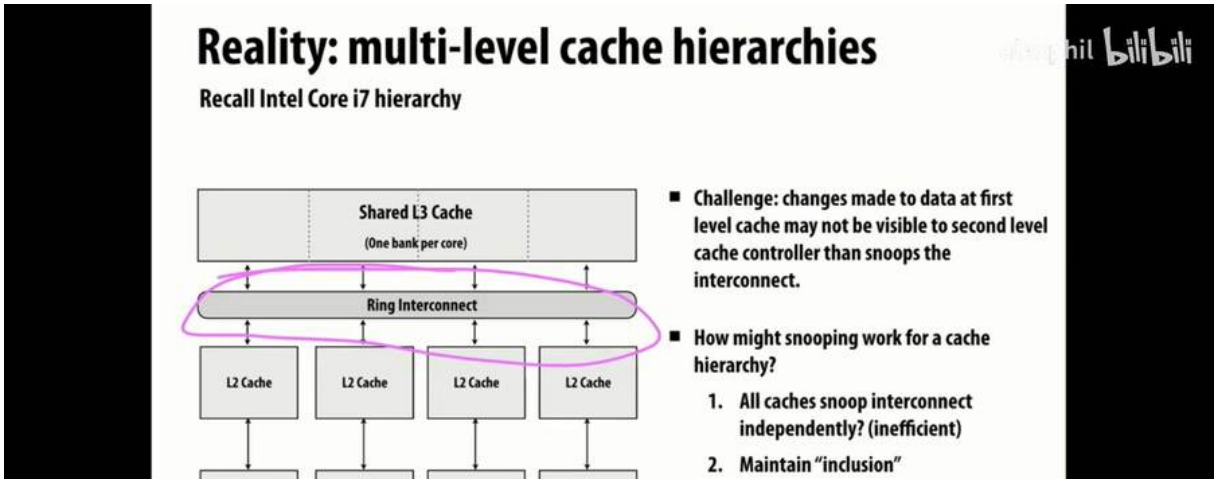

图 317: 现实情形：多级缓存层次，一致性流量发生在互连层[317]

讲者指出真实机器里已经不只是处理器和缓存，而是多级缓存。真正的一致性流量发生在某个互连层次上——它把各个核连到一个共享缓存。

为了讨论方便，他把靠近核的一侧叫缓存，另一侧叫内存。但他提醒：如例子所示，那个”内存”本身可能又是某个更全局状态的缓存。

由此引出一个关键问题：L2 缓存控制器如何能足够了解 L1 里有什么，从而正确地监听并响应总线上的流量?

本章小结

MESI 的四态语义是：独占是干净且唯一，已修改是脏的。图中黑线为本地处理器发起，蓝线为系统其余部分发起，缓存控制器据此调整本地状态。真实机器是多级缓存，一致性流量发生在互连层，而所谓内存本身可能又是更上层的缓存。由此引出核心问题：L2 如何知道 L1 的内容。

14.3 包含属性：L2 如何知道 L1 有什么

包含属性的定义

讲者给出答案：假设存在所谓的包含属性——内层缓存中的任何内容都是外层缓存的子集。

这样 L2 基本上就知道 L1 的全部内容。而且它不仅知道它们在那里，还知道它们的状态：是干净的还是脏的。

[317] 视频画面时间区间：00:05:30–00:05:45。若为裁剪图，时间区间仍对应原视频画面。

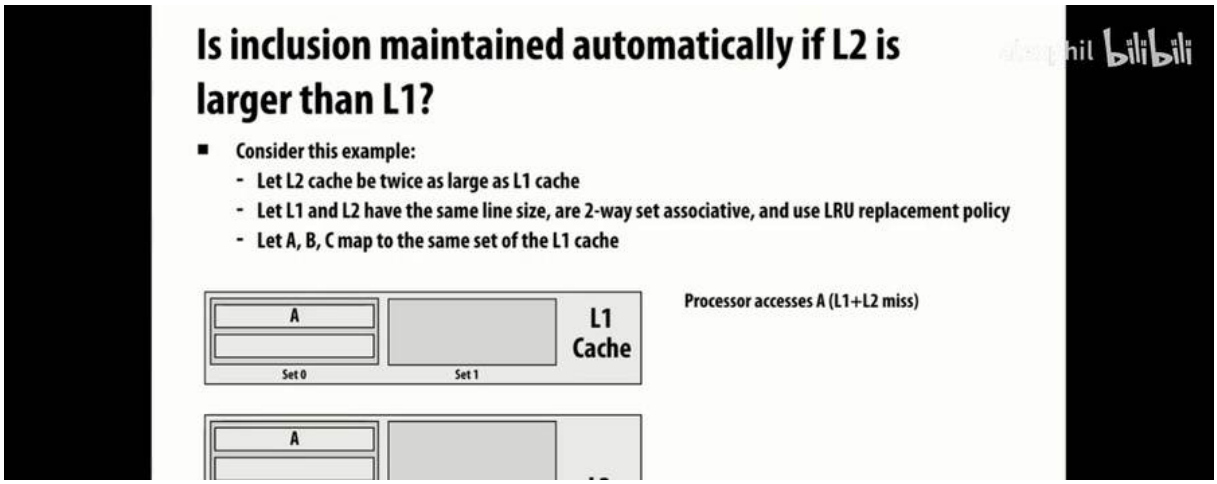

图 318: 若 L2 远大于 L1，包含属性会自动维持吗[318]

$$\mathrm{L1}_{\mathrm{contents}} \subseteq \mathrm{L2}_{\mathrm{contents}}$$

- 内层缓存持有的每一行，外层缓存必须也持有；
- 这样外层才能代替内层响应一致性请求；
- 状态（干净或脏）也必须在外层可见，否则无法判断是否需要取回数据。

有些实现放弃了包含性

讲者提到有些实现并不满足这个属性：它们允许 L1 缓存与 L2 不同步，保留那些已从 L2 淘汰的行。

他坦承自己不知道它们如何实现这套逻辑，但推测那是协议里又一层复杂度。

LRU 不保证包含性

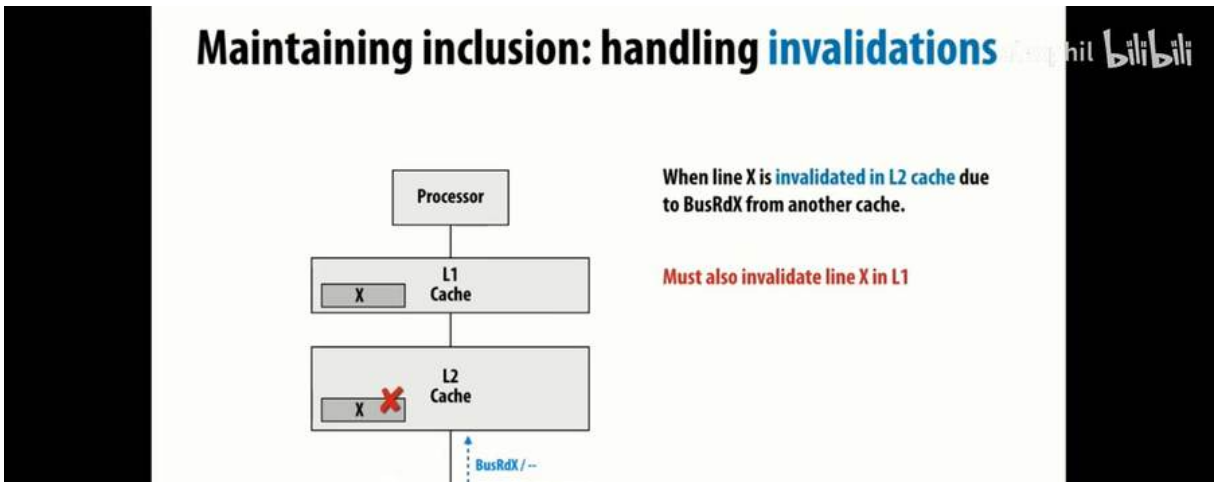

图 319: 维持包含属性：处理失效[319]

[318] 视频画面时间区间：00:08:00–00:08:15。若为裁剪图，时间区间仍对应原视频画面。

讲者举了一个具体反例。若两个缓存都用 LRU 策略，可以设想这样的场景：L1 中有大量活动，而这些活动 L2 看不到，因为 L1 自己处理了它们。

于是某个块被从 L2 中淘汰——尽管它在 L1 中非常活跃。

所以标准 LRU 策略并不能保证包含属性。而且这会是相当常见的情形。

因此必须在两个缓存之间人为强制这个策略，主要做法是：若 L2 决定要淘汰某行，就强制 L1 也淘汰它。

包含性是强加的策略而非自然结果

”L2 比 L1 大” 并不自动保证包含性，因为两个缓存各自独立地按 LRU 淘汰。要让包含性成立，必须在硬件里显式耦合两者的淘汰决策：外层淘汰时通知内层一起淘汰。这份耦合本身就是实现复杂度的来源。

L1 写命中与额外状态位

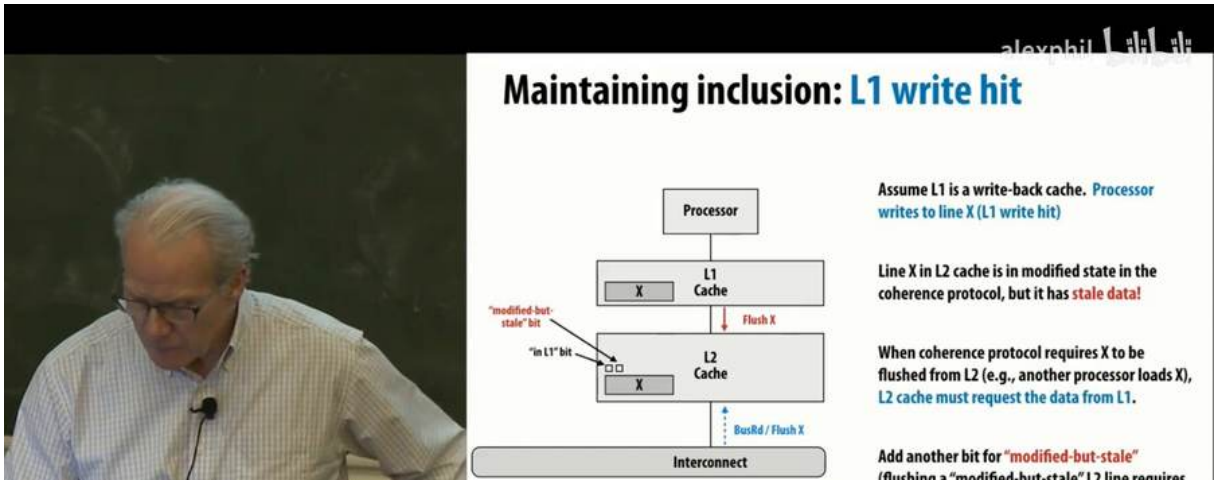

图 320: 维持包含属性：L1 写命中时的处理[320]

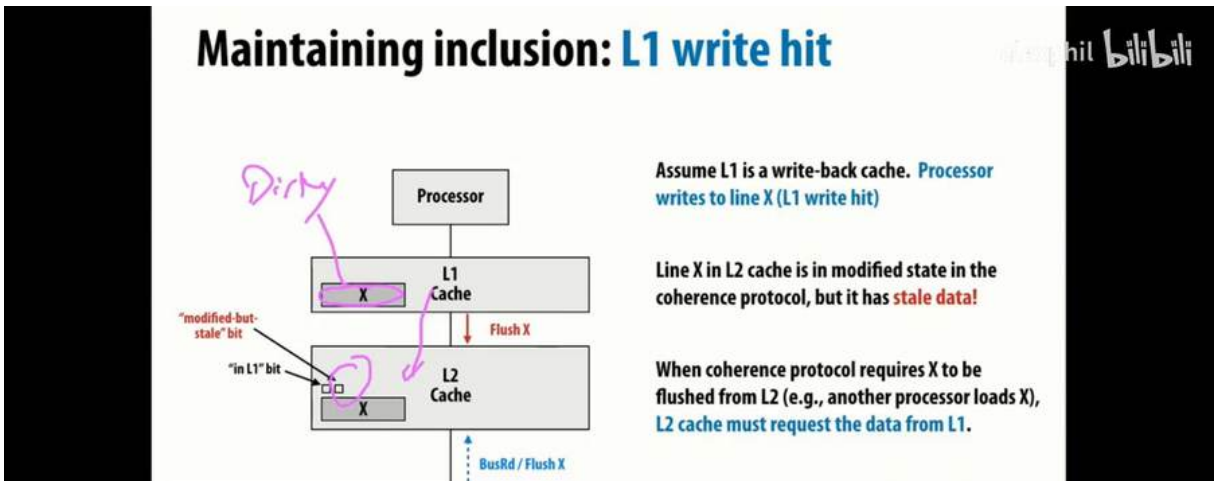

图 321: L1 写命中续：L2 需增加一位表示” 持有但副本不正确”[321]

[319] 视频画面时间区间：00:09:00–00:09:15。若为裁剪图，时间区间仍对应原视频画面。

[320] 视频画面时间区间：00:10:00–00:10:15。若为裁剪图，时间区间仍对应原视频画面。

讲者接着讨论另一种情形：假设 L1 是回写缓存，本地处理器一直在更新这一行，因此它被标记为脏。

他说你不希望每次 L1 写都穿透到 L2，否则就会有写直达缓存的全部问题。所以要让 L1 能持有这份数据并保持脏状态，甚至与 L2 不一致。

于是 L2 缓存里要增加一位额外的状态，表示：这个值确实在缓存中，但我没有它的正确副本。

那么当外部信号说”把你的 x 副本给我”时（比如有人要读），这份信息必须先由 L1 推到 L2，再由 L2 把数据放到总线上。

脏位归属决定了谁负责提供数据

若 L1 持有脏副本而 L2 只有过时值，那么任何外部请求都必须先由 L1 把数据推到 L2 才能应答。这意味着 L1 与 L2 之间会因一致性流量而产生额外通信。协议设计的目标是让这种通信只在必要时发生，否则流量会失控。

本章小结

包含属性要求 L1 内容是 L2 的子集，这样 L2 才知道 L1 有什么及其状态。有些实现放弃它，代价是额外复杂度。标准 LRU 不保证包含性，必须人为耦合淘汰决策：L2 淘汰时强制 L1 一起淘汰。当 L1 持有脏数据时，L2 需要一位表示”持有但副本不正确”，外部请求需由 L1 推到 L2 再上总线。

14.4 三个目标与三种失败

目标

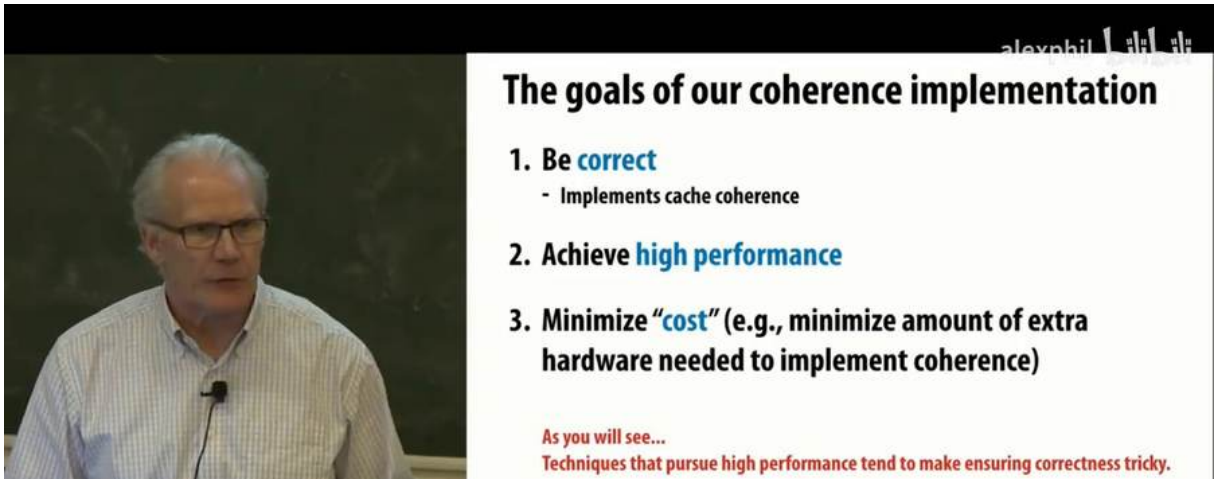

图 322: 一致性实现的目标：正确性与高性能[322]

讲者把目标列出来：第一是实现缓存一致性，也就是正确；第二是获得高性能。

他说这就是全局目标：我们必须正确，这是压倒性的要求，但同时也需要性能。而性能取决于流量有多少、以及不必要的 L1 与 L2 通信有多少。

因此原则是：每个缓存应该只在真正必须时才与其他缓存通信，否则就会增加过多流量。

[321] 视频画面时间区间：00:11:00–00:11:15。若为裁剪图，时间区间仍对应原视频画面。

[322] 视频画面时间区间：00:12:30–00:12:45。若为裁剪图，时间区间仍对应原视频画面。

正确性与流量是一对必须同时满足的约束

只追求正确可以退化为全部串行；只追求流量最小可以退化为不保证一致。因此协议设计的实质是在”正确”这个硬约束下最小化流量。任何优化都必须先证明不破坏正确性，这也是为什么形式验证在此领域如此重要。

死锁

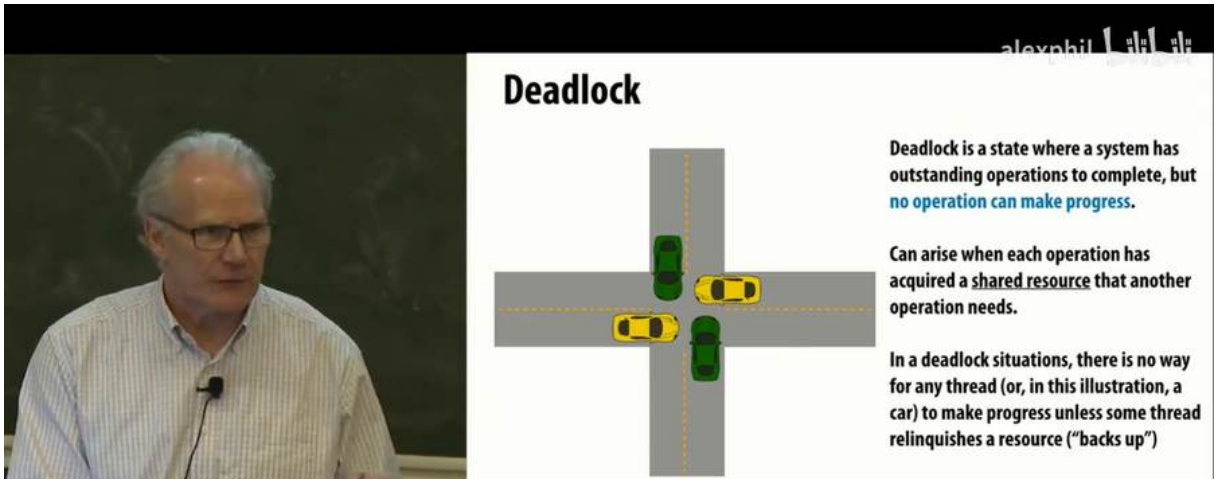

图 323: 死锁：系统有未完成操作，但没有任何操作能推进[323]

讲者给出死锁的定义：系统有未完成的操作，但没有任何操作能向前推进。

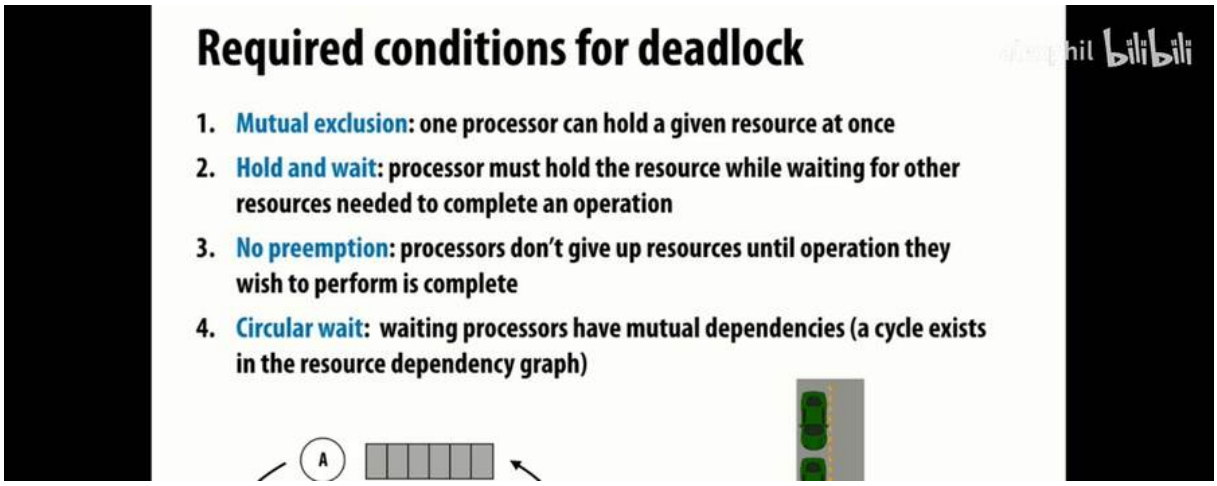

图 324: 死锁的必要条件：互斥、持有并等待、不可剥夺、循环等待[324]

他列出了死锁的必要条件，这是操作系统课上的经典结论：互斥（一个处理器能持有某资源）、持有并等待、不可剥夺、循环等待。四个条件同时成立才会死锁，因此打破任何一个就能避免。

[323] 视频画面时间区间：00:14:30–00:14:45。若为裁剪图，时间区间仍对应原视频画面。
[324] 视频画面时间区间：00:17:30–00:17:45。若为裁剪图，时间区间仍对应原视频画面。

活锁

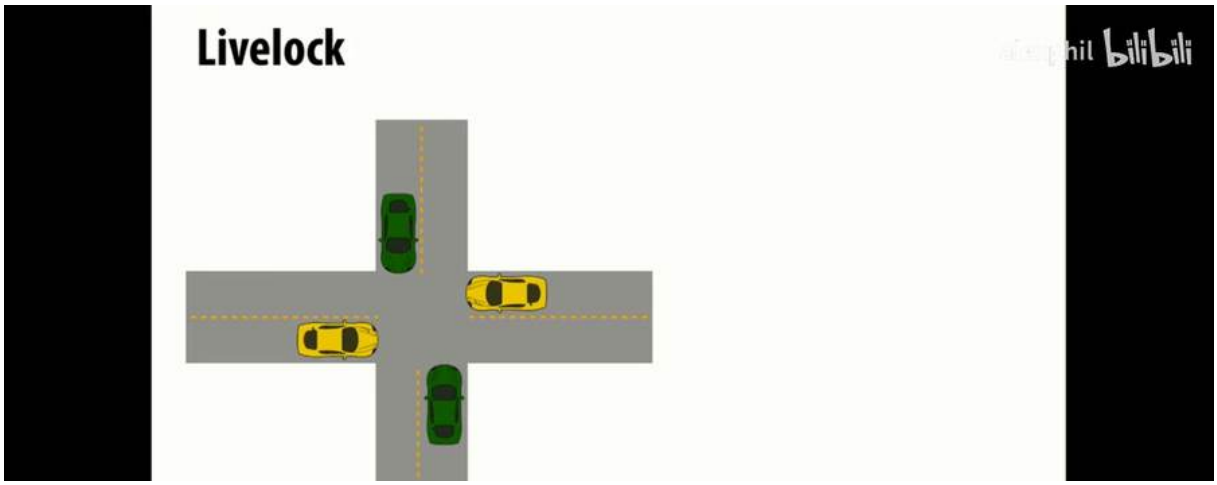

图 325: 活锁：有人在动，但没有任何有效推进[325]

讲者说还有一种较难理解的情况叫活锁。它不是” 什么都没发生”，而是” 发生的事情没有意义”。

他用汽车场景举例：四辆车同时说” 我倒车让你过”，结果没有一辆真正前进。它们同步得太紧，于是来回摆动，谁都无法推进。

死锁与活锁的区别在于” 是否在动”

死锁是所有操作都停住；活锁是操作一直在执行但状态在原地打转。因此监控”CPU 是否繁忙” 无法发现活锁——系统看起来在满负荷工作，实际没有任何进展。诊断活锁需要看” 状态是否有净推进”，而不只是看利用率。

讲者说活锁在现实中较少见，因为事情往往有随机性从而避免它。它可能出现的形式不是永久活锁，而是陷入这种来回摆动、拖慢系统。

指数退避

讲者提到早期以太网发展出的一个有意思的想法，它出现在很多协议里：指数退避。

用汽车场景想象：不是所有车都同时” 嗖嗖嗖” 地动，而是每辆车退让后等待一段随机时间再尝试前进。这样至少有一辆车能在下一轮通过，而不会所有车同时冲上去。

而” 指数” 的意思是：随机延迟应按指数分布逐渐变长：

$$\text{delay}_k = \text{rand}\left(0,\ 2^k \cdot \tau\right)$$

- k：连续冲突的次数；
- τ：基本时间单位；
- 上限随冲突次数指数增长，因此重试时间被逐渐拉开。

这样多次冲突后，各主体的重试时刻会分散到不同时间窗，避免持续同步碰撞。

[325] 视频画面时间区间：00:18:30–00:18:45。若为裁剪图，时间区间仍对应原视频画面。

饥饿

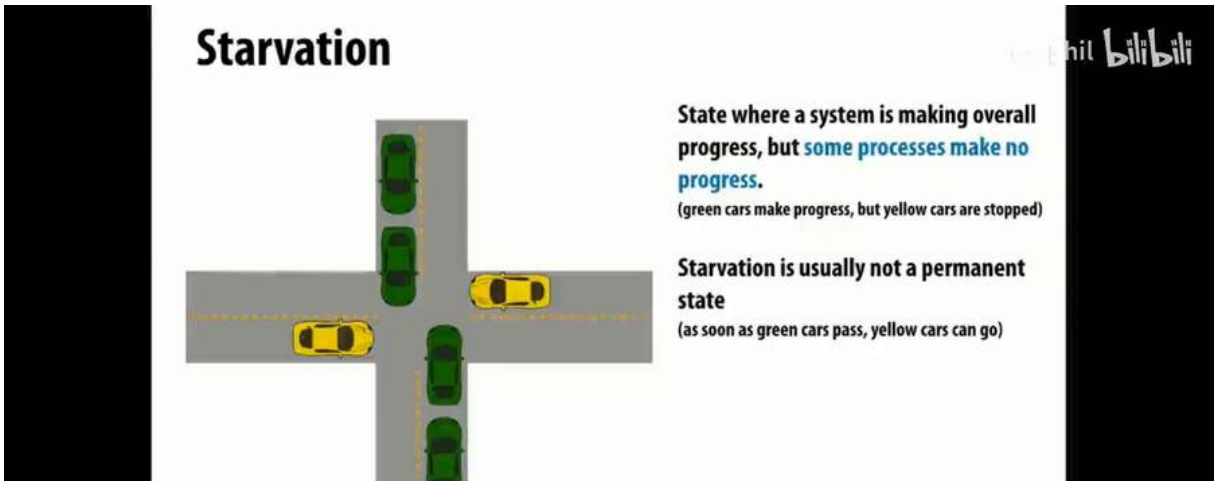

图 326: 饥饿：系统整体在推进，但某些进程毫无进展[326]

讲者区分第三种情况：饥饿。系统整体在取得进展，但某些进程完全没有进展。

他给出具体机制：若有 P 个处理器，而你总是把优先级给编号最小的那个，那么编号大的可能永远轮不到。因此必须建立公平系统——让优先级随机或不断变化，使没有谁会被永远挤出去。

固定优先级是最常见的饥饿来源

按编号、按到达顺序、按固定表项分配优先级的方案，都会在长时间运行后让某些主体长期得不到服务。因此仲裁逻辑必须引入随机化或轮转，代价是平均延迟略高，收益是消除最坏情况的无限等待。

本章小结

目标是正确且高性能，而性能取决于流量；原则是缓存只在必要时通信。三种失败形态是：死锁（全部停住，需四个必要条件同时成立）、活锁（在动但无有效推进，用指数退避缓解）、饥饿（整体推进但个别永远轮不到，需随机或变化的优先级）。

有了这些概念，就可以讨论具体的设计假设。

[326] 视频画面时间区间：00:20:00–00:20:15。若为裁剪图，时间区间仍对应原视频画面。

14.5 简化假设与总线事务

假设

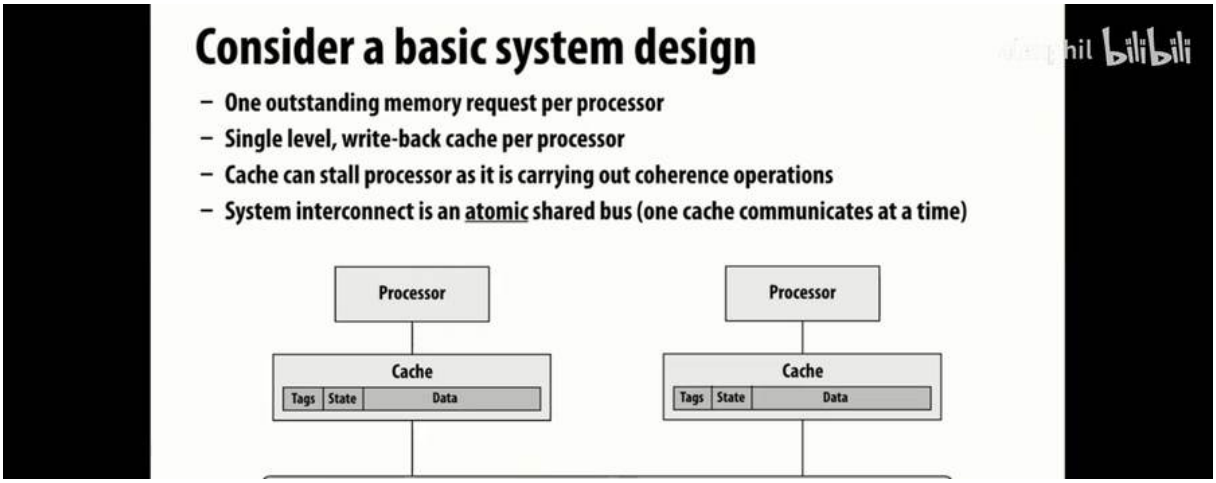

图 327: 基本系统设计：每处理器同时只有一个未完成请求，单级回写缓存[327]

讲者说先把事情简化。假设每个处理器同时只能有一个内存请求，必须完成该事务才允许发出下一个请求。这减少了并发活动的数量与流量规模。

假设只有简单的缓存，而且最初按回写缓存考虑，暂不担心多级缓存。

假设缓存可以回信号给处理器说”等一下，我正在处理一致性流量”。

还假设消息是原子的：总线上一次只有一个消息，它要完成的事务必须原子地完成，中间不插入任何其他活动。

原子性假设是简化，也是限制

原子总线让协议推理只需考虑单事务内部的顺序，不必处理事务交错。但这个假设依赖物理上的共享介质与仲裁机制，一旦改用分组交换互连，原子性就不再成立，协议必须重新设计以处理消息乱序与部分完成。

[327] 视频画面时间区间：00:21:30–00:21:45。若为裁剪图，时间区间仍对应原视频画面。

总线事务的步骤

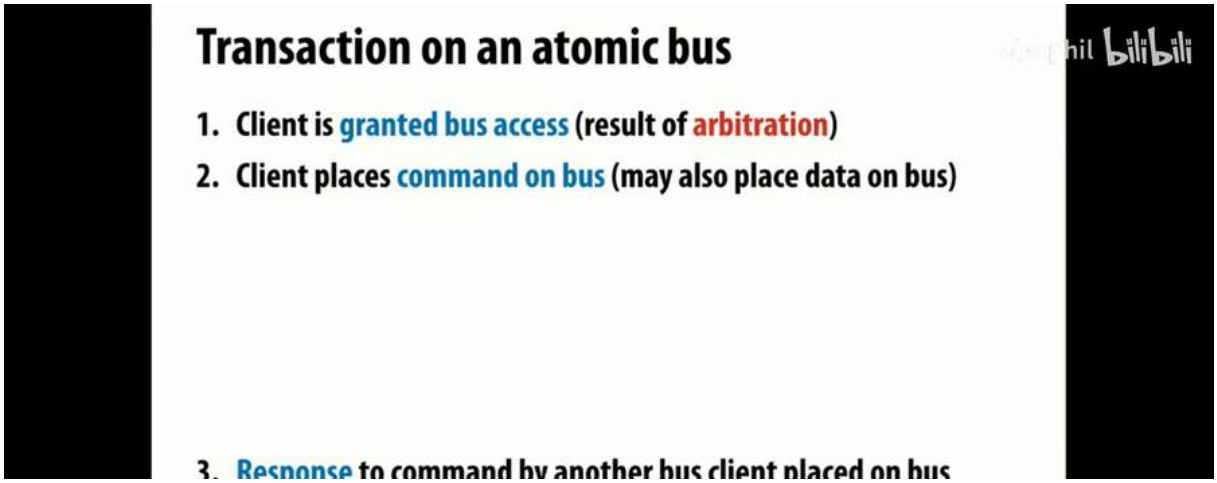

图 328: 原子总线上的事务：获得授权、广播请求、收集回应[328]

讲者描述一次事务：客户端先通过仲裁获得总线访问权，然后把请求广播出去，各缓存回应。

原子总线把并发问题降维成串行问题

若总线保证一次只有一个事务且不被插入，那么协议的正确性论证就只需考虑”单事务内各缓存的回应顺序”，而不必考虑事务之间的交错。这正是早期总线机器的优势，也是可扩展性的代价来源：原子性依赖物理上的共享介质。

本章小结

简化假设是：每处理器一个未完成请求、单级回写缓存、缓存可让处理器停顿、总线消息原子且不可插入。总线事务的步骤是获得授权、广播、收集回应。原子总线把并发问题降维成串行问题，这也是它难以扩展的原因。

[328] 视频画面时间区间：00:22:30–00:22:45。若为裁剪图，时间区间仍对应原视频画面。

14.6 多处理器控制器与 Snoop Pending

单处理器逻辑与多处理的差别

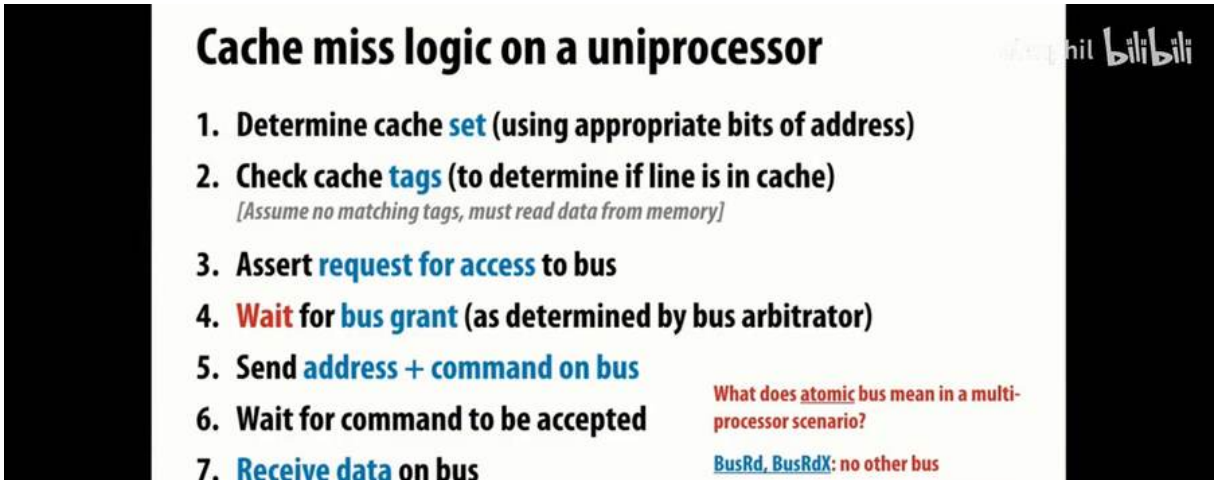

图 329: 单处理器上的缓存缺失逻辑：确定组、检查标签、选择牺牲行[329]

讲者先回顾单处理器上的缺失处理步骤：用地址的相应位确定缓存组，检查标签，若缺失则选择牺牲行。

核心挑战

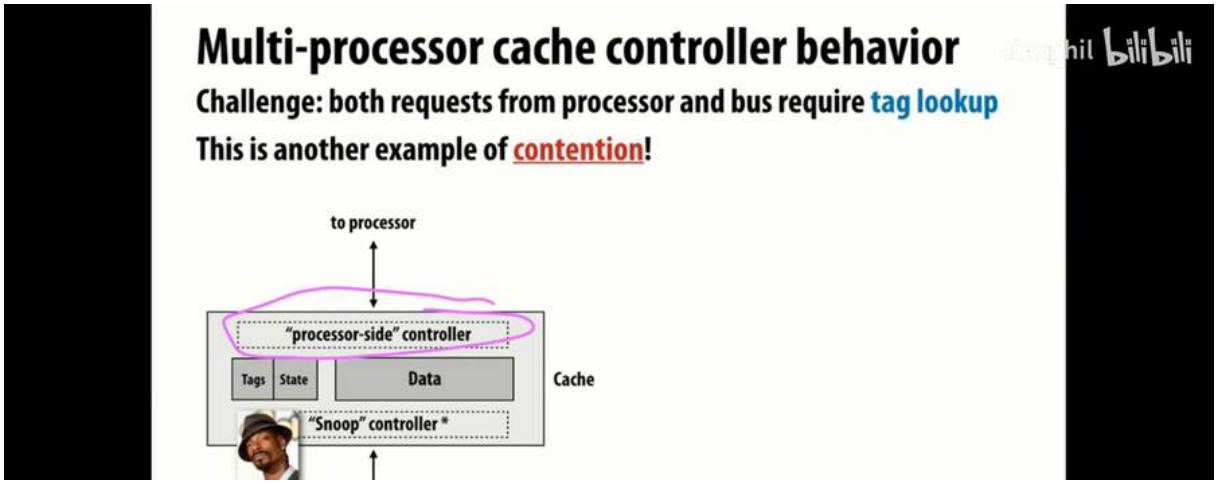

图 330: 多处理器控制器的挑战：处理器请求与总线请求同时需要标签查找[330]

[329] 视频画面时间区间：00:23:00–00:23:15。若为裁剪图，时间区间仍对应原视频画面。
[330] 视频画面时间区间：00:25:00–00:25:15。若为裁剪图，时间区间仍对应原视频画面。

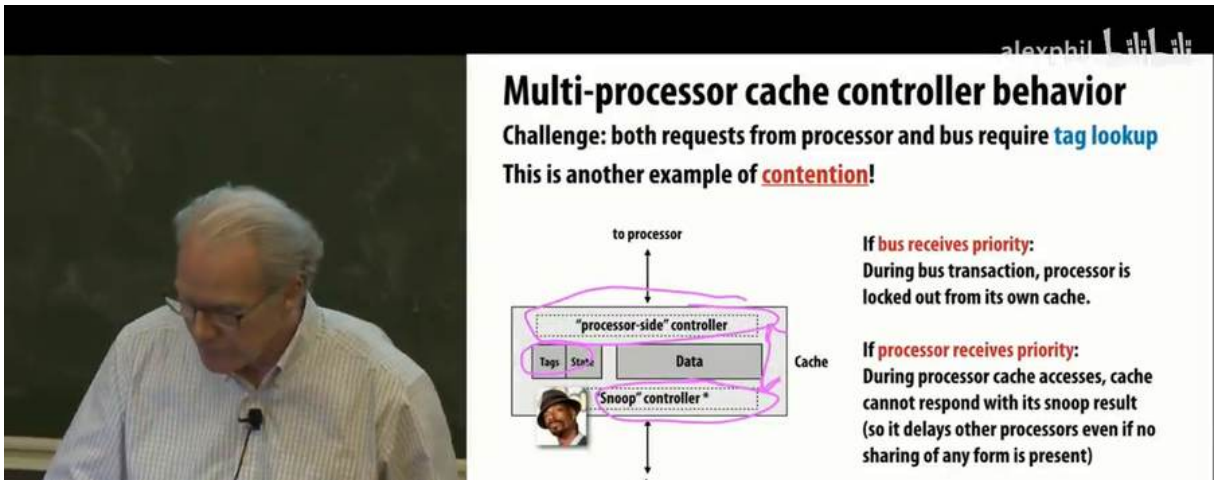

图 331：挑战续：如何在不冲突的情况下保证事务完整[331]

讲者点出多处理器的核心挑战：来自处理器的请求与来自总线的请求同时需要标签查找。而标签存储器是共享的硬件资源，不能同时服务两者。

一致性的实现代价一半在”资源争用”而非”状态机”

状态机只描述该做什么，而实现必须解决”两件事同时要做时谁先用硬件”。标签查找、缓存读写端口、总线驱动都是单份资源，因此控制器必须排队或复制这些资源。这类冲突在状态图上完全不可见，却是实现复杂度的主要来源。

Snoop Pending

讲者解释了硬件上的一个具体机制。总线事务一开始，所有缓存会暂时把某条线拉高，或发出信号说”等等，给我机会回应”。

然后当它们各自把消息发出后——无论是共享、脏，还是都不是——就把信号拉低。

一旦这条 pending 线重新变低，下一个动作才能发生。

因此用总线上的线与（OR）逻辑，让所有缓存能集体地表达信息，同时也能跟踪何时可以进入下一步。

Snoop Pending 是硬件级的会合点

它用一根线与线实现了一个分布式屏障：任何一个缓存拉高，全系统就等它。这让”所有监听者都已回应”成为一个可观测的硬件信号，而不需要中央控制器逐个查询。代价是它把最慢者的延迟暴露为全局延迟，与第 9 讲的掉队者问题同构。

本章小结

单处理器的缺失处理是确定组、查标签、选牺牲行。多处理器的挑战在于处理器请求与总线请求同时需要标签查找，这是状态图上不可见的资源争用。Snoop Pending 用线与逻辑实现分布式会合：任何缓存拉高则全系统等待，全部拉低才进入下一步。

[331] 视频画面时间区间：00:26:00–00:26:15。若为裁剪图，时间区间仍对应原视频画面。

14.7 数据从哪来：事务分裂

两种数据来源

讲者指出一个有趣的问题：数据可能来自内存，也可能来自另一个缓存。而内存的响应比缓存慢。

于是问题变成：是否应该在还不确定内存里是否有正确数据时就开始响应?

这是一个设计决策

讲者说这是个有意思的设计决策，因为它会产生额外的内存访问。

$$T_{\text{split}} = T_{\text{probe}} + T_{\text{data}}$$

- T_{probe}：先探测是否有缓存持有该数据所需的时间；
- T_{data}：真正取回数据的时间；
- 若等待探测结果再取，延迟是两者之和；
- 若并行启动，命中时更快，未命中时浪费一次内存访问。

提前启动响应是用带宽换延迟

在不确定数据来源时就启动内存响应，可以让命中缓存的常见情形更快；但若最终发现数据在缓存里，那次内存访问就浪费了。因此这个决策取决于” 缓存命中的比例” 与” 内存带宽的充裕程度”，而不是单纯的延迟优化。

本章小结

数据可能来自内存或另一缓存，而内存更慢。是否在不确定来源时就启动响应是个设计决策：并行启动降低常见情形延迟，代价是可能浪费内存访问。这个权衡取决于命中比例与带宽充裕度。

讲者在结尾回到一个观察：这类协议让人更体会私有缓存的价值——它们不必为单次访问付出总线流量。

14.8 总结与延伸

讲者在结尾说，这一讲让人更加体会到缓存的价值：尤其是那些非共享的私有缓存，它们不必为了取一个位置而做那么多总线流量。

把全讲串起来，这是一条从状态图走向实现的路径。状态图只能给出大致概念，实现层面的困难来自并发交错、多级层次、仲裁与超时。工业界认为这类协议极其复杂、极难做对，形式验证在此领域相当成功。真实机器是多级缓存，一致性流量发生在互连层，而所谓内存本身可能又是上层缓存。包含属性（L1 内容是 L2 的子集）让 L2 能代替 L1 响应请求，但标准 LRU 并不保证它——必须显式耦合两者的淘汰决策；当 L1 持有脏数据时，L2 需要额外一位表示” 持有但副本不正确”。目标是正确且高性能，而性能取决于流量，因此原则是只在必要时通信。三种失败形态是死锁、活锁与饥饿，分别用打破必要条件、指数退避、随机化优先级来应对。简化假设包括每处理器一个未完成请求与原子总线事务，后者把并发问题降维成串行问题，也是它难以扩展的原因。多处理器的核心挑战是处理器请求与总线请求同时需要标签查找——这类资源争用在状态图上不可见。Snoop

Pending 用线与逻辑实现分布式会合。最后，数据可能来自内存或缓存，是否提前启动响应是一个用带宽换延迟的决策。

本讲义在讲者材料之上做的压缩，有四点需要与讲者原话区分：

- 协议正确性必须针对实现论证，而不是针对状态机。状态机假设消息瞬时且无并发，而真实硬件有仲裁延迟、标签争用与超时。因此” 状态图看起来对” 不构成正确性证据，这正是形式验证在此领域必要的原因。
- 包含性是强加的策略，不是缓存大小的自然结果。两个独立按 LRU 淘汰的缓存不保证包含关系，必须显式耦合。理解这一点，才能理解为什么多级一致性协议比单级复杂得多。
- 死锁、活锁、饥饿是三种不同故障，诊断方法也不同。死锁看” 是否停住”，活锁看” 状态是否有净推进”，饥饿看” 个体是否长期得不到服务”。只看 CPU 利用率会把活锁误判为正常工作。
- 事务分裂的取舍取决于负载特征而非纯延迟。提前启动响应在命中率高时有利，在带宽紧张时有害。因此同一协议在不同工作负载下的最优策略可能不同。

这堂课的边界在于它是实现导论：讲者明确说真实协议极其复杂，本讲只给出几类典型问题的形态与对策，不含完整协议规范。

两个可以直接从本讲数据提出的下一步问题：其一，Snoop Pending 把最慢监听者的延迟暴露为全局延迟，那么在监听者数量增加时，这个会合机制如何成为可扩展性的瓶颈？其二，若放弃包含属性以允许 L1 保留已从 L2 淘汰的行，协议需要增加哪些机制才能正确响应总线请求？

Lecture 15 OpenMP

视频时长：1 小时 14 分 27 秒 来源：`https://www.bilibili.com/video/BV1Qp4y1J72A/`

15.1 OpenMP：用编译制导表达并行

讲者开场就说这是他最喜欢的东西之一，理由很实际：OpenMP 内建在所有 GCC 里，完全通过 pragma 支持，用起来非常自然。

一行 pragma 就能并行化循环

示例里写着 `#pragma omp parallel for`，讲者说这基本上就是并行化一个 for 循环所需的全部。

当然，他立刻补了一句关键的限定：这个循环是否应该被并行化，是另一个问题。我们要表达的是”允许这样做”，这是给编译器的注解。

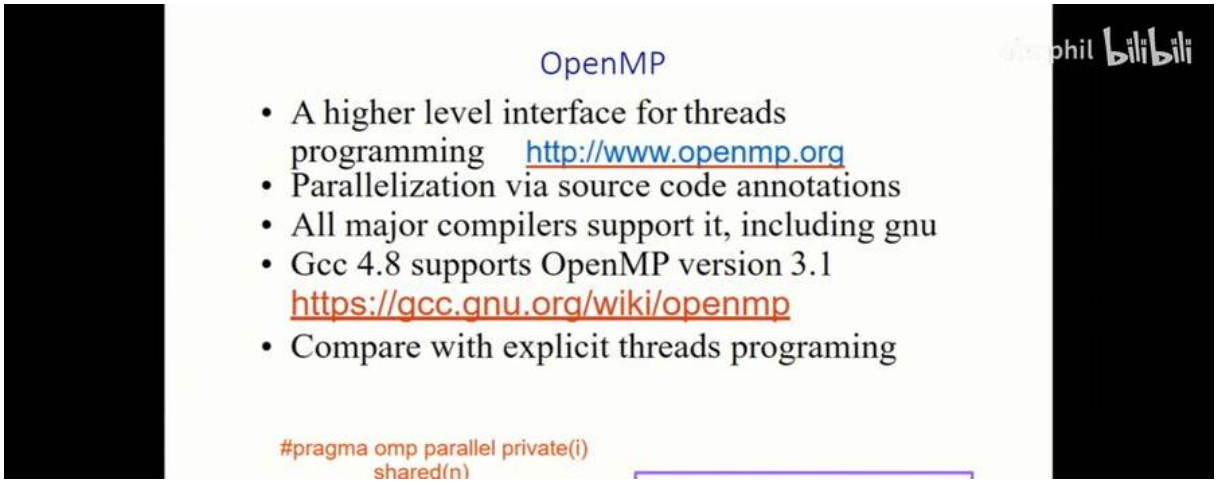

图 332：OpenMP：面向线程的高层接口[332]

pragma 是许可而非验证

`#pragma omp for` 的含义是”我确认这些迭代可以并行”，编译器不会（也不总能）验证这个断言。因此它与第四节讨论的 forEach 属于同一类：声明控制并行，不做依赖分析。加 pragma 前必须先自己确认迭代独立。

fork-join 模型

OpenMP 采用 fork-join 模型：启动一批工作线程，它们各自独立干活，到达同步点（屏障）后重新汇合，然后再启动、再汇合。

并行区内的并行度由线程数决定：

$$P_{\text{region}} = N_{\text{threads}}$$

[332] 视频画面时间区间：00:01:00–00:01:15。若为裁剪图，时间区间仍对应原视频画面。

- N_{threads}：当前并行区中的线程数，通常等于可用核数；
- P_{region}：该区段可达到的并行度；
- 串行区段的并行度为 1，因此整体加速受 Amdahl 定律约束。

讲者强调的那条贯穿全程的蓝色线程，就是并行度为 1 时仍在运行的主线程。

朴素地理解，这意味着反复创建和回收大量线程。但讲者说编译器在这方面做得相当好，会尽量减少线程创建。

图 333: OpenMP 的 fork-join 模型：程序以单线程开始，进入并行区后派生线程[333]

主线程的存在影响屏障语义

主线程在并行区内也承担一份工作，因此屏障要求它与其他线程一样到达。这意味着若主线程在并行区内被安排去处理某段只有它能做的代码，其余线程会在屏障处等它。理解主线程贯穿始终，才能理解 OpenMP 中屏障与任务划分的边界。

讲者展示了一张并行度示意图，其中并行区之间有串行执行的区段，并特意指出有一条蓝色的线程贯穿到底。他让学生猜它是什么——答案是主线程：程序启动时的那个线程从未消失，整个过程都在。

[333] 视频画面时间区间：00:02:00–00:02:15。若为裁剪图，时间区间仍对应原视频画面。

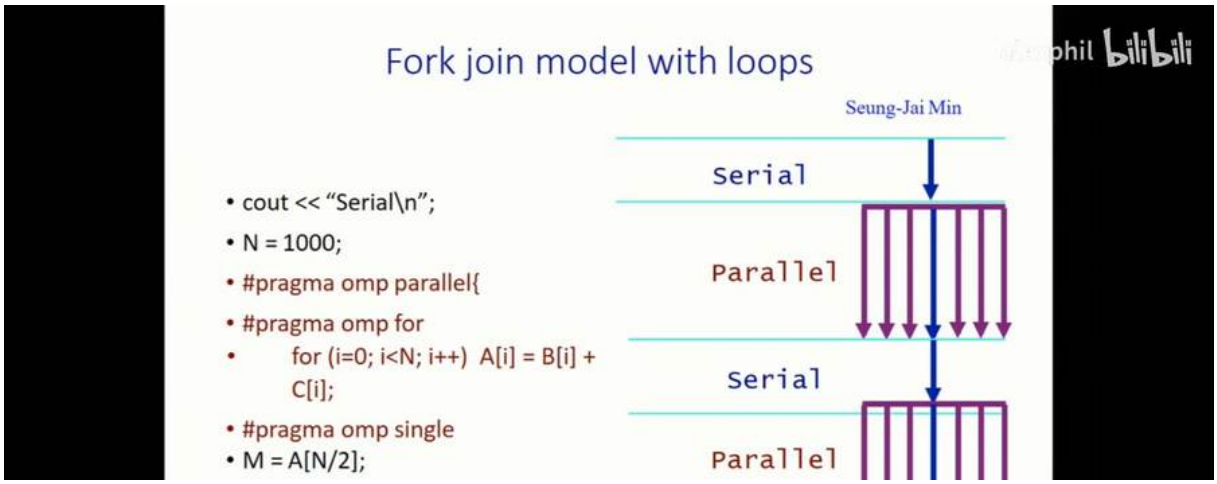

图 334: 循环上的 fork-join：并行区之间的串行段[334]

本章小结

OpenMP 通过 pragma 在高层表达并行，采用 fork-join 模型，主线程贯穿始终，编译器负责优化线程创建。一行 pragma 足以并行化循环，但它只是许可，不是正确性验证。

15.2 循环并行化时编译器替你做了什么

四件事

讲者列出翻译器自动完成的工作：生成各线程的局部循环边界，也就是替你划分；插入所需的屏障；处理循环次数不能被线程数整除时的余数；划分方式可以是静态或动态。

剩下的必须由程序员指定：哪些变量私有、哪些共享。

划分时每个线程分到的迭代数并不总能整除：

$$\text{chunk} = \left\lceil \frac{N}{T} \right\rceil, \qquad \text{last} = N - (T-1)\cdot \text{chunk}$$

- N：循环迭代总数；
- T：线程数；
- chunk：每个线程分到的迭代数，向上取整；
- last：最后一个线程实际处理的迭代数，可能少于 chunk。

编译器会自动生成这些边界，但最后一个线程的负载因此偏轻。

[334] 视频画面时间区间：00:02:30–00:02:45。若为裁剪图，时间区间仍对应原视频画面。

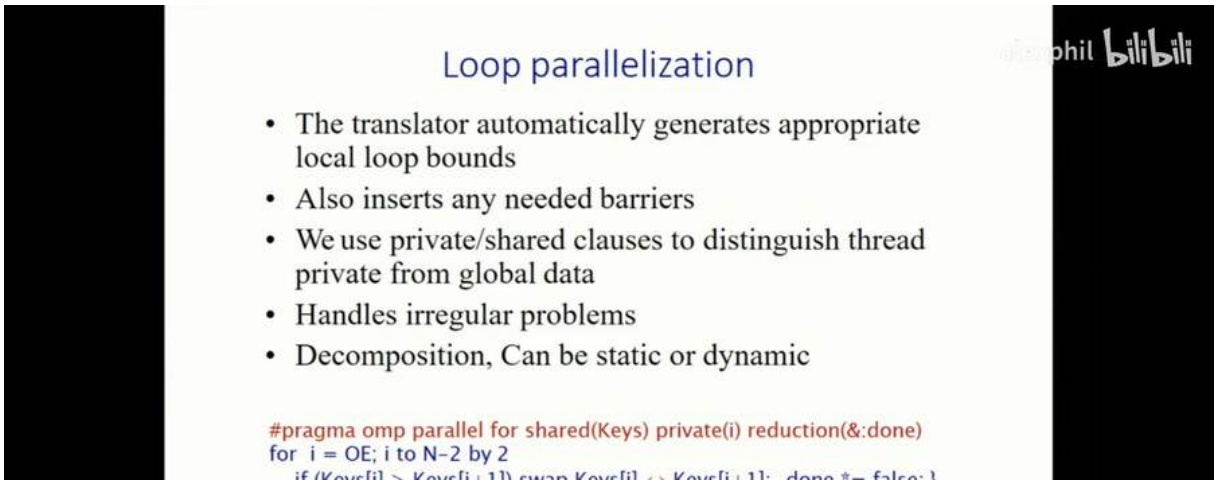

图 335: 循环并行化：翻译器自动生成局部边界、插入屏障、处理余数[335]

两种等价的写法

讲者指出 `omp for` 与 `omp parallel for` 表达的是同一件事，编译器与翻译器得到的信息相同。

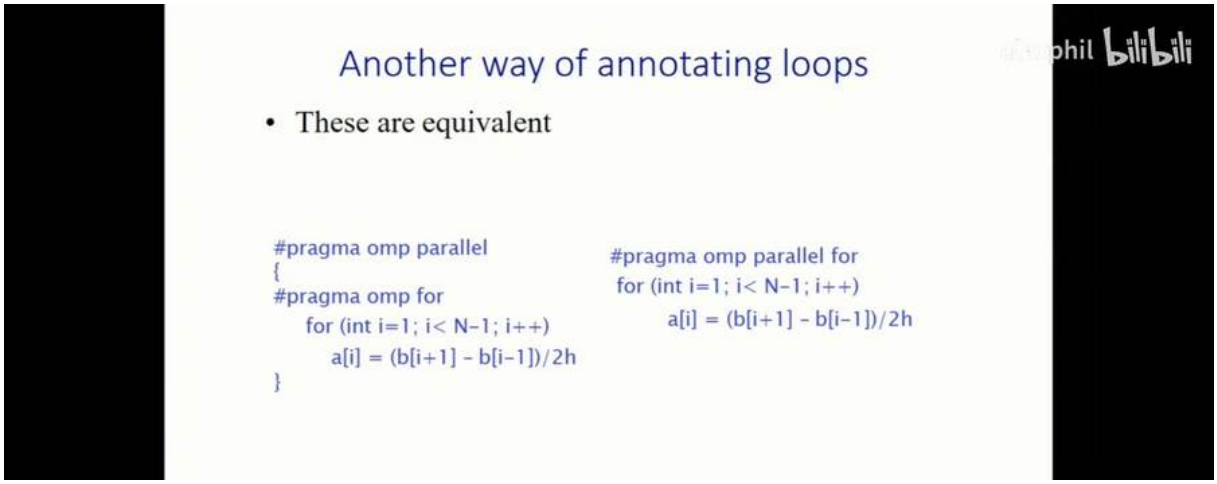

图 336: 两种等价的循环标注写法[336]

合并写法的隐含边界

`omp parallel for` 等于”开并行区”加”在区内划分循环”两步合一，因此它自带一个隐式的并行区边界。若在一个已开的并行区内部再写 `omp parallel for`，会嵌套出新的并行区，通常不是本意。这正是两种写法虽等价却不可随意互换的原因。

本章小结

编译器负责划分、插屏障、处理余数，并把私有与共享的判断留给程序员。两种 pragma 写法等价但边界含义不同。

[335] 视频画面时间区间：00:04:30–00:04:45。若为裁剪图，时间区间仍对应原视频画面。
[336] 视频画面时间区间：00:05:30–00:05:45。若为裁剪图，时间区间仍对应原视频画面。

那个被留下的判断正是最容易出错的地方，下一章专门讨论。

15.3 变量作用域：默认规则是个陷阱

两条默认规则

讲者给出 OpenMP 的默认约定：在并行区之前声明的变量，默认被所有线程共享；在并行区内部声明的变量，默认私有。

为什么默认是共享反而危险

讲者请学生想一件事：一旦开启并行，程序就变成多份实例。此时在”共享”与”私有”之间存在一个显然的选择。

他的判断很明确：私有才应该是常态，因为共享意味着并发控制问题——必须以某种方式管理，而这实际上会降低并行度。

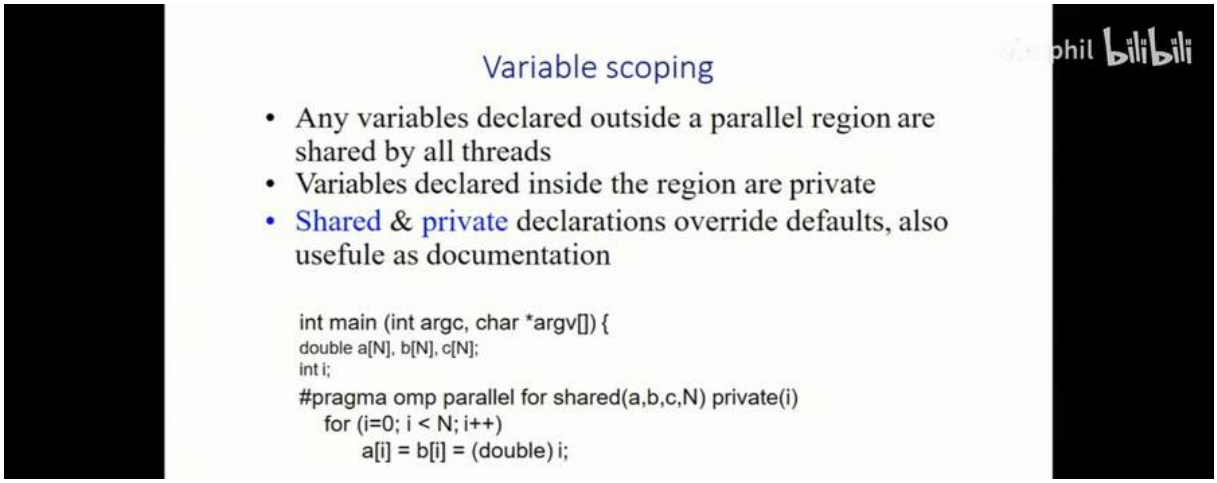

图 337: 变量作用域：并行区外声明的变量默认共享，区内声明的默认私有[337]

也就是说，默认规则把最危险的选择设成了默认值。

[337] 视频画面时间区间：00:06:30–00:06:45。若为裁剪图，时间区间仍对应原视频画面。

讲者的做法：100% 显式标注

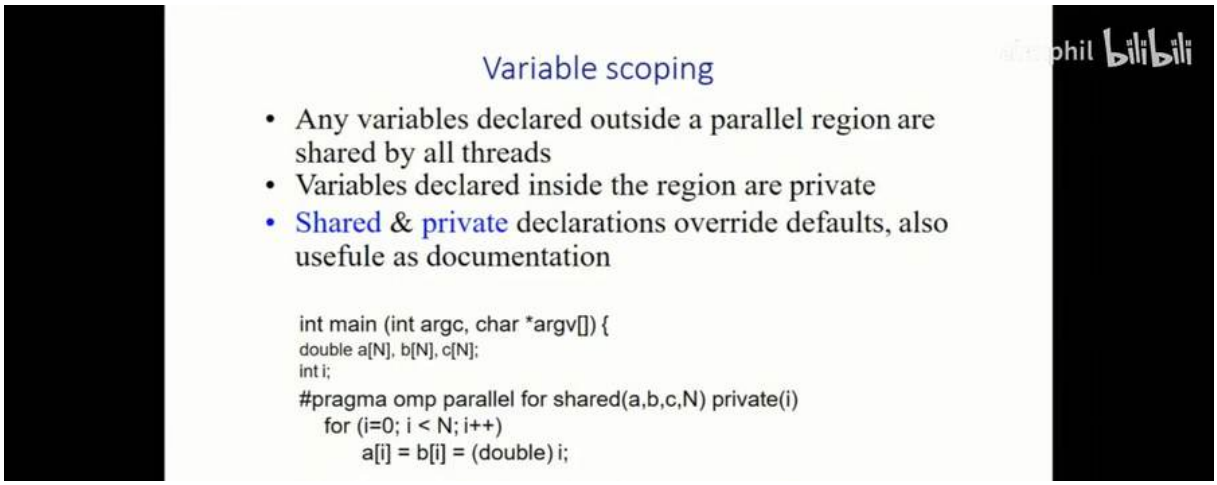

图 338: 变量作用域续：shared 与 private 子句的用法[338]

讲者说他写 OpenMP 代码时，即使不需要也会显式写 private，即使不需要也会显式写 shared。他给出的比例是 100% 的时间都这样做。

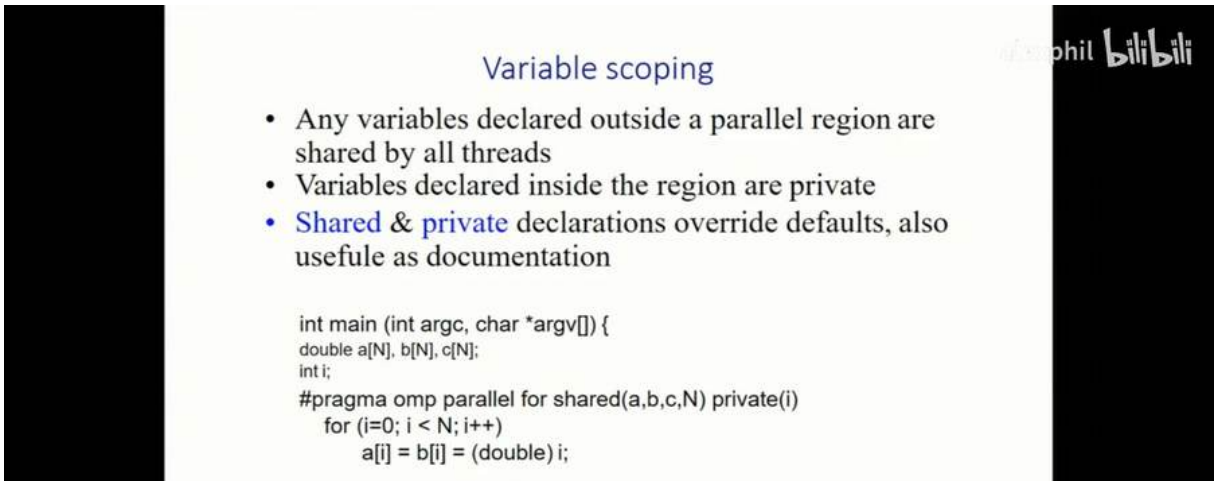

图 339: 变量作用域续：为何必须显式标注[339]

理由是两条。第一，若有人做维护性改动，把某个声明挪了位置——他以为自己在清理代码——却不知道这对 OpenMP 的含义，就会出问题。第二，看到一段并行代码时，我想直接知道我的假设是什么。要么去追溯变量在哪里声明，要么让它直接标出来。

显式标注把隐式契约变成文本契约

默认规则要求读者追溯变量声明位置才能推断共享性，而这对跨文件、宏展开或后续维护都不可靠。显式写出 shared 与 private，等于把并行区的假设写进代码本身。代价是冗长，收益是任何改动都会被阅读者立刻看到。

他问哪一种是正确答案，答案是后者。

[338] 视频画面时间区间：00:09:30–00:09:45。若为裁剪图，时间区间仍对应原视频画面。
[339] 视频画面时间区间：00:11:30–00:11:45。若为裁剪图，时间区间仍对应原视频画面。

默认规则让”搬动一行声明”变成语义变更

在普通 C 代码里把变量声明从函数开头移到循环内部，通常只是风格问题。但在 OpenMP 并行区里，这会把该变量从共享变成私有（或反之），可能让程序的结果完全改变，而且不会有任何编译警告。这正是讲者主张 100% 显式标注的根本原因。

本章小结

OpenMP 的默认规则是” 区外声明共享、区内声明私有”，而共享恰恰是需要并发控制的危险选项。因此讲者主张始终显式标注：既防止维护者无意改动引入语义变化，也让并行区的假设对读者直接可见。

15.4 CPU 与 GPU 的调度模型之别

讲者在回答学生提问时澄清了一件容易被混淆的事。

调度在哪里发生

在 GPU 上，warp 的调度由硬件负责：硬件持有这些 warp，逐条以锁步方式执行其中一个。

回到通用 CPU 模型后，调度不再以那种方式在芯片上发生，而是由操作系统负责。OS 持有一张待运行线程的列表，有些线程是进程的唯一线程，有些是更大任务的一部分。定时器设备周期性中断，操作系统调度器运行，决定此刻在发生什么、想让什么发生、想让哪个让出。

图 340: CPU 与 GPU 的调度模型不同：前者由操作系统调度，后者由硬件调度[340]

亲和性

讲者提到一个相关概念：亲和性，即让某组线程绑定在同一个核上运行，从而利用它们之间的缓存关系。

用 POSIX 线程库时可以指定线程如何成组，以利用缓存层次。理由是同一进程内的多个线程共享同一内存上下文。若共享同一内存上下文，它们的缓存复用就有价值。

[340] 视频画面时间区间：00:13:00–00:13:15。若为裁剪图，时间区间仍对应原视频画面。

图 341: 亲和性：让相关线程共享缓存层次[341]

亲和性是把”映射”这一步交给程序员

第 4 讲把线程映射到硬件单元列为四个步骤中的最后一步，通常交给系统。亲和性就是把这一步的决策权收回自己手里。当线程之间共享数据、需要共享缓存时，把它们绑到同一个插槽或用近邻核意味着更低延迟；反之若各线程独立工作，分散到不同插槽能获得更大总缓存。

本章小结

CPU 的线程调度由操作系统通过时钟中断完成，GPU 的 warp 调度由硬件完成。亲和性让程序员可以干预”线程落在哪个核上”，以利用共享缓存或避免跨插槽通信。

这些是背景知识。回到 OpenMP 本身，真正的陷阱在于依赖。

15.5 两类让结果出错的写法

竞争与伪共享同时出现

讲者展示了一个循环，其中各线程写相邻的元素。学生一眼指出两个问题。

第一是数据竞争：并行 for 的执行顺序没有任何保证，因此某个线程读到前一项时，完全不知道负责它的线程是否已经执行过。

第二是伪共享：这些元素是相邻的字，一个线程写就会作废另一个的缓存行，所以所有核都必须反复把这些缓存行拉回来。

[341] 视频画面时间区间：00:14:30–00:14:45。若为裁剪图，时间区间仍对应原视频画面。

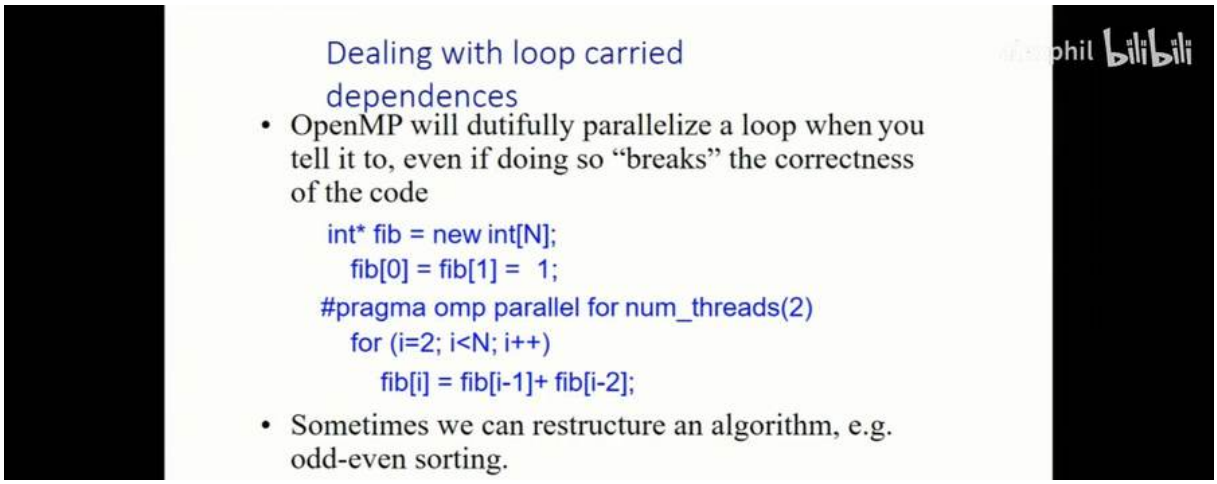

图 342: 相邻元素的并发写：竞争与伪共享同时发生[342]

同一段代码可能同时犯两类错误

数据竞争影响正确性，伪共享只影响性能，但它们可能源于同一处写法：多个线程处理相邻数据。此时既要修正确性（拆分依赖），也要修布局（避免同行争用）。只修一侧，程序要么还是错的，要么仍然很慢。

跨迭代依赖

讲者把问题抽象为跨迭代依赖。这类循环回读前一项、前两项，因此唯一能成功执行的调度就是串行。

而 OpenMP 会照你说的去做：你要求并行化，它就并行化，然后结果就错了。

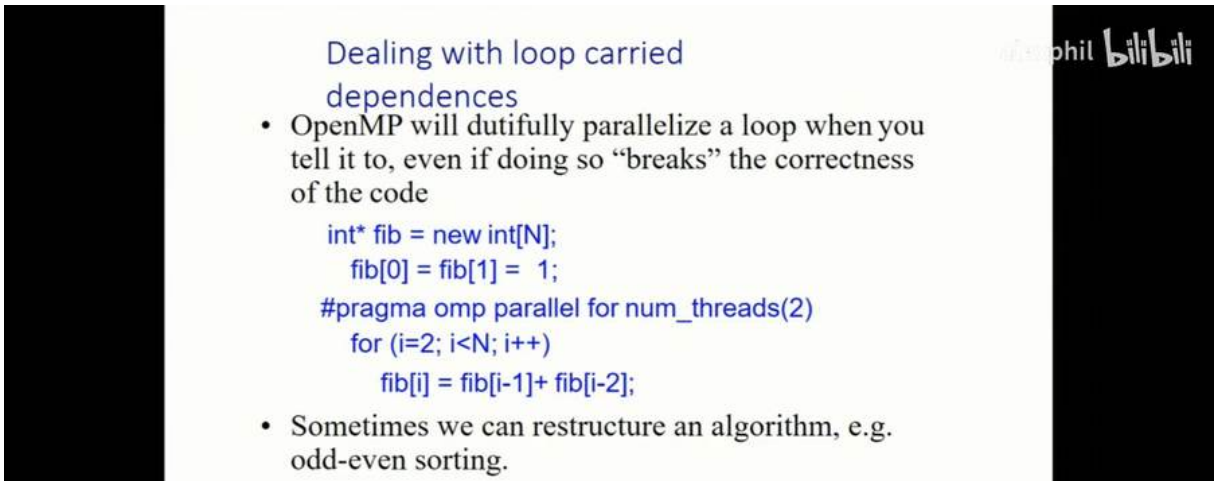

图 343: 跨迭代依赖：OpenMP 会照做并行化，但结果是错的[343]

讲者替 OpenMP 说了句公道话：它只是照你要的做，而且做得完全正确——错的是我们的要求。他还说明编译器有时能发现简单依赖并给出警告，但它并不真的知道全部情况，因此正确性责任在程序员。

[342] 视频画面时间区间：00:21:30–00:21:45。若为裁剪图，时间区间仍对应原视频画面。
[343] 视频画面时间区间：00:21:40–00:21:55。若为裁剪图，时间区间仍对应原视频画面。

不要指望编译器替你发现依赖

依赖分析在存在指针别名、间接寻址或不规则索引时会失效。OpenMP 有时给出警告，有时静默通过。因此”编译没报错”完全不构成”依赖已被处理”的证据。

本章小结

跨迭代依赖使唯一正确调度是串行，而 OpenMP 会照做，因此结果错误而责任在程序员。同一段代码还可能同时犯两类错误：相邻元素并发写既引发竞争（正确性）又引发伪共享（性能）。编译器有时能给出警告，但依赖分析在指针别名与间接索引下会失效。

除了循环本身，循环之间的同步也是一个独立的风险来源。

15.6 循环间同步：nowait 与 reduction

屏障默认与 nowait 子句

讲者接着解释一个默认行为：并行 for 结束时，OpenMP 会插入屏障，也就是这个循环必须全部完成后才能往下走。

若不想在循环末尾同步，可以加 nowait 子句，让线程完成即离开、直接执行后面的代码。

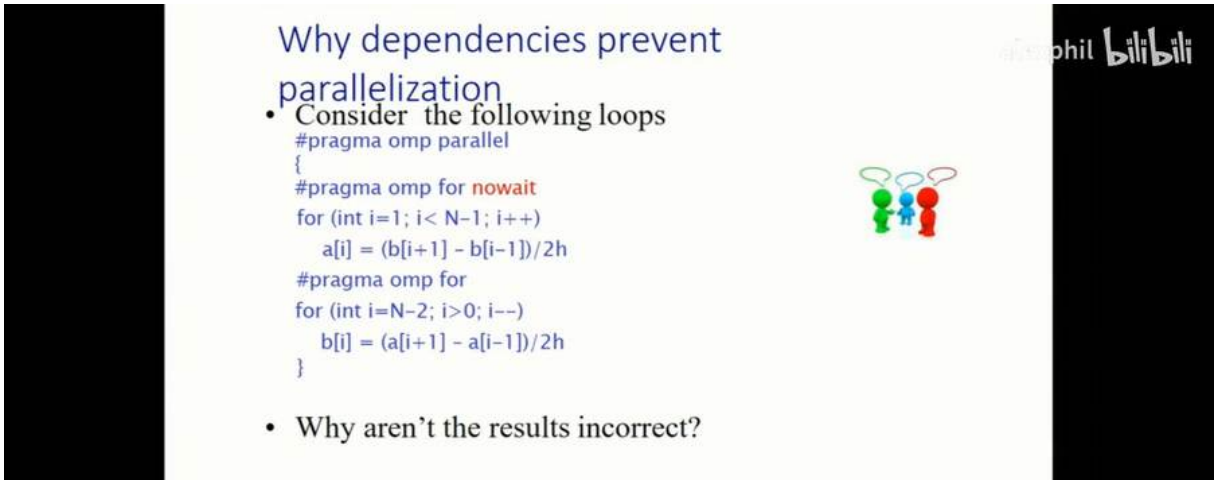

图 344: nowait 子句：去掉循环末尾的屏障[344]

他给出一个被”故意下毒”的例子：第一个循环带 nowait，第二个循环没有。第一个循环基于读 B 的结果给 A 赋值，第二个循环基于读 A 的结果给 B 赋值。两个循环之间存在依赖，而 nowait 恰好去掉了它们之间的同步。

nowait 与循环间依赖组合时最危险

两个相邻循环若存在数据流（前一循环写、后一循环读），默认屏障正是保证它们的正确性。加 nowait 后，线程可能在别的线程完成第一循环前就开始第二循环，读到旧值。这类错误的特征是偶尔出错、规模小时往往看不出来。

讲者的提示很明确：这个程序的唯一正确调度就是串行。

[344] 视频画面时间区间：00:26:00–00:26:15。若为裁剪图，时间区间仍对应原视频画面。

critical 与 reduction

讲者还讨论了 critical 区：需要保护共享更新时，把相关语句放进临界区。以及 reduction 子句：在某些应用里需要把一批值归约成单个全局值，OpenMP 提供声明式的支持。

其语义等价于部分和加最终合并：

$$\text{sum} = \bigoplus_{t=1}^{T} \text{priv}_t$$

- priv_t：第 t 个线程的私有累加值；
- $\bigoplus$：由 reduction 子句指定的结合运算（如加法、最大值）；
- T：线程数。

因为各线程只更新自己的私有副本，循环体内不需要任何加锁。

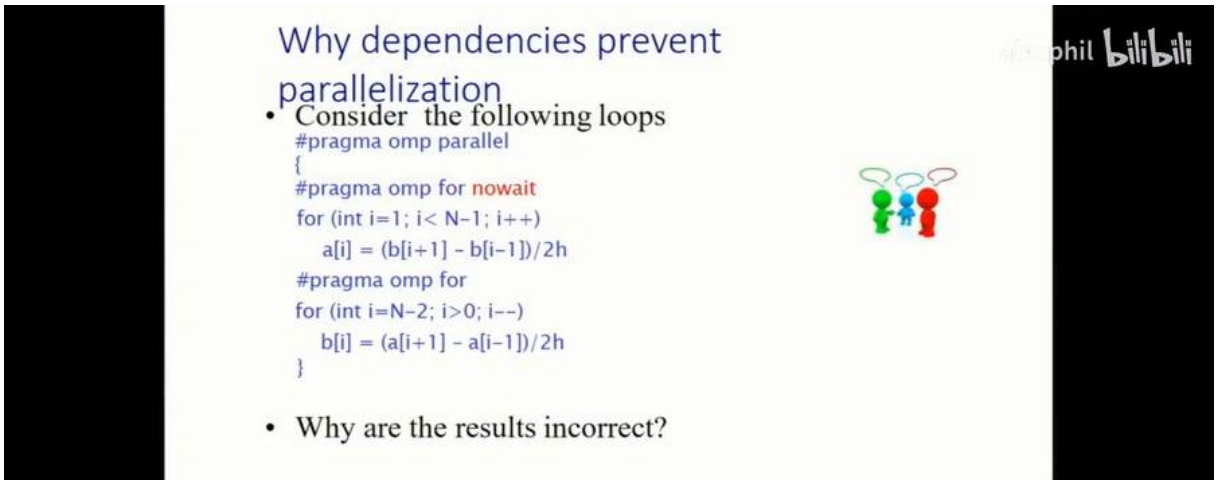

图 345: critical 区：保护共享更新[345]

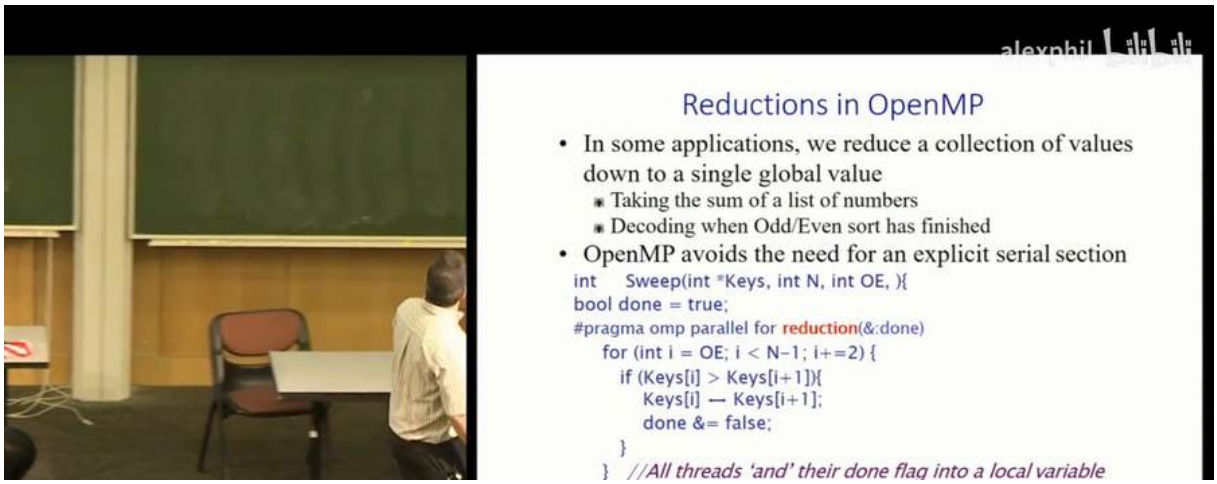

图 346: reduction 子句：自动汇总各迭代结果[346]

[345] 视频画面时间区间：00:31:00–00:31:15。若为裁剪图，时间区间仍对应原视频画面。
[346] 视频画面时间区间：00:55:00–00:55:15。若为裁剪图，时间区间仍对应原视频画面。

reduction 等价于部分和加最终合并

`reduction(+:sum)` 的语义是每个线程维护一份私有副本各自累加，循环结束后再合并。这与第 4 讲”部分和加串行合并”是同一件事，只是由编译器生成。因此它既避免了竞争，也避免了每次更新都加锁的开销。

本章小结

两类典型错误：相邻元素的并发写同时引发竞争与伪共享；跨迭代依赖使唯一正确调度是串行，而 OpenMP 会照做。默认的循环末尾屏障是保护措施，nowait 去掉它时必须确认后续代码不依赖该循环的结果。critical 保护共享更新，reduction 用私有副本加合并替代它。

这些陷阱都指向同一个结论：能并行化什么，取决于代码的真实依赖结构。

15.7 当 OpenMP 无法并行时：重组算法

一个关键判断

讲者在讨论重组时给出本讲最有价值的一句话：OpenMP 无法并行化某段代码，并不意味着人也无法并行化它。

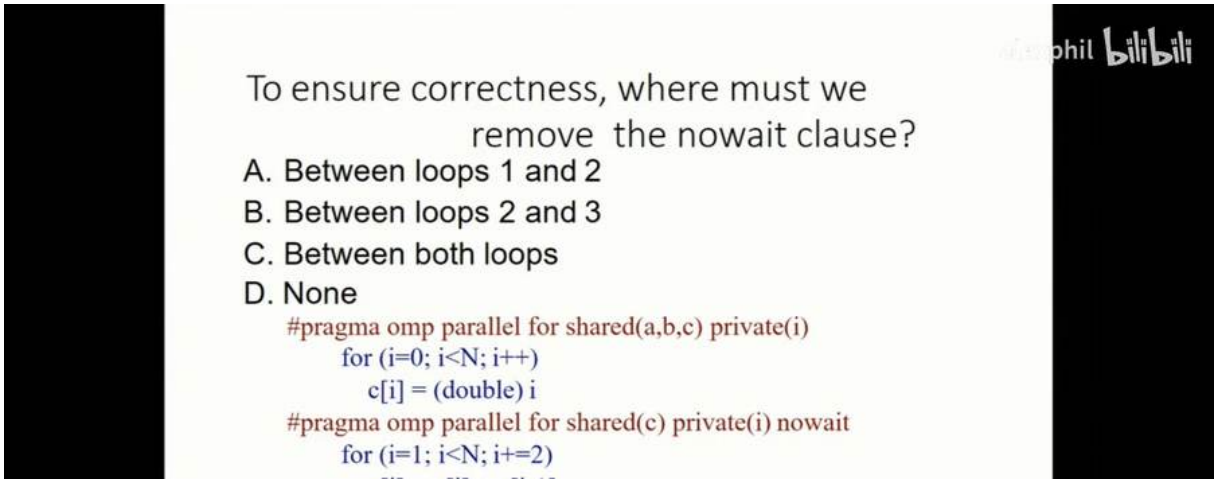

图 347：重组算法：拆成两个循环解开依赖[347]

只要理解代码在做什么，就可以做更高层的重组，得到相同的结果但换成可并行的计算方式。

拆成两个循环

举例：一个循环里，`B`[i] 依赖于 `C`[i] 的旧值，而 `C`[i] 又依赖前一次迭代的结果，于是既有”读到未更新值”的风险，也有”用未来值参与当前计算”的风险——两个方向的问题同时存在。

解法是把它拆成两个循环，从而解开纠缠。

[347] 视频画面时间区间：00:44:50–00:45:05。若为裁剪图，时间区间仍对应原视频画面。

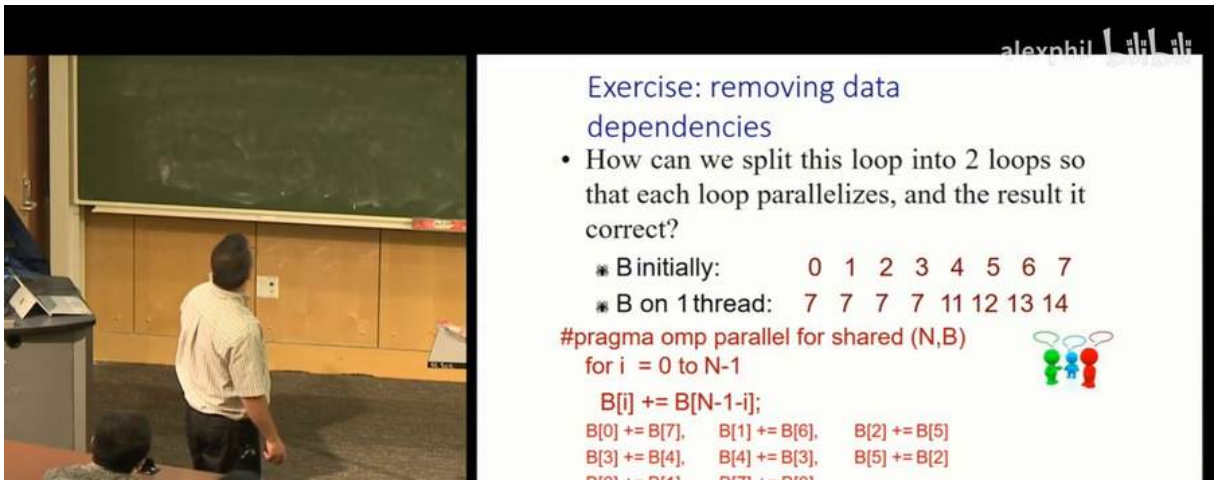

图 348: 从两端向中间配对遍历：等价于单向遍历但可并行[348]

算法重组与并行化无关，它改变的是计算的组织方式

拆成两个循环后，输出数组的内容完全不变，但依赖关系从”链式”变成”两个独立阶段”。这不是把串行代码伪装成并行，而是找到一个数学上等价、依赖结构更好的计算顺序。这类重组需要理解问题本身，而不是熟悉 OpenMP 语法。

一个反直觉的结论

讲者给出另一个例子：`B`$[i]$ 加上 $n-1-i$，即从外向内成对处理——索引 0 与最后一个配成一对，索引 1 与倒数第二个配成一对，逐步向中间收拢。

把这种对称性显式写出来后，原本的循环可以被改写。讲者问学生这么做如何解决问题，并在讨论后确认了一个反直觉的结论：如果去掉之前那个 nowait，反而更并行也更正确。因为前半段可以并行，而之前那样写则完全无法并行。从并行度角度看，这相当于把整段串行改成了「前半段并行、后半段串行」的 2 个阶段结构。

nowait 不是越多越好

nowait 消除的等待在某些场合是必须的同步。为了”让线程早点走”而加 nowait，常常把可以安全并行的结构变成错误结构。判断依据是后续代码是否依赖这个循环的输出，而不是线程是否能早点结束。

矩阵向量乘法

讲者最后用矩阵向量乘法作为经典案例说明这类问题在实际应用中的形态。这个例子的价值在于它同时具备两个特征：外层对输出向量的每个元素独立，因此天然可并行；而每个输出元素需要遍历整行，因此访存模式与缓存复用直接决定实际收益。讲者提醒，正因如此，同一份矩阵向量乘法按行并行与按列并行在性能上差别很大——前者每个线程顺序扫描一行，局部性良好；后者每个线程跨行取数，每一列都需要跳读。这与第 8 讲分块形状的结论一致：并行化的关键是数据布局与访问顺序，而不只是把循环拆开。

[348] 视频画面时间区间：00:50:00–00:50:15。若为裁剪图，时间区间仍对应原视频画面。

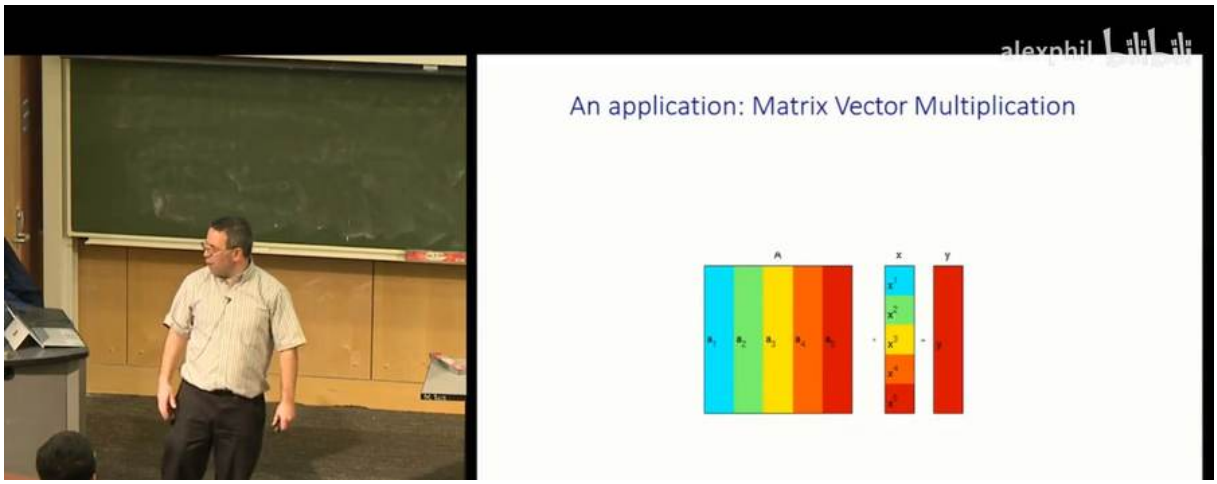

图 349: 矩阵向量乘法：一个应用案例[349]

本章小结

OpenMP 无法并行化不代表人无法并行化。通过拆分为两阶段、显式化对称配对等方式重组算法，可以在输出完全不变的前提下得到可并行的依赖结构。这类工作依赖对问题的理解，而不是语法技巧。反过来，滥用 nowait 会把安全的结构变危险。

15.8 总结与延伸

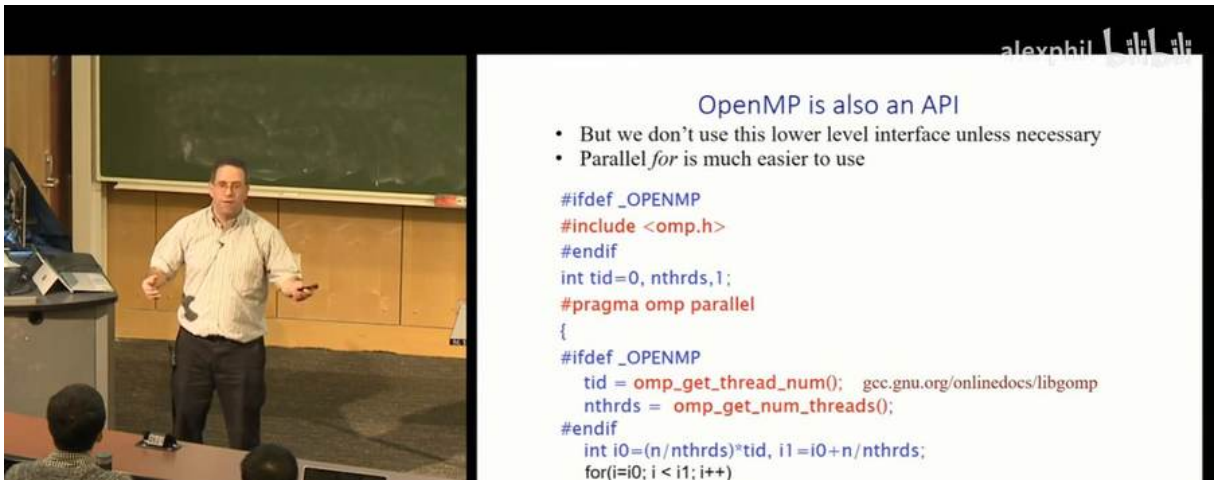

图 350: OpenMP 模型的适用边界：并行 for 最易用，更细控制需要更低层接口[350]

另外，讲者也说明 OpenMP 同时提供了一套底层的 API，但除非必要，他们并不使用这套较低层的接口，因为并行 for 更易用。这一点值得记住：同一个框架往往同时提供声明式与命令式两种入口，前者适合结构规整的循环，后者适合需要手工控制线程与同步的场景。选择哪一档，取决于代码的规整程度，而不是取决于哪种写法看起来更专业。

[349] 视频画面时间区间：01:00:00–01:00:15。若为裁剪图，时间区间仍对应原视频画面。
[350] 视频画面时间区间：01:12:10–01:12:25。若为裁剪图，时间区间仍对应原视频画面。

讲者在结尾坦率地评价了这个模型的边界：它是一个很快就会变得纠缠的模型。它确实是为”派生一堆线程、并行做一件事、同步、再并行、再同步”而设计的。如果你要做别的事情，更低层的模型可能更合适。但他也承认这个模型对很多问题非常好用且干净，他自己就很喜欢它。

把全讲串起来，OpenMP 用 pragma 在高层表达并行，采用 fork-join 模型，主线程贯穿始终。编译器自动划分循环、插入屏障、处理余数，但把私有与共享的判断留给程序员——而默认规则恰恰把最危险的选项（共享）设成了默认值，因此讲者主张 100% 显式标注。真正的陷阱有两类：相邻元素的并发写同时引发竞争与伪共享；跨迭代依赖使唯一正确调度是串行，而 OpenMP 会照做。循环末尾的屏障是保护措施，nowait 去掉它时必须确认后续不依赖结果。critical 保护共享更新，reduction 用私有副本加合并替代。而当 OpenMP 无法并行时，正确的方向是重组算法、拆成两阶段、显式化对称性，而不是硬加 pragma 或加 nowait。

本讲义在讲者材料之上做的压缩，有四点需要与讲者原话区分：

- 默认规则的危险不来自它本身错，而来自它让语义依赖于声明位置。同一个变量在函数开头和循环内部声明，行为不同且无警告。显式标注不是为了风格一致，而是为了把语义从”位置隐含”变成”文本可见”。
- pragma 与 forEach 属于同一类原语。它们声明控制并行而不验证依赖，因此正确性责任完全在程序员。这一判断在第 4 讲就已建立，这里只是换了一个语法。
- nowait 的效果是双向的。它既可能修复多余的同步，也可能破坏必需的同步。判据只有一个：后续代码是否依赖该循环的全部结果。
- 重组算法是并行工程的主要创造性工作。语法层面的 pragma 只是表达手段，而决定能达到多少并行度的是依赖结构。因此优化瓶颈往往不在”怎么写 pragma”，而在”怎么重排计算”。

这堂课的边界在于它是工具导论：讲者多次表示 OpenMP 的细节很多，学生需要在作业中体会。因此本文给出的是语义规则与陷阱清单，不含完整的子句参考。

两个可以直接从本讲数据提出的下一步问题：其一，既然默认规则危险，那么把 OpenMP 代码库整体改成显式标注，是否会掩盖本来应当被发现的依赖错误？其二，重组算法得到的是数学等价的另一个计算顺序，那么在浮点运算下”等价”是否严格成立，误差累积是否可能改变结果？

Lecture 16 Memory Consistency

视频时长：1 小时 8 分 58 秒　来源：https://www.bilibili.com/video/BV1Tj411z7U1/

16.1 问题：并行内存层次应该表现成什么样

上一讲讨论缓存协议时，我们只处理了单个缓存行内的一致性。这一讲要看更大的图景。讲者开场就说清了范围：之前的协议保证了某个缓存行上的读写按某种顺序一致的方式处理，但完全没有讨论 ** 不同地址之间 ** 的访问有没有约束。

如果没有约束，程序几乎无法写。

为什么”读到最新值”不可行

最直觉的定义是：每次读都拿到该地址最近被写入的值。

讲者说这在层次结构下不可行。有些操作一个时钟周期或更短就能完成，有些要几百个周期——从 1 个周期到 300 个周期，相差两个数量级。在一个大系统里，”时间”这个概念本身就不是严格对齐的。要求”这里的写必须立刻被别处的读看到”不切实际。

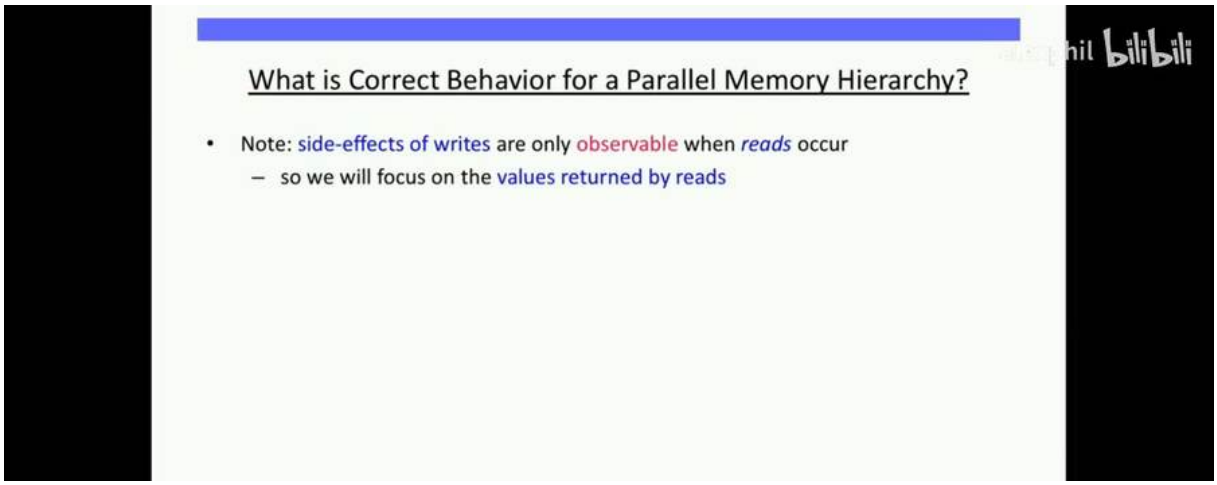

图 351：并行内存层次的正确行为：写的副作用只在被观察时才有意义[351]

另一个极端是完全混乱：谁读谁写，能不能看到全凭运气。这会让软件开发变得极其困难。

[351] 视频画面时间区间：00:01:30–00:01:45。若为裁剪图，时间区间仍对应原视频画面。

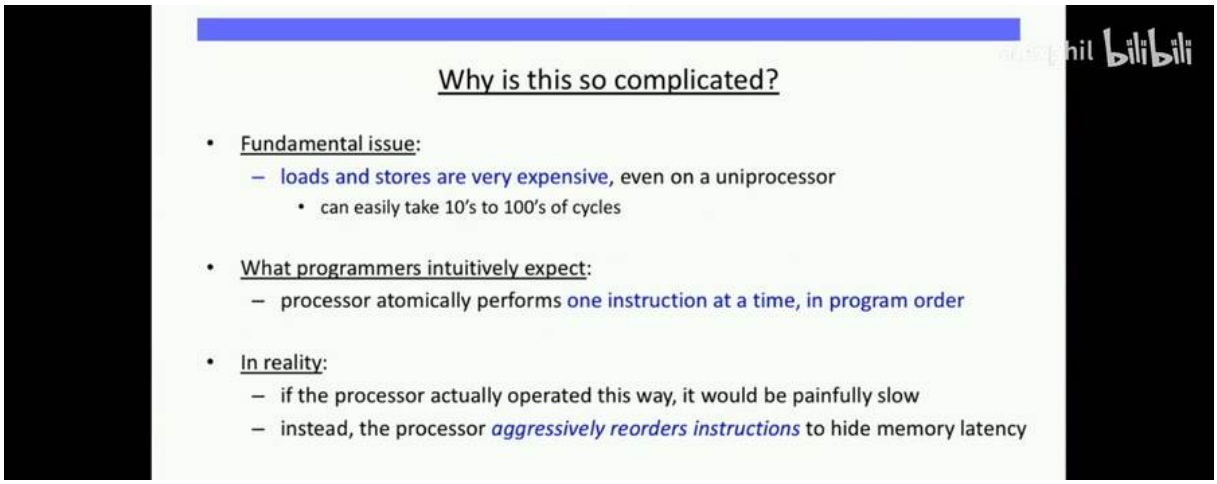

图 352: 为什么这件事这么复杂：访存本身在单核上就已经很贵[352]

因此几乎所有系统都提供某种一致性模型，但为了性能而采取了各种放宽。讲者把本节的核心定为这个取舍。

内存模型是性能与可编程性之间的协商结果

严格模型让程序行为符合直觉，代价是硬件必须频繁等待；宽松模型释放硬件自由度，代价是程序员必须显式表达依赖。因此”这个程序在多核上对不对”没有绝对答案，只能针对某个模型回答。这也是为什么同一份代码在不同架构上可能一个正确一个错误。

本章小结

”读到最新值”作为定义不可行，因为访存延迟跨越两个数量级，全局时间没有严格刻度。完全无序又让编程变得不可能。因此每个系统都在两者之间选择一个模型，模型就是对顺序约束的明确规定。

16.2 用一个实验建立顺序一致性的定义

讲者没有直接给定义，而是用一个观察者实验让定义自然浮现。

实验设置

程序里有三个线程。线程 0 把偶数依次写入共享变量 x；线程 1 把奇数依次写入同一个 x；线程 2 只是反复采样 x 的值。

问题变成了：线程 2 可能观察到哪些取值序列？哪些组合是非法的？

[352] 视频画面时间区间：00:10:00–00:10:15。若为裁剪图，时间区间仍对应原视频画面。

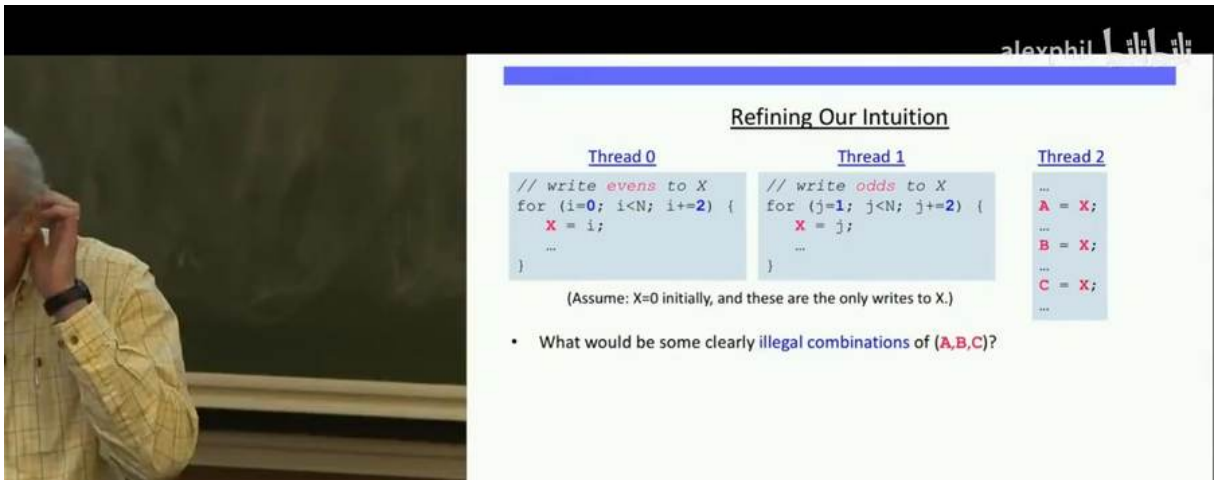

图 353: 细化直觉：线程 0 写偶数、线程 1 写奇数，线程 2 反复采样[353]

判断规则

讲者给出一个观察序列 4、8、1。学生很快判断这没问题。

理由正是定义的核心：只要偶数之间保持顺序、奇数之间保持顺序，那么这两组数的任意交错都是合法的。因为我们没有对线程 0 与线程 1 的相对速度做任何假设，只要求各自维持自己的顺序。

反过来，9、12、3 就有问题——3 不能出现在 9 之后，因为那意味着线程 1 自己的写序被打乱了。

而 7、19、31 是合法的：全是奇数且保持递增。

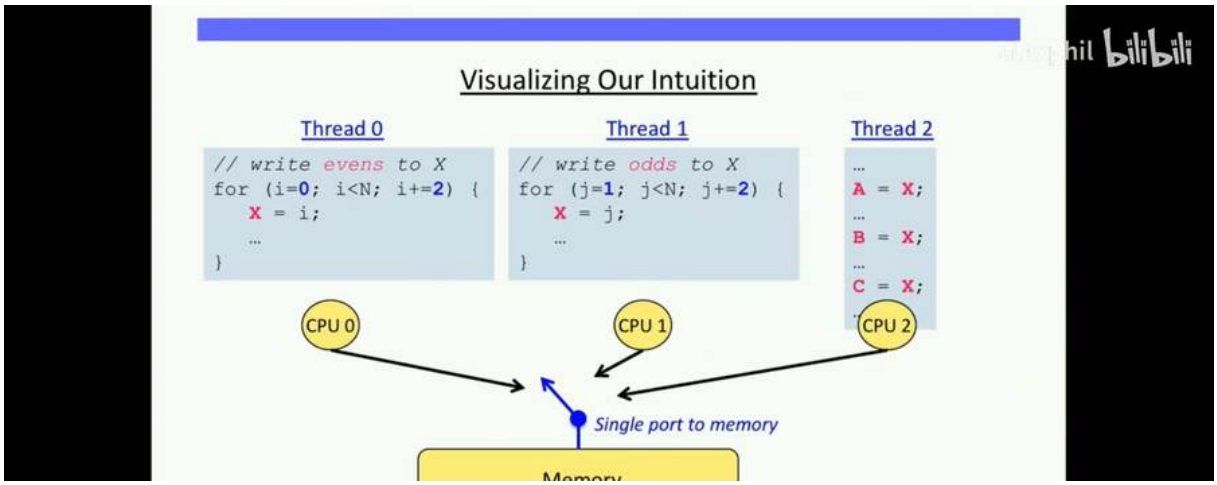

图 354: 可视化直觉：合法交错与非法序列的对比[354]

顺序一致性的判据是可交错性

对任意一个线程自己的写，任何外部观察者都必须按程序中的顺序看到它们；跨线程之间则必须存在某种合法的交错。这个定义不需要引入时间概念，只要求存在一个能解释全部观测的全局序列——这正是它可实现的原因。

[353] 视频画面时间区间：00:03:30–00:03:45。若为裁剪图，时间区间仍对应原视频画面。

[354] 视频画面时间区间：00:06:00–00:06:15。若为裁剪图，时间区间仍对应原视频画面。

对应的硬件想象

讲者给出一个假想模型来解释这个定义：一块完全全局的内存，没有任何缓存，每次只做一个操作。

每一步，各线程竞争执行自己那条操作的权限，这个”旋钮”选中其中一个，把它执行完。于是这个单端口内存就成了一个序列化器，强制把所有线程的操作编织成一个交错序列。

唯一保留的自由度是：我们不对线程之间的相对速度做假设，只要求每个线程内部保持程序序。

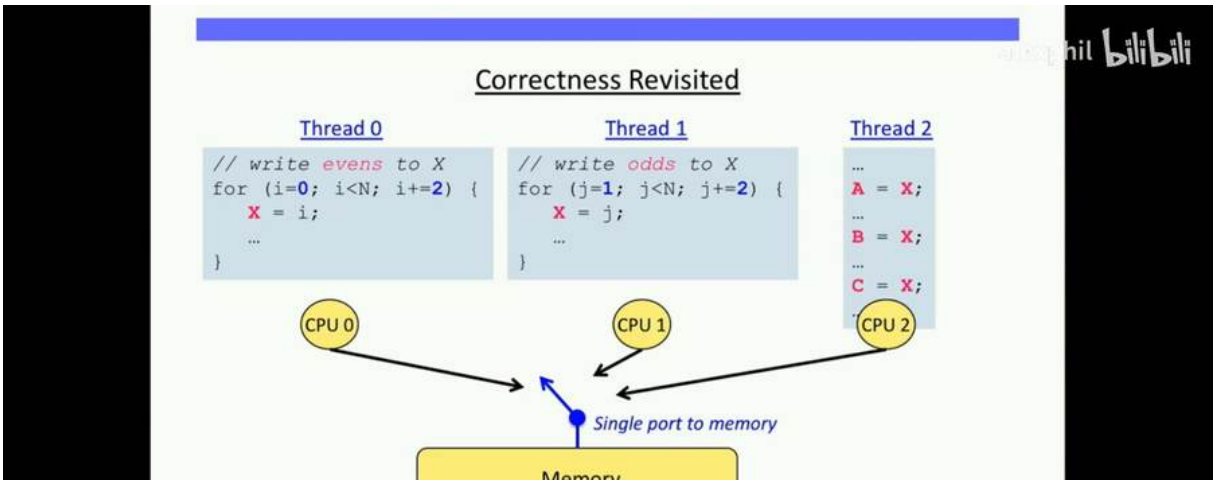

图 355：重新审视正确性：顺序一致性的判据[355]

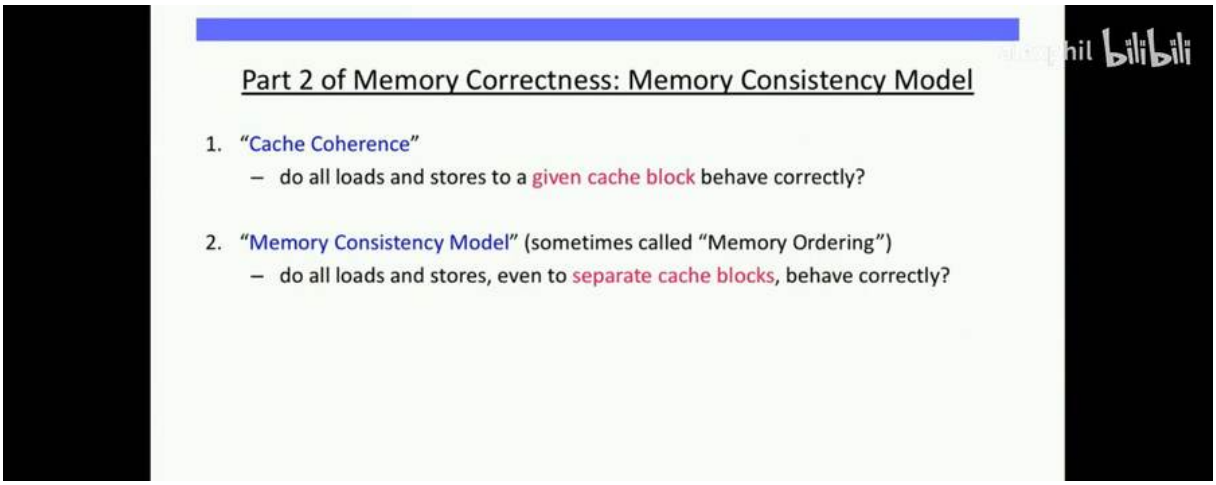

图 356：内存正确性的第二部分：一致性只覆盖单一变量，模型才覆盖跨地址[356]

Lamport 的顺序一致性定义

顺序一致性要求：每个进程的访存按程序序出现，且所有访存存在一个全局全序，该全序与各进程的程序序一致。讲者的单端口内存模型正是这个定义的操作化形式——把”存在全序”变成”由序列化器逐步产生全序”，从而可以推理。

[355] 视频画面时间区间：00:07:30–00:07:45。若为裁剪图，时间区间仍对应原视频画面。

[356] 视频画面时间区间：00:08:00–00:08:15。若为裁剪图，时间区间仍对应原视频画面。

本章小结

通过观察者实验，顺序一致性被表述为可交错性：各线程内部保持程序序，跨线程存在合法交错。单端口内存模型给出了它的操作化解释。这个定义只涉及单一变量的取值，尚未触及跨地址问题。

而这正是缺口所在：真实程序里读写不同地址的操作也必须有序。

16.3 横跨不同地址：更难的约束

顺序一致性的判据

把上面的规则推广到多个地址，就是经典的定义：

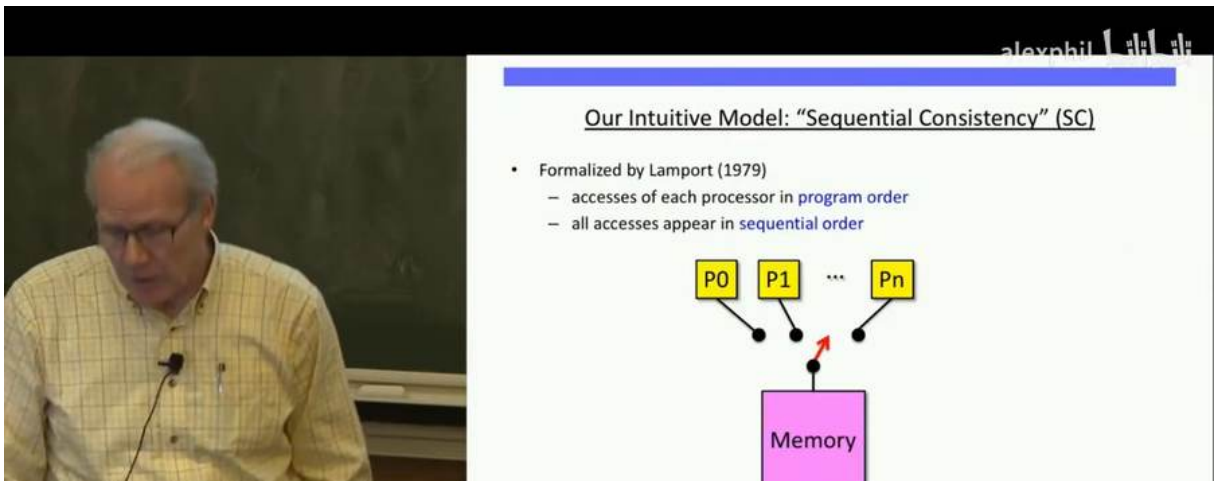

图 357: 直觉模型：顺序一致性（Lamport 1979），各进程访存按程序序[357]

$$\forall p:\ \text{order}_p \text{ 按程序序};\quad \exists \text{ 全序 } \sigma \text{ 与所有 } \text{order}_p \text{ 一致}$$

- order_p：进程 p 的访存序列，必须等于其程序中的顺序；
- σ：一个全局全序，用来解释所有观察者看到的顺序；
- 条件要求 σ 与每个 order_p 相容，即存在（而不要求唯一）这样的全序。

单核上已经不成立

讲者随即指出关键事实：即便在单处理器上，实现这个" 看起来是顺序执行" 的假象也需要大量硬件支持，而 ** 这些硬件不跨线程生效 **。

也就是说，即使在同一颗处理器上，不同线程的访存实际上可能不按你从程序里看到的顺序发生。

[357] 视频画面时间区间：00:26:30–00:26:45。若为裁剪图，时间区间仍对应原视频画面。

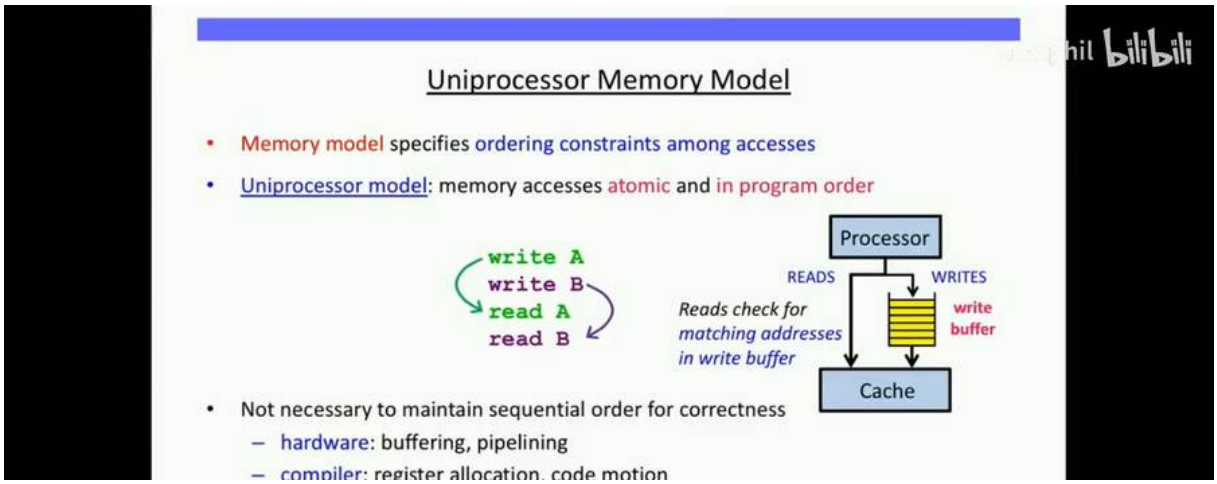

图 358: 单处理器内存模型：规定了访存之间的顺序约束[358]

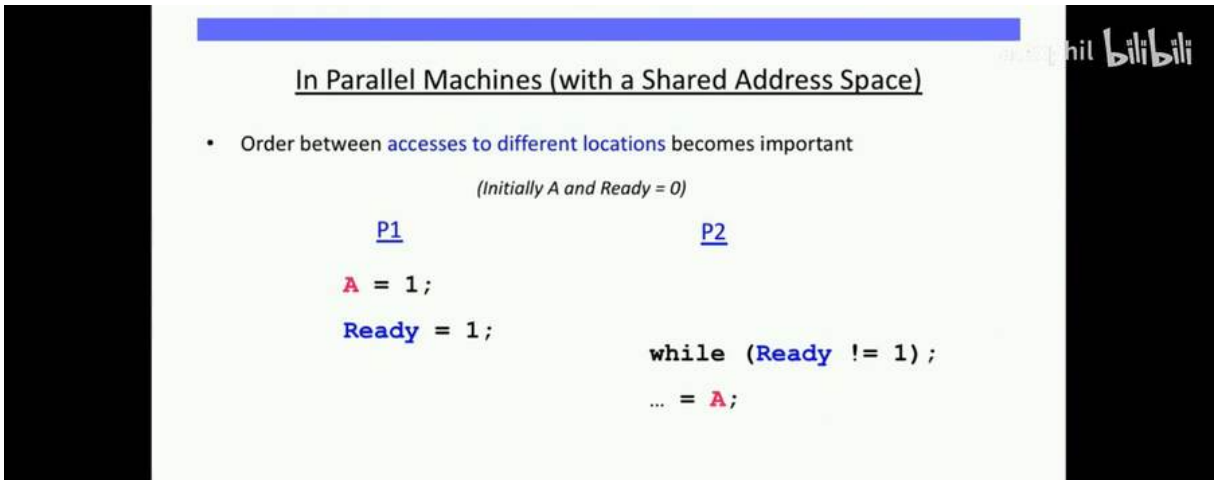

图 359: 共享地址空间并行机器中：不同地址之间的顺序变得重要[359]

写缓冲区：重排的物理来源

讲者给出机制。程序里写着” 先写 A，再读 B”，逻辑上后一条跟着前一条。但硬件会尝试：如果读 B 与写 A 无关，就可以在写 A 完成之前开始读 B。事实上完全可能出现读 B 先完成。

实现这件事的技术叫写缓冲区。处理器说” 把这个数据存到内存”，实际只是把它塞进队列，然后就可以认为这次写完成了。之后由包括缓存逻辑在内的硬件负责把它推出去。

[358] 视频画面时间区间：00:21:30–00:21:45。若为裁剪图，时间区间仍对应原视频画面。
[359] 视频画面时间区间：00:22:00–00:22:15。若为裁剪图，时间区间仍对应原视频画面。

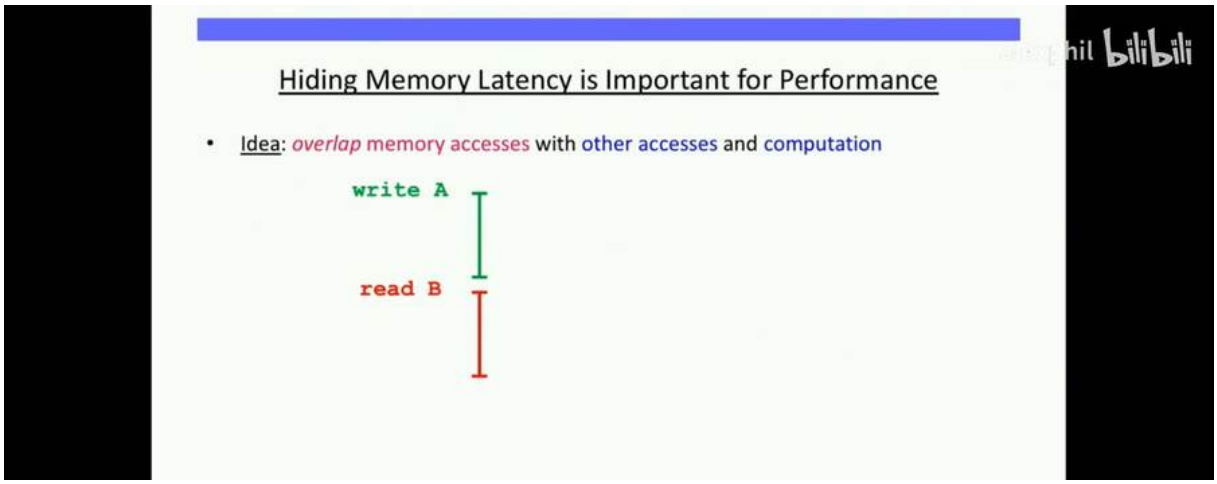

图 360: 隐藏内存延迟对性能很重要：让访存与其他工作重叠[360]

也有少量逻辑保证读写自己的一致性：读操作会先看一眼写缓冲区，若有对同一地址的待定写，就直接从缓冲区取值，而不去缓存读。

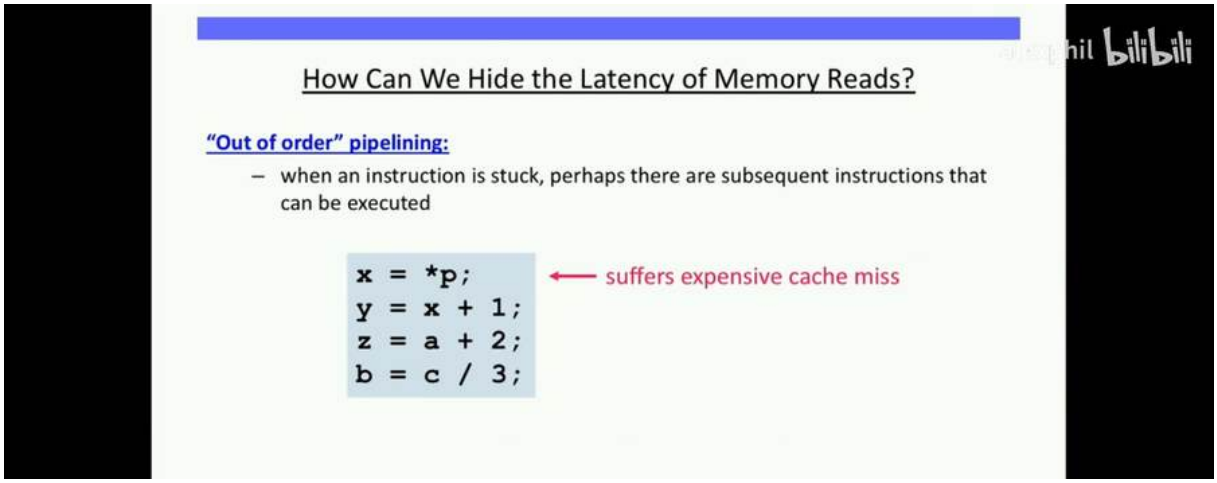

图 361: 如何隐藏内存读的延迟：乱序流水线在指令受阻时继续推进其他指令[361]

但关键在于：队列里可以积累一批还没进入内存系统的写，因此监听总线和缓存协议都不知道它们的存在。

[360] 视频画面时间区间：00:11:00–00:11:15。若为裁剪图，时间区间仍对应原视频画面。
[361] 视频画面时间区间：00:15:00–00:15:15。若为裁剪图，时间区间仍对应原视频画面。

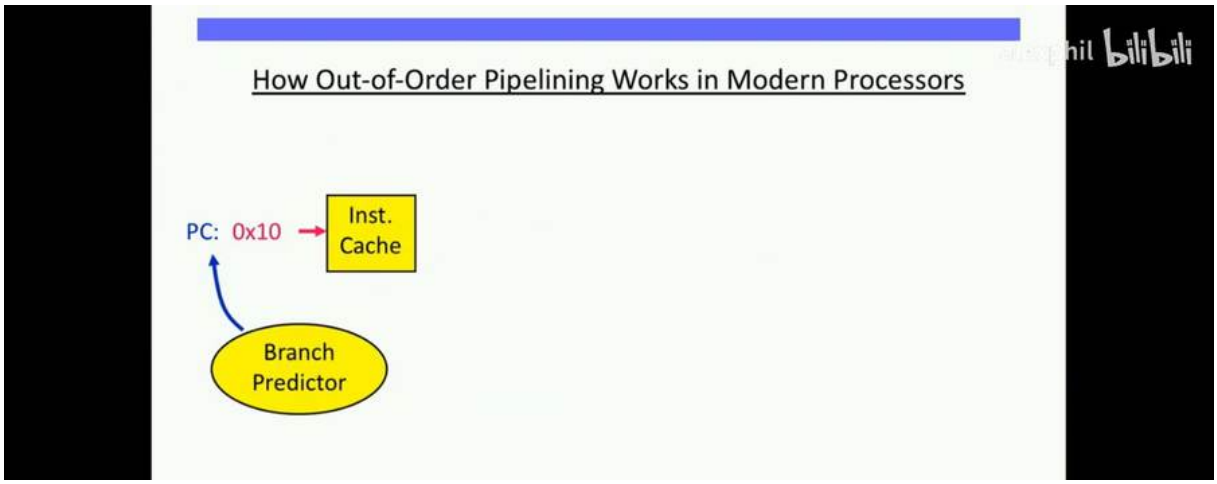

图 362: 现代处理器的乱序流水线：分支预测与乱序发射[362]

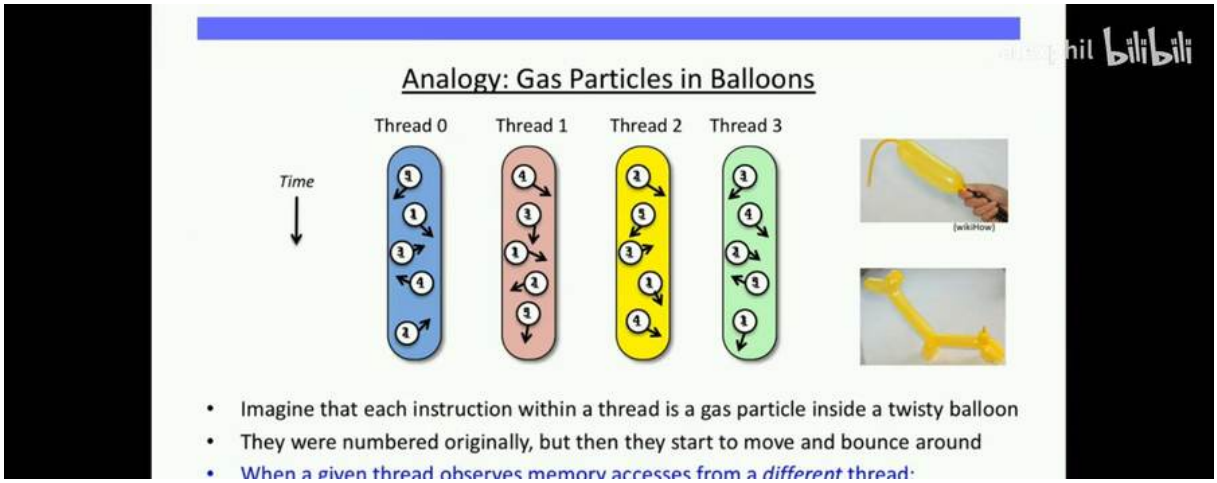

图 363: 类比：气球里的气体分子，各线程独立运动但仍受容器约束[363]

重排不是缺陷而是性能手段

隐藏访存延迟是性能的必要条件。若每条访存都必须等到前一条完成，流水线会在每次缺失时停摆数百周期。写缓冲区、乱序发射、分支预测都是为此存在。因此重排不可能被”修好”，只能被约束——内存模型的作用就是划定哪些重排可以接受。

本章小结

顺序一致性要求存在一个与各线程程序序相容的全局全序。但这个要求在单核上就已不成立：写缓冲区让写可以”提前完成”，乱序执行让读可以提前开始，而缓存协议完全看不到尚未推出的写。重排是为隐藏延迟而必然存在的。

16.4 反例：标志位同步为什么会失效

讲者用一个具体模式说明后果。

[362] 视频画面时间区间：00:19:00–00:19:15。若为裁剪图，时间区间仍对应原视频画面。
[363] 视频画面时间区间：00:20:30–00:20:45。若为裁剪图，时间区间仍对应原视频画面。

经典模式

处理器 1 更新数据 A，然后把标志位置为真。处理器 2 忙等这个标志位，一旦为真就认为可以安全读取 A。

这看起来是跨程序同步的标准做法。但如果读写相对顺序不保证与程序序一致，它就会失效。

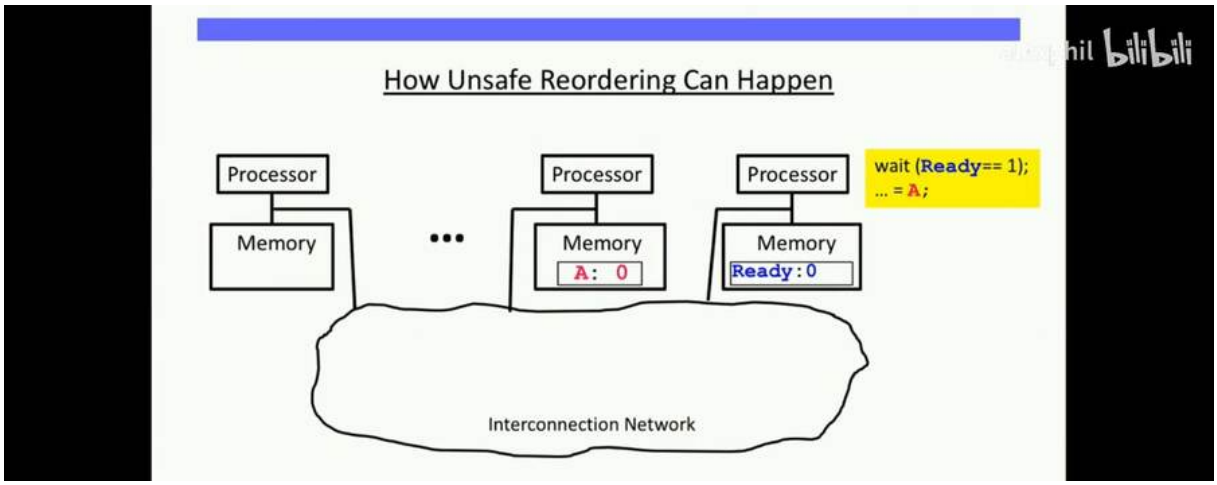

图 364: 不安全的重排如何发生：等待标志位后读到旧数据[364]

讲者描述了时序：处理器 2 持有 ready 的副本并在上面自旋，看到它是 0；处理器 1 把 A 设为 1，发出一系列事务，最终让 A 全局可见；但它设置 ready 为 1 的操作走得更快。

于是处理器 2 可能读到 ready 已经为 1，而去读 A 时拿到的还是旧值。

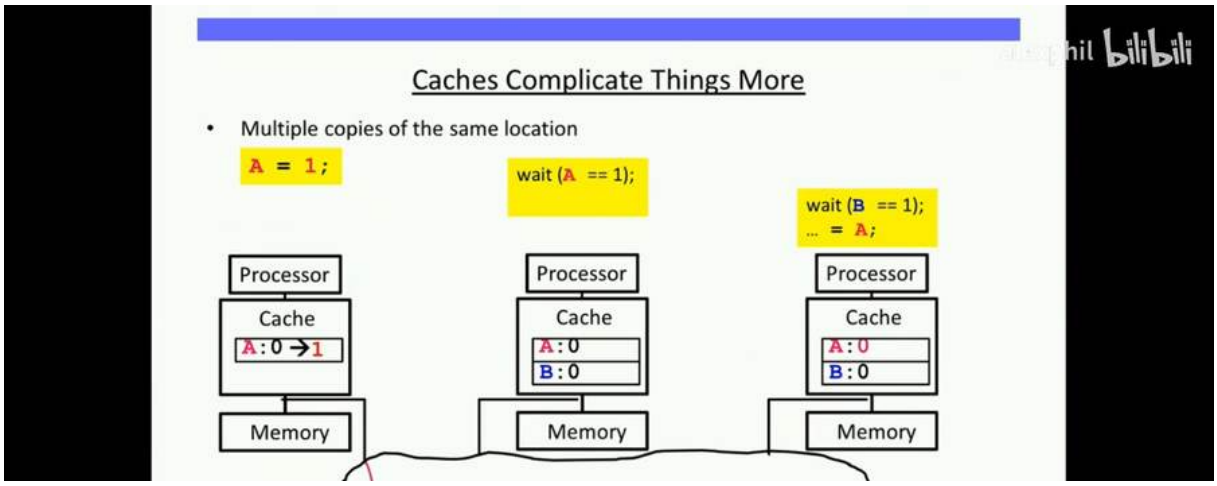

图 365: 缓存让情况更糟：同一位置的多个副本加剧了可见性错位[365]

标志位同步在弱模型上不成立

”先写数据、再置标志”依赖两条写之间的顺序对第三方可见，而这恰恰是模型允许打破的。写缓冲区、多级缓存与互连的延迟差异都可能导致标志先到。正确做法是使用释放/获取语义：写侧以释放方式置标志，读侧以获取方式读取，由硬件与编译器共同保证顺序。

[364] 视频画面时间区间：00:23:00–00:23:15。若为裁剪图，时间区间仍对应原视频画面。

[365] 视频画面时间区间：00:25:00–00:25:15。若为裁剪图，时间区间仍对应原视频画面。

链式依赖也救不了

讲者进一步给出更复杂的例子：三个线程构成一条依赖链。

左边把 A 设为 1；中间等待 A 变为 1 后把 B 设为 1；右边等待 B 变为 1 后去读 A。

即使有链式依赖，仍然可能出错：第一个线程启动了 A 的更新，很快触发了中间那个线程认为 A 是 1；但 A 的更新真正传遍全系统可能更慢。于是 B 先被置 1 并传到第三个线程，而那时 A 的更新还没到，第三个线程读 A 只能拿到旧值。

讲者总结说，这不只是" 这里写那里读" 的简单情形，一串有依赖的写读也可能因为延迟差异而拿不到正确值。

精简成可判定的反例

讲者把问题简化以便判定。去掉自旋，只问：处理器 1 可能读到哪些值组合？其中 (1,0) 这种组合不应出现。

$$(x, y) \neq (1, 0)$$

- x：处理器 1 读到的标志位值；
- y：处理器 1 读到的数据值；
- $(1, 0)$ 表示看到了" 数据已就绪" 的标志，却读到了初始值，这违背了写方的程序序。

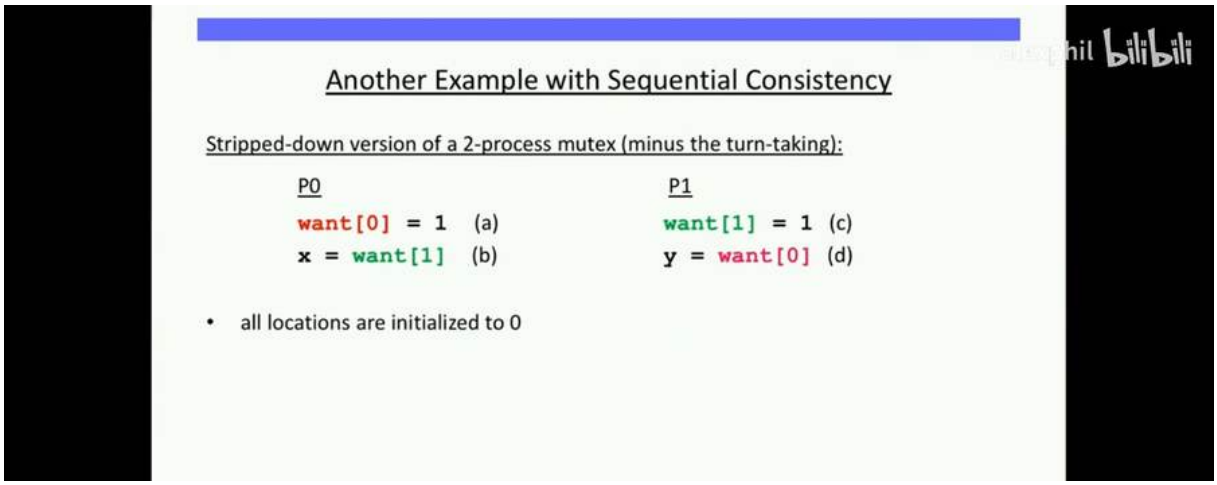

图 366: 互斥协议中不应出现的取值组合：两个进程不可能都读到初始值[366]

互斥协议的例子同理。两个进程各有一个标志，表示自己想进入临界区。若两个进程读到的都是初始值（组合 $(0, 0)$），就意味着双方都没看见对方的声明——而实际上至少有一方已经写过。因此 $(0, 0)$ 不可能。

讲者说这些都是相当标准的结论。而他提出的问题是：如果完全不能假设顺序一致性，那最基本的同步原语该怎么实现？

[366] 视频画面时间区间：00:29:00–00:29:15。若为裁剪图，时间区间仍对应原视频画面。

本章小结

标志位同步与互斥协议都依赖跨地址的顺序可见性，而弱模型允许打破它。链式依赖也无法补救，因为延迟差异会沿链累积。(1,0) 与 (0,0) 这类组合成为判断实现是否正确的探测器。

这引出一个两难：既要性能（允许重排），又要正确性（需要顺序）。下面的方案就是在这个张力中找平衡。

16.5 如何恢复顺序：从严格到逐步放宽

最严格的做法

一种办法是强制每个处理器同时只能有一个未完成访存。配合单地址的一致性协议，就足以保证每个处理器按程序序执行访存。

讲者说其实不必这么严厉：可以让后来的操作等待前一个被视为” 已完成”。

完成是什么意思

这里有个重要定义。一次访存的完成不是指数据真的到达内存，而是指它已经进入了内存系统的某个位置，从那里保证会被完成。

对写来说，塞进写缓冲区不算完成——它必须已经进入整个缓存协议、开始按要求作废相关的行。这个位置有时叫提交点，也就是说这次写在语义上确实要发生了。

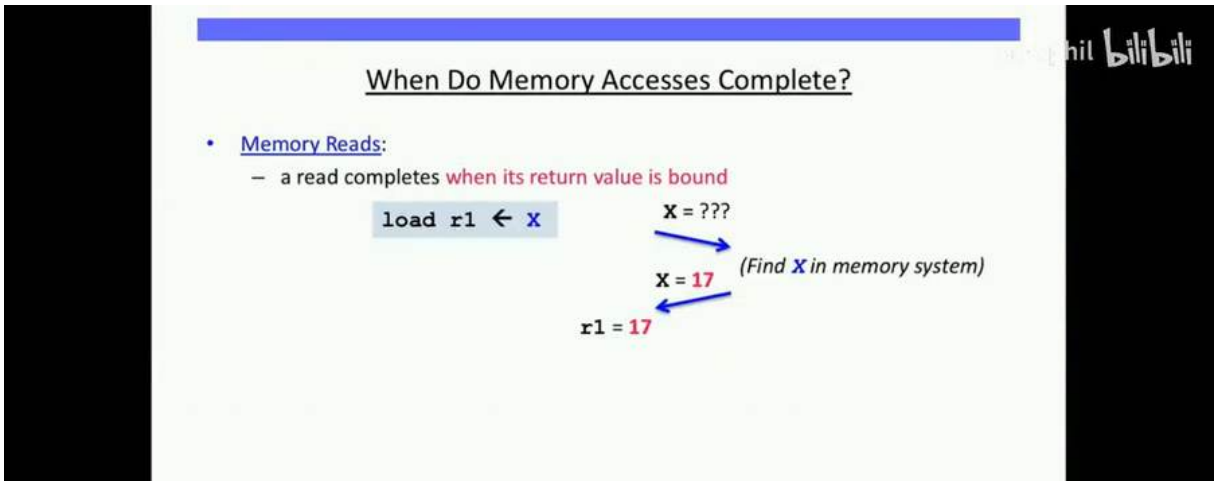

图 367: 访存何时算完成：读在返回值绑定时完成，写在到达提交点时完成[367]

对读来说，一旦确定了它要返回哪个值（无论来自内存还是别的缓存），就可以算完成。

” 完成” 是一个语义概念而非物理概念

读不需要真的把数值传回寄存器，写也不需要真的更新主存，只要已经推进到协议能保证必然完成的位置即可。这个定义让硬件可以在不牺牲正确性的前提下大幅重叠操作——因为等待的是” 承诺”，而不是” 结果”。

[367] 视频画面时间区间：00:32:00–00:32:15。若为裁剪图，时间区间仍对应原视频画面。

严格方案的代价

讲者说这就是在多处理器系统上实现经典顺序一致性的方式，但它是个相当严厉的规则，会让性能为写操作付出很大代价。

把” 每次只有一个未完成访存” 写成约束，就是：

$$\forall i:\ \mathrm{commit}(op_i) \prec \mathrm{begin}(op_{i+1})$$

- op_i：程序序中第 i 个访存操作；
- $\mathrm{commit}(op_i)$：该操作到达提交点的时刻；
- $\mathrm{begin}(op_{i+1})$：下一个操作可以开始的时刻；
- $\prec$：严格先于。

放宽模型的做法就是允许某些操作对违反这个约束——TSO 允许读越过尚未 commit 的写。

形象地说，这相当于在每次访存之间给气球拧一道结：相邻操作之间不能有任何重叠。

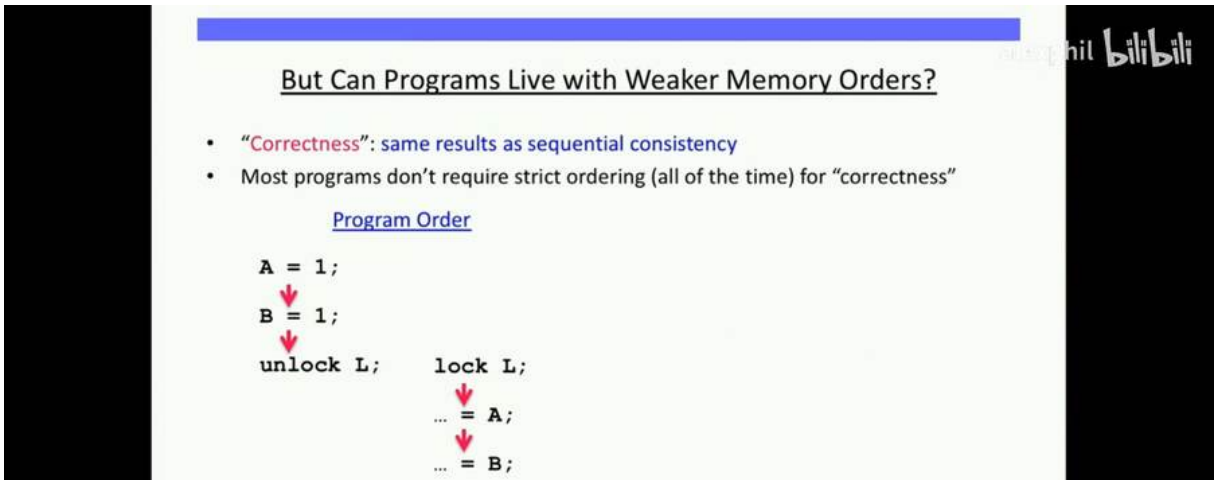

图 368: 程序能否接受更弱的顺序：停顿时间随模型放宽而大幅下降[368]

[368] 视频画面时间区间：00:39:00–00:39:15。若为裁剪图，时间区间仍对应原视频画面。

TSO：允许读越过写

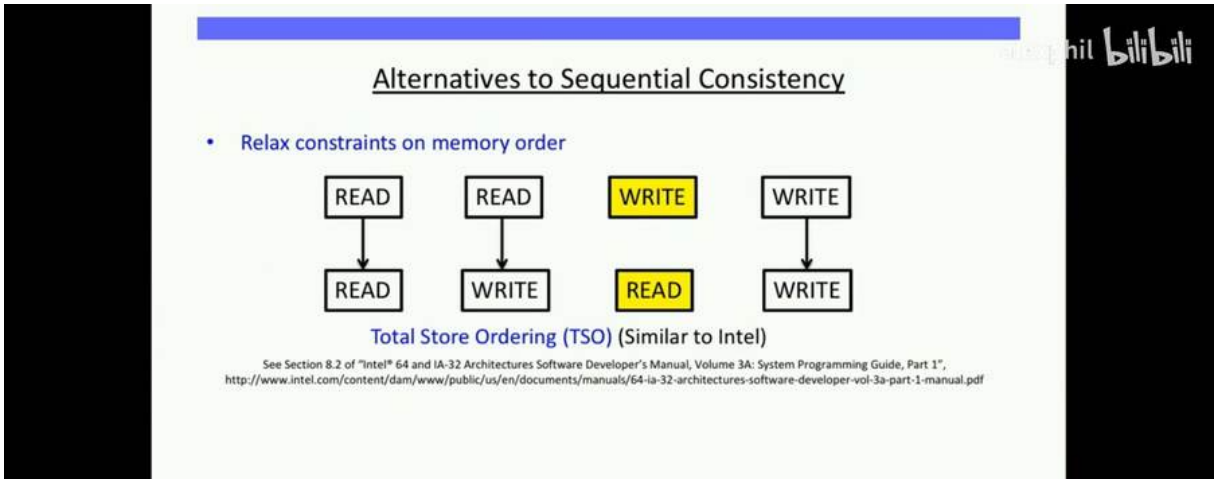

图 369: 顺序一致性的替代方案：放宽访存顺序约束[369]

最简单的放宽叫全存储顺序（TSO）。它允许在一次写尚未完成时就开始读——这正是做写缓冲区的人想要的，因为可以把写结果塞进队列就开始下一条操作。

但它保留了其他三条约束：读与读有序、读与写有序（必须先完成前一个读才能把写放进缓冲区）、写与写有序（用 FIFO 队列保证）。

讲者说这是硬件设计者可以" 不太费力就提供" 的方案，而事实上这正是 Intel 提供的基本语义（细节比这更复杂）。

模型	读-读	读-写	写-写
SC（顺序一致）	保序	保序	保序
TSO	保序	保序	保序（FIFO）
PSO	保序	保序	可重排

表 10: 各模型对访存顺序的约束（放宽的是写之后的操作）

PSO：连写也放宽

再放宽一步叫部分存储顺序（PSO）：写之后不做任何约束。

这在实现上的含义是写缓冲区不必是 FIFO，写可以任意顺序完成。

讲者给出性能对比：在这些基准上，从顺序一致性到 TSO，用于等待写而停顿的时间大幅减少，因为写被丢进缓冲区就可以先不管。

[369] 视频画面时间区间：00:36:00–00:36:15。若为裁剪图，时间区间仍对应原视频画面。

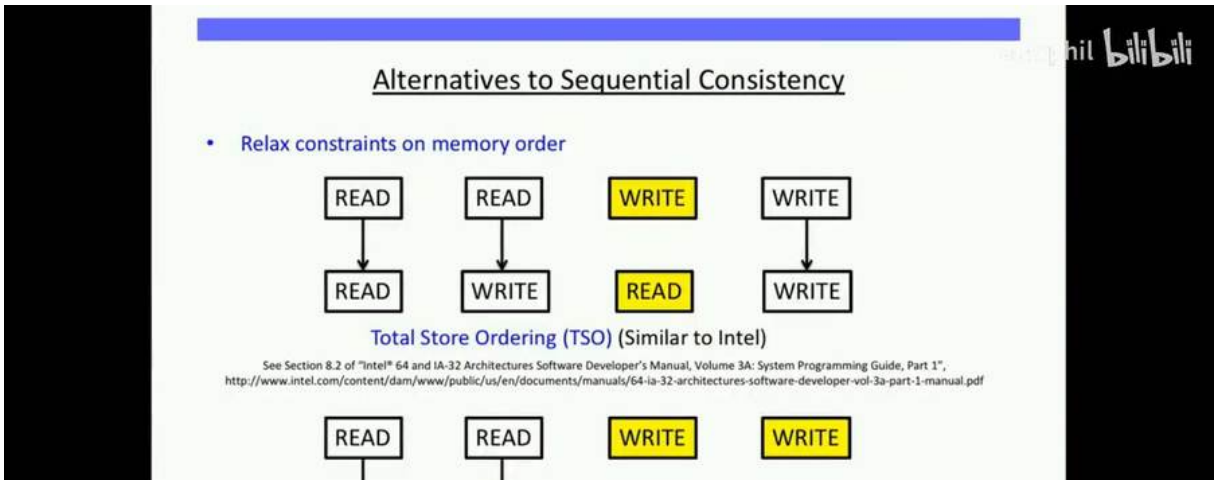

图 370: PSO：连写之间的顺序也放宽，写缓冲区无需 FIFO[370]

本章小结

恢复顺序的最严格做法是每次只有一个未完成访存，但代价巨大。关键是” 完成” 被定义为到达提交点，而不是物理完成。在此基础上可以逐条放宽：TSO 允许读越过写、保留其余三条约束，基本对应 Intel 语义；PSO 进一步放宽写之间的顺序，停顿时间显著下降。

16.6 弱序模型与栅栏

假设程序已正确同步

讲者转向另一条思路：不再要求硬件对所有访存保序，而是假设程序已经用正确的同步手段保护了共享状态。

具体来说，如果存在全局状态，那么除非持有适当的锁，程序不会去修改它。

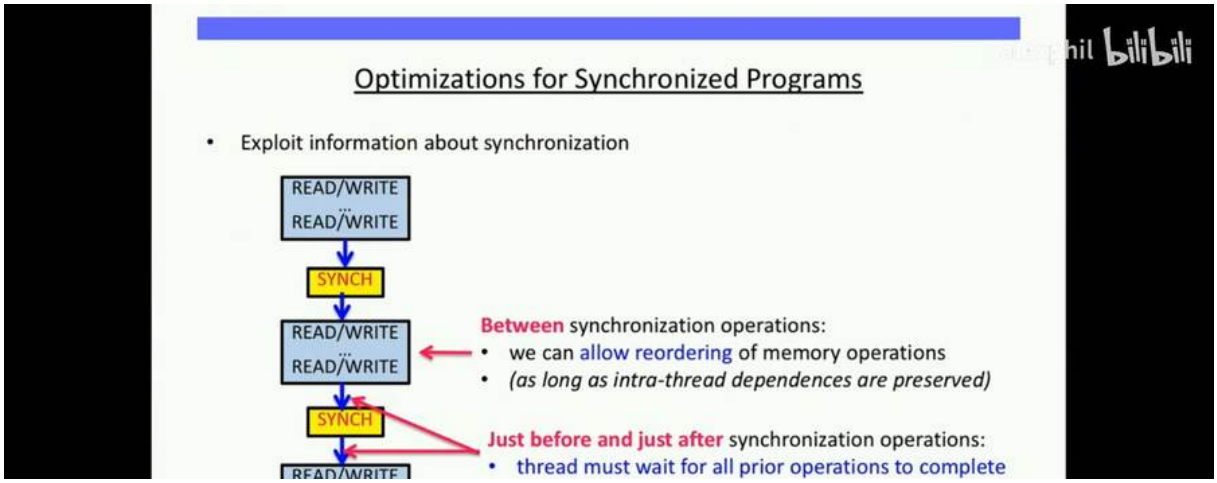

图 371: 为已同步程序做的优化：利用同步信息放宽顺序[371]

[370] 视频画面时间区间：00:38:00–00:38:15。若为裁剪图，时间区间仍对应原视频画面。
[371] 视频画面时间区间：00:47:00–00:47:15。若为裁剪图，时间区间仍对应原视频画面。

一个典型片段是：先做一些计算，然后取锁、更新全局数据结构、释放锁。此后任何操作都应当能看到这次更新。

关键落在两处：临界区内部的访存顺序无关紧要，因为这段代码只有自己能改这些数据；而释放锁之前，所有本该在临界区内完成的更新必须真的生效。

弱序的正确性完全依赖程序自身的同步纪律

硬件不再替你保证跨地址顺序，它把这份责任交给了程序和编译器。因此”我能读懂这段代码的意图”不足以保证正确——必须确认每一处共享访问都在同步保护之下。这也是为什么弱序模型下最简单的数据竞争都可能造成难以复现的错误。

只读共享数据的额外麻烦

讲者提到一个容易被忽略的细节：通常修改全局结构需要写权限，但程序经常允许其他线程以只读方式访问共享数据。例如一个计数器，更新它的线程必须持锁或用原子操作，但任何人都可以在任意时刻读它。

这就要求更细致地界定：对共享数据的只读访问，我们假设了什么样的顺序？

MFENCE

讲者给出 Intel 提供的原语。MFENCE 的语义是：到这一点为止所有此前未完成的读写都必须完成，并且在到达它之前不开始任何新操作。

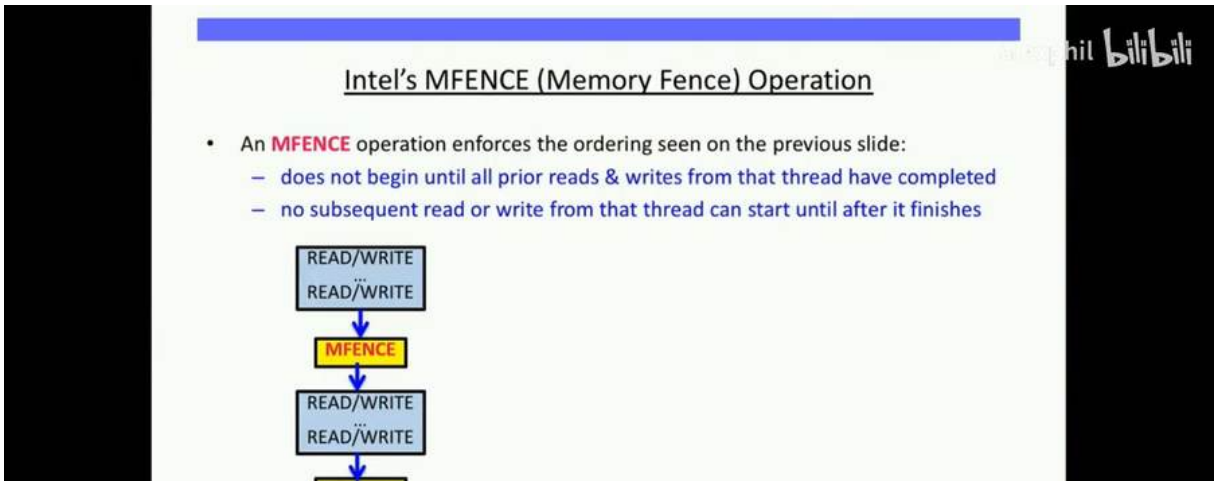

图 372：Intel 的 MFENCE：强制前序读写完成后再继续[372]

逻辑上它就是在气球上拧一道结。同样地，它不要求最终的写真的发生，只要求推进到协议能保证完成的位置。

实现上它并不需要等一切结束：遇到 MFENCE 时线程会停顿，直到写缓冲区排空。

[372] 视频画面时间区间：00:49:00–00:49:15。若为裁剪图，时间区间仍对应原视频画面。

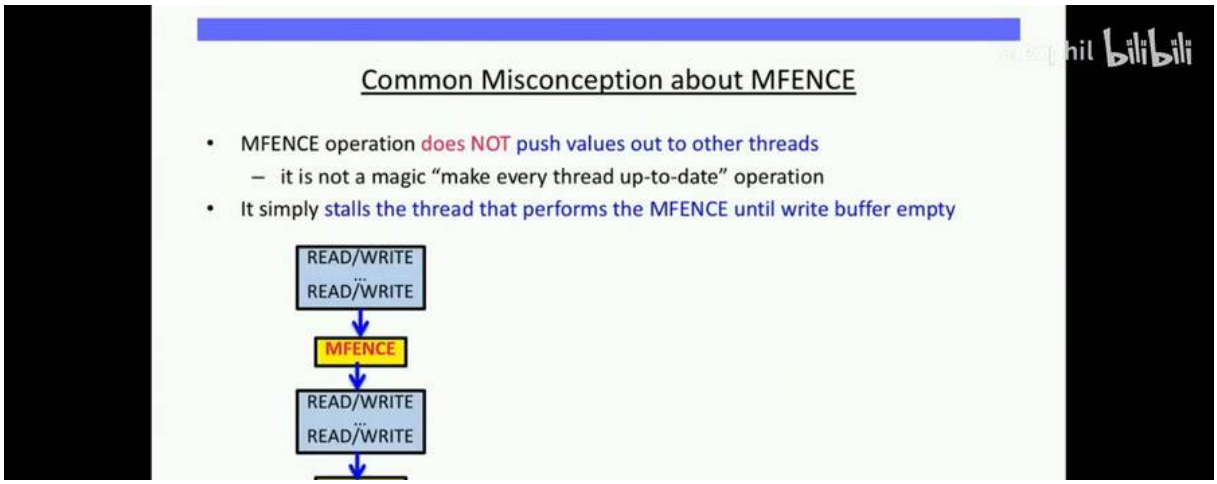

图 373: 关于 MFENCE 的常见误解：它不会把值推给其他线程[373]

讲者还专门纠正了一个常见误解：MFENCE 并不会把值推送到其他线程，它不是”让一切可见”的魔法指令。它只约束本线程的访存顺序。

栅栏约束的是顺序，不是可见性

栅栏保证”此前的操作在可见顺序上先于此后的操作”，但不负责把数据发送出去。数据何时被他人看到，仍由缓存一致性协议决定。混淆这两件事，会导致在需要同步可见性的场合误用栅栏而得不到效果。

ARM 的现实教训

讲者最后给出一个很有实践价值的提醒。ARM 处理器的内存模型比 x86 宽松得多。众所周知，为它编程要困难得多，但换来的是更高性能。

他的经验说法很有说服力：你的代码在所有你试过的 Intel 处理器上都跑得好，然后你把它移植到 ARM 上，突然就不工作了。

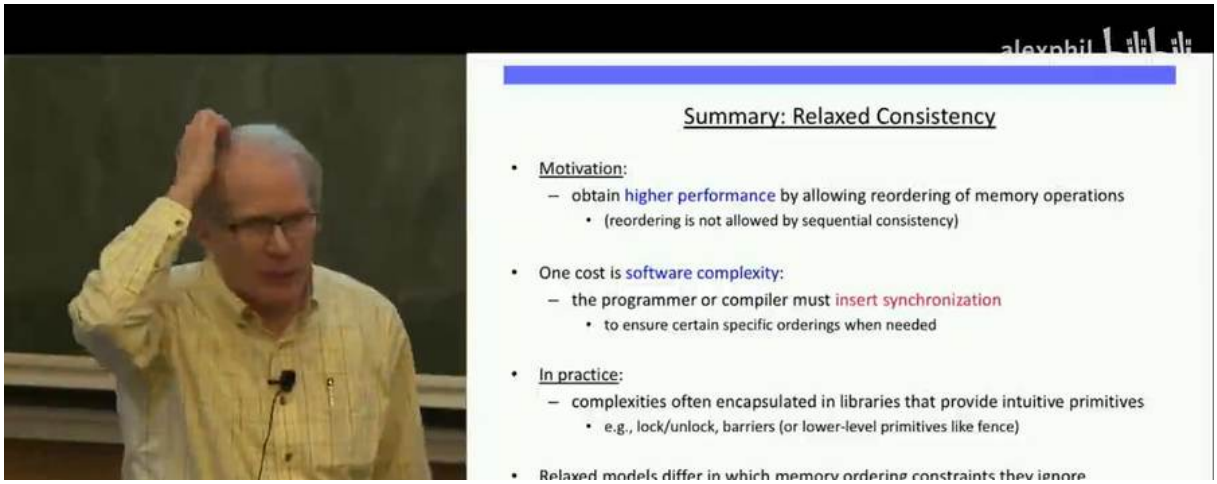

图 374: 放宽一致性的总结：纪律性编程与使用库原语的重要性[374]

[373] 视频画面时间区间：00:50:00–00:50:15。若为裁剪图，时间区间仍对应原视频画面。

[374] 视频画面时间区间：01:08:00–01:08:15。若为裁剪图，时间区间仍对应原视频画面。

由此得到这条建议：编程要有纪律，任何棘手的同步都应当使用现成的库，而不要自己发明。

本章小结

弱序模型把顺序责任交给程序：临界区内顺序无关，但释放锁前必须让更新生效。它要求区分读写共享数据时的假设，并对只读共享场景格外小心。MFENCE 这类栅栏只约束顺序、不负责可见性。模型越宽松性能越好，但跨架移植的风险越大，因此应尽量使用库提供的同步原语。

16.7 总结与延伸

讲者以一句话收束：这就是今天全部内容。

把全讲串起来，这是一条从直觉到反例再到工程规则的路径。”读到最新值”作为定义不可行，因为访存延迟跨越两个数量级、全局时间没有精确刻度；完全无序又无法编程。于是每个系统选择一个模型。用观察者实验可以把顺序一致性表述为可交错性：各线程内部保持程序序，跨线程存在合法交错，单端口内存是它的操作化模型。但一旦涉及不同地址，这个要求在单核上就已不成立：写缓冲区让写”提前完成”、乱序执行让读提前开始，而缓存协议看不到尚未推出的写。于是标志位同步失效，链式依赖也无法补救，(1,0) 与 (0,0) 成为可判定的错误探测器。恢复顺序的办法是把”完成”定义为到达提交点，并逐条放宽约束：TSO 允许读越过写，PSO 进一步放宽写序，停顿时间显著下降。另一条路是弱序模型，把顺序责任交给程序的同步纪律，并用 MFENCE 这类栅栏在关键处恢复顺序。最后，ARM 比 x86 更宽松，代码在 Intel 上正常不代表在 ARM 上正确，因此应当使用库提供的原语。

本讲义在讲者材料之上做的压缩，有四点需要与讲者原话区分：

- 一致性模型不是可有可无的抽象层，它决定了哪些代码写法是可移植的。同一份”看起来正确”的代码在 SC、TSO、ARM 上的表现可以完全不同。因此写并发代码时，目标模型必须作为前提明确下来。
- ”完成”的语义化定义是整套放宽方案的技术枢纽。正因为完成指到达提交点而非物理到达，硬件才能在不破坏模型的前提下重叠操作。理解这一点，才能理解为什么栅栏不必等到写真的落到内存。
- 栅栏与原子操作解决的是不同问题。栅栏约束顺序，原子操作保证不可分割，缓存协议保证单一地址的一致。缺任何一环，同步都可能失效，而三者都齐备时才等价于顺序一致性下的行为。
- 平台差异是实践中的主要风险来源。x86 的 TSO 相对宽松程度有限，很多错误写法恰好在它上面工作；ARM 的弱模型会把这些错误暴露出来。因此”在我机器上是对的”是并发编程中最危险的判断依据之一。

这堂课的边界在于它是概念导论：讲者明确表示内容不会出现在紧接着的考试里，而模型的形式化规则、各架构的具体条文都需要专门查阅。因此本文给出的是判断框架与陷阱地图。

两个可以直接从本讲数据提出的下一步问题：其一，若一份代码要在 x86 与 ARM 上都正确且高效，应当按最强的 SC 写还是按最弱的模型写，两者在性能与可维护性上如何取舍？其二，MFENCE 只排空写缓冲区，那么在一个需要”让其他线程看到我的更新”的场景里，还缺少哪一环，应当如何配合原子操作完成？

Lecture 17 OpenMP & MPI

视频时长：1 小时 14 分 27 秒 来源：https://www.bilibili.com/video/BV1Qj411B7sZ/

17.1 为什么需要消息传递

这一讲安排得很紧凑：先用较快节奏过一遍 MPI，然后回到更多优化实例。讲者说 MPI 本身相当直白，所以没必要在这些事情上花太多时间。

与共享内存的分野

MPI 是消息传递接口，设计初衷是给超算提供一个”靠通信解决问题”的接口。

讲者对比得很清楚：在共享内存中，我们假设处理器可以通过共享内存通信，无论访问是否一致。但如果涉及不同机器，或者 NUMA 远端访问代价太高，我们就不会忽略这一点，而是常常改用消息传递——发一个消息、做大量工作、把结果交回来。

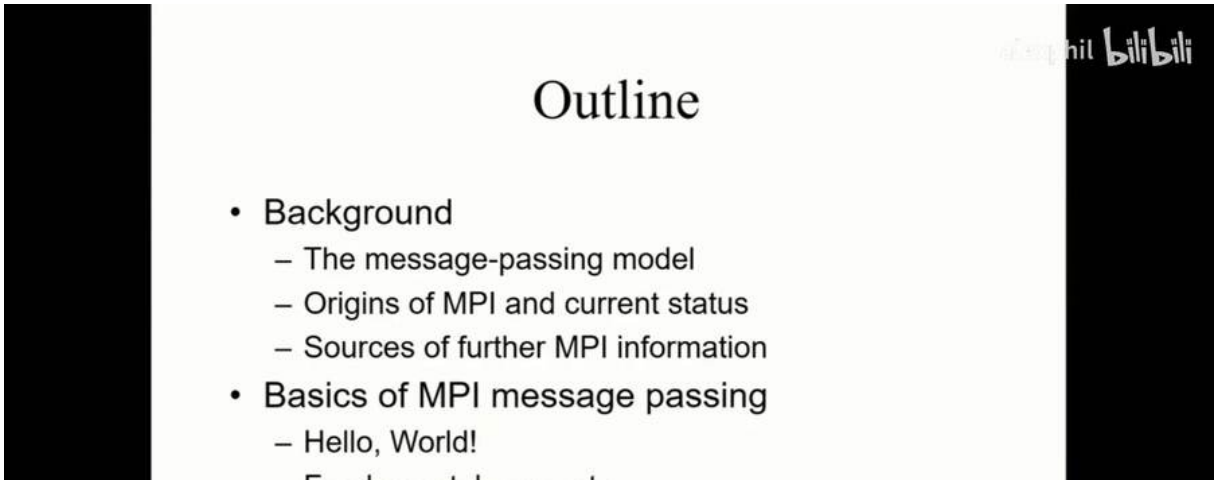

图 375: 本讲提纲：消息传递模型、MPI 的起源与现状，以及优化实例[375]

[375] 视频画面时间区间：00:00:30–00:00:45。若为裁剪图，时间区间仍对应原视频画面。

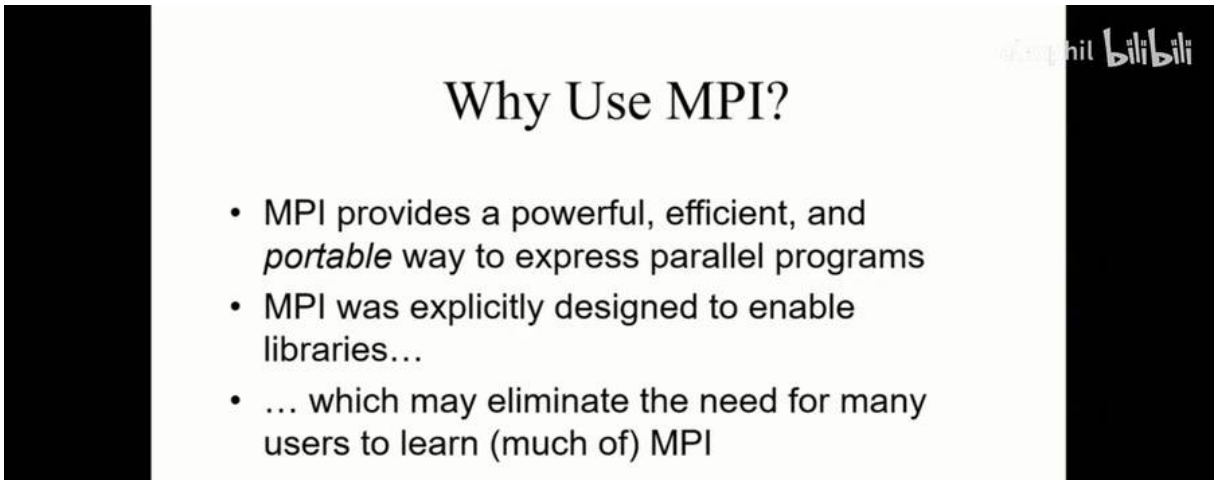

图 376: 为什么用 MPI：一种强大、高效、可移植的并行表达方式[376]

异步的本质与两种选择

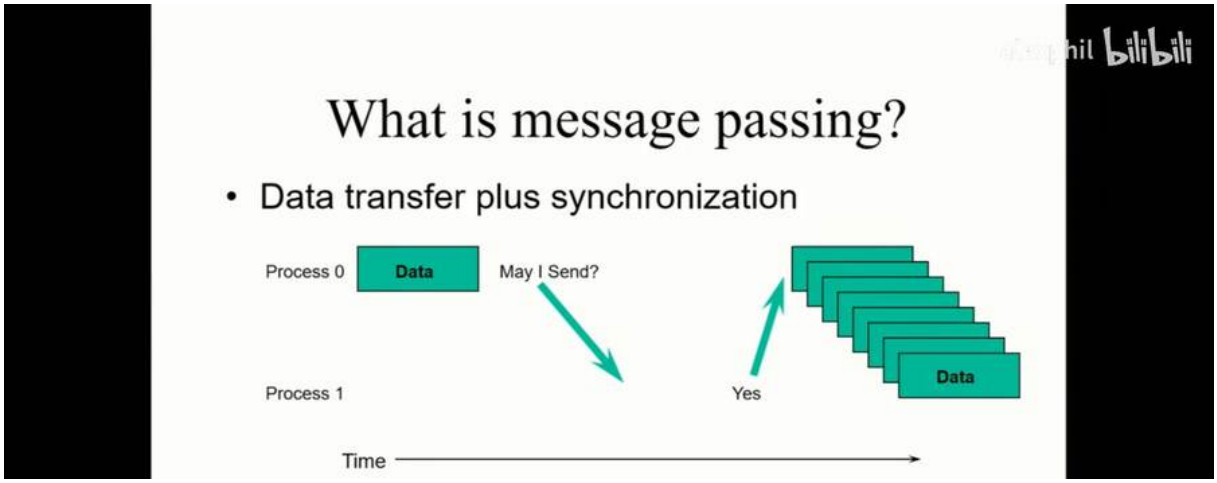

图 377: 什么是消息传递：数据传输加同步[377]

讲者强调消息传递本质上是异步的：发送方发消息给接收方，接收方把消息排队。然后我们有一个选择。

选项 A：作为发送方，我们等待接收方收到、处理、产生某种反应，并把回复发回来。

选项 B：发出即忘。我们把消息发出去，回去做其他正在做的事情，如果在意的话就周期性查看回复。

轮询的代价与频率是一对权衡

周期性检查回复比一直等待更灵活，但每次检查都要付一次系统调用或网络栈开销。检查太频繁会浪费 CPU，太稀疏则会错过及时响应。因此轮询间隔应与预期消息到达的粒度匹配。

他给了一个很贴切的类比：你可以寄一封信，然后每天站在信箱旁等回信；也可以寄出去，然后每天早上查一次邮箱，白天该干什么干什么。

[376] 视频画面时间区间：00:03:00–00:03:15。若为裁剪图，时间区间仍对应原视频画面。

[377] 视频画面时间区间：00:06:00–00:06:15。若为裁剪图，时间区间仍对应原视频画面。

阻塞与非阻塞不是风格差异而是结构差异

阻塞通信把”等待”变成调用者的状态，因此调用者在这段时间内无法做任何其他事。非阻塞通信把”等待”变成可查询的状态，调用者可以在等待期间计算。这个差别决定了程序能否重叠通信与计算，也就决定了通信延迟是否被隐藏。

本章小结

MPI 是为超算设计的消息传递接口，用于处理器无法靠共享内存廉价通信的场合。消息传递本质异步：发送方发出、接收方排队，之后可选择等待回复或发出即忘。这个选择对应阻塞与非阻塞两套语义，而它决定通信延迟能否被计算掩盖。

17.2 MPI 是什么：规范而非库

规范与实现

图 378: 最小的 MPI 程序：只做初始化与关闭[378]

讲者明确一点：MPI 本身是一个接口，不是库。它是一份规范，由许多不同的库在不同系统上实现。

规范的抽象边界决定了可移植性的范围

MPI 规范只保证调用语义，不保证性能特征。因此同一份程序在不同实现上正确性一致，但延迟与带宽可能差数倍。跨平台调优时必须重新测量，不能沿用旧参数。

因此它规定通信是什么样子，但不规定你怎么实现。

[378] 视频画面时间区间：00:03:30–00:03:45。若为裁剪图，时间区间仍对应原视频画面。

最小程序与错误处理

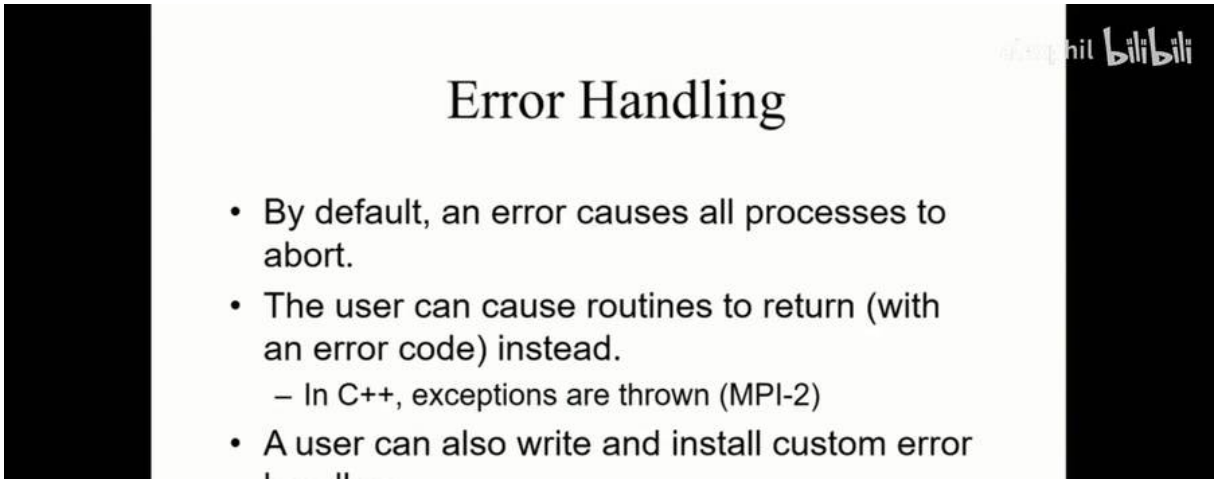

图 379: 错误处理：默认出错使所有进程终止，也可返回错误码[379]

最小的 MPI 程序只做两件事：初始化 MPI 库，然后关闭。

讲者打趣说，不要指望这个程序带来显著性能提升。但他可以保证：它会给你和第一次跑 Hello World 一样多的即时满足感——你会说「好，MPI 能跑了」，然后呢?

默认终止语义对长时运行作业不友好

默认出错即全体终止，意味着一个进程的局部错误会毁掉整个作业。在需要长时间运行的场景里，通常改为返回错误码并由程序自行决定是否降级或重试，但这要求每个调用点都检查返回值。

关于错误处理：默认情况下出错会让所有进程终止。用户可以改为让例程返回错误码。讲者补充说，若你用 Fortran，通常返回错误码；若用 C++，会得到异常。

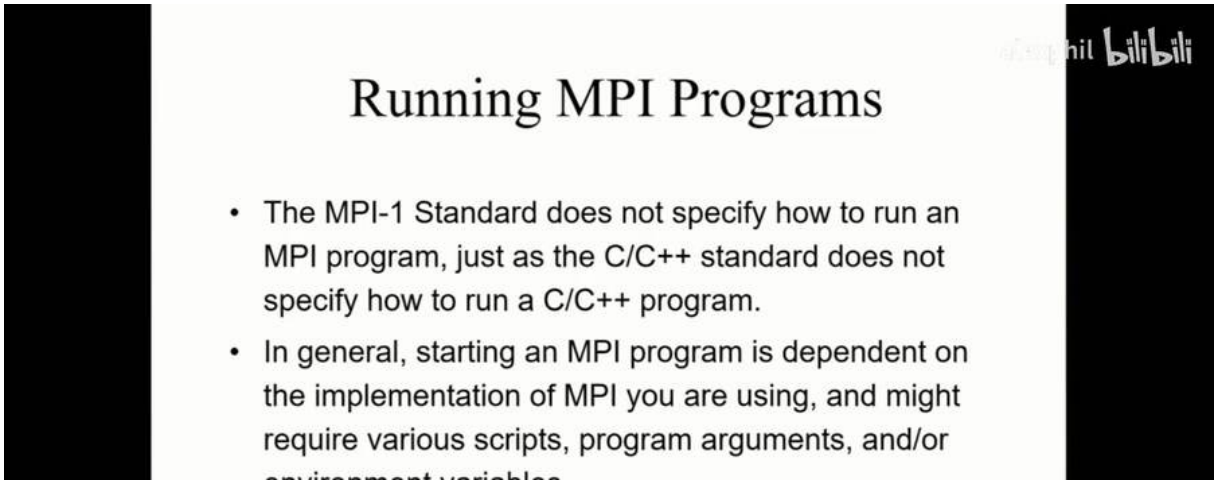

图 380: 运行 MPI 程序：标准不规定运行方式[380]

[379] 视频画面时间区间：00:04:00–00:04:15。若为裁剪图，时间区间仍对应原视频画面。

[380] 视频画面时间区间：00:04:30–00:04:45。若为裁剪图，时间区间仍对应原视频画面。

本章小结

MPI 是规范而非库，只规定通信语义不规定实现。最小程序只做初始化与关闭，它的价值是确认环境可用而非提升性能。错误处理可选返回码或异常，默认出错则全体终止。

规范之外还有一层：程序实际如何被启动与连接。

17.3 运行环境与启动方式

为什么没有标准环境

讲者指出一个容易被忽略的事实：运行 MPI 程序没有标准化的环境。这就像运行 C++ 程序也没有标准化环境一样——那不是接口或库的函数，而是运行时环境的函数。

他提到有建议说：若运行 MPI 程序，命令应叫 `mpiexec`，然后接程序名。这样会调用运行时，由它处理所有线程与已初始化的通信设施。

但这只是建议，并非所有 MPI 系统都必须这样。

规范与运行时的分工

MPI 规范定义的是”调用语义”，而进程如何启动、如何发现彼此、如何建立连接属于运行时实现。因此同一份 MPI 程序在不同集群上可能需要不同的启动命令与参数。移植时出问题往往在运行时层，而不在 API 层。

本章小结

MPI 是规范而非库，只规定通信语义不规定实现。最小程序只做初始化与关闭；错误处理可选返回码或异常。运行 MPI 程序没有标准环境，`mpiexec` 只是建议。规范与运行时的分工解释了为什么移植问题常出在运行时层。

17.4 基本概念：通信器、类型与标签

通信器与 rank

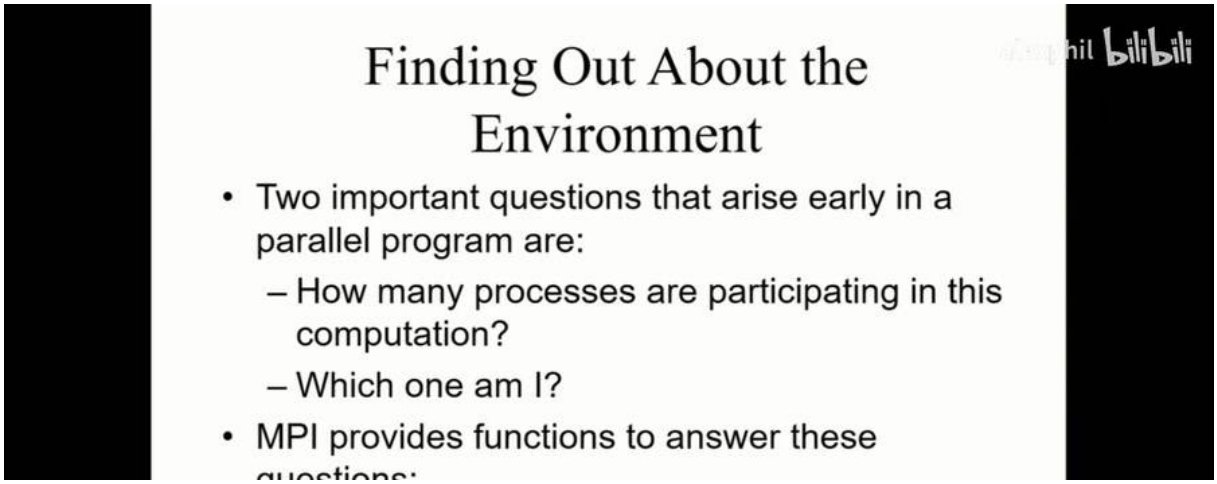

图 381：了解运行环境：并行程序早期要问的两个问题[381]

[381] 视频画面时间区间：00:05:00–00:05:15。若为裁剪图，时间区间仍对应原视频画面。

图 382: 改进版 Hello：获取组内进程数与自身 rank[382]

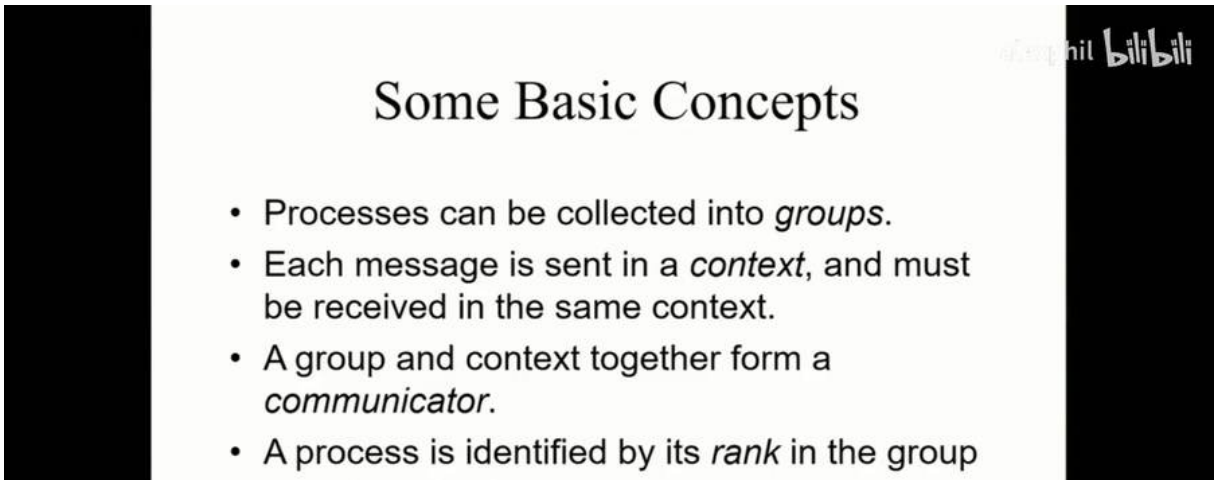

图 383: 基本概念：进程组成组，每条消息在通信器上下文中发送[383]

rank 是逻辑标识而非物理位置

rank 只表示进程在通信器内的编号，不携带拓扑信息。因此”rank 0 在第一个插槽” 这类假设不可移植。要获得拓扑信息，需要使用专门的拓扑查询接口或由启动器显式指定映射。

讲者说并行程序早期会问两个重要问题：有多少个进程？我是哪一个？

进程由它在通信器中的 rank 标识。有一个默认通信器，其组包含所有初始进程。这个默认通信器的名字是 `MPI_COMM_WORLD`。初始化 MPI 后你就属于它，除非你加入别的通信器。

通信器这个概念同时捕获了两件事：组成员关系，以及在组内通信的能力。讲者说这两件事名义上是分开的，但通信器把上下文与通信能力合并在一起。

[382] 视频画面时间区间：00:05:30–00:05:45。若为裁剪图，时间区间仍对应原视频画面。
[383] 视频画面时间区间：00:07:00–00:07:15。若为裁剪图，时间区间仍对应原视频画面。

通信器既是名字空间也是隔离域

同一个 rank 编号在不同通信器中指不同进程。因此通信器提供了消息隔离：在一个通信器里发的消息不会被另一个通信器的接收误取。这使得库可以安全地使用私有通信器，而不干扰调用者的消息。

数据类型

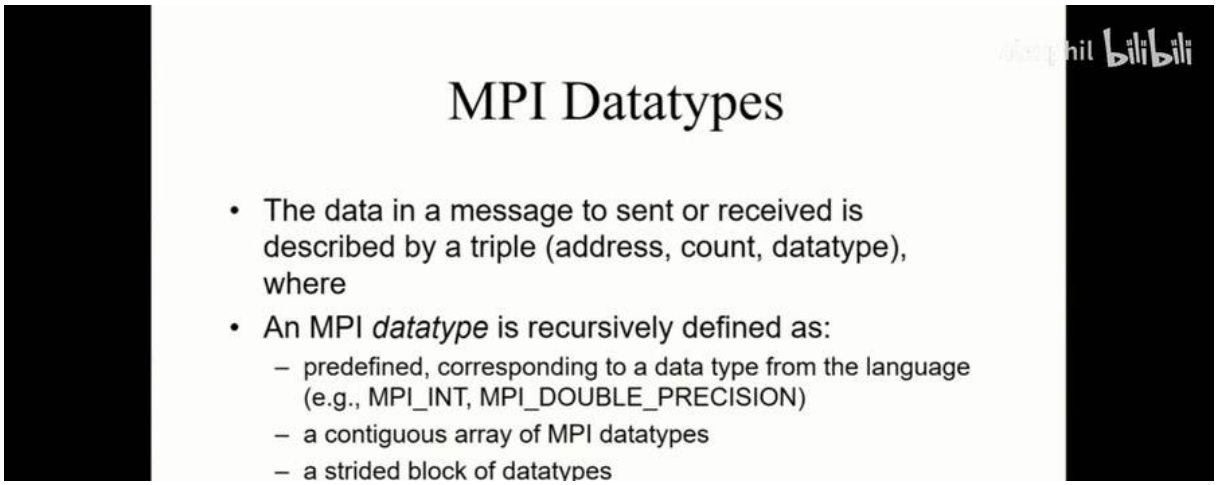

图 384: MPI 数据类型：由地址、计数与类型三元组描述消息数据[384]

因为 MPI 设计为在异构环境中工作，它有自己的一套数据类型：`MPI_INT`、`MPI_DOUBLE` 等等。

使用 MPI 数据类型带来两个好处：一是可以自由跨不同类型的系统工作，二是我们确切知道数据有多大。

类型标准化是异构通信的前提

若发送方与接收方的 int 宽度不同，直接按字节复制会得到错位的数据。MPI 的类型描述让运行库能在两侧之间做转换。因此通信时不应传裸类型的大小，而应传对应的 MPI 类型。

讲者点出一个 C 语言的经典问题：如果你说 `int`，人们会问这个 int 有多大。MPI 通过标准化数据类型、表示与大小解决了这个问题。

[384] 视频画面时间区间：00:08:00–00:08:15。若为裁剪图，时间区间仍对应原视频画面。

标签

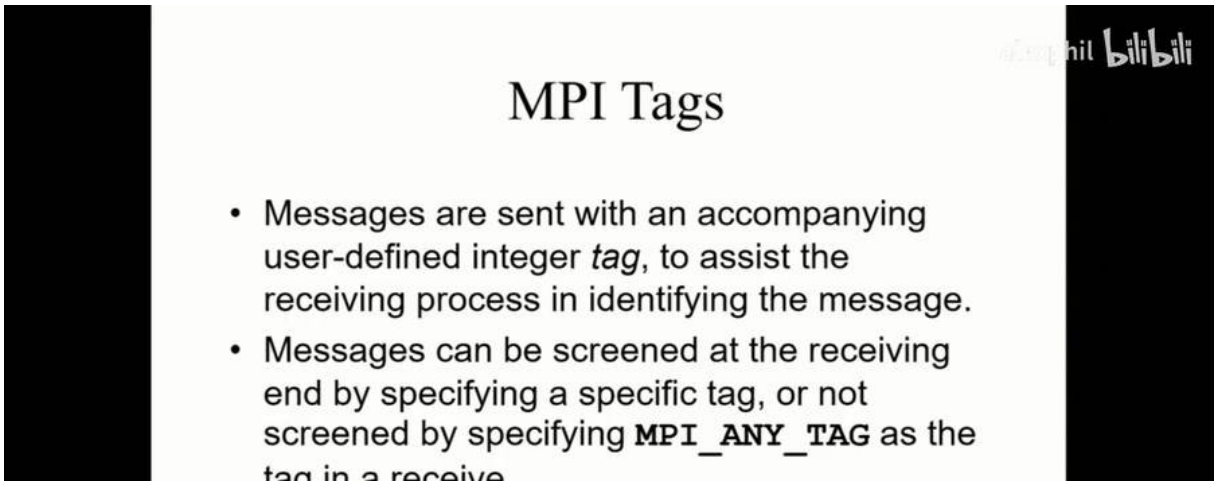

图 385: MPI 标签：用用户定义的整数标记消息类型[385]

MPI 标签曾经被用得很多。有人把它叫消息类型：发消息时可以打上标记说明这是哪类消息，接收方据此决定如何处理，从而实现对消息类型的内部组织。

讲者说标签现在仍然可以用。

本章小结

通信器同时捕获组成员关系与通信上下文，进程由 rank 标识，默认通信器含全部初始进程。MPI 自带数据类型以支持异构环境并确知大小。标签用于标记消息类型，接收方据此决定处理方式。

17.5 阻塞与非阻塞通信

发送接收的基本困难

讲者先描述经典问题：发送方想发数据，接收方可能还没准备好；接收方在等东西，发送方可能还没发。存在突发流量时发送方发了很多而接收方尚未就绪；也有接收方闲着而发送方还没准备好的时候。

缓冲就在中间起作用：暂存这些突发，在接收方就绪时把它们排空。

所需缓冲深度与发送速率和接收速率的差值有关：

$$B_{\min} \approx (r_{\text{send}} - r_{\text{recv}}) \cdot \Delta t_{\text{burst}}$$

- r_{send}：发送方突发期间的发送速率；
- r_{recv}：接收方在同期能消费的速率；
- Δt_{burst}：突发持续时间；
- 差值乘以时长即为需要暂存的字节数。

若缓冲不足，发送方就被迫阻塞，非阻塞语义随之退化。

[385] 视频画面时间区间：00:08:30–00:08:45。若为裁剪图，时间区间仍对应原视频画面。

阻塞通信

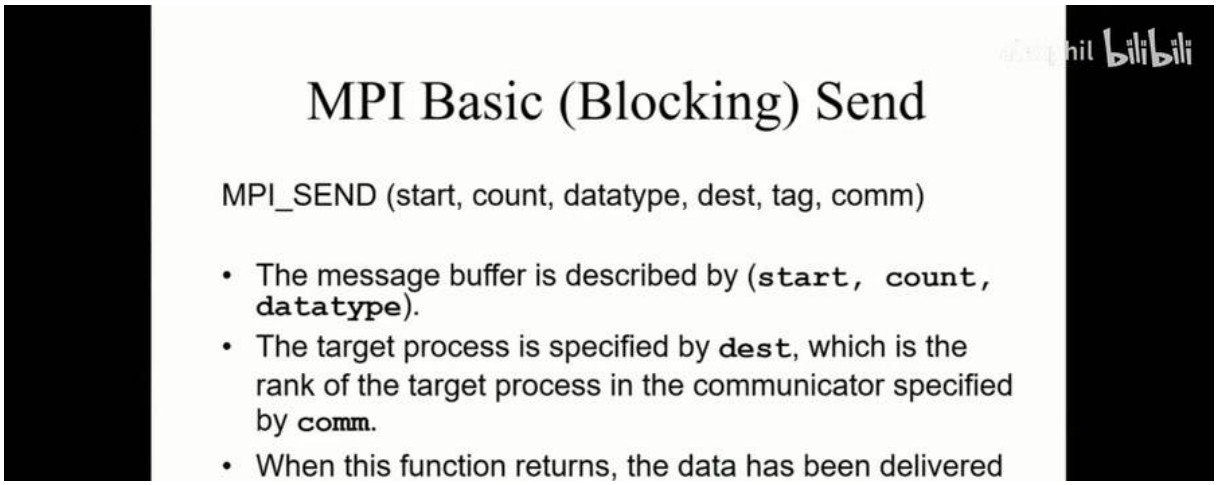

图 386: MPI 基本（阻塞）发送：`MPI_Send` 的参数[386]

讲者说阻塞通信很简单：发出、等确认、然后你知道宇宙的状态并继续。

问题在于它把本可并行的事情串行化了。有些东西本可以继续推进，却被冻结在那里等别人行动。

阻塞调用的代价可以用等待时间占循环总时间的比例来衡量：

$$f_{\text{stall}} = \frac{T_{\text{wait}}}{T_{\text{wait}} + T_{\text{compute}}}$$

- T_{wait}：每次阻塞调用中实际等待对端的时长；
- T_{compute}：同一轮中可与之重叠的计算时长；
- 若 $T_{\text{wait}} \gg T_{\text{compute}}$，则绝大部分时间被冻结；
- 非阻塞形式的目标就是让 T_{compute} 覆盖 T_{wait}，使 $f_{\text{stall}} \to 0$。

阻塞通信的代价在等待期间被放大

一次阻塞发送可能只等几微秒，但若在循环中反复调用，累计等待会吃掉大量本可用于计算的时间。因此在高性能代码中，阻塞调用常被移出热循环，或改用非阻塞形式批量提交、最后统一等待。

[386] 视频画面时间区间：00:09:00–00:09:15。若为裁剪图，时间区间仍对应原视频画面。

非阻塞通信

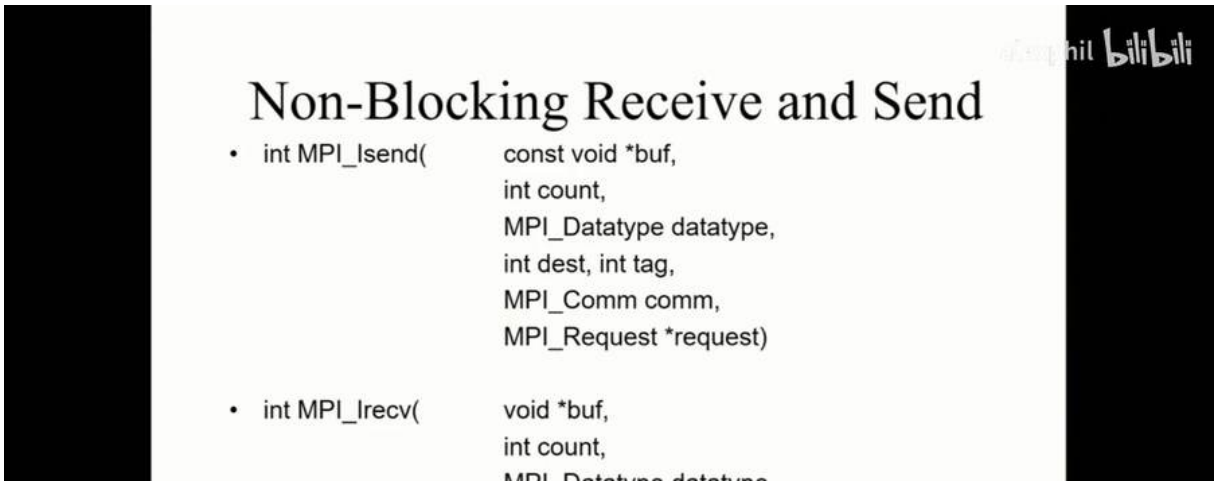

图 387: 非阻塞接收与发送：`MPI_Isend` 等接口[387]

非阻塞发送与接收放宽了这个模型。

非阻塞发送：你发出去，它被排队，然后你继续干自己的事。

非阻塞接收：你请求某个东西，要么拿到、要么还没有，但你不阻塞。

于是我们可以这样做：查看东西是否到了，然后回去做别的工作；再查看，再做别的工作。这就允许我们把计算重叠到等待时间里。

wait、test 与 probe

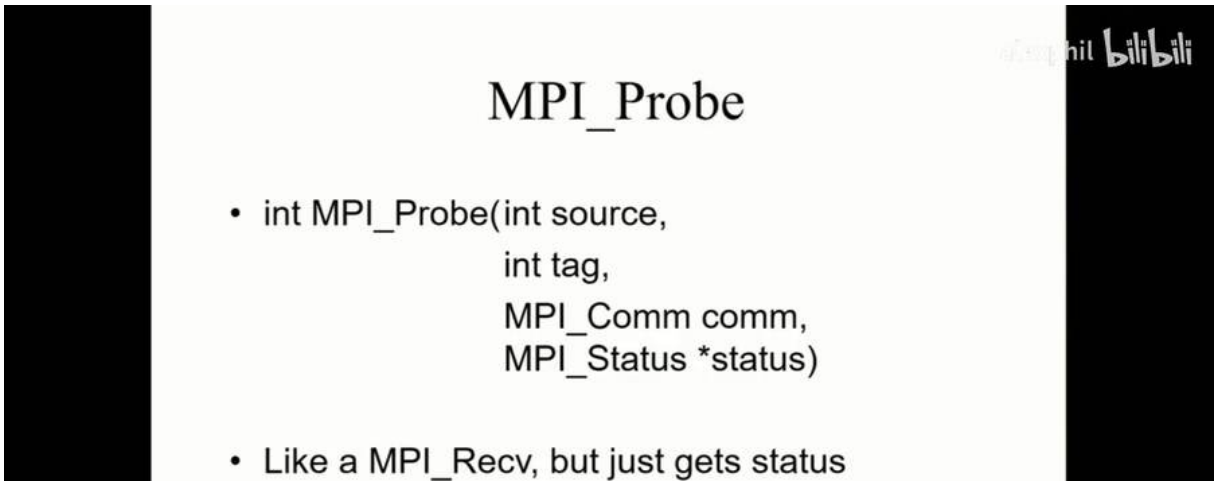

图 388: `MPI_Probe`：只取状态，不取数据[388]

讲者说明处理非阻塞操作的两个手段。`MPI_Wait` 与 `MPI_Test`：若我们有一个非阻塞操作，而现在想等它——因为能做的别的事都做完了——就调用 wait，此时会阻塞直到完成。

若用 test 并拿到状态，它可以告诉我们比如这次通信是否失败。

[387] 视频画面时间区间：00:10:30–00:10:45。若为裁剪图，时间区间仍对应原视频画面。

[388] 视频画面时间区间：00:12:30–00:12:45。若为裁剪图，时间区间仍对应原视频画面。

Probe 解决”不知道要收多少”的问题

接收方常需要先知道消息长度才能分配缓冲区。Probe 只读取消息头（来源、标签、大小），不消费消息体，因此可以先分配再接收。这是消息传递中处理变长消息的标准手法。

`MPI_Probe` 像一个接收，但只取状态、不取数据。讲者说它像是”偷看一眼”。

$$\text{进度} = \begin{cases} \text{MPI_Test} & \text{非阻塞，立即返回是否完成} \\ \text{MPI_Wait} & \text{阻塞直到完成} \\ \text{MPI_Probe} & \text{只看有没有消息，不取走} \end{cases}$$

- `MPI_Test`：适合在计算之间周期性检查，不打断计算流；
- `MPI_Wait`：适合已无事可做时，把剩余等待交给系统；
- `MPI_Probe`：适合先得知消息大小与来源，再决定分配多少缓冲区。

非阻塞通信有一个典型的配套用法，值得单独说明：先把若干发送与接收全部以非阻塞形式提交，在中间插入计算，最后统一调用 wait 等待全部完成。这样一次等待就覆盖了多个通信，而不是逐个阻塞。代价是需要为每个未完成操作保存一个请求句柄，且缓冲区在所有操作完成前不能复用。这个模式在需要交换边界数据的结构化网格应用中很常见：先发起所有邻居的发送与接收，再计算内部区域，最后等待边界数据到达。它把通信延迟完全藏在了内部计算之后。

本章小结

发送方与接收方的就绪时间不匹配，缓冲用于吸收突发。阻塞通信简单但把可并行的部分串行化。非阻塞发送入队即返回、非阻塞接收不阻塞，因此可把等待时间用于计算。`MPI_Wait` 阻塞等待完成、`MPI_Test` 非阻塞查询、`MPI_Probe` 只取状态不取数据。

17.6 集合操作与”MPI 其实很简单”

六个函数覆盖九成场景

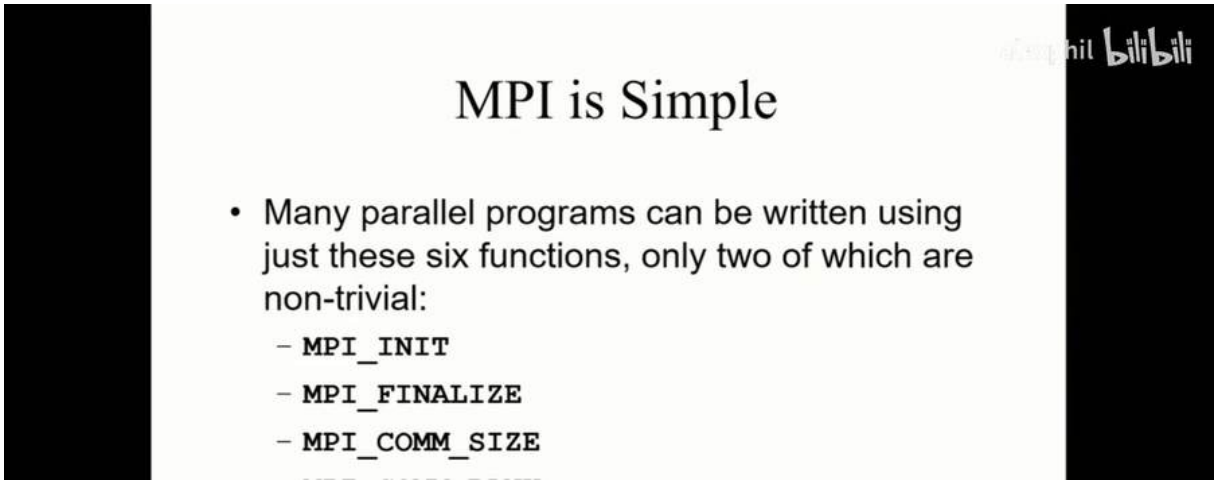

图 389: MPI 其实很简单：六个基本函数足以写多数并行程序[389]

讲者给出一个很有说服力的判断：许多并行程序只用这六个函数就能写出来，它们大概覆盖了我们会用到的 90%。

若再加上两个非阻塞操作与 probe，大约九个函数。他说这就是 MPI 的全部。

API 简单不等于并行化简单

六个函数足以表达多数通信模式，但”如何把问题切成可通信的块、如何安排通信与计算的重叠、如何避免某个进程成为瓶颈”这些问题无法由 API 解决。因此学习 MPI 的成本主要不在语法，而在分解策略。

而 MPI 的诀窍不在于怎么用这些库，而在于怎么解决问题。

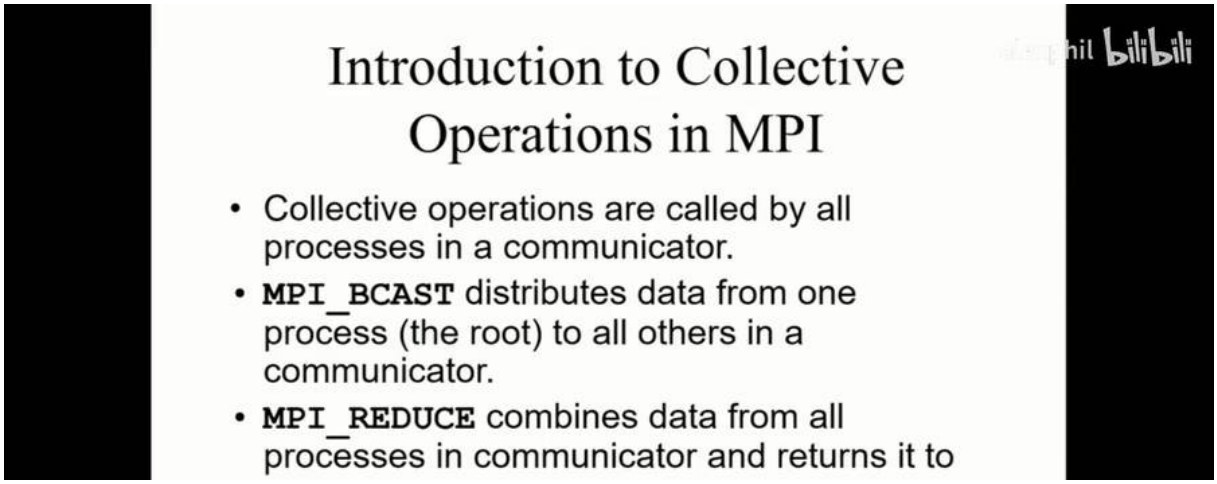

图 390: 集合操作入门：由所有进程共同调用[390]

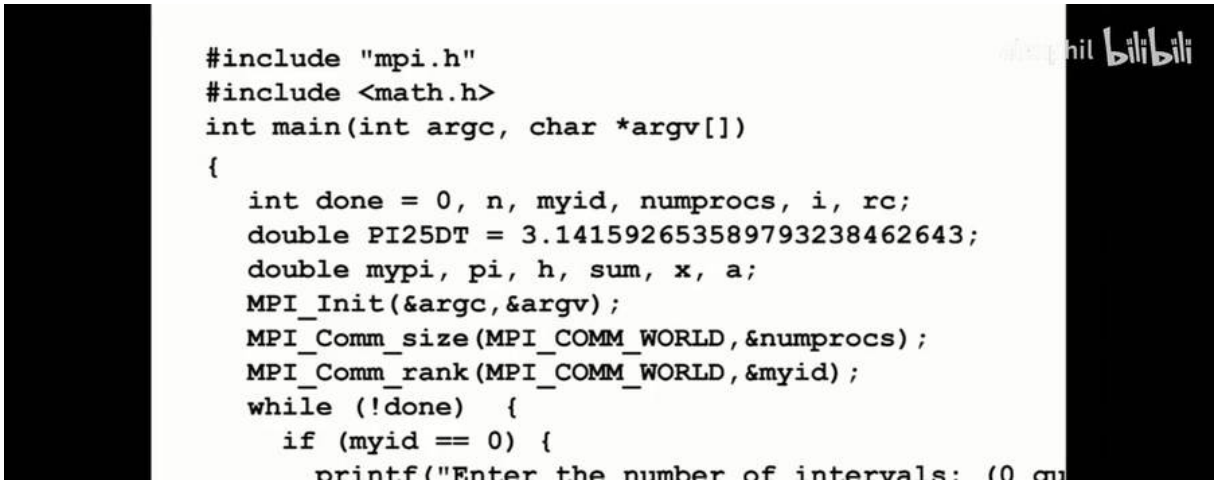

图 391: 示例：用 C 语言算圆周率，主从结构[391]

[389] 视频画面时间区间：00:13:30–00:13:45。若为裁剪图，时间区间仍对应原视频画面。
[390] 视频画面时间区间：00:14:30–00:14:45。若为裁剪图，时间区间仍对应原视频画面。
[391] 视频画面时间区间：00:15:00–00:15:15。若为裁剪图，时间区间仍对应原视频画面。

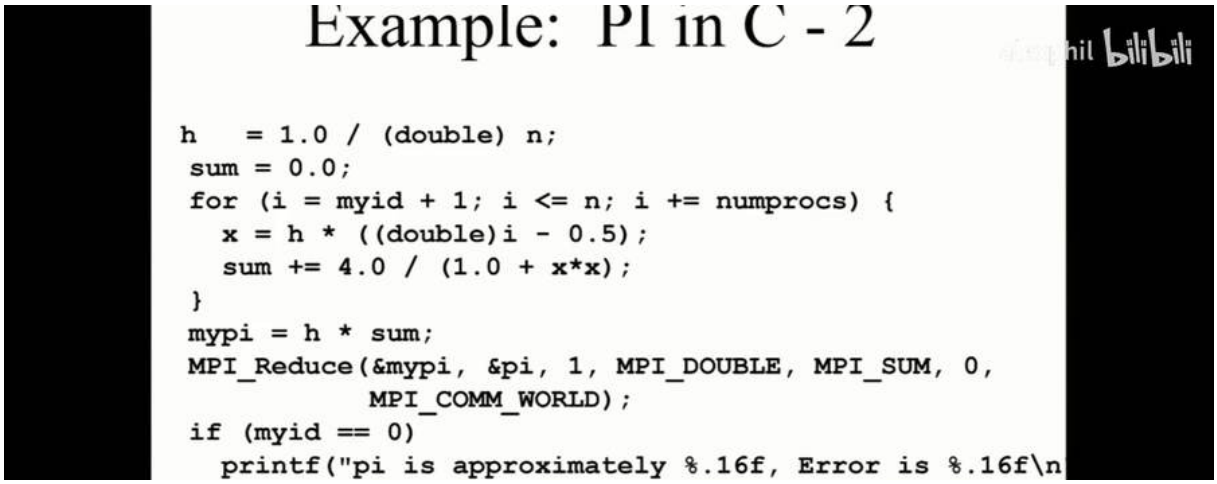

图 392: 圆周率示例续：各进程分片累加再归约[392]

软同步还有一个实际好处：它天然适合流水线式的分解。若把问题切成若干阶段，让不同进程分别处理不同阶段的数据块，那么每个进程只需在把结果交给下游时同步一次。上游不必知道下游内部如何实现，下游也不必关心数据从哪来。这种松耦合让系统更容易扩展——增加一级处理只需在链上插入一个新进程并修改相邻两级的通信端点，而不必改动整个程序的结构。共享内存方案要做到同样的扩展性，通常需要重新设计锁与数据布局。

本章小结

六个基本函数覆盖约九成使用场景，加上非阻塞与 probe 共约九个。集合操作由所有进程共同调用，用于广播、归约与扫描等集体模式。圆周率示例展示了典型的主从结构：各进程分片累加，再归约汇总。而 MPI 的诀窍不在 API 用法，而在问题分解。

最后一个概念值得单独讨论：消息传递改变了同步发生的位置。

17.7 软同步：同步发生在消息层

按通信而非按数据同步

讲者最后给出一个概念性的判断，值得单独记录。

消息传递是一种很好的解决问题的方式，因为它允许一种”软同步”：一个进程可以做一堆工作，做完后发一条消息。

于是你是在通信上同步，而不是在数据上同步。你不共享内存，因此不会为了抢同一个东西而堆积。你取到某个东西、处理它、把结果发出去。

软同步把”等待”从数据层移到消息层

共享内存的同步发生在数据上：多个线程抢同一块内存，必须用锁或原子操作协调。消息传递的同步发生在消息上：进程各自持有私有数据，只在发送与接收时对齐。因此消息传递天然避免了数据竞争，代价是必须显式搬运数据。

[392] 视频画面时间区间：00:15:30–00:15:45。若为裁剪图，时间区间仍对应原视频画面。

本章小结

六个基本函数覆盖约九成使用场景，加上非阻塞与 probe 共约九个；难点不在 API 而在问题分解。消息传递提供软同步：干完活再发消息，按通信同步而非按数据同步，因此不共享内存、不会为同一目标堆积。

17.8 总结与延伸

讲者在结尾处理了一些课程事务与反馈，随后结束。

把全讲串起来，这是一份关于”当处理器之间不能共享内存时如何表达并行”的速览。MPI 是为超算设计的消息传递接口，用于共享内存代价过高或跨机器的场合；它的模式是发消息、做大量工作、交回结果。消息传递本质异步：发送方发出、接收方排队，之后可选择等待回复（阻塞）或发出即忘（非阻塞），后者允许把计算重叠到等待时间。MPI 是规范而非库，只规定通信语义不规定实现；运行 MPI 程序没有标准环境，`mpiexec` 只是建议。通信器同时捕获组成员关系与通信上下文，进程由 rank 标识，默认通信器含全部初始进程；MPI 自带数据类型以支持异构环境并确知大小；标签用于标记消息类型。发送方与接收方就绪时间不匹配，缓冲用于吸收突发。阻塞通信简单但串行化本可并行的部分；非阻塞发送入队即返回、接收不阻塞，配合 `MPI_Wait`、`MPI_Test` 与 `MPI_Probe` 可以灵活控制。最后，六个基本函数覆盖约九成场景，而消息传递提供软同步——按通信同步而非按数据同步。

本讲义在讲者材料之上做的压缩，有四点需要与讲者原话区分：

- 消息传递与共享内存的差别不只是”能不能共享数据”，而是”同步发生在哪一层”。共享内存同步数据，消息传递同步消息。这个区别决定了竞争与死锁的形态完全不同。
- MPI 是规范而非库，这一点有实际后果。同一份程序在不同集群上可能需要不同的启动方式与参数，移植问题常出在运行时层而非 API 层。
- 非阻塞不等于更快，而是更可控。它把等待变成可查询的状态，但需要程序自己安排”什么时候检查”。若检查得太晚，收益会被浪费；若检查得太频繁，开销会抵消收益。
- 软同步的代价是显式数据搬运。消息传递避免了锁与数据竞争，但每次通信都要复制数据。因此它对”计算量相对通信量大”的问题最有利，这与第 23 讲片上与外接加速器的判据同构。

这堂课的边界在于它是速览：讲者明确说 MPI 本身很直白，因此只花很短时间过一遍，随后转入优化实例。因此本文不含完整的 MPI 调用参考与集合操作的细节语义。

两个可以直接从本讲数据提出的下一步问题：其一，非阻塞通信要求程序安排检查时机，那么检查频率应当如何与计算粒度匹配才不浪费收益？其二，软同步避免了数据竞争，但显式搬运数据会带来复制开销，在什么计算通信比之下消息传递会优于共享内存？

Lecture 18 Performance Measurement and Tuning

视频时长：1 小时 16 分 42 秒　来源：https://www.bilibili.com/video/BV1SX4y1W7fG/

18.1 “我的程序很慢”：先弄清楚是哪一种慢

这堂由 Kayvon Fatahalian 讲授的课是一个补充专题。他开场描述了一个他经常遇到的场景：学生走进办公室说，我的代码很慢、好像用了很多内存、被 SIGKILL 杀掉了。

他的第一句话是：有些情况下答案是去用调试器——但那只在 SIGKILL 这类情形下偶尔有效，而 SIGKILL 往往正是因为内存用得太多。

测量比猜测有价值

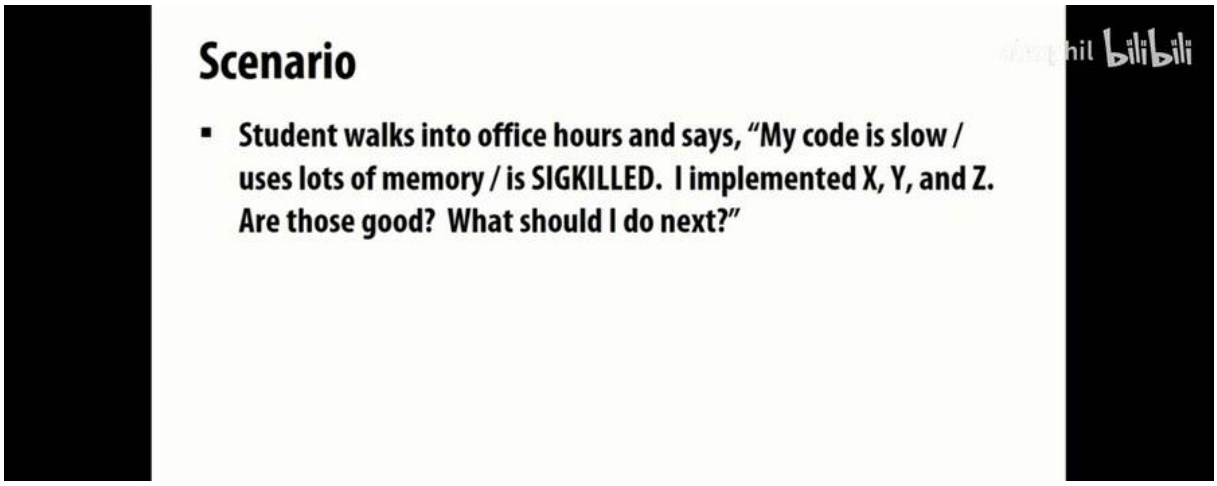

图 393: 场景：学生走进办公室说” 我的代码很慢、用了很多内存、被 SIGKILL 了”[393]

讲者给出本讲的核心命题：

图 394: 我的程序在做什么：测量比直觉更有价值[394]

[393] 视频画面时间区间：00:02:00–00:02:15。若为裁剪图，时间区间仍对应原视频画面。

”慢”是一个症状，不是诊断

程序慢可能意味着：机器被别人的进程占满、内存不足导致换页、某个函数占了七成时间、缓存缺失率过高、或者某处数据布局导致每次访问都多走一级指针。这五种情况的解法完全不同，而仅凭”慢”这一个观察无法区分。工具的作用是把症状转化为可定位的指标。

本讲的例子来自具体学期

讲者说明他的幻灯片基于 2016 年春季学期的作业三，当时是图处理负载：图中有数百万到数千万个节点，代码跑在 GHC 机器与 Xeon Phi 上。他坦承现在这门课的作业三换成了鼠群模拟，因此部分幻灯片只能说”当年是这样”。

他也提到自己 2000 年秋季就在这个教室上过课，那是他来 CMU 的第一堂课。这段插曲之后他转入正题。

本章小结

”我的程序慢”是一个症状而非诊断，可能对应机器被占用、内存不足、函数级热点、缓存缺失、数据布局等多种原因。工具的价值在于把症状转成可测量的指标。本讲的例子来自 2016 年春季的图处理作业。

下一章从最外层开始：先确认机器上还有没有别人。

18.2 第一层：机器上还有谁在跑

top

讲者给出的第一个工具就是 top。他描述了一次真实的困惑：SSH 进一台机器，看到有 50 个用户在线，于是怀疑是不是同学们都在跑代码。但 50 个人都登录同一台机器、且都恰好空闲地盯着屏幕，这不太可能——所以大概是有人在跑。

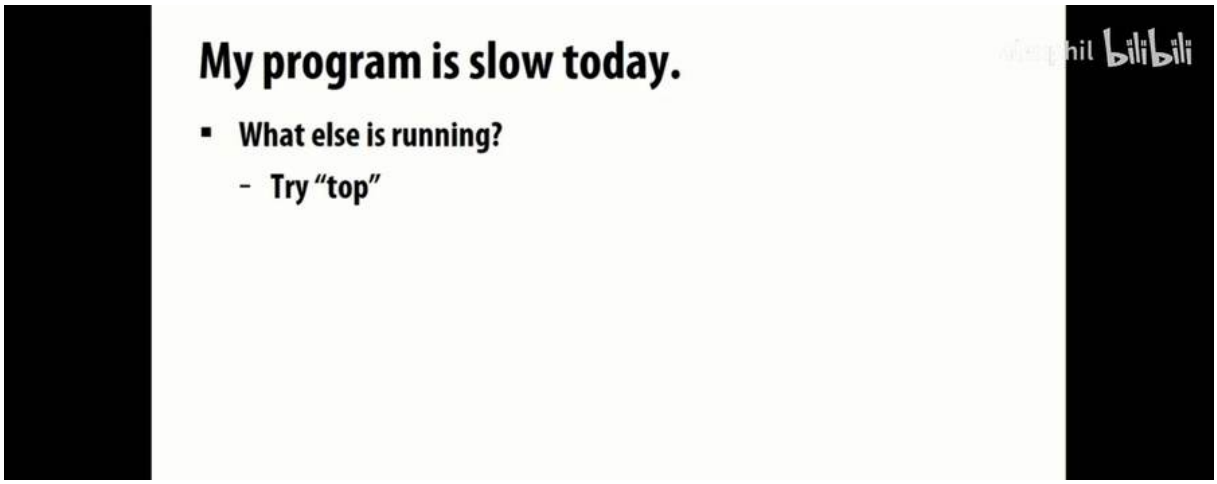

图 395：机器上还有什么在跑：先用 top 看整体负载[395]

[394] 视频画面时间区间：00:03:30–00:03:45。若为裁剪图，时间区间仍对应原视频画面。
[395] 视频画面时间区间：00:06:30–00:06:45。若为裁剪图，时间区间仍对应原视频画面。

结果那一次并不是。top 显示 99.9% 空闲，这台机器基本闲置。于是可以判断：慢的原因确实在自己的代码里。

top 还能看内存：机器共约 16 GB，空闲约 8 GB。

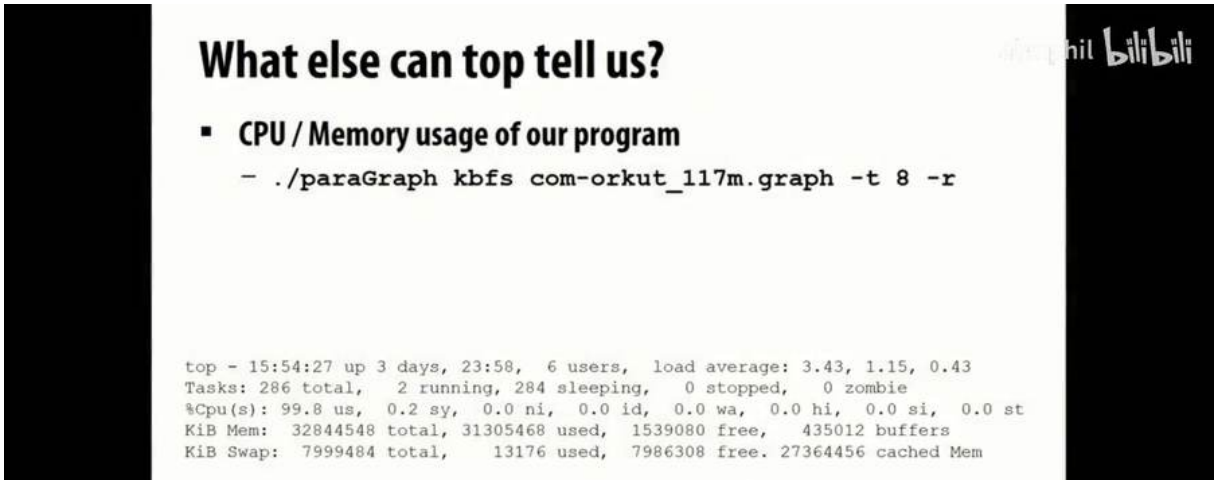

图 396: top 还能告诉我们什么：程序的 CPU 与内存使用[396]

那个让人困惑的百分比

讲者在这里专门停下来讲一个陷阱。他跑图处理负载并指定八个线程，top 显示他用了接近 800% 的 CPU。

讲者还提到另一次运行使用了 600% 的 CPU，那一次耗时 33 秒。这类数字经常引起争论。他提到关于” 百分比怎么加” 存在长期争论：这台机器有八个硬件线程，当代码用满八个线程时，你用的是系统的 100%，还是 800%？连工具本身都不一致——同一屏上系统整体 CPU 使用率显示 99%，而他的程序显示 796%。

图 397: top 的 CPU 百分比口径：按逻辑线程累加，八线程满载即 800%[397]

讲者的解释很清楚：这个数字告诉我们” 聚合来看有 99% 的 CPU 在被使用”，而恰好是他一个人用掉了相当于八个 CPU 的负载。

[396] 视频画面时间区间：00:08:30–00:08:45。若为裁剪图，时间区间仍对应原视频画面。
[397] 视频画面时间区间：00:10:00–00:10:15。若为裁剪图，时间区间仍对应原视频画面。

$$\mathrm{CPU\%}_{\mathrm{program}} \approx N_{\mathrm{threads}} \times 100\%$$

- N_{threads}：该程序实际使用的硬件线程数，本例为 8；
- 因此显示 796% 意味着接近八核满载；
- 系统整体百分比则是所有进程的聚合，上限是 100%。

读 top 前必须先确认百分比口径

不同工具、不同版本对 CPU 百分比的定义并不统一：有的按单核为 100%，有的按整机为 100%。看到 200% 时先问自己” 这台机器有几个硬件线程”，否则会把满载误读为异常，或把 25% 的利用误读为充分利用。

一定要用 top 吗

图 398: 一定要用 top 吗：作业三的计时也是评分的一部分[398]

讲者的回答是不一定。作业三本身就包含计时，提交时会跑并给出性能评分。因此也可以直接用评测脚本的输出，而不必手工盯着 top。

本章小结

第一层是系统级：用 top 确认机器是否被他人占满、内存是否充足。这里有一个必须警惕的细节——CPU 百分比的口径按逻辑线程累加，八线程满载会显示 800% 而非 100%。

确认了机器没问题，就可以转向” 我的代码里时间花在哪”。

18.3 第二层：函数级计时

插桩的基本问题

讲者先给出一个概念框架：插桩的核心问题是” 在什么时候注入测量”。可以在编译时注入，也可以在运行时注入。

[398] 视频画面时间区间：00:10:30–00:10:45。若为裁剪图，时间区间仍对应原视频画面。

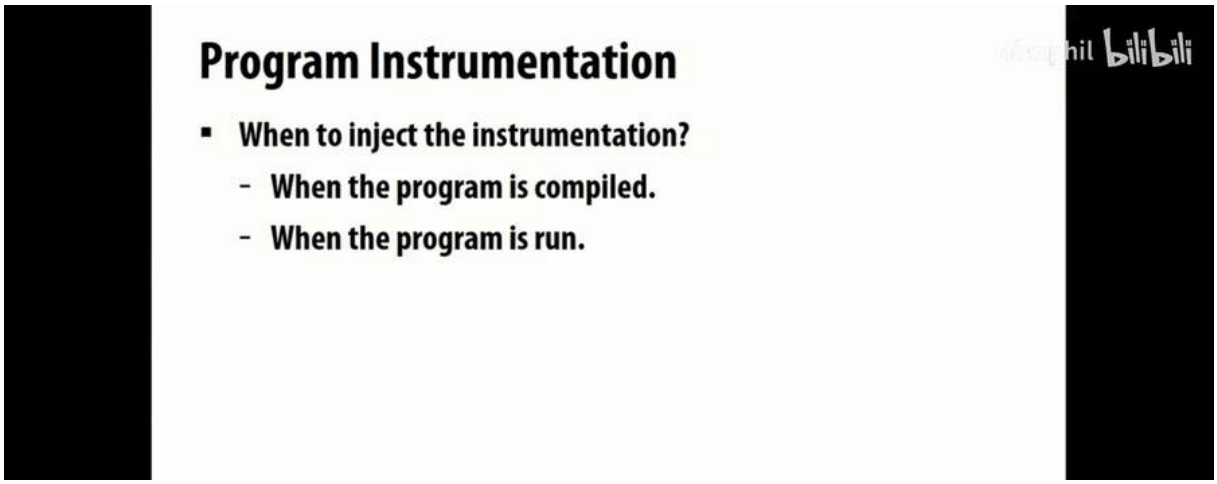

图 399: 程序插桩：在编译时或运行时注入测量代码[399]

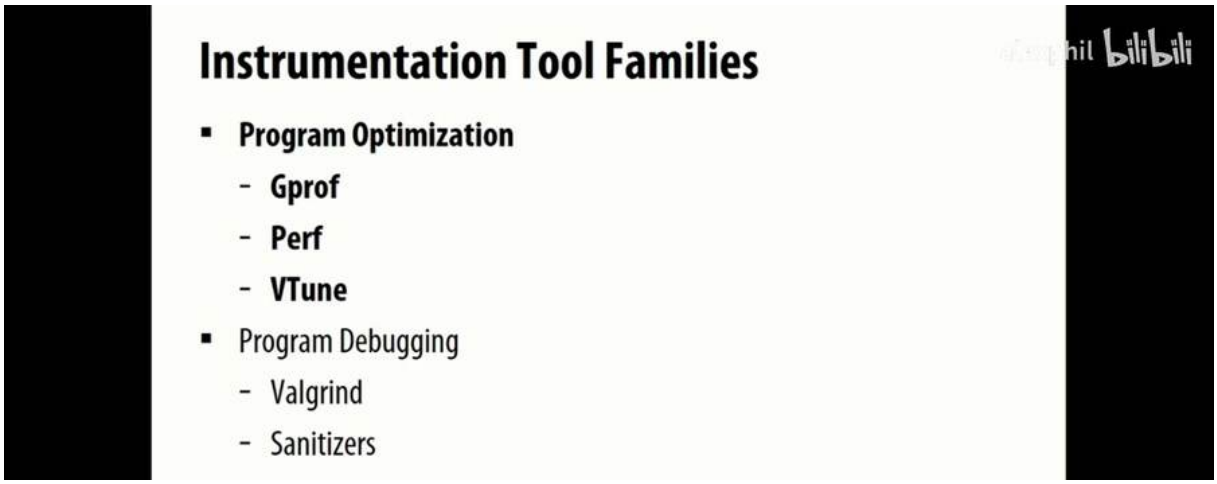

图 400: 插桩工具的几大类别：性能优化类与调试类[400]

两类工具回答不同问题

性能优化类工具回答” 时间花在哪里”，它假设程序行为正确、只是不够快。调试类工具回答” 程序为什么行为不对”，它关注状态与流程。用错类别会得到大量无关信息：拿调试器去找性能瓶颈，或拿 profiler 去查逻辑错误。

他把工具分成两大类：性能优化类（gprof、perf、VTune 等）与程序调试类。

[399] 视频画面时间区间：00:14:30–00:14:45。若为裁剪图，时间区间仍对应原视频画面。
[400] 视频画面时间区间：00:15:30–00:15:45。若为裁剪图，时间区间仍对应原视频画面。

重访 Amdahl 定律

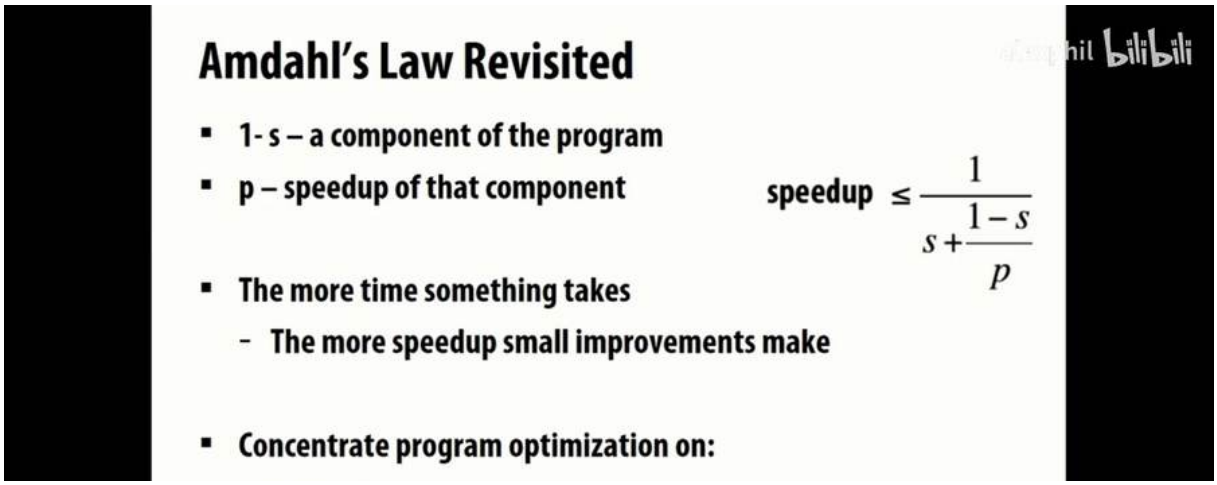

图 401: 重访 Amdahl 定律：优化占比为 s 的部分，整体加速受其限制[401]

讲者在这里用 Amdahl 定律说明为什么必须先定位热点：

$$S_{\text{total}} = \frac{1}{(1-s) + \dfrac{s}{p}}$$

- s：被优化部分占程序总时间的比例；
- p：该部分自身获得的加速倍数；
- 当 $p \to \infty$ 时，S_{total} 上界为 $1/(1-s)$。

若某函数只占 5% 时间，即使把它优化到零，整体也只快 5%。而若它占 50%，同样的努力收益大十倍。

gprof

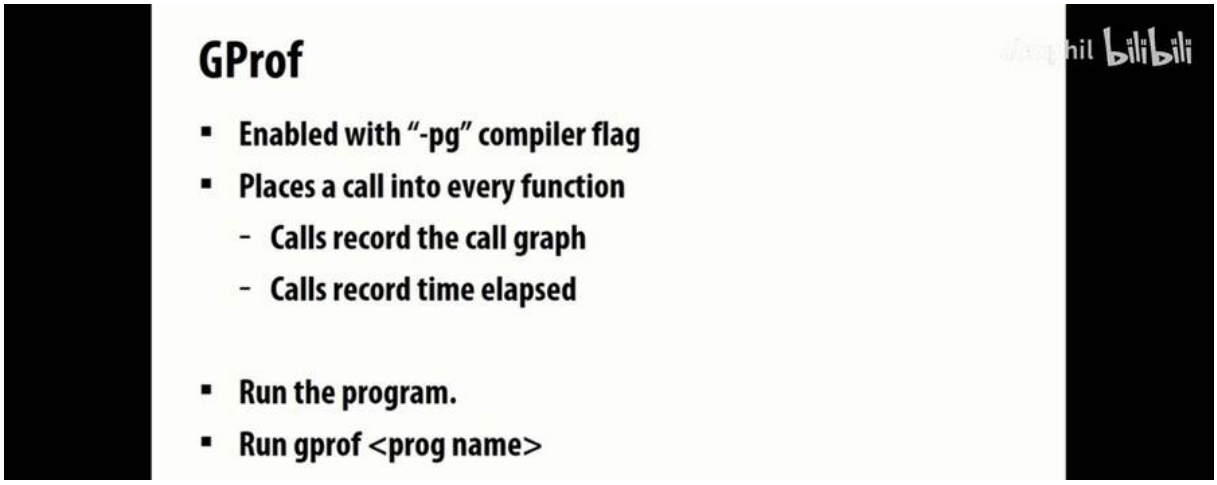

图 402: gprof：用 -pg 编译选项启用，在每个函数处插入调用[402]

[401] 视频画面时间区间：00:16:20–00:16:35。若为裁剪图，时间区间仍对应原视频画面。

gprof 通过 `-pg` 编译选项启用，它在每个函数处插入调用，记录调用关系与耗时。

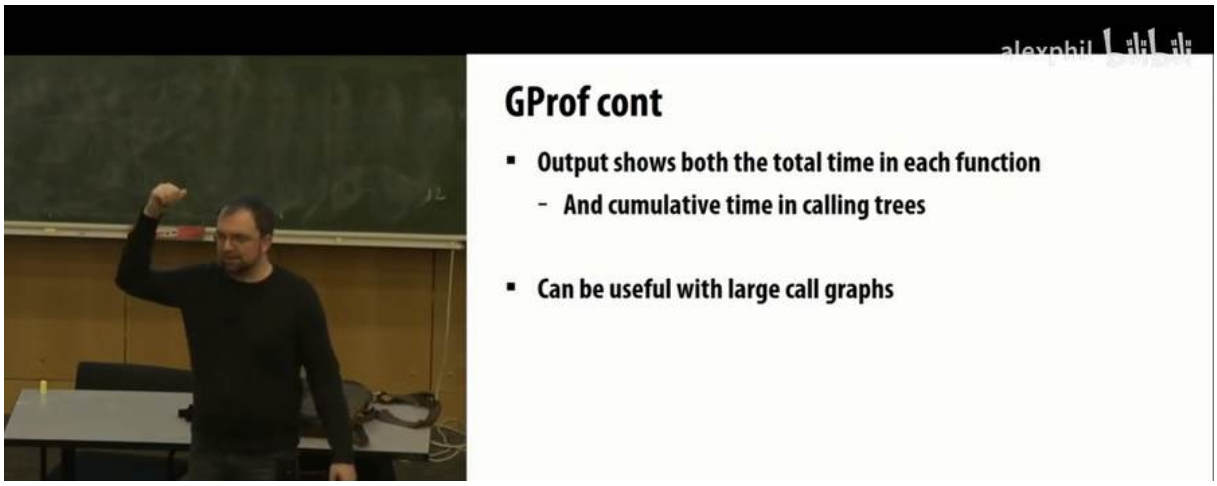

图 403: gprof 输出：各函数总耗时、调用树累积时间与调用次数[403]

输出会同时给出每个函数的总耗时、调用树中的累积时间，以及调用次数。

讲者给出实测结果：70% 的时间花在 `build_incoming_edges`（构建整张图），30% 花在 `pagerank` 上，还显示了调用次数——有的是 18 次，有的是 160 万次。

要排除与评分无关的框架代码

`build_incoming_edges` 是教师提供的代码，学生无法改变它，而且计时只针对 pagerank 部分。因此虽然它占了七成时间，却可以整个排除。这一步判断不是工具给出的，而是人根据评分规则做出的。

本章小结

第二层是函数级：gprof 用 `-pg` 插桩，给出每个函数的时间占比与调用次数。Amdahl 定律说明为什么必须先做这一步——占比决定收益上限。但工具给出的占比必须结合评分规则解读，否则会把无关的框架代码当成优化目标。

18.4 第三层：硬件性能计数器

架构提供的计数器

讲者转向硬件层面。现代架构暴露了一组性能计数器，可以统计缓存缺失、分支预测失败、每周期指令数等。

[402] 视频画面时间区间：00:18:00–00:18:15。若为裁剪图，时间区间仍对应原视频画面。
[403] 视频画面时间区间：00:19:00–00:19:15。若为裁剪图，时间区间仍对应原视频画面。

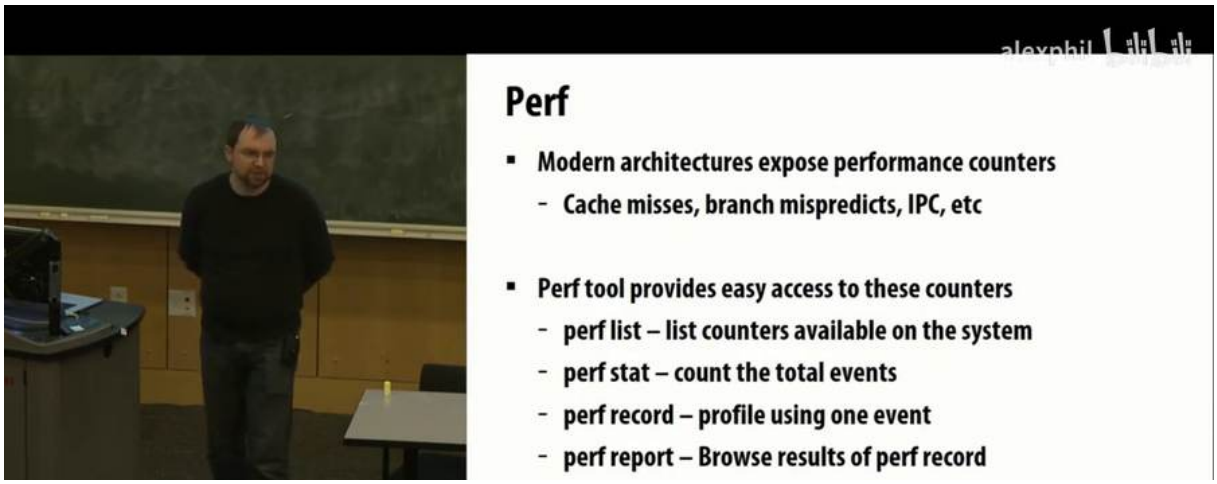

图 404: perf：访问现代架构暴露的性能计数器[404]

这些计数器是特权资源，因此相应工具需要由 root 安装。如果机器上没有，要么自己成为 root，要么找人装。

perf stat

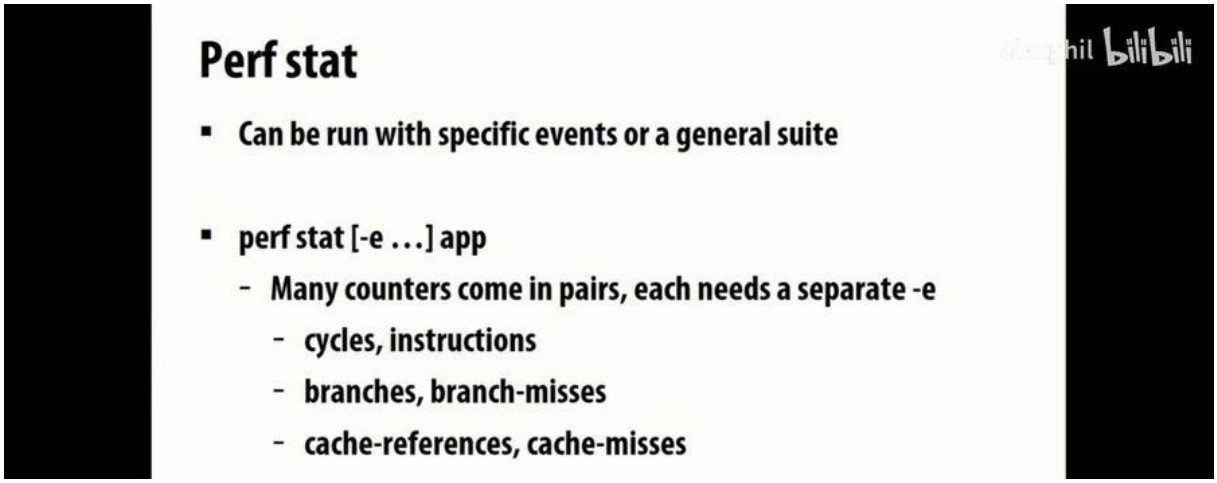

图 405: perf stat：可指定事件，也可用默认集合[405]

`perf stat` 是最简单的入口：可以指定事件，也可以让它收集一组常用计数器。

讲者强调了一条极易被忽视的规则：** 计数器必须成对使用 **。

比如有一个计数器统计分支总数，另一个统计分支预测失败数。如果只告诉你” 预测失败了 100 万次”，你无法判断这是 90% 的失败率还是 1%。缓存同理——” 有 100 万次缓存缺失” 是好是坏，取决于总访问数。

[404] 视频画面时间区间：00:21:00–00:21:15。若为裁剪图，时间区间仍对应原视频画面。

[405] 视频画面时间区间：00:22:30–00:22:45。若为裁剪图，时间区间仍对应原视频画面。

绝对计数必须除以总数才有意义

任何计数器给出的都是绝对量。缺失数、预测失败数、周期数本身都无法判断好坏，必须与对应的总数相比构成比率。因此收集计数器时要有意识地成对选取，否则拿到的是一堆无法解读的数字。

分时复用

分时复用会掩盖阶段性行为

程序若明显分为”构图”与”计算”两个阶段，分时复用会把两个阶段的数据混合外推，得到的比率既不代表构图也不代表计算。此时应当把两个阶段拆成两次运行分别采集，或改用基于采样的 perf record。

若一次执行中选了超过四个计数器，硬件会做分时复用：周期性关掉一个、打开另一个，假设你的代码行为足够稳定，然后把各段结果聚合起来，当作全程都在采集。

分时复用的前提是代码行为稳定

硬件能同时采集的计数器数量有限（通常四个）。超过时它轮流采样并外推。若程序有明显的阶段性（例如先构图后计算），这种外推会把两个阶段的数据混在一起，得到误导性的比率。此时应分段运行，或用采样方式替代。

读懂默认输出

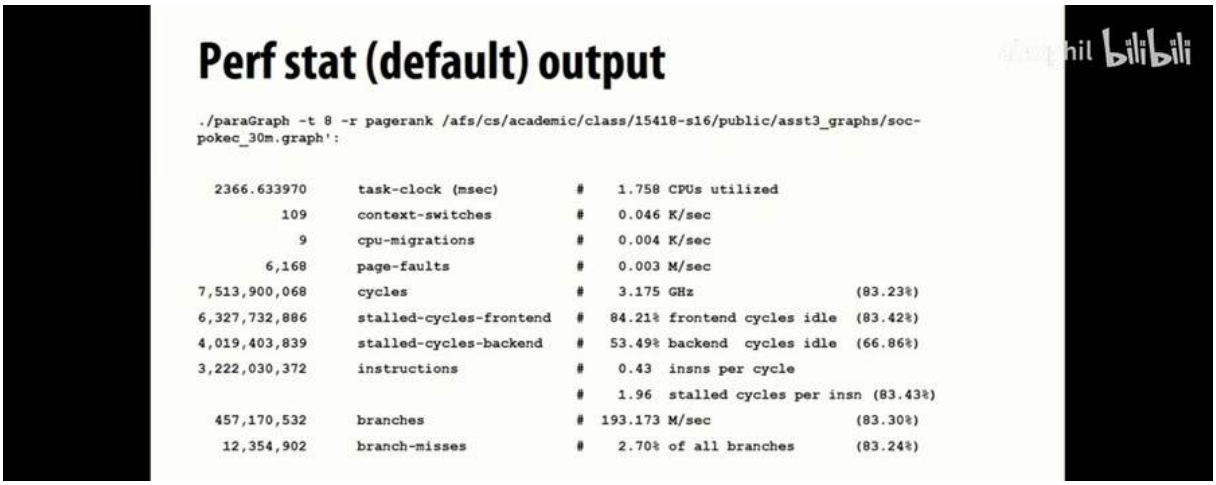

图 406: perf stat 默认输出：耗时、周期、停顿、指令数与 IPC[406]

讲者展示了 page rank 的实测输出，其中几个数字很关键：

耗时约若干毫秒、使用了若干 CPU；总周期数；前端与处理器内部的停顿时间；执行的指令数。由此算出每周期只执行 0.4 条指令——讲者的评语是”这不太好”。

分支方面：只有 2.7% 的分支被预测错误，这是正常的。

于是问题是：瓶颈在哪？这个默认集合里缺了什么？

答案很自然：缺缓存。讲者于是加上缓存计数器：

[406] 视频画面时间区间：00:24:30–00:24:45。若为裁剪图，时间区间仍对应原视频画面。

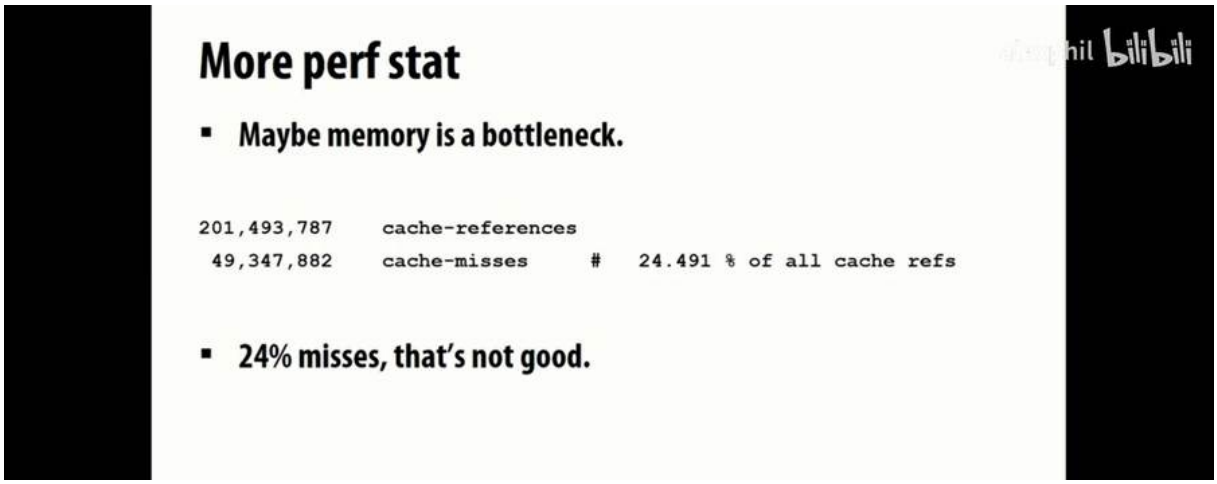

图 407: 更多 perf stat：缓存引用数与缺失数[407]

结果是约 5000 万次缓存缺失，而总访存约 2 亿次，缺失率 24.5%。讲者的评语同样是” 这不太好”。

缺失率由两个计数器相除得到：

$$\text{miss rate} = \frac{N_{\text{miss}}}{N_{\text{ref}}} = \frac{49{,}347{,}882}{201{,}493{,}787} \approx 24.5\%$$

- N_{miss}：`cache-misses` 计数器，约 4935 万；
- N_{ref}：`cache-references` 计数器，约 2.01 亿；
- 两者必须成对采集，单独任何一个都无法判断好坏。

用排除法读性能计数器

讲者的推理路径值得记住：先看 IPC（0.4 偏低），再看分支预测（2.7% 正常，排除），于是怀疑内存，加上缓存计数器确认缺失率 24.5%。每一步都是用一个指标排除一种可能，而不是一次性把所有计数器看一遍。这套顺序能把” 哪里不对” 缩小到具体子系统。

本章小结

第三层是硬件计数器。perf stat 给出周期、指令、IPC、分支与缓存等指标，但绝对数必须配总数才有意义，超过四个计数器会分时复用。读的方法是用排除法：IPC 偏低 → 分支正常 → 怀疑内存 → 缓存缺失率确认。

但函数级与子系统级的信息还不够，还需要知道具体是哪一行代码。

[407] 视频画面时间区间：00:25:00–00:25:15。若为裁剪图，时间区间仍对应原视频画面。

18.5 第四层：采样分析定位到行

perf record 与 report

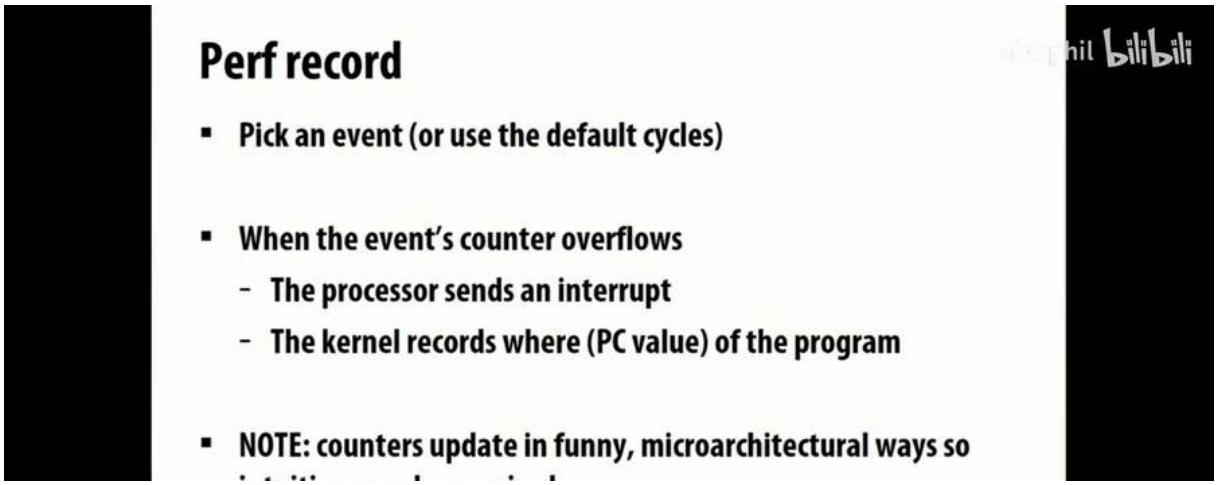

图 408: perf record：选择事件，计数器溢出时中断并记录位置[408]

`perf record` 选择一个事件（默认是周期），当该事件的计数器溢出时触发中断，记录当时执行到哪里。

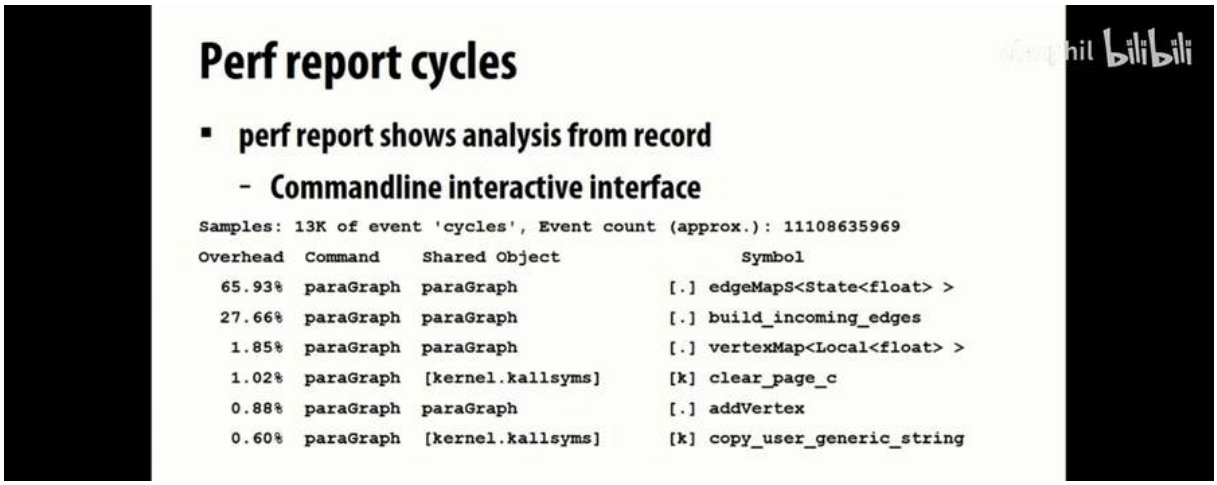

图 409: perf report：从记录生成交互式分析[409]

`perf report` 把记录转成分析。讲者说明采样点会自然集中在真正发生事件的位置——有 5000 万次缺失，若每 5 万次中断一次，就得到约 1000 个采样点，它们会散布在真正发生缺失的地方。

一个必须理解的精度限制

讲者在这里引用了架构手册的一段话，值得完整记录：由于微架构中存在延迟，从事件产生到溢出中断之间有时间差，因此很难在紧邻事件的位置产生中断。

这意味着工具告诉你的位置，未必就是事件真正发生的那条指令。

[408] 视频画面时间区间：00:26:00–00:26:15。若为裁剪图，时间区间仍对应原视频画面。

[409] 视频画面时间区间：00:29:00–00:29:15。若为裁剪图，时间区间仍对应原视频画面。

采样位置可能偏移一到两条指令

看到”某条不访问内存的代码行上有数百万次缓存缺失”时，不要断定工具坏了。真正的事件通常发生在其前后一到两条指令处。这要求使用者带着判断去复核周围的代码，而不是机械相信工具指向的那一行。

讲者说他的经验是偏移通常只有一两条指令。而能把范围缩小到两三条汇编指令，已经比之前所有工具都好得多。

从函数到行

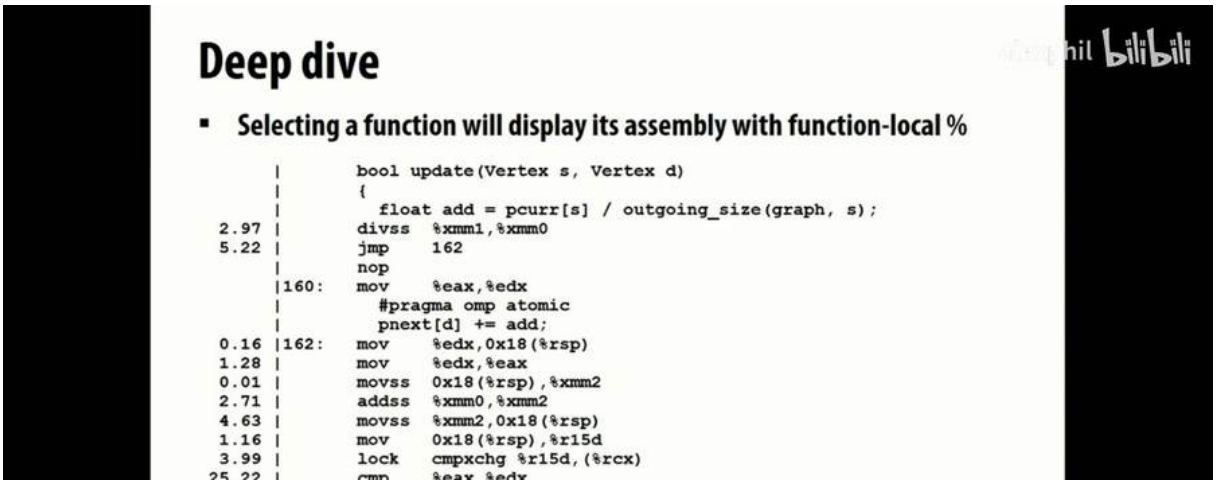

图 410: 深入：选中函数会显示其汇编与函数内的热点分布[410]

有了采样结果，就能看到缓存缺失在各函数间的分布。`edge_map` 这个自写例程占了 47 个百分点。构图代码占了另外 46 个百分点。两者合计 93 个百分点，几乎覆盖全部缺失。这已经很有用，但还可以继续深入——选中某个函数，工具会显示它的汇编，并标出函数内的热点百分比。

本章小结

第四层是采样分析：perf record 在事件溢出时中断记录位置，perf report 汇总成函数级与行级的热点。关键限制是微架构延迟会让位置偏移一到两条指令，因此必须结合人的判断复核。

这一层已经能把问题定位到具体代码行。最后一章看一个真实的优化案例。

[410] 视频画面时间区间：00:30:00–00:30:15。若为裁剪图，时间区间仍对应原视频画面。

18.6 案例：一个指针间接寻址的代价

先看反汇编

图 411: 反汇编：究竟是什么在消耗 KBFS 的时间[411]

讲者用 KBFS 负载（在图上做多次广度优先搜索）说明下一步该做什么。这一次，超过 80% 的执行时间都在他自己写的代码里——不能再归咎于教师提供的构图代码。

热点行常常紧邻真正的元凶

采样工具指向的行未必就是问题所在。本例中报出的位置是一个比较操作，而真正的开销来自它上方那一行的间接寻址。读报告时应把热点行连同前后几条指令一起看，而不是只盯被标记的那一行。

于是他看那个大循环：里面有一个操作，检查源与目标是否都访问过；若没有就做事，若有就跳过。

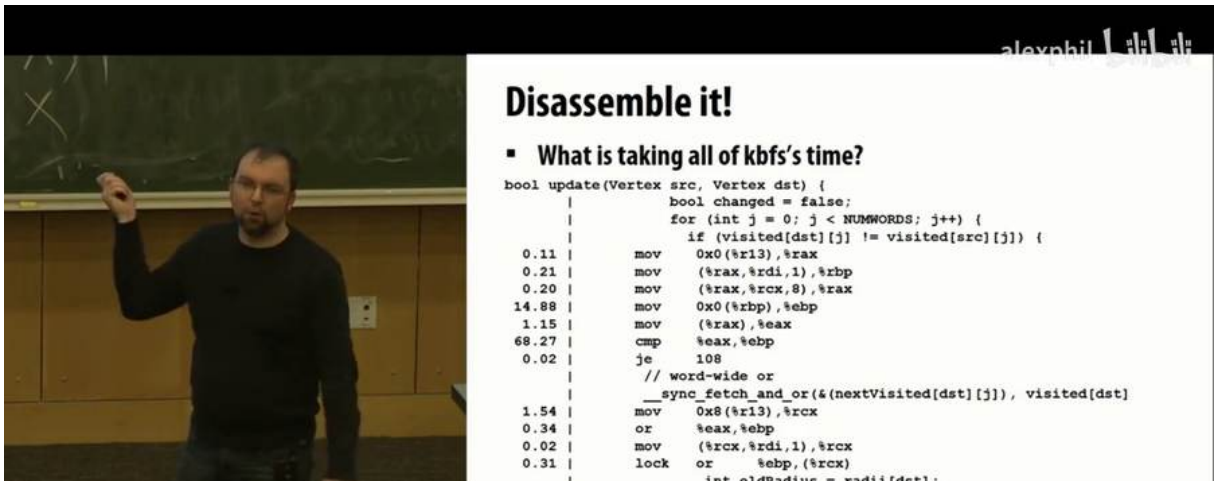

图 412: 双重指针数组造成两级间接寻址，每次访问都多付一次跳转[412]

[411] 视频画面时间区间：00:34:00–00:34:15。若为裁剪图，时间区间仍对应原视频画面。
[412] 视频画面时间区间：00:35:30–00:35:45。若为裁剪图，时间区间仍对应原视频画面。

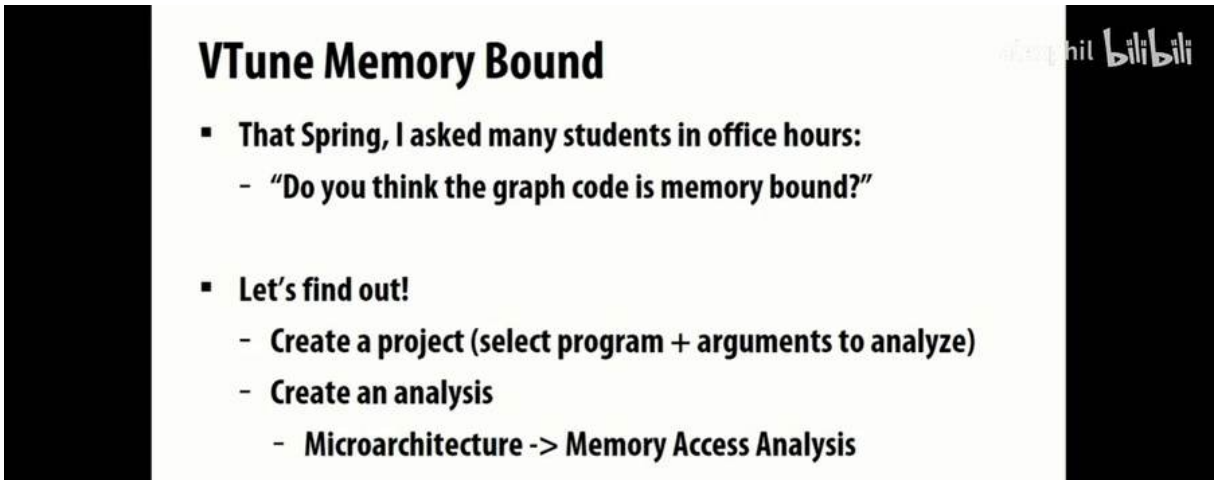

图 413: VTune 的 Memory Bound 视图：这类问题属于内存受限[413]

真正的瓶颈不在比较，而在上一行

讲者指出时间并不花在那个比较上，而是花在它上方那一行内存访问。

关键在于 `visited` 的类型：它被实现成指向指针数组的指针，看起来像二维数组，实际每次访问都要经过两级间接寻址。这个代价很大。

数据布局的代价常被算法分析忽略

从算法复杂度看，二维数组的访问是 $O(1)$；但从硬件看，" 指针数组 + 每行单独分配" 意味着每次访问多一次内存跳转，且各行的物理位置不连续，缓存局部性也差。复杂度相同的两种实现，实测可以差近十倍。这类问题只能通过工具定位，无法从算法推导出来。

改动与效果

讲者的改动很简单：去掉一级间接寻址。结果代码跑得显著变快，实测减速倍数为 9.8 倍——也就是说原来那一级指针让代码慢了近十倍。

他坦率地说当时班里没人找到这个原因——因为那时他还没讲过这堂课，大家都没有这套方法。这正是本讲存在的理由。他补充说，后来他用 VTune 的 Memory Bound 视图重新看这个问题，工具会直接把它归类为" 内存受限"，这让定位变得容易得多。这也说明不同工具的抽象层次不同：perf 给的是原始计数器与采样点，VTune 这类工具会在其上做一层归类，把纷杂的数字压缩成" 你的程序受限于内存" 这样一句结论。两者各有价值：原始数据更可信，归类结论更易读。他还提到曾问过很多同学" 你觉得这段代码受限于什么"，多数人的直觉判断与实际测量不符——这也正是为什么必须测量而不能凭感觉。

本章小结

KBFS 案例说明真正的瓶颈常在数据布局：双重指针数组让每次访问多付一级间接寻址，去掉后减速倍数从 9.8 倍回到正常。这类问题在算法复杂度上完全不可见，只能靠工具定位。它也印证了采样报告需要连同前后指令一起读——报出的行往往不是真正的元凶。

工具本身有代价，这决定了它给出的数字该怎么用。

[413] 视频画面时间区间：00:37:00–00:37:15。若为裁剪图，时间区间仍对应原视频画面。

18.7 工具的代价与使用边界

插桩带来的开销

讲者也给出这类工具的代价：插桩会带来数倍到数十倍的开销（他提到的量级是从 4× 到 20×），而且开销本身会改变程序的时间分布。

因此插桩测得的耗时需要按开销倍数修正：

$$T_{\text{real}} \approx \frac{T_{\text{instrumented}}}{1+o}$$

- $T_{\text{instrumented}}$：带插桩运行时测得的耗时；
- o：插桩带来的额外开销比例，量级从 4 到 20；
- 该修正只在开销均匀分布时有效，否则只能用于比较相对占比。

插桩开销会扭曲被测量的对象

gprof 在每个函数入口插桩，对频繁调用的小函数影响尤其大；缓存模拟器需要逐条模拟访存，比真实硬件慢几个数量级。因此用插桩工具得到的是”相对分布”而不是”绝对耗时”。要测真实性能，应当用未插桩的版本。

本章小结

KBFS 案例说明真正的瓶颈常在数据布局而非算法：双重指针数组让每次访问多付一级间接寻址，去掉后显著变快，这个倍数达到近十倍。插桩工具的开销可达数倍到数十倍，会扭曲时间分布，因此它给的是相对分布而非绝对耗时。

18.8 总结与延伸

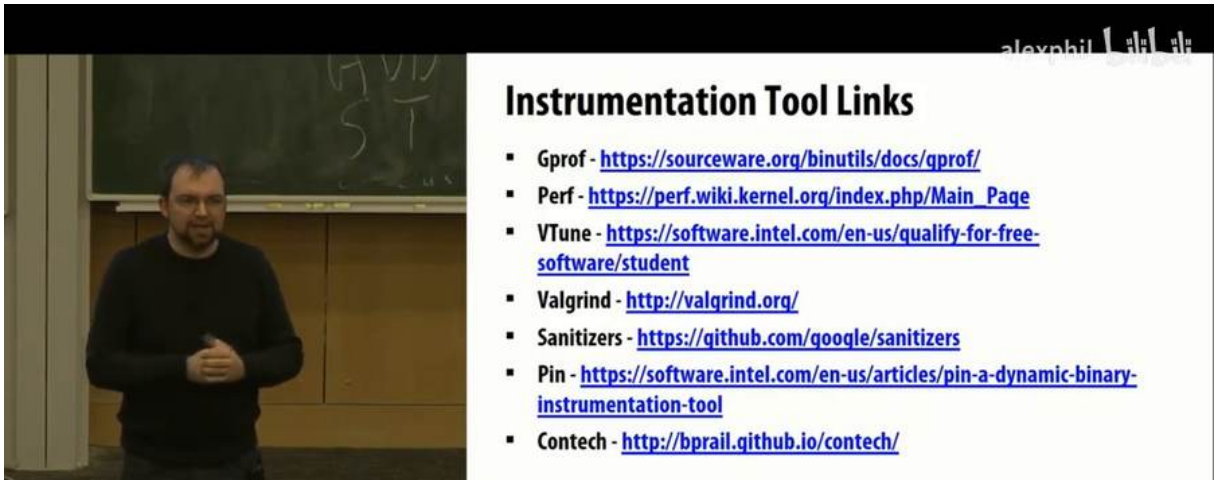

图 414：总结：各工具的资料链接与适用场景[414]

讲者在结尾给出各工具的资料链接，并把整讲收束。

把全讲串起来，这是一套由外向内逐层收窄的方法。第一层用 top 确认机器是否被他人占满、内存是否充足——注意 CPU 百分比按逻辑线程累加，八线程满载显示 800% 而非 100%。第二层

[414] 视频画面时间区间：01:12:40–01:12:55。若为裁剪图，时间区间仍对应原视频画面。

用 gprof 做函数级计时，并用 Amdahl 定律判断哪一段值得优化；但必须结合评分规则排除无关的框架代码。第三层用 perf stat 读硬件计数器，关键是绝对数必须配总数构成比率，超过四个计数器会分时复用；读法是用排除法——IPC 0.4 偏低、分支预测 2.7% 正常、于是怀疑内存、缓存缺失率 24.5% 确认。第四层用 perf record 采样定位到行，但要记住微架构延迟会让位置偏移一到两条指令。最后的案例说明真正的瓶颈常在数据布局：双重指针数组的两级间接寻址带来了近十倍的减速，去掉后显著变快。

本讲义在讲者材料之上做的压缩，有四点需要与讲者原话区分：

- 这套方法是分层排除，不是一次性收集所有指标。每一层都用最便宜的工具排除一大类可能，只在必要时才进入下一层。颠倒顺序会导致在无关数据上花时间，或过早进入昂贵的插桩分析。
- 工具的结论必须与人对其业务语义的理解结合。gprof 显示构图占七成，但那段代码不参与评分，因此不是优化目标。这一步判断工具无法代劳。
- 采样位置偏移是架构特性而非工具缺陷。理解这一点，才能在看到” 不访存的行报缺失” 时继续排查而不是放弃工具。这也解释了为什么读报告需要同时看汇编上下文。
- 最有效的优化常来自数据布局，而布局问题不会在复杂度分析中显现。双重指针数组是典型：算法上等价，硬件上差近十倍。因此性能工程不能只停留在算法层面。

这堂课的边界在于它讲的是工具的使用方法与判读逻辑，不含各工具的完整参数。讲者多次说明示例来自 2016 年的作业，学生需要把方法迁移到当前作业。

两个可以直接从本讲数据提出的下一步问题：其一，插桩开销会改变时间分布，那么用插桩工具得到的相对占比，在多大程度上可以外推到未插桩的真实运行？其二，缓存缺失率 24.5% 已经很高，但若要进一步区分是容量缺失、冲突缺失还是伪共享，还需要什么工具或手段？

Lecture 19 Interconnection Networks

视频时长：1 小时 14 分 38 秒　来源：https://www.bilibili.com/video/BV1Nw41197ZZ/

19.1 互连网络：一个在小规模下看不见的问题

这一讲讨论连接高性能计算系统中各计算节点的网络。讲者开场就做了一个区分：你在网络课上学的互联网与 TCP 都是很有意思的主题，但这些网络很不一样。

它们的设计目标是紧密局部、极高性能；另一方面，它们不必像互联网那样担心故障与断连。这类网络要连接的规模可以很大——超算中心可能有 10000 个节点需要互相连通。

此前我们一直假设有个总线

讲者回顾说，前面讲缓存协议时，我们只是假设缓存之间能以某种方式通信，通常就直接假设它是一条总线。

需要记住的是：这些连接点被称为节点，但实际通常是缓存控制器、内存控制器以及其他系统部件，它们充当处理器的代理。今天统称为节点。

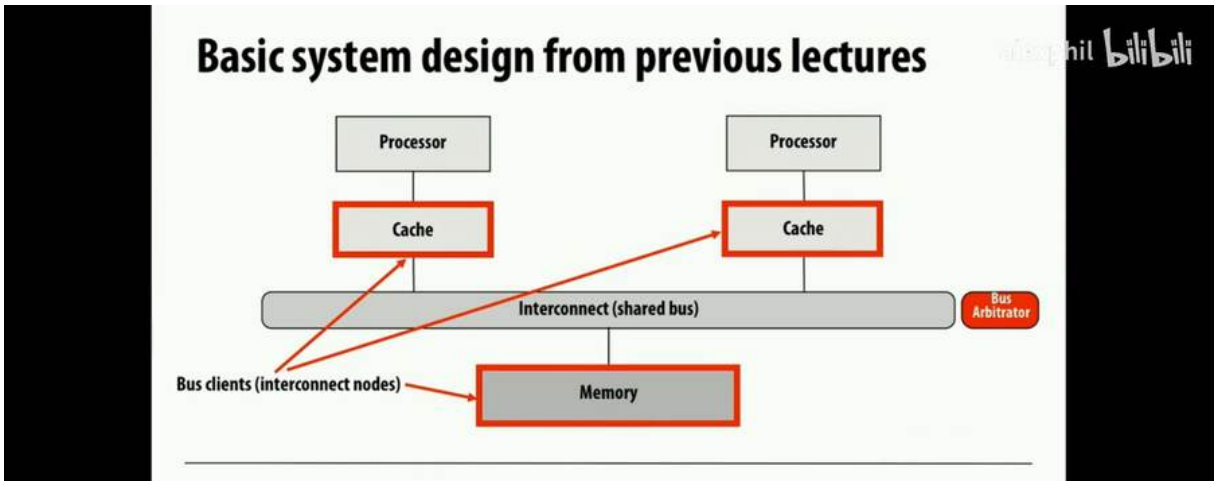

图 415: 前几讲的基本系统设计：处理器、缓存与互连的位置[415]

[415] 视频画面时间区间：00:01:30–00:01:45。若为裁剪图，时间区间仍对应原视频画面。

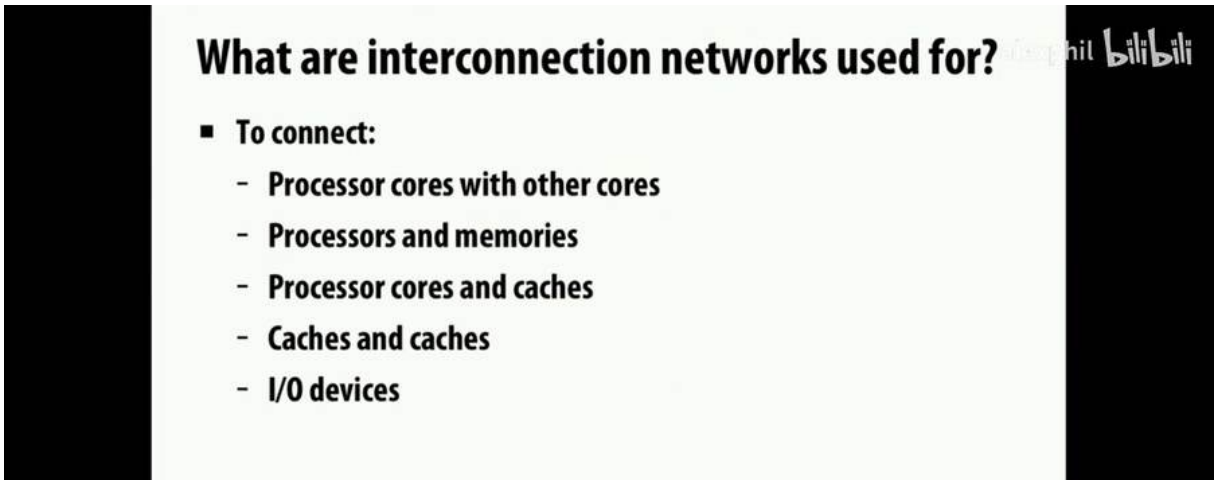

图 416: 互连网络用来做什么：连接核与核、核与内存、核与 IO[416]

总线的物理构造

总线是这件事最简单的形式，实际上就是一组导线。一组用于发出请求——把地址放上去；另一组用于返回响应，也就是读操作取回的数据。

讲者强调一个关键设计：这两组通常做成物理上分离的总线，以避免死锁——否则一堆请求可能堵住通道，让响应无法返回，系统就死锁了。

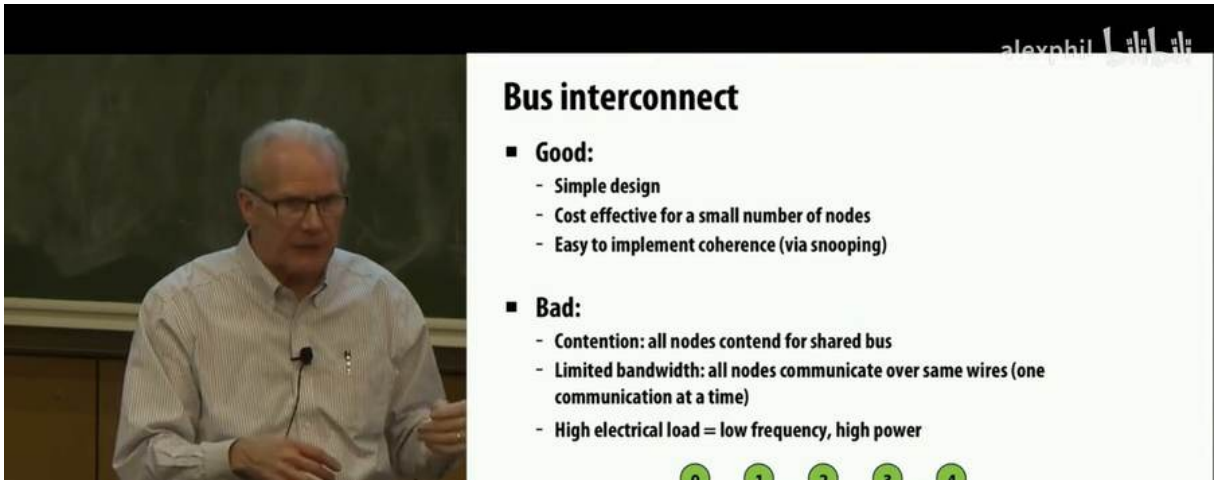

图 417: 总线互连：设计简单、对小规模节点成本有效[417]

请求与响应分离是为了避免循环等待

若请求与响应共用一条通道，当大量请求占满通道时，响应无法返回；而发出请求的一方在等到响应之前不会释放通道，于是形成循环等待。物理分离两条总线打断了这个环。这是互连设计中”用资源换正确性”的一个典型例子。

[416] 视频画面时间区间：00:03:30–00:03:45。若为裁剪图，时间区间仍对应原视频画面。
[417] 视频画面时间区间：00:20:30–00:20:45。若为裁剪图，时间区间仍对应原视频画面。

术语

讲者给出三个基本术语。网络节点是连接到路由器或交换机的端点，可以是处理器核、缓存控制器、内存控制器或 IO 设备。

链路是网络中的点对点连接，物理上通常是一束导线加两端能收发消息的电路。

交换机是一个能把消息沿不同路径导向目的地的模块。讲者说交换机内部的逻辑正是今天要看的一部分，它定义了路由元素。

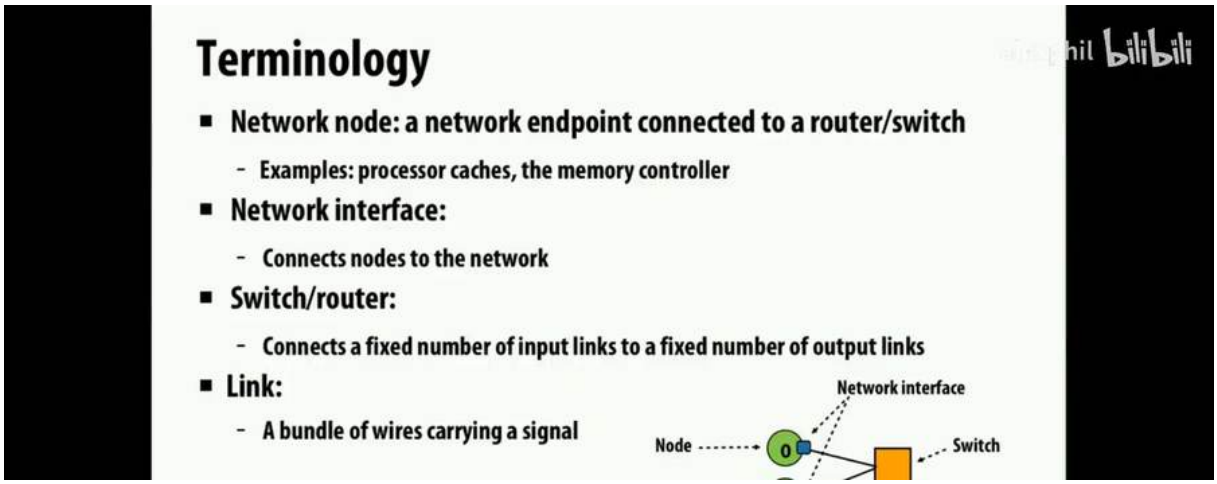

图 418: 术语：网络节点、链路与交换机[418]

本章小结

互连网络连接 HPC 系统的各节点，其目标与互联网不同：紧密局部、极高性能，但可以较少考虑故障。最简单的形式是总线，请求与响应需物理分离以避免死锁。三个基本术语是节点、链路与交换机。

下一章看为什么这个问题会变得重要，以及设计时需要权衡哪些维度。

19.2 为什么互连设计越来越重要

三个权衡维度

讲者列出使这个问题变得困难的几个方面。

第一是性能，包含延迟与带宽两方面。

第二是能耗，尤其是片上网络，能耗是主要关切。

第三是可扩展性。

可扩展性是一条产品约束而非技术偏好

若每个规模都需要定制网络，研发与验证成本会随规模种类线性增长。因此厂商倾向选择可递归构造的拓扑（环、mesh、胖树），即使它在极端规模下不是最优。这解释了为什么很多商用互连选择渐近指标并不最好的方案。

[418] 视频画面时间区间：00:08:00–00:08:15。若为裁剪图，时间区间仍对应原视频画面。

讲者说很少有人只想造一台系统：公司不会只做一千个处理器的机器，它们希望能卖 256、512、1024 等各种规模，并且能靠增加节点来扩展，而不必为每个规模定制一整套网络。

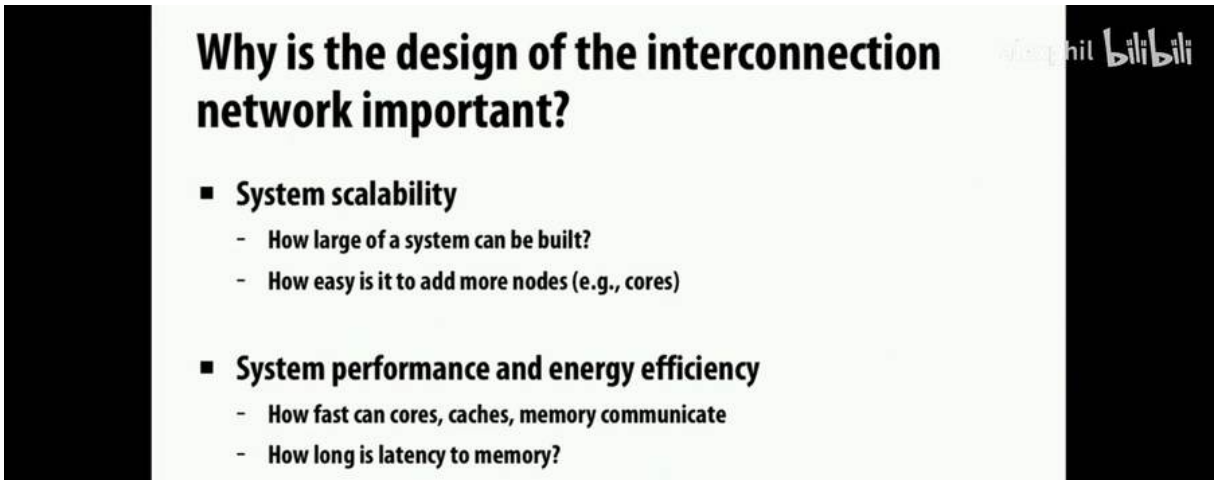

图 419: 为什么互连网络的设计很重要：系统可扩展性与成本[419]

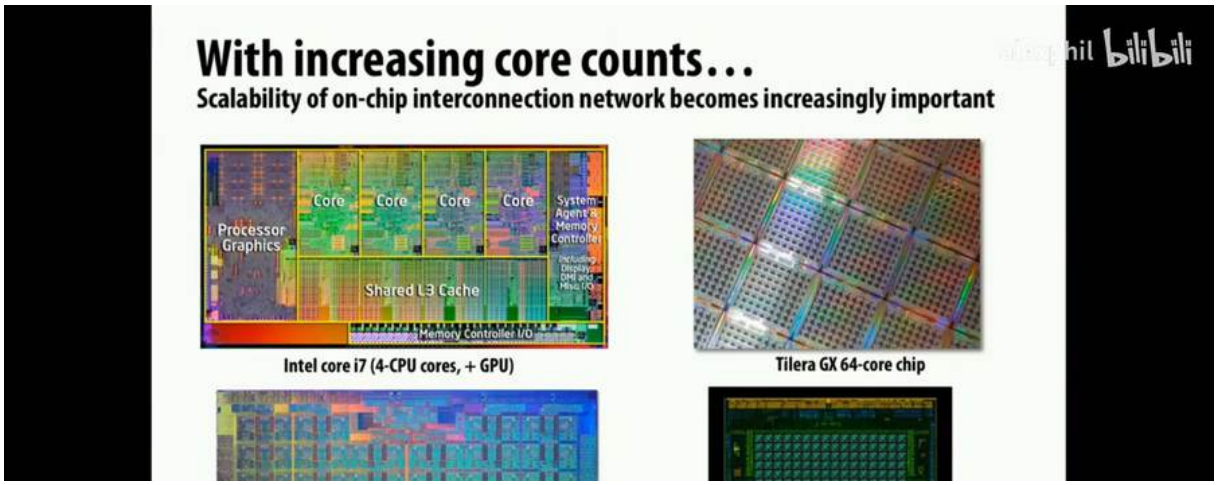

图 420: 核心数增长使片上互连的可扩展性日益重要[420]

物理上还有一个约束：在单芯片上，你可以负担更多导线、更粗的连接；跨芯片就做不到。因此系统设计层次越往上，方案差别越大。

真实机器的规模

讲者给出具体数字。课程用的多核芯片通常有六到八个核，规模相当小，它们通常用简单的环形网络。

Xeon Phi 则大得多：他们有的是 61 核机器，较新一代是 72 核。

[419] 视频画面时间区间：00:04:00–00:04:15。若为裁剪图，时间区间仍对应原视频画面。

[420] 视频画面时间区间：00:05:30–00:05:45。若为裁剪图，时间区间仍对应原视频画面。

规模决定拓扑选择

六到八核用环就够了，因为环的延迟随节点数线性增长，在个位数节点时仍可接受。一旦到几十上百个节点，环的直径就成了瓶颈，于是需要 mesh、树或胖树。拓扑选择不是偏好问题，而是由规模与流量模式共同决定的。

本章小结

互连设计需同时权衡性能（延迟与带宽）、能耗与可扩展性。厂商希望同一套设计覆盖 256 到 1024 等多种规模。物理约束使片上与跨芯片方案差别很大。真实机器中，六到八核用环，61 到 72 核的 Xeon Phi 则复杂得多。

要比较不同方案，需要一套描述与度量工具。

19.3 拓扑、路由、缓冲与流量控制

四个设计问题

讲者把设计问题拆成四类，这个划分贯穿全讲。

拓扑：网络的整体图结构，也就是交换机与链路如何连接节点。

路由：要把消息从 A 送到 B，选哪条路径，算法是什么。

缓冲：实际发送一条消息意味着什么？一个极端是必须预留整条通路，另一个极端是逐段搬运消息的一部分。

流量控制：如何管理拥塞，如何应对同一时刻大量消息都要通过的情形。

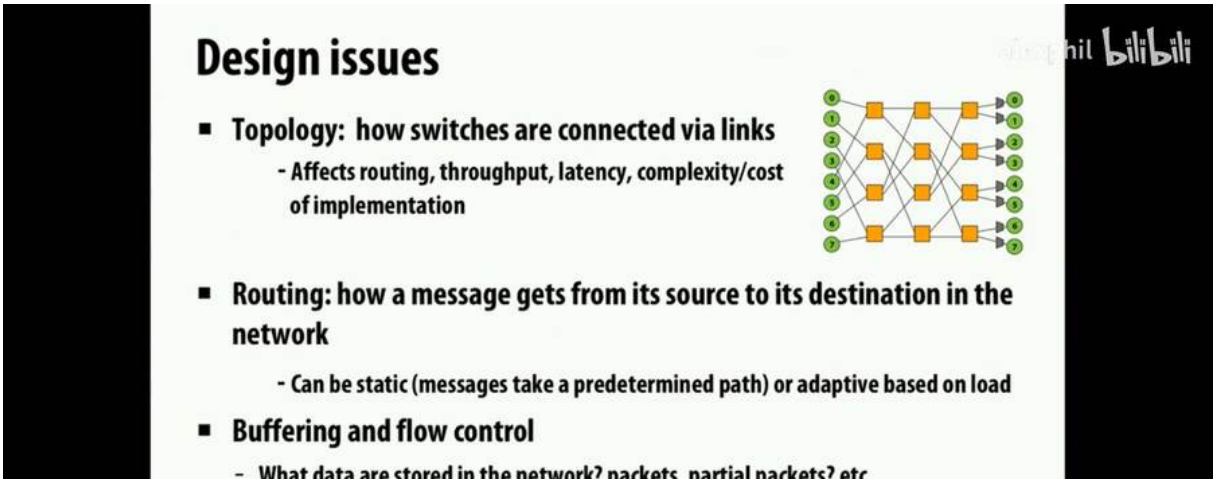

图 421：设计问题：拓扑影响路由与吞吐，缓冲与流量控制处理拥塞[421]

拓扑的两个度量

直径与平均距离要分开看

直径是最坏情况，用于界定延迟上界；平均距离决定典型性能。对 mesh 两者接近，因此直径能代表典型行为；但对某些不规则拓扑，最坏路径可能极少出现，此时只看直径会严重低估实际性能。评估时应两者并看。

[421] 视频画面时间区间：00:09:00–00:09:15。若为裁剪图，时间区间仍对应原视频画面。

路由距离：一条路径上经过的链路数，也就是跳数。
直径：图中最长的那条最短路径。讲者用 mesh 举例：$n \times n$ 的 mesh，直径是 $2n-2$。

$$D_{\text{mesh}} = 2n - 2, \qquad D_{\text{ring}} = \frac{N}{2}, \qquad D_{\text{hypercube}} = \log_2 N$$

- D_{mesh}：$n \times n$ 网格的最坏情况跳数，从一角到对角；
- D_{ring}：N 个节点绕环一周的最远距离；
- $D_{\text{hypercube}}$：N 个节点的超立方体只需逐位翻转，因此与位数同阶。

边缘效应使 mesh 的节点不等价

mesh 中心的节点有四条出边，边缘只有三条，角落只有两条。因此位于不同位置的节点其延迟与带宽并不相同。这会给负载均衡带来额外难度：把任务均匀分配到各节点，并不意味着通信代价也均匀。环面的边界回绕正是为消除这种不等价而设计的。

他还指出 mesh 的平均情况与最坏情况差别不大。

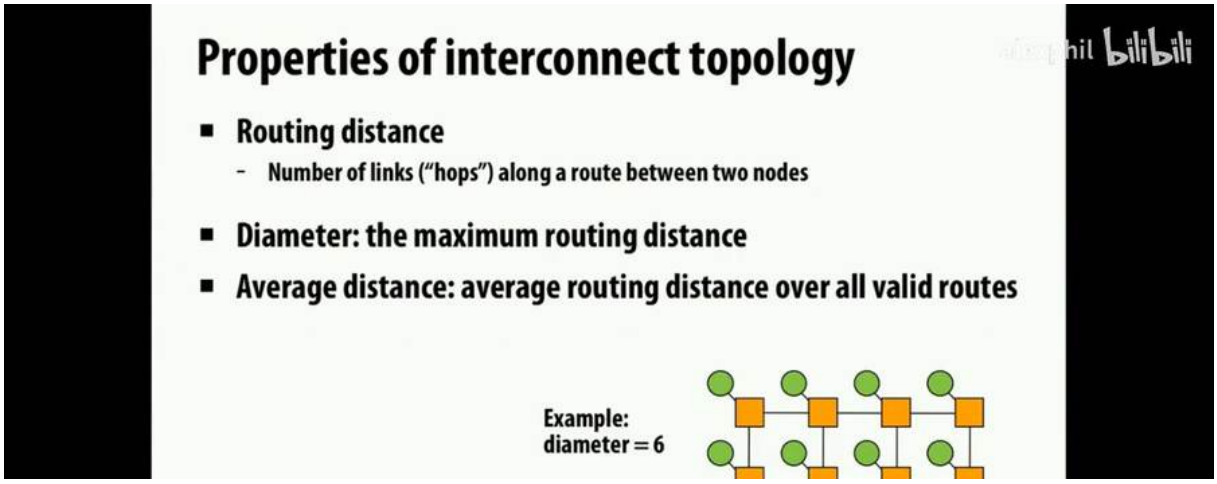

图 422: 互连拓扑的性质：路由距离（跳数）与直径[422]

对分带宽

这是讲者认为最有用的性能指标：把网络切成两半，数一数跨越切口的链路数。

[422] 视频画面时间区间：00:10:30–00:10:45。若为裁剪图，时间区间仍对应原视频画面。

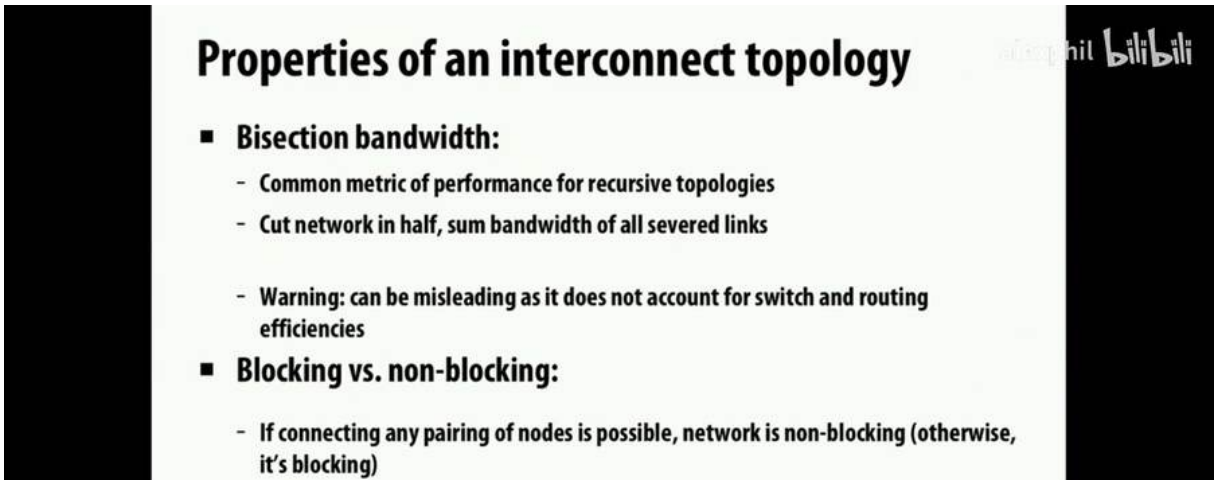

图 423: 对分带宽：衡量网络连通度的通用性能指标[423]

- 对 $n \times n$ 的 mesh，切成两半会切过约 n 条线。节点数是 n^2，因此对分带宽是节点数的平方根——讲者的评价是" 好，但不是很好"。
- 对简单的环或线性链，对分带宽是 1：切断任意一条线网络就断了。

写成通式：

$$B_{\text{mesh}}(N) = \sqrt{N}, \qquad B_{\text{ring}}(N) = 1, \qquad B_{\text{hypercube}}(N) = N$$

- N：网络节点总数；
- mesh 的对分带宽随 $\sqrt{N}$ 增长，环保持常数，超立方体随 N 线性增长；
- 三者的差距随规模扩大而急剧拉开。

为什么这重要？讲者给出具体场景：如果应用完全没有局部性，比如你正在做的作业，那么大量流量都要穿过系统，低对分带宽就成了瓶颈。

他做了个估算：若是完全随机的置换，每个人都去取数据，在中线处看过去，大约四分之一留在这一半、四分之一留在另一半、四分之一要从这半边到那半边，还有四分之一反向。

这可以写成跨越中线的流量比例：

$$f_{\text{cross}} = \frac{1}{4} = 0.25$$

- f_{cross}：随机置换下需要跨越中线的流量占总流量的比例；
- 每个节点对均匀分布时，四分之一的对子跨越中线；
- 若对分带宽为 B，则需承载 $0.25 \cdot N$ 量级的跨线流量。

这正是低对分带宽在随机访问下成为瓶颈的定量原因。

[423] 视频画面时间区间：00:12:30–00:12:45。若为裁剪图，时间区间仍对应原视频画面。

对分带宽是”最坏情况”的度量

它衡量的是网络在被切成两半时的最大承载能力，因此对无局部性的随机访问模式最敏感。若应用的通信有强局部性，实际瓶颈可能不在对分带宽而在局部链路。因此判断一个拓扑是否合适，必须同时看访问模式与这个指标。

阻塞与非阻塞

讲者用例子讨论这个概念：给定一个网络，同时让所有节点对之间都通信，是否都能无冲突地完成？能则是非阻塞，不能则是阻塞。

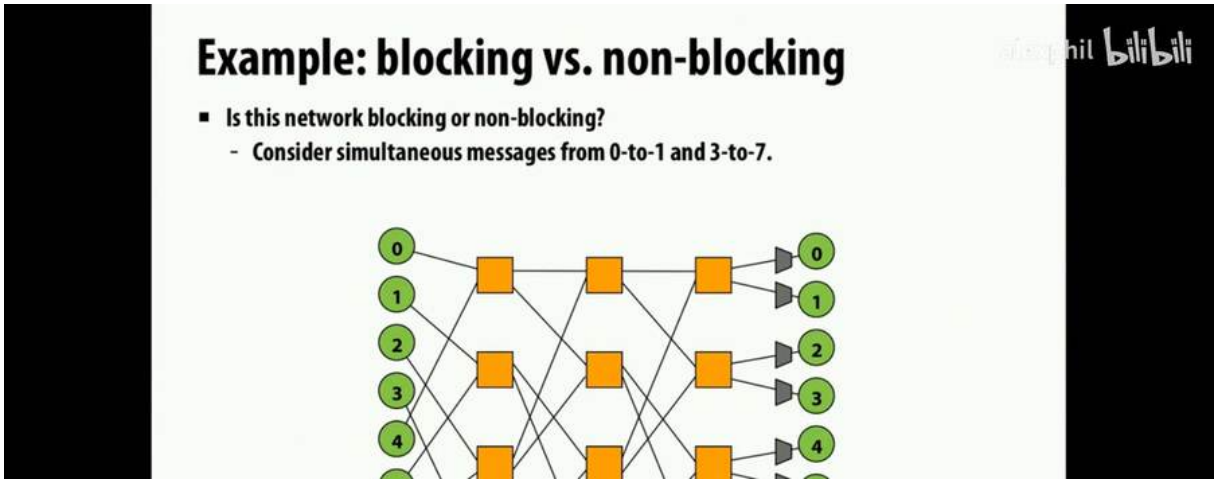

图 424: 例子：给定网络在同时通信时是否阻塞[424]

负载与延迟的关系

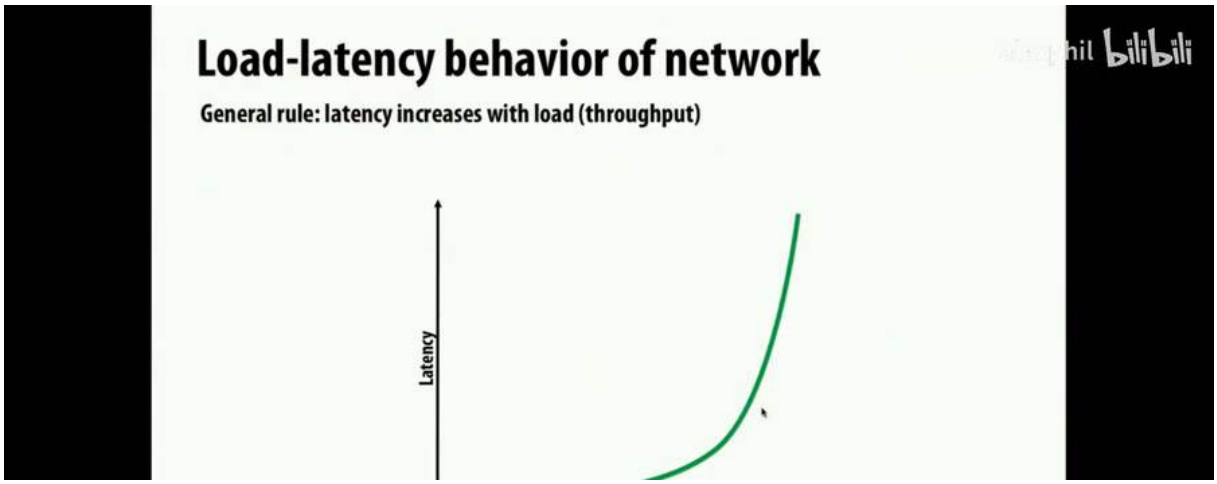

图 425: 网络的负载-延迟行为：延迟随负载上升而增加[425]

[424] 视频画面时间区间：00:15:30–00:15:45。若为裁剪图，时间区间仍对应原视频画面。
[425] 视频画面时间区间：00:17:00–00:17:15。若为裁剪图，时间区间仍对应原视频画面。

讲者给出一条通用规律：延迟随负载（吞吐）增加而增加。这是所有互连网络共有的行为——轻载时延迟接近理论最小值，接近饱和时延迟急剧上升。

网络性能必须用”负载-延迟曲线”描述而非单一数字

只报一个延迟值或一个带宽值都不足以刻画网络。真正决定应用性能的是在特定负载下的延迟，而负载又取决于应用的通信模式。因此评价网络要看曲线形状：拐点在哪里，饱和点在哪里。

本章小结

设计问题分四类：拓扑、路由、缓冲、流量控制。拓扑用路由距离与直径刻画，用对分带宽衡量连通度：mesh 为节点数的平方根，环为 1，超立方体为节点数。无局部性负载会让低对分带宽成为瓶颈。阻塞与非阻塞描述并发通信能力，而延迟随负载上升是通用规律。

有了这套度量，就可以逐个比较具体拓扑了。

19.4 拓扑巡礼：从交叉开关到环

交叉开关

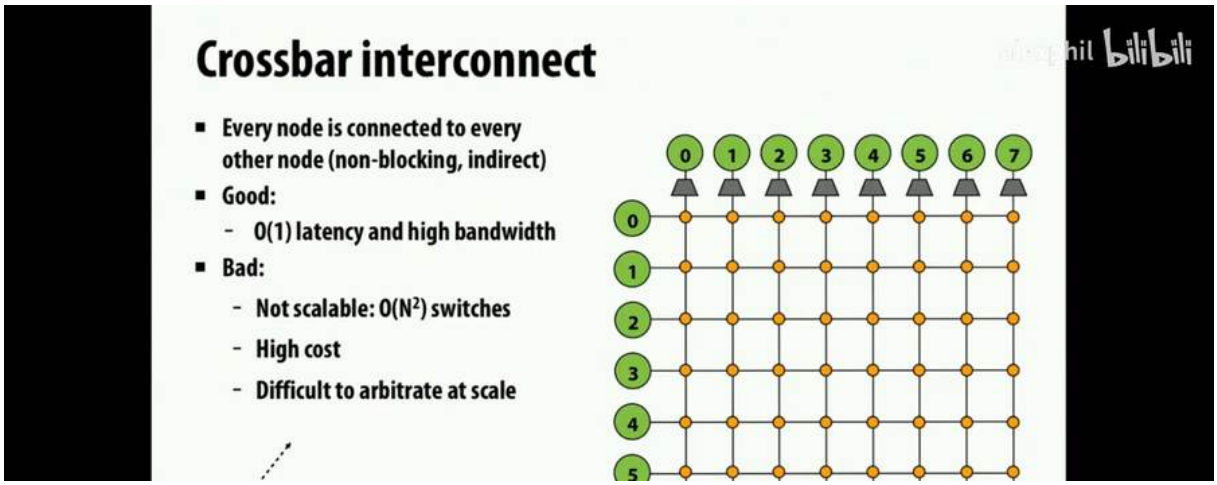

图 426: 交叉开关：每个节点与其他所有节点直连，非阻塞[426]

交叉开关是非阻塞的间接网络：每个节点都与其他所有节点相连。本质上它是一张完整图。

[426] 视频画面时间区间：00:22:00–00:22:15。若为裁剪图，时间区间仍对应原视频画面。

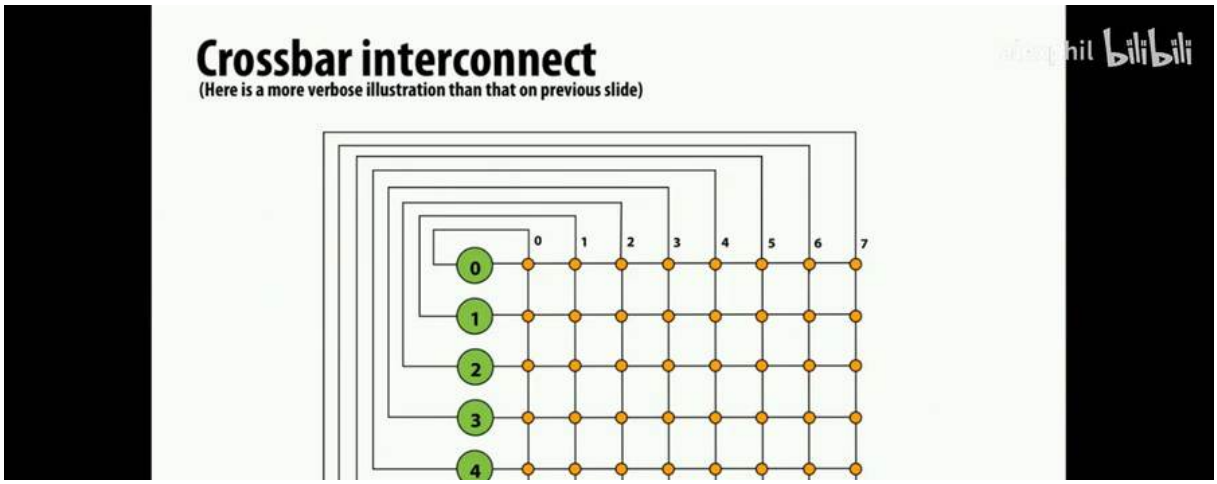

图 427: 交叉开关的更详细图示[427]

讲者澄清了一个容易误读的图示：那张图看起来像 mesh，需要逐跳再跳一下，但它的本意是完整图——每个节点到每个其他节点都有直连，不受限制。

本章小结

拓扑度量用路由距离与直径刻画延迟，用对分带宽度量连通度：mesh 为 $\sqrt{N}$、环为 1、超立方体为 N。三者差距随规模扩大而急剧拉开。阻塞与非阻塞描述并发通信能力，而延迟随负载上升是通用规律。无局部性的随机访问会让低对分带宽成为瓶颈——随机置换下约四分之一流量需跨越中线。

有了这套度量，就可以逐个比较具体拓扑。

19.5 小规模方案：交叉开关与环

交叉开关的面积代价

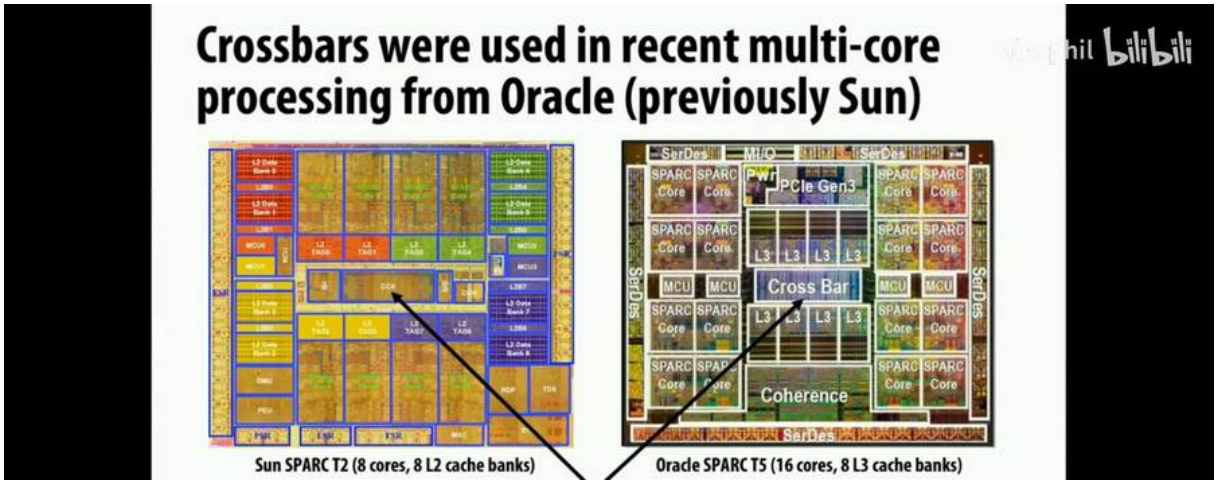

图 428: 交叉开关在 Oracle（原 Sun）多核处理器中的实际使用[428]

[427] 视频画面时间区间：00:25:30–00:25:45。若为裁剪图，时间区间仍对应原视频画面。

[428] 视频画面时间区间：00:24:00–00:24:15。若为裁剪图，时间区间仍对应原视频画面。

讲者展示了 Oracle（原 Sun）的一颗多核芯片。芯片内有一个交叉开关，物理上它占的面积大约相当于一个核——图中竖着的矩形是核。因此这是相当可观的一块面积。

但在芯片上，你确实可以造出交叉开关所需的连通度。这说明了交叉开关的适用边界：它提供最好的连通性，代价是面积随节点数平方增长。

$$\text{面积}_{\text{crossbar}} \propto N^2$$

- N：交叉开关连接的节点数；
- 每条连线交叉处都需要开关资源，因此面积按平方增长；
- 这决定了交叉开关只适用于小规模（如单芯片内的几个核）。

环

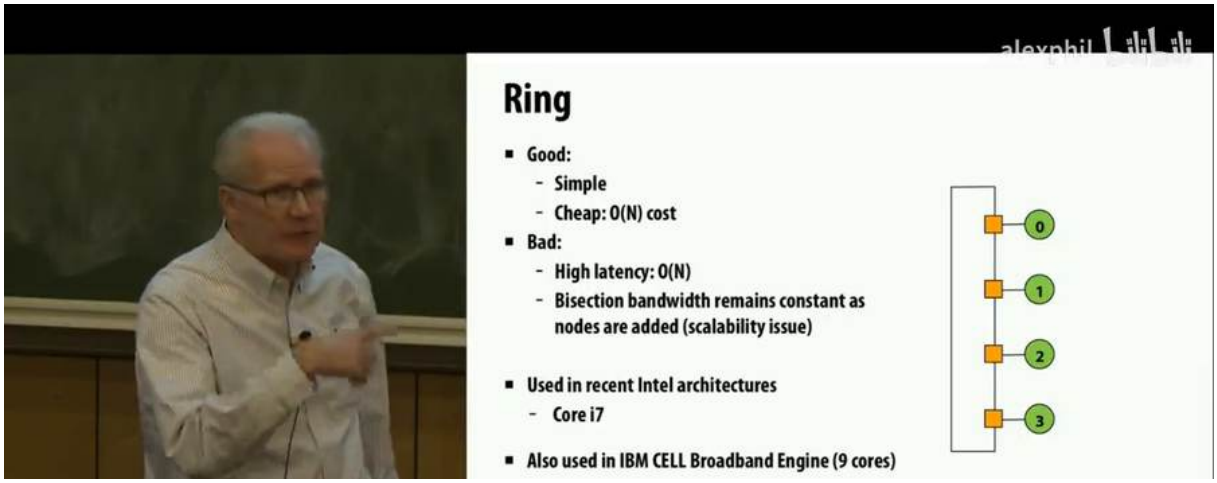

图 429: 环形互连：简单、成本 $O(N)$，但延迟 $O(N)$、对分带宽恒定[429]

环处在另一个极端：消息绕环流动。优点是极易扩展——加倍规模只需加倍环节点。成本是 $O(N)$。

缺点是延迟 $O(N)$，且对分带宽保持不变（因为切断两处就断网）。

讲者说环确实被实际使用，但通常只在四到八个元素的规模上。

[429] 视频画面时间区间：00:26:00–00:26:15。若为裁剪图，时间区间仍对应原视频画面。

Intel 的环

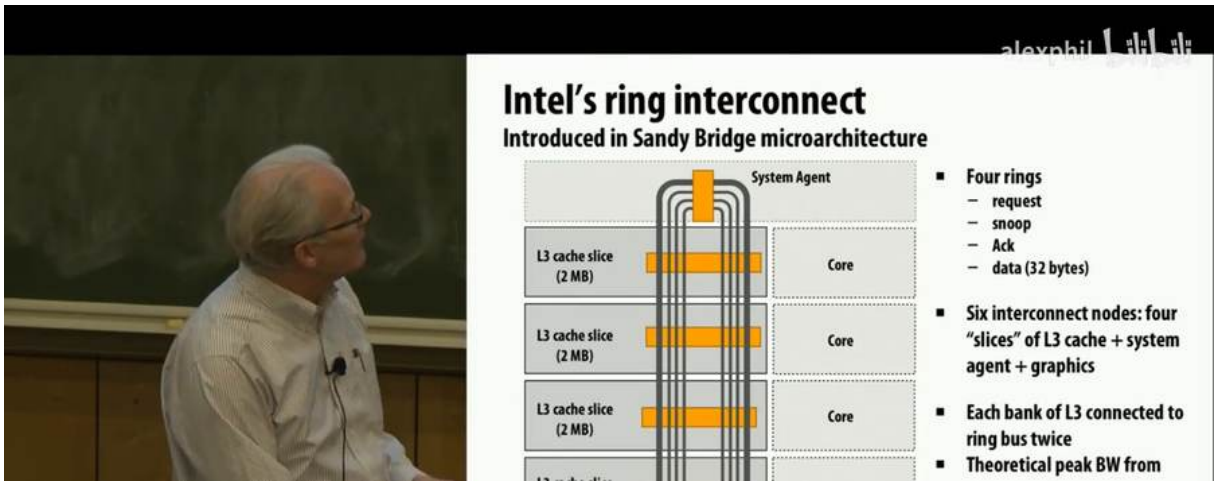

图 430: Intel 的环形互连：Sandy Bridge 引入，四条环连接缓存控制器[430]

讲者给出一个具体例子：Intel 的环形互连，从 Sandy Bridge 微架构引入。图中可以看到四条环，以及 System Agent。

他补充说最新一代叫 Skylake，用的是 mesh 网络，而之前各代都只用环连接缓存控制器。

他还指出课程集群里的 Xeon Phi 是环连接的，这也是它们性能不佳的部分原因。

Intel 从环迁移到 mesh 说明了什么

环在四到八个核时够用，但当核数增长到几十个，环的 $O(N)$ 延迟与恒定的对分带宽成为瓶颈。Skylake 改用 mesh 正是在这个压力下的必然选择：mesh 的对分带宽是节点数的平方根，延迟是 $O(\sqrt{N})$，在中等规模上明显优于环。

本章小结

交叉开关提供完整图连通性，面积按平方增长，只适合单芯片内小规模。环成本 $O(N)$、延迟 $O(N)$、对分带宽恒定，适合四到八个元素，Intel 曾用它连接缓存控制器，Skylake 才转向 mesh。Xeon Phi 用环也是其性能不佳的部分原因。

[430] 视频画面时间区间：00:27:00–00:27:15。若为裁剪图，时间区间仍对应原视频画面。

19.6 网格、环面与折叠环面

mesh

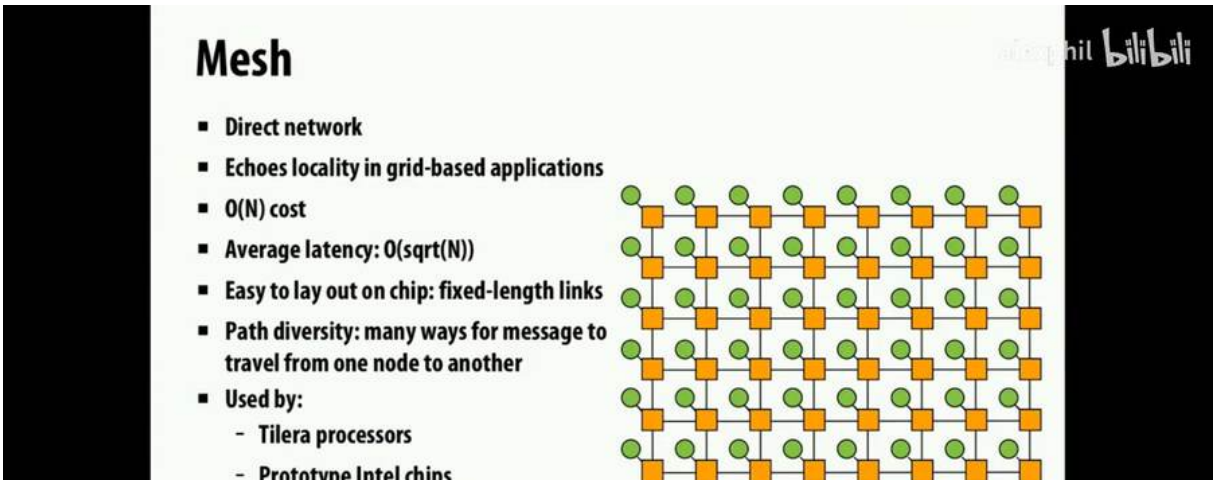

图 431: mesh：直接网络，呼应基于网格的应用的局部性，成本 $O(N)$[431]

mesh 是直接网络——交换机与节点是同一件事，交换逻辑内建在节点里。

拓扑与应用的局部性应当匹配

网格类应用（如二维分块的矩阵运算、格点模拟）的通信天然发生在相邻格点之间，因此 mesh 的物理布局与应用的数据布局同构，通信路径短且可预测。若把这类应用放到胖树上，相邻格点的数据可能被放到相距很远的叶子节点，反而增加跳数。

讲者说它呼应了基于网格的应用的局部性，成本是 $O(N)$，平均延迟优于环。

Xeon Phi 的 mesh

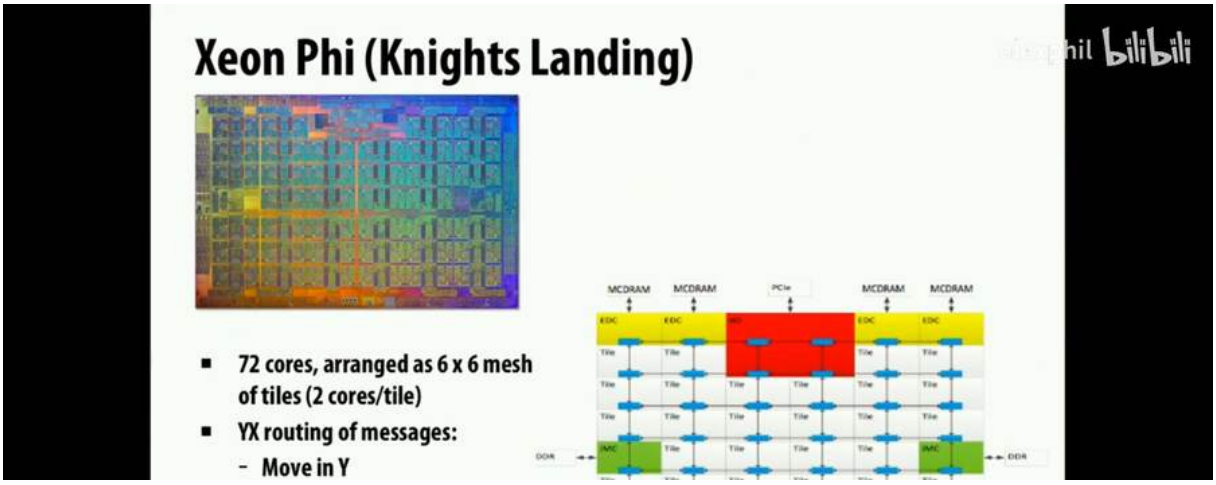

图 432: Xeon Phi（Knights Landing）：72 核排列为二维网格，含 MCDRAM[432]

[431] 视频画面时间区间：00:29:00–00:29:15。若为裁剪图，时间区间仍对应原视频画面。
[432] 视频画面时间区间：00:30:30–00:30:45。若为裁剪图，时间区间仍对应原视频画面。

讲者展示 Knights Landing 的布局：72 个核排列成二维阵列，周围是 MCDRAM（高带宽片上内存）。

环面

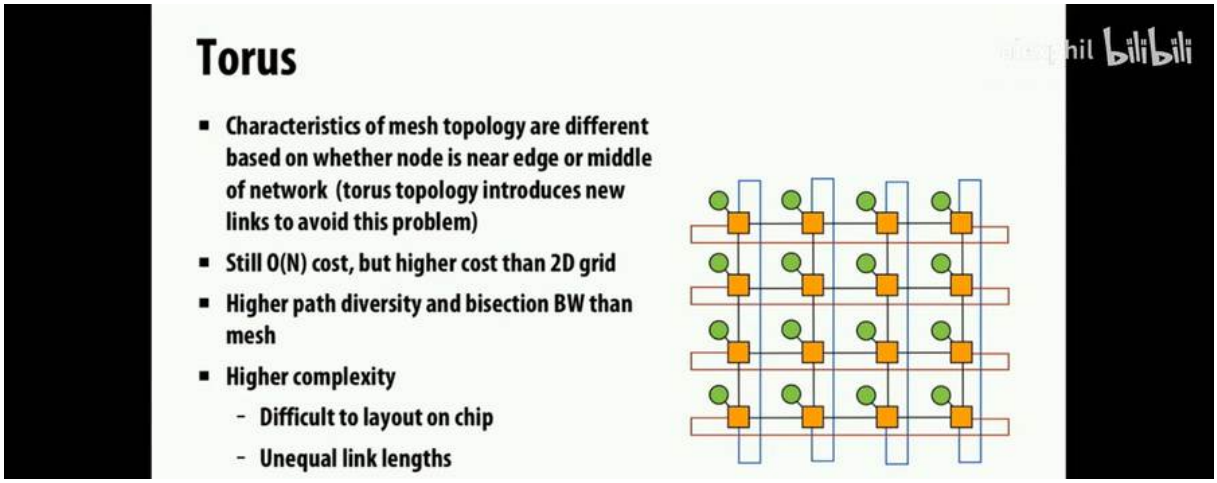

图 433: 环面：mesh 的边界回绕，使边缘节点的连通性接近中心[433]

环面就是 mesh 加上边界回绕。讲者指出 mesh 的特性会因节点位于边缘还是中间而不同，而回绕消除了这种差异。

折叠环面

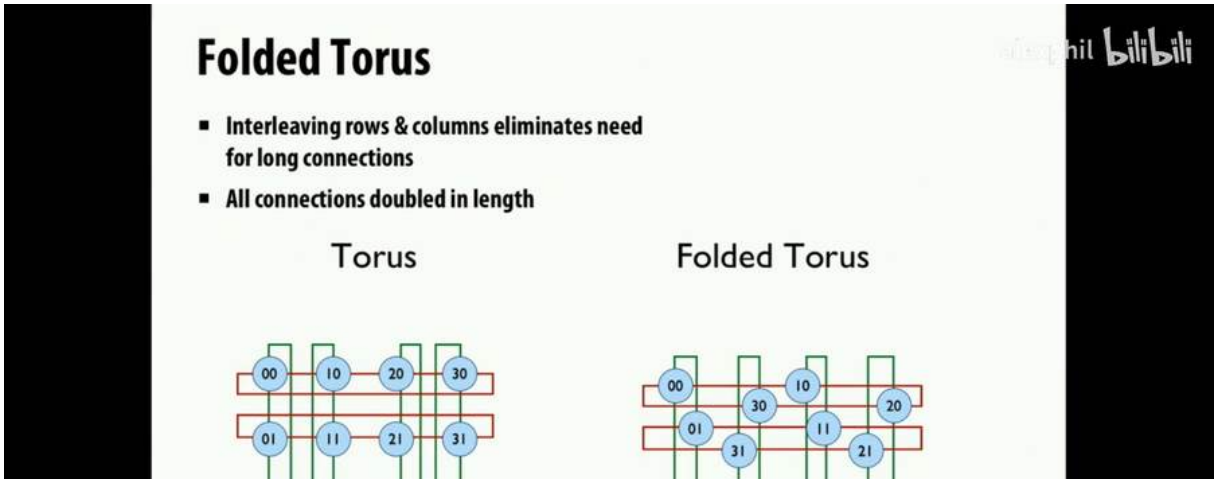

图 434: 折叠环面：交错行列以消除长连线，所有连接等长[434]

折叠环面通过交错行列来消除长连接的需要，使所有连接长度相同。这是物理实现上的优化——长导线在芯片上代价很高。

[433] 视频画面时间区间：00:31:30–00:31:45。若为裁剪图，时间区间仍对应原视频画面。
[434] 视频画面时间区间：00:32:30–00:32:45。若为裁剪图，时间区间仍对应原视频画面。

本章小结

mesh 是直接网络，交换逻辑在节点内，成本 $O(N)$，对分带宽为节点数的平方根，且呼应网格类应用的局部性。环面加边界回绕消除边缘与中心的差异，折叠环面进一步让所有连接等长以适应物理布局。

这些拓扑都依赖通信有局部性。若流量模式是全局的，树结构会更自然。

19.7 树、胖树与超立方体

树与它的路由

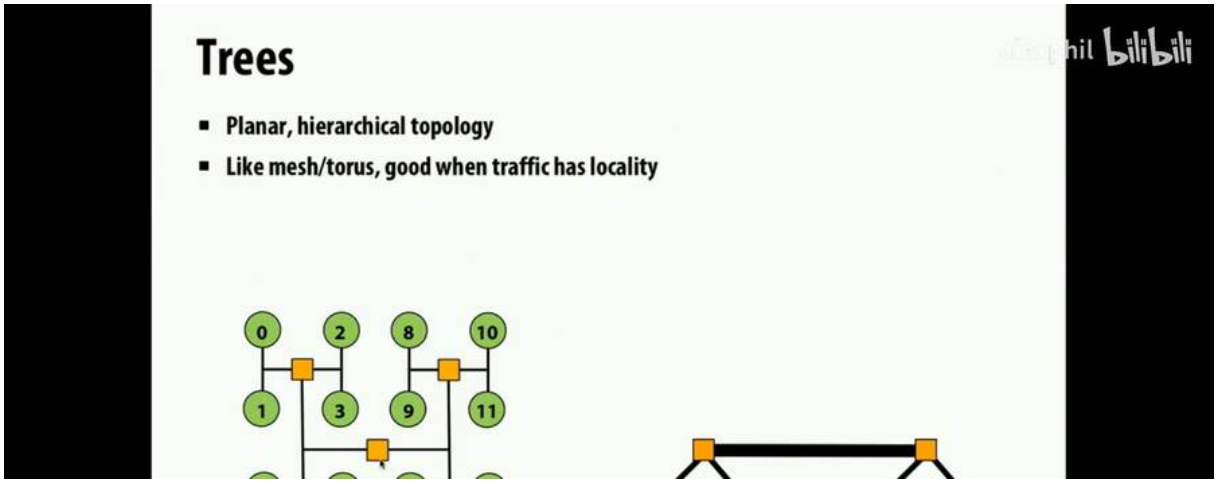

图 435: 树：平面层次化拓扑，流量有局部性时表现好[435]

树是平面层次化拓扑。和 mesh、环面一样，流量有局部性时它表现好。

树的路由极其简单，讲者用例子说明：

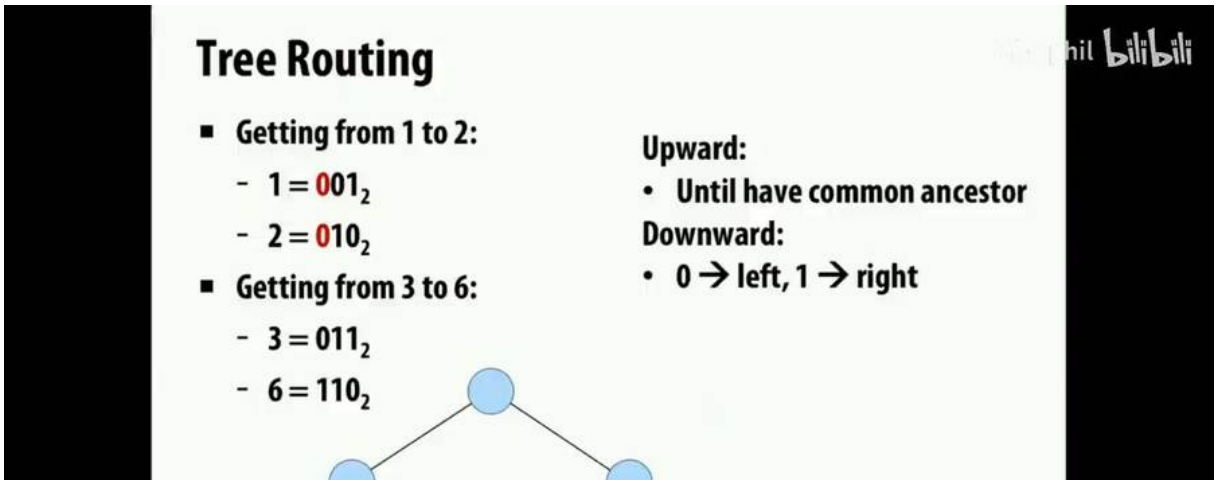

图 436: 树路由：上行到源与目的地址有公共祖先的层，再按位下行[436]

要从节点 1 到节点 2，先向上走。问题是走到哪停？基本规则是看发送方与接收方地址的位表示，上行到它们剩余低位相同的那一层。

[435] 视频画面时间区间：00:35:30–00:35:45。若为裁剪图，时间区间仍对应原视频画面。

[436] 视频画面时间区间：00:38:00–00:38:15。若为裁剪图，时间区间仍对应原视频画面。

例子里，从 1 到 2 需要上行两个位置，再往上就是通往根的共享路径，不必走。
下行时规则很简单：右是 1，左是 0，逐位进行。要到 2 就先右后左。
若要 3 到 6，没有公共祖先，就得一路走到根，然后按位下行 1、1、0。

树路由是位运算，不需要路由表

路由决策完全由源与目的地址的位决定，无需在每个交换机里维护转发表。这让树的硬件极其简单、延迟可预测。代价是树的连通性差，一旦某条上层链路成为热点就无法绕行。

树根的瓶颈与胖树

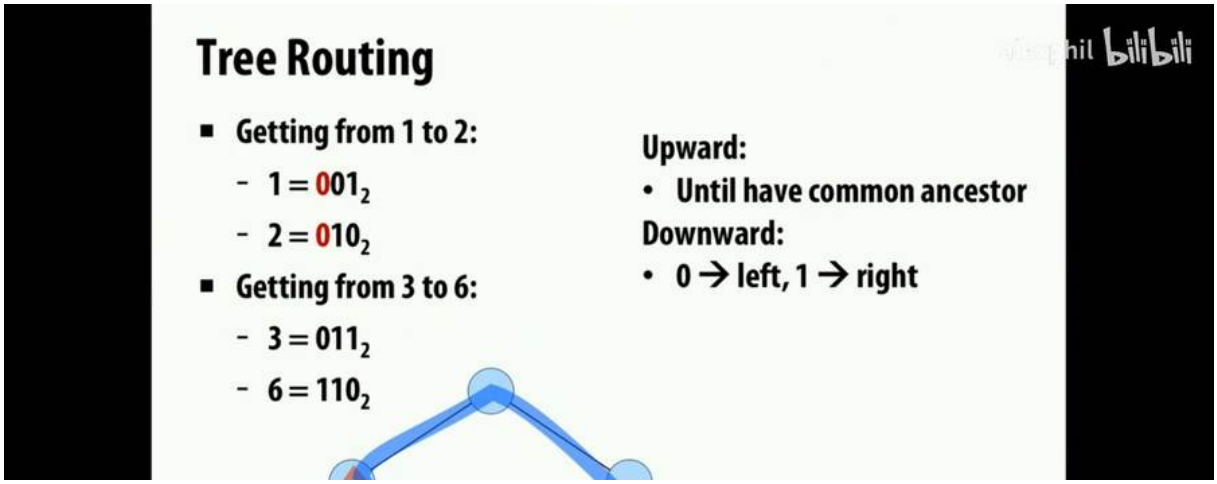

图 437: 胖树：在上层增加链路，使对分带宽不随深度受限[437]

经典树的问题是在根部有可怕的瓶颈。讲者说胖树的想法由 Charles Leiserson 提出（他在 CMU 获得博士学位，此后一直在 MIT 任教）。

他的观察正是讲者前面提到的：在树的上层，我们可以负担得起增加更多链路。这个设计里，若从中间切开，对分带宽是四条链路——和任何位置切开一样好。

胖树用”上层加宽”换取对分带宽

普通树的链路数在下层多、上层少，因此上层成为瓶颈。胖树让上层链路数成倍增加，使任意切口的对分带宽相同。代价是上层需要更多物理连线与交换机端口。这是用硬件资源换连通性的直接交易。

等宽胖树

讲者还提到一个变体：等宽胖树，有时也叫折叠 Clos 网络。它的特点是所有节点度固定，实现更简单。

[437] 视频画面时间区间：00:40:00–00:40:15。若为裁剪图，时间区间仍对应原视频画面。

超立方体

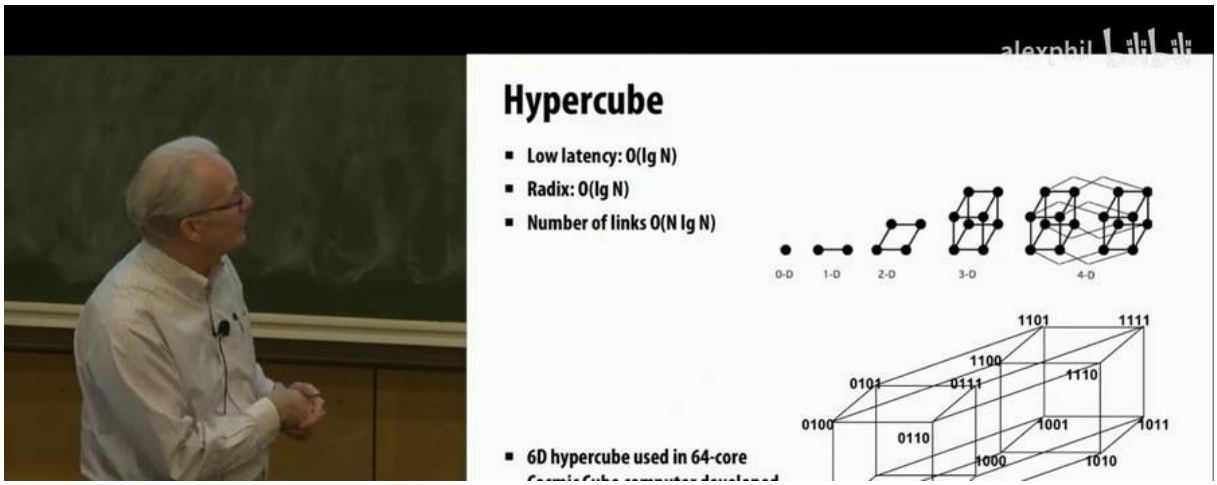

图 438: 超立方体：延迟 $O(\lg N)$、对分带宽随节点数增长，但物理难布线[438]

超立方体的指标很漂亮：延迟 $O(\lg N)$，节点度 $O(\lg N)$，链路数 $O(N \lg N)$，对分带宽随节点数线性增长。

讲者给出 0 维、1 维、2 维、3 维、4 维的递进图示。

但它有一个严重问题：物理上你无法在三维空间里表达更多维度。四维立方体还能靠塞进足够多的导线实现，但规模再往上，导线就成了一团密集的网，无法很好地布线。

于是它有一个漂亮的对分带宽，却有一个糟糕的性质：超过某个限度后，无法嵌入任何可实现的物理结构。

讲者补充说可以推广：不只每个地址位对应一个二选一，而是让每维有更高的度。他说 Cray 造过的一些超算确实用了四维立方体，但不是二元的，每个节点的度更高。

理论指标优秀不等于可实现

超立方体的延迟与对分带宽在所有拓扑中都属最优，但它的连线在三维空间中会交叉成灾。这说明拓扑评估不能只看渐近指标，还要考虑物理布局成本——导线长度、层数、交叉。实际系统因此在指标与可实现性之间折中，选择了胖树或高维环面。

[438] 视频画面时间区间：00:50:00–00:50:15。若为裁剪图，时间区间仍对应原视频画面。

omega 网络

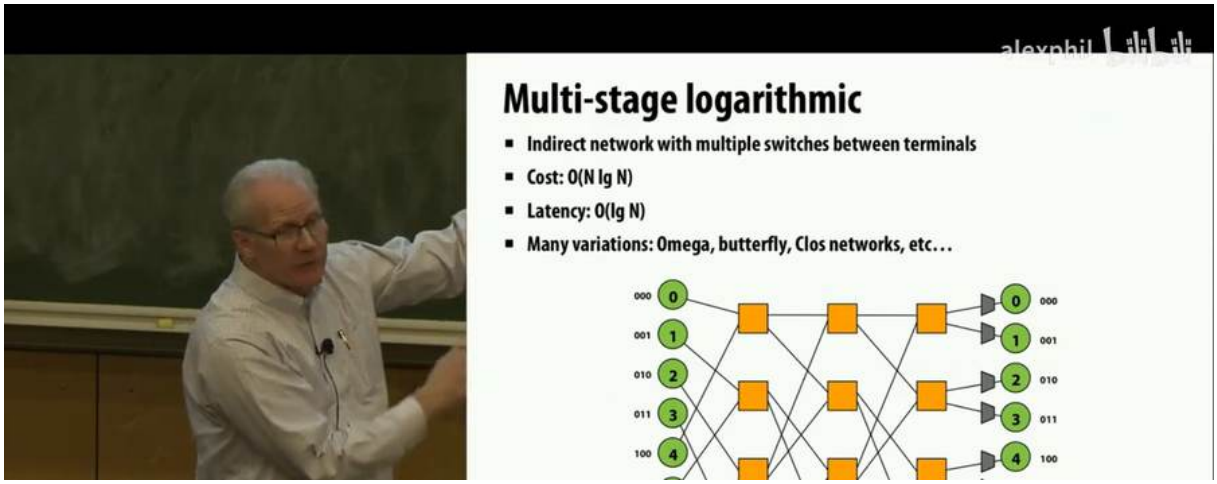

图 439: omega 网络：多级对数网络，用约 $N \log N$ 个开关连接 N 个终端[439]

omega 网络是一个经典设计。讲者指出画法上有误导：左端和右端的节点其实是同一批，只是画成两列便于理解。

实际结构是 N 个节点，中间大约 $N \log N$ 个开关（更准确地说是每级 $N/2$ 个开关、共 $\log N$ 级），提供从一端到另一端的通路。

它的特点是：第一级跨越较大间隔，到末端只做相邻的交换。

$$\text{开关数}_{\text{omega}} = \frac{N}{2} \log_2 N$$

- N：终端节点数；
- $\log_2 N$：级数；
- 每级 $N/2$ 个二选一开关，因此总数与 $N \log N$ 同阶。

本章小结

树的路由完全由地址位决定，硬件极简但根部是瓶颈。胖树在上层加宽链路，使对分带宽不随切口位置变化，代价是更多连线。等宽胖树让节点度固定以简化实现。超立方体指标最优但物理上难以布线，实际系统用更高节点度的推广版本。omega 网络用 $N \log N$ 个开关构成多级网络。

[439] 视频画面时间区间：00:52:00–00:52:15。若为裁剪图，时间区间仍对应原视频画面。

19.8 总结与延伸

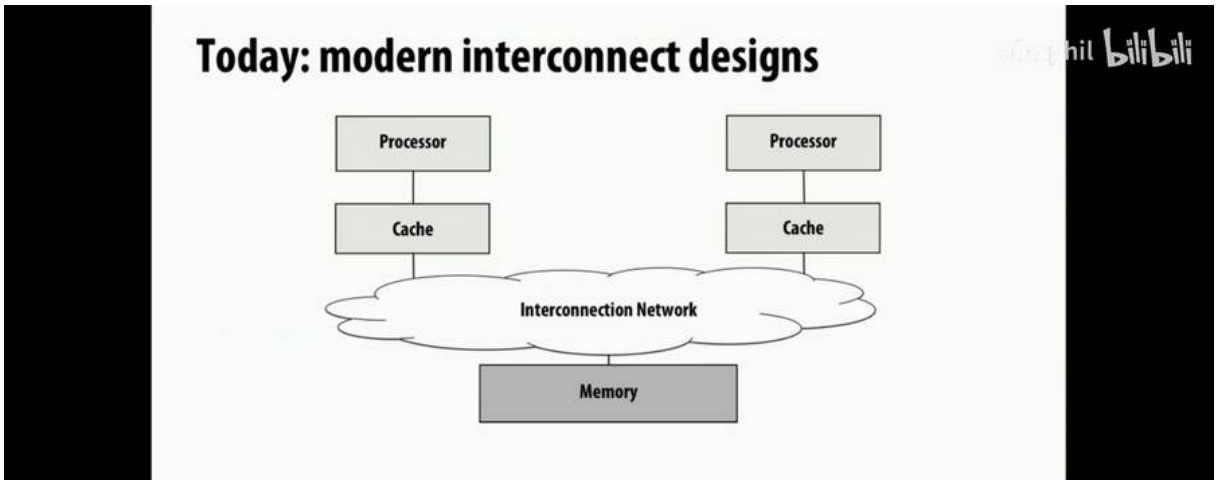

图 440: 现代互连网络设计：处理器、缓存与互连的整体视图[440]

讲者在结尾回到一个现实观察：网络正越来越成为从机器中取得性能的关键问题。他提到一个具体的经验——把相关的作业都集中放在数据中心的同一个角落、以确保它们之间有足够带宽，这件事比直接把网络配置得足够宽、让作业可以随机放置并仍取得不错性能要困难得多。

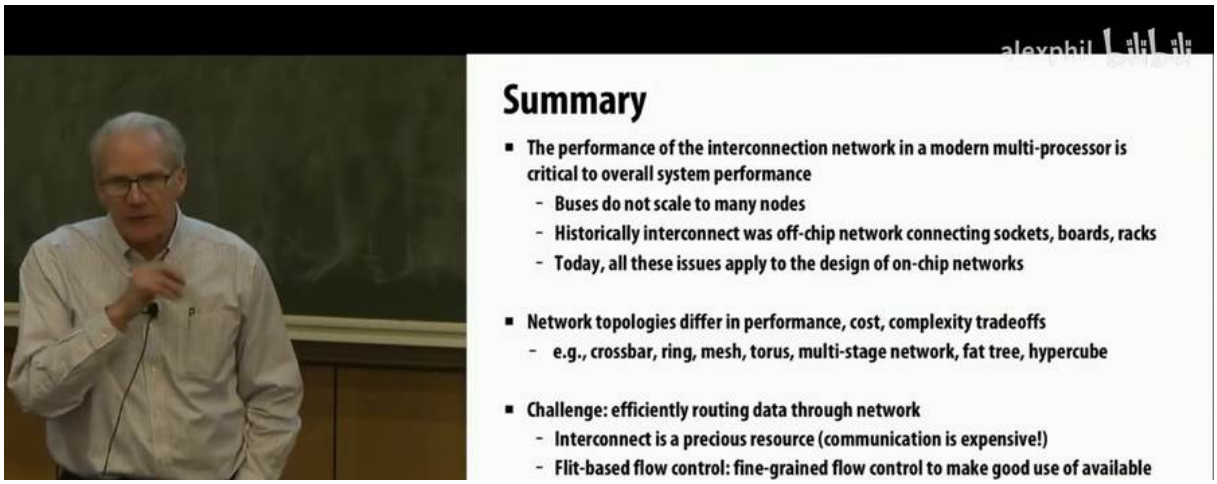

图 441: 总结：互连网络的性能对现代多核处理器的整体性能至关重要[441]

把全讲串起来，这是一份关于”如何把消息送到远处”的设计地图。互连网络连接 HPC 节点，目标与互联网不同：紧密局部、极高性能，可较少考虑故障。最简单形式是总线，请求与响应须物理分离以避免死锁。设计需权衡性能、能耗与可扩展性，厂商希望同一设计覆盖 256 到 1024 等多种规模。四个设计问题是拓扑、路由、缓冲与流量控制；拓扑用直径与对分带宽度量，mesh 为节点数的平方根、环为 1、超立方体为节点数。无局部性负载会让低对分带宽成为瓶颈，而延迟随负载上升是通用规律。具体拓扑从交叉开关（完整图，面积平方增长）到环（$O(N)$ 延迟、恒定对分带宽）、mesh 与环面（呼应局部性）、树与胖树（路由由地址位决定，胖树在上层加宽消除根瓶颈）、超立方体（指标最优但难布线）、omega 网络（$N \log N$ 个开关的多级网络）。

本讲义在讲者材料之上做的压缩，有四点需要与讲者原话区分：

[440] 视频画面时间区间：00:03:00–00:03:15。若为裁剪图，时间区间仍对应原视频画面。
[441] 视频画面时间区间：01:12:40–01:12:55。若为裁剪图，时间区间仍对应原视频画面。

- 对分带宽是最坏情况指标，不能单独用于判断拓扑优劣。它假设流量均匀跨越中线，而真实应用的通信常有局部性。因此评估时应把对分带宽与应用的访问模式一起看，否则会低估局部性友好的拓扑。
- 拓扑选择由规模与流量模式共同决定，不存在普遍最优。环在四到八核时够用，几十核时 mesh 更好，上百节点则需要胖树。同一套设计要覆盖多个规模，这正是厂商选择层次化方案的原因。
- 物理可实现性是理论指标之外的一等约束。超立方体在所有渐近指标上都优秀，却因布线困难而很少被采用。评价任何拓扑都必须同时问” 能不能造出来”。
- 路由简单性本身有价值。树路由只需地址位运算，无需路由表，因此延迟可预测、硬件成本低。复杂拓扑的灵活路由往往要付出表项与查找延迟的代价。

这堂课的边界在于它是导论：讲者只给出拓扑与指标的比较，未深入具体路由算法、缓冲分配策略与流量控制协议。真实互连的工程细节远超本讲范围。

两个可以直接从本讲数据提出的下一步问题：其一，既然对分带宽是最坏情况指标，那么对于一个通信有明显局部性的应用，应当用哪个指标来替代它做评估？其二，胖树通过在上层增加链路消除了根瓶颈，但上层链路数增加会带来什么新的代价，这个代价在什么规模下开始主导？

Lecture 20 Implementing Synchronization

视频时长：1 小时 5 分 45 秒　来源：`https://www.bilibili.com/video/BV1au4y1i7DD/`

20.1 同步原语的实现：从线程映射说起

讲者开场先回顾线程如何映射到硬件，因为同步的开销与这个映射直接相关。

硬件线程与活跃线程

一个处理器有若干核，每个核支持若干个执行上下文。Intel 把这个叫超线程，正式名称是同时多线程。典型情况下每个核两个。

因此在任一时刻，硬件能同时处理的线程数上限就是执行上下文的数量。

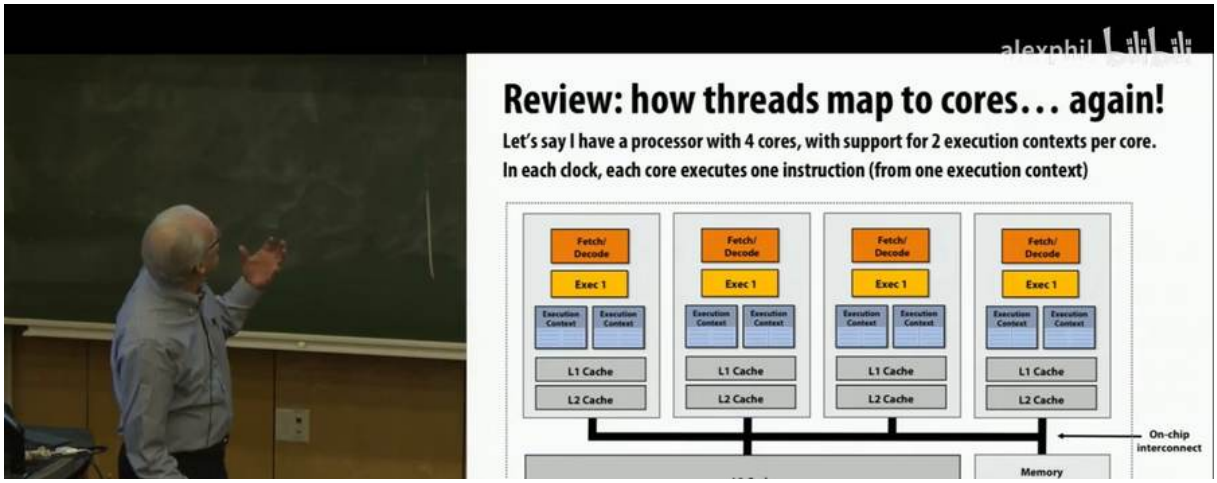

图 442: 回顾：线程如何映射到核心与执行上下文[442]

但若在机器上跑 top 或列出活跃进程，通常会发现多得多的线程。讲者用一台笔记本举例：浏览器、各种主动使用的程序，加上 Dropbox 之类被动运行的服务——它们都在后台保持进程，以便有事发生时能响应。

于是可能有 200 个以上的潜在活跃线程，远多于可执行的硬件线程数。

[442] 视频画面时间区间：00:12:00–00:12:15。若为裁剪图，时间区间仍对应原视频画面。

图 443: 一台机器上可以并发运行许多程序[443]

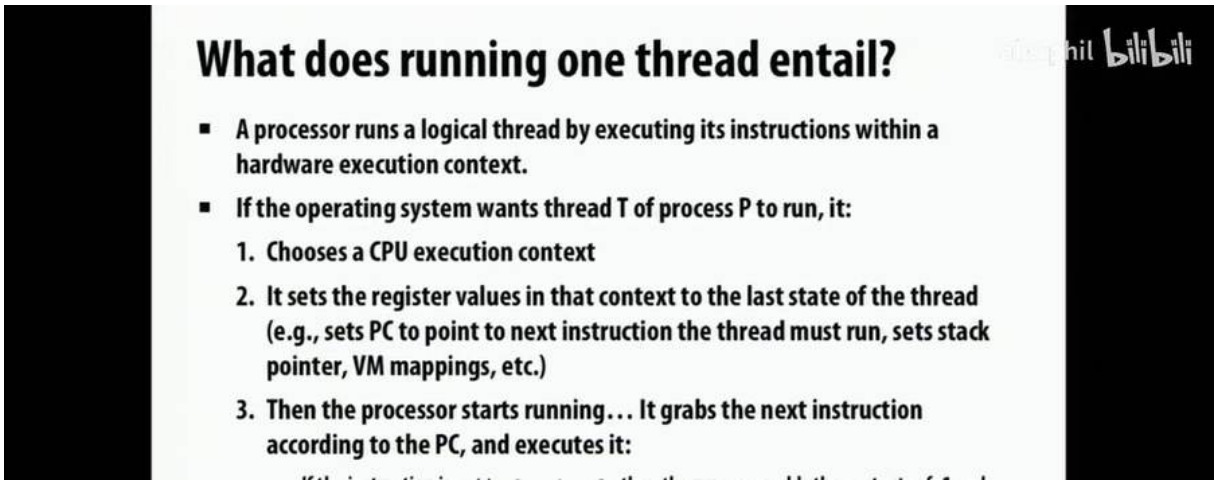

图 444: 运行一个线程意味着什么：处理器执行其指令流[444]

操作系统做了什么

操作系统必须决定哪些线程应该运行。它会挑一个核、挑该核上的一个执行上下文，把线程分配过去。这涉及从存储的状态中取出寄存器值。

[443] 视频画面时间区间：00:13:00–00:13:15。若为裁剪图，时间区间仍对应原视频画面。
[444] 视频画面时间区间：00:14:00–00:14:15。若为裁剪图，时间区间仍对应原视频画面。

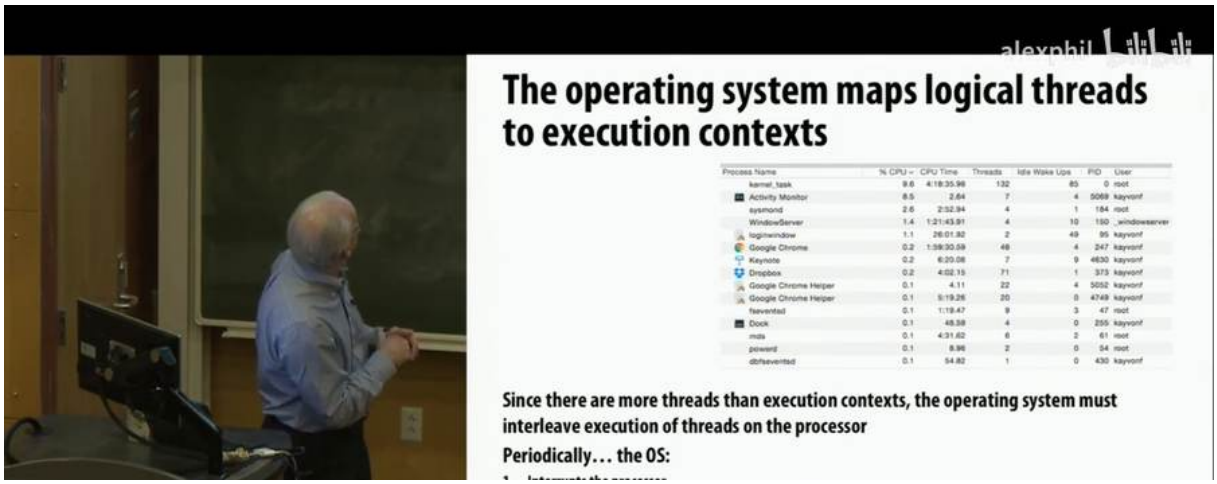

图 445: 操作系统把逻辑线程映射到执行上下文[445]

图 446: 两个执行上下文如何在一个每周期只能发一条指令的核上运行[446]

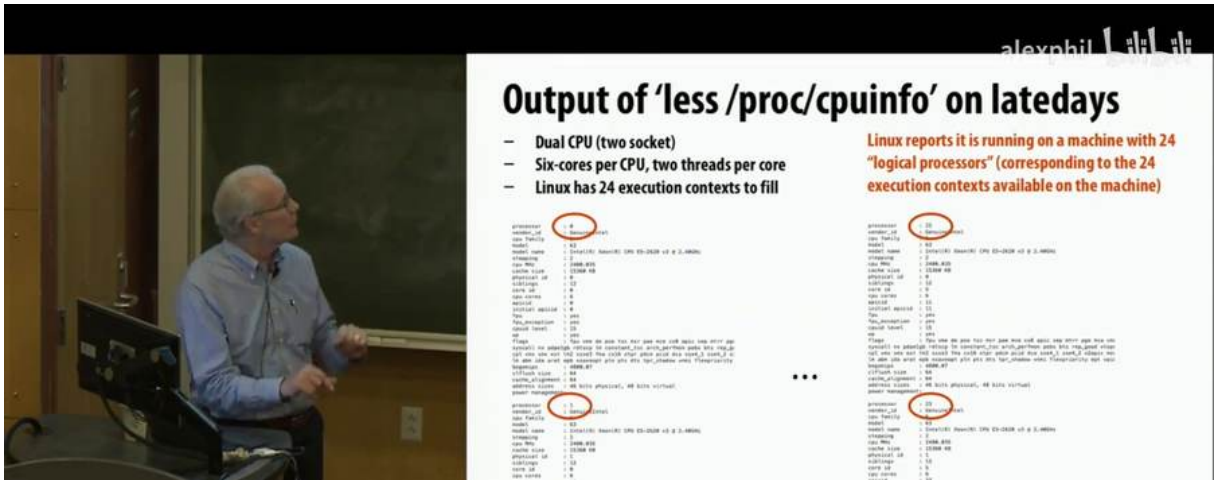

图 447: late days 机器上 /proc/cpuinfo 的输出：双插槽[447]

[445] 视频画面时间区间：00:15:30–00:15:45。若为裁剪图，时间区间仍对应原视频画面。

[446] 视频画面时间区间：00:16:00–00:16:15。若为裁剪图，时间区间仍对应原视频画面。

同步开销的量级由”是否牵涉操作系统”决定

纯硬件的原子操作在几十到几百个周期量级，而任何涉及操作系统调度的操作（把线程放入等待队列、切换上下文）在数千周期量级。两者相差一到两个数量级，因此选择忙等待还是阻塞，本质上是在选择付出哪一档开销。

本章小结

硬件线程数远少于活跃线程数，操作系统负责把线程映射到执行上下文。同步的开销量级取决于是否牵涉操作系统：纯硬件是几十到几百周期，阻塞式是数千周期。这个差距决定了后面所有设计取舍。

下一章看同步事件本身包含哪几个阶段。

20.2 同步事件的三个阶段

获取、等待、释放

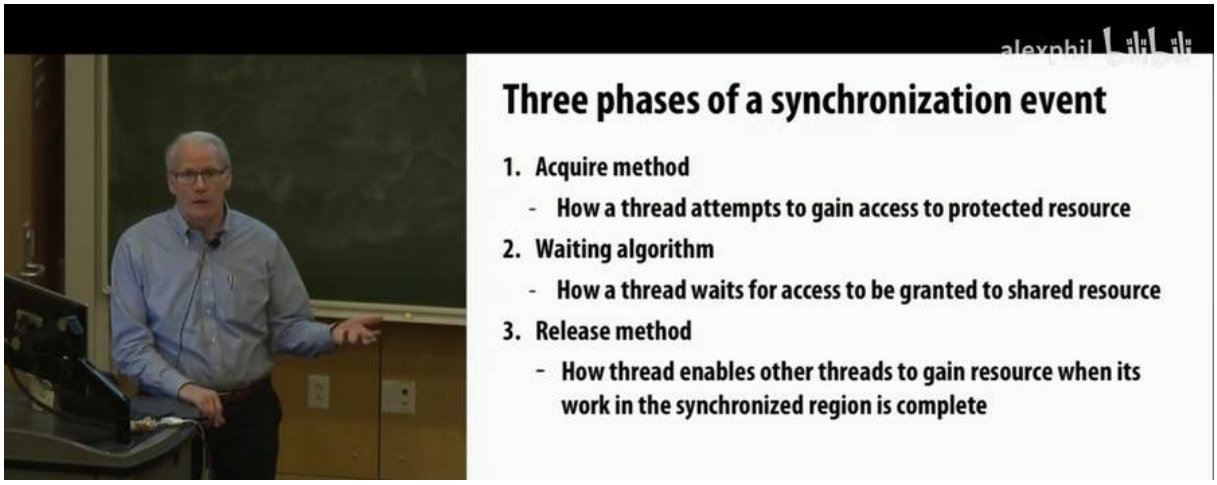

图 448: 同步事件的三个阶段：获取方式、等待期间做什么、释放方式[448]

讲者把一次同步拆成三段。第一是获取：一个线程如何尝试取得锁或某种访问受保护资源的许可。第二是等待：如果没能取得，在等待期间做什么。第三是释放：用完这段临界区之后，如何把锁交出去。

三阶段框架让比较变得可比

不同锁实现在获取方式上可能差别很大，但等待与释放的方式往往决定了它在竞争下的表现。把实现拆成三段后，就可以分别问：获取在无争用时是否够快、等待是否产生流量、交接是否公平。这比笼统比较”哪个锁更快”更有信息量。

这个三段划分是后面所有比较的框架。

[447] 视频画面时间区间：00:17:00–00:17:15。若为裁剪图，时间区间仍对应原视频画面。

[448] 视频画面时间区间：00:19:00–00:19:15。若为裁剪图，时间区间仍对应原视频画面。

忙等待

最朴素的做法是忙等待，也叫自旋：在循环里检测条件，不成立就再测一次。释放则是把条件置为真，简单得几乎不需要解释。

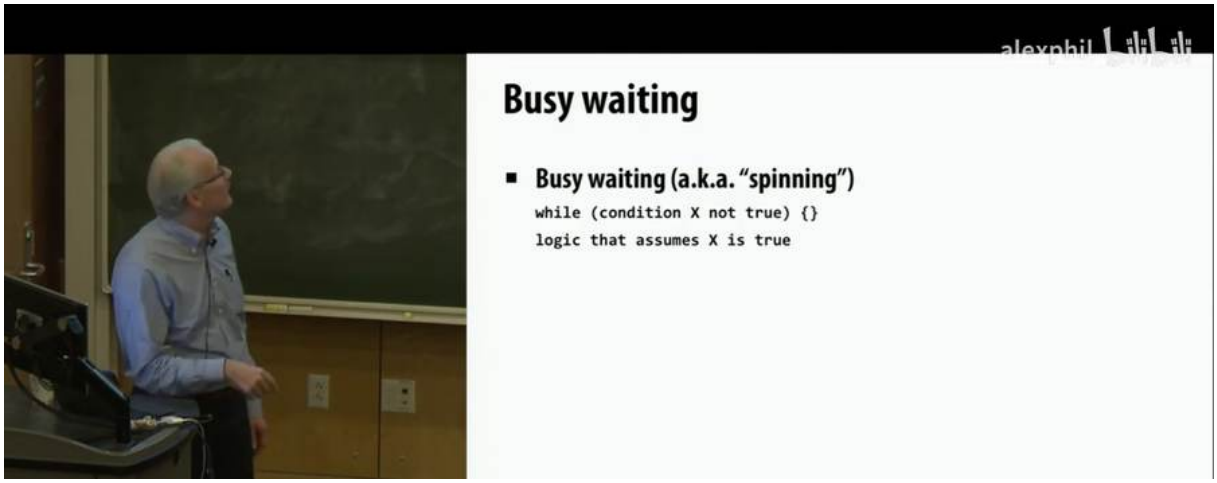

图 449: 忙等待：循环检测条件直至成立[449]

你可能听说过忙等待是坏事——这大体上正确，因为处理器在那个循环里没做任何有用的事，只是绕回自己。

但在并行计算里它未必坏

讲者给出一个重要的限定：在高并行、以性能为导向的系统中，忙等待没那么糟。因为如果你没在等待，本来也没有别的事可做。

他举例说，我们试图用满全部十二个处理器；如果不执行应用代码，那确实没什么有用的活可干。

因此自旋的机会成本可以写成：

$$\mathrm{cost}_{\mathrm{spin}} = \begin{cases} 0, & \text{若没有其他可运行线程} \\ T_{\mathrm{wait}} \cdot \text{可复用算力}, & \text{否则} \end{cases}$$

- 在并行计算程序里通常落在第一种情形，因为所有核都已被占满；
- 在多用户系统里落在第二种情形，自旋挤占了本可给别人用的时间片。

而且忙等待有个明确的优势：等待与释放的开销都很低。

[449] 视频画面时间区间：00:19:30–00:19:45。若为裁剪图，时间区间仍对应原视频画面。

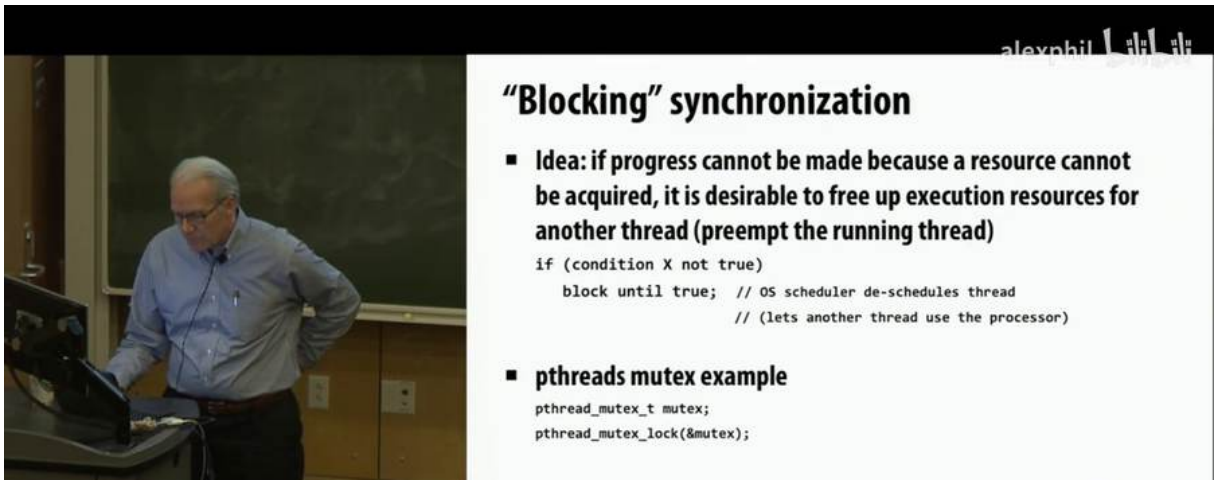

图 450: 阻塞式同步：若资源无法获取则让出控制权[450]

阻塞式等待

另一种做法是：如果等待的条件不成立，就交出对这些硬件资源的控制权，把当前线程放到一边，换一个已就绪的线程进来执行。

讲者说在使用 pthreads 互斥量时就是这个行为——线程要么自旋，要么阻塞。

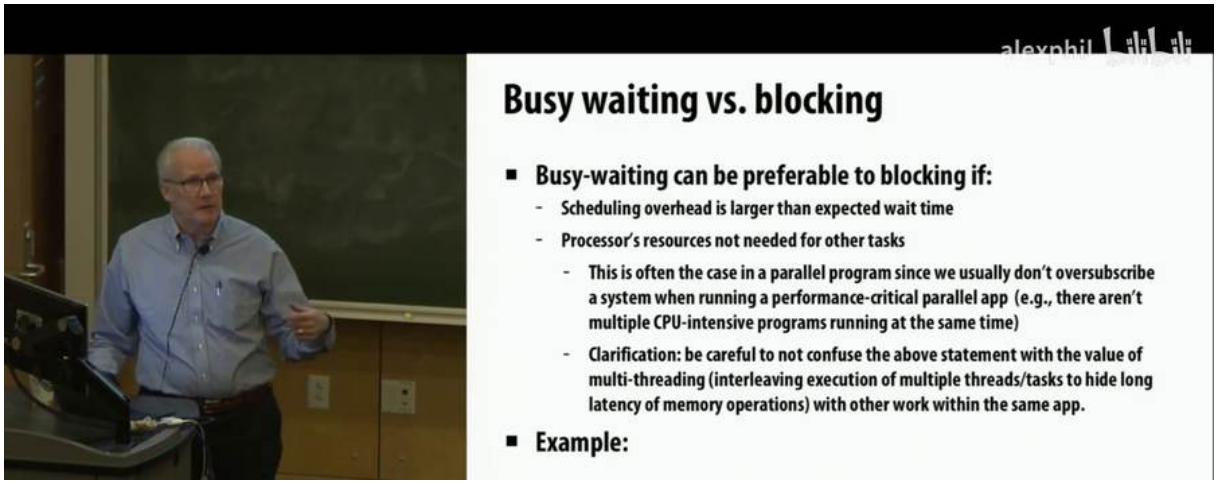

图 451: 忙等待与阻塞的对比：开销量级差异明显[451]

在被许多用户或许多类型进程共享的系统里，阻塞显然更可取，但它有不小的开销。忙等待完全不需要操作系统介入，是纯硬件的事；阻塞则需要被操作系统放入某个队列。

于是量级差别出来了：任何涉及操作系统的操作都在 2000 个时钟周期以上；纯硬件的操作只需 50 到 200 个周期。

[450] 视频画面时间区间：00:21:00–00:21:15。若为裁剪图，时间区间仍对应原视频画面。

[451] 视频画面时间区间：00:22:00–00:22:15。若为裁剪图，时间区间仍对应原视频画面。

忙等待的适用条件

讲者给出的判据其实很明确：若处理器除了等待没有别的活可干，自旋的代价只是浪费算力而不浪费别人的时间；若系统里还有别的线程可以推进，自旋就浪费了本可复用的执行资源。因此并行计算程序倾向自旋，而通用操作系统倾向阻塞。

本章小结

同步分获取、等待、释放三阶段。忙等待是循环检测，开销极低，在高并行计算中并不坏，因为等待时本无别事可做。阻塞式等待让出控制权，适合多用户共享系统，但需操作系统介入，开销高数千倍。

这个框架可以用来分析任何锁的实现。

20.3 一个看似正确却有缺陷的自旋锁

朴素实现

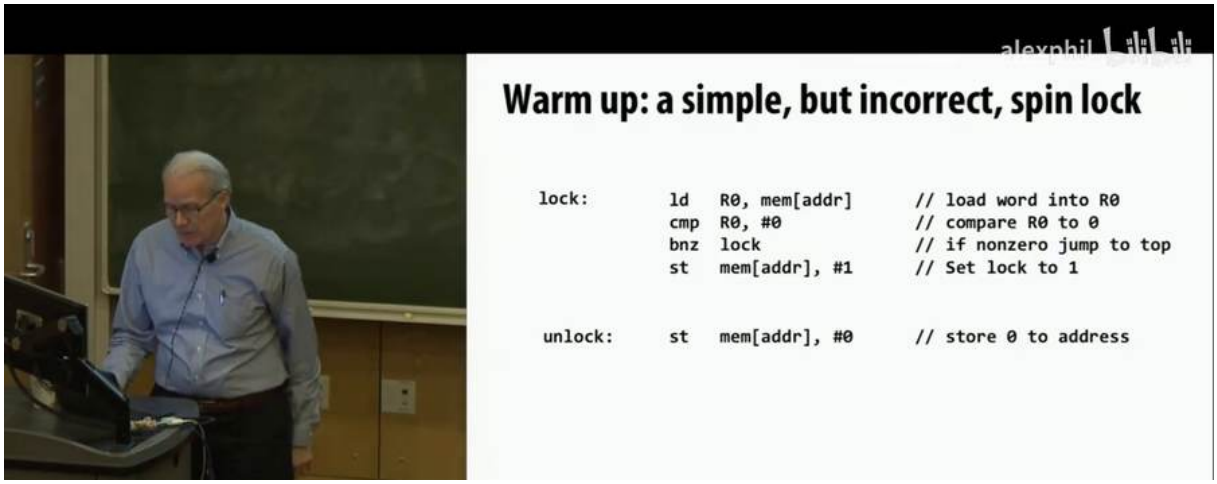

图 452: 热身：一个简单但有缺陷的自旋锁[452]

讲者用一段伪汇编说明。加锁部分：从内存把共享变量读入寄存器，与零比较；若非零则跳回开头继续自旋；若为零则把该内存位置置为一。解锁就是把内存置零，好让其他在自旋的程序看到变化、抢到它并再次置一。

讲者说大多数人已经知道这段代码有 bug。他的提问得到的回答是：它不是原子的。

[452] 视频画面时间区间：00:23:00–00:23:15。若为裁剪图，时间区间仍对应原视频画面。

缺陷在哪

图 453: 缺陷所在：测试与置位之间存在可被插入的窗口[453]

问题正是这个：别的进程可能恰好插进中间。你做完测试、发现是零，正准备写一，而另一个进程同时也这么想。于是可能有多个线程一起通过这道锁。

讲者补充说现实中不太容易发生，因为——现在你们已经看过缓存协议了——尽管内存操作相当快，但要作废共享副本、把它们推出去、再取回一份，是件不小的工程。所以实际出错概率不高。

这个窗口的大小可以粗略表示为：

$$T_{\text{window}} \approx T_{\text{load}} + T_{\text{invalidate}} + T_{\text{transfer}}$$

- T_{load}：读取锁值的时间；
- $T_{\text{invalidate}}$：作废其他持有者副本的时间；
- T_{transfer}：把缓存行取回本地的时间；
- 窗口越长，其他线程插入的机会越大。

因此朴素自旋锁的安全性依赖一个未声明的假设：窗口足够短。这个假设在核数增加时会失效。

但流量问题很严重

讲者的分析很关键：若一堆进程在跑同一段代码、都在等同一把锁，它们会把这份锁的只读副本拉进各自缓存。于是自旋会跑得飞快，几乎不产生总线流量。

但一旦解锁发生，就会有一场巨大的争夺，所有等待者都要去更新它。因此确实很可能出现同步错误。

低出错概率不等于正确

”实际不太容易发生”是缓存协议延迟带来的副作用，而不是任何保证。处理器数量增加、缓存一致性实现变化、或编译器重排都可能让这个窗口变得更容易命中。依赖概率来保证正确性在并发代码中是不可接受的——必须用原子原语。

[453] 视频画面时间区间：00:25:00–00:25:15。若为裁剪图，时间区间仍对应原视频画面。

本章小结

朴素自旋锁用普通读写实现，测试与置位之间存在窗口，多个线程可能同时通过。实际出错概率因缓存协议的开销而降低，但这不构成保证。更严重的是解锁瞬间所有等待者一起争抢，产生大量一致性流量。

这引出对硬件原子原语的需求。

20.4 硬件原语：test-and-set 与 cmpxchg

test-and-set

多数处理器提供了专门为此设计的指令，最经典的是 test-and-set。

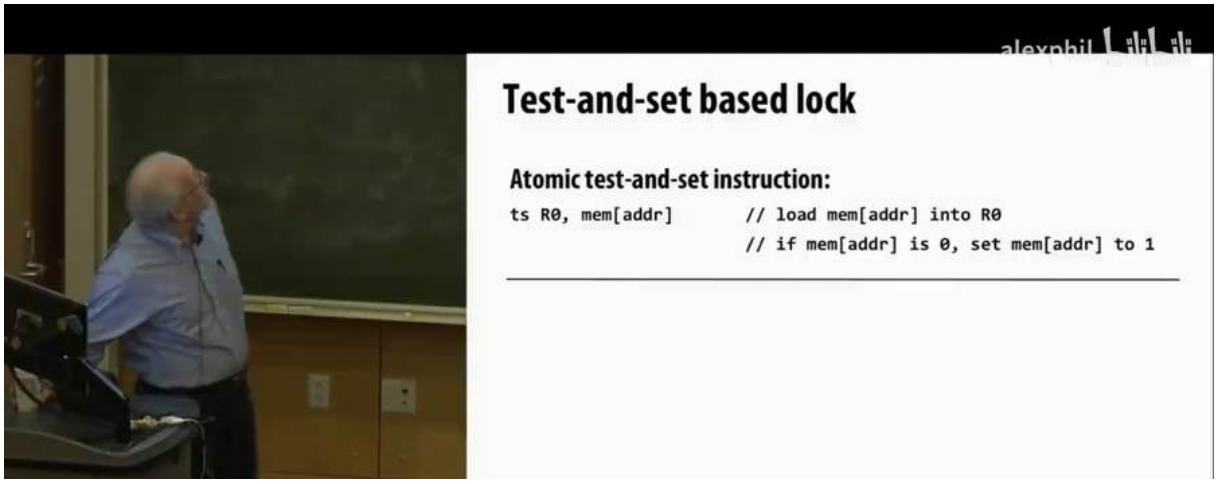

图 454: 基于 test-and-set 的锁：原子地读取并把零置一[454]

它的语义是：原子地把一个内存位置读入寄存器，若该值为零则置一。还有各种变体，但经典形式就是这样。

讲者强调这些语义非常专用，显然只对同步有用。关键点是它原子——读、测试、写永远是一个不可中断的动作，读写被保证锁在一起。事后可以通过查看寄存器零判断是否成功取得。

原子性保证的是”不可分割”，不是”不会失败”

test-and-set 的原子性确保读与写在硬件层面不被插入，但并发线程仍可能在你之前成功获取锁，此时你读到的是旧值一、获取失败。因此原子指令解决的是正确性，循环重试解决的是进展性——两者缺一不可。

锁应独占一个缓存行

讲者顺带提到一个实现细节：锁虽然逻辑上只需要一位，但通常实现为一个数据字，而且往往要放在单独的缓存行里。

[454] 视频画面时间区间：00:26:00–00:26:15。若为裁剪图，时间区间仍对应原视频画面。

把锁放在单独缓存行是伪共享的预防措施

若锁与其他频繁访问的变量共享一行，那么每次加解锁都会作废那一行，连带影响其他变量的缓存。把锁独占一行，使加解锁的流量只影响锁本身。这与第 13 讲的伪共享是同一问题的两种表现。

一致性流量分析

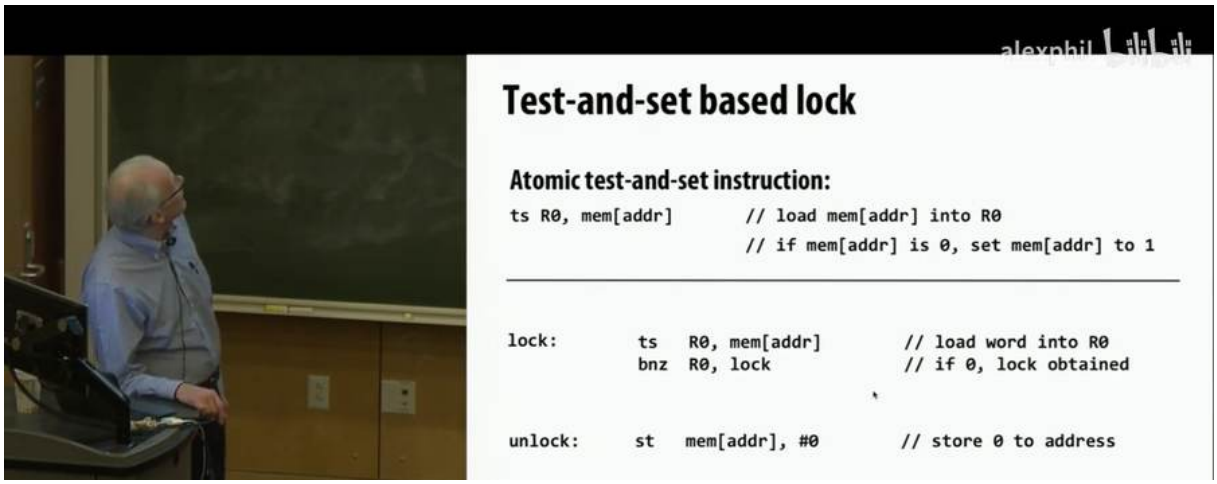

图 455: test-and-set 锁：分析其一致性流量[455]

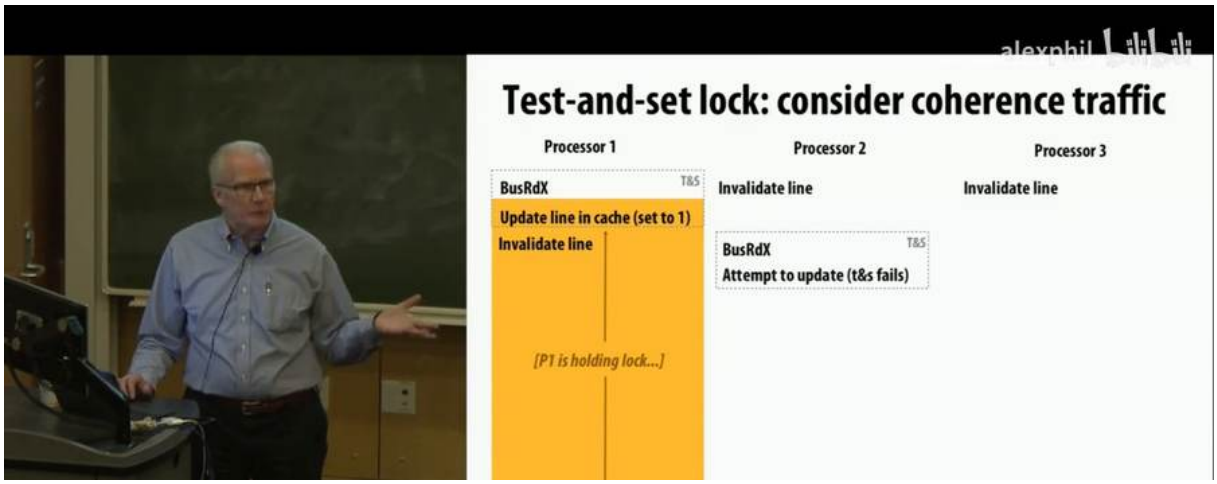

图 456: 流量分析续：解锁时所有处理器都要作废自己的副本[456]

讲者用 3 个处理器的场景分析：每次 test-and-set 都要向总线发出读并申请独占，使其他 2 个持有者的副本作废。

[455] 视频画面时间区间：00:27:30–00:27:45。若为裁剪图，时间区间仍对应原视频画面。

[456] 视频画面时间区间：00:29:00–00:29:15。若为裁剪图，时间区间仍对应原视频画面。

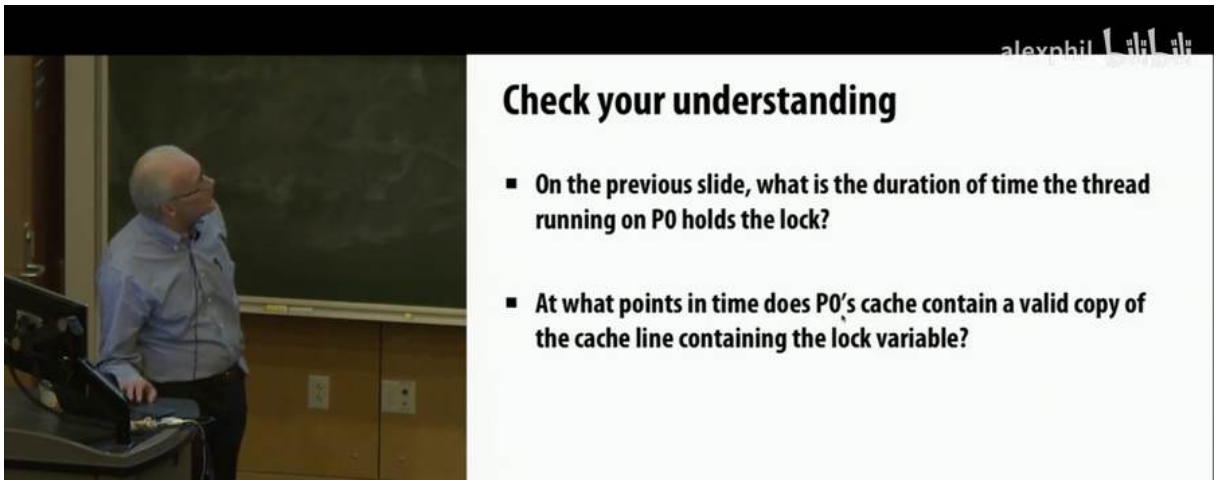

图 457: test-and-set 锁的性能基准：执行 N 次加解锁[457]

x86 的 cmpxchg

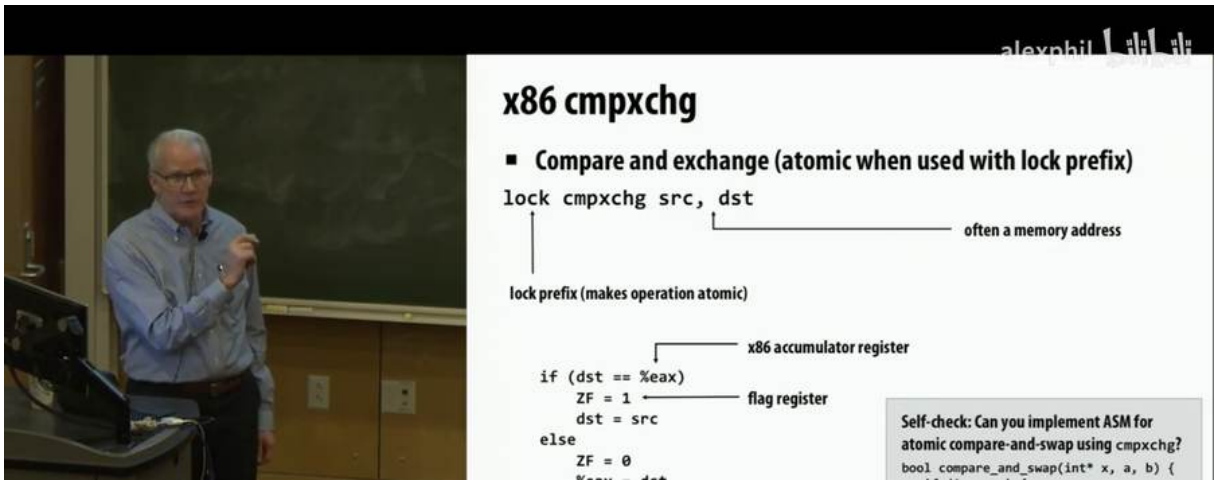

图 458: x86 的 cmpxchg：配合 lock 前缀即为原子比较交换[458]

讲者给出真实指令的例子：x86 的比较并交换。它本身是一个比较交换操作，与 lock 前缀配合使用时才是原子的。

本章小结

test-and-set 原子地完成读、测试、写三步，锁应独占一个缓存行以避免连带失效。但每次 test-and-set 都要申请独占，解锁时所有等待者一起争抢，产生大量流量。x86 用 cmpxchg 加 lock 前缀提供同类能力。

那么一把” 好锁” 应该满足什么?

[457] 视频画面时间区间：00:31:00–00:31:15。若为裁剪图，时间区间仍对应原视频画面。
[458] 视频画面时间区间：00:33:00–00:33:15。若为裁剪图，时间区间仍对应原视频画面。

20.5 理想锁的四个特征

图 459: 理想的锁性能特征：低延迟、低流量、低存储、公平[459]

讲者给出评价锁的四个维度，并逐项对照 test-and-set。

维度	含义	test-and-set 的表现
低延迟	锁空闲且无争用时，加锁应几乎无额外代价	好（等价于一条存储）
低流量	等待与交接不应产生大量互连流量	差（交接时全部争抢）
低存储	每把锁占用的空间应很小	好（一个数据字）
公平	不应让某个线程长期饥饿	差（无顺序保证）

表 11: 理想锁的四个特征与 test-and-set 的对照

讲者的总结是：test-and-set 赢了一些，但不多。

四个维度互相冲突，没有全面最优的锁

低延迟要求简单，公平要求排队，低流量要求错开，低存储要求紧凑。任何改进都是在这些维度间重新分配代价。因此评价一个锁实现必须指明它针对哪种竞争程度与哪种负载模式，而不是笼统地说”这个锁更快”。

本章小结

理想锁需低延迟、低流量、低存储、公平。test-and-set 在延迟与存储上表现好，在流量与公平上差。这四项互相冲突，因此改进方案都是在其中重新分配代价。

下面三个方案分别针对流量与公平。

[459] 视频画面时间区间：00:35:00–00:35:15。若为裁剪图，时间区间仍对应原视频画面。

20.6 三个改进方案

test-and-test-and-set

图 460: test-and-test-and-set：先只读自旋，看到空闲才尝试获取[460]

这个改进只有看过缓存协议才能体会其价值。与其反复猛敲 test-and-set，不如先在只读模式下忙等到锁变为零。

只读自旋不会作废他人的副本，因此几乎不产生流量。

一旦看到锁空闲，再用 test-and-set 去抓。若成功就完成了。

$$\text{流量}_{\text{TTAS}} \approx \frac{\text{流量}_{\text{TAS}}}{N_{\text{waiters}}}$$

- N_{waiters}：同时等待该锁的处理器数；
- TAS 每次尝试都要申请独占，成本随等待者数线性增长；
- TTAS 的自旋阶段只读，因此总流量降到接近单次交接的量级。

只读自旋利用了缓存共享状态

在 MSI/MESI 中，多个处理器可以同时持有同一行的共享副本。只读自旋让所有等待者停留在共享状态，不触发失效；只有真正尝试获取时才升级为独占。这把”等待”的流量从每次尝试降为一次。

[460] 视频画面时间区间：00:37:00–00:37:15。若为裁剪图，时间区间仍对应原视频画面。

退避

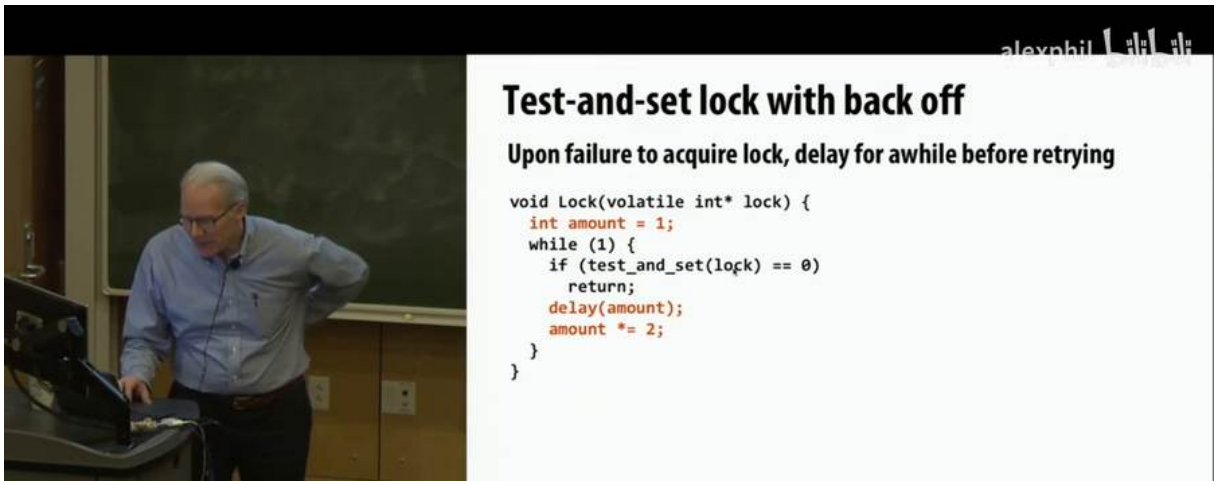

图 461: 退避策略：获取失败后延迟一段时间再重试[461]

第二个改进是退避：获取失败后，先等一会儿再重试。

讲者讨论了一个朴素版本的缺陷：如果我不断把延迟加倍，可能得到一个很大的数；而当第一个进程释放锁时，我还在等我的延迟到期。因此这个版本并不好，但基本思想是对的。

退避的另一个好处是改善交接时的可扩展性。特别是若延迟经过精心设置，有些线程会退后、有些会插入，它们不会一起扑上去。

退避参数必须与实际临界区长度匹配

退避时间过短则等待者仍会同时重试，起不到错开效果；过长则可能在锁已释放后仍在睡眠，浪费机会窗口。理想值取决于临界区平均长度与等待者数量，而这些在运行时变化，因此实际系统常采用自适应或指数退避并设上限。

票号锁

图 462: 票号锁：取号后按序获得服务，保证公平[462]

[461] 视频画面时间区间：00:42:00–00:42:15。若为裁剪图，时间区间仍对应原视频画面。

讲者介绍第三个方案，并说明它真正涉及操作系统层面的思考。票号锁也叫面包店算法，得名于在有些地方你确实会拿到一张实体票（现在多是屏幕上的版本）。

想法是：进入时先取一个号，服务方保证按号码顺序处理这些请求，因此你终会成为队首。

好处是公平——一旦拿到号，就保证在某个时刻会得到服务。

票号锁把竞争转化为排队

test-and-set 的问题在于所有等待者在释放瞬间同时争抢，谁抢到取决于时序而非等待时长。票号锁用两个计数器（取号与叫号）把这种竞争变成有序队列：等待者只需检测叫号是否轮到自己，因此既保证公平，也把交接流量降到每人一次读。

本章小结

三个改进各针对一个问题：test-and-test-and-set 用只读自旋把等待流量降到接近单次交接；退避用随机错开重试时刻缓解争抢，但参数需与临界区长度匹配；票号锁用取号排队保证公平并把竞争转化为有序检测。

这些方案都针对锁。屏障是另一类同步，它有自己的难点。

20.7 屏障与后续

集中式屏障

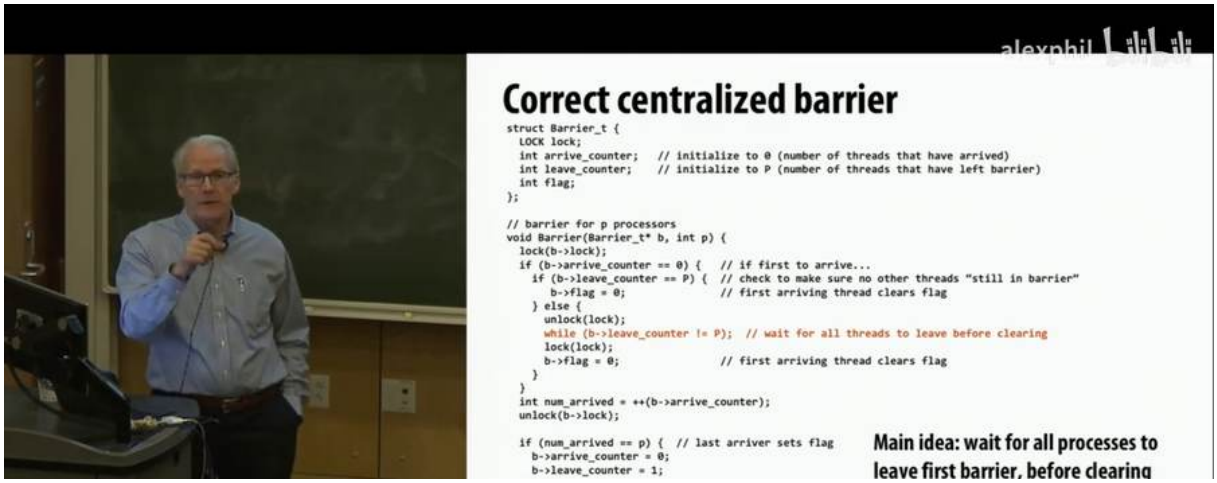

图 463: 集中式屏障：用一个锁与到达计数器实现[463]

讲者展示了一个正确的集中式屏障实现：结构里有一把锁、一个到达计数器，以及释放时用到的标志。

屏障的要求是：所有线程都到达后才能继续。因此实现需要计数——每个线程到达时递增，最后一个到达的线程负责释放其余所有等待者。

屏障的等待时间随线程数增长：

$$T_{\text{barrier}} \approx T_{\text{last}} + T_{\text{wake}} \cdot N_{\text{threads}}$$

[462] 视频画面时间区间：00:43:00–00:43:15。若为裁剪图，时间区间仍对应原视频画面。

[463] 视频画面时间区间：00:58:00–00:58:15。若为裁剪图，时间区间仍对应原视频画面。

- T_{last}：最后一个线程到达的时刻，取决于负载最重的那个；
- T_{wake}：唤醒一个等待者所需的通信代价；
- N_{threads}：线程数，唤醒是串行或广播的，因此该项随线程数增长。

屏障是”一对多”的同步，锁是”一对一”的交接

锁的语义是互斥：同一时刻只有一个线程在临界区内。屏障的语义是集合：所有线程必须都到达才算完成。后者需要区分两种角色（最后到达者与等待者），因此实现上必然比锁多一层状态管理。

讲者指出这比锁更复杂：锁只需让一个线程通过，屏障要让一批线程同时通过，因此需要额外的状态来区分”我是最后一个”与”我在等别人”。

屏障的代价随线程数增长

每次屏障都需要一次全局计数更新与一次全体唤醒。前者是串行点，后者产生广播流量。因此屏障的延迟大致随线程数线性增长，而它在迭代式并行程序（如每轮都要同步的模拟）中被频繁调用，往往成为可扩展性的主要瓶颈。

预览：事务内存

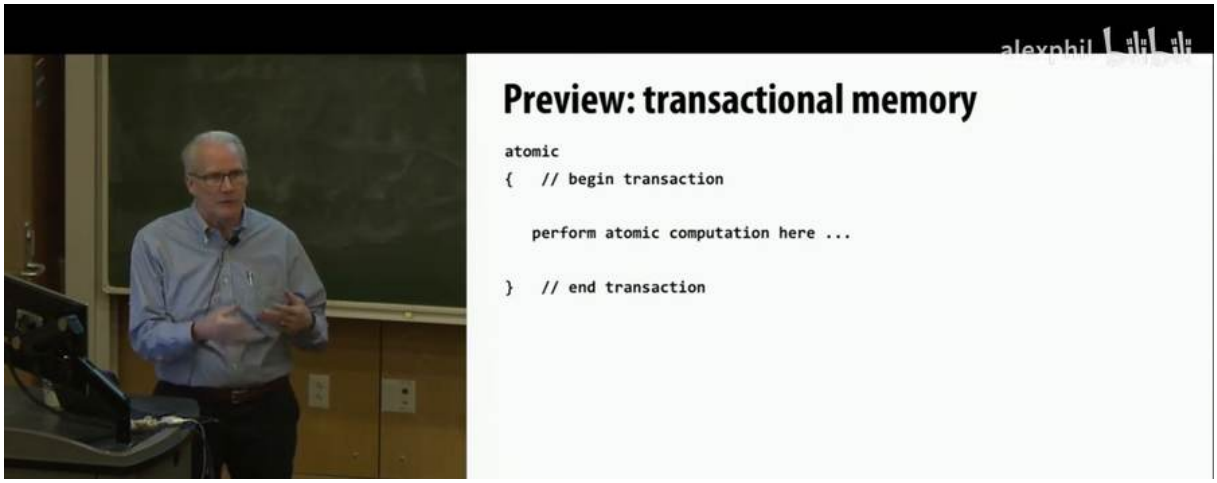

图 464: 预览：事务内存，把一段计算声明为原子[464]

讲者在结尾预告后续内容：事务内存。它的想法是把一段计算声明为原子的，由硬件负责检测冲突并在必要时回滚。

他提到有意思的是，近年处理器确实加入了支持这类操作的特殊指令与能力。

本章小结

集中式屏障用锁加到达计数器实现，比锁更复杂，因为要让一批线程同时通过。它的代价随线程数增长，在迭代式程序中常成为瓶颈。事务内存把一段计算声明为原子，由硬件处理冲突，是后续讲座的主题。

[464] 视频画面时间区间：01:05:00–01:05:15。若为裁剪图，时间区间仍对应原视频画面。

20.8 总结与延伸

把全讲串起来，这是一条从” 同步事件由什么构成” 到” 各种实现如何取舍” 的路径。硬件线程数远少于活跃线程数，操作系统负责映射；同步的开销量级取决于是否牵涉操作系统，纯硬件几十到几百周期，阻塞式数千周期。同步分获取、等待、释放三阶段。忙等待开销极低，在高并行计算中并不坏，因为等待时本无别事可做。朴素自旋锁用普通读写实现，测试与置位之间存在窗口，多个线程可能同时通过；实际出错概率因缓存协议开销而降低，但不构成保证。test-and-set 提供原子的读-测-写，但它每次尝试都要申请独占，解锁瞬间所有等待者一起争抢，产生大量流量，且不公平。理想锁需低延迟、低流量、低存储、公平，而 test-and-set 只在前两项之一与第三项上表现好。三个改进分别针对流量与公平：test-and-test-and-set 用只读自旋把等待流量降到接近单次交接；退避用随机错开重试时刻；票号锁用取号排队保证公平。屏障用锁加计数实现，代价随线程数增长。事务内存是后续方向。

本讲义在讲者材料之上做的压缩，有四点需要与讲者原话区分：

- 忙等待与阻塞的取舍取决于” 等待期间是否有别的事可做”，而不是取决于哪个更先进。并行计算程序通常用满所有核，因此自旋不浪费他人的执行机会；通用操作系统则总有别的线程可跑。脱离这个前提讨论优劣没有意义。
- 锁的四项特征互相冲突，因此不存在全面最优的实现。低延迟要求简单，公平要求排队，低流量要求错开。评价任何锁都必须指明竞争程度与负载模式。
- 原子性不能靠概率保证。朴素自旋锁在实际中很少出错，是因为缓存一致性协议的开销拉长了窗口两侧的时间，而不是因为它在逻辑上安全。处理器数增加或协议改变都可能让问题暴露。
- 只读自旋的有效性依赖缓存一致性协议的共享状态。若硬件不支持多副本共享读，这一优化就无从谈起。这说明锁的实现与底层一致性模型紧密耦合，不能跨平台照搬。

这堂课的边界在于它讲的是硬件层面同步原语的实现思路，未深入具体语言的内存序语义与事务内存的完整机制。讲者也说明真实实现远比示意版本复杂。

两个可以直接从本讲数据提出的下一步问题：其一，票号锁保证了公平，但它把交接变成串行的叫号传递，在处理器很多时延迟如何增长？其二，屏障的代价随线程数线性增长，那么在设计一个每轮都要同步的迭代程序时，应当增大每轮的工作量以减少同步次数，还是改用无屏障的算法结构？

Lecture 21 Lock-free Programming

视频时长：1 小时 21 分 51 秒　来源：https://www.bilibili.com/video/BV17N411B71Z/

21.1 为什么不用一把大锁

讲者开场就给了个提醒：这是课程中材料相当棘手的部分。你看着这些代码会隐约觉得懂了，但最好回去再核对一遍。

动机：互斥量太重

问题的起点很实际：pthread 互斥量是重量级代码，它需要与操作系统交互，可能让作业被挂起、放入队列、之后重新唤醒。

这个过程可能耗时 20000 个时钟周期以上。因此你不希望在并发程序的应用级数据结构上做这件事。

锁的开销量级决定了它的适用范围

第 20 讲区分过：纯硬件原子操作在几十到几百周期，涉及操作系统调度的在数千周期以上。而 pthread 互斥量作为最重的同步原语，可以到数万周期。用它保护一个每次只做几条指令的临界区，开销会比临界区本身大几个数量级。

自旋锁的适用范围

有时自旋锁是合适的：如果你只想抓住某个东西足够久，完成几次更新而不让别人来搅局，自旋锁是很有效的方式。

但无论用哪种锁，若把锁加在整个数据结构上——比如一个哈希表——你就把对该表的所有访问串行化了，这会使它不适合并行编程。

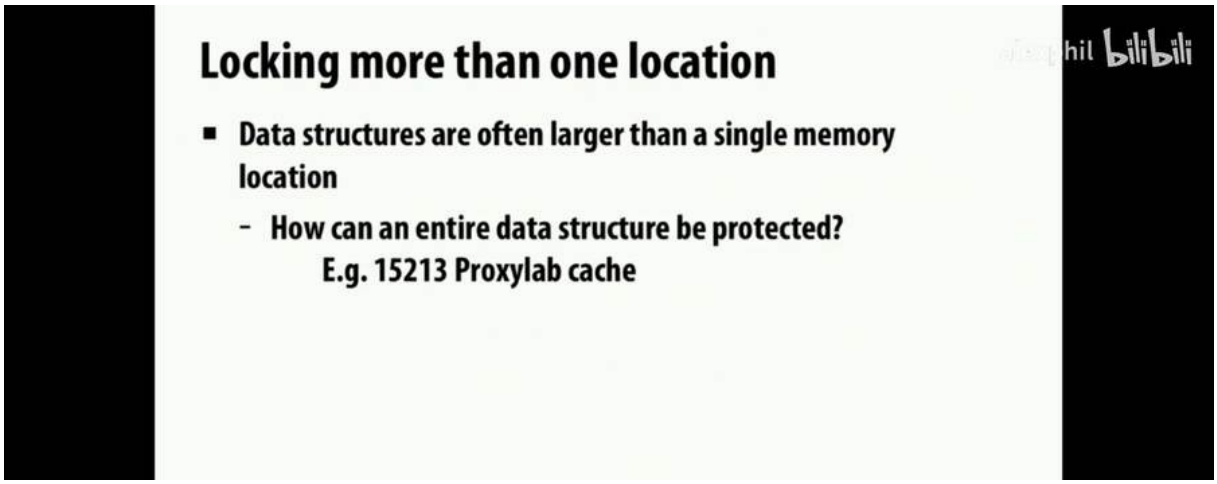

图 465：锁定多个位置：数据结构往往大于一个内存位置[465]

[465] 视频画面时间区间：00:17:00–00:17:15。若为裁剪图，时间区间仍对应原视频画面。

三个层次

三层方案对应三种不同的失败模式

一把大锁的失败模式是串行化——正确但无并行。细粒度锁的失败模式是死锁与实现错误——并行但容易出错。无锁的失败模式是 ABA 与回收难题——无阻塞但极难正确实现。选择层次时，要先明确自己更怕哪种失败。

讲者把改进方向分成三层。最粗的是用一把大锁；再细一步是给数据结构的各个组成部分分别加锁；最极端的是完全去掉锁，另找机制保证共享与更新的正确性。

最后这一层就是无锁编程，这个领域有相当多的研究成果。

本章小结

pthread 互斥量开销可达数万周期，不适合应用级数据结构。自旋锁适合短暂持有，但把锁加在整个结构上会串行化所有访问。改进方向有三层：一把大锁、细粒度锁、完全无锁。

下一章从最基础的原语开始。

21.2 比较并交换：无锁的基石

原子操作家族

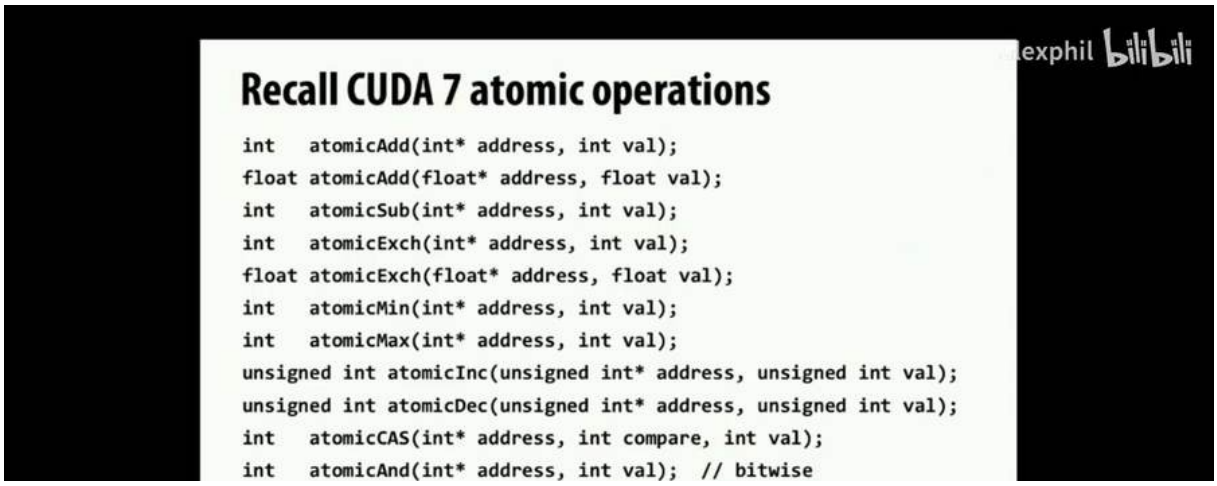

图 466: 回顾 CUDA 的原子操作：原子加、原子自增等[466]

细粒度方案依赖原子操作。讲者展示了 CUDA 提供的一组原子操作（GCC 也有类似的列表），包括各种原子加、原子自增或执行某种简单算术运算的方式。

[466] 视频画面时间区间：00:05:00–00:05:15。若为裁剪图，时间区间仍对应原视频画面。

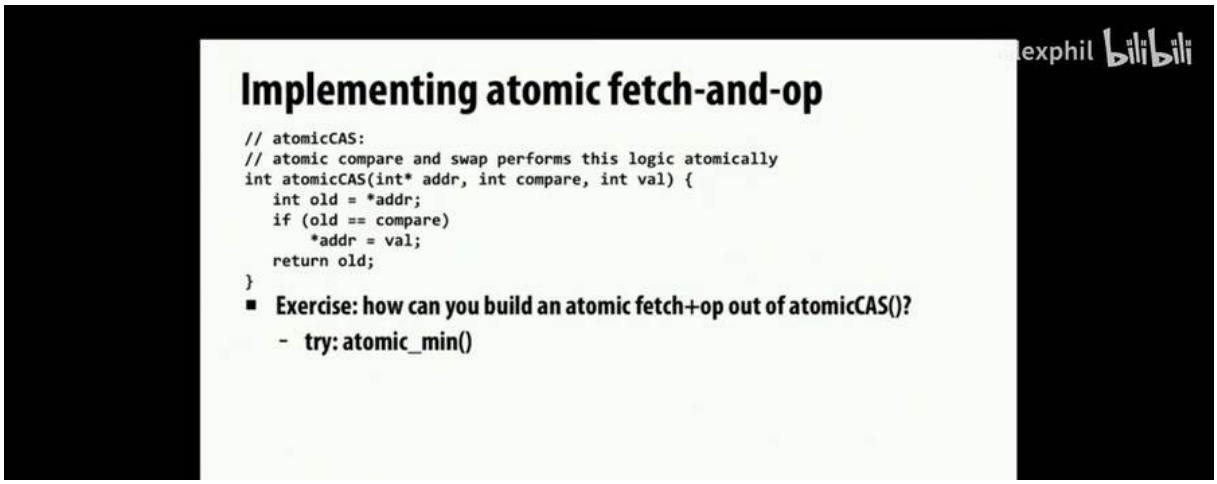

图 467: 实现原子取并运算：原子 CAS 的接口[467]

另一类叫比较并交换，讲者说这些实际上是硬件实现的核心基础。

三参数语义

比较并交换是一个三参数操作：第一个是地址，第二个是你认为该地址上当前是什么值，第三个是你想存进去的值。

整个操作原子地完成，也就是说这些步骤被归在一起执行。具体是：先检查该地址当前的值为多少，称为 old；若它与期望值匹配，就执行更新，并把 old 作为结果返回。

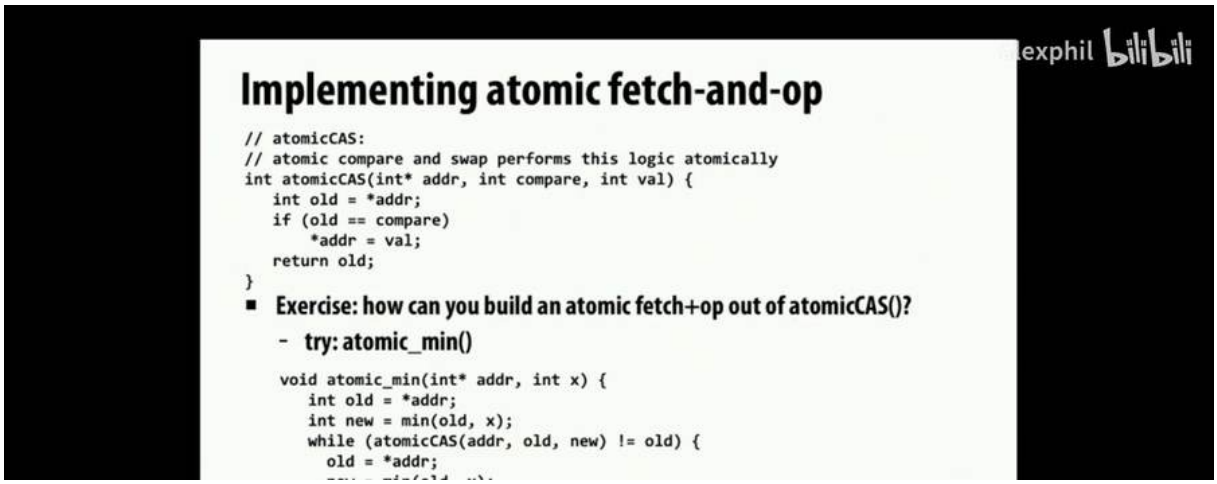

图 468: 用 CAS 循环实现原子更新：反复尝试直到成功[468]

讲者补充说，设计上你会确保要写入的新值与期望值不同，否则这个操作没什么意义。而当 old 返回后若与你传入的比较值不匹配，你就知道这次操作失败了。

用公式表达即：

$$\mathrm{CAS}(addr, expected, new) = \begin{cases} \text{成功，写入 } new, & \text{若 } \mathrm{mem}[addr] = expected \\ \text{失败，不写入}, & \text{否则} \end{cases}$$

[467]视频画面时间区间：00:05:30–00:05:45。若为裁剪图，时间区间仍对应原视频画面。
[468]视频画面时间区间：00:09:40–00:09:55。若为裁剪图，时间区间仍对应原视频画面。

- 返回值总是该地址原来的内容，因此可用它判断成败；
- 成功条件依赖” 当前值恰好等于期望值”，这正是 ABA 问题的根源；
- 操作整体不可分割，期间没有其他线程能观察到中间状态。

循环重试与无限循环

CAS 通常在循环里使用：不断尝试直到成功。
讲者提醒说，最好把这段代码改一下使它不会无限循环——这通常是个好习惯。
他还说，实现原子自增会更合理一些，因为那样就没有无限循环的可能。

CAS 重试循环可能不终止

在极高争用下，一个线程可能反复失败。若不加超时或退避，理论上可能长期无法推进。虽然实际中概率很低，但在需要确定性响应的系统（实时系统、持有锁期间被换出）里，这种无界重试是不可接受的。

用 CAS 实现锁

讲者指出，用这个原语实现锁也很直接：假设某地址上有一个标志，你不断尝试把一或零换进去，并检查旧值是否等于一。

锁与原子操作是同一种原语的两种用法

CAS 既能实现” 读-改-写” 这类无锁更新，也能实现互斥锁。区别在于用途：前者让多个线程都尝试推进数据结构，后者只允许一个线程进入临界区。因此无锁编程并不是抛弃原子原语，而是改变使用它的方式。

本章小结

比较并交换是三参数原子操作：地址、期望值、新值，返回旧值以判断成败。它通常在循环中重试，需注意无限循环风险。用它可以实现原子自增与自旋锁。它是硬件实现所有同步的核心基础。

有了原语，先看语言层面的封装。

21.3 语言封装：C11 的原子类型

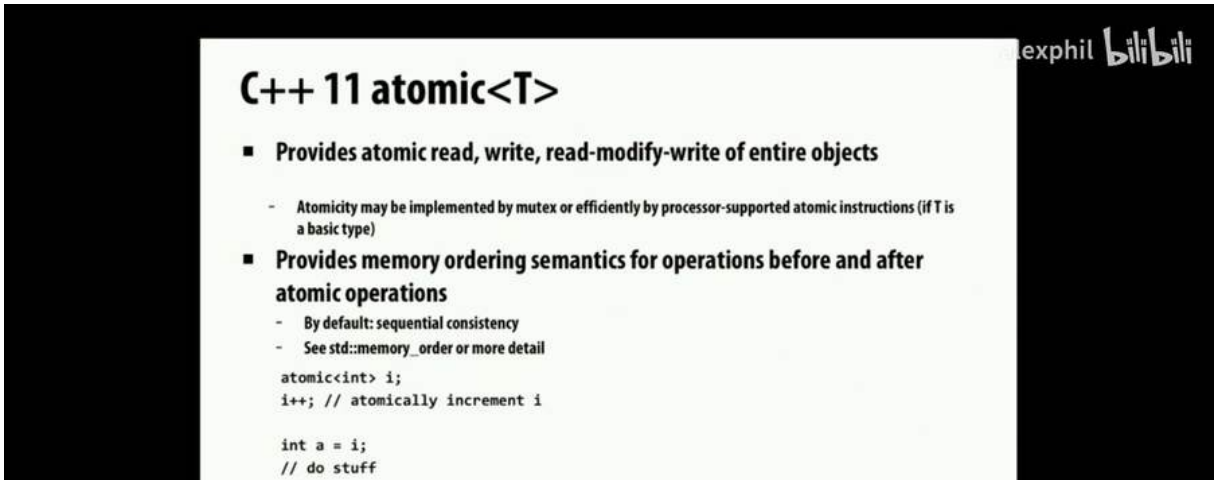

图 469: C++11 的 atomic 模板：可把任意类型做成原子[469]

讲者指出 C++ 的 C11 版本有一个有意思的能力：通过模板类型把任何东西做成原子的。

对 int 或任何基本数据类型使用时，实现会利用为支持原子操作而设计的原生指令。比如对原子变量做自增，通常会用到取并加之类的操作。

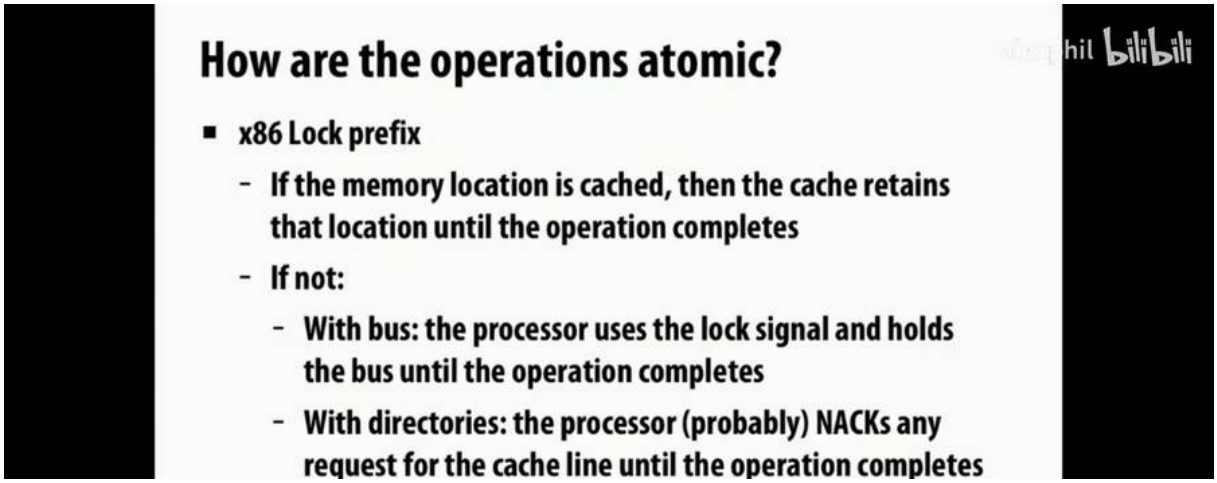

图 470: 这些操作如何做到原子：x86 的 lock 前缀[470]

对于更复杂的类型，你可以对几乎任何数据类型这么做；但若是庞大复杂的数据结构，它就干脆在里面放一把大锁。讲者说它会尽量聪明地选择内部实现，尤其是对标准数据类型。

抽象层的实现可能是”聪明”也可能是”退化为锁”

同一个 atomic 接口，对基本类型映射到单条原子指令，对复杂结构则可能退化为内部加锁。因此不能假设”用了原子类型就一定无锁”。要判断实际行为，需要看类型大小与平台实现，必要时用工具验证。

[469] 视频画面时间区间：00:11:00–00:11:15。若为裁剪图，时间区间仍对应原视频画面。

[470] 视频画面时间区间：00:12:30–00:12:45。若为裁剪图，时间区间仍对应原视频画面。

本章小结

C11 的 atomic 模板把原子性抽象成类型，对基本类型映射到原生指令，对复杂结构则可能退化为加锁。这个抽象很方便，但不能假设它总是无锁实现。

下面进入本讲的主体：并发数据结构。

21.4 并发链表的三种方案

串行版本

图 471: 示例：有序链表的节点结构与列表结构[471]

讲者用标准有序链表代码作为例子。插入时沿链表走，直到遇到大于待插入值的位置；此时已创建好新节点 n，把它插入：cur 成为 n 的后继，而 cur 原来的前驱 prev 改为指向 n。

删除同理：沿链表走到匹配值处，摘除并释放该元素。

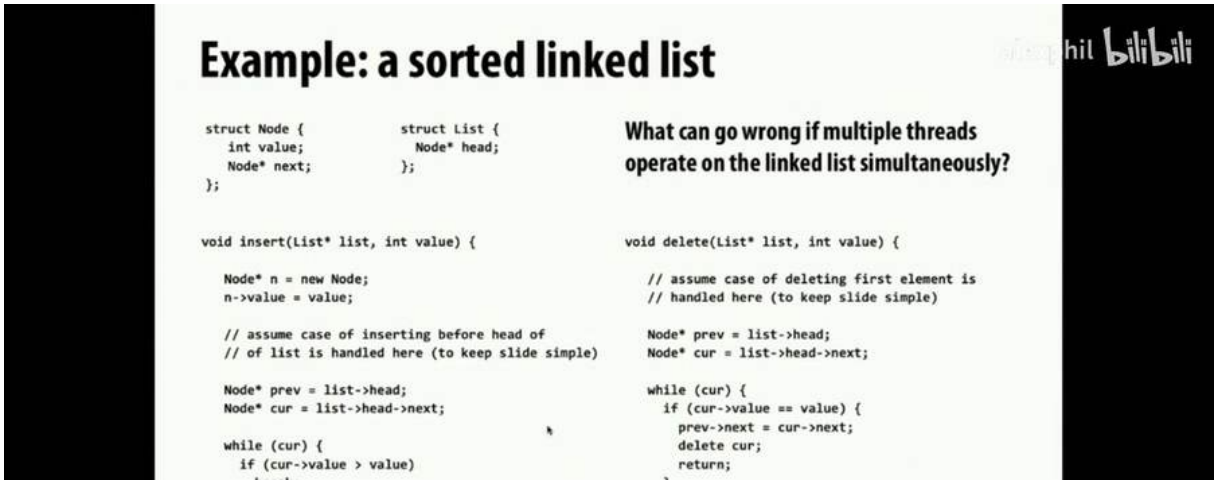

图 472: 有序链表的插入：把新节点接到前驱与后继之间[472]

[471] 视频画面时间区间：00:18:00–00:18:15。若为裁剪图，时间区间仍对应原视频画面。

[472] 视频画面时间区间：00:19:30–00:19:45。若为裁剪图，时间区间仍对应原视频画面。

并发时出了什么问题

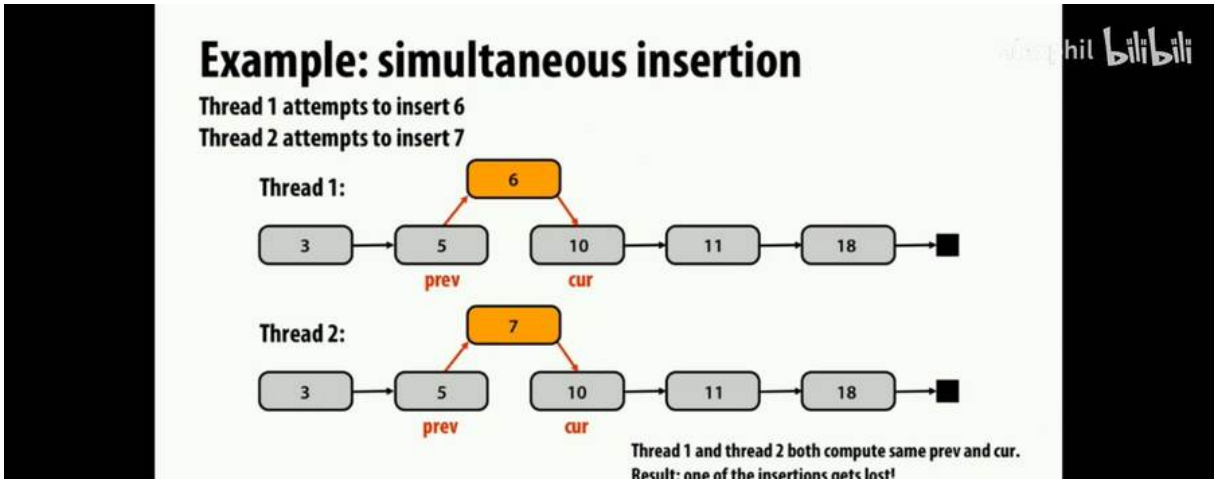

图 473: 同时插入：线程 1 插入 6，线程 2 插入 7[473]

讲者给出具体场景：一个线程想插入元素 6，另一个想插入 7。

线程 1 先到达并开始拼接；与此同时线程 2 到达同一位置也开始拼接。于是两者竞争谁真正把元素接进链表。

例子里显示一个插入成功、另一个没有。讲者说其实基本只有这两种结果，因为这段代码中唯一被更新的共享部分是那个前驱指针。

删除也有类似问题：一个线程已经准备好删除，同时有插入发生；或者两个删除作用于相邻元素。各种问题都会出现。

方案一：一把锁

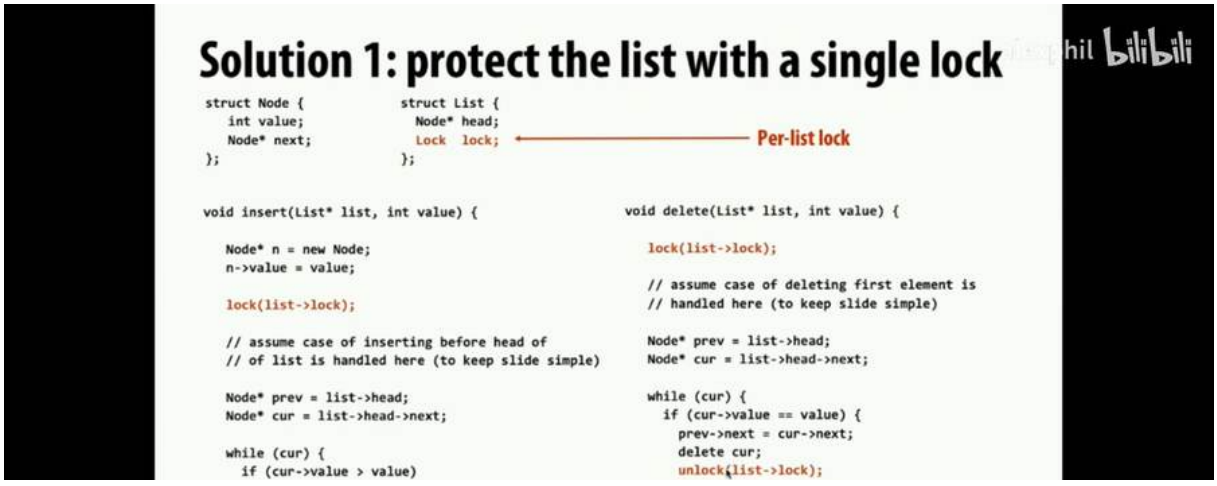

图 474: 方案一：用一把锁保护整个链表[474]

最快捷的做法是给整个结构加一把锁：只要有一方在遍历这个链表，它就持有整条链表的锁，直到完成操作才释放。

[473] 视频画面时间区间：00:20:30–00:20:45。若为裁剪图，时间区间仍对应原视频画面。

[474] 视频画面时间区间：00:22:00–00:22:15。若为裁剪图，时间区间仍对应原视频画面。

讲者提醒一个细节：代码里若有多个返回点，必须确保每条路径都解锁，不能有些解有些不解。

但坏处正如前面所说：这让链表变成完全串行的数据结构。

图 475: 挑战：谁能做得更好[475]

本章小结

有序链表的并发问题集中在拼接时的前驱指针更新，两个线程会竞争同一个位置。最简方案是用一把锁保护整个链表，正确但完全串行，且需注意多返回点的解锁。

这引出”谁能做得更好”的挑战。

21.5 方案二：手递手细粒度锁

核心不变式

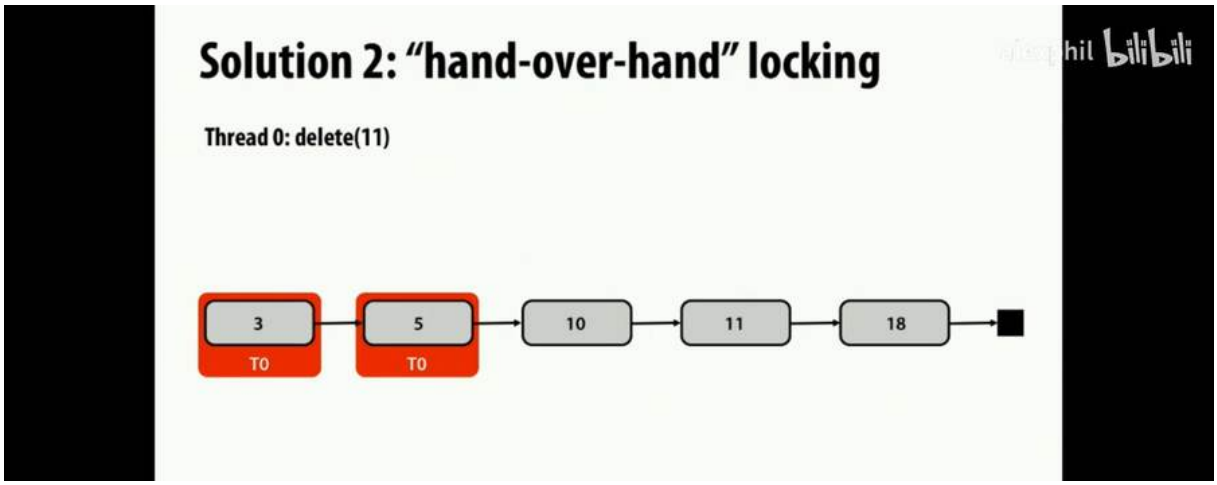

图 476: 方案二：手递手加锁遍历，同时持有前驱与当前两把锁[476]

讲者展示手递手遍历的过程：遍历循环有一个循环不变式——循环开始时保证持有前驱与当前这两把锁。

[475] 视频画面时间区间：00:22:30–00:22:45。若为裁剪图，时间区间仍对应原视频画面。

[476] 视频画面时间区间：00:23:00–00:23:15。若为裁剪图，时间区间仍对应原视频画面。

随着循环推进，它释放一把再获取下一把。因此会有一个瞬间只持有一把锁，但 ** 永远不会有不持锁的瞬间 **。

图 477: 方案二：细粒度加锁，每个节点一把锁[477]

手递手的正确性来自”始终持有至少一把锁”

若允许出现不持锁的瞬间，另一个线程就可能在你松手的位置插入或删除节点，使你的前驱指针失效。始终保持至少一把锁，保证了你脚下的路径不会被抽走。这个不变式比”每步都加锁”更强，也更精确。

顶层锁的作用

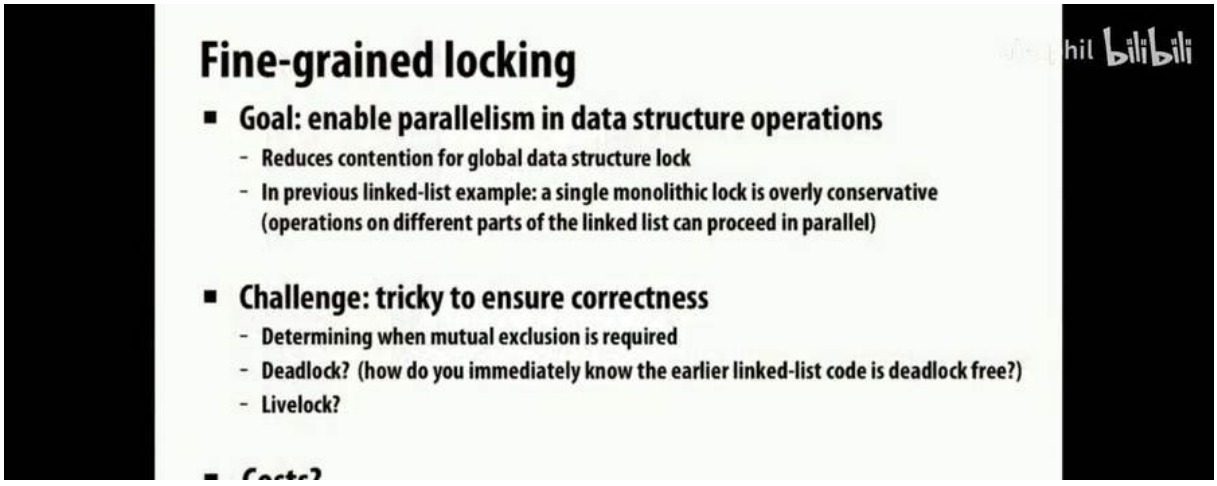

图 478: 细粒度加锁的目标：让数据结构操作的并行化成为可能[478]

讲者解释为什么还需要一把覆盖整个结构的锁：它是用来控制初始访问的，让所有人一开始被串行化，进入顺序决定了它们沿链表推进的次序。

[477] 视频画面时间区间：00:27:00–00:27:15。若为裁剪图，时间区间仍对应原视频画面。
[478] 视频画面时间区间：00:31:30–00:31:45。若为裁剪图，时间区间仍对应原视频画面。

但这把锁只持有到你取得第一个元素的锁为止，然后释放——这就是手递手。别人可以跟在你后面进来，但不能越过你。

图 479: 练习：实现一个细粒度锁定的二叉搜索树[479]

实用变体：哈希桶

讲者提到一个很实际的做法：用哈希表，每个桶是一条链表。这是实现哈希表的标准方式。

第一层哈希把争用分散开，使这些链表通常最多 10 到 20 个元素长。这样就解除了串行化，而且完全合理。

代价

细粒度锁的出错点随锁数量增长

讲者明确说，与一把大锁相比，细粒度方案有很多地方可以引入 bug，而且都是微妙的问题。锁越多，需要维护的不变式越多：谁持有哪把、何时释放、哪些路径可能提前返回。正确性论证的复杂度显著上升。

讲者还说明细粒度方案若用标准锁实现，开销高到不实用；而自旋锁通常只是一个整数，因此存储开销不算大问题。

并发度与锁的数量成正比，但受链表中元素数限制：

$$P_{\text{concurrent}} \approx \min\left(N_{\text{locks}}, N_{\text{threads}}\right)$$

- N_{locks}：数据结构上可独立加锁的位置数；
- N_{threads}：活跃线程数；
- 若链表很长而锁只在每个节点上，则可并发推进的线程数受链表长度限制；
- 哈希桶方案通过增加独立桶数来提高这个上界。

[479] 视频画面时间区间：00:33:00–00:33:15。若为裁剪图，时间区间仍对应原视频画面。

手递手在需要修改多个指针的结构上难度陡增

链表插入只改两个指针，删除只改一个。而二叉搜索树的删除可能涉及节点替换、子树搬移，需要同时持有多个节点的锁并按固定顺序获取以避免死锁。锁数量增加使”始终持有至少一把”这个不变式难以维护。

他也提到去年曾实现过一个完整的二叉搜索树手递手遍历版本，说它极其优雅但并不简单——尤其是删除本身已经棘手，再叠加手递手遍历。

本章小结

手递手遍历的不变式是始终持有前一与当前两把锁，因此路径不会被抽走。顶层锁只用于让进入者有序，取得第一个元素的锁后即释放。实用变体是用哈希桶把争用分散到短链表上。代价是出错点随锁数量增长，实现复杂度显著上升。

这些都是基于锁的方案。下一章看完全不用锁的思路。

21.6 无锁算法：定义与两个实例

基于锁方案的根本弱点

讲者指出基于锁的系统有一个可能被长期延迟的风险：某个进程被页换出、甚至崩溃。

若它崩溃，可能造成大麻烦；若被换出，整个访问会挂起等待该进程回来。这些都是操作系统层面的问题。

于是无锁算法的定义是：不会发生这种情况——至少有一个线程总能推进。

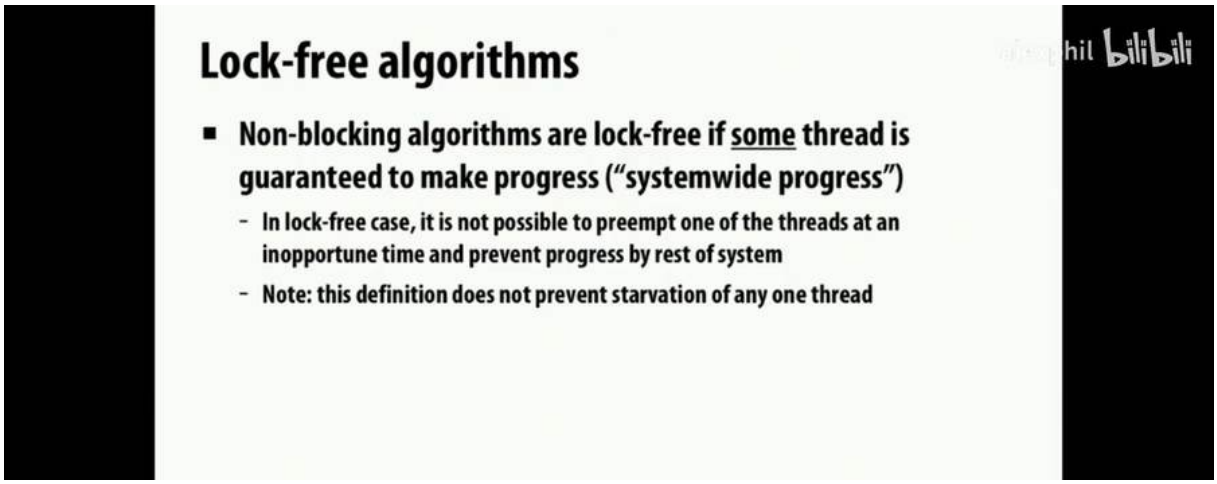

图 480: 无锁算法：若某些线程总能推进则为无锁[480]

讲者自己也承认这个保证有点宽松。因为用 CAS 循环时我们只是不断尝试直到成功，而在严重争用下可能所有人反复失败。

可以把它写成失败概率的形式：

$$P(\text{某线程成功}) \geq 1 - (1-p)^{N_{\text{retries}}}$$

[480] 视频画面时间区间：00:36:00–00:36:15。若为裁剪图，时间区间仍对应原视频画面。

- p：单次 CAS 尝试成功的概率，随争用加剧而下降；
- N_{retries}：某线程的重试次数；
- 只要 $p > 0$，重试次数增加时成功率趋于 1，但单次延迟无上界。

因此无锁给的是概率性进展，而不是时间上界。因此说”绝对保证推进”是过强的说法，实际含义是：以极高概率，某个线程能继续前进，且不受操作系统上下文切换问题的影响。

无锁的保证是”系统层面”而非”个体层面”

无锁只保证至少有一个线程能推进，不保证某个特定线程会推进。这意味着单个线程可能饥饿，而整体系统仍在前进。若应用要求每个线程都有响应时间上界，无锁并不足够，还需要额外的公平性机制。

单读单写有界队列

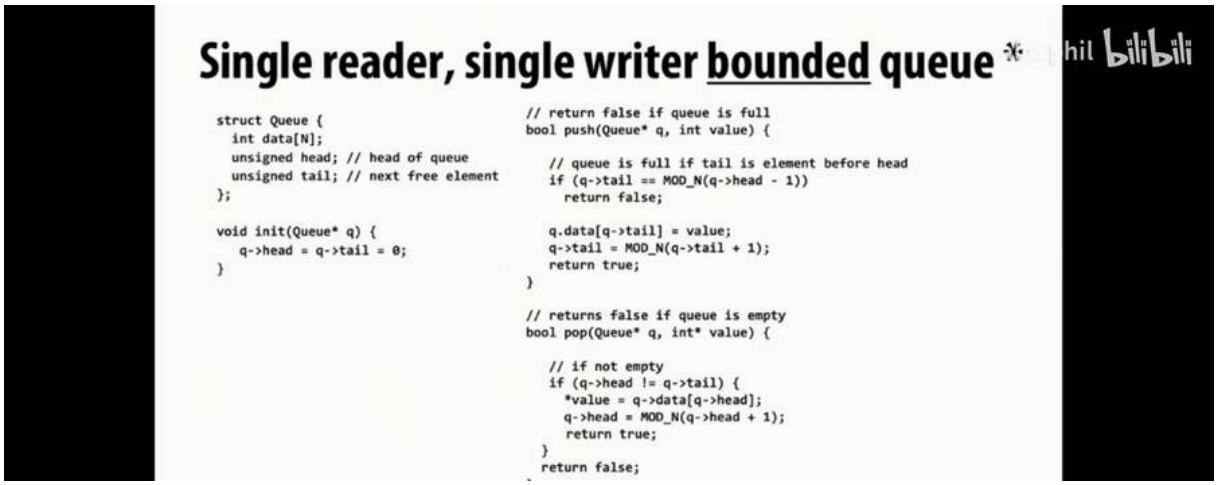

图 481：单读单写的有界队列：环形缓冲区实现[481]

讲者给出一个很简单的例子，并声称这段代码可用——前提是只有一个读者和一个写者，且它们不是同一个线程：一个线程插入，一个线程删除。

结构是一个环形队列：head 表示最旧的值，tail 指向下一个应该插入的位置。插入就是写入该位置再把 tail 前移一格；弹出就是读出该位置的值再把 head 前移一格。两者都对 n 取模以回绕。

[481] 视频画面时间区间：00:37:00–00:37:15。若为裁剪图，时间区间仍对应原视频画面。

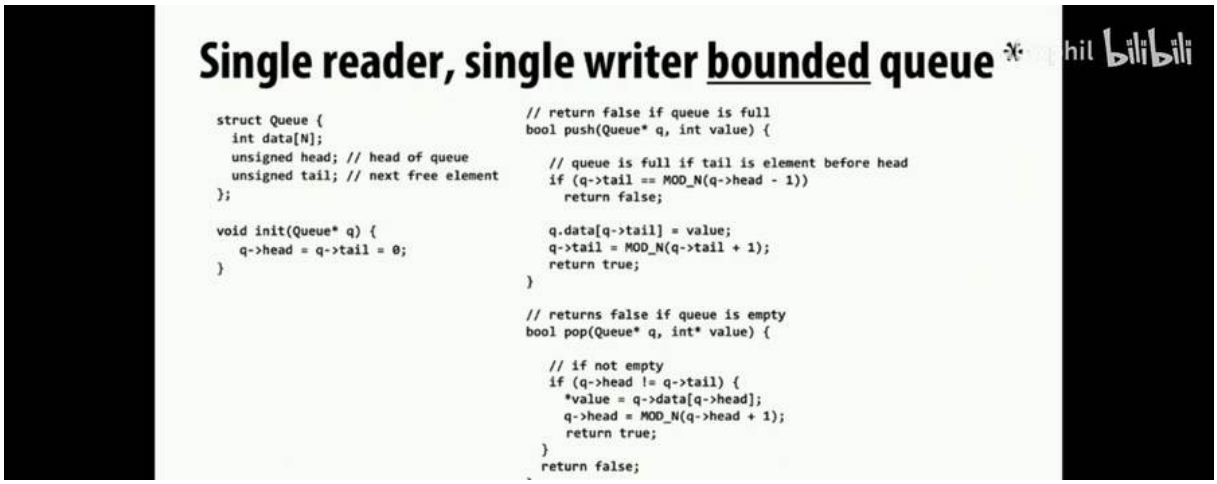

图 482: 队列满与空的判定：牺牲一个空位以区分两者[482]

这里有个经典问题：如何区分满与空？这段代码的做法是让出一个空位——永远不让 tail 越过 head。当 tail 到达 head 的位置时就宣布队列满，因此实际容量只有 n 减 1。

$$\text{capacity} = n - 1$$

- n：环形缓冲区的槽位数；
- 牺牲一格用于区分” 满” 与” 空” 两种状态；
- 否则 head 与 tail 相等时无法判断是哪一种。

讲者说明也可以用别的方案，但那样就需要另一个标志来告知空或满。

他最后指出这个方案的前提：对内存序做合理假设，且更新一个字时不会出现” 改了半个字” 导致读到不一致的值。它甚至不需要取模操作本身是原子的。

单读单写无界队列

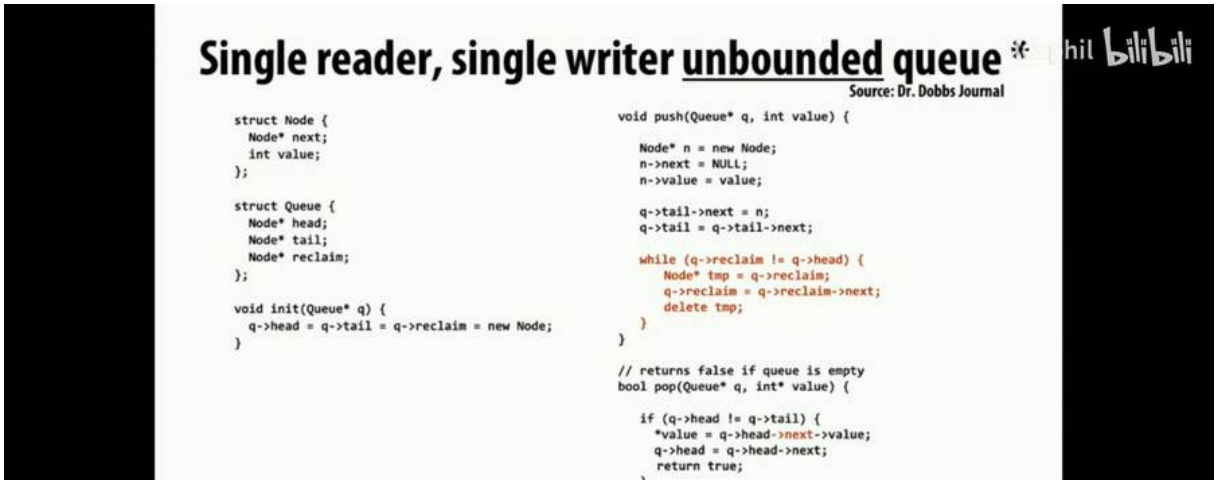

图 483: 基于链表的无锁队列：head、tail 与 reclaim 三个指针[483]

[482] 视频画面时间区间：00:40:00–00:40:15。若为裁剪图，时间区间仍对应原视频画面。

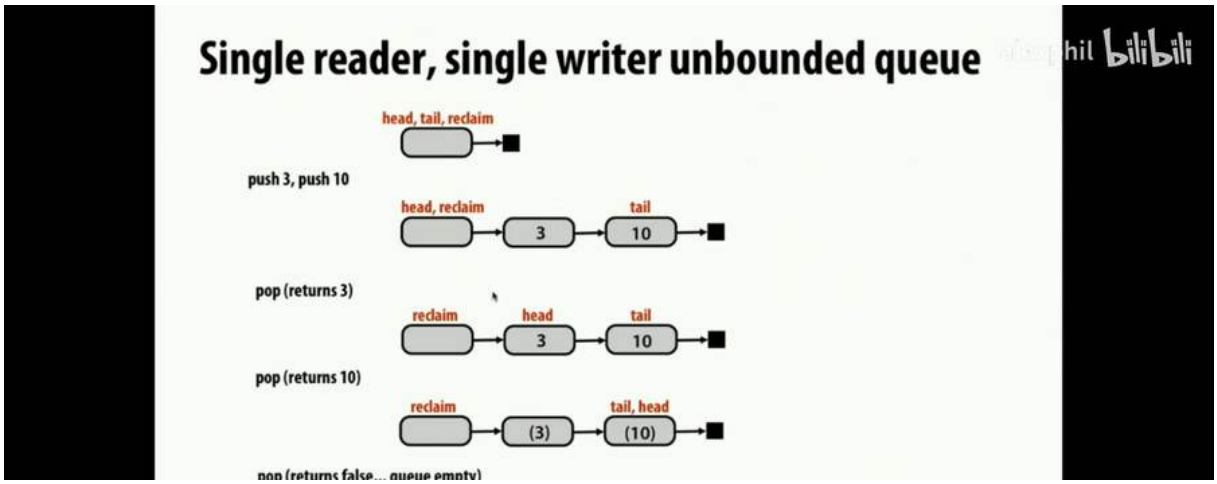

图 484: reclaim 指针：pop 只移动 head，从不真正删除节点[484]

讲者接着给出一个同样基于单读单写、但没有大小限制的方案，因为它用链表实现。

这个想法很有意思：队列有三个指针——标准的 head 与 tail，外加第三个叫 reclaim。

关键在于：pop 虽然从链表中移除元素，但 ** 从不真正删除任何节点 **。只有 pusher 才被允许用 new 创建节点、用 delete 释放节点。

head 是一个哑元素，从不会有值放进去。初始时是空链表，三个指针都指向同一个节点。

push 时不断往 tail 加节点。pop 只是让 head 沿链表前进一格，而不删除前一个元素。

reclaim 指针负责跟踪链表中还有哪些节点——即使它们已经不再是有效数据。

讲者用括号表示那些已被移除、但物理上仍留在链表元素中的值：它们只是残留值。

单写者负责回收是无锁队列的关键分工

若 pop 也能释放节点，那么一个慢的读者可能仍持有指向该节点的指针，而节点已被释放并被复用，于是读到垃圾数据——这正是 ABA 问题的温床。把分配与释放权限集中到唯一的写者，就消除了这类竞态。

本章小结

无锁算法保证至少有一个线程总能推进，且不受操作系统上下文切换影响；这个保证是系统层面而非个体层面。两个实例是单读单写的环形队列（牺牲一格区分满空）与单读单写的链表队列（只有写者可分配释放节点，pop 只移动 head）。

这些方案都限定了单读单写。放宽这个限制会引出更棘手的问题。

[483] 视频画面时间区间：00:44:00–00:44:15。若为裁剪图，时间区间仍对应原视频画面。

[484] 视频画面时间区间：00:46:00–00:46:15。若为裁剪图，时间区间仍对应原视频画面。

21.7 ABA 问题

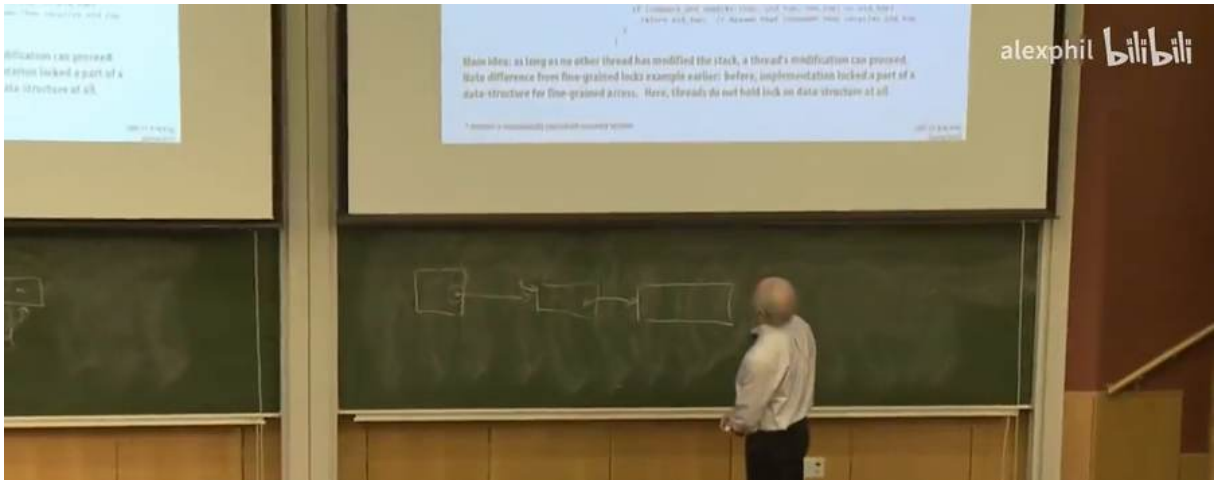

图 485: 从细粒度锁转向无锁实现时遇到的新问题[485]

看起来应该可行的做法

讲者描述了一个自然想法：我取得指向某个节点的指针，把它作为新栈顶，想把它塞进链表前端。但要确保在这个过程中没有别人插入新元素或删除那个元素。

讲者问：这看起来应该可行吧？然后自己回答：你大概猜到了，不行。

问题的形态

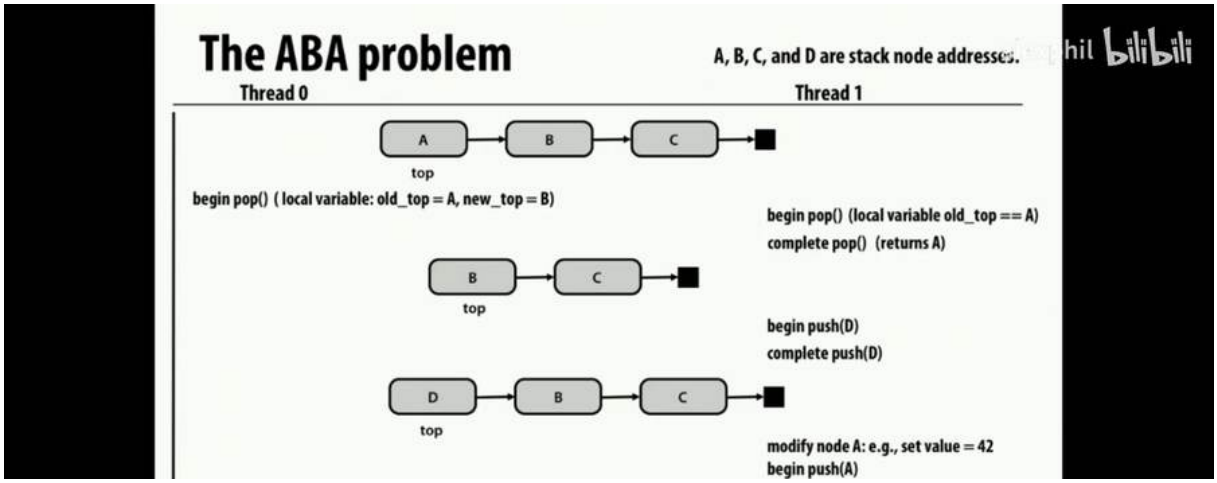

图 486: ABA 的具体场景：pop 后重新插入导致 CAS 误判[486]

这个问题的名字很简短：ABA 问题。做这行的人一听说” 我有个 ABA 问题”，立刻就知道你在说什么。

场景是这样的：两个线程都在操作这个栈。线程 0 开始一次弹出操作，它认为旧栈顶是 A、新栈顶是 B，想在做完比较并交换后确认自己是对的。

[485] 视频画面时间区间：00:54:30–00:54:45。若为裁剪图，时间区间仍对应原视频画面。

[486] 视频画面时间区间：00:57:00–00:57:15。若为裁剪图，时间区间仍对应原视频画面。

与此同时线程 1 完成了一次弹出并成功返回，真的从链表中移除了一个元素。然后线程 1 又插入了一些东西。

如果线程 1 插入的恰好是 A，那么当线程 0 执行 CAS 时，它看到栈顶仍是 A，就以为没有任何变化，于是成功地把栈顶设为 B。但此时 B 可能已经被释放了。

CAS 只能比较值，无法感知中间状态

ABA 的根源在于比较并交换只看” 当前值是否等于期望值”，而不看” 这个值是否一直是这个值”。若中间经历了 A→B→A 的变化，CAS 无法察觉，于是基于旧假设的更新被错误地接受。这是无锁算法与基于锁算法最本质的差别之一。

应对方向

讲者给出两类常见对策。其一是给指针附加一个版本计数，使每次修改都改变计数，因此 A→B→A 会变成 (A,1)→(B,2)→(A,3)，CAS 便能识别出变化。其二是把节点回收推迟到确定无人引用之后，例如让唯一的写者负责释放，或使用引用计数、风险指针等机制。

内存回收是无锁设计的核心难点

在有锁的世界里，” 删除一个节点” 只需在临界区内摘链并释放。在无锁世界里，你无法知道是否还有别的线程正持有指向该节点的指针。因此释放时机成为一个独立的设计问题，ABA 只是它的一种表现形式。

本章小结

ABA 问题指值从 A 变回 A 而中间状态未被察觉，导致 CAS 误判成功。对策包括附加版本计数与推迟节点回收。内存回收时机是无锁设计的核心难点，ABA 只是其表现之一。

21.8 总结与延伸

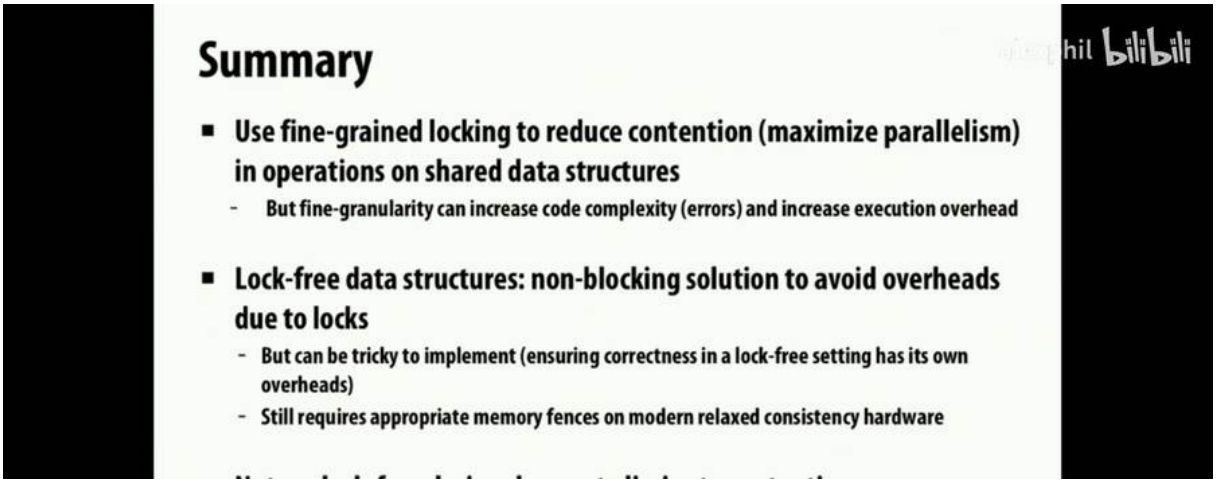

图 487: 总结：用细粒度锁降低争用，用无锁避免阻塞[487]

[487] 视频画面时间区间：01:21:00–01:21:15。若为裁剪图，时间区间仍对应原视频画面。

讲者在结尾给出参考资料，并说如果在维基百科上搜 ABA 问题或无锁队列，会找到相当不错的解释与文献链接。

把全讲串起来，这是一条从”锁太重”到”完全不用锁”的路径。pthread 互斥量开销可达数万周期，不适合应用级数据结构；自旋锁适合短暂持有，但把锁加在整个结构上会串行化所有访问。改进方向有三层：一把大锁、细粒度锁、完全无锁。比较并交换是三参数原子操作，返回旧值以判断成败，它既能实现无锁更新也能实现锁，是所有同步的硬件基础。C11 的 atomic 模板把原子性抽象成类型，但对复杂结构可能退化为加锁。并发链表的最简方案是一把锁，正确但完全串行；手递手细粒度锁的核心不变式是始终持有前一与当前两把锁，因此路径不会被抽走，顶层锁只用于让进入者有序，实用变体是用哈希桶把争用分散到短链表。无锁算法保证至少有一个线程总能推进，不受操作系统上下文切换影响，这个保证是系统层面而非个体层面。两个实例是单读单写的环形队列与链表队列，后者只有写者能分配释放节点。ABA 问题指值从 A 变回 A 而中间态未被察觉，使 CAS 误判成功；对策是版本计数与推迟回收，而内存回收时机是无锁设计的核心难点。

本讲义在讲者材料之上做的压缩，有四点需要与讲者原话区分：

- 无锁不等于无同步。它仍然依赖原子原语与内存序约束，只是不再依赖互斥。因此”去掉锁就更快”是误解：原子操作在高争用下同样昂贵，只是失败方式不同。
- 无锁的保证是系统级的，不保证个体线程的响应时间。若应用要求每个线程都在有限时间内推进，需要更强的无等待（wait-free）保证，而这类算法通常更难实现。
- 细粒度锁与无锁解决的是不同问题。前者降低争用范围，仍可能阻塞；后者消除阻塞，但引入 ABA 与回收难题。选择取决于应用更怕争用还是更怕阻塞。
- 节点回收是无锁设计的独立难题，不是 ABA 的附属。风险指针、引用计数、单写者负责释放都是为此存在。忽视这一点会让看似正确的无锁结构在长时间运行后崩溃。

这堂课的边界在于它是导论：讲者明确说这部分材料很棘手，建议回去反复核对。因此本文给出的是方案框架与问题清单，不含可直接照搬的完整实现。

两个可以直接从本讲数据提出的下一步问题：其一，ABA 问题可以用版本计数缓解，但计数器会回绕，那么回绕周期应当如何与并发规模匹配才安全？其二，单读单写的无锁队列依赖”只有一个写者”这一强限制，若要多生产者，需要哪些额外机制，代价是什么？

Lecture 22 Transactional Memory

视频时长：1 小时 12 分 38 秒　来源：`https://www.bilibili.com/video/BV1nP411W7qr/`

22.1 上一讲的遗留：ABA 与两种解法

上一讲讲了一个不涉及锁的无锁队列实现，并遇到了 ABA 问题。这一讲先把它收尾。

ABA 的形态

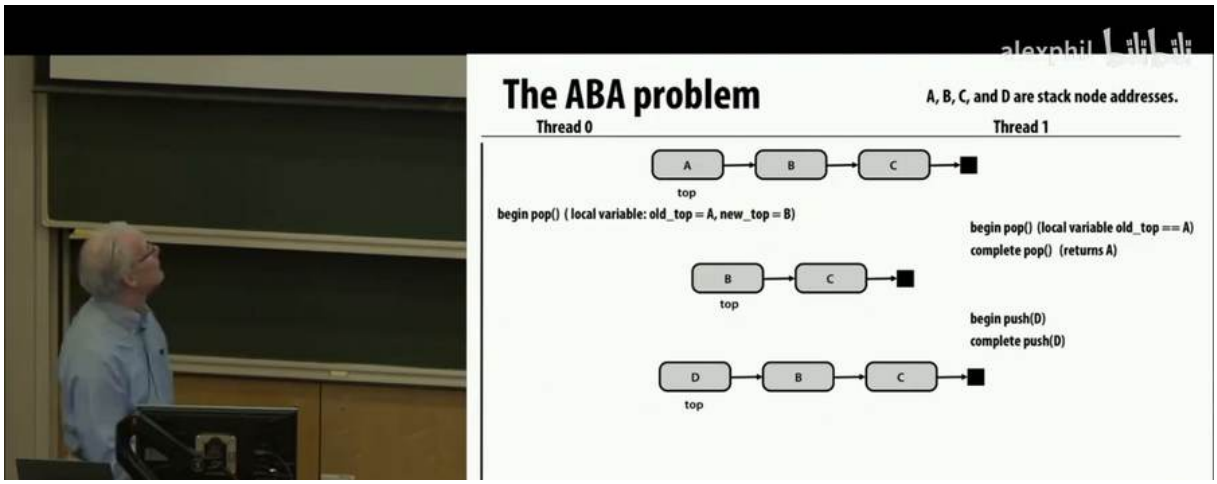

图 488: ABA 问题：指针值相同但所指内容已被改动[488]

讲者复述问题：你取得一个指向某物的指针，后来做比较并交换时看到同一个指针，于是认为它一定没变过。但在此期间，别的线程可能改动过这个指针或它所指向的东西，使其变得无效。

ABA 的本质是”值相等”不等于”历史相同”

比较并交换只能比较当前值，无法区分”一直没变”与”变过又变回来”。因此任何依赖 CAS 的判断都必须回答一个问题：这个值在被比较期间是否可能经历过变化再复原？若可能，就必须引入额外的历史信息（如计数器）或外部保护机制。

[488] 视频画面时间区间：00:01:30–00:01:45。若为裁剪图，时间区间仍对应原视频画面。

解法一：计数器与双重比较交换

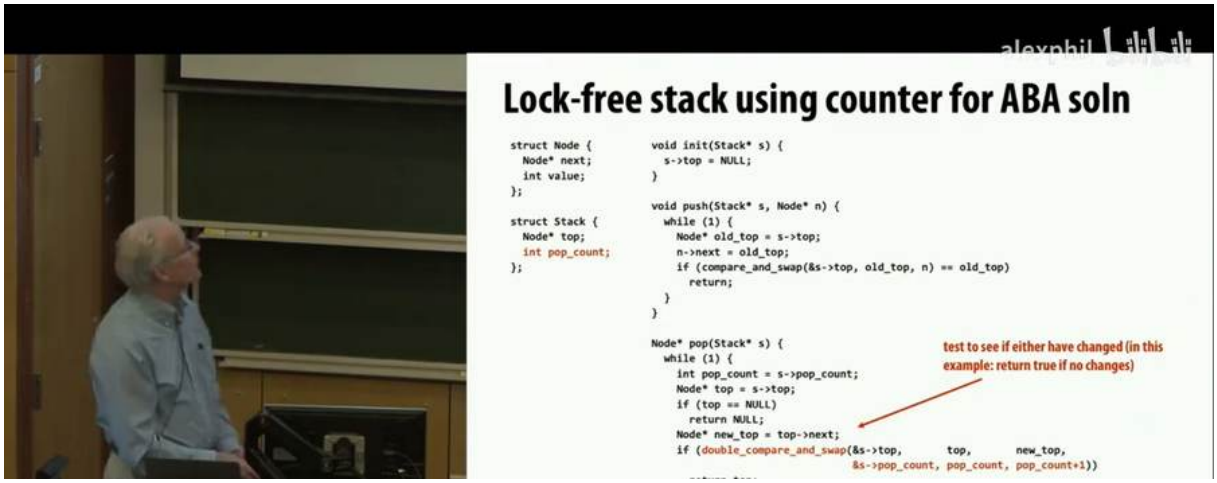

图 489: 用计数器解决 ABA 的无锁栈[489]

讲者回顾上一讲提过的做法：给数据结构加一个额外计数器，用所谓的双重比较交换，使得你可以同时把这个元素从链表中摘除并原子地递增计数器。

于是你不仅能判断”数据是否还是原来那个”,还能判断”计数器是否成功递增”,从而避免 ABA。

他在这里做了一个诚实的更正：上次讲完后他说这里还有另一个问题，但课后有人指出这段代码其实完全正确，这是相当经典的实现。若去查 ABA 问题，会看到关于双重比较交换的讨论。讲者还提到一个边界情形：如果那个 pop 计数器只是 32 位整数，那么它会回绕。回绕之后，一个足够长的运行就可能让计数器回到旧值，从而重新暴露 ABA。因此在长时间运行的系统里，计数器的位宽必须与并发规模匹配。

解法二：危险指针

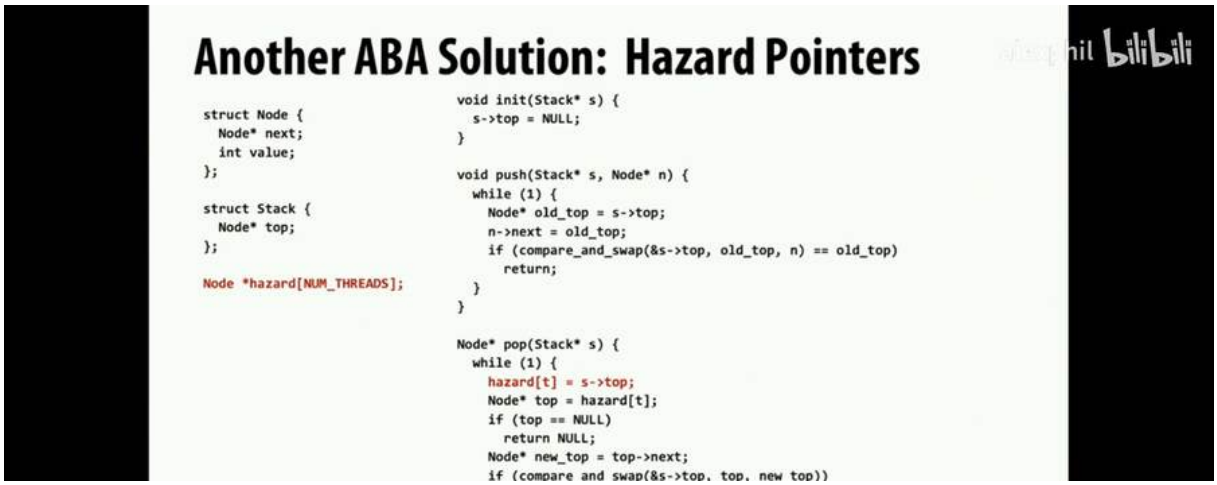

图 490: 另一种 ABA 解法：危险指针[490]

[489] 视频画面时间区间：00:02:00–00:02:15。若为裁剪图，时间区间仍对应原视频画面。

[490] 视频画面时间区间：00:05:30–00:05:45。若为裁剪图，时间区间仍对应原视频画面。

讲者接着介绍危险指针，说它其实是解决 ABA 的另一种完全不同的思路，而且是真正被使用的技术。

它的主要优势是不需要双重比较交换，普通 CAS 就够。

做法是：有一个按线程组织的指针数组，每个线程可以列出它正在持有的一个项，表示”我正在用这个，别人不要动它”。

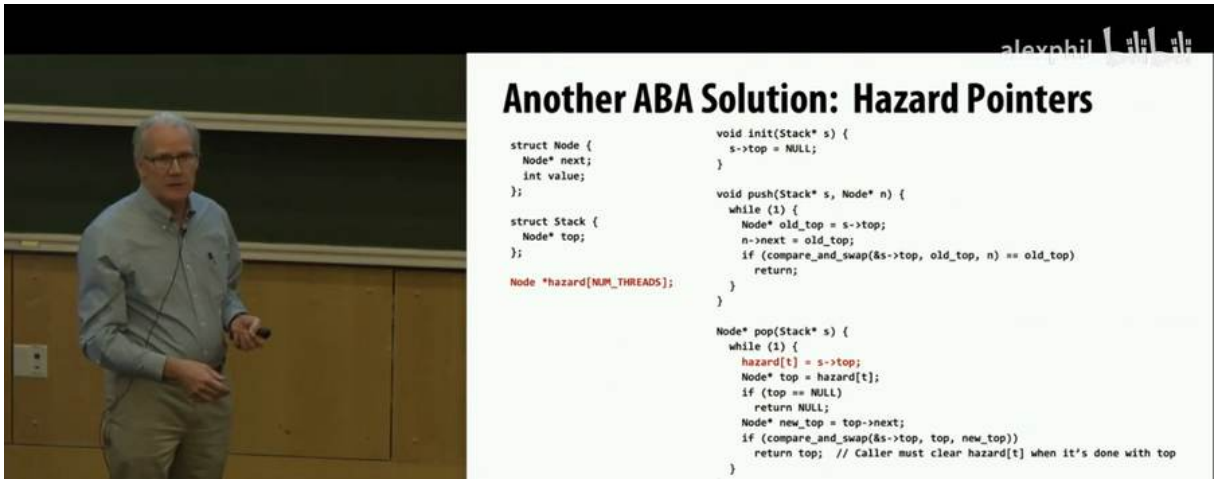

图 491: 危险指针续：调用者取到栈顶后保持标记为真[491]

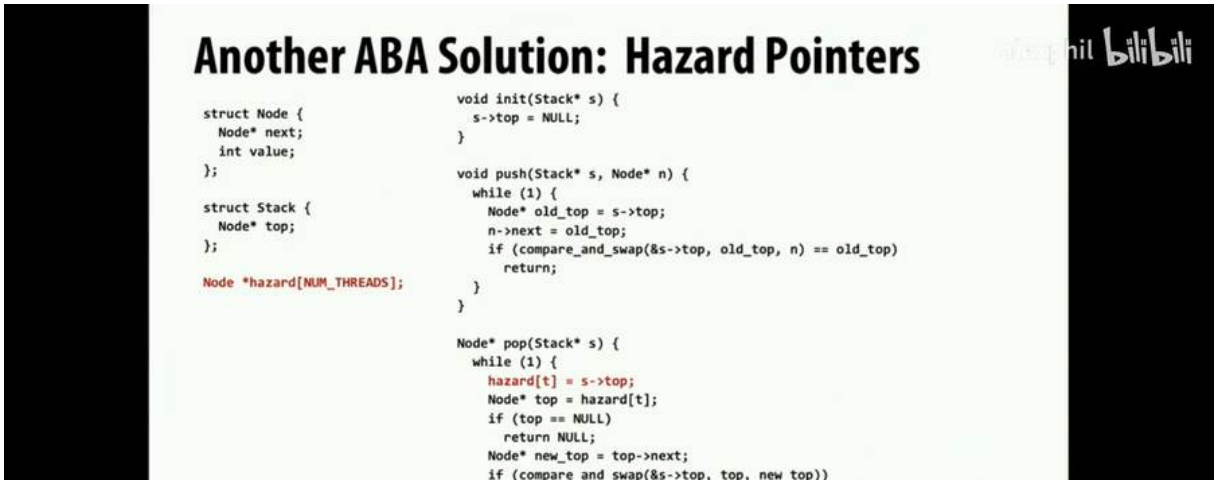

图 492: 危险指针续：回收节点前必须确认无人标记[492]

即使操作完成后，调用者仍会得到一个栈顶值；只要它还在引用那个栈顶，就应把危险标志保持为真。

一般规则是：任何时候你想回收或释放一个节点，只要还有线程把它标记为自己的危险指针，就不能这么做。

[491] 视频画面时间区间：00:07:00–00:07:15。若为裁剪图，时间区间仍对应原视频画面。

[492] 视频画面时间区间：00:07:30–00:07:45。若为裁剪图，时间区间仍对应原视频画面。

危险指针把”谁在用”变成可查询的全局状态

双重 CAS 用版本号记录”历史变过没有”；危险指针用共享标记记录”现在谁在用”。前者解决 ABA，后者同时解决”引用已释放内存”。因此两者不是同一问题的两种写法，而是针对不同侧面的机制。

两者是独立的解法

讲者澄清一个上次没说清的点：上次他讲危险指针时把它说成”要在计数器之外额外做的事”，但实际上它们是两种独立的实现方式。

危险指针不需要特殊指令，但需要某种更全局的控制与状态跟踪。

他还说危险指针在”不只是字级别、而是想对整个对象做原子操作”时也有用，因此它更多出现在高层协议里。

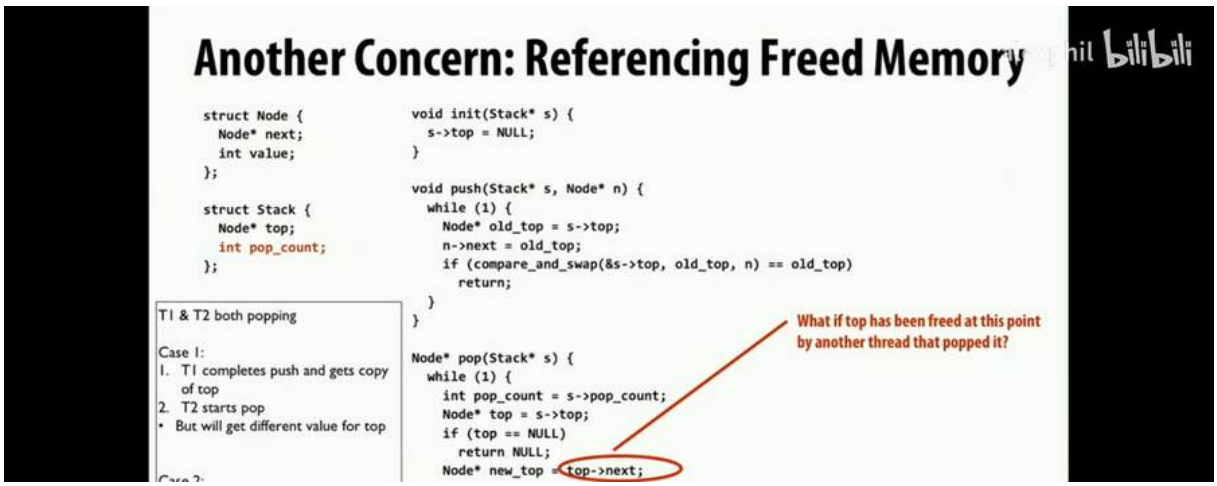

图 493: 另一个问题：引用已释放的内存[493]

本章小结

ABA 指指针值相同而所指内容已被改动，本质是”值相等”不等于”历史相同”。两种独立解法是：加计数器并用双重比较交换，可同时摘除元素与递增计数；或用危险指针，让每个线程标记正在持有的节点，回收前须确认无人标记。后者不需要特殊指令，但需要全局协调，也适用于整对象的原子操作。

[493] 视频画面时间区间：00:03:00–00:03:15。若为裁剪图，时间区间仍对应原视频画面。

22.2 为什么需要更高层的抽象

手工无锁的代价

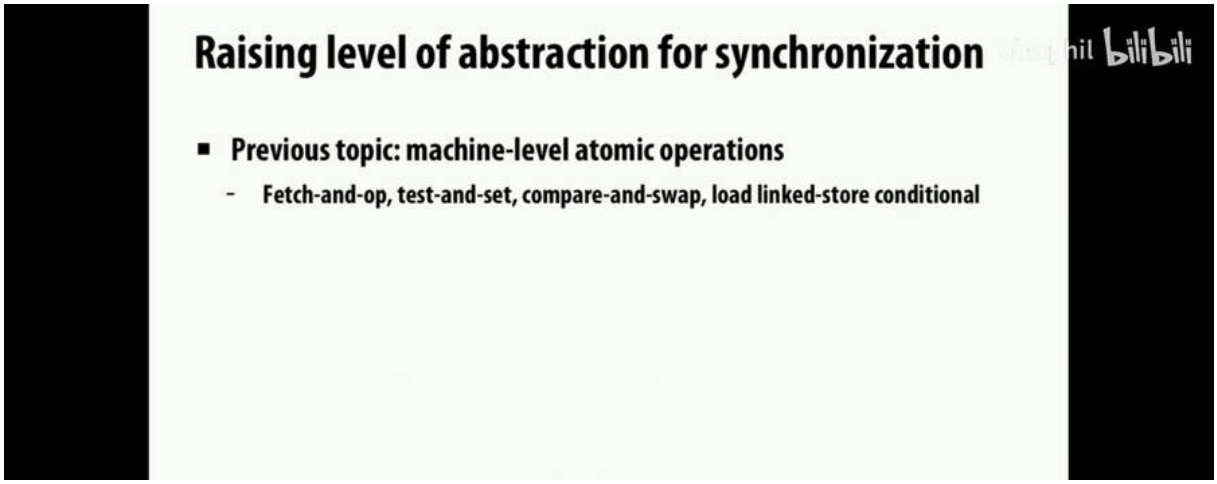

图 494: 提升同步的抽象层次：从机器级原子操作往上走[494]

讲者回顾上一段内容：我们一直在各种锁与技巧之间打转，或者像这个例子一样，直接停在数据结构层面思考如何无锁地实现。

他的评价很直接：这很枯燥，风险很高。

风险具体在哪？你可能觉得它工作正常，在某种处理器上测过；然后换一个型号，比如从 x86 换到 ARM，突然就不工作了。

抽象的目标

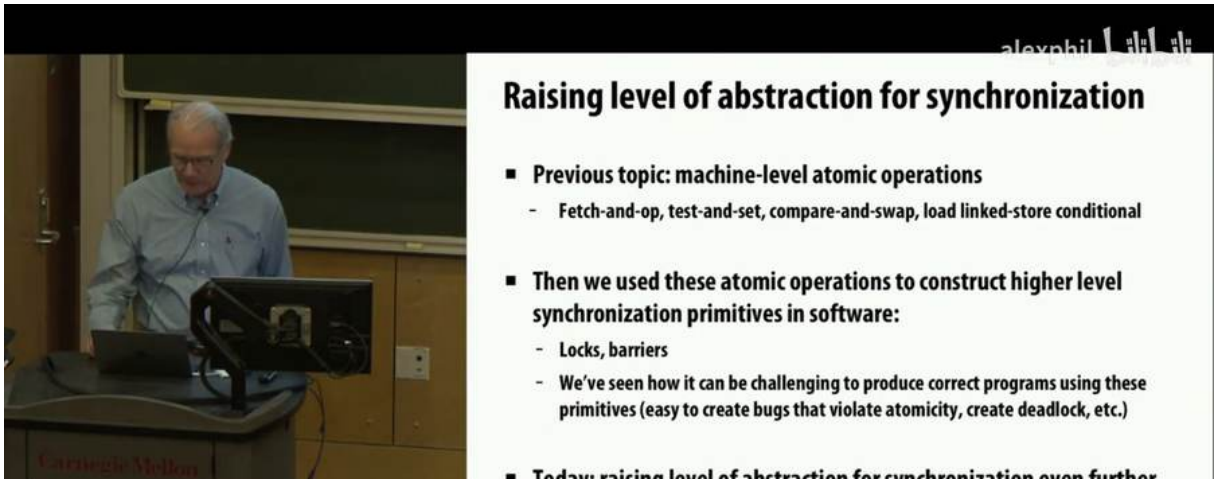

图 495: 提升抽象层次续：由编译器与硬件的组合决定实现[495]

因此今天想做的是：能否提升到更高的抽象层，表达一些我们想实现的高层原则，然后用编译器与硬件的某种组合让它们工作，并由系统去判断在具体情形下什么是最好的做法。

[494] 视频画面时间区间：00:10:00–00:10:15。若为裁剪图，时间区间仍对应原视频画面。

[495] 视频画面时间区间：00:11:00–00:11:15。若为裁剪图，时间区间仍对应原视频画面。

讲者加了一句玩笑：这除了会减少系统程序员的工作机会之外，一切都很好——而这目前似乎不是问题。

抽象提升的代价是失去控制权

当编译器与硬件替你决定”如何实现原子性”时，你也失去了对开销、延迟、回退策略的控制。因此在性能敏感的场合，高层抽象可能不够用。这也是事务内存在实践中长期处于”可用但不总是更快”状态的原因。

本章小结

手工实现无锁结构枯燥且风险高：在一种处理器上正常不代表换架构后仍正常。因此目标是提升抽象层，由编译器与硬件的组合决定实现。代价是失去对开销与策略的控制，这也是高层抽象在实践中不一定更快的原因。

22.3 事务：来自数据库的想法

经典例子

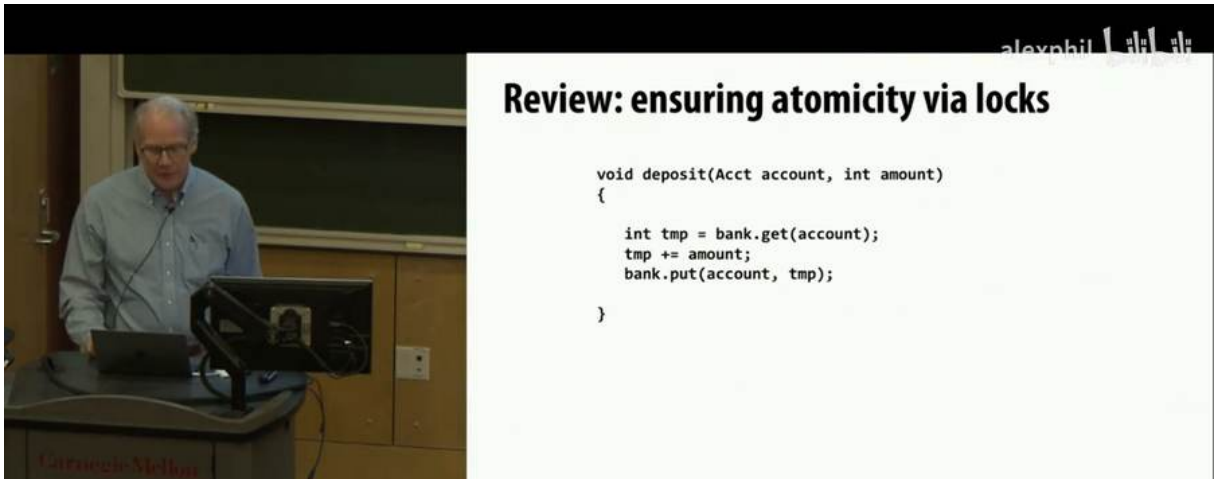

图 496：回顾：用锁保证原子性[496]

讲者说明事务的想法来自数据库社区，他们已经研究了几十年。

经典例子是银行账户：如果要把钱从账户 A 转到账户 B，我希望这件事以一种不会留下不一致状态的方式完成——比如钱从 A 扣了却没能存进 B。

这就是原子性：要么完全做完，要么完全不做。若某个事务失败，我希望它就像从未发生过一样。

他补充说，即使只是单纯往账户里存款（不涉及转账），也希望它要么成功且计数确实增加了，要么失败，并且事后我能知道它没有发生。

最直接的做法与它的不足

讲者说起点就是给它套一把大锁。这至少能保证不存在竞争条件导致账户被错误递增。

[496] 视频画面时间区间：00:12:30–00:12:45。若为裁剪图，时间区间仍对应原视频画面。

但这不是完整的方案。他真正想要的是类似在 OpenMP 里见过的那种：直接声明某段代码是原子的，然后由编译器、操作系统与硬件去想办法实现。

本章小结

事务思想来自数据库，已有数十年历史。经典例子是银行转账：原子性要求要么全做完要么全不做，失败则如同从未发生。最直接的做法是套一把大锁，它能避免竞争但不是完整方案——理想是直接声明某段代码为原子。

22.4 声明式与命令式

两种抽象的差别

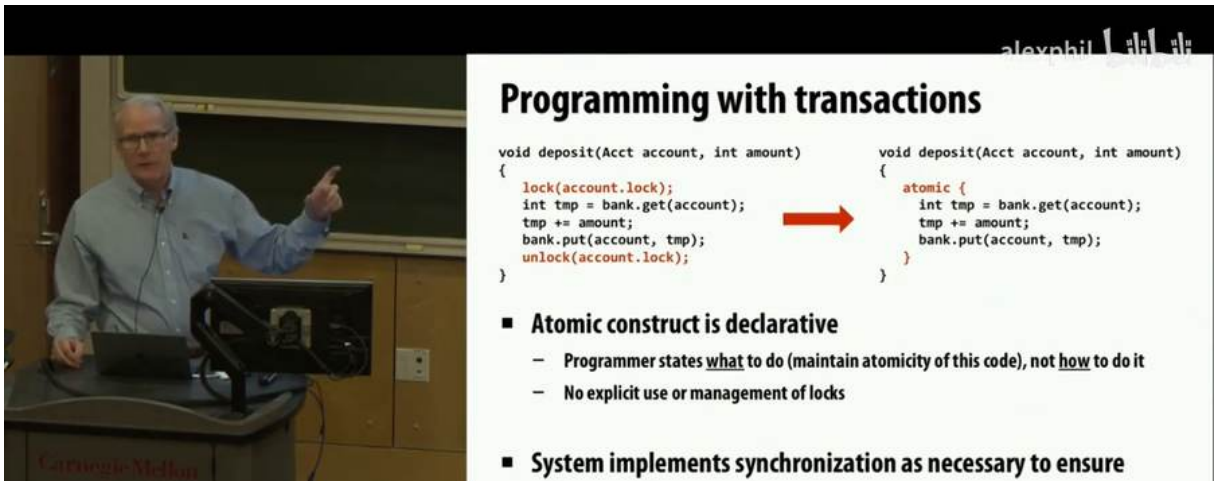

图 497: 用事务来编程：声明一段原子操作[497]

图 498: 声明式与命令式抽象的区别[498]

讲者给出定义。声明式是指：我只给出想做什么的意图。命令式是指：我要按下面的方式做——设置一把锁、做一些操作、释放锁。

[497] 视频画面时间区间：00:13:30–00:13:45。若为裁剪图，时间区间仍对应原视频画面。
[498] 视频画面时间区间：00:14:30–00:14:45。若为裁剪图，时间区间仍对应原视频画面。

事务内存的想法就是为内存系统提供这种原子能力，靠硬件、软件与编译器的某种组合来实现。

$$\text{声明式}:\ \text{atomic}\{\ \text{operations}\ \} \quad \longrightarrow \quad \text{命令式}:\ \text{lock};\ \text{operations};\ \text{unlock}$$

- 左侧只说明”这段要原子”，具体机制由系统选择；
- 右侧显式指定用锁，实现细节暴露给程序员；
- 左侧可以随硬件演进更换实现而不改代码，右侧不能。

声明式的价值在于把”怎么做”留给系统

同一段声明式代码，在不同硬件上可以被编译成锁、硬件事务、或软件事务。这意味着程序的可移植性从”语法层”提升到”语义层”。而命令式代码把机制写死，换平台时必须重写。

本章小结

声明式只给出意图，命令式给出做法。事务内存为内存系统提供原子能力，用硬件、软件与编译器组合实现。声明式的价值在于同一份代码可随硬件演进更换实现而不必重写。

22.5 事务的三个保证

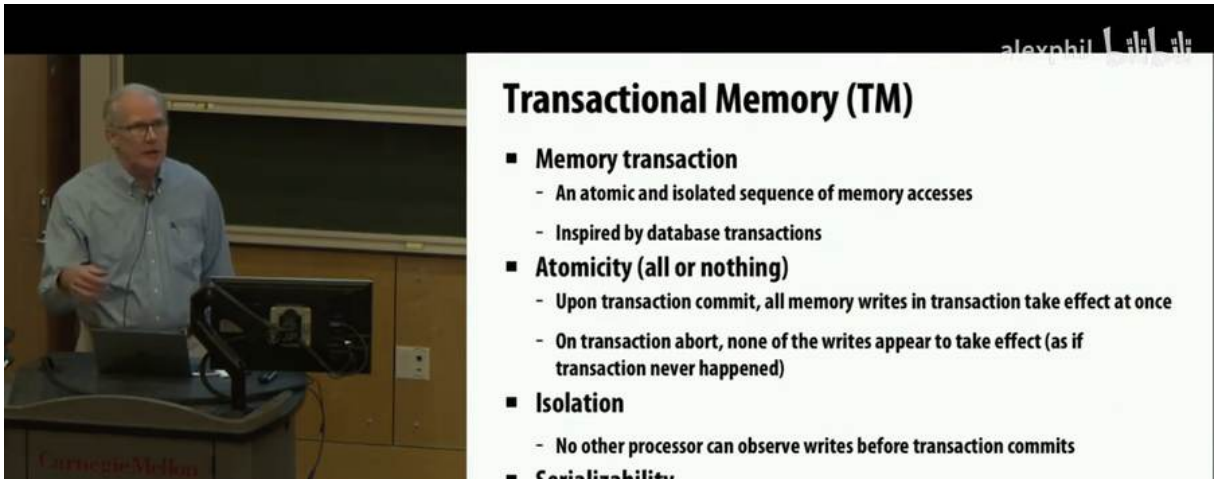

图 499: 事务内存：一段原子且隔离的内存访问序列[499]

[499] 视频画面时间区间：00:15:00–00:15:15。若为裁剪图，时间区间仍对应原视频画面。

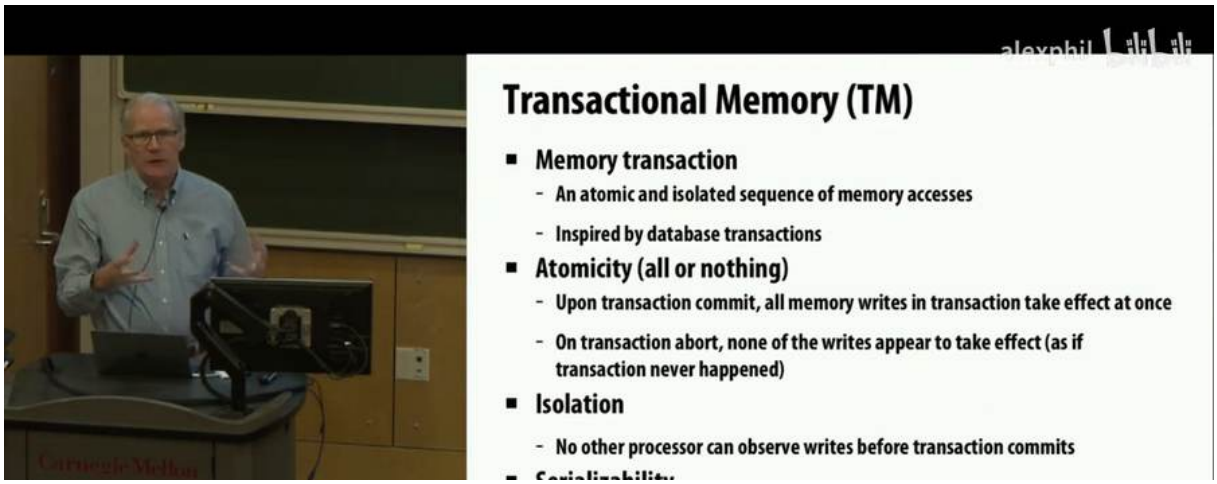

图 500: 事务内存的特性：原子性、隔离性与可串行化[500]

讲者列出希望具备的几个特性，它们都超出此前看过的普通同步。

原子性

第一个是原子性：要么全做要么全不做。括号里的每一个步骤都要发生，或者一个都不发生，不留任何残余。

比如作为转账的一部分，我绝不会出现” 从一个账户取出了钱但没能存进另一个账户”。

隔离性

第二个比此前看过的更强，叫隔离性：当事务进行时，如果有其他观察者对这些状态有只读访问权，他们不应看到任何中间状态。

也就是说，若他们在任意时刻查看银行账户，不应看到转账过程中某一个账户被扣减而另一个尚未增加的瞬间。对外部观察者而言，即使他们没有写权限，也应看到” 突然一下，两个账户的值一起变了”。

可串行化

第三个是可串行化，此前讨论过：我不关心事务在时钟意义上的确切顺序，但我关心它们是否可串行化——我可以把这段执行重放成某个序列，使它看起来像纯顺序发生的一样。

三个保证是递进关系而非并列

原子性保证” 不半途而废”，隔离性保证” 外人看不到中间态”，可串行化保证” 整体等价于某个串行顺序”。前两者约束单个事务的可见性，第三者约束多个事务之间的等价性。缺任何一条，程序员的推理都会失效。

本章小结

事务的三个保证是：原子性（全做或全不做，不留残余）、隔离性（外部观察者看不到中间状态，即使只读也不行）、可串行化（不关心时钟顺序，但可重放为某个串行序列）。三者递进：分别约束单事务的完整性、单事务的可见性、多事务间的等价性。

[500] 视频画面时间区间：00:16:30–00:16:45。若为裁剪图，时间区间仍对应原视频画面。

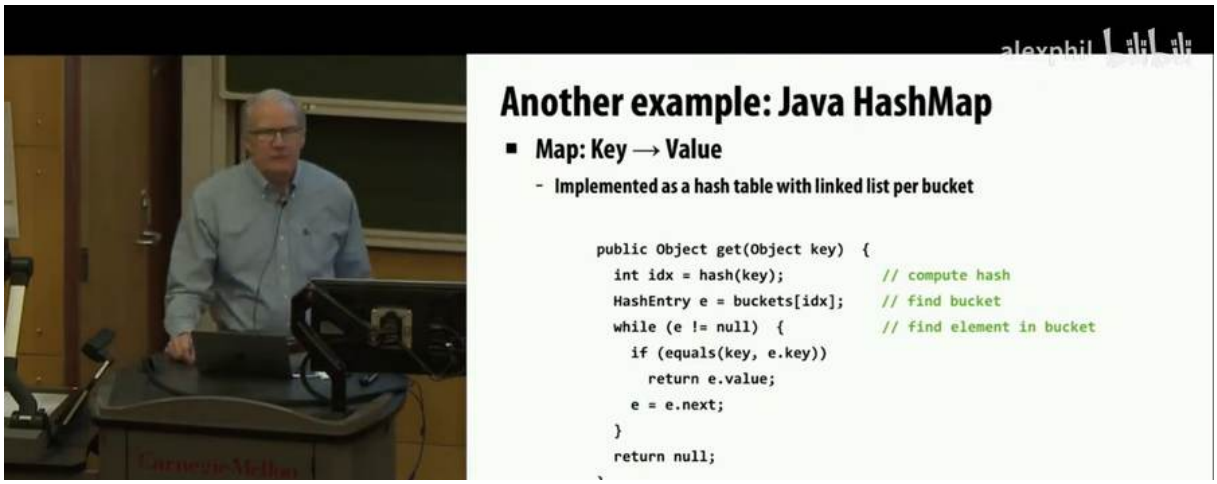

图 501: 事务内存的实现思路：按串行版本执行并检测冲突[501]

22.6 乐观执行与冲突检测

本章小结

事务的三个保证是：原子性（全做或全不做）、隔离性（外部看不到中间状态）、可串行化（可重放为串行序列）。三者递进：分别约束单事务的完整性、单事务的可见性、多事务间的等价性。

有了保证，接下来的问题是系统如何实现它们。

22.7 乐观执行与冲突检测

理想情形

讲者描述理想情形：我们希望能直接说”就原子地做这件事”，然后由系统去想办法实现。

而且可能不必靠细粒度锁，而是采用另一种思路：先假设没问题，就像串行版本那样执行，同时检查途中是否发生了冲突。

冲突的判据可以写成读写集合的交集：

$$\text{conflict} \iff (R_i \cap W_j) \cup (W_i \cap R_j) \cup (W_i \cap W_j) \neq \varnothing$$

- R_i, W_i：事务 i 读过的地址集合与写过的地址集合；
- 任意两项非空即表示两个事务之间存在冲突；
- 检测到冲突时，至少有一个事务必须回滚重试。

若两个事务的读写集合完全不相交，则它们可以并行完成而不必协调。

乐观执行把”预防”换成”检测”

细粒度锁是预防式的：进入前先取得权限，因此冲突不会发生。乐观执行是检测式的：先做，再检查是否被别人干扰过，若被干扰则回滚重试。前者在低冲突时开销偏高（每次都要加锁），后者在低冲突时几乎无开销，但在高冲突时会反复回滚。

[501] 视频画面时间区间：00:25:30–00:25:45。若为裁剪图，时间区间仍对应原视频画面。

重试的期望次数随冲突概率上升而急剧增加：

$$E[\text{retries}] = \frac{1}{1 - p_{\text{conflict}}}$$

- p_{conflict}：一次尝试中与并发事务冲突的概率；
- 当 p_{conflict} 接近 1 时，期望重试次数趋于无穷；
- 因此乐观执行在高冲突下会失效，必须辅以退避或切换到悲观模式。

一个具体对比：哈希表与树

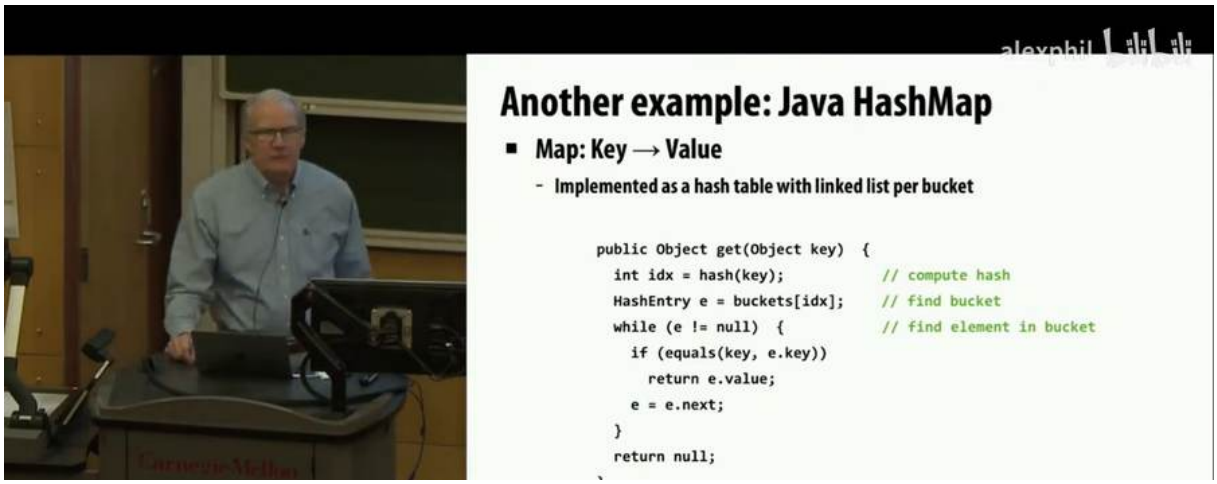

图 502: 具体例子：Java HashMap 的桶式实现[502]

讲者用哈希表与树的对比说明为什么这个区别重要。

哈希表可以直接做到 N 路分裂：计算哈希函数本身不需要锁，因此可以立刻跳到许多互不相关的链表上并行操作。

而树结构不一样：所有人都必须先穿过根，再穿过头几层。若树足够大，之后可以散开并行，但他们在根处是串行化的。

数据结构的形状决定了并行的天然上限

哈希表在入口处就把工作分成独立分支，因此并行度随桶数增长；树的入口是单一节点，因此并行度被根的限制压住。这不是锁实现得好不好的问题，而是结构本身的性质。因此”选什么数据结构”在并行场景下比”怎么加锁”更根本。

本章小结

理想情形是声明”原子地做”并由系统实现，且可以不走细粒度锁，而是乐观执行：按串行版本做，同时检测冲突，有冲突则回滚。哈希表因哈希函数无需加锁而可 N 路分裂；树的入口单一，所有人须先穿过根，因此天然串行化。这说明数据结构的形状决定了并行度的上限。

[502] 视频画面时间区间：00:28:00–00:28:15。若为裁剪图，时间区间仍对应原视频画面。

22.8 总结与延伸

讲者在结尾承认事务方法仍有许多已讨论过的现实困难，但能在更高层次表达这件事本身非常有价值。

把全讲串起来，这是一条从” 手工无锁的泥潭” 走向” 声明式原子性” 的路径。上一讲的无锁队列遇到 ABA：指针值相同而所指内容已被改动，本质是” 值相等” 不等于” 历史相同”。两种独立解法是加计数器配双重比较交换，或使用危险指针——后者让每线程标记正在持有的节点，回收前须确认无人标记，不需要特殊指令但需要全局协调，也适用于整对象的原子操作。手工实现无锁结构枯燥且风险高：在一种处理器上正常不代表换架构后仍正常。因此目标是提升抽象层，由编译器与硬件决定实现，代价是失去对开销与策略的控制。事务思想来自数据库：原子性要求全做或全不做、失败如同未发生；最直接做法是套一把大锁，但那不是完整方案。声明式只给出意图，命令式给出做法，而声明式的价值在于同一份代码可随硬件演进更换实现。事务的三个保证是原子性、隔离性与可串行化，它们递进而非并列。实现上可以采用乐观执行：按串行版本做，同时检测冲突，有冲突则回滚。哈希表因哈希函数无需加锁而可 N 路分裂，树的入口单一因此天然串行化——数据结构的形状决定了并行度的上限。

本讲义在讲者材料之上做的压缩，有四点需要与讲者原话区分：

- 三个保证是递进的：原子性约束单事务的完整性，隔离性约束其对外可见性，可串行化约束多事务之间的等价性。只满足前两者而违反第三者，程序仍可能得到无法用任何串行顺序解释的结果。
- 危险指针与双重 CAS 解决的是不同侧面。前者回答” 现在谁在用这个节点”，后者回答” 这个值的历史是否变过”。因此它们可以互补，但不是同一问题的两种等价写法。
- 乐观执行的适用性取决于冲突率。低冲突时它几乎无开销，高冲突时会反复回滚，实际吞吐可能低于细粒度锁。因此” 事务比锁好” 不成立，只有” 在特定冲突率下更好”。
- 数据结构的形状比同步机制更根本。哈希表天然可分裂，树天然在根处串行。这一点在选择数据结构时就已决定，后续任何同步优化都无法突破。

这堂课的边界在于它是概念导论：讲者明确说事务方法仍有大量现实困难，本讲给出的是动机、保证与实现思路，不含具体的硬件事务指令或软件事务库的使用细节。

两个可以直接从本讲数据提出的下一步问题：其一，乐观执行在高冲突下会反复回滚，那么是否存在一种自适应策略，能在运行时检测冲突率并切换到悲观（加锁）模式？其二，可串行化允许重排为任意串行顺序，那么对于依赖特定顺序的算法，事务内存是否提供了表达顺序约束的手段？

Lecture 23 Heterogeneous Parallelism

视频时长：1 小时 20 分 12 秒 来源：https://www.bilibili.com/video/BV1R8411Q7cB/

23.1 功耗：改变设计思路的主导约束

这一讲讨论一个有意思的话题：异构计算环境有什么好处，以及从纯软件走向更接近硬件（甚至自己做硬件设计）能带来什么。

从手机到超算的共同约束

讲者开场给出一个贯穿性观察：从手机到超级计算机，都在使用非常混合的计算引擎，而且原因各不相同。

但真正主导当代计算机设计的是功耗。超算机房供电不能超过 10 兆瓦；手机若耗电太多，既会耗尽电池，也会热到无法握在手里或放在口袋里。

因此无论最小的还是最大的计算机，功耗都是那个问题。过去十年里，这一点改变了很多关于”什么该做、什么不该做” 的思考。

功耗约束把” 性能” 重新定义为” 每瓦性能”

在供电受限的前提下，增加算力的唯一途径是提高每瓦效率，而不是简单地堆更多晶体管。这解释了为什么专用加速器有吸引力：它在特定任务上能效远高于通用核，因此同样功耗预算下能完成更多工作。

已有硬件大多被闲置

讲者提醒一个事实：我们在单核系统上已经见过极大的性能差异，取决于你愿意花多大力气。

作业一里，如果你使劲用上 SIMD 向量单元和多线程，就能在不换任何硬件的情况下让应用大幅加速。

” 买更多硬件” 与” 用更好现有硬件” 是两条不同的路

前者需要更换系统，后者只需改代码与算法。第 8 讲与第 15 讲的例子都说明后者的收益可以很大且成本更低。讲者在此提出这一点，是为了说明异构讨论之前还有一层更基础的问题：现有资源是否已被充分利用。

他的判断是：典型机器拥有的硬件远多于多数程序实际使用的部分。特别是整个 SIMD 向量单元，除了那些有人花大力气优化过的库之外，基本处于闲置状态。

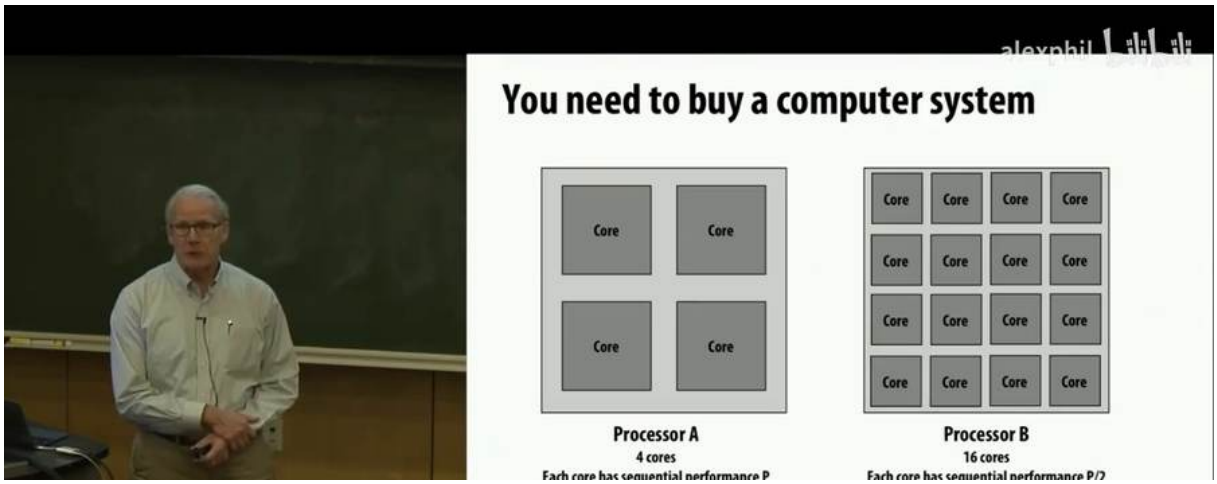

图 503: 你需要买一台计算机系统：固定面积下如何分配核[503]

本章小结

功耗是从手机到超算的共同主导约束，它把性能问题重新定义为每瓦性能问题。同时，现有硬件的相当部分（尤其 SIMD 单元）在多数程序中被闲置。因此”异构”与”更充分利用已有硬件”是同一件事的两面。

下一章用一个固定面积的简化模型，把胖核与瘦核的取舍算清楚。

23.2 固定面积下的胖核与瘦核

一个选择题

讲者先回顾一道考题：如果可以在 k 个性能为 1 的处理器与 1 个性能为 k 的处理器之间选择，你选哪个？

Amdahl 定律使”等价交换”不成立

k 个性能为 1 的核，理论峰值是 k；1 个性能为 k 的核，峰值也是 k。但前者受串行部分与并行开销拖累，实际只能达到远低于 k 的加速比，而后者没有这些损失。因此两者峰值相同但实际收益不同。

答案应当是后者，因为你几乎拿不到理想加速比，所以这不是等价交换。

[503] 视频画面时间区间：00:04:00–00:04:15。若为裁剪图，时间区间仍对应原视频画面。

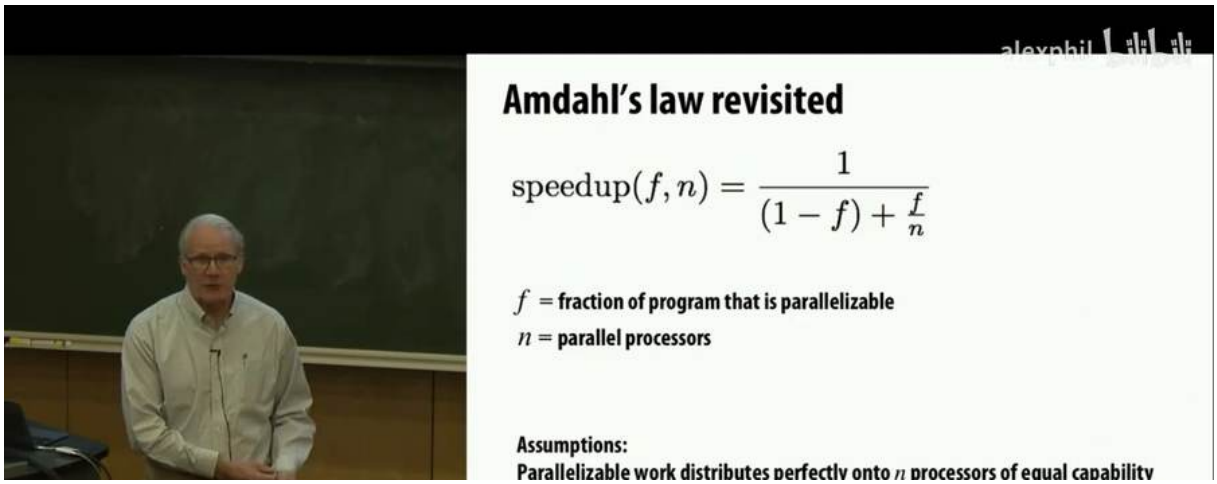

图 504: 重访 Amdahl 定律：加速受未并行部分限制[504]

但他接着把这个问题做得更细：设想在固定芯片面积下，面积并不均等划分。假设胖核的性能恰好是瘦核的两倍，那么右侧系统的原始算力可达八倍。

把 Amdahl 定律改写成资源形式

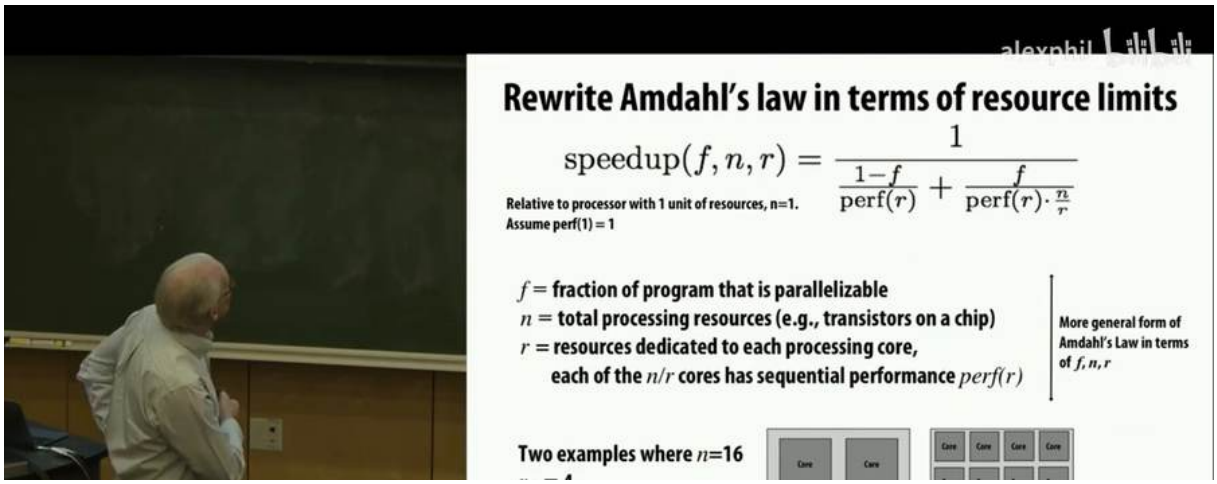

图 505: 用资源限制改写 Amdahl 定律[505]

讲者把 Amdahl 定律重新表述。经典形式是：

$$S(f,n) = \frac{1}{(1-f)+\frac{f}{n}}$$

- f：程序中可并行部分的比例；
- n：并行处理器数；
- 当 $n \to \infty$ 时，S 的上界为 $1/(1-f)$。

[504] 视频画面时间区间：00:05:30–00:05:45。若为裁剪图，时间区间仍对应原视频画面。
[505] 视频画面时间区间：00:07:30–00:07:45。若为裁剪图，时间区间仍对应原视频画面。

而资源形式需要引入核大小。假设总资源为 N，每核大小为 r，则核数为 N/r，每核性能为 $\mathrm{perf}(r)$：

$$S(f,N,r)=\frac{1}{\dfrac{1-f}{\mathrm{perf}(r)}+\dfrac{f\cdot r}{N\cdot\mathrm{perf}(r)}}$$

- $\mathrm{perf}(r)$：单核性能随其分得资源 r 的函数；
- N/r：核的数量；
- 关键点是 $\mathrm{perf}(r)$ 呈收益递减，而非线性增长。

收益递减是这套模型的唯一关键假设

讲者说你可以选任何喜欢的函数，但收益递减是普遍性质。他取的示例是：

$$\mathrm{perf}(r)=\sqrt{r}$$

- r：单个核分得的资源份数；
- $r=1$ 时性能为 1，作为归一化基准；
- $r=4$ 时性能为 2，$r=16$ 时性能为 4。

面积翻四倍而性能只翻两倍，这就是”递减”的具体含义。他取 $\mathrm{perf}(r)=\sqrt{r}$ 作示例：$r=4$ 时性能只有 2 倍，$r=16$ 时只有 4 倍。这个选择的具体形式不重要，重要的是它必须递减——只要递减，后面的结论就成立。

曲线怎么读

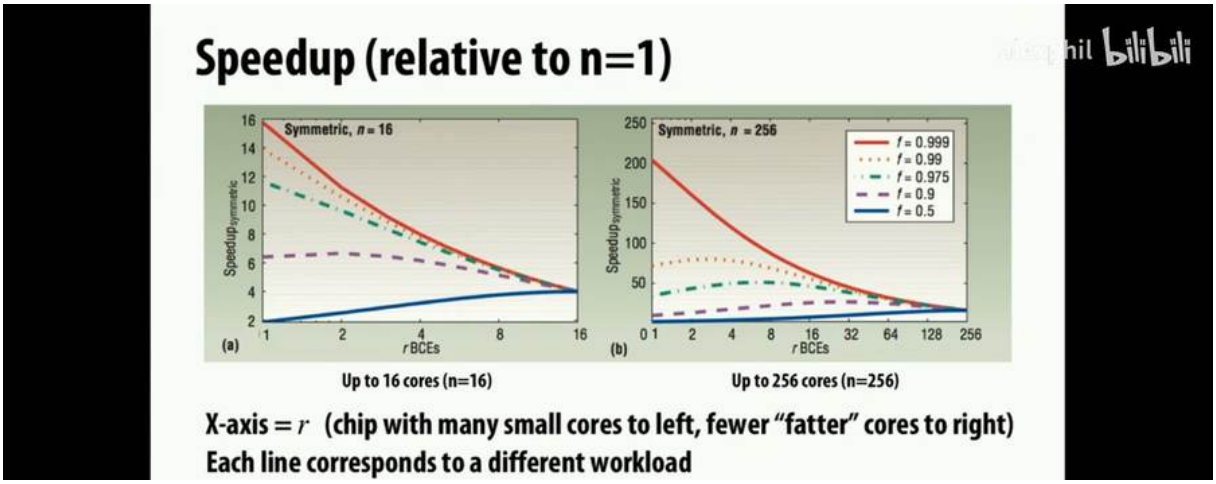

图 506: 加速比曲线（相对单核）：对称 16 核情形[506]

讲者画出”对称情形”的曲线族，也就是所有核都相同、每核分得 r 份资源。

横轴是核大小，纵轴是相对”一个极小的单核”的加速比。问题是：把面积放大 16 倍，究竟能得到多少实际收益？

[506] 视频画面时间区间：00:10:30–00:10:45。若为裁剪图，时间区间仍对应原视频画面。

结果取决于并行比例：

- 若几乎完美并行（串行只占 0.01%），那么把芯片切成尽可能多的小核就是最优，几乎能拿到完美的 16 倍；
- 但只要串行比例升到 1%，就开始出现明显损失；
- 到 10% 时，最多只能拿到约 6 倍多；
- 若只有一半可并行，加速比只有 2 倍左右。

收敛点揭示了模型的边界

所有曲线收敛到同一个点，说明无论并行比例如何，把整块面积做成一个核都是同一个结果。这意味着模型的差异全部体现在” 中间区域”：核不太大也不太小的配置。因此设计决策的关键区间正是中间地带，而两端是退化情形。

所有曲线都收敛到同一个点：单核占满整个芯片，此时 $r = 16$，性能为 4 倍（按平方根模型）。

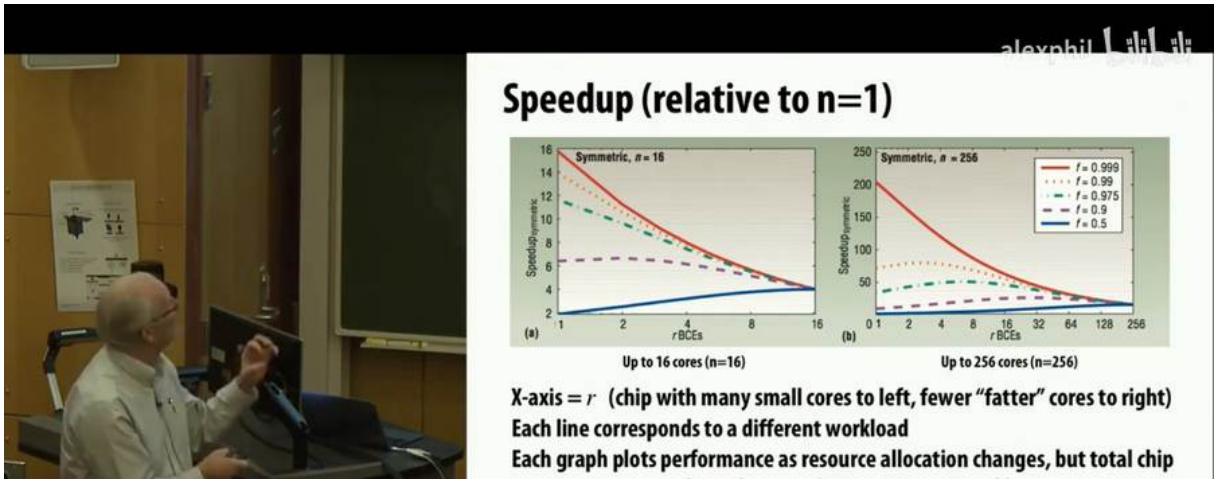

图 507: 面积放大 256 倍的加速比曲线：Amdahl 效应更严峻[507]

若把面积放大到 256 倍，原理相同但更极端。即便九成九可以并行，加速比也只有 70 倍左右，尽管芯片面积大了 256 倍，这个数字远低于直觉预期。

同构设计在两端都不讨好

若把面积都做成胖核，收益因递减而有限；若都做成瘦核，则串行部分会成为硬瓶颈。因此同构方案在” 并行度不高” 与” 并行度极高” 两种极端下都不是最优。这正是异构设计的出发点。

本章小结

固定面积下，胖核与瘦核的取舍由串行比例决定：几乎全并行时小核最优，串行比例升高后需要更大的核。收益递减使同构方案在两端都不理想。Amdahl 定律的资源形式把核大小纳入模型，使这个权衡可以定量计算。

那么把串行部分专门交给一个胖核会怎样？

[507] 视频画面时间区间：00:13:30–00:13:45。若为裁剪图，时间区间仍对应原视频画面。

23.3 异构方案：一个胖核加若干瘦核

模型设定

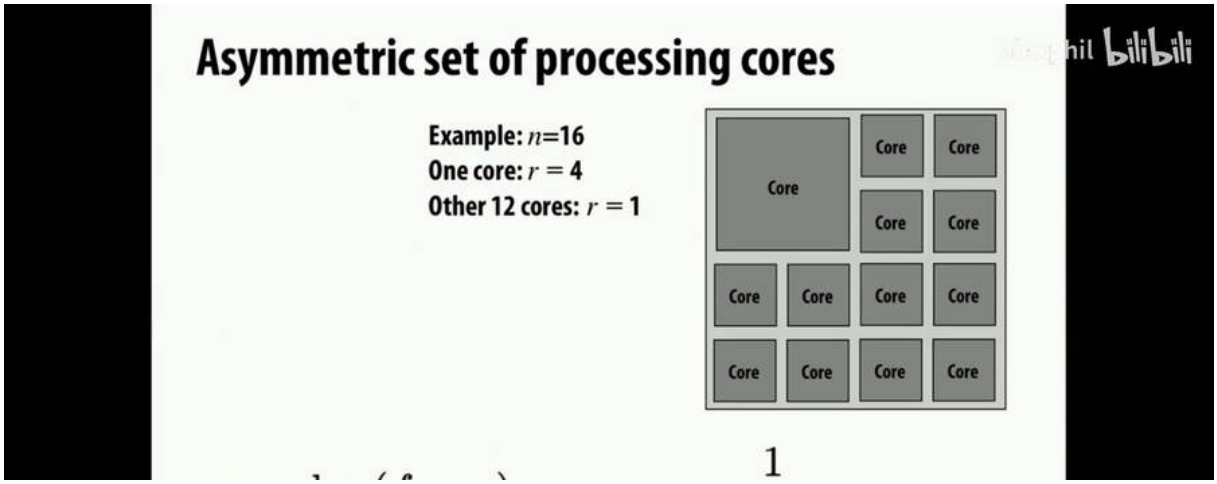

图 508: 非对称处理核集合：一个 $r = 4$ 的胖核加若干瘦核[508]

讲者提出一个很自然的问题：与其假设所有核都相同，不如做一个胖核加一批单位大小的瘦核。

三个参数是：可加速部分的比例、以基本小核为单位衡量的芯片总大小 N、以及分配给那个胖核的资源量 R。

基本想法是把串行部分映射到胖核上，把并行部分映射到瘦核上，从而从这个组合中获得收益。

$$S_{\text{hetero}} = \frac{1}{\dfrac{1-f}{\text{perf}(R)} + \dfrac{f}{\dfrac{N-R}{1}\cdot 1}}$$

- 第一项：串行部分在胖核上执行，其性能为 $\text{perf}(R)$；
- 第二项：并行部分分散到 $N - R$ 个瘦核上，每个性能为 1；
- 隐含假设是映射完美——串行代码全部跑在胖核上，并行代码全部跑在瘦核上。

[508] 视频画面时间区间：00:15:30–00:15:45。若为裁剪图，时间区间仍对应原视频画面。

曲线更平缓

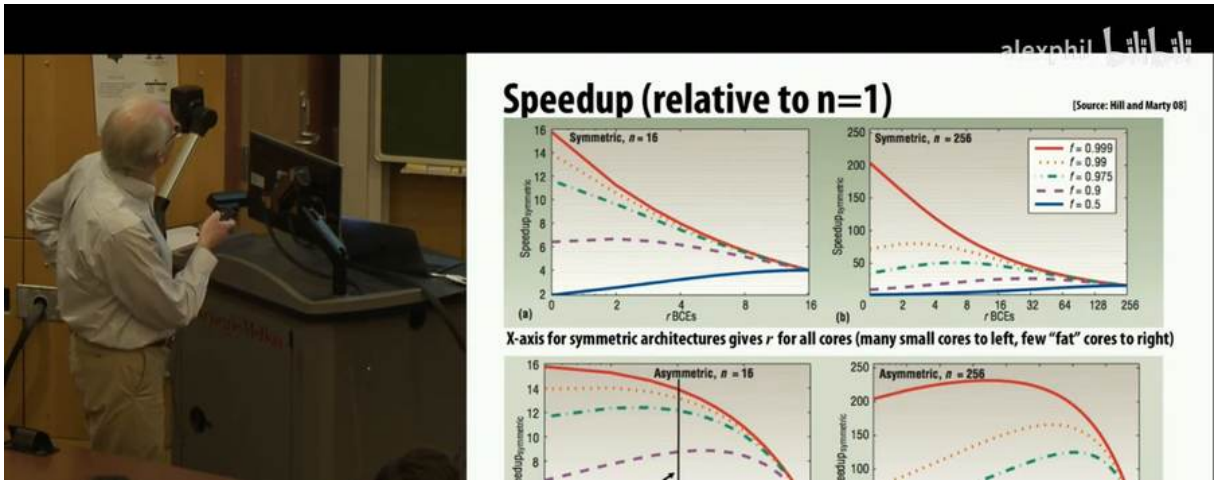

图 509: 非对称方案的加速比曲线（Hill 与 Marty 08）[509]

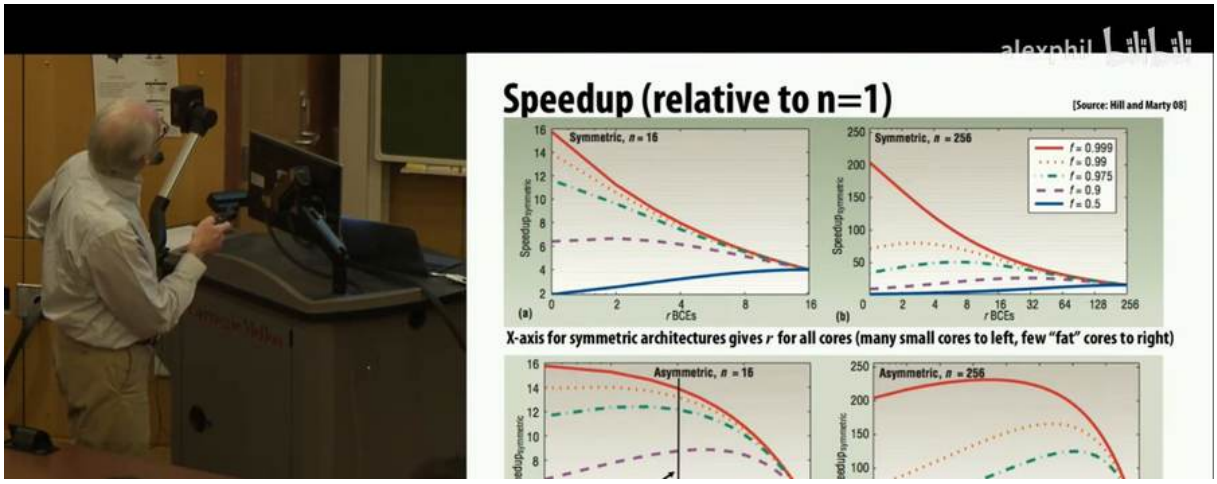

图 510: 非对称方案劣化更平缓，最优区间更宽[510]

讲者把非对称曲线与之前对称曲线放在一起比较。以 16 份总资源为例，若胖核占 0.25 的资源（即 $r = 4$），其余 12 份留给瘦核。结论很清楚：对称情形下曲线是陡峭下降，而非对称情形下劣化更平缓。

即便串行比例极低（0.01%），非对称方案也能在很宽的范围内接近理想并行。而更有意思的是那条 99% 并行的虚线，以及 2.5% 串行的情形——它们都" 撑得住"。

所有曲线仍会收敛到" 一个胖核占满整个芯片" 的点，但在此之前，非对称方案给出的收益明显更好。

讲者总结：把一部分面积专门用于顺序处理，然后把尽可能多的面积撒上一堆能做并行的小处理器，在这个模型下是个好主张。

最优的胖核大小可通过求导得到，其定性结论是：

$$R^* \propto \sqrt{1-f}$$

[509] 视频画面时间区间：00:16:30–00:16:45。若为裁剪图，时间区间仍对应原视频画面。

[510] 视频画面时间区间：00:17:30–00:17:45。若为裁剪图，时间区间仍对应原视频画面。

- R^*：最优分配给胖核的资源量；
- f：可并行部分的比例；
- 串行比例越高（f 越小），胖核应分得越多资源。

这解释了为什么异构方案的最优区间比同构方案更宽：它多了一个可调参数。

异构的价值在于”让最坏情形也不至于太差”

同构方案的问题是最优核大小对串行比例极其敏感，一旦比例偏离假设就会大幅退化。异构方案牺牲一小块面积给胖核，换来的是对串行比例的鲁棒性：无论比例落在哪个量级，性能都不至于崩塌。这是用面积换稳健性的交易。

本章小结

异构方案把串行部分映射到胖核、并行部分映射到瘦核，曲线劣化更平缓、最优区间更宽。其价值在于对串行比例的鲁棒性，而不是更高的峰值。代价是一块面积被专用化。

但这个模型建立在若干乐观假设之上，现实约束会削弱它。

23.4 现实约束：为什么不能照模型定制

三条限制

讲者随即给出三条现实限制。

第一，这个模型假设我们能完美地把串行计算映射到胖核、把其余部分映射到其他核。这是乐观假设。

第二，很少有工作负载被刻画得如此清楚，可以围绕它设计整个系统。

第三，芯片开发成本达数百万美元，只有卖得足够多才能摊薄。而几乎没有哪个应用有足够的量来支撑一颗专用芯片。

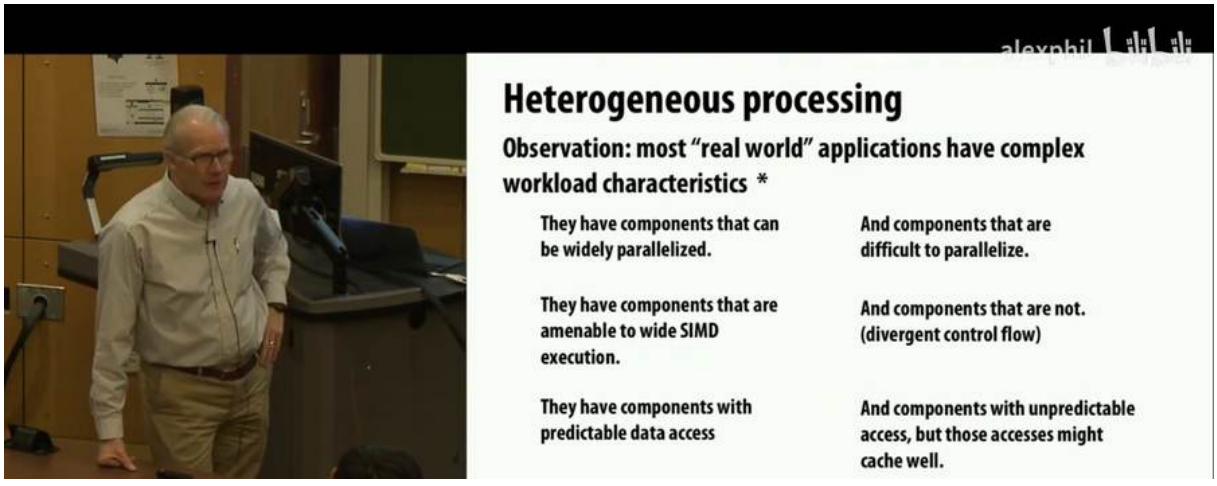

图 511：异构处理：真实应用的工作负载特征往往复杂多变[511]

[511] 视频画面时间区间：00:22:00–00:22:15。若为裁剪图，时间区间仍对应原视频画面。

专用化与通用性是一对反向约束

专用化提升能效，但缩小了适用范围；通用性扩大适用范围，但牺牲能效。商用芯片必须服务未知负载，因此只能在两者之间取折中。这正是”异构”而非”专用”成为主流的原因：异构在保持通用性的同时获得部分专用化的收益。

他甚至指出手机也不是”一个负载”——它有各种各样不同的负载。因此把这些想法应用到实践中是个困难的工程权衡。

异构设计的真实约束是”通用性”

理想模型可以为一个已知串行比例定制芯片，但商用芯片必须服务未知负载。因此实际设计不追求针对某个比例的绝对最优，而追求在广泛负载范围内都不至于太差。这解释了为什么商用芯片的异构配置往往是”一个大核 + 若干中核 + 若干小核”这样的组合。

本章小结

异构方案把串行部分映射到胖核、并行部分映射到瘦核，结果是劣化更平缓、最优区间更宽，对串行比例更鲁棒。但现实限制有三：映射假设过于乐观、负载难以提前刻画、芯片成本要求足够销量。因此商用设计追求广泛负载下的稳健而非针对某点的最优。

下一章看真实系统如何落地这些想法。

23.5 真实系统里的异构

移动 SoC

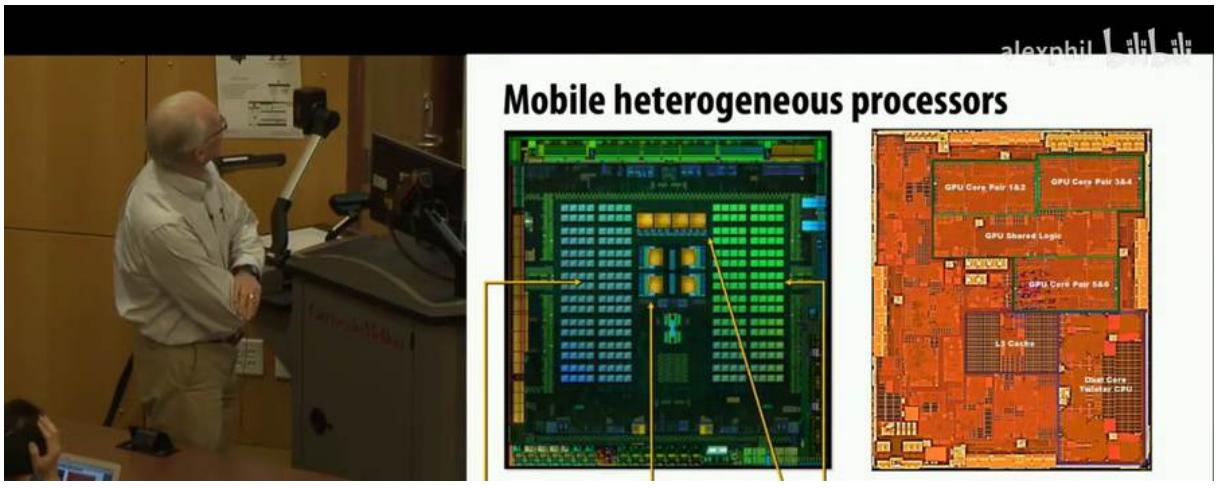

图 512: 移动端异构处理器：常规核、GPU 与共享逻辑[512]

讲者展示了一个移动处理器的布局，指出它已经在用异构核的想法。

常规核——也就是你登录进去跑线程的那些——只占整颗芯片的一部分，包含各自的一级与二级缓存。图中还有一块叫共享 LLC，即最后一级缓存（L3 的缓存控制器）。

[512] 视频画面时间区间：00:29:00–00:29:15。若为裁剪图，时间区间仍对应原视频画面。

然后是一块 GPU。讲者特意说明：这不是 NVIDIA 那种可编程、能做非图形工作的 GPU，它们相当专门化于图形处理，可编程性与灵活性都不如 NVIDIA 的产品。但大多数笔记本（除非有独立显卡）就靠它。

他还指出，这个示例是四核版本，其他芯片会塞进更多核。但相当大一块面积专门给了图形控制。媒体引擎则负责 MPEG 视频解码之类的处理，这些是人们日常都在用的。

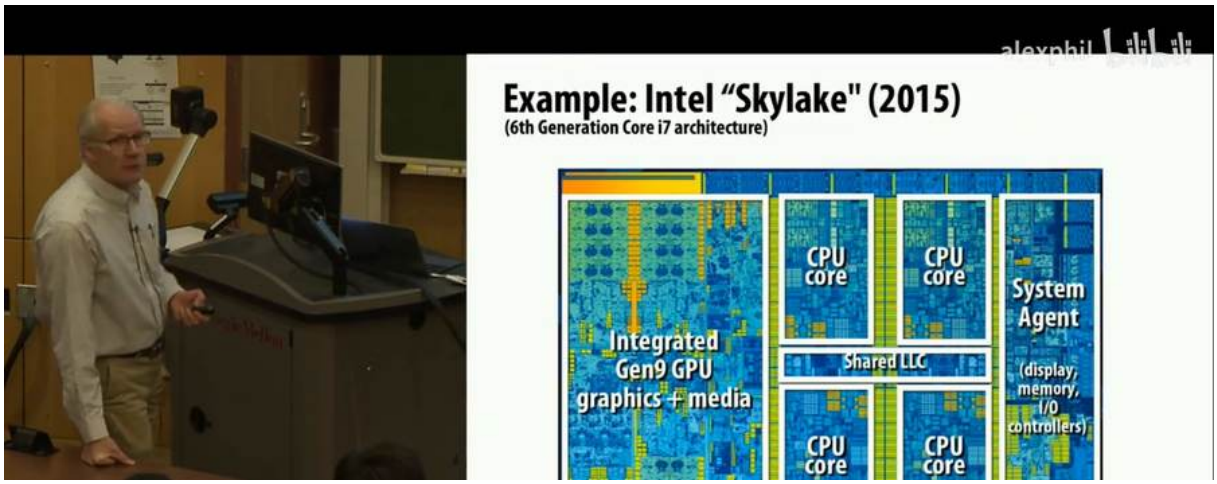

图 513: 实例：Intel Skylake（2015，第六代 Core i7）的布局[513]

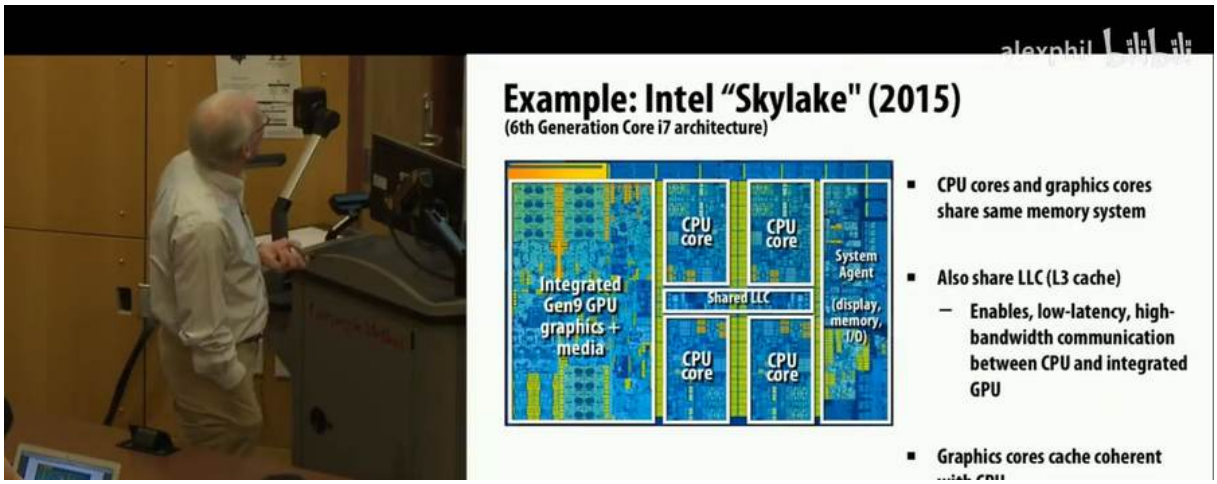

图 514: Skylake 中 CPU 核与图形核共享最后一级缓存[514]

片上集成的关键优势

讲者在这里点出一个性能上的关键差别：把所有东西放在一颗芯片上的好处之一是，GPU 也能共享这最后一级缓存。

于是它与内存系统之间有了非常高带宽的耦合。

这正好对应第 7 讲里你们在 Gates 机器上遇到的 NVIDIA GPU 的问题：你必须做 cudaMemcpy 来回搬运数据。那是个真正的性能杀手，因为它依赖外部总线的带宽，而不是芯片内部能提供的带宽。

[513] 视频画面时间区间：00:24:00–00:24:15。若为裁剪图，时间区间仍对应原视频画面。
[514] 视频画面时间区间：00:26:00–00:26:15。若为裁剪图，时间区间仍对应原视频画面。

数据搬运的代价由互连带宽决定

外接加速器的算力再强，若每次调用都要把数据经外部总线搬进搬出，实际性能就会被搬运时间主导。片上集成把这条路径换成共享缓存，带宽高出若干倍，因此对”计算量相对数据量不大”的任务收益尤其明显。

再加一块独立 GPU

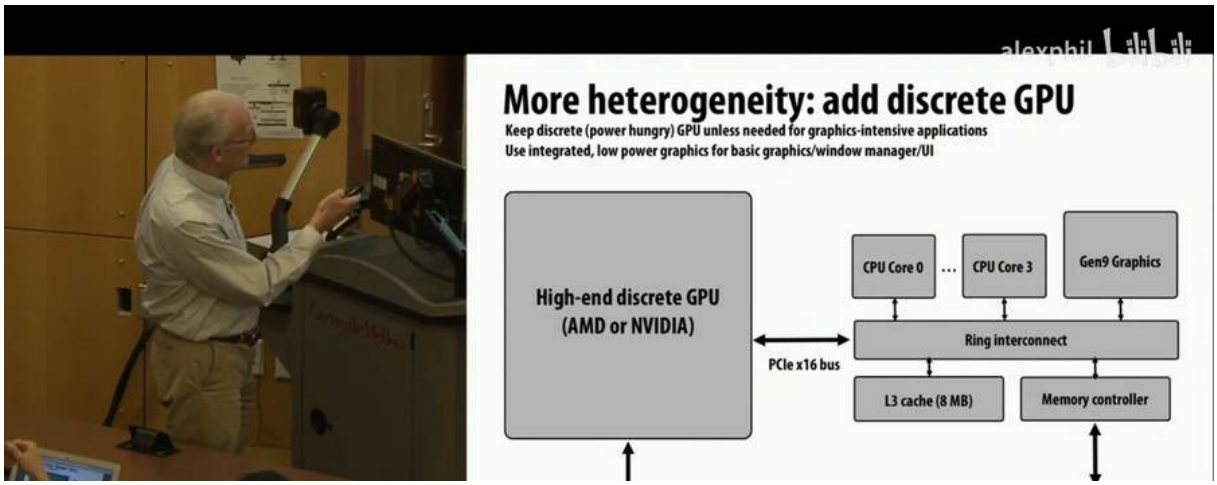

图 515: 更多异构：保留耗电的独立 GPU，仅在需要时启用[515]

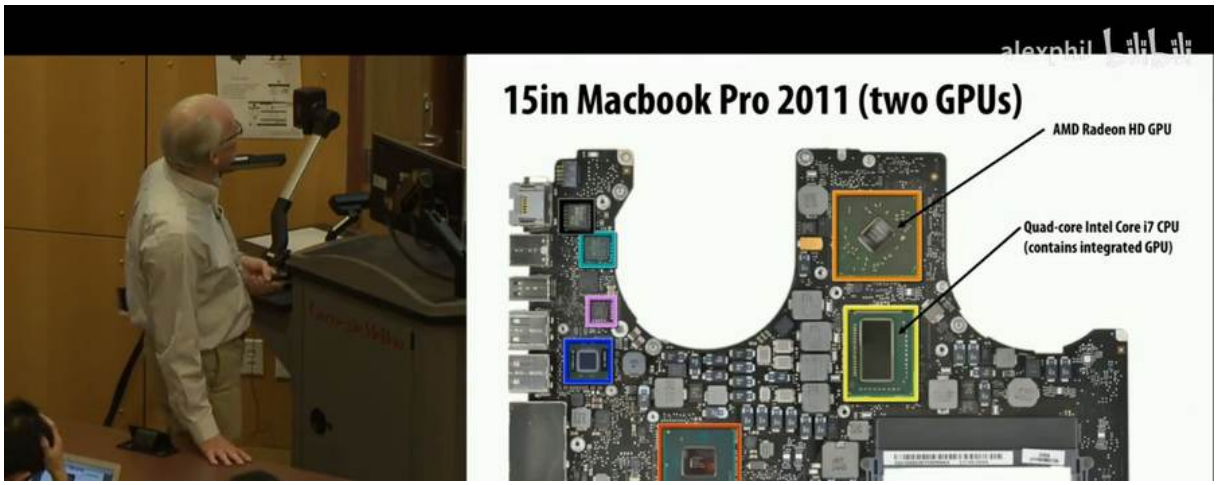

图 516: 15 寸 MacBook Pro 2011：四核 CPU 加两块 GPU[516]

讲者接着讨论更进一步的异构：加一块独立 GPU。策略是保留这块耗电的 GPU，但只在需要图形性能时才启用。

他用 2011 年 15 寸 MacBook Pro 举例：四核 Intel Core i7（内含集成 GPU）加一块 AMD Radeon 独立 GPU。因此典型配置是：一块轻度的集成 GPU 处理日常图形，加一块重量级 GPU 供需要时使用，中间由某种总线连接。

[515] 视频画面时间区间：00:27:00–00:27:15。若为裁剪图，时间区间仍对应原视频画面。

[516] 视频画面时间区间：00:28:00–00:28:15。若为裁剪图，时间区间仍对应原视频画面。

多档加速器是功耗管理的具体手段

保留高性能但耗电的单元，仅在需要时启用，本质上是把”平均功耗”与”峰值性能”解耦。这与第 10 讲提到的手机芯片峰值 5 瓦、典型 100 到 200 毫瓦是同一逻辑：平时用低功耗单元，需要时短暂启用高功耗单元。

本章小结

真实系统普遍异构：移动 SoC 含常规核、专用 GPU 与媒体引擎；Skylake 让 CPU 核与图形核共享最后一级缓存；更高端配置再加一块独立 GPU 并按需启用。片上集成的关键优势是共享缓存带来的高带宽，避免了外接方案的数据搬运瓶颈。

23.6 超算与专用化

本章小结

真实系统普遍异构：移动 SoC 含常规核、专用 GPU 与媒体引擎；Skylake 让 CPU 核与图形核共享最后一级缓存；更高端配置再加一块独立 GPU 并按需启用。片上集成的关键优势是共享缓存带来的高带宽，避免了外接方案的数据搬运瓶颈。多档加速器本质上是把平均功耗与峰值性能解耦。

把视野放大到超算，异构还有另一层含义：基准如何定义性能。

23.7 超算排名与专用化

单一基准的问题

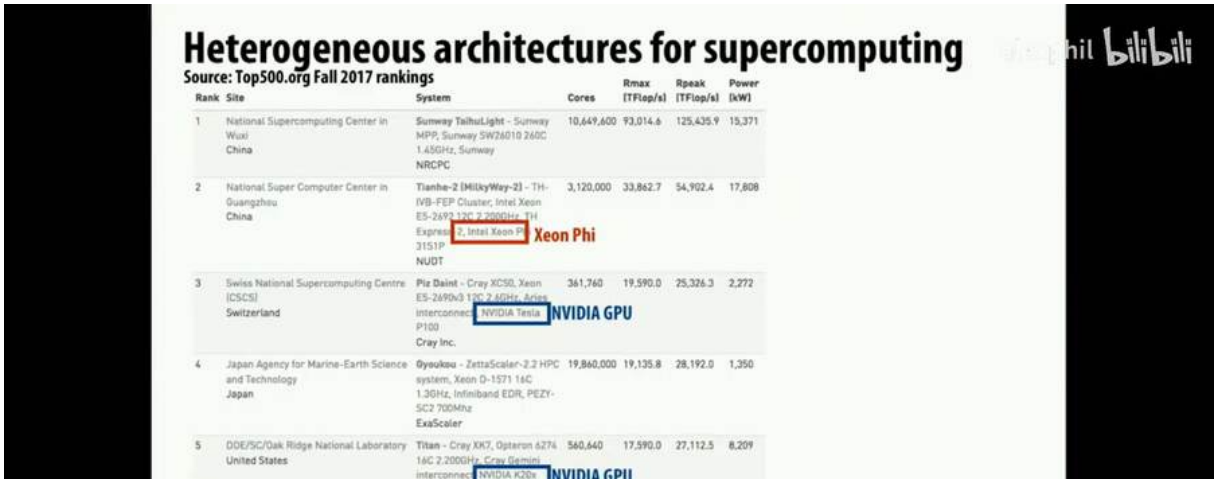

图 517: 超算的异构架构：Top500 排名数据[517]

[517] 视频画面时间区间：00:38:00–00:38:15。若为裁剪图，时间区间仍对应原视频画面。

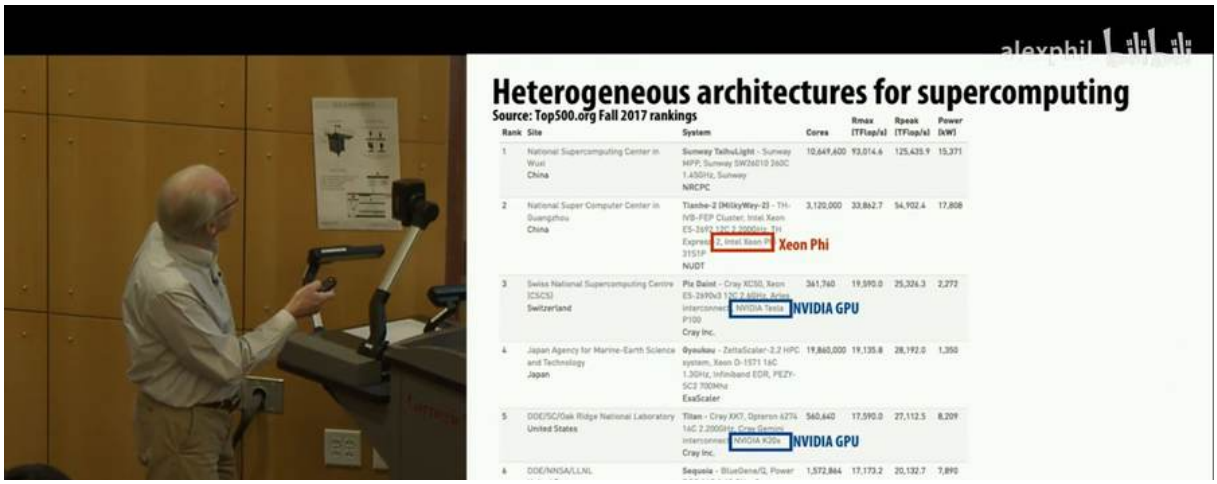

图 518: 以 Titan 为参照比较异构超算系统[518]

讲者对 Top500 排名提出了一个尖锐的批评，值得记录。

排名只跑一个基准：高斯消元，也就是用一个有 300 年历史的算法做矩阵求逆。这不能反映真实应用——即便是数值应用也会用更聪明的算法，通常处理稀疏矩阵，而这个是稠密矩阵。

因此这个基准把大量权重放在原始浮点能力上，而很少反映可编程性、互连网络质量这些实际决定应用性能的因素。

讲者还补充了一个现实原因：做这件事的人会花整整一年甚至更久、用整个团队去调优这一个基准。而如果调十个基准，工作量就是十倍，所以做基准的人本身也不愿意扩展。

另一个原因是要锁定整台机器：这种规模的机器通常不会只跑一个应用，而是被分成几十份。但做这些测试需要完全不受干扰的访问权。因此部分原因是后勤问题。

单一基准会系统性偏向某一类架构

只用稠密浮点基准排名，会高估那些为稠密矩阵优化的架构，低估那些强在可编程性、互连或数据局部性的架构。因此排名可作为参考，但不能当作”通用性能”的度量。评价系统必须结合目标应用的通信与访存特征。

[518] 视频画面时间区间：00:40:00–00:40:15。若为裁剪图，时间区间仍对应原视频画面。

专用化：走向硬件

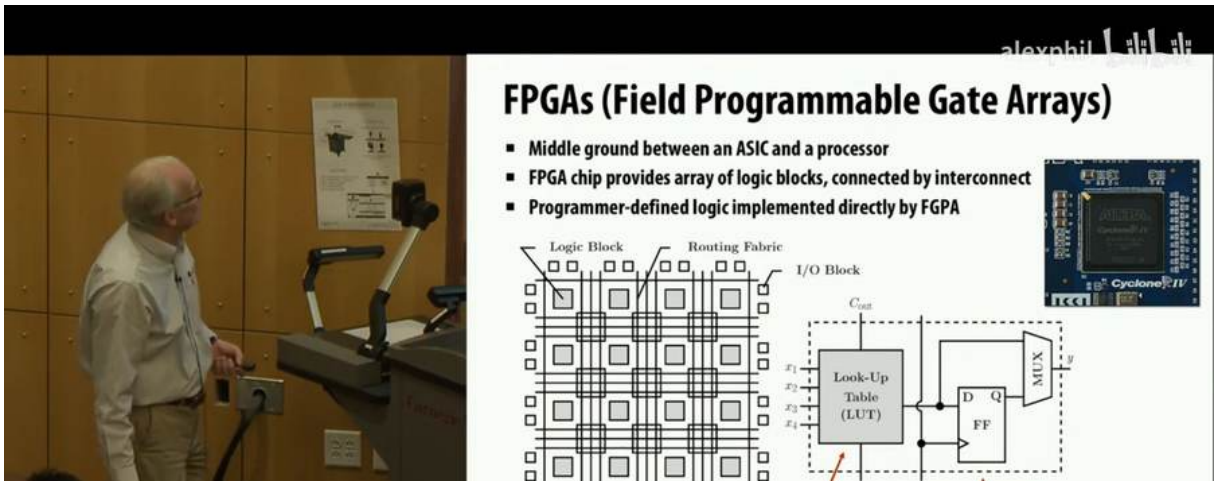

图 519: FPGA：介于 ASIC 与处理器之间的中间地带[519]

讲者最后讨论另一个维度：从纯软件走向更接近硬件，甚至直接做硬件设计。

他提到 FPGA 处在一个中间地带：FPGA 芯片提供一片可编程逻辑阵列，因此既能获得接近硬件的能效，又保留了一定的可重配置性，不必像 ASIC 那样一次性流片。

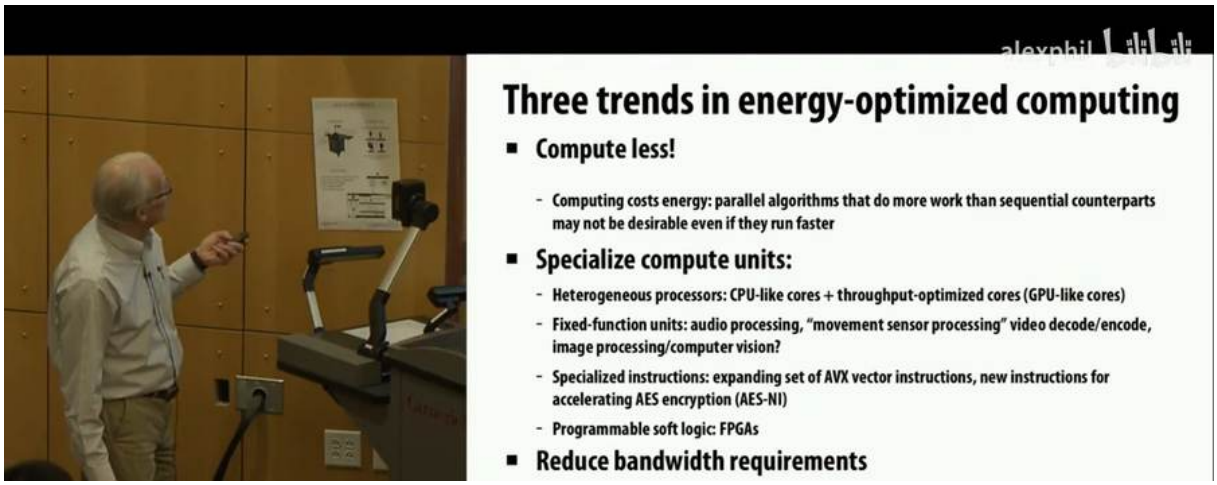

图 520: 能效优化的三条趋势：减少计算量、并行算法、专用化[520]

本章小结

Top500 只用高斯消元一个基准，系统性地偏向稠密浮点能力，忽略可编程性与互连质量，且调优成本与后勤约束使这一状况难以改变。专用化是另一条路径，FPGA 提供了介于 ASIC 与通用处理器之间的中间地带。

[519] 视频画面时间区间：01:10:00–01:10:15。若为裁剪图，时间区间仍对应原视频画面。

[520] 视频画面时间区间：01:19:00–01:19:15。若为裁剪图，时间区间仍对应原视频画面。

23.8 总结与延伸

讲者在结尾说这是一个有大量工作在进行、并且随时间不断变化的领域，是整个系统中非常活跃的一部分。今天的系统原理与十年前差别很大，部分原因正是这些功耗问题。

把全讲串起来，这是一条从” 功耗约束” 到” 异构设计” 再到” 真实系统” 的路径。功耗是从手机到超算的共同主导约束，它把性能问题重新定义为每瓦性能问题；同时现有硬件的相当部分在多数程序中被闲置。在固定面积模型下，胖核与瘦核的取舍由串行比例决定：几乎全并行时小核最优，串行比例升高后需要更大的核，而收益递减使同构方案在两端都不理想。异构方案把串行部分映射到胖核、并行部分映射到瘦核，结果是劣化更平缓、对串行比例更鲁棒，代价是一块面积被专用化。但现实限制有三：映射假设过于乐观、负载难以提前刻画、芯片成本要求足够销量。真实系统普遍异构：移动 SoC 含常规核、专用 GPU 与媒体引擎；Skylake 让 CPU 核与图形核共享最后一级缓存；更高端配置再加独立 GPU 并按需启用，片上集成的关键优势是避免外接方案的数据搬运瓶颈。最后，Top500 的单一基准会系统性偏向稠密浮点能力，而专用化（如 FPGA）是另一条提升能效的路径。

本讲义在讲者材料之上做的压缩，有四点需要与讲者原话区分：

- 异构的价值不是” 更多算力”，而是” 对负载变化的鲁棒性”。同构设计在已知负载下可以做到最优，但对负载比例极其敏感。异构用一小块专用面积换取跨负载的稳定表现，这是稳健性交易而非峰值交易。
- 收益递减是整套论证的关键前提。若单核性能能随面积线性增长，把所有面积给一个核就是最优，异构便没有意义。因此评估异构是否值得，首先要问单核性能的实际面积缩放曲线。
- 片上与外接的差别主要是带宽而非算力。外接加速器算力可能更强，但数据搬运受外部总线限制。因此” 计算量相对数据量越大” 的任务越适合外接，” 数据量大而计算相对轻” 的任务越适合片上。
- 基准的选择本身就是一种架构主张。只用稠密浮点基准排名，等于宣布这类能力最重要。因此读排名时必须同时问：这个基准与我的应用有多相似。

这堂课的边界在于它是导论性讨论：讲者给出的是权衡模型与真实系统的观察，未深入具体加速器的编程模型与调度机制。

两个可以直接从本讲数据提出的下一步问题：其一，异构方案依赖” 串行部分能映射到胖核” 这一假设，那么当串行部分本身无法被识别或分离时，异构的收益还剩多少？其二，片上共享缓存避免了数据搬运，但也意味着 CPU 与 GPU 竞争同一份缓存容量，这个竞争在什么工作负载下会抵消共享带来的收益？

Lecture 24 Domain-Specific Programming Languages

视频时长：1 小时 20 分 1 秒　来源：`https://www.bilibili.com/video/BV16N4y197cm/`

24.1 普通程序远未用尽硬件

这一讲讨论领域专用语言。讲者的切入方式不是先讲语言设计，而是先说明一个观察：普通程序在普通机器上运行时，远远没有用尽可用的硬件能力。

基线是什么

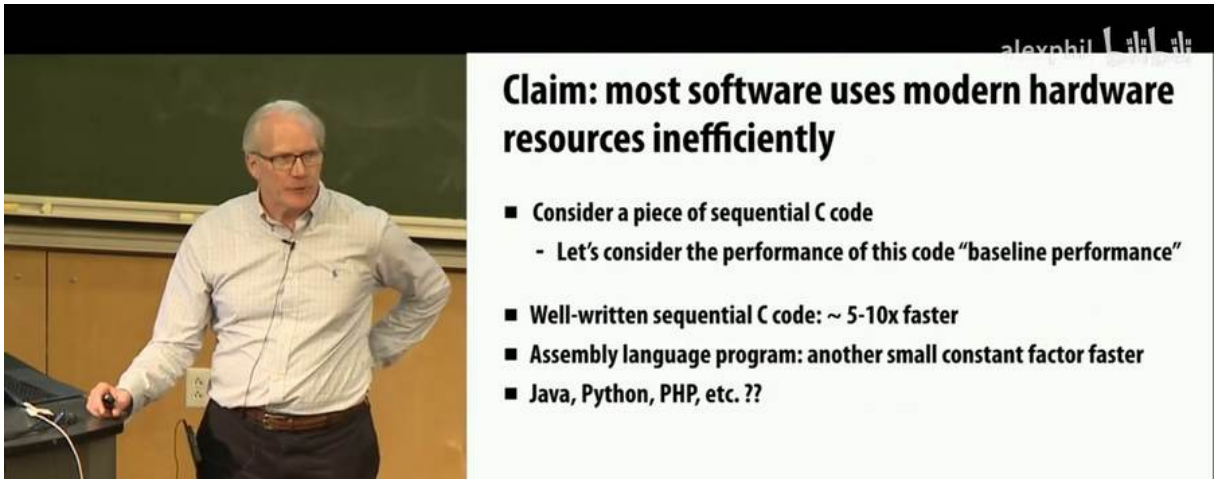

图 521：论点：多数软件对现代硬件资源的使用效率很低[521]

讲者先明确基线：这里默认假设你用 C 或 C++ 编写，因此你已经处于编程层面上相当低的位置，有能力去压性能。

在这个前提下，传统智慧是：一段写得不错的串行程序，若有人坐下来认真调优、用好编译器而不走极端，可以获得大约 5 到 10 倍的性能。

[521] 视频画面时间区间：00:25:00–00:25:15。若为裁剪图，时间区间仍对应原视频画面。

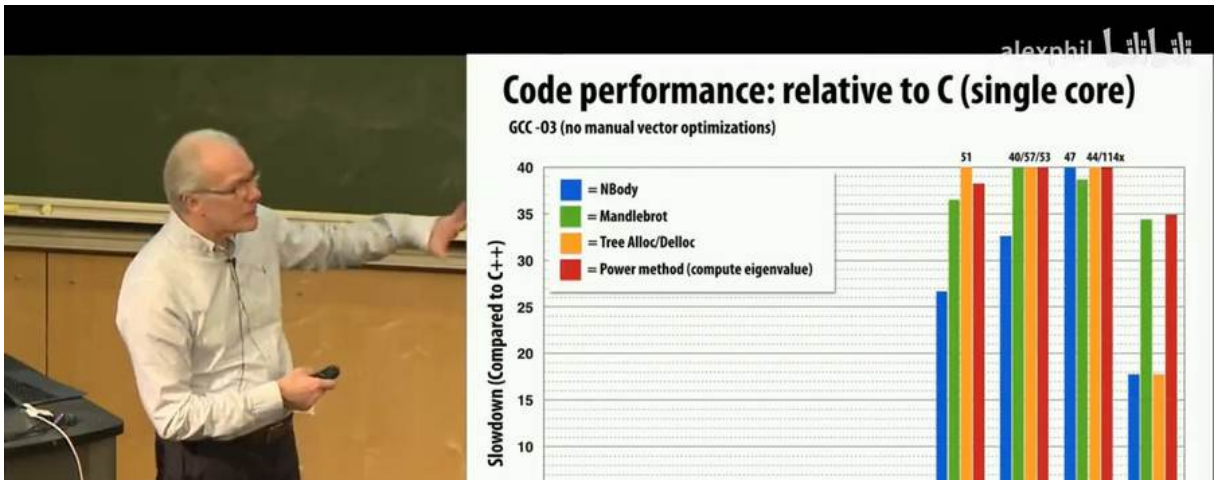

图 522: 各语言相对 C（单核，GCC -O3，无手工向量优化）的性能对比[522]

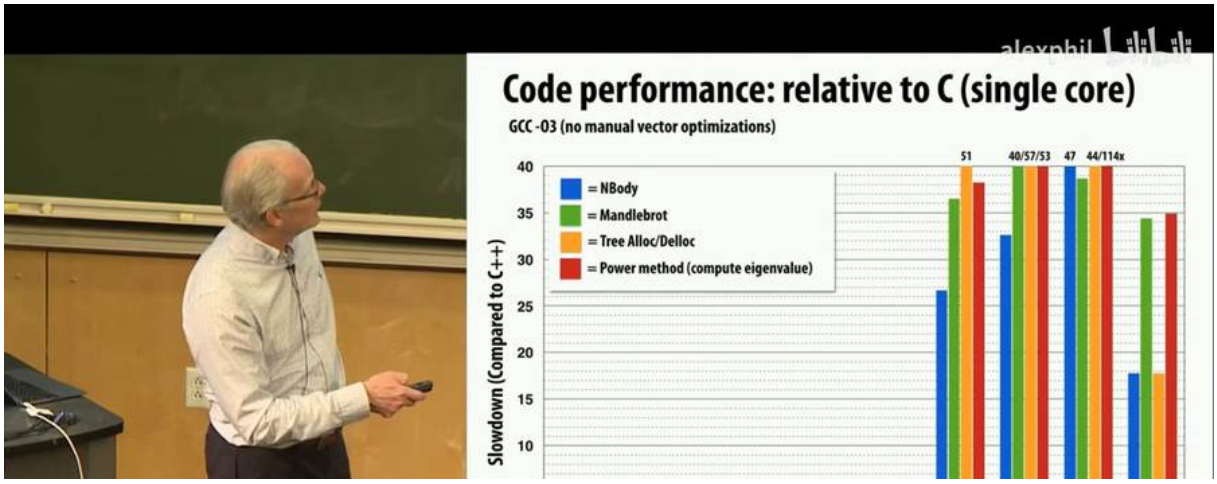

图 523: 性能对比续：纯解释型语言通常慢得多[523]

其他语言留下了多少性能

讲者接着问一个更有意思的问题：世界上大量程序是用别的语言写的，那些语言把多少性能留在了桌上?

他的答案是：很多。

他展示了一张对比图。基准是同一段程序用 C 实现、用相当常规的编译选项 GCC -O3（没有手工向量优化），其性能记为 1。其余各项表示相对 C 慢多少。

图中列出 Java、Scala（构建在 Java 之上）、C#（Windows 上的托管版 C）、Haskell（函数式语言）、Go（另一种托管语言）。讲者特别提到 Go 在这里跑得异常慢，而 JavaScript 的表现意外地相当不错。

而像 Python 这类纯解释型语言，通常要慢得多。

若把 C 实现记为基准 1，则其他语言可表示为相对倍数：

[522] 视频画面时间区间：00:26:00–00:26:15。若为裁剪图，时间区间仍对应原视频画面。

[523] 视频画面时间区间：00:27:00–00:27:15。若为裁剪图，时间区间仍对应原视频画面。

$$\text{speed}_{\text{lang}} = \frac{T_{\text{C}}}{T_{\text{lang}}}$$

- T_{C}：同一算法用 C 加常规优化编译后的耗时；
- T_{lang}：同一算法用目标语言实现的耗时；
- 该比值小于 1 表示比 C 慢，且往往小一个数量级甚至更多。

这个比值在写第一行代码时就基本确定，后续优化只能在其基础上改善。

语言选择本身就是一次性能决策

同一算法在不同语言下的性能差距可达数十倍，而这个差距在写代码的第一天就已确定。因此”用什么语言实现”不是风格问题，而是性能预算的分配问题。这也解释了为什么数值密集的核心循环常被下沉到 C 或 Fortran，而外围逻辑用高层语言。

本章小结

基线是写好的串行 C 代码；在此基础上认真调优可获得 5 到 10 倍。而其他语言相对 C 的性能落差很大：托管语言与函数式语言各有损耗，纯解释型语言慢得多。因此语言选择本身就是性能决策，差距在写第一行代码时就已确定。

24.2 即使好代码也没用尽机器

并行带来的额外收益

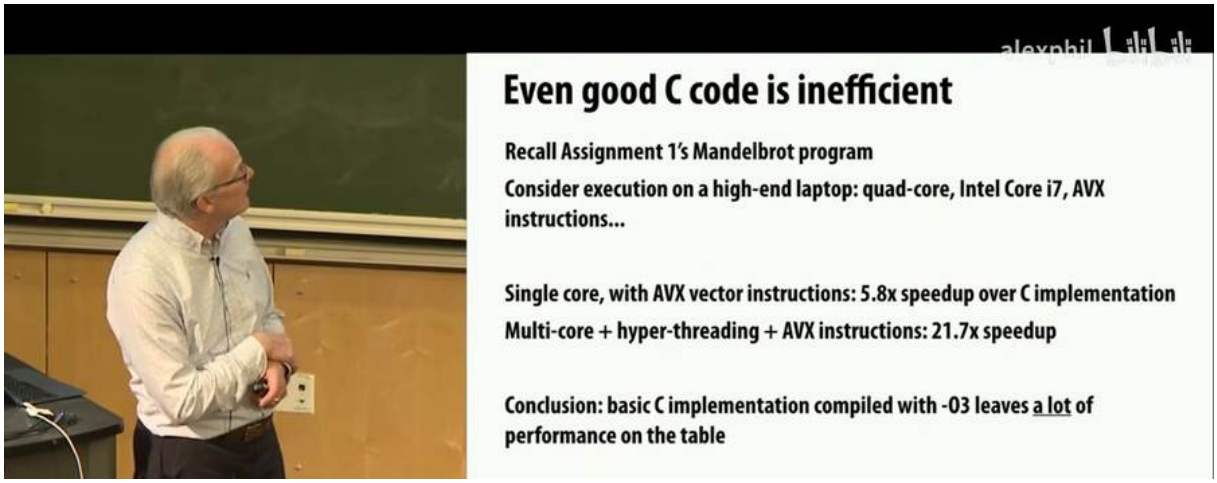

图 524：即使好的 C 代码也效率不高：作业一的 Mandelbrot 程序[524]

讲者用课程作业一的 Mandelbrot 程序说明。通过利用超线程与多核，他们在 Gates 机器上获得了大约 20 倍的加速。

他补充说，即使在一台高端四核笔记本上，也能测到 20 倍。而这只是更好地利用了那台笔记本内置的资源，并不是把它映射到什么特别大的机器上。

所以他强调：相对于”写好的串行 C 代码”这个基线，同一台机器上仍有大量性能可以挖出来。

[524] 视频画面时间区间：00:27:30–00:27:45。若为裁剪图，时间区间仍对应原视频画面。

这两类收益可以分解相乘：

$$S_{\text{total}} = S_{\text{tune}} \times S_{\text{parallel}}$$

- S_{tune}：对串行代码本身调优的收益，典型为 5 到 10 倍；
- S_{parallel}：把已有并行资源用起来的收益，本例约 20 倍；
- 两者作用于不同层面，因此可以叠加而非互相替代。

$$\frac{T_{\text{naive C}}}{T_{\text{tuned parallel}}} \approx 20$$

- 分子：常规编译的串行 C 实现；
- 分母：用上 SIMD、多线程与超线程后的实现；
- 这个 20 倍并非来自更大的机器，而是来自把已有资源用起来。

但做起来比想起来难

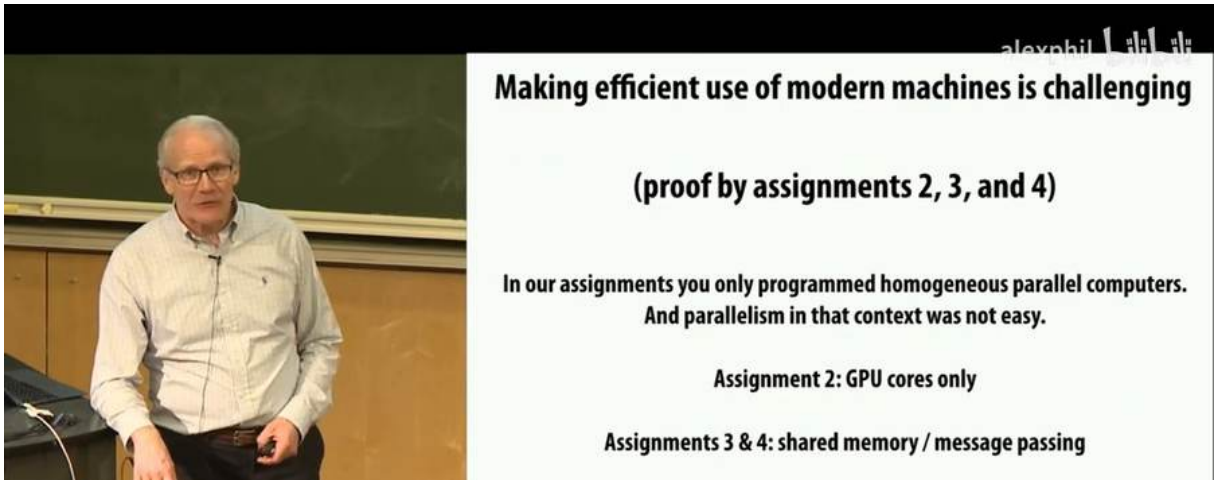

图 525: 高效利用现代机器很有挑战：作业二、三、四就是证明[525]

讲者说这是并行计算里一个特别好的案例。但他随即给出一个诚实的限定：通过他们的各项作业，我们发现实际做起来比想起来要难得多。

把一段程序改造成真正能利用并行机能力的形式，需要大量工作。

[525] 视频画面时间区间：00:28:30–00:28:45。若为裁剪图，时间区间仍对应原视频画面。

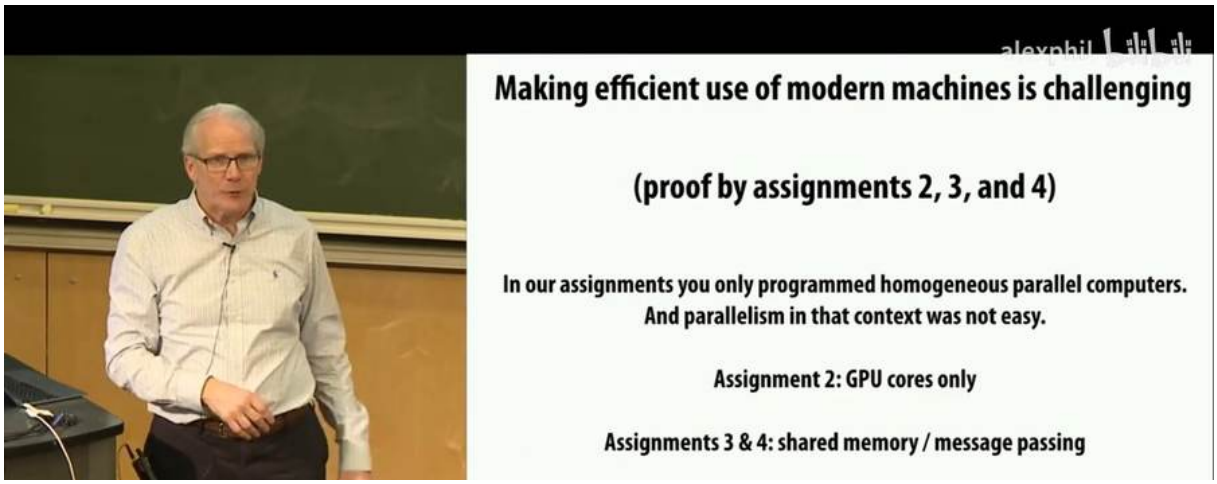

图 526: 不同并行模型几乎是不同的思维方式[526]

更难的是：映射到 GPU 与映射到 CPU，最终会形成很不同的代码风格。即使在 CPU 上，基于共享内存模型与基于消息传递模型，写法也差别很大。

所以从软件视角看，在这些机器之间移动时，你几乎需要不同的思维方式来推理程序。

”同一个算法”在不同并行模型下不是同一份代码

算法描述的是计算，而并行模型决定了如何组织数据、如何同步、如何划分工作。因此移植不是”改几个 API”，而是重写程序结构。这正是”生产力”问题在并行领域被放大的原因。

本章小结

即使好的串行 C 代码也没用尽机器：利用超线程与多核可获得约 20 倍加速，而且这在四核笔记本上就能测到，并非需要大机器。但做起来比想起来难得多：把程序改造成能利用并行机能力需要大量工作，而 GPU 与 CPU、共享内存与消息传递各自要求不同的思维方式。

[526] 视频画面时间区间：00:29:00–00:29:15。若为裁剪图，时间区间仍对应原视频画面。

24.3 多个层次与多个模型

机器内部的层次

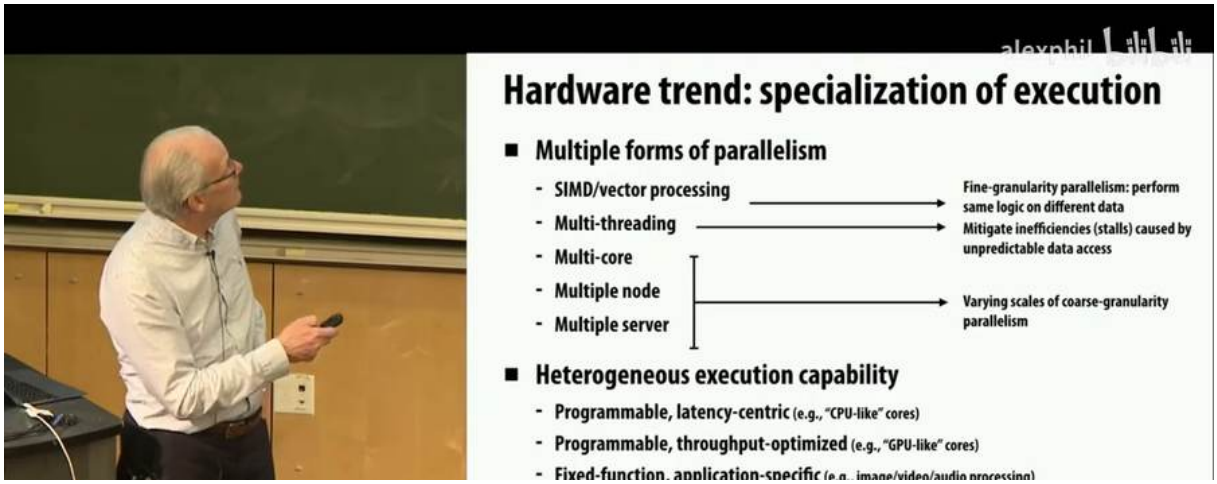

图 527: 硬件趋势：执行单元的专门化与多种并行形式[527]

讲者列举机器内部可挖掘的层次。

线程级，甚至超线程级：利用多个功能单元的未用潜力，隐藏因缓存缺失造成的访存延迟。

多核，甚至多服务器：上升到任务级或更粗粒度的并行，思考程序之间如何通信与交互。

所以即使在常规 CPU 内部，就已经有这么多不同层次可以挖掘。而加入各种异构的、某种程度上专门化的单元之后，这件事变得更棘手。

必须同时思考多个层次

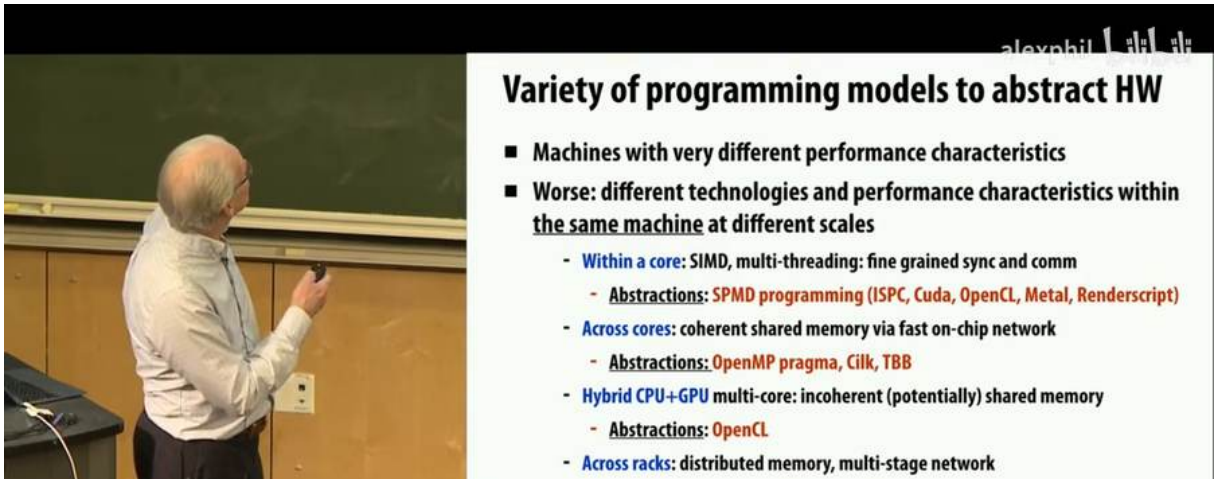

图 528: 为不同硬件提供多种编程模型：性能特征差异很大[528]

讲者的结论是：要真正拿到全部性能，你必须在多个层次上同时思考。

这对软件开发者与维护者已经是个问题，而且在变得更糟。

[527] 视频画面时间区间：00:30:00–00:30:15。若为裁剪图，时间区间仍对应原视频画面。

[528] 视频画面时间区间：00:31:30–00:31:45。若为裁剪图，时间区间仍对应原视频画面。

他还指出：若思考每个层次上最适合的编程模型，会发现它们各不相同。比如 SIMD 单元与 GPU 适合单程序多数据风格。

层次越多，”统一抽象”的代价越高

若为每一层各写一套代码，维护成本随层数线性增长；若强行用一套抽象覆盖所有层，抽象就必须足够通用，从而失去各层的优化机会。领域专用语言正是选择后一条路，但把优化交给编译器——它的成败取决于编译器能否在该领域内做对。

这些层次之间还有耦合，使问题进一步复杂。比如用多线程隐藏访存延迟时，线程数增加会加剧缓存争用；而缓存争用又会反过来降低每个线程的有效带宽。因此在某一层上做出的选择，常常改变另一层的代价结构。这意味着「逐层独立优化」并不成立——必须把几层放在一起考虑，这也解释了为什么并行程序的调优往往比串行程序更需要全局视角。讲者提到，作业里同学们常见的失败模式正是在一层上优化、却让另一层退化。

本章小结

机器内部有多层可挖掘：线程级与超线程用于隐藏访存延迟，多核与多服务器可上升到任务级，加入异构专用单元后更棘手。要拿到全部性能必须在多个层次上同时思考，而每层适合的编程模型不同。这使统一抽象与逐层优化成为一对必须权衡的代价。

24.4 领域专用语言的策略

牺牲通用性

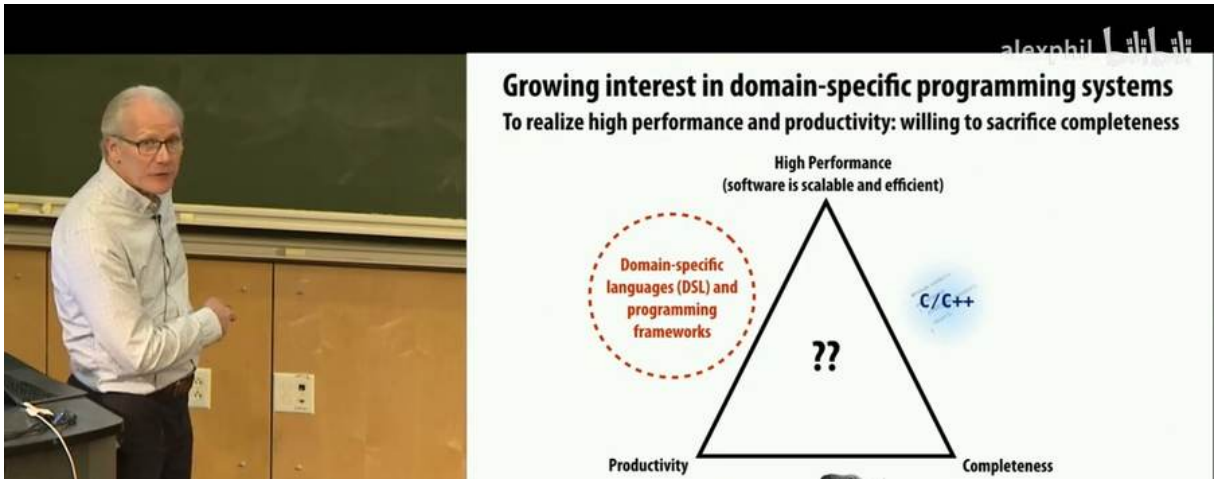

图 529: 领域专用编程系统的兴趣在增长：兼顾高性能与生产力[529]

讲者把领域专用语言的策略说得很直接：为了实现高性能与生产力的兼得，就要牺牲通用性。

也就是说，不追求一个能表达任何计算的通用语言，而是针对某一类问题设计语言，把该领域的知识编码进编译器与运行时。

[529] 视频画面时间区间：00:35:00–00:35:15。若为裁剪图，时间区间仍对应原视频画面。

领域专用语言的赌注是”领域内的规律足够稳定”

它假设该领域的计算模式足够固定，因此可以预先设计好优化。若领域内的需求快速变化，专用语言的优化就会迅速过时。因此这类语言最适合那些问题形态多年不变、而硬件不断变化的领域——把变化的负担从程序员转移到编译器。

这里有一个值得注意的取舍：领域专用语言把优化责任从程序员转移给编译器，但编译器只能优化它「理解」的模式。因此语言设计者必须预先决定哪些模式值得支持。支持得越多，编译器越难对每种情况都做到最优；支持得越少，语言的适用范围就越窄。这个张力决定了专用语言的成败——它必须在「足够覆盖领域」与「足够专注以优化」之间找到平衡点。

图作为抽象

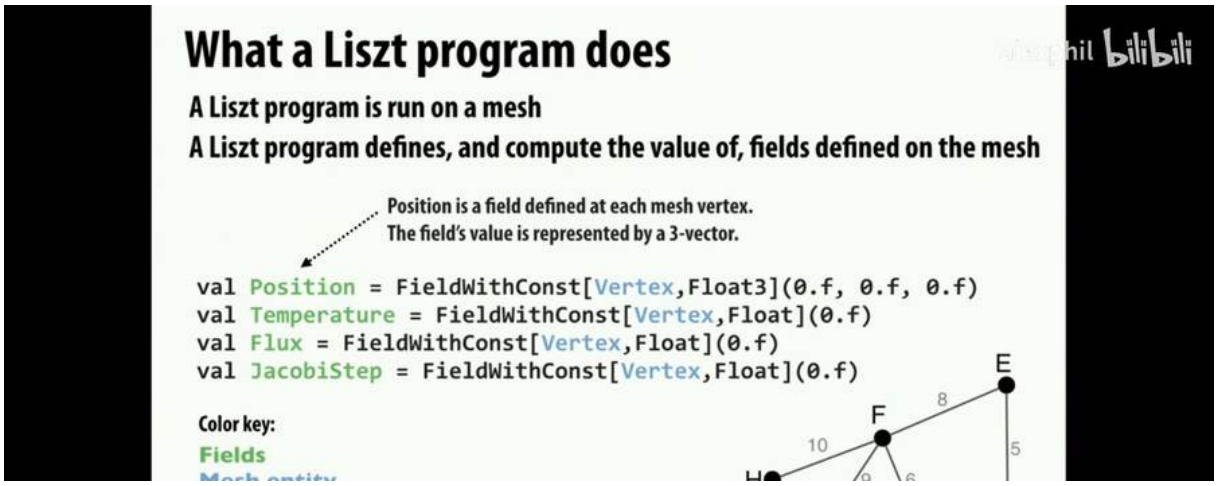

图 530: Liszt 程序在网格上运行，定义并计算面与顶点的量[530]

讲者举图为例：图本身就是一种自然的抽象，节点表示实体，边表示两个不同实体之间的关系。因此可以在这些不同的抽象层次上表达计算。

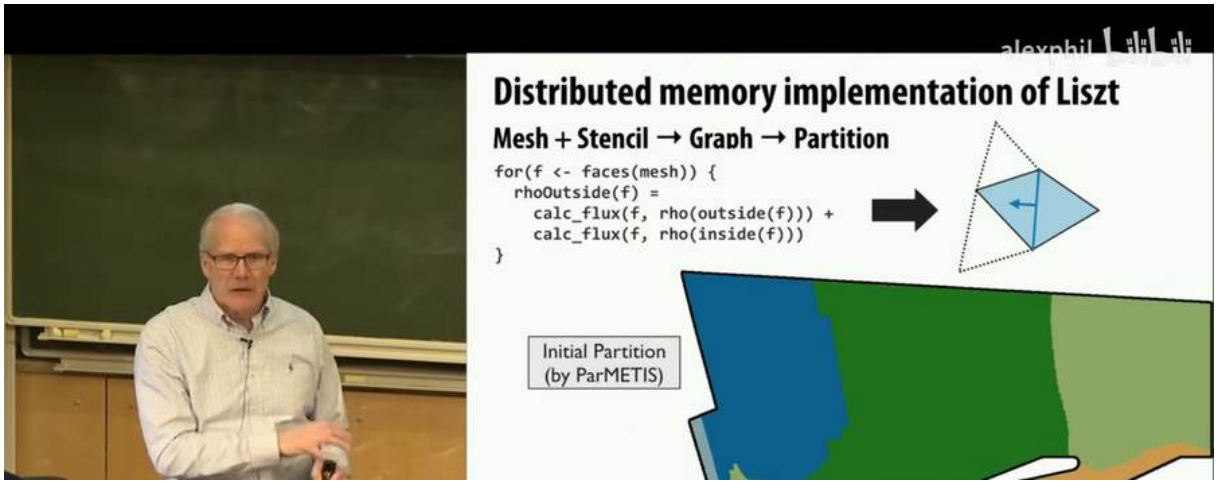

图 531: 网格通常也不是均匀的：Liszt 的分布式实现把网格转成图再分区[531]

[530] 视频画面时间区间：00:40:00–00:40:15。若为裁剪图，时间区间仍对应原视频画面。

他指出一个实际细节：网格通常也不是均匀的。因此这类语言需要处理不规则性，而这正是它比通用语言更有优势的地方——不规则性在领域内是已知的，可以被编译器预先考虑。

本章小结

领域专用语言的策略是牺牲通用性换取高性能与生产力：针对一类问题设计语言，把领域知识编码进编译器与运行时。图是自然的抽象（节点是实体、边是关系），而网格通常并不均匀，处理这种不规则性正是专用语言的优势所在。

24.5 两个具体项目

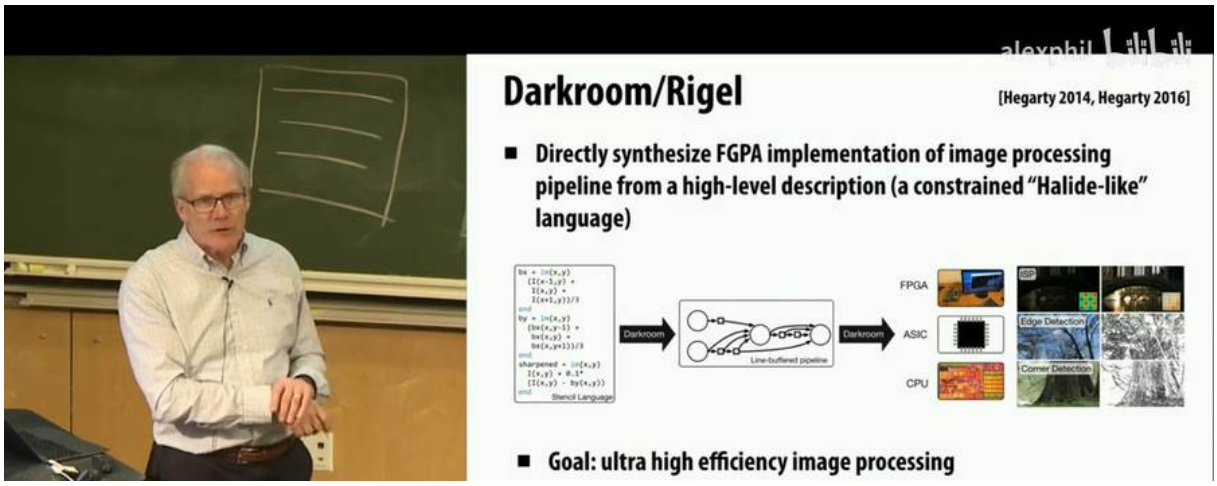

图 532: 两个具体项目：Darkroom/Rigel 直接合成 FPGA 实现[532]

讲者在结尾介绍了两个具体项目。其中之一是 Darkroom 与 Rigel：直接把图像处理流程合成为 FPGA 实现。

另一个的思路是把计算函数表达为图上的运算。讲者说这类项目并非独创，已有各种类似的工作，但这两个项目在”映射到我们所关心的计算机”这一点上特别有力。

专用化的终点是硬件生成

当领域语言足够受限时，编译器不仅能生成优化代码，还能直接生成硬件描述。这把这门语言的”目标机器”从处理器扩展到了 FPGA 或专用电路，从而绕过通用处理器的能效限制。这是第 23 讲异构与专用化思路的极端形式。

本章小结

两个具体项目展示了专用化的两个方向：Darkroom/Rigel 直接把图像处理流程合成为 FPGA 实现；另一个把计算函数表达为图上的运算。两者都在”映射到目标机器”这一点上特别有力。专用化的极端形式是直接生成硬件，从而绕过通用处理器的能效限制。

[531] 视频画面时间区间：00:50:00–00:50:15。若为裁剪图，时间区间仍对应原视频画面。

[532] 视频画面时间区间：01:19:00–01:19:15。若为裁剪图，时间区间仍对应原视频画面。

24.6 总结与延伸

讲者在结尾说，这类思路并非独创，已有各种类似工作，但把它们映射到我们所关心的计算机上这件事特别有价值。

把全讲串起来，这是一条从” 硬件远未被用尽” 到” 用领域专用语言改变这一状况” 的路径。基线是写好的串行 C 代码，在此基础上认真调优可获得 5 到 10 倍；而其他语言相对 C 的落差很大，纯解释型语言慢得多，因此语言选择本身就是性能决策。即使好的 C 代码也没用尽机器：利用超线程与多核可获得约 20 倍，而且这在四核笔记本上就能测到。但做起来比想起来难得多：把程序改造成能利用并行机能力需要大量工作，而 GPU 与 CPU、共享内存与消息传递各自要求不同的思维方式。机器内部有多层可挖掘，每层适合的编程模型不同，因此统一抽象与逐层优化是一对必须权衡的代价。领域专用语言的策略是牺牲通用性：针对一类问题设计语言，把领域知识编码进编译器与运行时。图是自然的抽象，而网格通常并不均匀，处理这种不规则性正是专用语言的优势。最后，两个具体项目展示了专用化的两个方向，其极端形式是直接生成硬件。

本讲义在讲者材料之上做的压缩，有四点需要与讲者原话区分：

- ”20 倍” 与”5 到 10 倍” 是两个不同性质的数字。前者来自把已有并行资源用起来，后者来自对串行代码本身的优化。它们可以叠加，而不是互相替代。
- 语言之间的性能落差在写第一行代码时就已确定，这一点常被低估。因此” 用高层语言写原型、把热点下沉到 C” 是常见策略，但下沉本身有接口与维护成本。
- 领域专用语言的赌注是领域规律足够稳定。若领域需求快速变化，预置的优化会迅速过时。因此它适合问题形态多年不变而硬件不断变化的场景。
- 专用化的终点可以是硬件生成。当领域足够受限时，编译器能直接输出硬件描述，从而绕过通用处理器的能效限制。这是异构思路的极端形式，也是专用语言与硬件设计的交汇点。

这堂课的边界在于它是动机与项目介绍：讲者用大量时间讨论项目要求与选题，DSL 部分给出的是策略与两个例子，未深入语言的具体设计或编译技术。

两个可以直接从本讲数据提出的下一步问题：其一，既然语言选择决定了数十倍的性能落差，那么在什么规模的程序中” 换语言重写” 比” 在原语言内优化” 更划算？其二，领域专用语言把优化责任转移给编译器，那么当编译器无法处理某个新模式时，程序员还有什么手段？

Lecture 25 Domain-Specific Programming on Graphs

视频时长：48 分 52 秒 来源：https://www.bilibili.com/video/BV1iN411i7pv/

25.1 领域专用系统：为什么值得为图单独做一套

上一讲讲的是领域专用语言，也就是为特定任务类设计的新编程语言。这一讲接着讨论。

两种形态

讲者先做了一个区分：领域专用语言是”发明一门新语言”，但更大的一类做法是提供领域专用的编程系统或软件包——它们给你同样的能力，但不必发明新语言。

今天要看的是其中为描述图计算而设计的一些系统。

图 533: 上次内容：领域专用编程系统的接受度在提升[533]

新语言与专用包是两种不同代价的路线

新语言表达力更强、能承载领域专有的优化，但要求用户学习并迁移工具链；专用包沿用宿主语言，学习成本低，但能做的优化受宿主语言表达能力限制。选择哪条路取决于领域复杂度与用户群的迁移意愿。

四维取舍

讲者给出一个反复出现的框架。他的核心判断是：不可能有一个通用系统同时给你高性能（尤其是在这些古怪的并行机器上）、完全通用、易于表达、以及易于编写。

因此专用化愿意在某几个维度上妥协，换取另一些。

[533]视频画面时间区间：00:02:30–00:02:45。若为裁剪图，时间区间仍对应原视频画面。

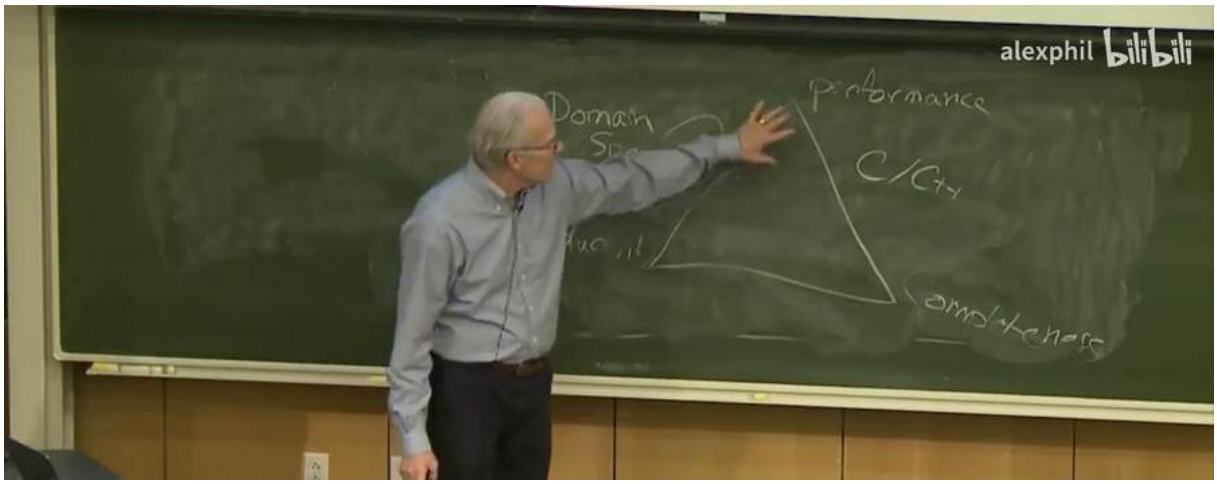

图 534: 领域：性能、完备性、通用性与生产力的四维取舍[534]

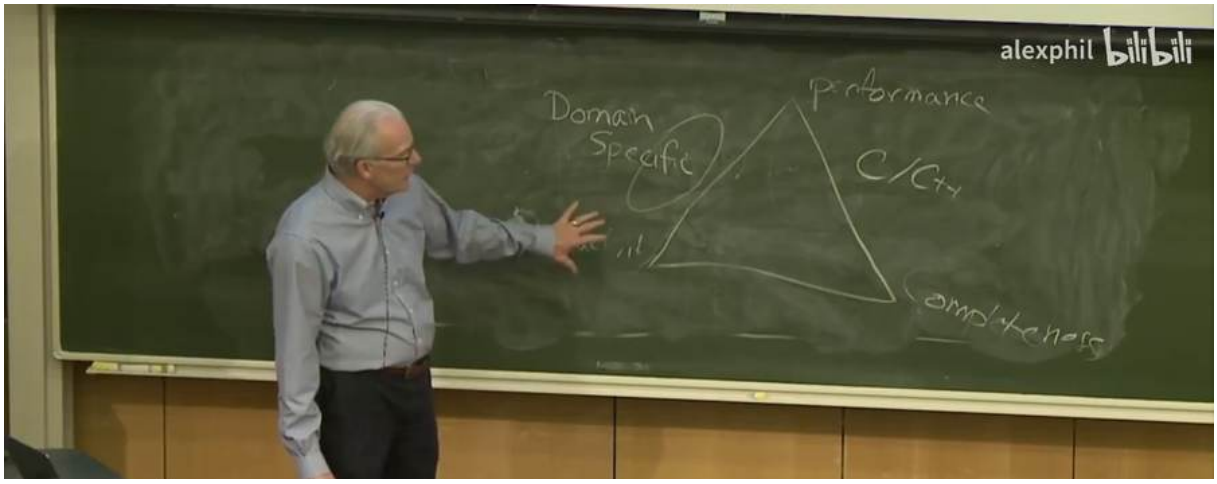

图 535: 领域专用系统在这四维空间中的位置[535]

他点明 C 或 C++ 的位置：语言非常完备，只要你肯下功夫就能拿到好性能，但生产力不高，因为大量底层工作要自己重写。而领域专用系统处在另一个区域。

[534] 视频画面时间区间：00:08:30–00:08:45。若为裁剪图，时间区间仍对应原视频画面。

[535] 视频画面时间区间：00:09:00–00:09:15。若为裁剪图，时间区间仍对应原视频画面。

图为什么适合专用化

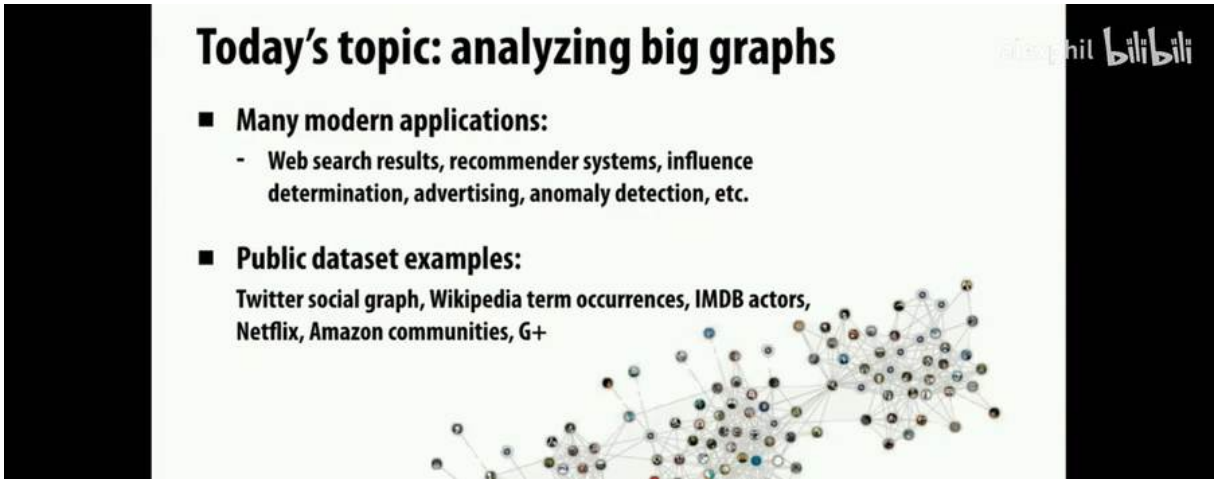

图 536: 今日主题：分析大图，现代应用的基础[536]

讲者给出图的几个典型应用。最著名的一个起点是 Google 由两位斯坦福研究生创立，他们观察到：若把网页看作节点、把链接看作引用，就能从图结构中推断哪些内容最有信息量，由此得到 PageRank。

同类分析还用于社交网络的行为模式：把 Facebook 好友、Twitter 关注等关系构成图，只看整体结构就能获得很多洞察。不少公司在此基础上建立了重要业务，而这个领域不只吸引计算机科学家，社会科学家也用它研究社群与群体。

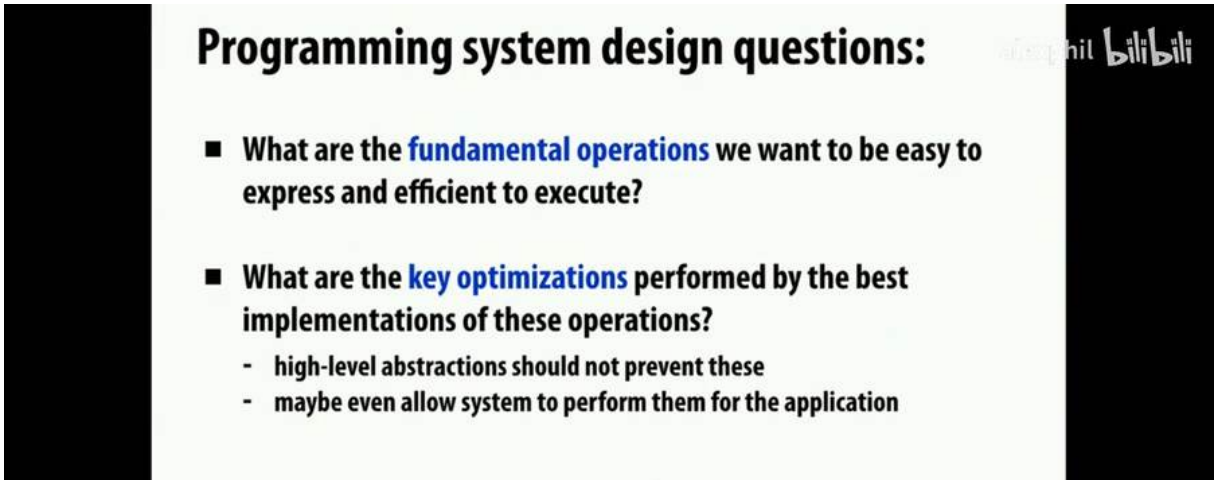

图 537: 编程系统设计问题：哪些基本操作应当易于表达[537]

挑战在于这些图可能极其巨大：节点或边可达百万级，极端情况下到 10 亿级。把它们映射到标准处理器上是困难的。而算法本身你不想反复重写。因此这是专用化能力的理想场景：专为图设计、能扩展到很大的图、且容易做不同类型的分析。

他提到斯坦福有一个叫 SNAP 的项目，主导者 Yuri Leskovec 在 CMU 做的博士研究就是可扩展图算法。

[536] 视频画面时间区间：00:05:00–00:05:15。若为裁剪图，时间区间仍对应原视频画面。
[537] 视频画面时间区间：00:11:00–00:11:15。若为裁剪图，时间区间仍对应原视频画面。

本章小结

领域专用系统有”新语言”与”专用包”两种形态。核心取舍在性能、完备性、通用性、易用性四维之间，C/C++ 位于”完备且可压性能但生产力低”的一角。图计算因规模巨大（十亿级）、映射困难、算法不应反复重写，成为专用化的理想场景。

下一章看这类系统如何设计才能既好用又不比手写慢。

25.2 设计难点：性能与可调性

抽象的基础

讲者指出一个前提：图本身已经是一种抽象，它把各种我们想关注的关系抽象掉了。因此可以在这个抽象之上继续搭建。

他想问的高层问题是：能在这个方向上走多远，通过提供该领域通用、但底层实现很费力的功能。

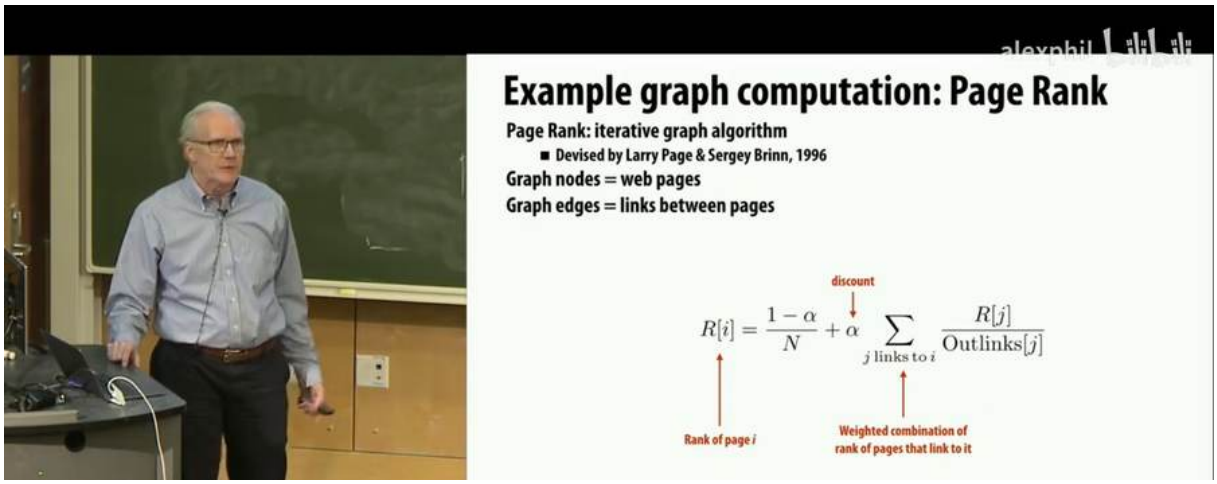

图 538: 示例图计算：PageRank 的迭代算法[538]

必须达到手写性能

讲者点出最关键的约束：专家们已经积累了相当多的知识，知道如何从这类算法中压出性能。

如果这些知识无法以某种方式被纳入系统，那么人们就不会真正采用它——因为如果他们重写代码能拿到更好的性能（哪怕很痛苦），至少会有一部分人这么做，而你就会失去潜在收益。

因此系统必须具备达到同等性能的能力。

专用系统的采用门槛是”不劣于手写”

一个专用系统即使表达力再强、开发再快，只要性能明显低于专家手写版本，就会被关键用户放弃。因此”易用”不是充分条件，必须同时满足”性能可比”。这也是这类系统最难的部分：既要抽象，又不能因抽象损失性能。

[538] 视频画面时间区间：00:11:30–00:11:45。若为裁剪图，时间区间仍对应原视频画面。

两条达成路径

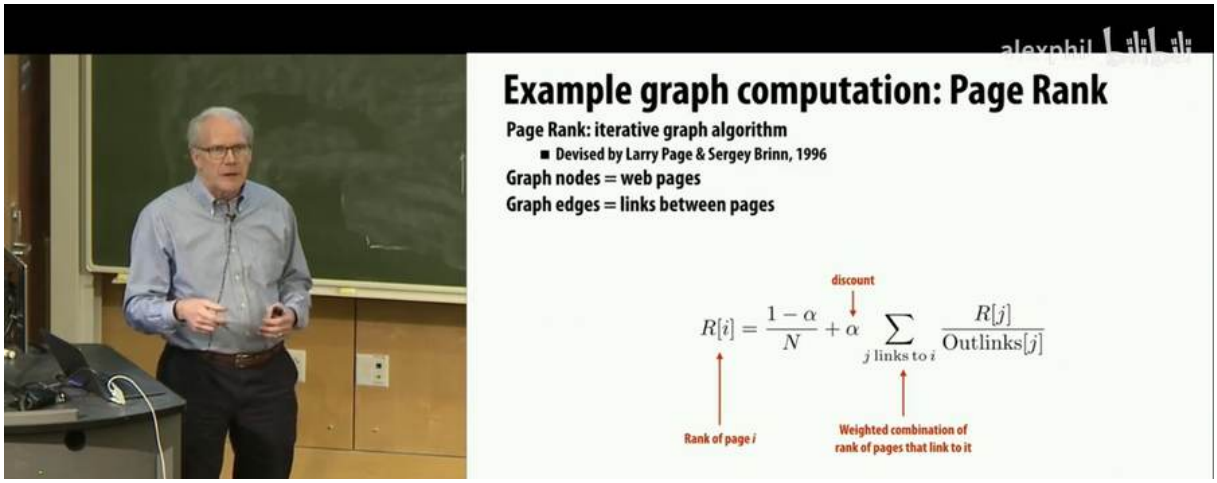

图 539: PageRank 迭代：按出链把权重分给邻居[539]

讲者给出两条路。其一，代码足够聪明，能自动完成优化。其二，给用户与开发者提供足够的钩子、足够的” 小杠杆”，让他们能推拉出那些手写时会做的优化。

他提到此前讨论 Halide 图像处理程序时见过这个模式：它有一个很有意思的特性，给你一门单独的语言来描述图像处理流水线应该如何分阶段。

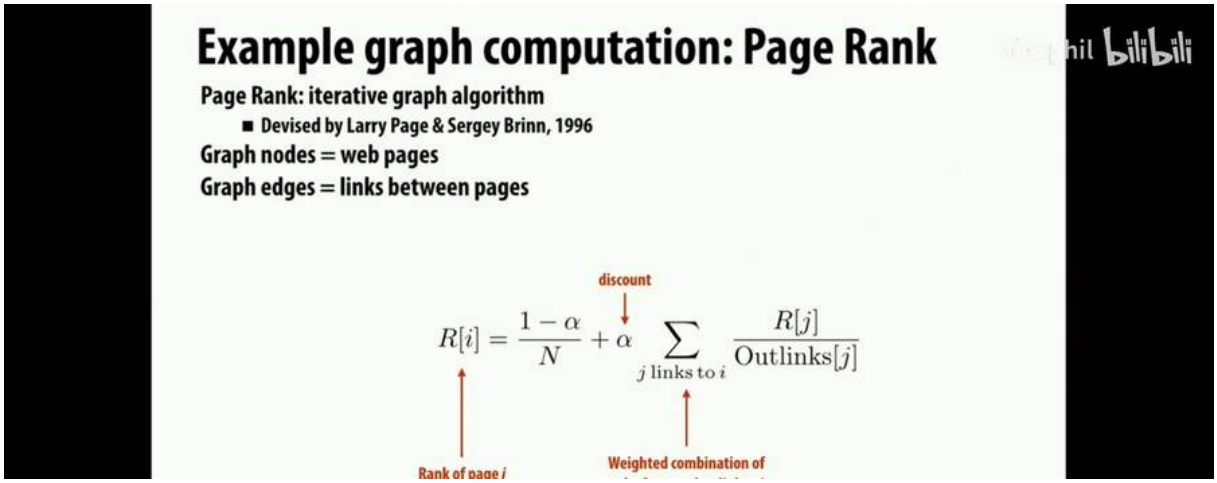

图 540: PageRank 继续迭代直至收敛[540]

[539] 视频画面时间区间：00:12:30–00:12:45。若为裁剪图，时间区间仍对应原视频画面。
[540] 视频画面时间区间：00:13:30–00:13:45。若为裁剪图，时间区间仍对应原视频画面。

双语言设计

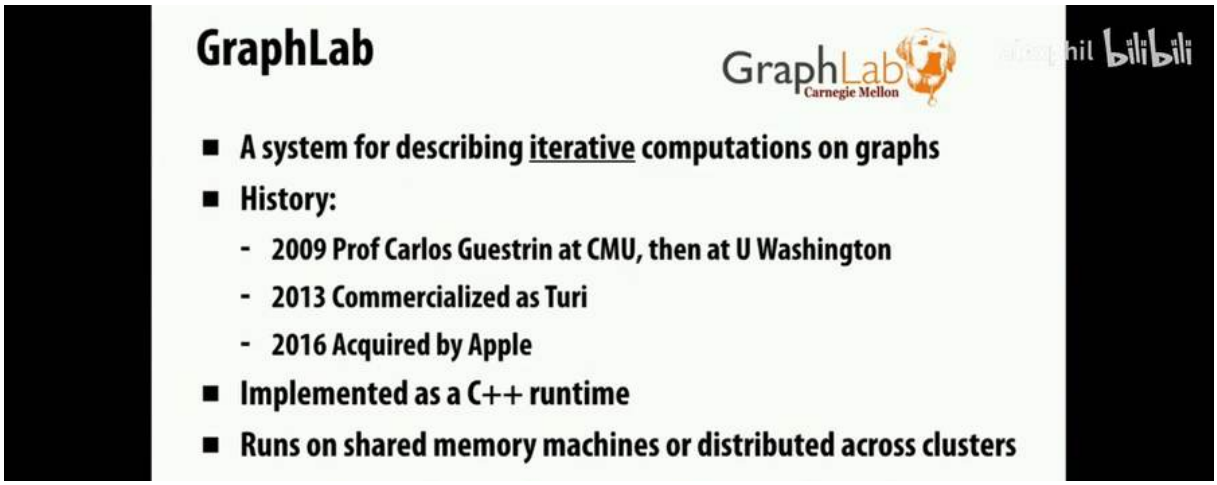

图 541：GraphLab：CMU 的图计算框架[541]

讲者把这个模式讲得更清楚：不必追求一个能自动搞定的完美程序，而是让用户能做调优，而不必从头重写一切。

他对比了两种风格。一种是高度自动化的系统：它自己做映射，做复杂的编译期分析。另一种像基础版 Halide：它本身很精巧，但把大量优化决策交给用户。

而这门课追求的是两者之间：既有表达力，又能让系统（通过人工指导或自动分析）把性能上的最佳做法纳入进来。

把” 做什么” 与” 怎么做” 分成两门语言

计算描述回答” 算什么”，调度描述回答” 怎么排布与并行”。分开的好处是：算法部分可以复用，而针对不同机器只需重写调度部分。这正是 Halide 与后续图计算系统的共同结构。

本章小结

设计难点是必须达到手写性能，否则系统不会被采用。两条路径是自动优化与提供可调钩子，Halide 展示了后者的典型形态：用单独语言描述流水线如何分阶段。这演化为” 计算描述 + 调度描述” 的双语言设计。

下面用 PageRank 与 GraphLab 具体看这个结构。

25.3 PageRank 与 GraphLab

PageRank 的直觉

讲者说 PageRank 由 Page 与 Brin 在 1996 年提出，算” 上古时代” 了。他们从斯坦福退学、没有毕业，之后创办了 Google（大约 1999 或 2000 年）。

他用一个通俗的类比解释算法直觉：某些人之所以是名人，是因为他们认识其他名人，于是自己也成了名人——他们本身没有内在价值，只因为认识了” 对的人”，人们就开始关注他们。

PageRank 大致就是这个意思：你的重要性由你认识谁决定。

其迭代式可以写成：

[541] 视频画面时间区间：00:16:00–00:16:15。若为裁剪图，时间区间仍对应原视频画面。

$$R[i] = \frac{1-\alpha}{N} + \alpha \sum_{j \to i} \frac{R[j]}{\text{outlinks}[j]}$$

- $R[i]$：节点 i 的排名分数；
- N：节点总数，$\frac{1-\alpha}{N}$ 是随机跳转项；
- α：阻尼系数，通常取 0.85 左右；
- outlinks$[j]$：节点 j 的出链数，用于把权重平均分给邻居。

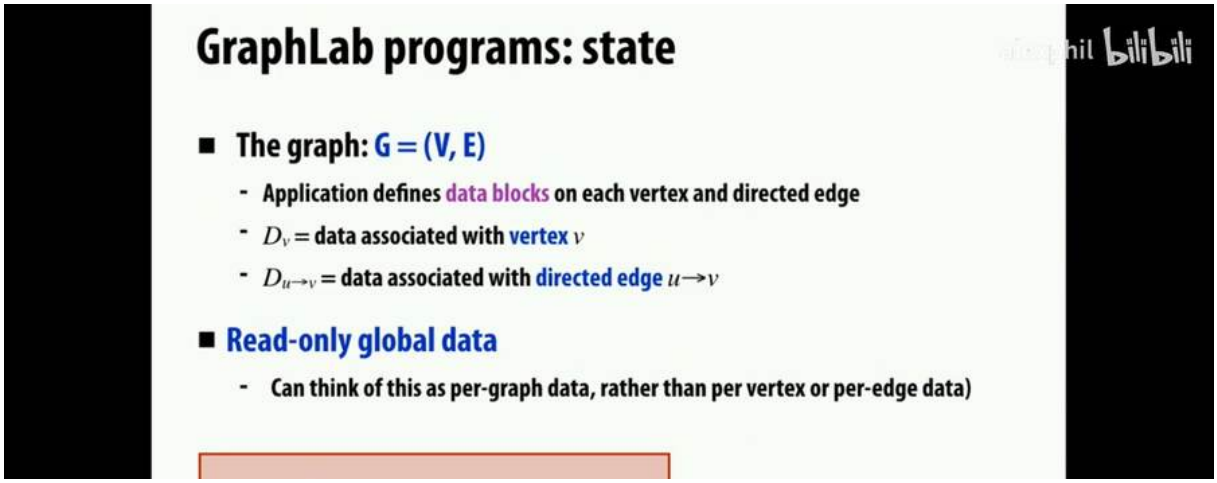

图 542: GraphLab 程序的状态：图与顶点上的数据块[542]

GraphLab 的结构

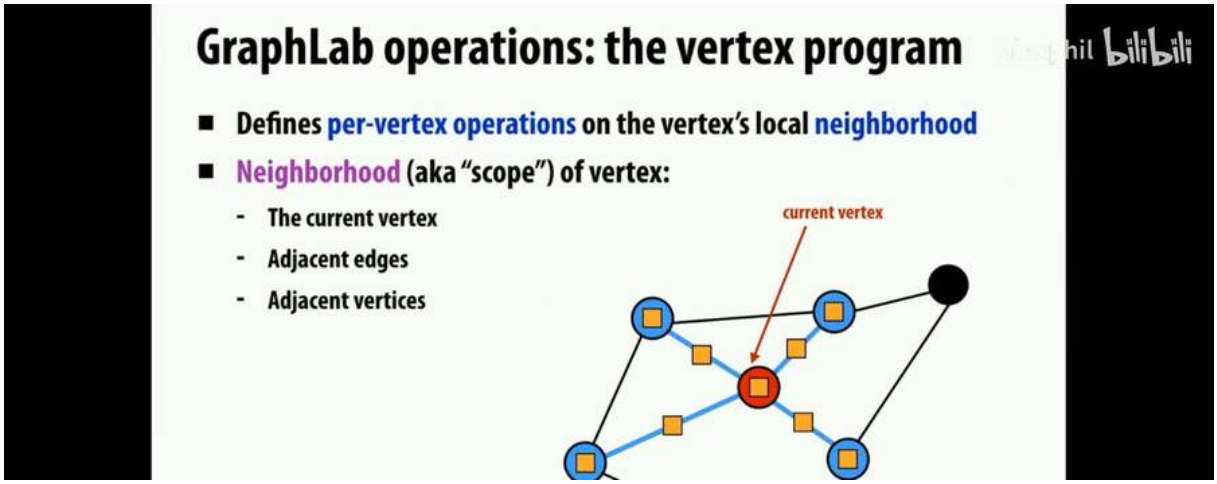

图 543: GraphLab 操作：顶点程序定义每个顶点在其局部邻域上的计算[543]

GraphLab 是为描述图上的迭代计算而设计的系统。应用在每个顶点上定义数据块，然后提供一个顶点程序——它定义每个顶点在其局部邻域上的操作。

[542] 视频画面时间区间：00:17:30–00:17:45。若为裁剪图，时间区间仍对应原视频画面。
[543] 视频画面时间区间：00:18:00–00:18:15。若为裁剪图，时间区间仍对应原视频画面。

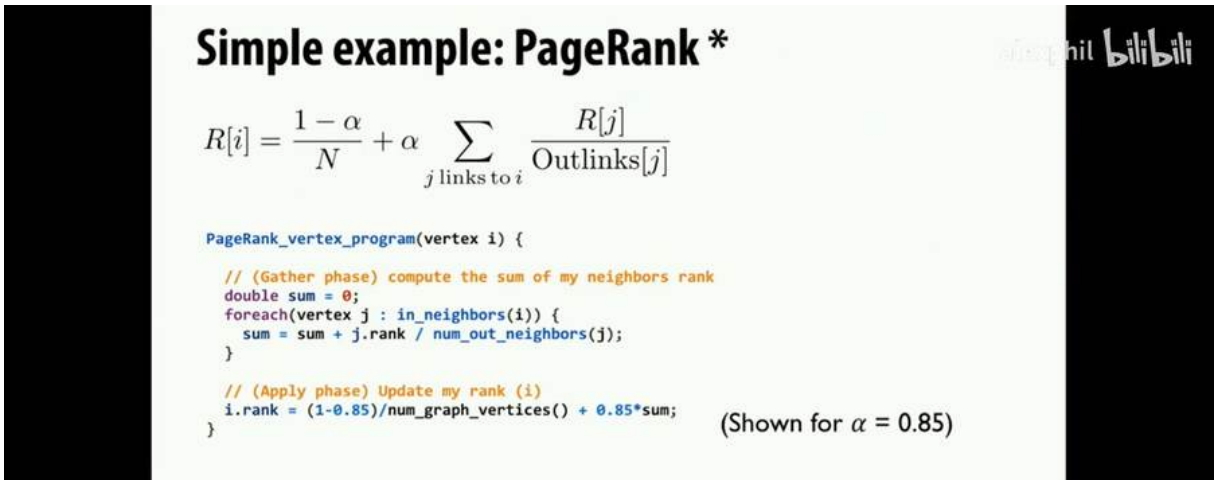

图 544: 简单示例：用顶点程序写 PageRank[544]

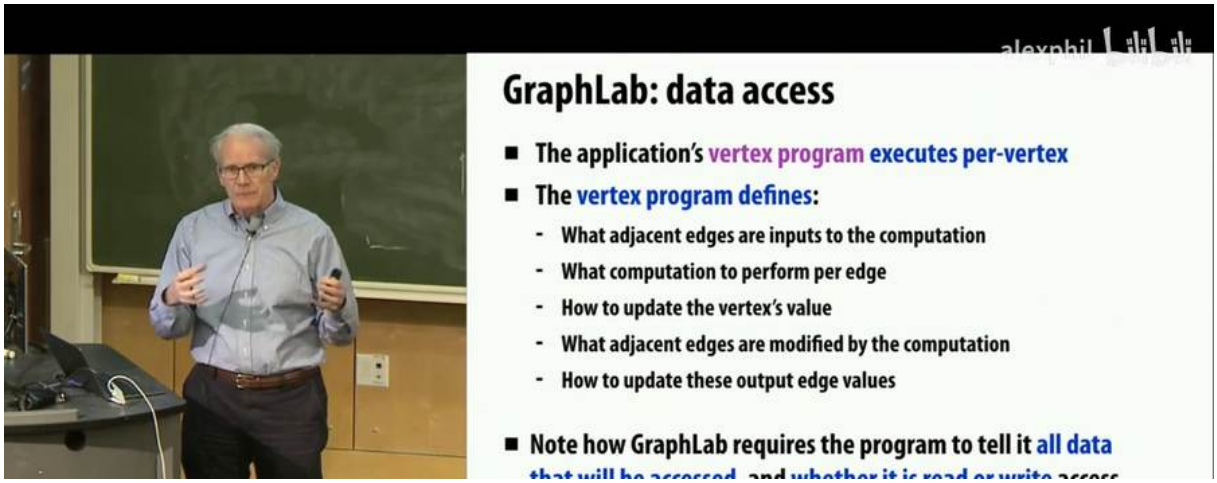

图 545: GraphLab 的数据访问：顶点程序可读写其邻域[545]

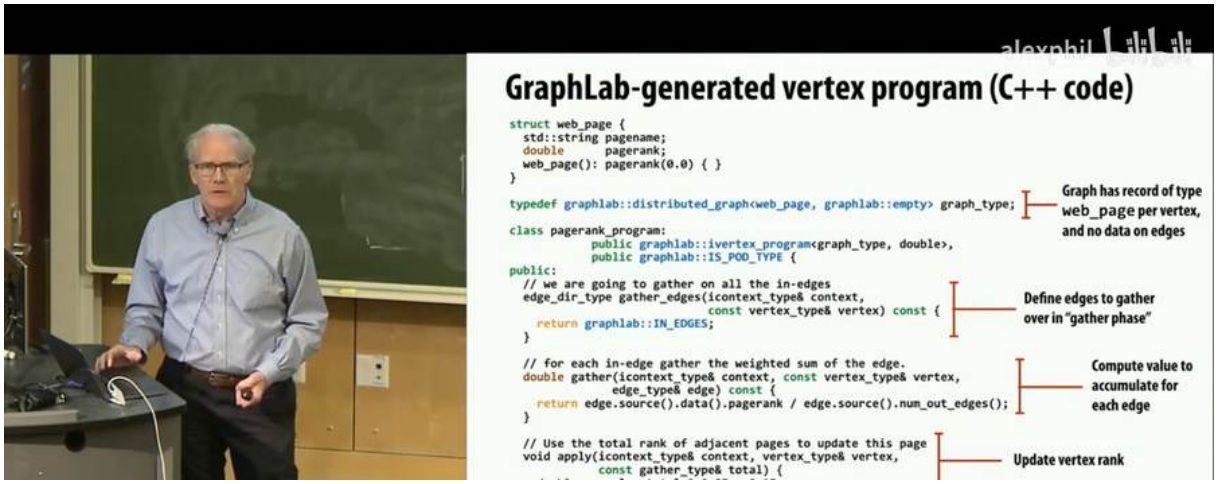

图 546: GraphLab 生成的顶点程序（C++ 代码）[546]

[544]视频画面时间区间：00:18:30–00:18:45。若为裁剪图，时间区间仍对应原视频画面。
[545]视频画面时间区间：00:19:30–00:19:45。若为裁剪图，时间区间仍对应原视频画面。

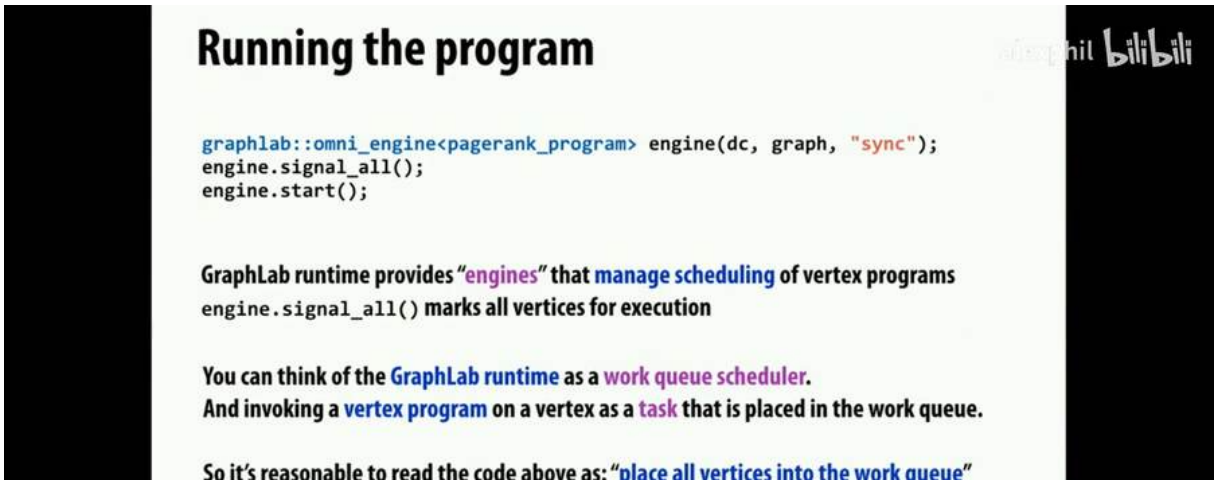

图 547: 运行程序：引擎以同步模式执行顶点程序[547]

顶点信号

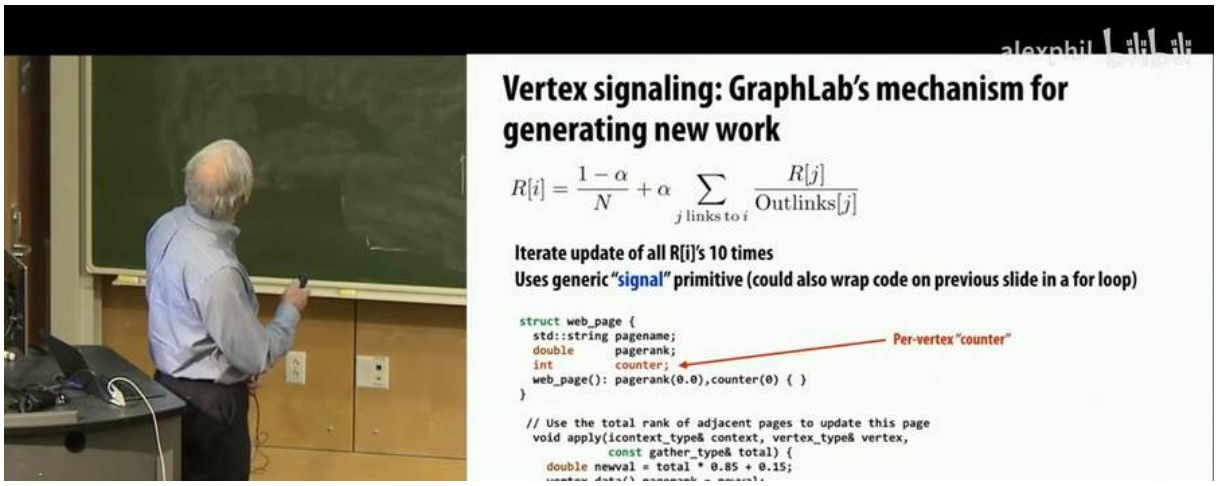

图 548: 顶点信号：GraphLab 产生新工作的机制[548]

[546] 视频画面时间区间：00:20:00–00:20:15。若为裁剪图，时间区间仍对应原视频画面。
[547] 视频画面时间区间：00:21:00–00:21:15。若为裁剪图，时间区间仍对应原视频画面。
[548] 视频画面时间区间：00:21:30–00:21:45。若为裁剪图，时间区间仍对应原视频画面。

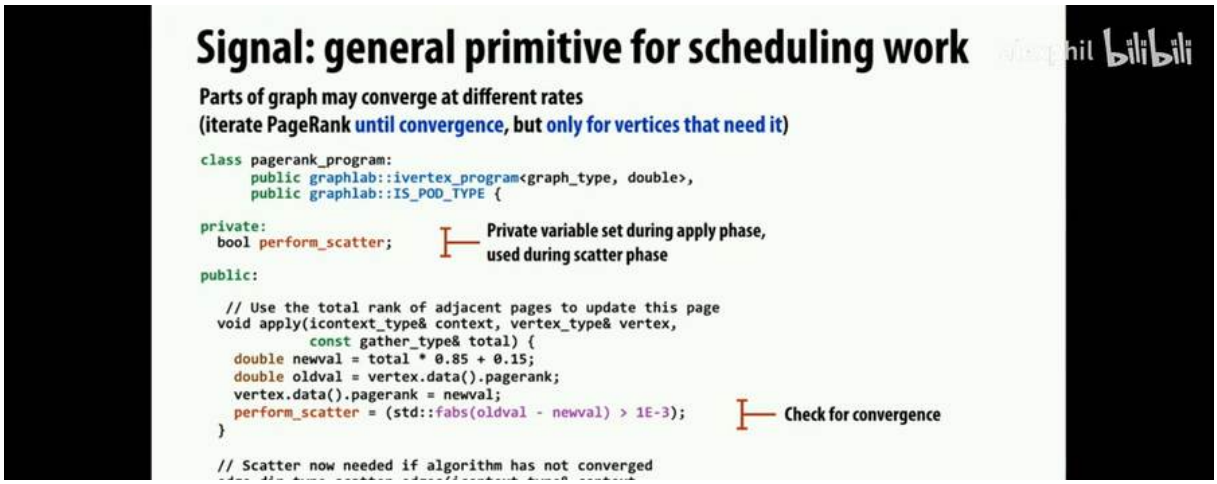

图 549: 信号：调度工作的通用原语[549]

GraphLab 用”信号”机制来产生新工作：一个顶点在更新后可以通知邻居，让它们重新计算。讲者说信号是一个通用的调度原语——图中不同部分可能以不同速率收敛，因此只重算真正需要更新的部分比每轮全部重算更高效。

信号机制把”何时重算”变成可编程决策

若每轮全部顶点都重算，工作量与顶点数成正比，但很多顶点其实没变。信号让程序只唤醒真正受影响的顶点，因此工作量与”实际变化的传播范围”成正比。这是异步图计算能显著快于同步版本的关键原因。

本章小结

PageRank 的直觉是”重要性由你认识谁决定”。GraphLab 用顶点程序定义局部计算，用信号机制产生新工作，使只重算受影响部分成为可能。

这引出并行执行的一致性问题。

[549] 视频画面时间区间：00:22:30–00:22:45。若为裁剪图，时间区间仍对应原视频画面。

25.4 一致性规则与调度顺序

一致性的选项谱

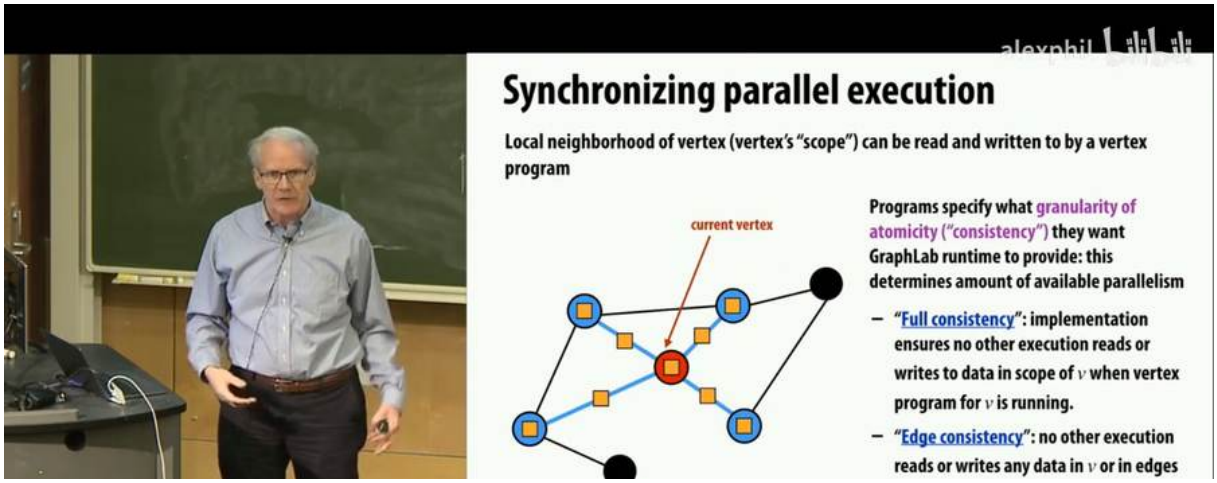

图 550: 同步并行执行：顶点邻域可被读写[550]

讲者说明 GraphLab 因为设计得极为通用、被很多人用于很多用途，因此提供了多种一致性规则选项。

一种是非常宽松的：想做什么就做，想什么时候做就什么时候做。另一种给出更多约束，例如是否允许对同一顶点或同一条边同时读写。

同步与异步

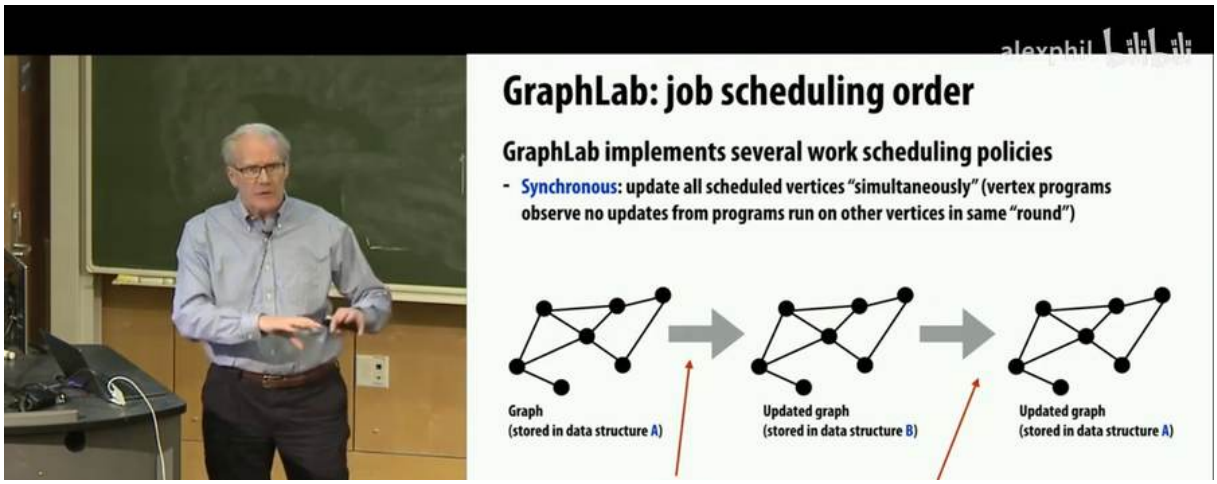

图 551: GraphLab 的任务调度顺序：同步与异步策略[551]

讲者接着讲调度规则。一种是同步模式，看起来就像第 12 讲的同步鼠群例子：所有更新先全部算完，然后才改变状态。

实现上它维持两份状态副本：计算新状态时基于旧状态 A 算出 B，然后 B 成为当前状态，接着复用 A 的数据结构作为下一轮的新状态。这个翻转避免了任何相互干扰的可能。

[550] 视频画面时间区间：00:23:30–00:23:45。若为裁剪图，时间区间仍对应原视频画面。
[551] 视频画面时间区间：00:24:00–00:24:15。若为裁剪图，时间区间仍对应原视频画面。

但它通常收敛更慢。讲者说因为存在振荡（与第 12 讲同样的现象），异步风格的更新往往更好。

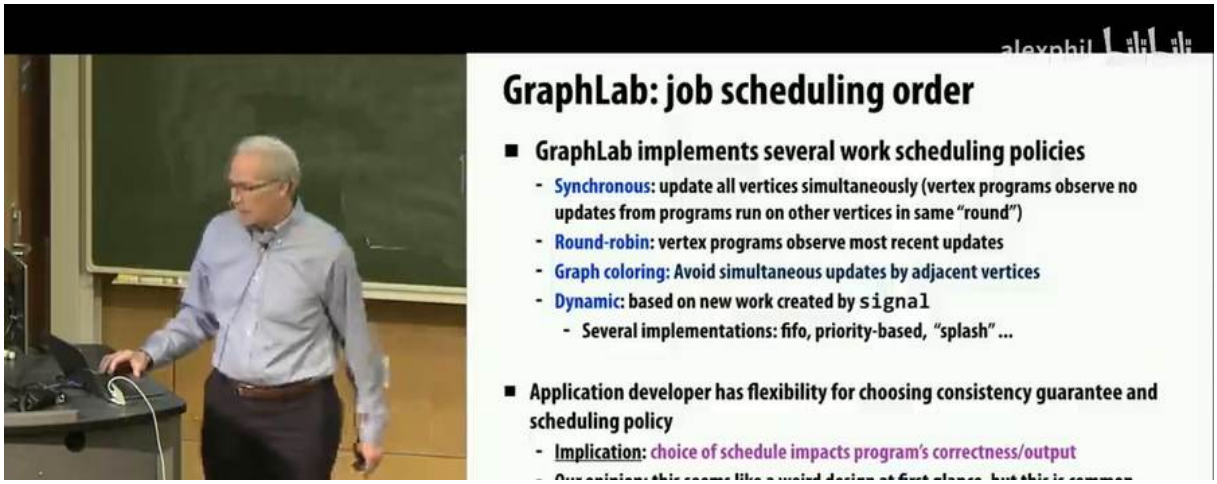

图 552: 调度顺序续：同步、轮转与动态信号的收敛差异[552]

另一种是动态模式，由”更新了哪些顶点”的信号来驱动。

$$\text{工作量}_{\text{sync}} = N_{\text{vertices}} \times N_{\text{rounds}}, \qquad \text{工作量}_{\text{async}} \propto \text{实际传播范围}$$

- 同步模式每轮处理全部顶点，工作量与顶点数乘轮数成正比；
- 异步模式只重算被信号唤醒的顶点，工作量与实际变化传播的范围成正比；
- 因此当图中只有局部区域在变化时，异步显著更省。

异步更快但正确性论证更难

异步更新意味着不同顶点可能基于不同轮次的数据计算，因此最终结果依赖于执行顺序。若算法对顺序敏感（例如需要严格单调收敛），异步可能给出与同步不同的结果。因此选择调度策略时，必须确认算法在所选一致性规则下仍满足正确性条件。

本章小结

GraphLab 提供从极宽松到有约束的一致性规则，以及同步、轮转、动态信号等多种调度策略。同步模式用双缓冲避免干扰但收敛更慢；异步因避免振荡而收敛更快，代价是结果依赖执行顺序。

这些选项都以钩子形式暴露，而这带来一个新问题。

[552] 视频画面时间区间：00:26:00–00:26:15。若为裁剪图，时间区间仍对应原视频画面。

25.5 三条经验与一个警告

经验一：识别真正重要的场景

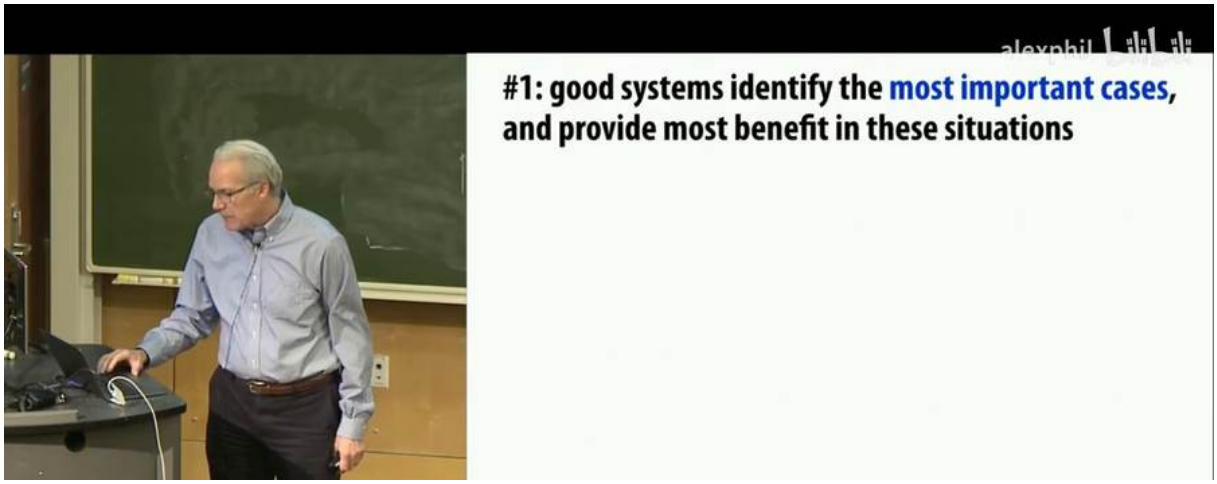

图 553: 经验一：好的系统识别最重要的场景并在这些场景中提供最大收益[553]

讲者说这是个好教训：你要做的是识别哪些重要场景真正值得这样一个系统，然后做出一个利用问题已有抽象（图就是好例子）的编程系统，并让它能给你想要的性能。

经验二：保持简单

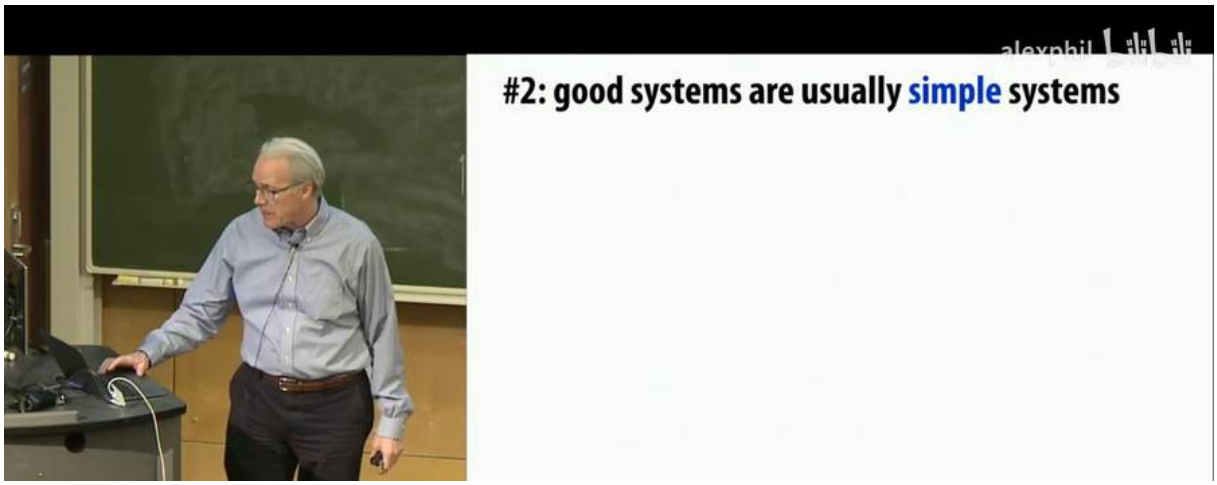

图 554: 经验二：好的系统通常是简单的系统[554]

第二条经验，也是讲者认为构建这类系统最难的事情之一：让它们保持简单。

他描述了失控的过程：GraphLab 起步时概念很简单，但随着你加入更多特性、更多能力、更精细的处理，它就开始变复杂。每有人说不喜欢某种做法，就会有开发者说” 好，我们也实现那个”，于是又多了一个选项——很快就不受控制了。

[553] 视频画面时间区间：00:26:30–00:26:45。若为裁剪图，时间区间仍对应原视频画面。
[554] 视频画面时间区间：00:27:00–00:27:15。若为裁剪图，时间区间仍对应原视频画面。

选项累积是专用系统的主要腐化路径

每个新增选项都看似合理，但它们组合后的语义空间会爆炸。用户面对的选择越多，越难判断哪个组合是正确的。因此健康的专用系统往往有意识地拒绝一部分需求，用”不支持的场景请用通用语言”来换取语义清晰。

经验三：原语应当可组合

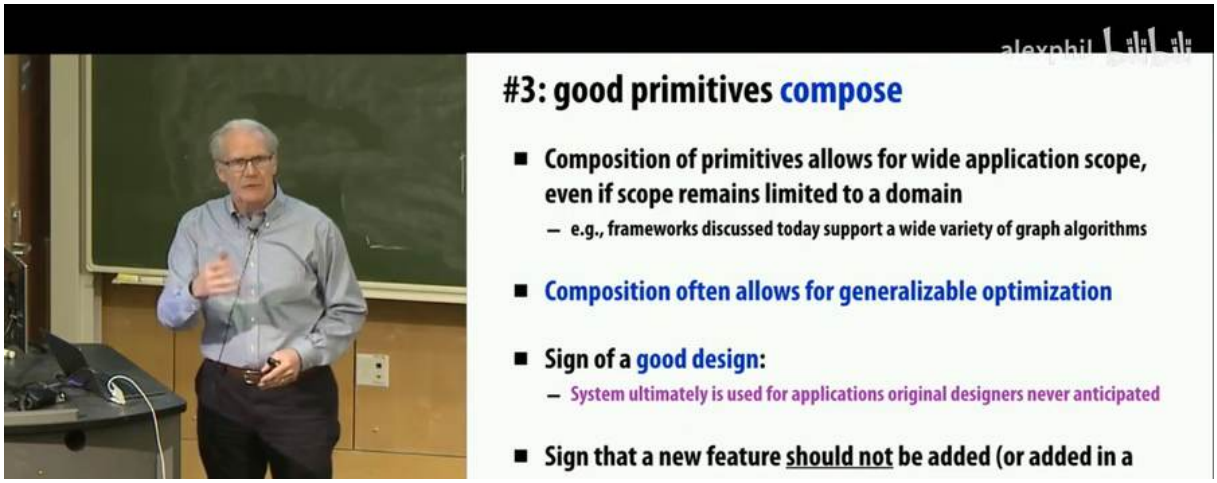

图 555: 经验三：好的原语可以组合，从而覆盖广泛的应用范围[555]

第三条：好的原语可以组合。原语的组合使得应用范围可以很广，即使每个原语本身很简单。

本章小结

三条经验是：识别真正重要的场景、保持系统简单、让原语可组合。讲者特别强调第二条最难——每加一个选项都看似合理，但组合后的语义空间会爆炸，最终使系统失控。

下面看一个把局部性做到极致的实例。

25.6 分片：把局部性做到极致

本章小结

三条经验是：识别真正重要的场景、保持系统简单、让原语可组合。讲者特别强调第二条最难，因为每加一个选项都看似合理，但组合后的语义空间会爆炸。

下面看一个把局部性做到极致的实例。

[555] 视频画面时间区间：00:28:00–00:28:15。若为裁剪图，时间区间仍对应原视频画面。

25.7 GraphChi：用分片把随机访问变成顺序读

分片表示

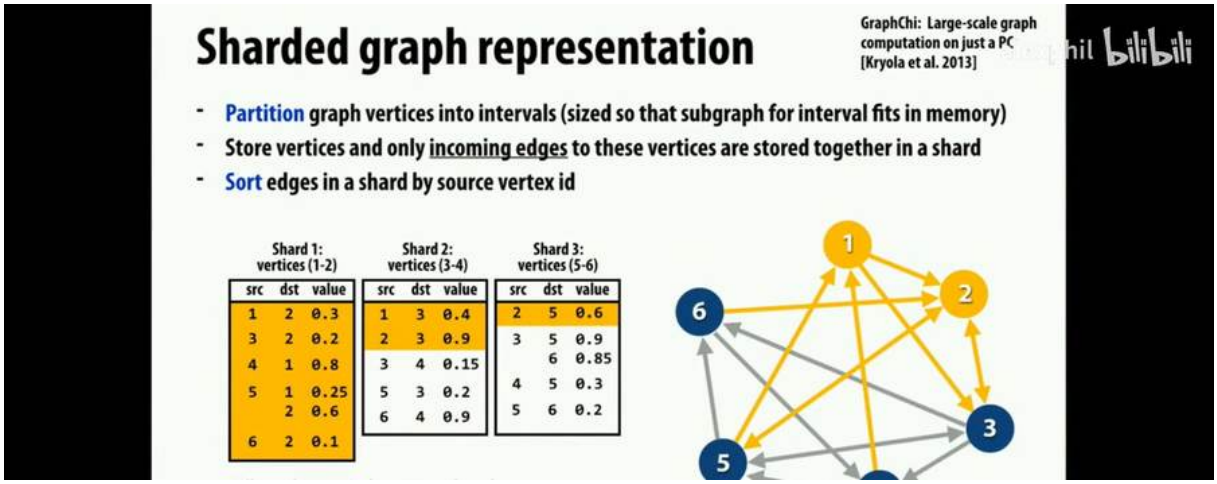

图 556: GraphChi：大图在单机上的分片图表示[556]

讲者最后介绍 GraphChi——一个在单台 PC 上做大图计算的系统。它的核心是分片的图表示。

分片的划分规则是：对于某一片，所有指向该片内节点的入边都算这一片的一部分，而出边不算（它们属于对方的分片）。

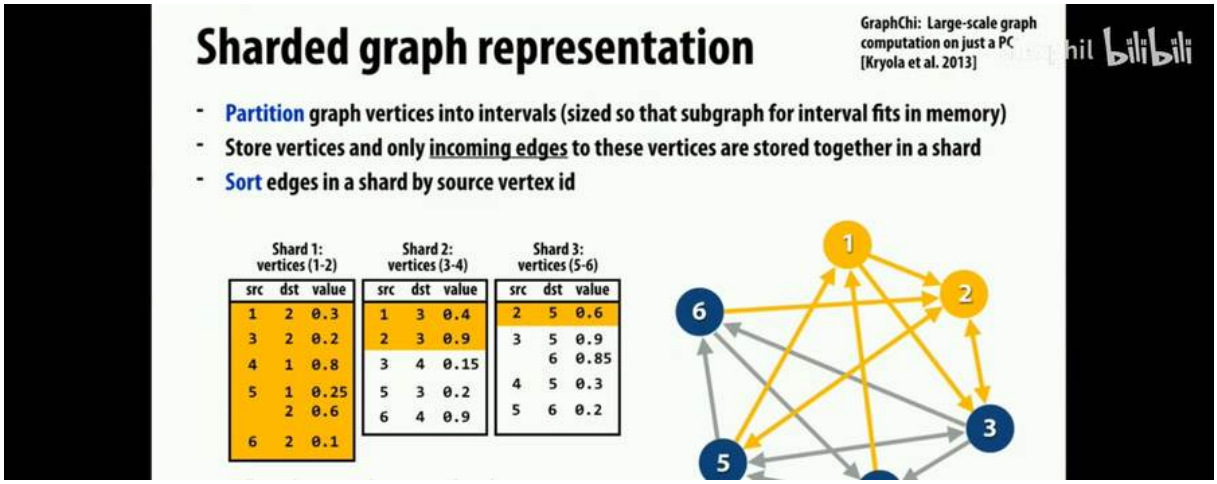

图 557: 局部性：按源排序后只需读取其他表中的连续片段[557]

关键在于排序。若对每一片，把边表按源排序，并且假设排序顺序（节点编号）也与分片划分一致，那么处理某一片时：

本片需要的信息是完整的；而从其他分片（邻居的分片）需要的，只是那些指向本片的边——而由于已排序，这些边在对方表里是连续的片段。

因此局部性来自两处：一是本片信息本来就集中存放；二是即使信息分布在别处，也能定位到它的连续存放位置。

若跨片边在对方表中是连续区间，则访问成本从随机变为顺序：

[556] 视频画面时间区间：00:34:00–00:34:15。若为裁剪图，时间区间仍对应原视频画面。

[557] 视频画面时间区间：00:35:00–00:35:15。若为裁剪图，时间区间仍对应原视频画面。

$$T_{\text{shard}} \approx \frac{B_{\text{local}}}{\text{BW}_{\text{seq}}} + \frac{B_{\text{cross}}}{\text{BW}_{\text{seq}}}$$

- B_{local}：本片数据量，天然连续；
- B_{cross}：需从其他分片读取的边数据量；
- 两者都按顺序带宽计价，因此总时间只与实际字节数成正比，不随随机跳转次数恶化。

未排序时第二项的分母会退化为随机访问带宽，通常低一个数量级。

排序使”稀疏的跨片访问”变成”连续的范围读”

图计算的主要成本常是随机访存。按源排序后，一个分片所需的跨片边变成对方表中的一段连续区间，于是随机访问变成顺序读取。这个转换不需要改变算法，只需改变数据布局——与第 8 讲的结论一致：局部性首先是布局问题。

流式计算

讲者说明这样就可以写出流式计算：对每个分区、每个分片，加载该片的信息以及与之相关的其他分片信息，然后处理。

他用一个示意：假设分片是节点 1 到 5、6 等，那么每次访问时都是读一个大表加上其他表中的连续部分。他评价说这是一个相当简单的技巧，就是如何组织信息。

本章小结

GraphChi 用分片表示让大图能在单机处理：入边归本片、出边归对方，并按源排序，使跨片访问变成连续范围读。局部性因此同时来自本片集中存放与跨片定位。这是一个纯数据布局技巧，不改变算法。

25.8 总结与延伸

讲者在结尾说今天就到这里，让学生们去忙项目。

把全讲串起来，这是一条从”为什么专用化”到”怎么实现”的路径。领域专用系统有”新语言”与”专用包”两种形态，核心取舍在性能、完备性、通用性、易用性四维之间。图计算因规模巨大（可达十亿级）、映射困难、算法不应反复重写，成为专用化的理想场景。设计难点是必须达到手写性能，否则不会被采用；两条路径是自动优化与提供可调钩子。Halide 展示了后者的形态，演化为”计算描述 + 调度描述”的双语言设计。GraphLab 用顶点程序定义局部计算、用信号机制产生新工作，并提供从极宽松到有约束的一致性规则与多种调度策略；同步模式用双缓冲避免干扰但收敛更慢，异步因避免振荡而收敛更快。三条经验是识别真正重要的场景、保持系统简单、让原语可组合，其中”保持简单”最难，因为选项累积会让语义空间爆炸。最后，GraphChi 用分片表示与按源排序，把跨片随机访问变成连续范围读，在单机上处理大图。

本讲义在讲者材料之上做的压缩，有四点需要与讲者原话区分：

- 专用系统的价值不只是”写起来快”，而是”不牺牲性能的前提下写起来快”。若性能劣于手写，关键用户会离开，系统的生态就无法建立。因此这类系统的成败取决于它能否把专家知识编码进调度层。

- 双语言设计是对” 自动优化” 与” 手工调优” 的折中。纯自动的系统难以覆盖所有机器特性，纯手工的系统失去抽象收益。分成两层后，算法层可复用，调度层可针对机器重写。
- 异步更新的加速来自” 只做必要的工作”，但也把正确性责任转移给了算法。它不保证与同步版本得到相同结果，因此使用前必须确认算法在所选一致性下仍然收敛到正确解。
- 局部性的改善往往来自布局而非算法。GraphChi 的排序技巧不改变任何计算步骤，只改变数据的组织方式，却把随机访问变成了顺序读。这与第 8 讲的结论是同一条原理在不同层次上的体现。

这堂课的边界在于它是导论：讲者给出的是设计理念与几个系统的结构，未深入具体编译流程与运行时实现。他也坦承这类系统最难的是长期维护其简单性，而这部分没有通用解法。

两个可以直接从本讲数据提出的下一步问题：其一，双语言设计让调度层可针对机器重写，那么当目标机器从单机变成集群时，调度层需要增加哪些新的原语？其二，异步更新更快但结果依赖执行顺序，是否存在一类算法必须在同步语义下运行，判据是什么？

Lecture 26 Parallel Deep Neural Networks

视频时长：1 小时 17 分 26 秒　来源：https://www.bilibili.com/video/BV1594y167Ue/

26.1 深度神经网络：回到结构化数值计算

这一讲讲并行计算与深度神经网络的关系。讲者开场说这个话题想必很多人感兴趣。

一场革命

深度神经网络彻底改变了 AI 与机器学习领域，其影响之广难以描述。讲者举了几个例子：Google DeepMind 在围棋上的胜利；图像识别的巨大进步——起初问题只是" 这张图里有猫吗"，现在则变成" 猫在图中哪里"，还能判断它在走、在睡还是在吃，以及是什么品种。语音识别与语言翻译也已经完全交给深度神经网络。

他回忆几年前访问 Google 时，那里的高层说他们基本上把为特定应用开发的所有精巧基础设施都拆掉了，全部换成深度神经网络，用海量数据训练然后放手让它跑。

许多过去需要精细领域知识的问题，现在只需投入大量训练数据。

图 558: 训练与评估深度神经网络：近年多项 AI 突破背后的技术[558]

深度学习把 AI 拉回数值计算的地盘

讲者特别指出这一点：深度学习让 AI 重新变成结构化的数值计算问题，看起来更像科学计算而非符号推理。这个转变是它与并行计算天然契合的根本原因——它由大量规则化的乘加运算构成，而不是由不规则的控制流构成。

[558] 视频画面时间区间：00:01:30–00:01:45。若为裁剪图，时间区间仍对应原视频画面。

基本单元

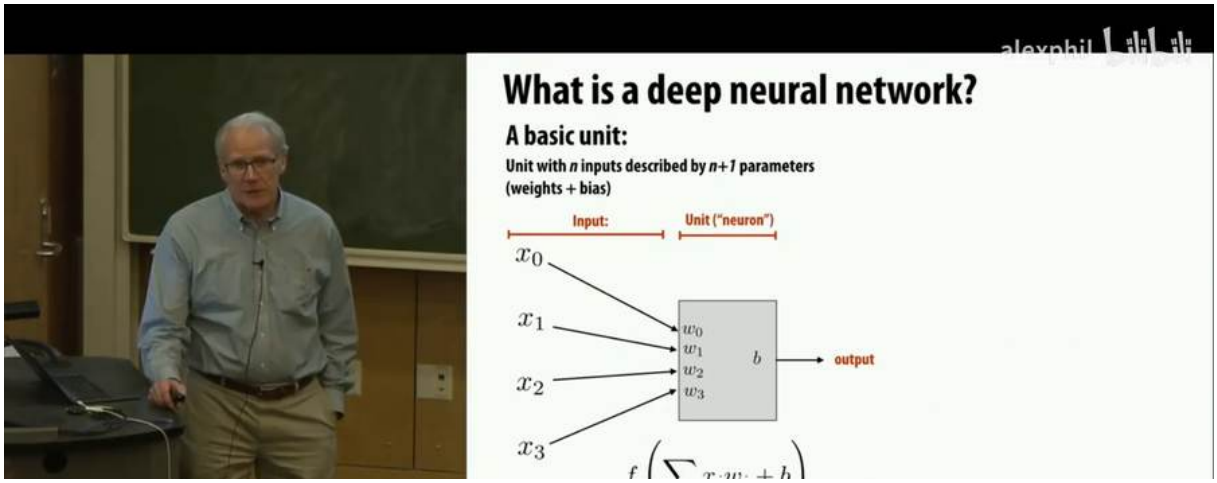

图 559: 深度神经网络的基本单元：n 个输入与 $n+1$ 个参数（权重与偏置）[559]

讲者说明基本想法源自七十年代末甚至更早：一种非常初等的计算，由一组计算元件构成的网络完成。

基本单元是：对若干输入做加权求和。假设这些是实数值，权重也是实数。算出加权和，加上某个偏置，然后对这个结果应用一个非线性函数。

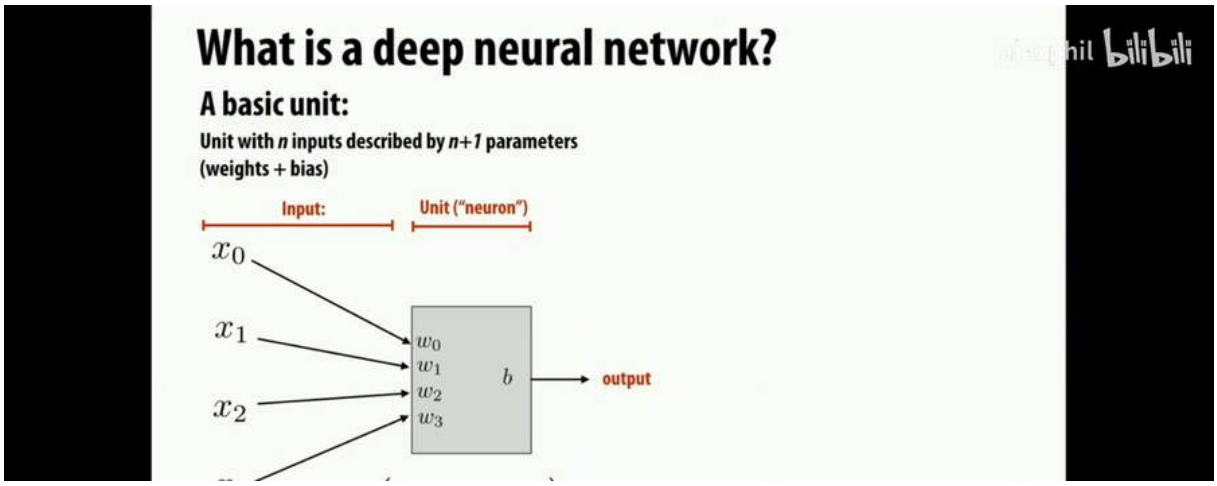

图 560: 激活函数：早期采用饱和型非线性函数[560]

$$y = \sigma\left(\sum_{i=1}^{n} w_i x_i + b\right)$$

- x_i：第 i 个输入值；
- w_i：对应的权重，共 n 个；
- b：偏置，因此该单元共有 $n+1$ 个参数；
- σ：非线性激活函数。

[559] 视频画面时间区间：00:03:00–00:03:15。若为裁剪图，时间区间仍对应原视频画面。

[560] 视频画面时间区间：00:04:00–00:04:15。若为裁剪图，时间区间仍对应原视频画面。

ReLU：一个极其简单的非线性

过去人们设计过各种精巧的函数：输出被限制在零到一之间或某个有界范围，随着输入增大而衰减但不完全消除——不是硬截断，更像是趋近某个极限。

但最近变成了一个简单到离谱的函数：取该值与零的最大值。

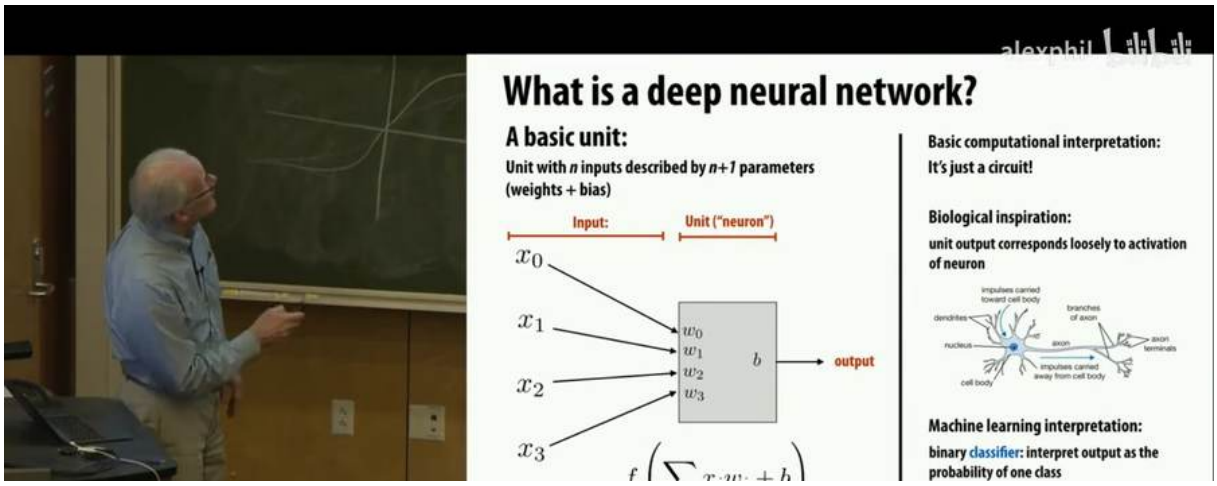

图 561: ReLU：取输入与零的最大值，正区间直通、负区间关闭[561]

$$\mathrm{ReLU}(z) = \max(z, 0)$$

- z：加权和加偏置的结果；
- 若 z 为正，输出就是 z 本身；
- 若 z 为负，输出为零，相当于该单元被”关闭”。

这被称为修正线性单元，也叫铰链函数。它显然是非线性的，但极其简单，而且被证明非常有效。

”脑启发”是设计直觉的来源，不是正确性的依据

神经网络的组件选择确实受生物神经元启发，但它们的有效性由经验结果而非神经科学论证支持。因此调参、换激活函数、改结构都是工程行为：以验证集表现为准，不以”更像大脑”为准。

讲者还做了一个重要澄清：这东西隐约受生物学启发，反映了真实神经元的某些特点；但说它基于某种关于大脑活动的根本理论，则完全是胡说。它是受大脑启发，而非模仿大脑。

本章小结

深度神经网络彻底改变了 AI，并把问题拉回结构化数值计算。基本单元是加权和加偏置再加非线性激活。早期用饱和型函数，现在普遍用 ReLU——取与零的最大值，简单却有效，且只是受大脑启发而非模仿。

下一章区分两个不同的问题：训练与推理。

[561] 视频画面时间区间：00:06:10–00:06:25。若为裁剪图，时间区间仍对应原视频画面。

26.2 训练与推理：两个不同的问题

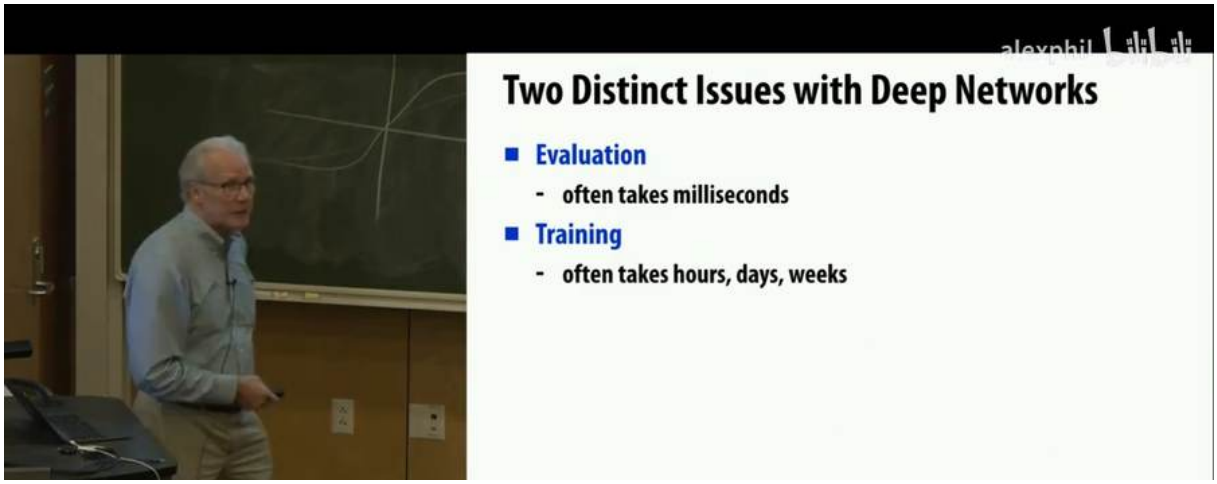

图 562: 深度网络的两个不同问题：推理常以毫秒计，训练常以天计[562]

量级差异

讲者把这件事拆成两个问题，这个区分决定了后续所有优化方向。

训练：让模型学会模仿观察到的相关性，并且针对目标语言专门训练。这通常需要 7 天以上，长则数周甚至数月，以及巨量算力——Google 的数据中心会动用大量节点连续运行多日，只为把这个模型建起来。

推理：某人往搜索框里输入英文、想翻译成中文，这就是把文本喂进网络、计算所有神经元的值。这相对快速简单，但仍然涉及可能十亿次以上的运算。

推理的算力需求由”调用次数 × 单次代价”决定

单次推理远便宜于一次训练，但推理的调用次数可能比训练多出许多个数量级，而且常在资源受限的设备（手机）上进行。因此总能耗与总延迟的瓶颈往往在推理侧。这解释了为什么移动端神经网络加速是一个独立且重要的研究方向。

[562] 视频画面时间区间：00:06:30–00:06:45。若为裁剪图，时间区间仍对应原视频画面。

网络拓扑

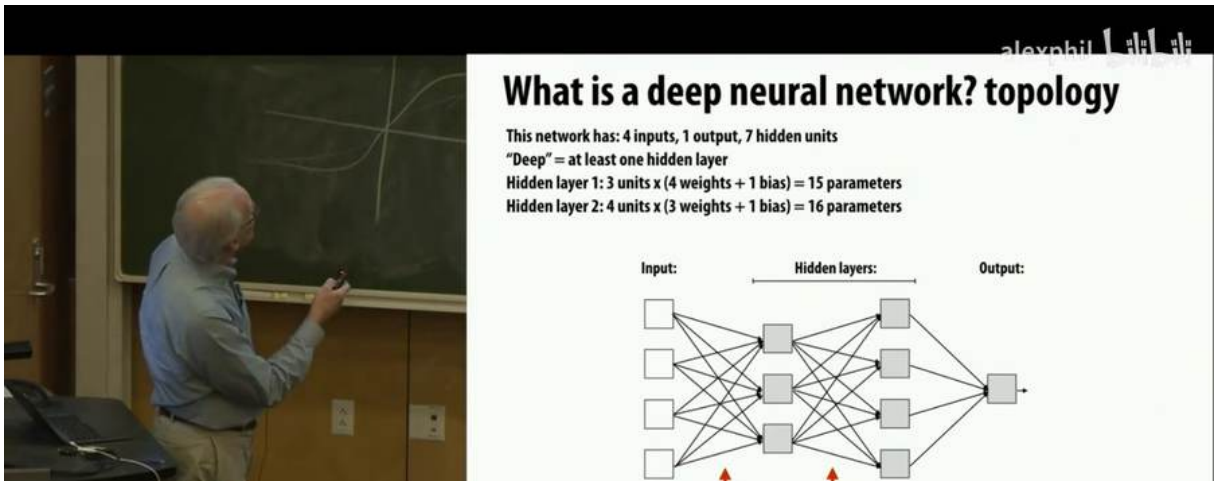

图 563: 网络拓扑：4 个输入、1 个输出、7 个隐藏单元[563]

网络的构成方式是：有一批计算元件或神经单元，按某种结构连成网络。输入是一张图，输出是一组分类器结果。输入与输出之间是隐藏单元。

每个单元都有与其所有输入值对应的一组权重，以及偏置。

讲者说为了讨论推理，先假设这些权重已经确定，稍后再讲怎么得到它们。推理过程就是从输入到输出把这些计算全部做一遍。

他也坦承网络的结构（该有几层、每层多少单元）仍带有相当程度的”黑魔法”成分。

结构选择缺乏理论指导意味着必须靠实验

既然层数与宽度没有可靠的理论推导，网络设计就只能靠经验与搜索。这带来两个后果：一是训练成本高，要试很多配置；二是知识难以迁移，在一个数据集上最优的结构未必在另一个上最优。

连接方式

讲者区分两类连接。全连接层允许一层所有神经元向下一层所有神经元提供值——这当然相当于完整图，是昂贵的东西，因此相对少见。

但在图像处理中，我们天然有局部性要保留，因此常做稀疏连接。

本章小结

训练与推理是两个不同的问题：训练需要数天到数月与巨量算力，推理单次快速但调用次数极多且常在受限设备上进行。网络由输入、隐藏单元与输出分类器构成，每个单元有 $n+1$ 个参数。连接方式分全连接与稀疏连接两类。

图像处理中的稀疏连接正是卷积。

[563] 视频画面时间区间：00:09:00–00:09:15。若为裁剪图，时间区间仍对应原视频画面。

26.3 卷积：权重共享带来的三重节省

从图像卷积到卷积层

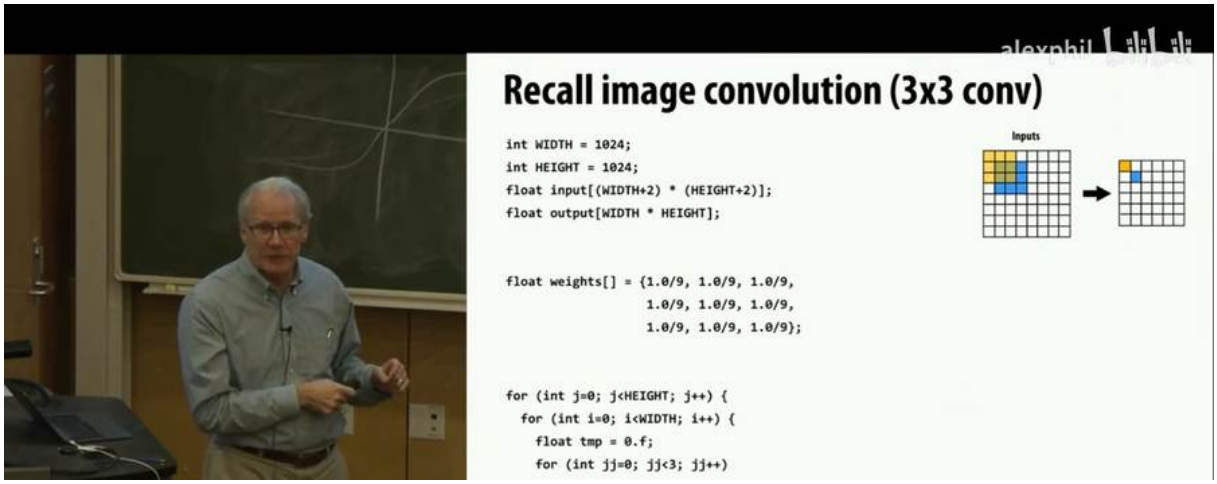

图 564: 回顾图像卷积：1024×1024 输入上的 3×3 卷积[564]

讲者回到图像卷积。此前看过一个简单情形：对某像素最近的九个值使用相同权重。把它推广到任意权重，就很像神经网络的第一部分——对输入做加权和。

卷积的另一个特点是：在图像的每一点都应用同一组权重。

因此有一类网络就叫卷积神经网络，正是这个操作的神经网络版本：有权重，但在给定的连接范围内，权重处处相同——同一个滤波操作应用于整个输入。

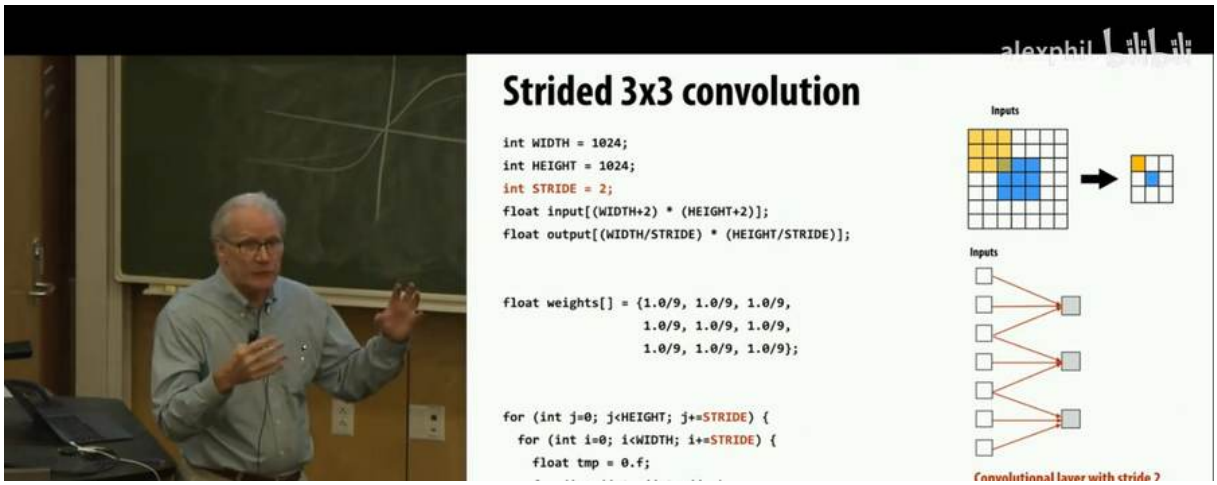

图 565: 带步长的 3×3 卷积：以步长 2 降采样[565]

权重共享是卷积层的核心收益

全连接层每个连接都有独立权重，参数量随输入输出规模乘积增长；卷积层只保留一个滤波器的参数，与图像尺寸无关。因此卷积层更易训练（参数少）、更易存储（模型小）、更易推理（计算少）。讲者的结论很直接：能卷积时就优先卷积。

[564] 视频画面时间区间：00:11:00–00:11:15。若为裁剪图，时间区间仍对应原视频画面。

[565] 视频画面时间区间：00:12:30–00:12:45。若为裁剪图，时间区间仍对应原视频画面。

滤波器在做什么

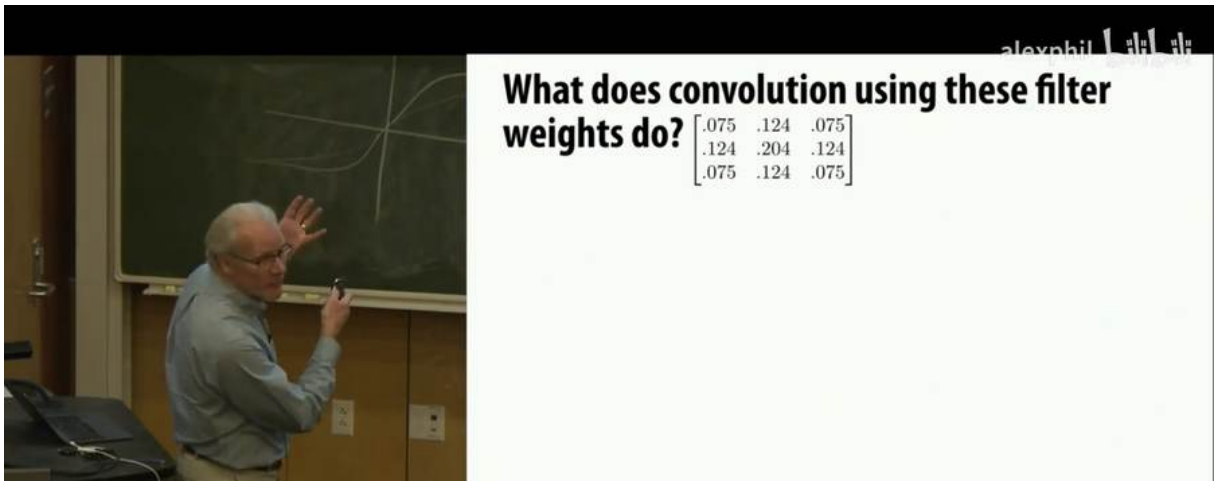

图 566: 这组滤波器权重会做什么：中心权重最高、向外递减[566]

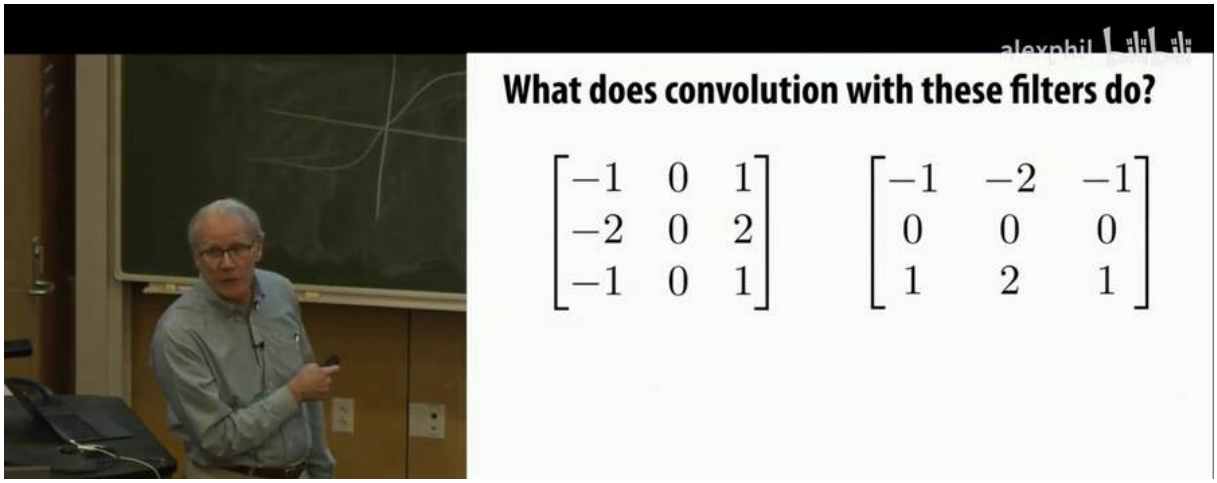

图 567: 高斯模糊：权重按半径呈高斯衰减[567]

讲者展示了一组权重：被保留的像素权重最高，随着距离增加权重递减，按半径呈某种函数关系下降。这实际上是对高斯模糊的近似——权重随半径呈高斯衰减。这也是往图片里引入模糊的常见做法。

[566]视频画面时间区间：00:13:30–00:13:45。若为裁剪图，时间区间仍对应原视频画面。
[567]视频画面时间区间：00:14:30–00:14:45。若为裁剪图，时间区间仍对应原视频画面。

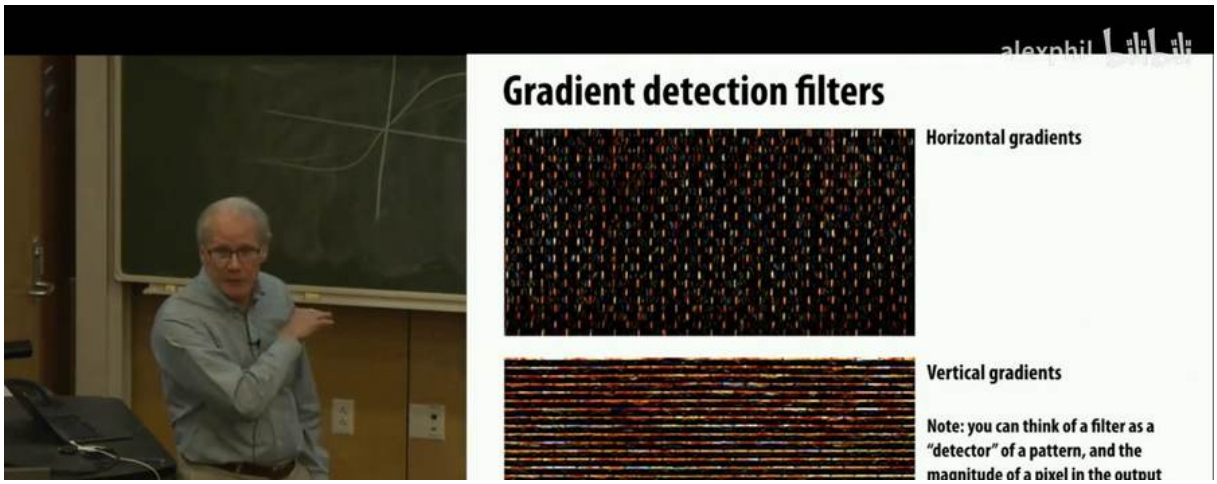

图 568: 梯度检测滤波器：水平与垂直边缘[568]

滤波器权重可以对应到经典图像处理算子

高斯模糊对应权重随半径递减；边缘检测对应正负交替的差分权重。这说明卷积层并非全新的数学，而是把经典图像滤波的算子族参数化了——网络要学的是”该用哪些算子、以什么顺序组合”。

他接着展示含负值的滤波器。这类是做边缘检测的——本质上在求导数，可以想成 x 或 y 方向的空间导数，也就是梯度。

多滤波器

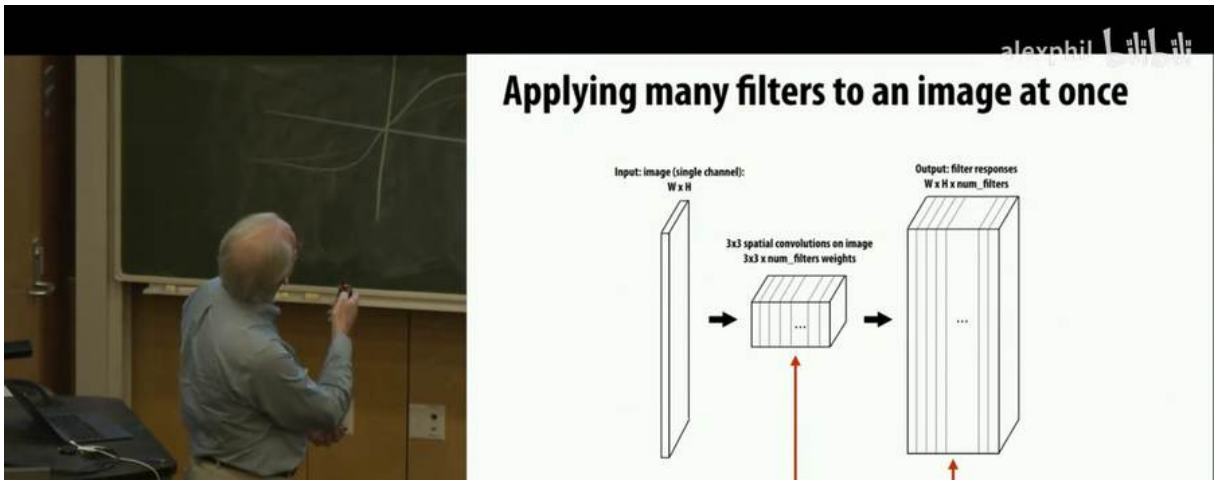

图 569: 同时应用多个滤波器：k 个独立计算产生 k 个响应[569]

讲者在此澄清一个容易误解的图示：我们常想把同类型的多个滤波器应用于一张图，但使用不同的权重。图中画成一摞，但那不表示这些块之间在传递信息。

[568] 视频画面时间区间：00:15:00–00:15:15。若为裁剪图，时间区间仍对应原视频画面。
[569] 视频画面时间区间：00:15:30–00:15:45。若为裁剪图，时间区间仍对应原视频画面。

实际上每一个都是独立的滤波操作，各自有自己的权重，各自产生自己的输出。所以看到这个堆叠时，不要以为它们之间有信息流动——那只是画法，不是计算模型。

叠加更多层

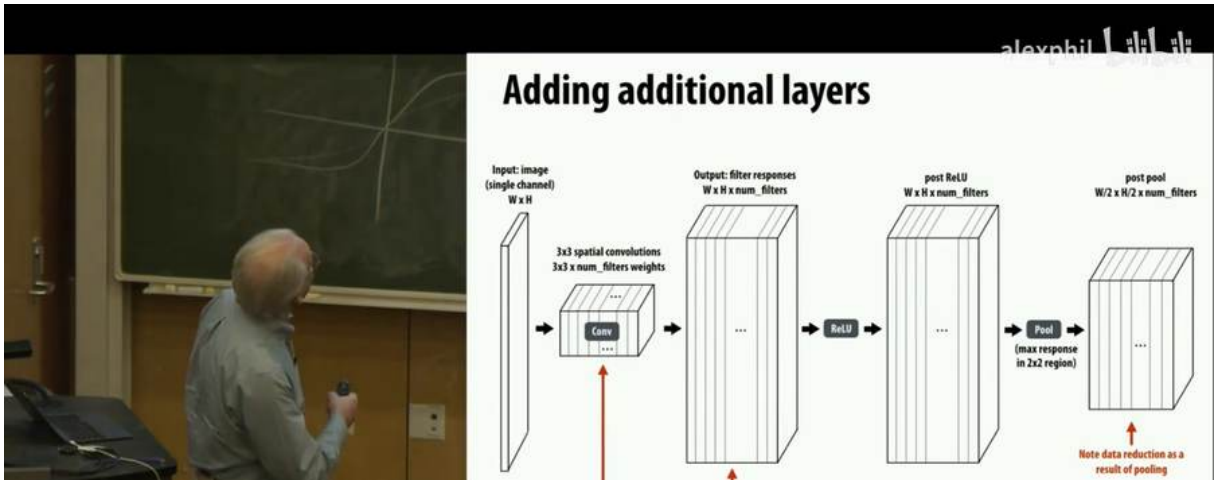

图 570: 叠加更多层：卷积、ReLU 与池化的序列[570]

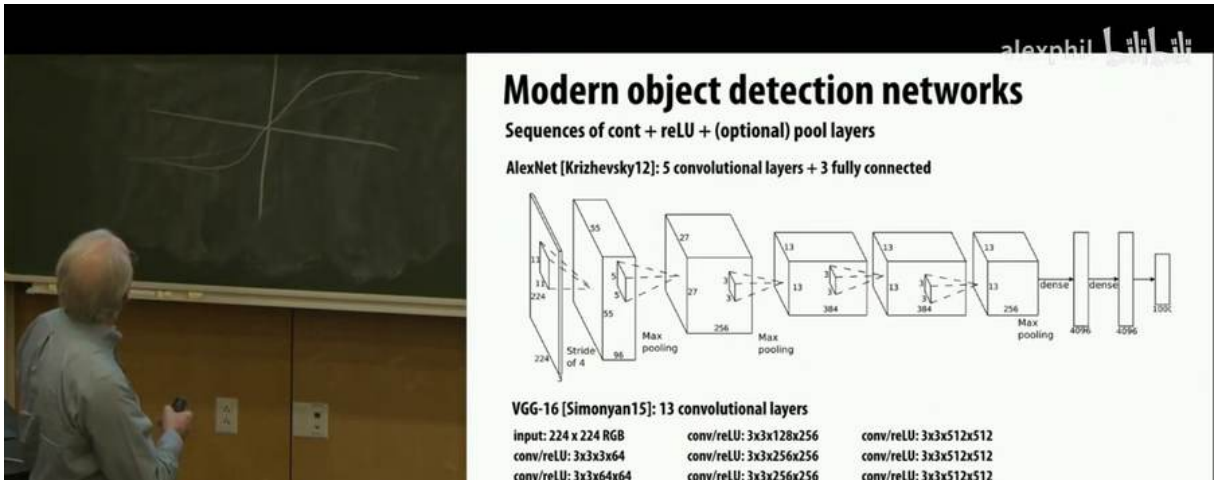

图 571: 现代目标检测网络：卷积 + ReLU + 可选池化的序列（AlexNet）[571]

讲者说明为什么必须有非线性：如果只用线性函数，得不到有趣的结果。这是最早的尝试者（他们把这些叫感知机）遇到的困境——他们不明白为什么什么有意思的事都没发生，原因就是纯线性函数不够，系统必须有非线性才有效。

因此所有滤波响应都要过一个非线性函数。此时这些东西的尺寸仍与原始图像相同，我们可能想把它缩小。

[570] 视频画面时间区间：00:17:00–00:17:15。若为裁剪图，时间区间仍对应原视频画面。
[571] 视频画面时间区间：00:19:00–00:19:15。若为裁剪图，时间区间仍对应原视频画面。

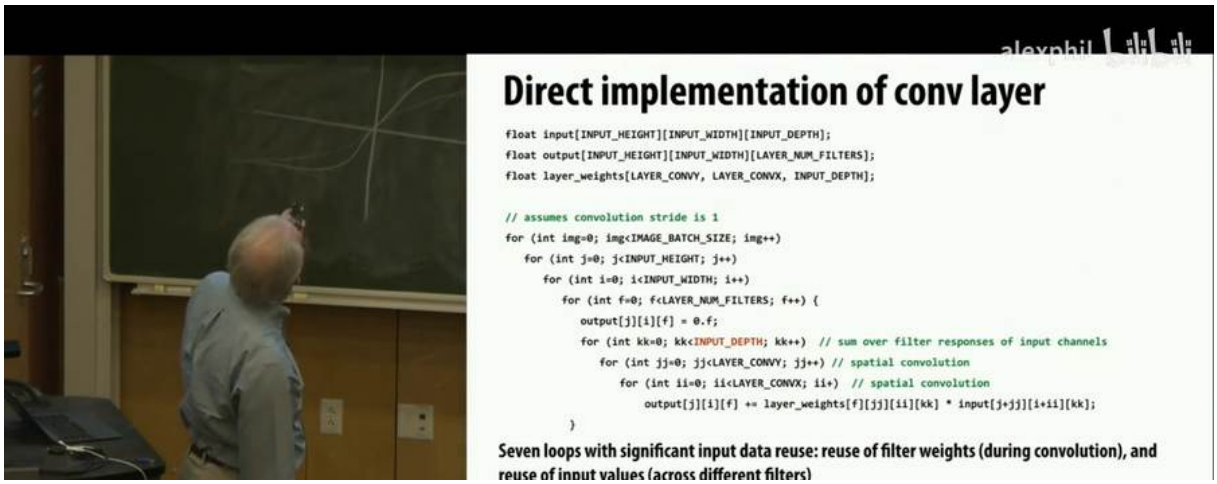

图 572: 卷积层的直接实现：多层嵌套循环[572]

池化

常见做法是某种池化：比如把四个像素平均成一个。但更常用的是最大池化——取四者中的最大值让它通过。

池化的作用是把”位置”信息换成”存在”信息

最大池化保留区域内最强的响应，丢弃其精确位置。这让网络对小的平移不敏感，从而获得一定的不变性。代价是空间分辨率下降，因此需要精确定位的任务不能过度池化。

讲者说这同样是数据缩减方案：你只想把最重要的东西推过去。

他展示了一个实际在用的图像网络：初始计算把 224×224 的图像用带步长的卷积降到 55×55，然后用 5×5 最大池化继续降，再用 3×3 最大池化继续降。末端用 softmax 给出分类。

他也指出：有些层是卷积的（权重在所有滤波器上相同），而稠密区域的那些则完全任意——每个权重对每个位置的每个副本都可以不同，因此可能的权重数量极其庞大。

本章小结

卷积层因权重共享而三重节省：更易训练、更易存储、更易评估。滤波器可以近似高斯模糊，也可以是边缘检测（求空间导数）。多滤波器是 k 个独立计算而非相互传递。纯线性无效，因此必须加非线性；池化做数据缩减，最大池化最常用。典型网络是”卷积 + ReLU + 池化”的序列，末端用 softmax 分类。

26.4 推理的计算结构

任务规模

讲者给出具体任务：给一张新的 $224 \times 224 \times 3$ 的 RGB 图像，如何算出那一百个输出值?

[572] 视频画面时间区间：00:22:00–00:22:15。若为裁剪图，时间区间仍对应原视频画面。

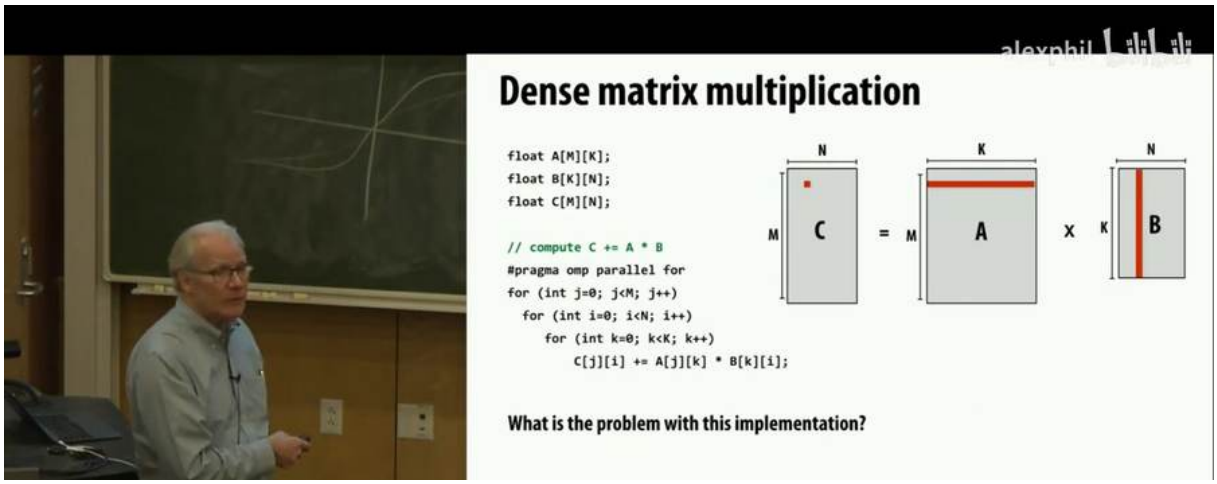

图 573: 稠密矩阵乘法：卷积与它在计算结构上相同[573]

一种方式是写很深的嵌套循环。以单个卷积层为例：外层遍历批中的每张图，再遍历输出的空间维度、滤波器数量与输入通道，最内层遍历卷积核的 x 与 y 维度。

最终就是大量乘加运算。

单层卷积的乘加次数可以估为：

$$\text{MACs} = H \cdot W \cdot K \cdot C_{\text{in}} \cdot k_h \cdot k_w$$

- H, W：输出特征图的空间尺寸；
- K：本层滤波器个数（输出通道数）；
- C_{in}：输入通道数，首层为 3（RGB）；
- k_h, k_w：卷积核的高与宽，3×3 时各为 3。

以 224×224 输入、3 通道、3×3 核为例，单层就已是百万量级的乘加，整网因此达十亿次以上。

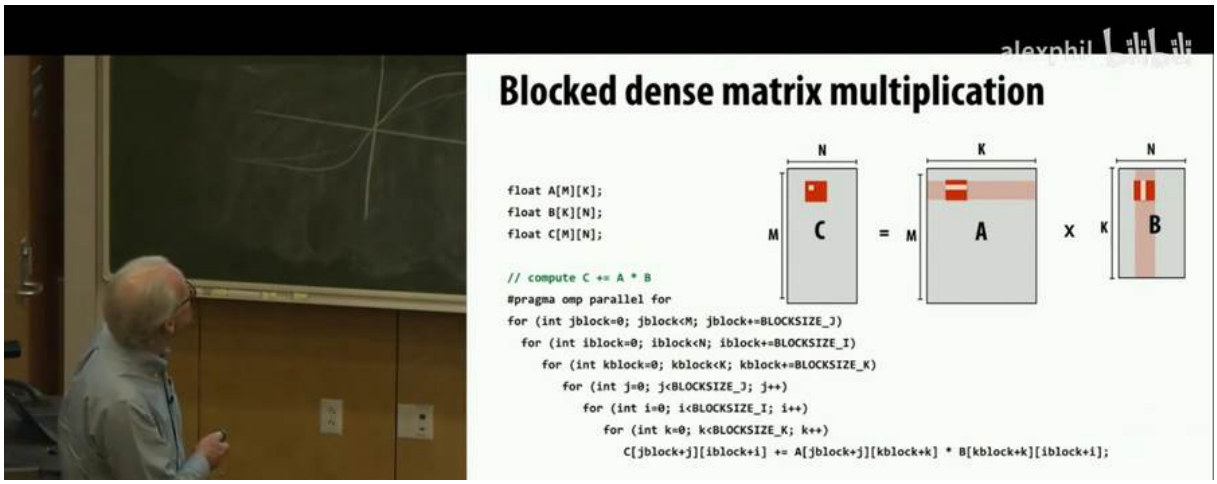

图 574: 分块稠密矩阵乘法：改善空间局部性[574]

[573] 视频画面时间区间：00:23:30–00:23:45。若为裁剪图，时间区间仍对应原视频画面。

[574] 视频画面时间区间：00:24:30–00:24:45。若为裁剪图，时间区间仍对应原视频画面。

本章小结

卷积层因权重共享而三重节省：更易训练、存储与评估。滤波器可近似高斯模糊，也可做边缘检测（求空间导数）。多滤波器是 k 个独立计算而非相互传递。纯线性无效，因此必须加非线性；池化做数据缩减，最大池化最常用。典型网络是「卷积 + ReLU + 池化」的序列，末端用 softmax 分类。

有了网络结构，接下来的问题是：怎么把它算得快。

26.5 推理的计算结构

循环嵌套与局部性

讲者做了一段很有价值的分析：计算内层某个元素时，我们在 A 的所有行上跨步，这不算太糟，因为至少有空间局部性；而在 B 的所有列上跨步，这就很糟，因为没有空间局部性。

但两种情况下都没有任何时间局部性。因此缓存利用得很不好。

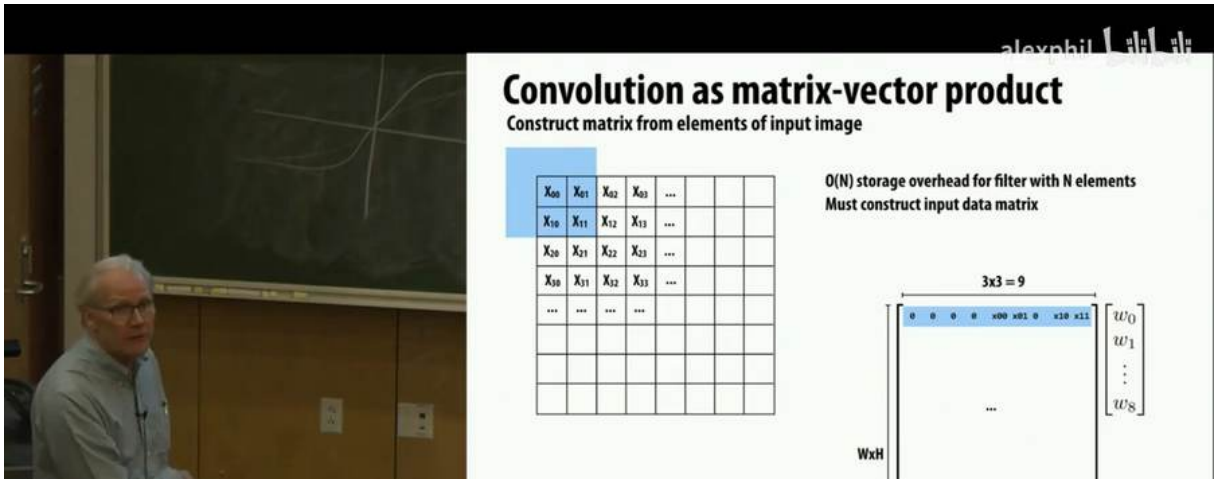

图 575：把卷积写成矩阵向量积：从输入图像元素构造矩阵[575]

解法与前面各讲一致：引入分块以改善空间局部性——把 A 与 B 的某个子矩阵留住，在其元素上把计算做完再换。

卷积的优化与矩阵乘法同构

把卷积展开成矩阵乘后，所有针对 GEMM 的优化技术（分块、寄存器阻塞、向量化、多线程划分）都可以直接复用。这也是为什么深度学习框架的卷积实现往往调用高度优化的 GEMM 库，而不是自己写循环。

[575] 视频画面时间区间：00:25:00–00:25:15。若为裁剪图，时间区间仍对应原视频画面。

[576] 视频画面时间区间：00:26:30–00:26:45。若为裁剪图，时间区间仍对应原视频画面。

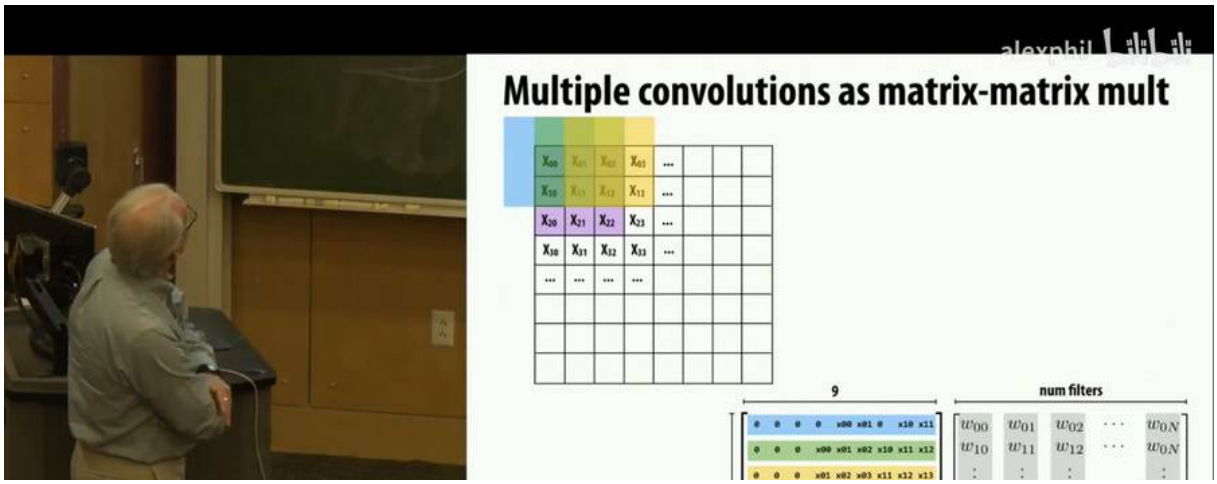

图 576: 多个卷积作为矩阵矩阵乘法[576]

内存占用

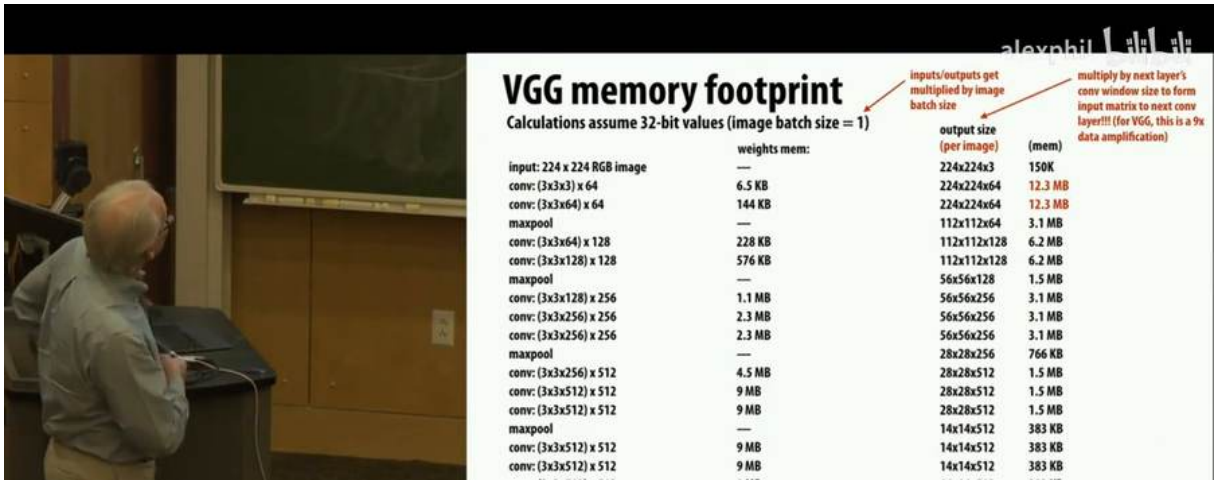

图 577: VGG 的内存占用：输入输出随层数逐级放大[577]

[577] 视频画面时间区间：00:28:30–00:28:45。若为裁剪图，时间区间仍对应原视频画面。

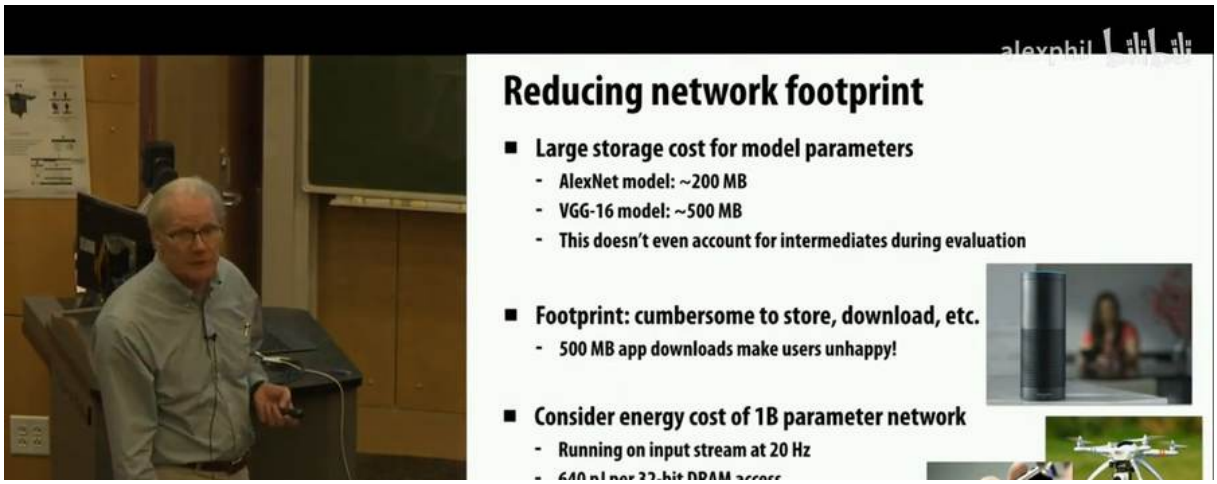

图 578: 降低网络内存占用：参数存储与中间结果[578]

讲者讨论了 VGG 的内存占用：输入与输出会逐层乘以滤波器数量与卷积窗口大小，因此中间结果的存储需求随层数快速增长。

参数量与算力是两个独立的约束

参数量决定模型能否装进片上存储、能否快速加载；算力决定单次推理需要多少周期。二者不一定同向变化：有些网络参数少但计算密，有些参数多但每参数只算一次。优化时必须先测出哪个是瓶颈，再决定压缩策略。

他给出具体数字：AlexNet 模型参数约 200 MB，而 VGG-16 更大。这说明推理的瓶颈可能不在算力而在内存带宽与容量。

推理的瓶颈常在内存而非算力

卷积的计算量大，但参数与中间结果的存储同样巨大。若每层都要把整个中间结果写回内存再读回，带宽就会成为限制。因此实际实现常做层融合（把相邻的卷积与 ReLU 合并）以避免中间结果的往返。

本章小结

推理任务是把 $224 \times 224 \times 3$ 的输入算成约一百个输出。朴素实现是多层嵌套循环，缓存行为差：B 的列遍历无空间局部性，且两者都无时间局部性。解法是分块，与矩阵乘法同构。内存占用随层数快速增长，AlexNet 参数约 200 MB，因此瓶颈常在带宽而非算力。

26.6 训练：梯度下降与损失

本章小结

推理任务是把 $224 \times 224 \times 3$ 的输入算成约一百个输出。朴素实现是多层嵌套循环，缓存行为差：B 的列遍历无空间局部性，且两者都无时间局部性。解法是分块，与矩阵乘法同构。内存占用随层数快速增长，AlexNet 参数约 200 MB，因此瓶颈常在带宽而非算力。

推理只需前向计算。训练则要额外完成误差回传与参数更新，是更重的任务。

[578] 视频画面时间区间：00:30:00–00:30:15。若为裁剪图，时间区间仍对应原视频画面。

26.7 训练：梯度下降与损失

泛化与评估集

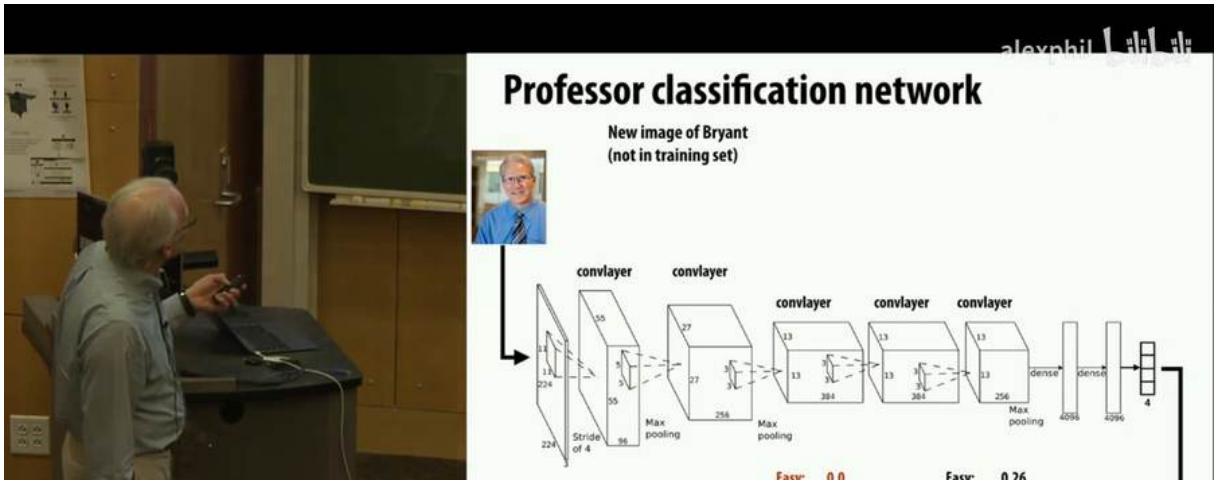

图 579: 训练循环：前向传播、计算损失、反向传播更新权重[579]

讲者说明训练要做什么：我们已经知道网络对该图像的输出，也知道它应该预测什么，现在想回去把这些权重稍微调整一点，让它变得更好——但不要改得太剧烈。

他指出了一个关键的张力：对某一张图像而言，让它在这一个样本上变得完美是相对容易的。但我们希望在整个巨大样本集上都能给出普遍良好的结果。

更重要的是，我们要它不仅能对训练过的样本工作，而且在丢进其他已知或未标注的图像时也能给出正确的值。

因此常见做法是把原始训练数据切出一部分，纯用于评估：在一个集合上训练，然后在另一个来自统计上相似分布的集合上评估质量。

泛化的代价决定了训练必须”小步走”

如果为单张图大幅调整权重，训练集上的误差会迅速下降，但模型会失去泛化能力。因此训练必须用很小的步长在大量样本上累积调整。这个约束解释了为什么训练需要极多轮迭代与极大数据量，也就解释了为什么训练的算力需求远高于推理。

[579] 视频画面时间区间：00:40:00–00:40:15。若为裁剪图，时间区间仍对应原视频画面。

softmax 与损失

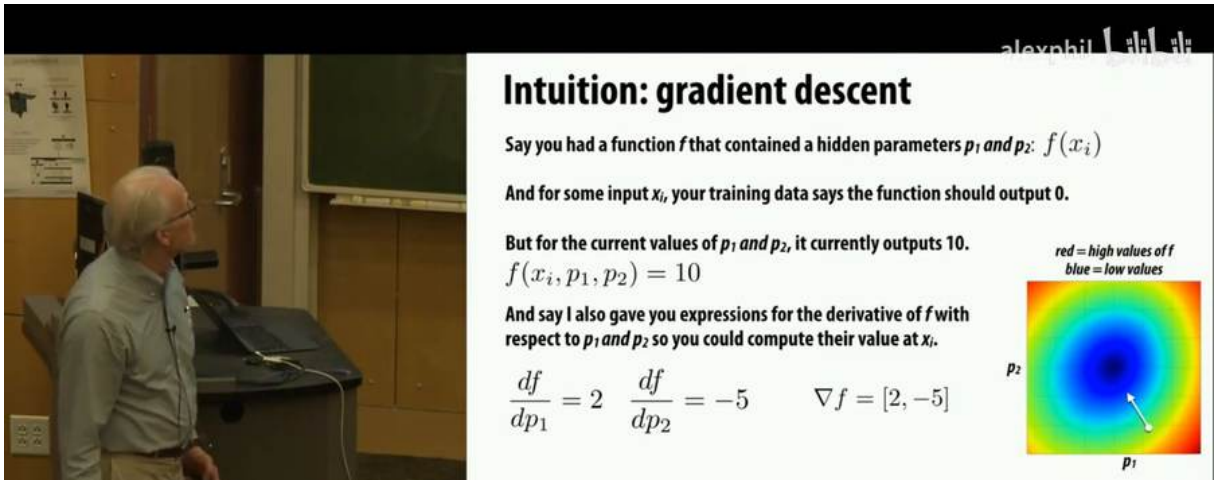

图 580: 梯度下降的直觉：沿函数下降方向调整参数[580]

softmax 的可微性是训练可行的前提

若直接取最大值，该操作在绝大多数点上梯度为零，参数无法通过梯度下降更新。softmax 用一个平滑的归一化替代硬选择，使梯度处处存在且能反映”次优类别与最优类别的差距”，这是反向传播能工作的关键。

讲者说明输出处理：若有若干实数输出，直接取最大值也算合理。但通常会让它们经过一个叫 softmax 的算子——它会强调最大的那个数，但不会把其余的全丢掉。

这样做部分是为了让函数可微，从而获得比简单取最大值更细的分辨率。

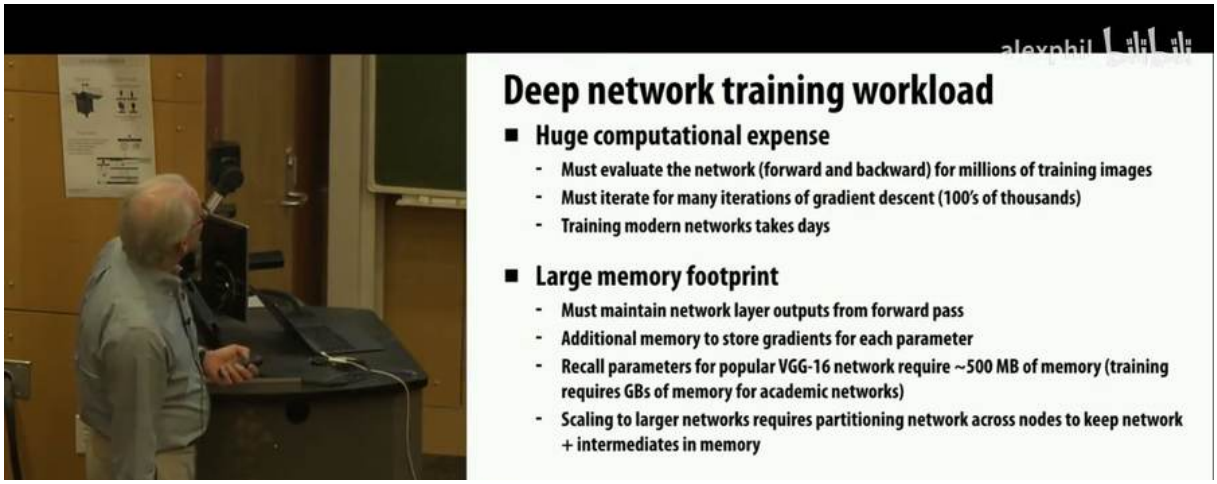

图 581: 深度网络的训练负载：前向与反向传播的巨大计算量[581]

于是损失就是预测值与目标值之间差异的某个函数，训练的目标是最小化这个损失。

$$L = -\sum_{c=1}^{C} y_c \log(\hat{y}_c), \qquad \hat{y}_c = \frac{e^{z_c}}{\sum_{j=1}^{C} e^{z_j}}$$

[580] 视频画面时间区间：00:45:00–00:45:15。若为裁剪图，时间区间仍对应原视频画面。

[581] 视频画面时间区间：00:52:30–00:52:45。若为裁剪图，时间区间仍对应原视频画面。

- z_c：第 c 类的原始输出分数；
- $\hat{y}_c$：经 softmax 归一化后的概率；
- y_c：真实标签的独热编码；
- C：类别数，本例约 100。

讲者强调 softmax 的可微性使梯度可以逐层回传，这正是反向传播得以实现的前提。

本章小结

训练的目标是通过小幅调整权重让网络在大量样本上普遍表现良好，并能泛化到未见样本；因此必须切出评估集衡量质量。输出经 softmax 强调最大值但不丢弃其余，使其可微从而支持梯度回传。损失衡量预测与目标的差异，训练即最小化该损失。

26.8 总结与延伸

讲者最后说讲了这么久，问有没有人熟悉这些内容，然后结束了这一讲。

把全讲串起来，这是一条从” 神经网络是什么” 到” 如何高效计算” 的路径。深度学习彻底改变了 AI，并把问题拉回结构化数值计算，这是它与并行计算天然契合的原因。基本单元是加权和加偏置再加非线性激活；早期用饱和型函数，现在普遍用 ReLU——取与零的最大值，简单却有效，且只是受大脑启发而非模仿。训练与推理是两个不同的问题：训练需要数天到数月与巨量算力，推理单次快速但调用次数极多且常在受限设备上进行。卷积层因权重共享而三重节省：更易训练、存储与评估；滤波器可以是高斯模糊，也可以是边缘检测；多滤波器是 k 个独立计算；纯线性无效，因此必须加非线性；池化做数据缩减。推理的朴素实现缓存行为差，解法是分块，与矩阵乘法同构；内存占用随层数快速增长，因此瓶颈常在带宽而非算力。训练的目标是泛化而非拟合单样本，输出经 softmax 使其可微，训练即最小化损失。

本讲义在讲者材料之上做的压缩，有四点需要与讲者原话区分：

- 深度学习的并行性来自它的数值结构，而非它的” 智能”。它由规则化的乘加构成，因此可以映射到 SIMD、GPU 与专用加速器。这也解释了为什么它对硬件演进极其敏感——算力提升直接转化为模型能力。
- 卷积的价值是参数效率，不是表达力。它用权重共享把参数量从” 与图像尺寸的乘积成正比” 降到” 只与滤波器大小有关”。因此它适用于具有平移不变性的任务，不适合所有任务。
- 推理与训练需要不同的优化策略。推理关注单次延迟与内存带宽，训练关注吞吐与数值稳定性（前向与反向都要算）。因此同一网络在两侧的最优实现往往不同。
- 非线性不是可选项而是必要条件。若全网络都是线性变换，多层叠加等价于单层线性变换，深度毫无意义。ReLU 的价值在于它足够简单，使梯度计算廉价，同时打破线性。

这堂课的边界在于它是导论：讲者明确说网络结构的设计仍带” 黑魔法” 成分，未深入反向传播的推导与具体训练算法细节。

两个可以直接从本讲数据提出的下一步问题：其一，卷积的权重共享依赖平移不变性，那么对于不具备这种不变性的任务（如需要精确定位的检测），应当如何改造卷积层？其二，推理的瓶颈常在内存带宽，那么把相邻层融合以减少中间结果往返，与把网络做窄以减少参数量，这两种策略在什么条件下更有效？

Lecture 27 Implementing Message Passing

视频时长：1 小时 19 分 10 秒　来源：https://www.bilibili.com/video/BV1Gh4y1S75H/

27.1 消息传递为什么容易实现

这一讲讲所用框架的内部实现。讲者先用约 25 分钟讲消息传递实现中数据的实际搬运，然后交给另一位教授讲其他内容。

基本操作

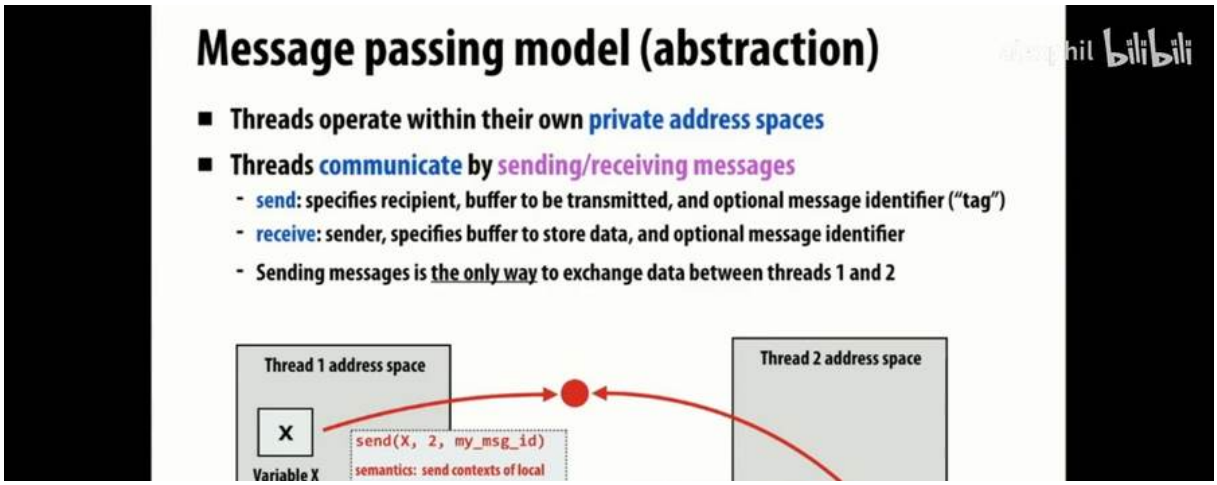

图 582: 消息传递模型：线程在各自的私有地址空间中操作[582]

讲者回顾：大家现在既有共享地址空间的编程经验，也有消息传递的经验。消息传递中的原语通信操作，就是把一条消息从一个处理器发到另一个处理器。

消息传递的复制开销无法回避

共享地址空间可以靠缓存让多个处理器共享同一份数据；消息传递每次通信都要复制。因此消息传递的成本与通信量成正比，而共享地址空间的成本与缓存行迁移次数成正比。这决定了它们在不同计算通信比下的优劣。

发送方执行 send，接收方执行 receive。这个操作最终把数据从发送方的私有地址空间复制到接收方私有地址空间的某一部分。

[582] 视频画面时间区间：00:01:00–00:01:15。若为裁剪图，时间区间仍对应原视频画面。

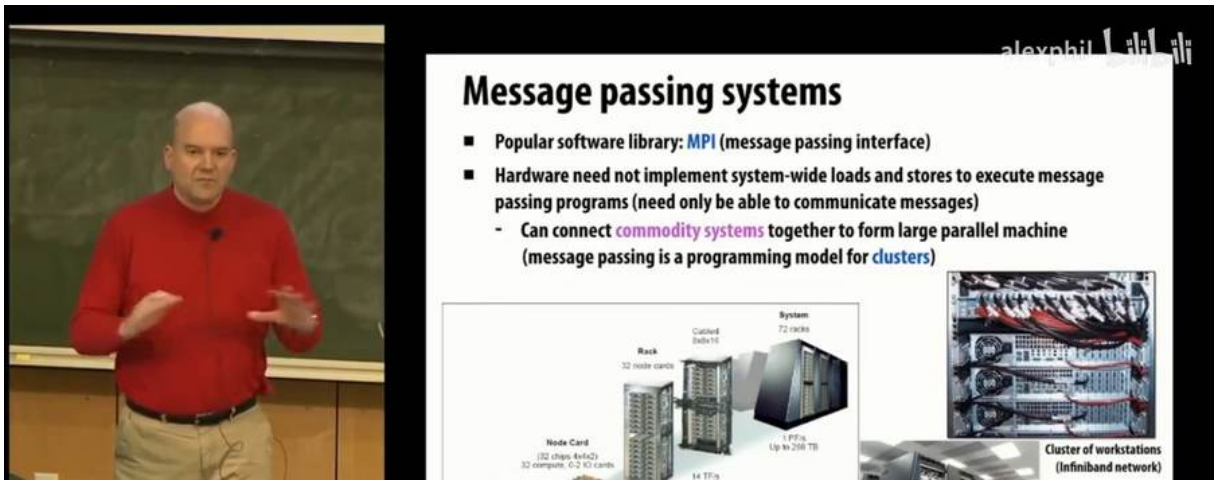

图 583: 消息传递系统：流行库为 MPI，硬件需求很少[583]

硬件上为什么受欢迎

讲者给出一个很实际的理由：市面上很多系统采用消息传递，原因很简单——搭建大规模消息传递计算机非常容易，因为你除了一个足够快的互连之外不需要任何特殊硬件。

特别是与共享地址空间相比，你不需要缓存一致性，也不需要内存系统的任何特殊支持。

消息传递的可扩展性上界由互连决定

既然不需要一致性硬件，那么规模上限就只取决于互连的带宽与延迟。这也意味着消息传递系统的性能瓶颈几乎总是网络，而不是处理器。因此第 19 讲的拓扑与对分带宽分析在这里直接适用。

他甚至说：理论上你可以把手机、物联网设备之类全都拉进来，只要它们能通过某个（希望足够快的）网络互相发消息，就构成一台大的并行计算机。

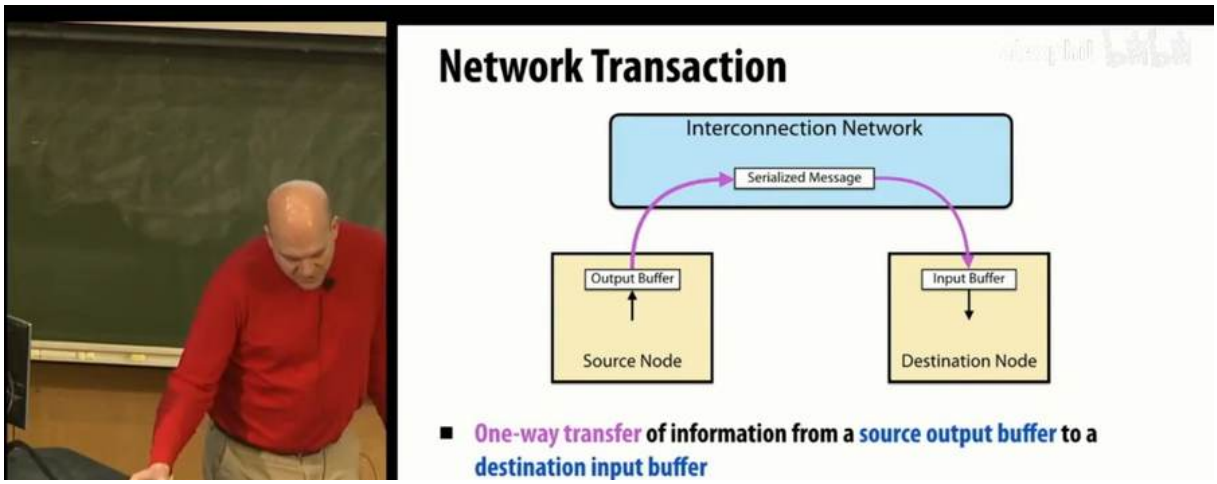

图 584: 网络事务：序列化消息、输出缓冲、输入缓冲与互连网络[584]

[583] 视频画面时间区间：00:02:00–00:02:15。若为裁剪图，时间区间仍对应原视频画面。
[584] 视频画面时间区间：00:03:30–00:03:45。若为裁剪图，时间区间仍对应原视频画面。

消息传递的规模优势来自”不需要一致性”

共享地址空间要跨节点就必须维护缓存一致性，而一致性协议的复杂度与节点数密切相关（见第 11、13 讲）。消息传递把这层复杂度完全移出硬件，因此可以靠增加节点线性扩展。代价是软件必须显式管理数据放置。

在整个并行计算历史上，这都是搭建大型机器相当流行的方式。今天在云计算中，很多服务器就是这样工作的：独立的刀片或节点之间并没有缓存一致性，因此让软件跑起来最容易的方式就是消息传递。

讲者补了一句很关键的限定：它受欢迎并不一定因为它是程序员写软件时最方便的首选方式——你们做完作业可能各有看法——但从硬件角度看它确实很棒。

”硬件容易”与”编程容易”是两条独立的评价轴

消息传递不需要缓存一致性，因此搭建大规模机器几乎只依赖互连；但程序员必须显式搬运数据、显式安排同步。因此它是一套”硬件成本低、软件成本高”的方案。这个不对称正是它在大规模系统中长期占优的原因。

本章小结

消息传递的原语是把数据从发送方私有地址空间复制到接收方。它在硬件上极受欢迎：只需足够快的互连，不需要缓存一致性或特殊支持，因此云计算的刀片间也用它。但它未必是程序员最方便的写法——这是”硬件成本低、软件成本高”的不对称。

27.2 与共享地址空间的对比

共享地址空间怎么搬数据

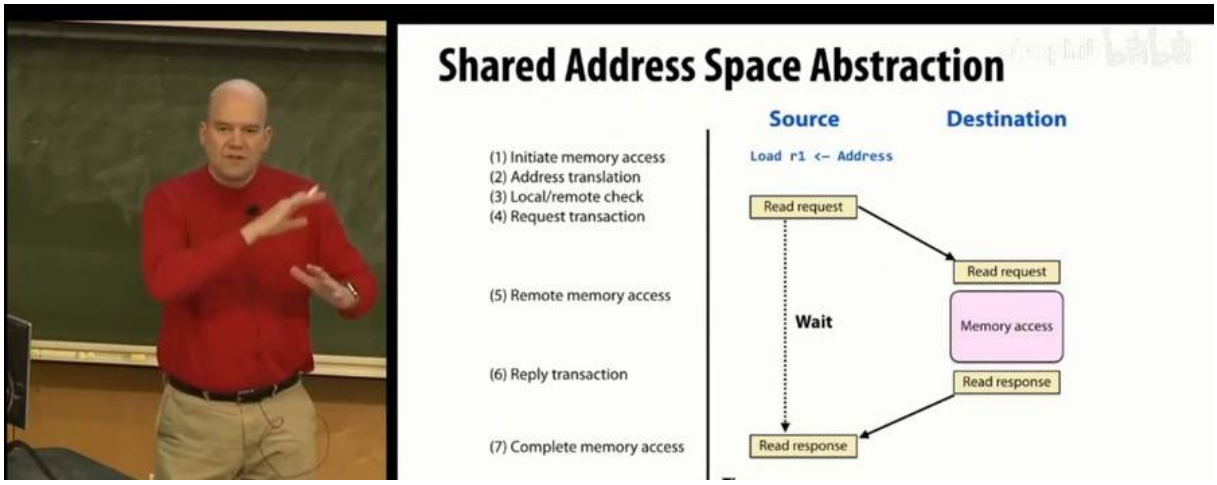

图 585: 共享地址空间抽象：发起访问、地址转换、取回数据[585]

[585] 视频画面时间区间：00:04:30–00:04:45。若为裁剪图，时间区间仍对应原视频画面。

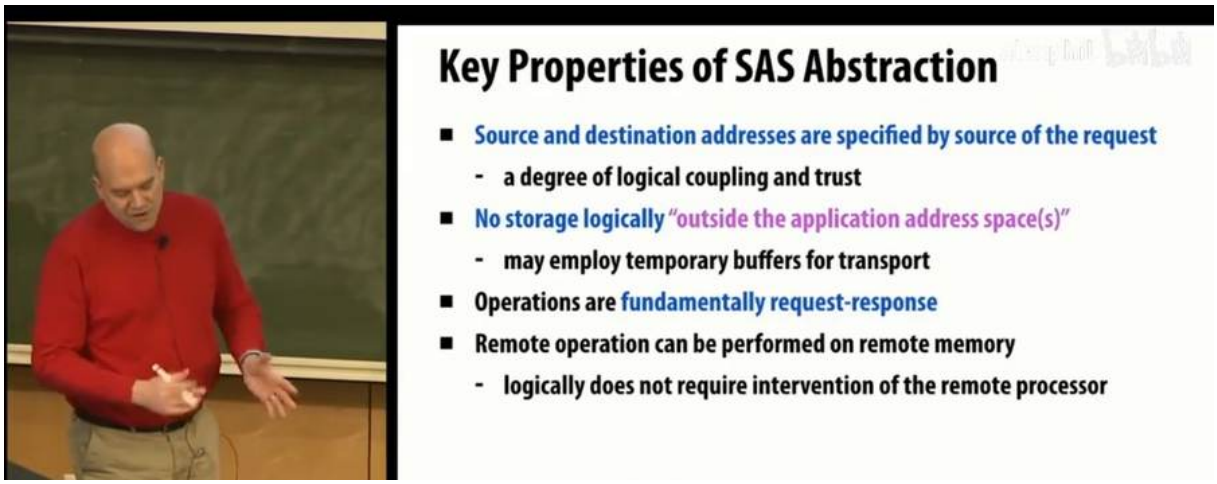

图 586: 共享地址空间的关键性质：地址由请求方指定[586]

讲者先展示共享地址空间下数据的实际搬运，以便与消息传递对比。

有趣之处在于：在共享地址空间中，数据主要通过 ** 读 ** 被搬进消费方的缓存。写操作只是更新数据，真正把数据拉过来的是读。

共享地址空间把”数据在哪”这个问题交给硬件

请求方只给出地址，其余由地址转换与一致性协议完成。因此程序员不必知道数据实际位于哪个节点的内存。消息传递没有这层机制，因此”数据在哪”必须由程序自己跟踪。

一次读会把某个虚拟地址转成物理地址，硬件据此知道去哪里取数据。

关键差别

讲者指出一个当时看起来不明显、但后面会冒出来的要点：共享地址空间不需要在地址空间之外的额外缓冲或存储。

它本质上是一个请求-响应协议，这让它简单。

共享地址空间的隐含假设是”地址双方都知道”

在共享地址空间中，请求方只需给出地址，硬件负责把它翻译成物理位置并取回数据。发送方不需要知道接收方的缓冲区在哪，接收方也不需要预先安排。消息传递没有这个前提，因此必须显式交换地址或复制数据。

本章小结

共享地址空间靠读把数据拉进消费方缓存，写只更新数据；读时由硬件做地址转换。它不需要地址空间之外的额外缓冲，本质是请求-响应协议，因此简单。其隐含前提是双方都能通过地址访问同一份数据，而消息传递没有这个前提。

[586] 视频画面时间区间：00:06:30–00:06:45。若为裁剪图，时间区间仍对应原视频画面。

27.3 两种语义：同步与异步

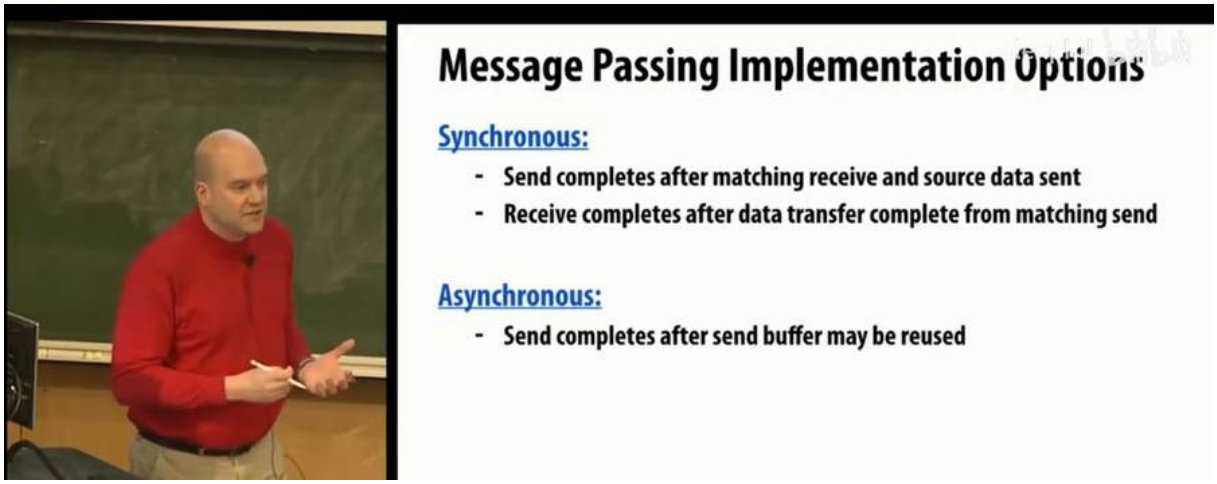

图 587：消息传递的实现选项：同步与异步[587]

讲者回顾几周前看网格求解代码时引入的概念。

同步消息传递：发送方停顿，直到接收方已经收到。

异步消息传递：你发出去、把它缓冲起来、继续干自己的事，最终接收方会收到它，然后双方各自继续。

直觉的陷阱

讲者说，在完全不了解底层发生什么的情况下，如果只听说这两种，你的直觉大概会说：异步听起来好得多，因为它不让发送方停顿。发送方发消息时凭什么要停？我们宁愿继续跑。

异步的”不停顿”是有条件的

异步发送之所以不停顿，是因为消息被暂存在某处。若暂存空间耗尽，发送方仍会被迫阻塞。因此异步不是”永不阻塞”，而是”把阻塞推迟并集中到缓冲耗尽时”。理解这一点才能理解后面两种协议的取舍。

[587] 视频画面时间区间：00:08:00–00:08:15。若为裁剪图，时间区间仍对应原视频画面。

消息的尺寸

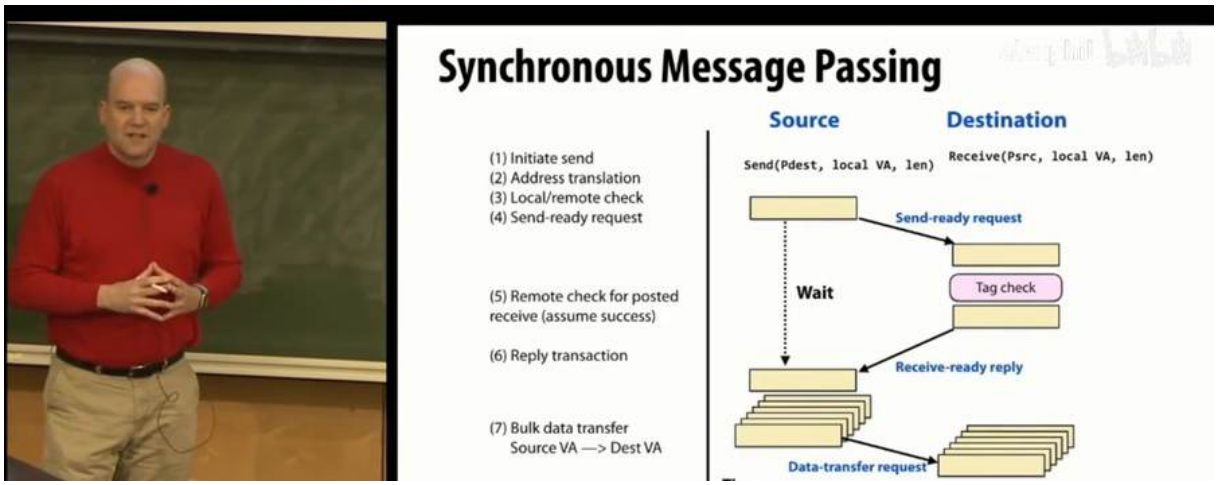

图 588: 同步消息传递：发起发送与匹配接收[588]

讲者提醒消息不只是单个缓存行：它们可能是 1024 字节这样的千字节量级，甚至兆字节级的数据。

而且课上早先讲过：发送大消息是好主意，你们大概也亲身经历过。发很小的消息——单个字、单个字节——会带来大量开销。更好的做法是把要发的数据合并成大消息。

因此系统必须支持传输任意长度的消息，而且它们可能很大。

本章小结

同步消息传递让发送方停顿到接收方收到；异步则缓冲后继续。直觉会偏向异步，但异步的”不停顿”以缓冲空间为前提。消息可能达千字节甚至兆字节，小消息开销大，因此应合并成大消息，而系统必须支持任意长度。

[588] 视频画面时间区间：00:09:00–00:09:15。若为裁剪图，时间区间仍对应原视频画面。

27.4 eager 协议：三段流程

先发通知，不发数据

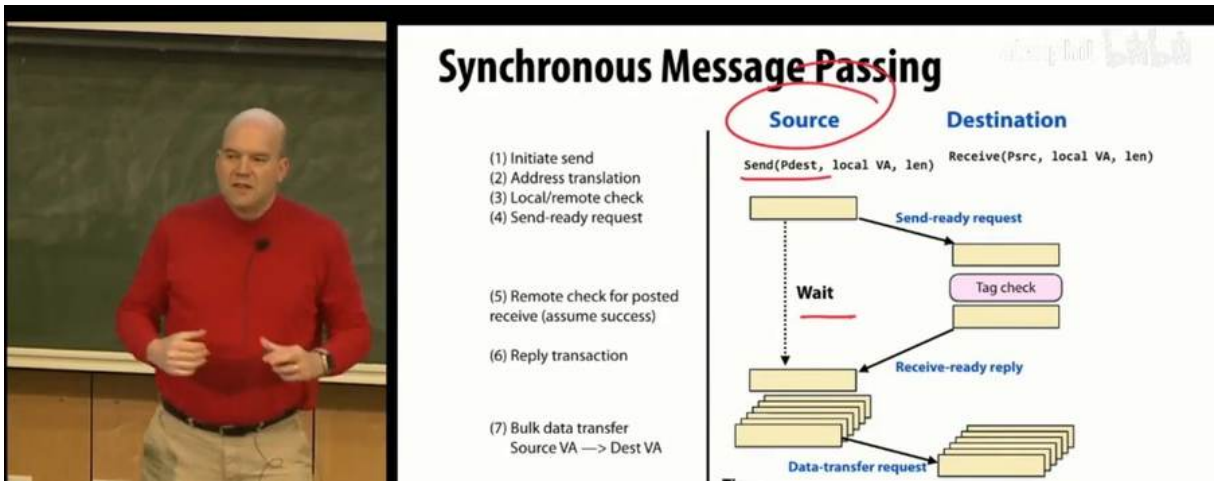

图 589: 同步消息传递续：发送方先通知而不发数据[589]

讲者用图示逐步说明。发送方想发消息，于是执行 send。

它先要做的是通知接收方” 我想发一条消息”，但此时还不发送消息里的实际数据。

这个通知要带上消息类型之类的信息，以便与接收方正等待的消息类型匹配。

本章小结

同步消息传递让发送方停顿到接收方收到；异步则缓冲后继续。直觉偏向异步，但异步的「不停顿」以缓冲空间为前提。消息可能达千字节甚至兆字节，小消息开销大，因此应合并成大消息，而系统必须支持任意长度。

下面看两种协议各自如何组织数据的实际搬运。

[589] 视频画面时间区间：00:10:30–00:10:45。若为裁剪图，时间区间仍对应原视频画面。

27.5 同步协议的三段流程

回应地址

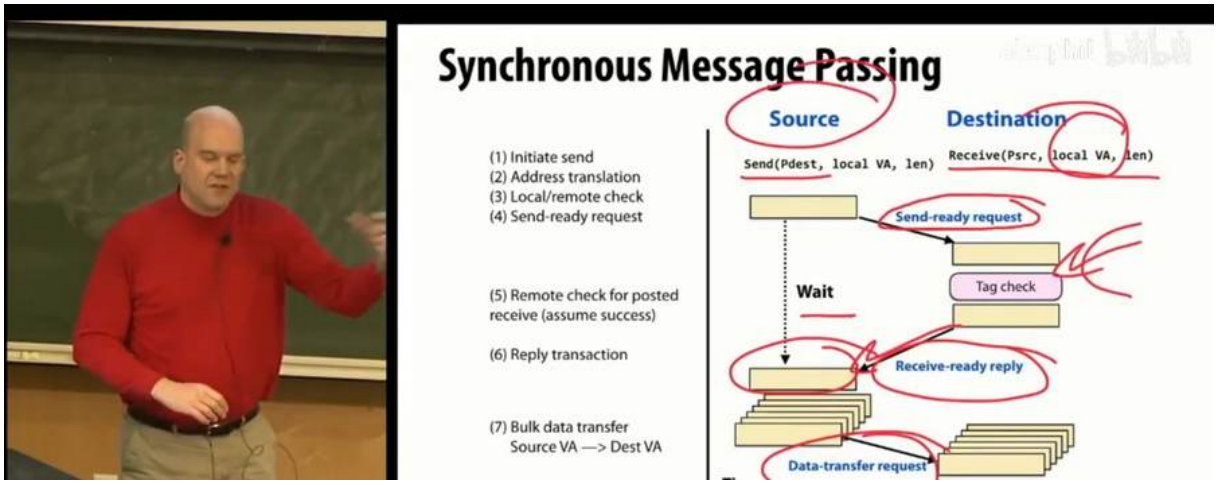

图 590: 同步消息传递续：接收方回应并告知目标地址[590]

假设接收方已经执行了一个匹配的 receive，那么它已经准备好、也确实想要这条消息。

于是它把一条应答消息发回给发送方，说” 我们准备好接收了”，并顺便告诉它：这是目标地址，数据传过来时应该放到这里。

像 DMA 一样传输

现在发送方知道了目的地址，可以开始传输数据。讲者说好处是：这时它就只是一次 DMA 传输——直接内存访问传输。

它只是把数据从发送方地址空间中一个已知位置，复制到接收方地址空间中一个已知位置，因为发送方已经知道目的地址。所以这就是内存之间的复制。

而这个方案的美妙之处在于：不需要在发送方与接收方地址空间之外的任何额外缓冲。

$$\text{跳数} = 1_{\text{通知}} + 1_{\text{回应地址}} + 1_{\text{数据}} = 3$$

- 第一跳：发送方把通知送到接收方；
- 第二跳：接收方把目标地址送回发送方；
- 第三跳：数据从发送方直接写入接收方地址空间；
- 因此即使是极小的消息，也至少需要三跳。

零额外缓冲换来的是往返延迟

这个协议不在地址空间外占用缓冲，代价是每次发送都要一次完整往返才能开始传数据。因此它的延迟下限由三跳决定。对延迟敏感的通信（如反复传递一个标志位），这个下限会成为瓶颈。

[590] 视频画面时间区间：00:13:00–00:13:15。若为裁剪图，时间区间仍对应原视频画面。

本章小结

eager 协议分三段：发送方先发通知（含消息类型），接收方匹配后回应并告知目标地址，然后数据以 DMA 方式从已知位置复制到已知位置。它不需要地址空间外的额外缓冲，代价是至少三跳的往返延迟。

27.6 异步的麻烦与流控

发送方阻塞反而是好事

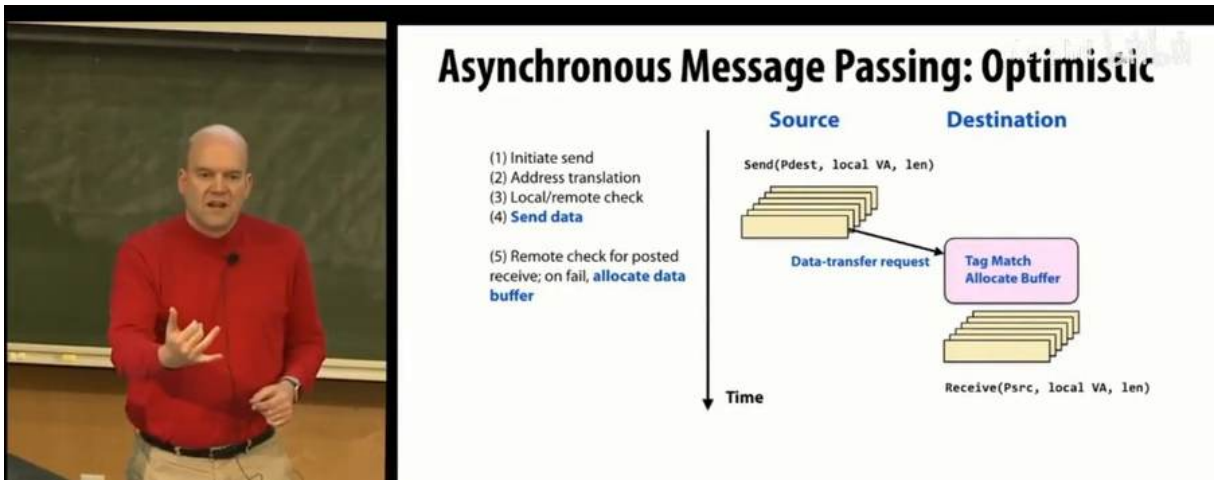

图 591: 异步消息传递（乐观）：发送后缓冲并继续[591]

讲者接着分析乐观的异步方案会遇到什么问题。

如果发送方缓冲用尽，它就停顿、阻塞，直到那些消息消失。这实际上是一种自然的流控——它把发送方节流回去，于是它们的发送不再真正异步，而会阻塞一段时间。

但

本地可判定性是分布式算法设计的关键

一个机制若能在本地判定是否需要等待，就不需要全局协调，因此实现简单、扩展性好。发送方缓冲检查满足这个条件；接收方的涌入问题不满足，因此必须靠预先约定的信用额度来转化。

讲者说这其实是容易处理的问题：因为你在发送方本地就能察觉”糟糕，缓冲用完了”，那好，我现在不再往消息层塞东西，就这么阻塞着，最终它会清空。

[591] 视频画面时间区间：00:14:00–00:14:15。若为裁剪图，时间区间仍对应原视频画面。

接收方的问题难得多

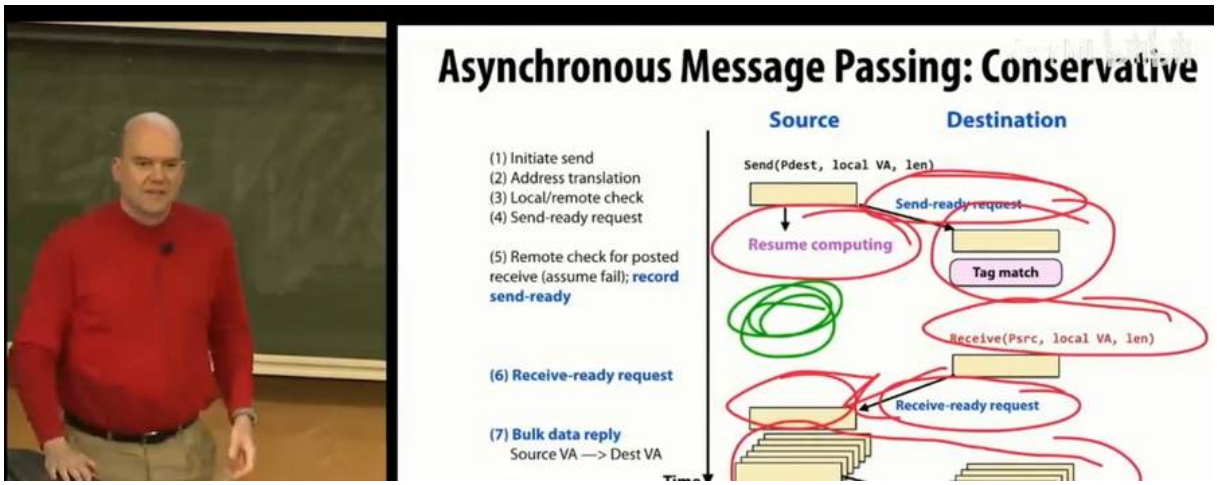

图 592: 异步（保守）方案：地址转换后确认接收方已就绪[592]

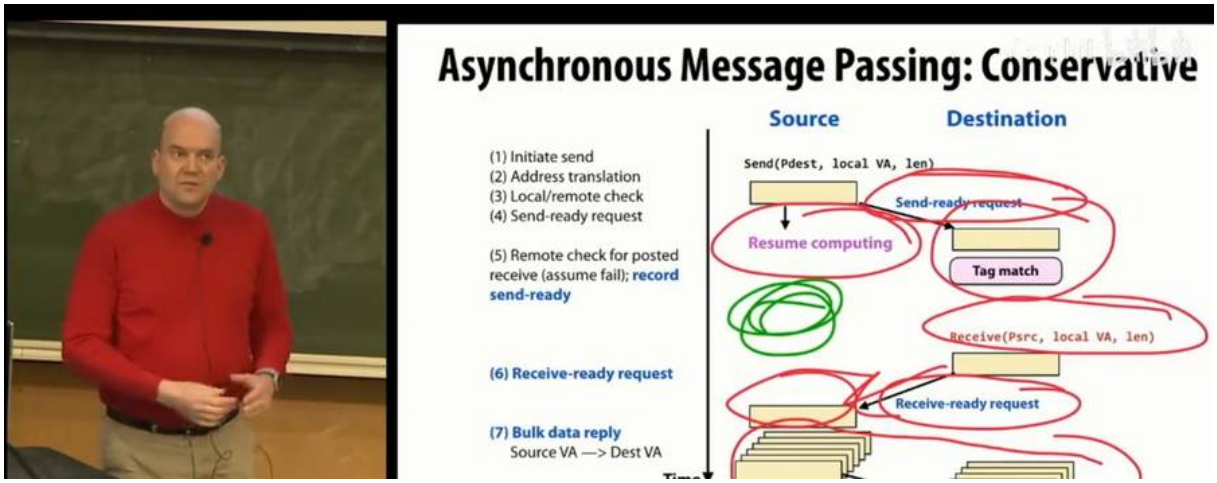

图 593: 异步消息传递（保守）：先确认再发送[593]

讲者说接收方侧就难得多。因为如果东西持续涌入，你很难回过头去说” 嘿，等一下，你那边别再给我发了”——因为它已经在给你发了很多东西。

页面标记是一种” 写时重定向” 技巧

接收方把正在使用的页面标记为不可写，发送方的数据便只能落到其他位置。这样接收方可以安全地继续使用当前页面，而不必担心被覆盖。代价是多了一层页级映射与异常处理。

他提到一个常用的技巧：把你正在用的那个页面标记出来，让发送方写到别处去。但这也带来一个重要的正确性问题——如果你在” 执行了 send” 与” 数据真正传输” 之间写了那个地址，就会出问题。

[592] 视频画面时间区间：00:21:40–00:21:55。若为裁剪图，时间区间仍对应原视频画面。
[593] 视频画面时间区间：00:22:00–00:22:15。若为裁剪图，时间区间仍对应原视频画面。

乐观异步的风险在于接收方无法回溯

发送方缓冲耗尽时它可以自我节流，这是本地决策；但接收方一旦被涌入的数据填满，就无法让已经在途的消息停下。因此保守方案必须事先获得许可，这正是信用机制要解决的问题。

接收方要应对涌入，除了信用机制还有另一类做法：把数据直接写入最终目的地。若接收方事先知道消息将放到哪个缓冲区，就不必先暂存再复制，而可以让发送方一次写到那里。这正是 eager 协议在 rendezvous 之后能做到零额外缓冲的原因。但要让接收方事先知道目的地，就必须有一次往返确认——于是又回到延迟与缓冲的取舍上。因此这两类方案不是互相替代，而是分别优化延迟与空间：乐观方案省掉往返但需要缓冲，保守方案省掉缓冲但需要往返。信用机制的价值就在于它允许大部分发送走乐观路径，只在信用耗尽时才退化为等待。

本章小结

乐观异步下，发送方缓冲耗尽会阻塞，形成自然流控，而且这在本地容易处理。接收方则难得多：数据已在涌入，无法回溯要求对方停止。常用的页面标记技巧还引出”send 与传输之间写入” 的正确性问题。

27.7 信用式混合方案

预分配与通告

讲者提出混合思路：能否在安全的时候享受乐观异步的速度，而不陷入可怕的缓冲问题?

做法是信用式方案：在接收方为所有发送方预分配一定量的空间。然后消息传递层会向它们通告” 已经为你预留了多少空间”。

他举例说比如四个字节或若干千字节。这个额度不需要很大就能显著减少往返等待的次数：只要发送方在一次突发中不超出额度，它就可以连续发送而不必等待确认，而接收方只需为每个发送方预留这一小块空间，总开销随参与者数量线性增长但系数很小。

跟踪用量

然后发送方在尝试发消息时，会跟踪自己已经用掉了多少那部分空间。

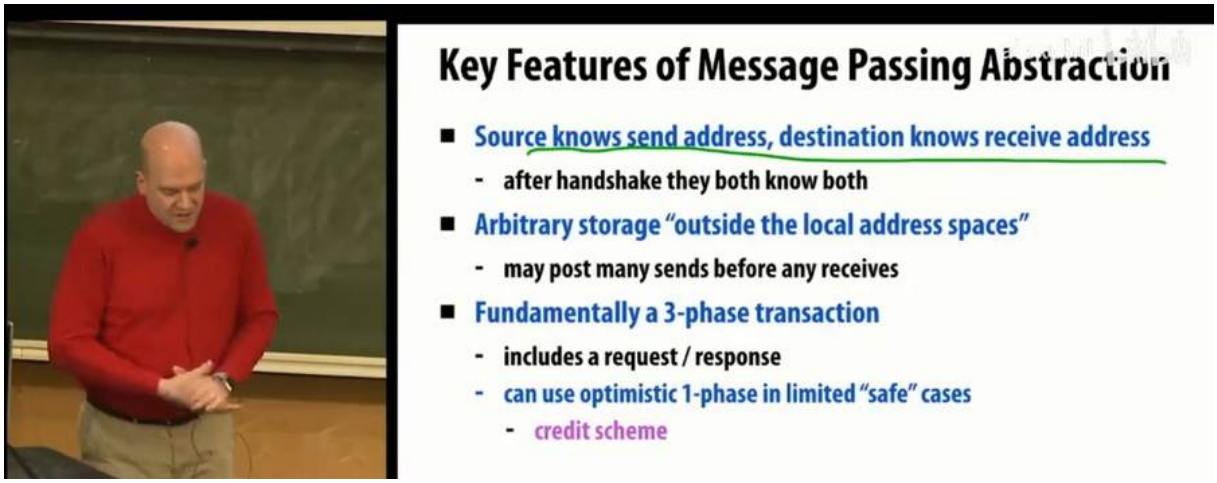

图 594: 消息传递抽象的关键特征：发送方知道接收方身份[594]

只要还有信用，它就可以继续以乐观方式发送，不必等待往返确认；一旦信用耗尽，它就必须等待接收方归还信用。

信用额度与等待频率的关系可近似为：

$$N_{\text{wait}} \approx \frac{B_{\text{total}}}{C}$$

- B_{total}：一次通信阶段要发送的总字节数；
- C：预分配的信用额度（字节）；
- N_{wait}：该阶段中发送方需要等待信用归还的次数；
- 额度越大，等待次数越少，但接收方需预留的缓冲也越大。

$$\text{可用信用} = \text{预分配额度} - \text{已发送未确认字节}$$

- 预分配额度：接收方为该发送方预留的缓冲上限；
- 已发送未确认字节：已经发出但接收方尚未消费的数据量；
- 只要可用信用为正，发送方无需等待确认即可继续发送。

信用机制把”全局流控”变成”逐对局部记账”

接收方不必在涌入时喊停，而是在建立连接时就声明”你能发多少”。发送方据此自行记账，用尽则自停。这把一个需要全局协调的问题化成了每个发送方本地可判定的条件，因此既安全又不必每次往返。

信用额度的大小需要权衡。额度设得大，发送方可以在长时间内不等待，但接收方必须为每个可能的发送方预留相应空间，总缓冲需求随参与者数量线性增长。额度设得小，接收方的缓冲压力小，但发送方会更频繁地等待信用归还，通信效率下降。因此实际系统常按连接建立时的可用内存与预期通信模式动态协商额度，并在运行中根据实际吞吐调整。这个调参过程与第 19 讲讨论的缓冲深度问题是同一个数学结构：都需要在「空间」与「等待」之间找到平衡点。

此外，信用归还的粒度也值得注意：若每消费一小段就归还一点，通信开销会累积；若只在整块消费完后归还，发送方又可能等待过久。实际实现常取折中，按固定批次归还。

本章小结

信用式方案在接收方为每个发送方预分配空间并通告额度，发送方跟踪已用量，只要信用为正就可持续乐观发送，用尽则等待归还。它把全局流控变成逐对局部记账，因此兼顾了速度与安全。

27.8 总结与延伸

讲者在结尾结束了这一部分，随后由另一位教授继续。

把全讲串起来，这是一条从”消息传递为何容易实现”到”如何在速度与安全之间取舍”的路径。消息传递的原语是把数据从发送方私有地址空间复制到接收方，它只需足够快的互连，不需要缓存一致性，因此在云计算刀片间也广泛使用——这是”硬件成本低、软件成本高”的不对称。共享地

[594] 视频画面时间区间：00:29:30–00:29:45。若为裁剪图，时间区间仍对应原视频画面。

址空间靠读把数据拉进消费方缓存，由硬件做地址转换，本质是请求-响应协议，不需要额外缓冲，前提是双方都能通过地址访问同一份数据。消息传递有两种语义：同步让发送方停顿到接收方收到，异步则缓冲后继续。直觉偏向异步，但异步的不停顿以缓冲空间为前提。消息可达千字节至兆字节，小消息开销大因此应合并。eager 协议分三段：通知（含类型）、回应地址、DMA 传输，它不需要额外缓冲但至少三跳。乐观异步下发送方缓冲耗尽会阻塞并形成自然流控，这在本地易处理；接收方则难得多，因为数据已在涌入而无法回溯。最后，信用式混合方案在接收方为每个发送方预分配空间并通告额度，发送方跟踪已用量，用尽则等待归还——把全局流控变成逐对局部记账。

本讲义在讲者材料之上做的压缩，有四点需要与讲者原话区分：

- ” 异步不停顿” 是有条件的。它的前提是缓冲空间可用；一旦耗尽，发送方仍会阻塞。因此异步的真实收益是” 把阻塞推迟并集中”，而不是消除阻塞。
- eager 协议的零额外缓冲与三跳延迟是一对交换。它把缓冲责任推给地址空间本身，代价是每次发送都要一次往返。因此它对大消息友好、对小消息不友好。
- 发送方与接收方的流控难度不对称。发送方可以本地自停，接收方却无法让在途数据回头。这个不对称正是信用机制存在的理由。
- 信用机制把全局问题化为局部记账。接收方在建立连接时声明额度，发送方自行记账，因此既避免全局协调，又不必每次往返确认。这是分布式流控的通用模式。

这堂课的边界在于它聚焦数据搬运与缓冲策略，未涉及路由算法、网络拓扑对消息延迟的影响，也未讨论 MPI 的具体实现细节。

两个可以直接从本讲数据提出的下一步问题：其一，信用额度的大小应当如何选择？过大则接收方缓冲压力大，过小则发送方频繁等待归还。其二，eager 协议的三跳延迟对延迟敏感的小消息不利，那么是否存在只针对小消息优化的第三条路径？